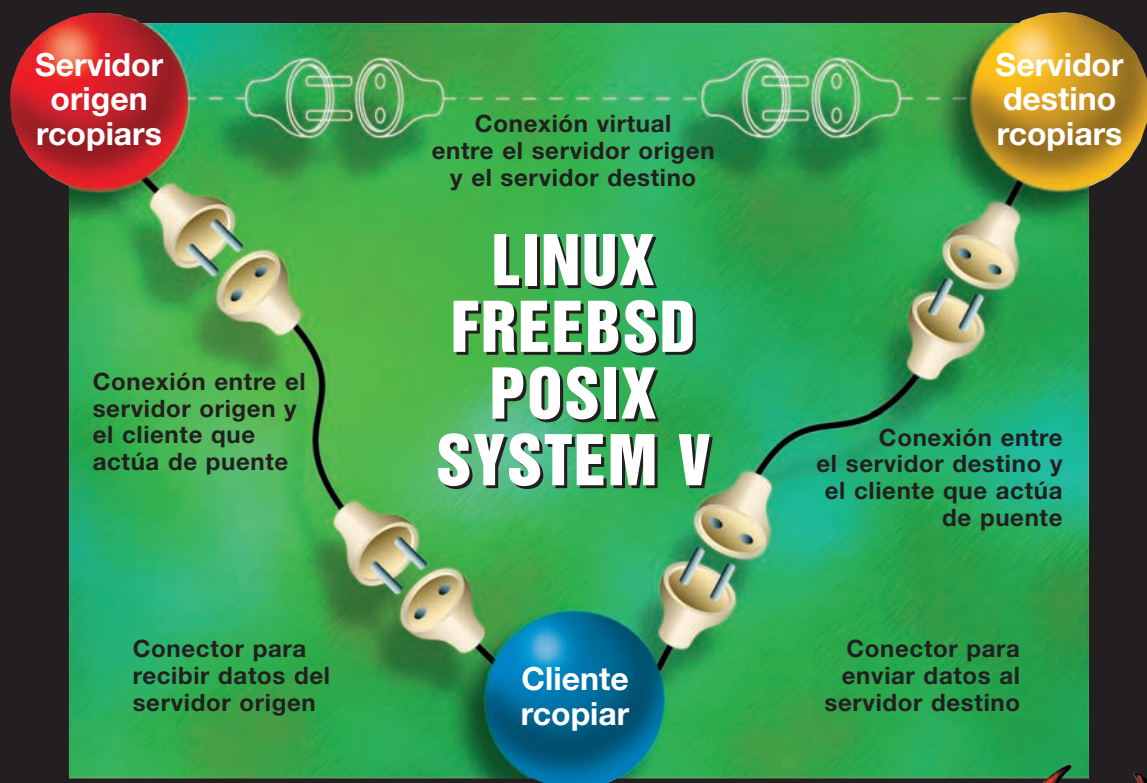


UNIX

Programación avanzada

3ª EDICIÓN AMPLIADA Y ACTUALIZADA



Francisco M. Márquez



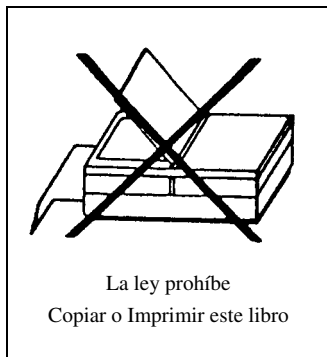
Ra-Ma®

UNIX®
PROGRAMACIÓN AVANZADA
(TERCERA EDICIÓN)

UNIX[®]
PROGRAMACIÓN AVANZADA
(TERCERA EDICIÓN)

FRANCISCO MANUEL MÁRQUEZ GARCÍA
Universidad de Alcalá





UNIX® es una marca comercial registrada de The Open Group

UNIX®. PROGRAMACIÓN AVANZADA, 3ª EDICIÓN

© Francisco Manuel Márquez García

© De la Edición Original en papel publicada por Editorial RA-MA

ISBN de Edición en Papel: 978-84-9964-603-4

Todos los derechos reservados © RA-MA, S.A. Editorial y Publicaciones, Madrid, España.

MARCAS COMERCIALES. Las designaciones utilizadas por las empresas para distinguir sus productos (hardware, software, sistemas operativos, etc.) suelen ser marcas registradas. RA-MA ha intentado a lo largo de este libro distinguir las marcas comerciales de los términos descriptivos, siguiendo el estilo que utiliza el fabricante, sin intención de infringir la marca y solo en beneficio del propietario de la misma. Los datos de los ejemplos y pantallas son ficticios a no ser que se especifique lo contrario.

RA-MA es una marca comercial registrada.

Se ha puesto el máximo empeño en ofrecer al lector una información completa y precisa. Sin embargo, RA-MA Editorial no asume ninguna responsabilidad derivada de su uso ni tampoco de cualquier violación de patentes ni otros derechos de terceras partes que pudieran ocurrir. Esta publicación tiene por objeto proporcionar unos conocimientos precisos y acreditados sobre el tema tratado. Su venta no supone para el editor ninguna forma de asistencia legal, administrativa o de ningún otro tipo. En caso de precisarse asesoría legal u otra forma de ayuda experta, deben buscarse los servicios de un profesional competente.

Reservados todos los derechos de publicación en cualquier idioma.

Según lo dispuesto en el Código Penal vigente ninguna parte de este libro puede ser reproducida, grabada en sistema de almacenamiento o transmitida en forma alguna ni por cualquier procedimiento, ya sea electrónico, mecánico, reprográfico, magnético o cualquier otro sin autorización previa y por escrito de RA-MA; su contenido está protegido por la Ley vigente que establece penas de prisión y/o multas a quienes, intencionadamente, reprodujeren o plagiaren, en todo o en parte, una obra literaria, artística o científica.

Editado por:

RA-MA, S.A. Editorial y Publicaciones

Calle Jarama, 33, Polígono Industrial IGARSA

28860 PARACUELLOS DE JARAMA, Madrid

Teléfono: 91 658 42 80

Fax: 91 662 81 39

Correo electrónico: editorial@ra-ma.com

Internet: www.ra-ma.es y www.ra-ma.com

Maquetación: Autor

Diseño Portada: Autor

ISBN: 978-84-9964-311-3

E-Book desarrollado en España en septiembre de 2014

*Dedico esta obra a mi hermana Carmen,
su ejemplar lucha por la vida es
una lección para todos nosotros.*

Tabla de materias

Índice de figuras	xv
Índice de programas	xvii
Índice de tablas	xxi
Prólogo	xxiii
1 Introducción	1
1.1 Estructura del sistema	1
1.2 Arquitectura del sistema operativo UNIX	3
1.3 Interfaz de las llamadas al sistema	5
1.4 Ejercicios	8
 I El sistema de ficheros	 11
2 Arquitectura del sistema de ficheros	13
2.1 Características del sistema de ficheros	13
2.2 Estructura del sistema de ficheros	14
2.2.1 El superbloque	15
2.2.2 Nodos índice (<i>inodes</i>)	16
2.2.3 Los bloques de datos. Estructura de un fichero ordinario	18
2.3 Tipos de ficheros en UNIX	23
2.3.1 Ficheros ordinarios	23
2.3.2 Directorios	24
2.3.3 Ficheros especiales	27
2.3.4 Tuberías con nombre	28
2.4 Extensiones del sistema BSD	29
2.4.1 Grupo de cilindros	30
2.5 Tablas de control de acceso a los ficheros	31
2.5.1 Tabla de nodos-i	31
2.5.2 Tabla de ficheros	31
2.5.3 Tabla de descriptores de fichero	33
2.6 Administración de los sistemas de ficheros	35

2.6.1	Particiones del disco	35
2.6.2	Formato del disco	35
2.6.3	Ficheros de dispositivo del disco	36
2.6.4	Construcción de un sistema de ficheros	38
2.6.5	Comprobación del estado de un sistema de ficheros	41
2.6.6	Montaje y desmontaje de un sistema de ficheros	42
2.6.7	Monitorización y contabilidad del sistema de ficheros	43
2.7	Ejercicios	44
3	Manejo de ficheros ordinarios	51
3.1	Introducción	51
3.2	Entrada/salida sobre ficheros ordinarios	53
3.2.1	Apertura de un fichero (open)	53
3.2.2	Lectura de datos de un fichero (read)	55
3.2.3	Escritura de datos en un fichero (write)	56
3.2.4	Cierre de un fichero (close)	57
3.2.5	Creación de un fichero (creat)	57
3.2.6	Duplicado de un descriptor (dup)	58
3.2.7	Acceso aleatorio (lseek)	58
3.2.8	Consistencia de un fichero	59
3.3	Biblioteca estándar de funciones de entrada/salida	59
3.3.1	Interfaz de la biblioteca estándar	59
3.3.2	Entrada/salida de caracteres con la biblioteca estándar	63
3.3.3	Implementación de la biblioteca estándar de entrada/salida	64
3.4	Control de ficheros abiertos (fcntl)	72
3.5	Administración de ficheros	77
3.5.1	stat , lstat y fstat	77
3.5.2	Modos de un fichero	78
3.5.3	Cambio de la información estadística de un fichero	81
3.5.4	Ejemplo de utilización de la información estadística de un fichero	83
3.6	Compartición y bloqueo de ficheros	87
3.6.1	Implementación de lockf a partir de open	91
3.7	Ejercicios	94
4	Manejo de directorios y ficheros especiales	117
4.1	Acceso a directorios	117
4.1.1	Creación de un directorio (mkdir y mknode)	118
4.1.2	Borrado de un directorio (rmdir)	119
4.1.3	Creación de nuevas entradas en un directorio (link)	120
4.1.4	Directorios asociados a un proceso (chdir y chroot)	121
4.1.5	Biblioteca estándar de acceso a directorios	122
4.2	Acceso a ficheros especiales	129
4.2.1	Entrada/salida sobre terminales	129
4.2.2	Control de terminales	135
4.3	Administración del sistema de ficheros	137
4.3.1	Montar y desmontar un sistema de ficheros (mount y umount)	137

4.3.2	Consistencia del sistema de ficheros (sync)	138
4.3.3	Estado de un sistema de ficheros	139
4.4	Ejercicios	144

II Procesos e hilos 157

5	Estructura de un proceso	159
5.1	Programas y procesos	159
5.2	Estados de un proceso	161
5.3	Tabla de procesos y área de usuario	164
5.4	Contexto de un proceso	165
5.5	Hilos y procesos ligeros	167
5.5.1	Hilos del núcleo	169
5.5.2	Procesos ligeros	169
5.5.3	Hilos del usuario	170
5.6	Distribución y planificación de procesos	171
5.7	Sistemas con requisitos de tiempo real	173
5.7.1	Planificación con prioridades fijas	173
5.7.2	Planificación con prioridades dinámicas	179
5.7.3	Planificación de tareas dependientes	179
6	Gestión de procesos e hilos	183
6.1	Ejecución de programas mediante exec	183
6.2	Creación de procesos (fork)	188
6.3	Terminación de procesos (exit y wait)	189
6.4	Información sobre procesos	195
6.4.1	Identificadores de proceso	195
6.4.2	Identificadores de usuario y de grupo	197
6.4.3	Variables de entorno	198
6.4.4	Parámetros relativos a ficheros	200
6.5	Control de la memoria asignada a un proceso	201
6.5.1	Asignación dinámica de memoria	202
6.5.2	<i>Sticky bit</i>	210
6.5.3	Bloqueo de memoria	211
6.6	Creación de hilos	212
6.7	Tratamiento de los errores	215
6.8	Atributos de un hilo	217
6.9	Cancelación de hilos	220
6.10	Funciones seguras y reentrantes	225
6.10.1	Funciones seguras para un uso concurrente	225
6.10.2	Funciones reentrantes	226
6.11	Planificación de hilos	227
6.12	Ejercicios	233

7	Señales y funciones de tiempo	239
7.1	Concepto de señal	239
7.2	Tipos de señales	240
7.3	Señales en el UNIX System V	243
7.3.1	Envío de señales (kill y raise)	243
7.3.2	Tratamiento de señales (signal)	245
7.3.3	En espera de señales (pause)	249
7.3.4	Salto globales (setjmp y longjmp)	250
7.4	Señales en el sistema 4.3BSD	252
7.4.1	Tratamiento de señales (sigvec)	253
7.4.2	Protección de zonas críticas	259
7.4.3	En espera de señales (sigpause)	260
7.4.4	Concepto de máquina virtual en el sistema BSD	262
7.5	Gestión de señales en POSIX	262
7.5.1	Tratamiento de señales (sigaction)	262
7.5.2	Envío de señales	267
7.5.3	Máscara de señales	268
7.5.4	Recepción síncrona de señales	274
7.6	Ejemplo de aplicación de las señales	276
7.6.1	Software tolerante a fallos	276
7.6.2	Sincronización de procesos	278
7.7	Funciones de tiempo	281
7.7.1	Fijación de la fecha del sistema (stime)	281
7.7.2	Lectura de la fecha del sistema (time y gettimeofday)	282
7.7.3	Tiempos de ejecución asociados a un proceso (times)	285
7.7.4	Temporizadores	288
7.8	Relojes, retardos y temporizadores en POSIX	294
7.8.1	Retardos	294
7.8.2	Temporizadores	295
7.9	Ejercicios	298
8	Perfilado, contabilidad y depuración	309
8.1	Perfil de un proceso	309
8.2	Contabilidad	315
8.3	Depuración de programas	322
III	Comunicación entre procesos	331
9	Comunicación mediante tuberías	333
9.1	Comunicación entre procesos	333
9.2	Tuberías sin nombre	334
9.3	Comunicación bidireccional	337
9.4	Tuberías en los intérpretes de órdenes	340
9.5	Tuberías con nombre	342
9.6	Comunicación dúplex	350

9.6.1	Interrogación periódica	351
9.6.2	Lectura conducida por eventos	356
9.6.3	Multiplexación mediante select	357
9.7	Ejercicios	359
10	Comunicación local entre procesos e hilos	369
10.1	Mecanismos <i>IPC</i> del UNIX System V	369
10.1.1	Formación de llaves	370
10.1.2	Control de las facilidades <i>IPC</i> desde la línea de órdenes	371
10.2	Semáforos	372
10.2.1	Conceptos generales	372
10.2.2	Petición de semáforos (semget)	373
10.2.3	Control de las estructuras de semáforo (semctl)	374
10.2.4	Operaciones $\mathcal{P}$ y $\mathcal{V}$ (semop)	376
10.2.5	Ejemplo de aplicación. Sincronización de procesos	377
10.3	Memoria compartida	379
10.3.1	Petición de memoria compartida (shmget)	379
10.3.2	Control de una zona de memoria compartida (shmctl)	380
10.3.3	Operaciones con la memoria compartida (shmat y shmdt)	381
10.3.4	Ejemplo: multiplicación concurrente de matrices	382
10.4	Colas de mensajes	387
10.4.1	Petición de una cola de mensajes (msgget)	389
10.4.2	Control de las colas de mensajes (msgctl)	389
10.4.3	Operaciones con colas de mensajes (msgsnd y msgrcv)	390
10.4.4	Ejemplo: bases de datos centralizadas	391
10.5	Semáforos en POSIX	397
10.5.1	Críticas a los semáforos	399
10.6	Cerros en POSIX	400
10.6.1	Atributos de un cerrojo	402
10.7	Monitores con interfaz de procedimiento	405
10.7.1	Variables de condición en POSIX	406
10.7.2	Atributos de las variables de condición	415
10.8	Ejercicios	415
11	Comunicaciones en red	423
11.1	Mecanismos <i>IPC</i> del sistema BSD	423
11.1.1	Protocolos y conexiones	426
11.1.2	Direcciones de red	427
11.1.3	Modelo cliente-servidor	428
11.1.4	Esquema general de un servidor y de un cliente	429
11.2	Llamadas para el manejo de conectores	430
11.2.1	Apertura de un punto terminal en un canal (socket)	431
11.2.2	Nombre de un conector (bind)	433
11.2.3	Disponibilidad para recibir peticiones de servicio (listen)	434
11.2.4	Petición de conexión (connect)	435
11.2.5	Aceptación de una conexión (accept)	435

11.2.6	Lectura o recepción de mensajes de un conector	436
11.2.7	Escritura o envío de mensajes a un conector	438
11.2.8	Cierre del canal (<code>close</code>)	439
11.3	Ejemplos de servidores y clientes	439
11.3.1	Ejemplos con conectores de la familia <code>AF_UNIX</code>	440
11.3.2	Ejemplos con conectores de la familia <code>AF_INET</code>	452
11.4	Miscelánea de llamadas y funciones	464
11.4.1	Nombres de un conector (<code>getsockname</code> y <code>getpeername</code>)	464
11.4.2	Nombre del nodo actual (<code>gethostname</code>)	464
11.4.3	Construcción de tuberías a partir de conectores (<code>socketpair</code>)	465
11.4.4	Cierre de un conector (<code>shutdown</code>)	465
11.4.5	Lectura del fichero <code>/etc/hosts</code>	466
11.5	Ejemplo de aplicación. Transferencia de ficheros	467
11.6	Ejercicios	483

IV Apéndices 485

A	El lenguaje de programación C	487
A.1	Introducción	487
A.2	Ciclo de creación de un programa	488
A.2.1	Componentes léxicos del lenguaje	488
A.2.2	Estructura de un programa C	489
A.3	Tipos de datos	490
A.3.1	Tipos fundamentales	490
A.3.2	Operador <code>sizeof</code>	491
A.3.3	Tipos derivados	492
A.3.4	Punteros	492
A.3.5	Alias para los nombres de tipo	496
A.4	Expresiones y operadores	497
A.4.1	Operadores aritméticos	497
A.4.2	Operadores de relación y lógicos	498
A.4.3	Operadores para el manejo de bits	498
A.4.4	Expresiones abreviadas	499
A.5	Sentencias de control de flujo	499
A.5.1	Proposiciones y bloques	499
A.5.2	Selección (<code>if-else</code>)	500
A.5.3	Selección múltiple (<code>else-if</code>)	500
A.5.4	Selección por casos (<code>switch</code>)	501
A.5.5	Bucle <code>for</code>	502
A.5.6	Bucle (<code>while</code>)	502
A.5.7	Bucle (<code>do-while</code>)	502
A.5.8	<code>break</code> y <code>continue</code>	503
A.6	Funciones	503
A.6.1	Paso de parámetros por valor y por referencia	504

B	Desarrollo de aplicaciones en el entorno UNIX	509
B.1	Programación modular	509
B.2	Fases de la compilación	511
B.3	Compilación de programas multimodulares	512
B.4	El preprocesador de C	513
B.5	Ejemplo de programa con varios ficheros	516
B.6	Gestión de bibliotecas de funciones	519
B.6.1	Programas <code>ar</code> y <code>ranlib</code>	519
B.6.2	Programa <code>nm</code>	523
B.7	Distribución de los ficheros de una aplicación	524
B.8	Automatización de trabajos	526
B.9	Documentación	531
B.10	Ejercicios	536
C	Resumen de llamadas al sistema	541
	Referencias	575
	Índice alfabético	581

Índice de figuras

1.1	Arquitectura del sistema UNIX.	3
1.2	Diagrama de bloques del núcleo.	4
2.1	Primer nivel en la estructura de un sistema de ficheros.	15
2.2	Reserva de espacio para ficheros y fragmentación del espacio libre.	19
2.3	Tabla de direcciones de bloque de un nodo-i.	20
2.4	Ejemplo de tabla de direcciones de un nodo-i.	22
2.5	Ejemplo de estructura del directorio <code>/etc</code> para el UNIX System V.	24
2.6	Formato de las entradas de un directorio en el sistema BSD.	26
2.7	Organización del disco en el sistema BSD.	31
2.8	Tablas de control de acceso a los ficheros: tabla de descriptores, tabla de ficheros y tabla de nodos-i.	32
2.9	Ficheros estándar de entrada, salida y salida de mensajes de error de un proceso.	34
3.1	Máscara de modo de un fichero.	79
3.2	Estructura de los ficheros a ordenar.	97
4.1	Estructura lógica de directorios después de montar el sistema de ficheros <code>/dev/dsk/0s2</code> sobre el directorio <code>/usr</code> del sistema raíz.	138
5.1	Estados de un proceso y transición entre estados.	162
5.2	Diagrama completo de transición de estados de un proceso.	163
5.3	Componentes del contexto de un proceso.	168
6.1	Representación de los argumentos <code>argc</code> y <code>argv</code>	185
6.2	Creación de un nuevo contexto de proceso mediante <code>fork</code>	190
6.3	Sincronización entre los procesos padre e hijo.	193
6.4	Estructuras empleadas por <code>rmem</code> y <code>lmem</code> para gestionar la memoria.	204
6.5	Estructura de una cola.	207
7.1	Tratamiento de las señales recibidas por un proceso.	241
7.2	Salto globales con <code>setjmp</code> y <code>longjmp</code>	251
7.3	Acción de la macro <code>sigmask</code>	254
7.4	Sincronización de dos procesos mediante el intercambio de señales.	279

9.1	Primer ejemplo de comunicación a través de tuberías.	336
9.2	Comunicación bidireccional usando dos tuberías.	337
9.3	Comunicación bidireccional con protocolo.	338
9.4	Uso de tuberías para redirigir los ficheros estándar de entrada y salida. . . .	341
9.5	Lectura y escritura en una tubería con nombre.	343
9.6	Estructura de los canales de comunicación de un programa conducido por mensajes.	358
10.1	Representación gráfica del tipo <code>matriz</code>	388
10.2	Estructura de una cola de mensajes.	388
10.3	Flujo de mensajes entre los procesos cliente y el gestor de la base de datos. .	392
10.4	Red de Petri que modela el sistema productor-consumidor.	417
10.5	Estructura de una RdP.	419
10.6	Disparo de una transición. Arcos con peso igual a la unidad.	419
10.7	Disparo de una transición. Arcos con peso superior a la unidad.	420
11.1	Estructura en capas del modelo <i>OSI</i> para comunicación entre ordenadores. .	424
11.2	Estructura de los programas servidores y clientes.	430
11.3	Secuencia de llamadas para una comunicación orientada a conexión.	431
11.4	Secuencia de llamadas para una comunicación no orientada a conexión. . . .	432
11.5	Protocolos <i>Internet</i>	453
11.6	Formatos de almacenamiento de palabras multibyte.	457
11.7	Comunicación entre los servidores y el cliente de la aplicación <code>rcopiar</code>	467
B.1	Fases del proceso de compilación.	511
B.2	Compilación de un programa formado por varios módulos.	513
B.3	Estructura de un polinomio.	516
B.4	Distribución de los módulos de la aplicación <code>polinomios</code>	525
B.5	Página del manual para la función <code>leer_polinomio</code>	535

Índice de programas

1.1	Listado de códigos de error (errno.c)	7
1.2	Ejemplo para probar la función error (ej1-2.c)	9
3.1	Copiar ficheros. Primera versión (copiar.c)	52
3.2	Copiar ficheros. Segunda versión (fcopiar.c)	62
3.3	Fichero de cabecera para la biblioteca estándar de E/S (mstdio.h)	64
3.4	Biblioteca estándar de E/S: implementación de mfopen y mfclose (mfopen.c)	66
3.5	Biblioteca estándar de E/S: implementación de m_fillbuf y m_flushbuf (mfio.c)	67
3.6	Biblioteca estándar de E/S: implementación de mfflush (mfflush.c)	69
3.7	Copiar ficheros. Tercera versión (mfcopiar.c)	70
3.8	Compilación de la biblioteca estándar de E/S (makefile)	71
3.9	Uso de fcntl para realizar accesos con bloqueo a ficheros (fcntl.c)	74
3.10	Lectura y presentación del nodo-i de un fichero (estado.c)	84
3.11	Bloqueo de ficheros, con comprobación previa y sin ella (bloquear.c)	89
3.12	Implementación de una versión simplificada de lockf (lockf.c)	91
4.1	Recorrido de directorios, orden tree (tree.c)	126
4.2	Envío de mensajes a un usuario (mensaje-para.c)	131
4.3	Nueva versión de utmp.h para implementar la función getutent (utmp.h)	133
4.4	Implementación de getutent (getutent.c)	133
4.5	Forma de usar ioctl (entrada.c)	135
4.6	Estado de un sistema de ficheros – lectura del superbloque (estado-sf.c)	141
5.1	Programa para planificar tareas con el criterio <i>DMS</i> (dms.c)	176
6.1	Espera por un acontecimiento (esperar-por.c)	186
6.2	Ejemplo de uso de fork (fork.c)	188
6.3	Creación y terminación de procesos hijo (procesos.c)	192
6.4	Codificación de la función system (sistema.c)	194
6.5	Uso de las llamadas de identificación de procesos (pid.c)	196
6.6	Gestión dinámica de memoria (rmem.c)	203
6.7	Aplicación de la gestión dinámica de memoria (mensajes.c)	207
6.8	Creación de hilos y espera hasta su terminación (hilos.c)	214
6.9	Macro para facilitar la gestión de errores (errores.h)	216
6.10	Atributos de un hilo (atributos.c)	218
6.11	Cálculo con refinamientos progresivos y limitación temporal. (exp.c)	222
6.12	Planificación y distribución de hilos (plan.c)	230
7.1	Ejemplo del uso de kill (kill-1.c)	244

7.2	Uso de <code>signal</code> (<code>signal-1.c</code>)	246
7.3	Uso de <code>pause</code> (<code>pause-1.c</code>)	249
7.4	Uso de <code>setjmp</code> y <code>longjmp</code> (<code>setjmp-1.c</code>)	251
7.5	Uso de <code>sigvec</code> (<code>sigvec-1.c</code>)	255
7.6	Uso de <code>sigvec</code> (<code>sigvec-2.c</code>)	256
7.7	Uso de <code>sigvec</code> (<code>sigvec-3.c</code>)	258
7.8	Uso de <code>sigsetmask</code> y <code>sigblock</code> (<code>sigmsk-1.c</code>)	259
7.9	Protección de secciones críticas (<code>sigmsk-2.c</code>)	261
7.10	Macros para gestionar excepciones (<code>exception.h</code>)	270
7.11	Funciones invocadas por las macros <code>try-catch-finally</code> (<code>exception.c</code>) . .	271
7.12	Prueba de la estructura de control <code>try-catch-finally</code> (<code>excep.c</code>)	274
7.13	Gestión de los fallos en coma flotante (<code>sigfpe.c</code>)	277
7.14	Sincronización mediante el intercambio de señales (<code>sincro.c</code>)	278
7.15	Medida del tiempo de ejecución de un programa (<code>timeval.c</code>)	284
7.16	Tiempos de ejecución asociados a un proceso. (<code>tiempo.c</code>)	286
7.17	Implementación de la función <code>sleep</code> (<code>dormir.c</code>)	288
7.18	Ejemplo de implementación de la orden <code>at</code> (<code>a-las.c</code>)	291
8.1	Creación del perfil de un proceso (<code>perfil.c</code>)	311
8.2	Contabilidad de procesos (<code>cont.c</code>)	317
8.3	Depurador basado en <code>ptrace</code> (<code>dep.c</code>)	325
8.4	Programa que depuraremos con <code>dep</code> (<code>traza.c</code>)	329
9.1	Envío de mensajes mediante tuberías sin nombre (<code>pipe-1.c</code>)	335
9.2	Envío de mensajes entre dos procesos mediante dos tuberías sin nombre (<code>pipe-2.c</code>)	338
9.3	Uso de las tuberías por parte del intérprete de órdenes (<code>tuberia.c</code>)	340
9.4	Simulación de conversaciones por radio (<code>llamar-a.c</code>)	345
9.5	Simulación de conversaciones por radio (<code>responder-a.c</code>)	348
9.6	Comunicación dúplex mediante interrogación periódica (<code>llamar.c</code>)	351
10.1	Sincronización de procesos mediante semáforos (<code>sem-1.c</code>)	377
10.2	Multiplicación concurrente de matrices (<code>matrices.c</code>)	383
10.3	Cabecera para los clientes y el gestor de una base de datos centralizada (<code>censo.h</code>)	392
10.4	Gestor de una base de datos centralizada (<code>gestor.c</code>)	393
10.5	Cliente de una base de datos centralizada (<code>cliente.c</code>)	395
10.6	Definición del tipo abstracto <code>cola</code> (<code>colas.h</code>)	408
10.7	Operaciones del tipo abstracto <code>cola</code> (<code>colas.c</code>)	409
10.8	Comunicación entre hilos (<code>prueba-colas.c</code>)	412
11.1	Fichero de cabecera (<code>scomun.h</code>)	440
11.2	Fichero con funciones comunes para los ejemplos de conectores (<code>scomun.c</code>)	441
11.3	Fichero de cabecera (<code>u-addr.h</code>)	442
11.4	Funciones para comunicarse con conectores del tipo <code>STREAM</code> (<code>str-com.c</code>) . .	443
11.5	Servidor que usa conectores <code>AF_UNIX</code> del tipo <code>SOCK_STREAM</code> (<code>u-str-se.c</code>) .	445
11.6	Cliente que usa conectores <code>AF_UNIX</code> del tipo <code>SOCK_STREAM</code> (<code>u-str-cl.c</code>) . .	447
11.7	Funciones para comunicarse con conectores del tipo <i>datagrama</i> (<code>dg-com.c</code>)	448
11.8	Servidor que usa conectores <code>AF_UNIX</code> del tipo <i>datagrama</i> (<code>u-dg-se.c</code>) . . .	450
11.9	Cliente que usa conectores <code>AF_UNIX</code> del tipo <i>datagrama</i> (<code>u-dg-cl.c</code>)	451

11.10	Fichero de cabecera (<code>i-addr.h</code>)	458
11.11	Serv. concurrente con conectores <code>AF_INET</code> del tipo <code>SOCK_STREAM</code> (<code>i-tcp-se.c</code>)	458
11.12	Cliente con conectores <code>AF_INET</code> del tipo <code>SOCK_STREAM</code> (<code>i-tcp-cl.c</code>)	459
11.13	Servidor interactivo con conectores <code>AF_INET</code> del tipo <code>SOCK_DGRAM</code> (<code>i-udp-se.c</code>)	460
11.14	Cliente con conectores <code>AF_INET</code> del tipo <code>SOCK_DGRAM</code> (<code>i-udp-cl.c</code>)	461
11.15	Compilación de los distintos programas cliente – servidor (<code>makefile.1</code>) . .	462
11.16	Fichero de cabecera (<code>rcopiar.h</code>)	468
11.17	Funciones comunes para los clientes y servidores de la utilidad de copia remota (<code>rcop-com.c</code>)	469
11.18	Servidor de la utilidad de copia remota de ficheros (<code>rcopiars.c</code>)	472
11.19	Cliente de la utilidad de copia remota de ficheros. (<code>rcopiar.c</code>)	478
11.20	Fichero para la compilación de las utilidades de transferencia remota de ficheros. (<code>makefile.2</code>)	482
A.1	Multiplicación de matrices (<code>matr.c</code>)	505

Índice de tablas

2.1	Capacidad de direccionamiento de los bloques de direcciones de un nodo-i. Suponemos bloques de 1 Kbyte.	21
3.1	Constantes definidas en <code><sys/stat.h></code> para el modo de un fichero.	80
4.1	Modos de un fichero.	119
5.1	Atributos de la tarea τ_i	173
5.2	Sistema formado por 4 tareas con $D_i \leq T_i$	175
5.3	Resultado de la planificación de las tareas descritas en la tabla 5.2.	176
6.1	Composición de un fichero ejecutable.	186
6.2	Normas POSIX y sus extensiones.	212
6.3	Puntos de cancelación.	221
6.4	Perfiles definidos en POSIX 1003.13 para sistemas de tiempo real.	228
7.1	Señales del UNIX System V.	244
7.2	Comparación entre la máquina hardware y la máquina virtual UNIX.	262
7.3	Señales POSIX.	263
7.3	Señales POSIX (continuación).	264
7.4	Macros para consultar el campo <code>si_code</code>	265
7.4	Macros para consultar el campo <code>si_code</code> (continuación).	266
7.5	Manejador asociado a cada señal.	298
7.6	Mensaje asociado a cada señal.	298
7.7	Punto de repliegue asociado a cada señal.	300
7.8	Sesión de trabajo con el programa <code>saltos</code>	301
10.1	Semáforos de la aplicación <i>productores-consumidores</i>	420
10.2	Ciclos básicos de los productores y los consumidores.	421
11.1	Familias de direcciones.	433
A.1	Funciones asociadas a los operadores lógicos.	498
A.2	Expresiones abreviadas.	499
A.3	Ejecución de la proposición <code>else-if</code>	501

Prólogo

UNIX es un sistema operativo que goza de gran popularidad en los entornos industriales, académicos y, recientemente, incluso domésticos. Su implantación en máquinas con gran capacidad de proceso gráfico y de cálculo es cada vez mayor. Se puede decir que casi todas las estaciones de trabajo y miniordenadores dedicados a tareas específicas, como CAD/CAM, autoedición, diseño gráfico, servidores de disco, servidores de páginas *web*, etc., se sirven de este sistema operativo desarrollado a principios de los años setenta.

Es mucha la bibliografía que explica la forma de usar el sistema en sus tareas más comunes: inicio de una sesión de trabajo, edición de documentos, administración de un árbol de directorios, impresión, etc. Son menos los textos que tratan sobre la administración del sistema, por lo que, generalmente, el usuario administrador tiene que enfrentarse a la documentación técnica que suministra el fabricante que, en última instancia, es quien tiene la palabra. Pero, desde luego, el terreno menos documentado sobre UNIX es su entorno de programación. Al margen de las herramientas de desarrollo de aplicaciones, como pueden ser los compiladores, los programas depuradores, editores, archivadores, etc., UNIX le brinda al programador un juego de piezas básicas, conocidas como *llamadas al sistema*, que posibilitan la escritura de programas que explotan al máximo los recursos del sistema.

Haciendo uso de las llamadas al sistema, el programador podrá dar solución a sus necesidades de escribir programas claros, rápidos y fiables que:

- Hagan uso del sistema de ficheros,
- ejecuten varios procesos en paralelo o concurrentemente,
- implementen algunos aspectos de programación en tiempo real,
- sean tolerantes ante los fallos imprevistos,
- se recuperen de las condiciones de error y
- se comuniquen con otros programas a través de una red local.

El presente libro hace un recorrido por las técnicas de programación necesarias para crear aplicaciones con las características anteriores y muestra cómo implementarlas a partir del conjunto de llamadas al sistema UNIX y sus funciones de biblioteca asociadas.

El libro está organizado en tres partes que guardan estrecha relación con la estructura del sistema UNIX. Tras un primer capítulo de introducción, en el que se presenta la

arquitectura del sistema, se inicia el estudio de la primera parte, dedicada por completo al sistema de ficheros de UNIX. Esta parte consta de tres capítulos y analiza los siguientes temas:

- Arquitectura del sistema de ficheros.
- Manejo de ficheros ordinarios.
- Manejo de directorios y ficheros especiales.

Tras el estudio del sistema de ficheros, pasamos a ver los conceptos de proceso y de hilo en torno a los cuales se construye la segunda parte del libro y donde se estudian los siguientes temas:

- Estructura de un proceso.
- Gestión de procesos e hilos.
- Señales y funciones de tiempo.
- Perfilado, contabilidad y depuración.

El objeto de estudio de la tercera parte es la comunicación entre procesos y entre hilos, tanto si éstos se están ejecutando en una misma máquina —comunicación local—, como si lo hacen en máquinas distintas conectadas por medio de una red —comunicación remota—. Los tres capítulos de esta parte estudian los siguientes temas:

- Comunicación mediante tuberías.
- Comunicación local entre procesos e hilos.
- Comunicaciones en red.

Para presentar todos estos temas, junto con las explicaciones teóricas, se incluye el código de alrededor de 90 programas en los que se intenta dar al lector una visión sobre cómo enfocar una aplicación con UNIX. La mayor parte de los programas constituyen versiones simplificadas de las órdenes UNIX que se suelen emplear durante una sesión de trabajo. El hecho de que un usuario pueda escribir sus propias órdenes da una idea de la flexibilidad del sistema. Algunos de los programas presentados no son triviales, por lo que recomiendo al lector que, antes de realizar una lectura pormenorizada de los mismos, realice una lectura para captar su arquitectura. He procurado escribir programas claros y con abundantes comentarios para ayudar a la comprensión de los mismos, sin embargo soy consciente de que el lenguaje C, a veces, favorece la programación con un estilo lleno de expresiones idiomáticas que hacen excesivamente lacónicos los programas. Los programas han sido compilados con diferentes máquinas y diferentes versiones de UNIX¹, con objeto de vigilar por su transportabilidad. La experiencia me ha demostrado que los esfuerzos

¹Algunos de los sistemas con los que he desarrollado las aplicaciones han sido Linux —distribución Red Hat—, FreeBSD, AIX —de la firma IBM—, HP-UX —de la firma Hewlett-Packard— y XENIX —de la firma SCO—.

realizados para asegurar la compatibilidad de un programa a la larga suponen un gran ahorro en tiempo y recursos humanos.

El lenguaje de programación más utilizado para programar con UNIX es C. Este lenguaje fue desarrollado para poder reescribir el sistema en un lenguaje de alto nivel, por ello es la forma natural de escribir programas para el sistema. Es aconsejable que el lector que accede al libro sepa programar en C, o por lo menos tenga conocimiento de la programación estructurada en lenguajes de alto nivel. Con objeto de dotar al libro de cierta autonomía, he incluido un apéndice en el que se repasan los aspectos más importantes de la programación en C. He recibido sugerencias por parte de los lectores de ediciones anteriores para que amplíe este apéndice y con mucho gusto realizaría tal tarea, pero me temo que, por mucho que lo ampliase, ese apéndice no pasaría de ser un resumen muy reducido de un tema muy amplio como es la programación en C. No obstante, ya a partir de la segunda edición añadí un nuevo apéndice titulado «*Desarrollo de Aplicaciones en el entorno UNIX*», que aborda un tema poco tratado por la mayor parte de los libros de programación que hay en el mercado. En él se realiza una exposición general de los pasos más frecuentes que se deben seguir para el desarrollo de una aplicación multimodular y de las herramientas estándar que UNIX pone a nuestra disposición. En concreto, los puntos más importantes que se abordan son:

- Concepto de programación modular.
- Compilación de programas multimodulares.
- Gestión de bibliotecas de funciones.
- Distribución de los ficheros de una aplicación.
- Automatización de trabajos.
- Documentación.

Si el lector tiene algún problema a la hora de seguir el libro porque el lenguaje C le supone una barrera, le aconsejo que consulte el excelente libro del profesor Francisco Javier Ceballos Sierra titulado *C/C++ Curso de programación*, 2ª edición editado por RAMA [Ceballos, 2001]. Por otro lado, si el lector está interesado en ampliar sus conocimientos en el uso del sistema operativo UNIX a nivel de órdenes, puede consultar el libro del profesor Sebastián Sánchez Prieto titulado *UNIX y Linux: Guía práctica*. 2ª edición editado también por RAMA [Sánchez, 2001].

Ha sido grato para mí conocer, a través de la correspondencia que los lectores han enviado tanto a la editorial como a mi buzón de correo electrónico, la buena acogida que ha tenido el libro en el entorno universitario y profesional; esto me ha animado para elaborar esta tercera edición corregida y ampliada. Asimismo, he sabido que el contenido del libro se ajusta en gran medida al programa de la asignatura de *Laboratorio de Sistemas Operativos* que se imparte en muchas universidades españolas formando parte de planes de estudio como los de Ingeniería Informática en Ingeniería de Telecomunicaciones. Por ello he dedicado un gran esfuerzo a la confección de los ejercicios. Así, en esta tercera edición, aparecen más de 100 ejercicios con una doble finalidad: servir de entrenamiento para que el lector pueda practicar y asentar las ideas expuestas en la parte teórica de

cada capítulo y ampliar algunos conceptos muy específicos o poco estándar que no se mencionan en la parte teórica.

Los ejercicios del libro se pueden clasificar en dos categorías:

- Ejercicios de aplicación directa de los conceptos expuestos en la teoría. Estos ejercicios pueden resolverse en unos minutos y sólo exigen consultar y trabajar con el material explicado en el capítulo. Suelen ser preguntas de respuesta corta o programas de fácil construcción.
- Ejercicios de desarrollo libre de aplicaciones. Estos ejercicios persiguen que el lector aborde la construcción de programas complejos y que se familiarice tanto con los problemas que la construcción de estos programas plantea como con las soluciones que se pueden dar a los mismos. Estos ejercicios pueden utilizarse para tomar ideas de cara a construir alguna práctica de laboratorio como el de *Sistemas Operativos*, ya que van acompañados de una exposición bastante detallada tanto de los objetivos que persiguen como de las especificaciones que deben cumplir las aplicaciones finales.

Lo ideal es que el lector se enfrente a los ejercicios e intente resolverlos con todos los medios a su alcance. Éste es el mejor método para asentar las ideas que se exponen en cada capítulo. No obstante, he preparado un cuadernillo con las soluciones de los ejercicios, que los lectores pueden obtener a través de la editorial en las direcciones www.ra-ma.es y www.ra-ma.com.

Algunos lectores me han comunicado que sería interesante que ampliara el capítulo dedicado a las comunicaciones en red. Ese capítulo está concebido como una primera toma de contacto para un programador con conocimientos sobre llamadas al sistema. El tema de las redes de comunicaciones y las interfaces que se han diseñado para usarlas desde UNIX es tan extenso que tratarlo en toda su profundidad requeriría varios volúmenes; sirva de ejemplo la obra de Stevens y Wright *TCP/IP Illustrated* —3 volúmenes: [Stevens, 1994], [Wright & Stevens, 1995] y [Stevens, 1996]— y la obra de Comer y Stevens *Internetworking with TCP/IP* —3 volúmenes: [Comer, 2000], [Comer & Stevens, 1999] y [Comer, 2001]—.

Cambios con respecto a ediciones anteriores

La presente edición ha sido procesada con L^AT_EX 2_ε y METAPOST con lo que ha mejorado, a mi modo de ver, en su aspecto tipográfico.

Por lo que respecta al contenido se ha conservado todo el contenido de la edición anterior —corregido y actualizado— y se ha añadido material nuevo para describir la interfaz de programación POSIX, distribuido de la siguiente forma:

- *Capítulo 5.* Hilos y procesos ligeros. Distribución y planificación de procesos. Sistemas con requisitos de tiempo real.
- *Capítulo 6.* Creación de hilos. Tratamiento de los errores. Atributos de un hilo. Cancelación de hilos. Funciones seguras y reentrantes. Planificación de hilos.
- *Capítulo 7.* Gestión de señales en POSIX. Relojes, retardos y temporizadores en POSIX.

- *Capítulo 10.* Semáforos en POSIX. Cerrojos en POSIX. Monitores con interfaz de procedimiento. Variables de condición en POSIX.
- *Capítulo 11.* Descripción de las nuevas familias y tipos de conectores.

Es mucho lo que se puede escribir sobre la programación en el entorno UNIX, sobre todo si se estudian las interfaces de las extensiones que cada fabricante realiza en su propia versión de UNIX y las llamadas para trabajar con redes de ordenadores —algunas de las cuales no han llegado a tener aceptación en el mercado— o con entornos gráficos —en este aspecto el sistema X-WINDOW ha sido unánimemente aceptado—.

Ante la proliferación de versiones de UNIX² y sus correspondientes interfaces han surgido intentos de normalización que han culminado con la creación en 2001 de una interfaz común —conocida como «*Single UNIX Specification Version 3*»— fruto de la colaboración entre *The Open Group* y el *Institute of Electrical and Electronics Engineers Inc. (IEEE)*. Este esfuerzo de unificación recoge una idea iniciada por el *IEEE* a finales de los años ochenta para crear una *interfaz de aplicación para sistemas abiertos* que dio lugar a la familia de normas *IEEE Std. 1003.1* —también conocidas como normas POSIX— y cuya última versión es la ya mencionada *Single UNIX Specification Version 3* publicada conjuntamente por el *IEEE* y *The Open Group* con el título *1003.1TM Standard for Information Technology – Portable Operating System Interface (POSIX®) – System Interfaces, Issue 6* [IEEE, 2001b] y que puede ser consultada en www.UNIX-systems.org.

Expansión de los sistemas *libres*

No puedo dejar de comentar que en el período transcurrido desde la aparición de la primera edición del libro hasta hoy se ha producido una enorme expansión del sistema operativo Linux. El primer núcleo de este sistema fue codificado por Linus Torvald en 1993 y ha crecido con la aportación altruista de miles de programadores de todo el mundo guiados por dos únicos intereses: el amor por las ciencias de la computación y el deseo de que el software sea patrimonio cultural de la humanidad, y que hasta el estudiante más modesto pueda tener acceso a los últimos avances en ciencias de la computación. El canal de comunicación entre esta pléyade de programadores es la red Internet y no deja de ser sorprendente que un sistema como es Linux, creado por tanta gente, sin la financiación de grandes multinacionales y sin pretensiones de ser el sistema del futuro, sea a la vez tan robusto, abierto y barato. Sobre todo barato, su coste asciende a cero céntimos. Para conseguirlo sólo es necesario un punto de conexión a la red Internet y una dirección de contacto. He aquí la principal: www.linux.org. A partir de ella podemos encontrar distribuciones de Linux en diferentes idiomas —incluido el español—, para diferentes microprocesadores: *Intel*, *Motorola*, *PPC*, *Alpha*, *Sparc*, etc.; y con diferentes propósitos: uso personal, sistemas empuetrados, dispositivos móviles, etc.

El lector no debe sentirse perdido entre la enorme cantidad de distribuciones de Linux que hay, ya que todas ellas comparten algo en común: el núcleo, y sus diferencias se deben a detalles menores como puede ser la forma de instalar una u otra. Es más, los esfuerzos por desarrollar herramientas comunes han llevado a que implementaciones completamente

²Para conocer la evolución de UNIX puede consultar [McKusick *et al.*, 1996] y [Raymond, 2004].

distintas de UNIX como son Linux y FreeBSD tengan ambientes gráficos idénticos como son KDE y GNOME.

Otra de las versiones de UNIX con larga tradición en el entorno universitario es BSD —*Berkeley Software Distributions*— creada en los años ochenta en la Universidad de Berkeley cuando AT&T reclamó la propiedad intelectual de la marca UNIX y del software del sistema. Uno de los principales proyectos en los que se embarcó la comunidad UNIX de Berkeley fue la creación de un sistema para las comunicaciones en red de la *Defense Advance Research Projects Agency (DARPA)*, lo que dio lugar a la creación del sistema 4BSD y de la interfaz de conectores aceptada hoy día como estándar en la industria del software para redes de ordenadores. Al finalizar el aporte económico de *DARPA* al proyecto BSD en los años noventa surge, al igual que ocurre con Linux, una comunidad internacional de usuarios que de forma altruista revisan y mejoran el sistema que ahora pasa a llamarse FreeBSD. Este sistema también es gratuito y lo podemos encontrar en www.freebsd.org. Además está considerado como uno de los más robustos, prueba de ello es que *Apple* ha diseñado su sistema *Mac OS X* incorporando las mejores características de *Mach 3.0* y de FreeBSD 5.

Lo anteriormente expuesto es una de las razones por la que todos los programas, s de este libro, excepto el programa 4.6 que ha sido diseñado para XENIX, han sido compilados en Linux y en FreeBSD con el compilador C del proyecto *GNU* —*gcc*, compilador de distribución gratuita desarrollado por la *FSF (Free Software Foundation)* que forma parte del paquete de desarrollo de aplicaciones tanto de Linux como de FreeBSD y no se ha producido ningún problema por falta de transportabilidad.

En este libro hay algunos ejercicios, alrededor de 20, que han sido diseñados para trabajar sólo con Linux. Estos ejercicios aparecen debidamente indicados, para que el lector no los tenga en cuenta en otras instalaciones UNIX.

Observaciones sobre la terminología

«BERGANZA.—[...] Hay algunos romancistas que en las conversaciones disparan de cuando en cuando con algún latín breve y compendioso, dando a entender a los que no lo entienden que son grandes latinos, y apenas saben declinar un nombre ni conjugar un verbo.

[...]

CIPIÓN.— Para saber callar en romance y hablar en latín, discreción es menester, hermano Berganza.

BERGANZA.— Así es, porque también se puede decir una necedad en latín como en romance, y yo he visto letrados tontos, y gramáticos pesados, y romancistas vareteados con sus listas de latín, que con mucha facilidad pueden enfadar al mundo, no una sino muchas veces.»

(Cervantes, *El coloquio de los perros*)

Las observaciones sobre la terminología que acompañan a cada capítulo en forma de notas a pie de página no tienen como finalidad presumir de gramático sino más bien todo lo contrario, reconocer mis limitaciones —y creo hablar en nombre de muchos hablantes

de español— para adaptar a nuestra lengua el creciente número de términos anglosajones acuñados y adoptados en el ámbito de la informática. Este aluvión terminológico desborda nuestra capacidad de respuesta y la mayor parte de las veces nos limitamos a usar los términos tal y como los importamos, adaptándolos únicamente a nuestro sistema fonético pero conservando la grafía original. Esto suele producir resultados grotescos que nos llevan a expresarnos en jerga *espanglis* con la que conseguimos hacernos entender. Algunas naciones, ante el peligro de colonización cultural, han creado comisiones para salvaguardar la pureza de su lengua, comisiones, como en el caso francés, que publica en el *Journal officiel* listas de términos sancionados de obligado empleo por los investigadores que reciben ayuda estatal para su trabajo [Lázaro-Carreter, 1997, *La adopción de tecnicismos extranjeros*, art. de 1992], o como el Instituto de la Lengua Islandesa responsable de la planificación y preservación de una lengua con poco más de 280.000 hablantes pero que ha acuñado términos como *vélbúnaður*, *hugbúnaður* y *tölva*³ para referirse a *hardware*, *software* y computador; términos, los dos primeros, que la Real Academia Española —RAE— ha incorporado como *préstamos* en su última edición del diccionario —*DRAE*—.

Ante la carencia de una normativa clara en nuestra lengua —si exceptuamos el *DRAE*, única voz autorizada al respecto— que ponga un poco de orden en la terminología científica, me he guiado por la única norma que realmente hace evolucionar una lengua: el uso que los hablantes hacen de ella; siempre poniendo un poco de sentido común para desechar usos que, por exceso de purismo o de ignorancia, rozan el absurdo. He procurado ser comedido en las dos posturas, la de usar directamente los anglicismos y la de adaptar todos los términos a nuestra lengua. Cada vez que introduzca una traducción o deje de hacerlo procuraré justificarlo a través de estas notas léxicas. Para preparar estas notas he consultado, además del *DRAE* [RAE, 2001], el *Diccionario Etimológico de Corominas* [Corominas & Pascual, 1996], el *Vocabulario Científico y Técnico* de la Real Academia de Ciencias Exactas, Físicas y Naturales [RACEFyN, 1996], el *Diccionario Webster* [Webster, 1986] y el *Diccionario de Covarrubias* [Covarrubias, 1611] para los aspectos históricos.

Agradecimientos

En primer lugar quisiera agradecer a Gema Álvarez, a María José Serrano y a Joaquín Cardoso la atención, cariño y comprensión que me han brindado durante estos últimos años. Poder contar con su amistad es un verdadero privilegio para mí.

También quisiera agradecer a todas las personas que, bien a través de la editorial o a través del correo electrónico, me han hecho sugerencias y críticas sobre las ediciones anteriores del libro.

³Podemos encontrarlas en *Orðabanki Íslenskrar málstöðvar* www.ismal.hi.is/ob.

Por último, me resta decir que la presente edición, al igual que las anteriores y al igual que otras muchas creaciones humanas, no estará libre de erratas, por lo que de antemano agradezco todas las sugerencias que ayuden a mejorar su calidad. Con tal fin, los lectores pueden dirigirse al Departamento de Automática, Campus Universitario, Ctra. Madrid-Barcelona, Km, 33,600. 28871 Alcalá de Henares (Madrid). Los lectores también pueden dirigirse, mediante correo electrónico, a la dirección francisco.marquez@uah.es.

Alcalá de Henares
Enero de 2004

Francisco Manuel Márquez García

Capítulo 1

Introducción

«Pero, con todo eso, yo me esforzaré a decir una historia que, si la acierto a contar y no me van a la mano, es la mejor de las historias; y estéme vuestra merced atento, que ya comienzo.»

(Cervantes, Quijote I-20)

1.1	Estructura del sistema	1
1.2	Arquitectura del sistema operativo UNIX	3
1.3	Interfaz de las llamadas al sistema	5
1.4	Ejercicios	8

1.1 Estructura del sistema

UNIX es el núcleo¹ de un sistema operativo de tiempo compartido. El núcleo del sistema es un programa que siempre está residente en memoria y, entre otros, brinda los siguientes servicios:

- Controla los recursos del hardware.
- Controla los dispositivos periféricos (discos, terminales, impresoras, etc.).
- Permite a distintos usuarios compartir recursos y ejecutar sus programas.
- Proporciona un sistema de archivos que administra el almacenamiento de información (programas, datos, documentos, etc.).

En un sentido más amplio, UNIX abarca también un conjunto de programas estándar, como pueden ser:

¹Es frecuente encontrar el término inglés *kernel* para referirse al núcleo de un sistema operativo. Si rastreamos la etimología de esta palabra nos encontramos que procede del inglés antiguo *cyrnel*, relacionada con los siguientes términos: inglés antiguo *corn*, alto alemán *corn*, gótico *kaiurn*, latín *grānum*. *Kernel* tiene el mismo origen que el término castellano *grano*.

- Compilador de lenguaje C (`cc`).
- Editor de texto (`vi`).
- Intérprete de órdenes (`sh`, `ksh`, `csh`).
- Programas de gestión de archivos y directorios (`cp`, `rm`, `mv`, `mkdir`, `rmdir`, etc.).
- Entorno gráfico de ventanas (X-WINDOW).

En la figura 1.1 se pueden ver distintos niveles dentro de la arquitectura UNIX. El nivel más interno —hardware— no pertenece realmente al sistema operativo, es la máquina física en la que se ejecuta el sistema y cuyos recursos queremos gestionar. Directamente en contacto con el hardware se encuentra el núcleo del sistema. Este núcleo está escrito en lenguaje C en su mayor parte, aunque coexistiendo con código en ensamblador. Las primeras implementaciones de UNIX se hicieron en lenguaje ensamblador, pero en 1973 Ritchie reescribió el sistema en lenguaje C, que había sido creado para recodificar UNIX de una forma independiente de la máquina [Kernighan & Ritchie, 1988].

En el tercer nivel de nuestra estructura se encuentran programas estándar de cualquier sistema UNIX —`vi`, `grep`, `sh`, `who`, etc.— y programas generados por el usuario —`a.out` programa ejecutable estándar creado por defecto por el compilador `cc`, o cualquier otro programa ejecutable—. Hay que hacer notar que estos programas del tercer nivel nunca interaccionan con el hardware de forma directa; por lo tanto debe existir algún mecanismo que nos permita indicarle al núcleo que necesitamos operar sobre un determinado recurso hardware. Este mecanismo es lo que se conoce como llamadas al sistema —*system calls*— y es el objeto principal de estudio de este libro. Así pues, cualquier programa que se esté ejecutando bajo el control de UNIX, cuando necesite hacer uso de alguno de los recursos que le brinda el sistema, deberá efectuar una llamada a alguna de las *system calls*.

Por encima del tercer nivel tenemos aplicaciones que se sirven de otros programas ya creados para llevar a cabo su función. Estas aplicaciones no se comunican directamente con el núcleo. Un ejemplo de estos programas es el compilador de lenguaje C (`cc`). Cuando invocamos al compilador con una orden como la siguiente

```
$ cc -o programa programa.c -lm
```

`cc` llama de forma secuencial a los programas:

- Preprocesador de C (`cpp`).
- Compilador de C (`comp`).
- Ensamblador (`as`).
- Enlazador (`ld`).

La jerarquía de programas no tiene por qué verse limitada a cuatro niveles. El usuario puede crear tantos niveles como necesite. Puede haber también programas que se apoyen en diferentes niveles y que se comuniquen con el núcleo por un lado, y con otros programas ya existentes, por otro.

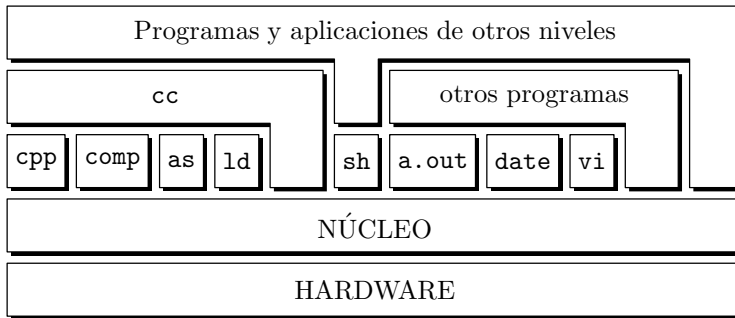


Figura 1.1: Arquitectura del sistema UNIX.

1.2 Arquitectura del sistema operativo UNIX

Vamos a esbozar a continuación, desde un punto de vista muy general, los bloques funcionales básicos de que consta el núcleo de UNIX. Los dos conceptos centrales sobre los que se basa la arquitectura de UNIX son los ficheros y los procesos. El núcleo está pensado para facilitarnos servicios relacionados con el sistema de ficheros y con el control de procesos. La figura 1.2 muestra los tres niveles que vamos a estudiar en la arquitectura del sistema: hardware, núcleo y usuario. Las llamadas al sistema y su biblioteca asociada representan la frontera entre los programas del usuario y el núcleo. La biblioteca asociada a las llamadas es el mecanismo mediante el cual podemos invocar una llamada desde un programa C. Esta biblioteca se enlaza por defecto al compilar cualquier programa C y se encuentra en el fichero `/usr/lib/libc.a`. Los programas escritos en lenguaje ensamblador pueden invocar directamente a las llamadas al sistema sin necesidad de ninguna biblioteca intermedia. Las llamadas al sistema se ejecutan en modo protegido², y para entrar en este modo hay que ejecutar una sentencia en código máquina conocida como *interrupción software* —*trap*—. Es por esto que las llamadas al sistema pueden ser invocadas directamente desde ensamblador y no desde C.

En la figura 1.2 vemos que el núcleo está dividido en dos subsistemas principales: *subsistema de ficheros* y *subsistema de control de procesos*. El subsistema de ficheros controla los recursos del sistema de ficheros y tiene funciones como reservar espacio para los ficheros, administrar el espacio libre, controlar el acceso a los ficheros, permitir el intercambio de datos entre los ficheros y el usuario, etc. Los procesos interactúan con este subsistema a través de unas llamadas específicas (`open`, `read`, `write`, `status`, `chown`, etc.).

El *subsistema de ficheros* se comunica con los dispositivos de almacenamiento secundario —discos duros, unidades de cinta, etc.— a través de los *manejadores de dispositivo* —*device drivers*—. Los manejadores de dispositivo se encargan de proporcionar el protocolo de comunicación —*handshake*— entre el núcleo y los periféricos. Se consideran dos tipos de dispositivos según la forma de acceso: *dispositivos modo bloque* —*block devices*— y *dispositivos modo carácter* —*row devices*—. El acceso a los dispositivos en modo blo-

²También llamado *modo kernel* o *modo supervisor* en la terminología de los microprocesadores.

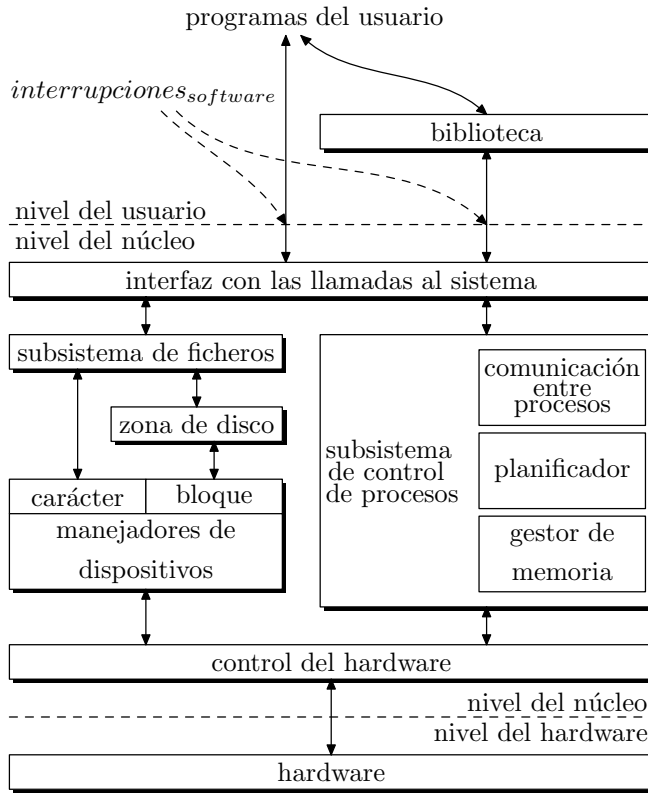


Figura 1.2: Diagrama de bloques del núcleo.

que se lleva a cabo con la intervención de memorias intermedias —*buffers*³— que mejoran enormemente la velocidad de transferencia. El acceso a dispositivos en modo carácter se

³Se han propuesto multitud de traducciones para el término *buffer*, entre otras: antememoria, memoria intermedia, memoria tampón y memoria zulo. La idea de una jerarquía de memorias ya aparece en los primeros trabajos sobre estructura de computadores [Burks *et al.*, 1946] y el concepto de memoria intermedia está relacionado funcionalmente con la jerarquía de memorias. El término *buffer* procede de *buff*, éste del francés *buffle* y éste del italiano *bufalo*. Comparemos esta evolución etimológica con la perteneciente a la palabra *bufalo*, del lat. tardío *bufālus*, éste del lat. *bubālus*, y éste del gr. *βουβαλος*, gacela; parece imposible encontrar una traducción enraizada en el origen de la palabra que a todas luces se muestra onomatopéyico —comparemos con el verbo *bufar*, ya aparece en Covarrubias [Covarrubias, 1611]—, sin embargo, volviendo al término inglés, podemos encontrar una relación clara entre *buffer* y *bufar*; en efecto *buffer* tiene un origen onomatopéyico relacionado con el significado *reaccionar como un cuerpo blando cuando es golpeado*, de ahí que *buffer* tenga la acepción de *dispositivo o pieza empleado para reducir los golpes y el deterioro debido al contacto*, es decir, un amortiguador de los golpes, que por lo general se construye con partes blandas. Una memoria intermedia cumple la función amortiguadora al adaptar las distintas velocidades de dos dispositivos que interactúan, por ejemplo, el procesador y los periféricos. Cuando el procesador envía datos a los periféricos, su rendimiento aumenta enormemente si esos datos son almacenados en una memoria intermedia que los absorbe a la misma velocidad que son generados y se los entrega a un periférico a la velocidad —mucho más lenta— con la que éste los requiere. Con esta misma acepción aparece en [RACEFyN, 1996].

lleva a cabo de forma directa, sin la intervención de estas memorias. Un mismo dispositivo físico puede ser manejado tanto en modo bloque como en modo carácter, dependiendo de qué manejador usemos para acceder a él.

El *subsistema de control de procesos* es el responsable de la planificación de los procesos —*scheduling* o *dispatching*—, su sincronización, comunicación entre los mismos —*IPC inter process communication*— y del control de la memoria principal. Algunas de las llamadas usadas para controlar procesos son: `fork`, `exec`, `exit`, `wait`, `brk`, `signal`, etc. El *módulo de gestión de memoria* se encarga de controlar qué procesos están cargados en la memoria principal en cada instante. Si en un momento determinado no hay suficiente memoria principal para todos los procesos que lo solicitan, el gestor de memoria debe recurrir a mecanismos de *intercambio* —*swapping*— para que todos los procesos tengan derecho a un tiempo mínimo de ocupación de la memoria y se puedan ejecutar. El intercambio consiste en llevar los procesos cuyo tiempo de ocupación de la memoria expira a una memoria secundaria en el disco —área de *swap*— que se monta como un sistema de ficheros aparte, y traer de esa memoria secundaria los procesos a los que se les asigna tiempo de ocupación de la memoria principal. Al módulo gestor de memoria se le conoce también como *intercambiador* —*swapper*—. El *distribuidor* —también *despachador* o *scheduler*— se encarga de gestionar el tiempo de CPU que tiene asignado cada proceso. El distribuidor entra en ejecución controlado por la interrupción del reloj y decide si el proceso actual tiene derecho a seguir ejecutándose —esto depende de su prioridad y de sus privilegios— o ha de conmutarse de contexto —asignarle la CPU a otro proceso—.

La comunicación entre procesos puede realizarse de forma asíncrona —señales— o síncrona —colas de mensajes, semáforos—.

Por último, el *módulo de control del hardware* es la parte del núcleo encargada del manejo de las interrupciones y de la comunicación con la máquina. Los dispositivos pueden interrumpir a la CPU mientras está ejecutando un proceso. Si esto ocurre, el núcleo debe reanudar la ejecución del proceso después de atender a la interrupción. Las interrupciones no son atendidas por procesos, sino por funciones especiales, codificadas en el núcleo, que son invocadas durante la ejecución de cualquier proceso.

1.3 Interfaz de las llamadas al sistema

Las llamadas al sistema de UNIX tienen un formato estándar, tanto en la documentación que nos brinda el sistema sobre ellas —páginas del manual— como en la forma de invocarlas⁴. La documentación de las llamadas se encuentra en la sección 2 de las páginas del manual de UNIX. Cada entrada del manual presenta una sinopsis que muestra cómo se declara la función de la biblioteca que permite realizar la llamada correspondiente. El manual también describe los códigos de retorno de cada función y los códigos de error en caso de que algo falle. Para algunas funciones se muestra un ejemplo indicando cómo se usa la función. Este último aspecto dependerá, como es natural, de la aplicación que estemos diseñando. A continuación se muestra el formato que tiene la documentación del manual para la función `pipe`. La forma de desplegar esta información por pantalla es escribiendo:

⁴Si el lector no está familiarizado con la programación en lenguaje C, es recomendable que lea primero el Apéndice A. *El lenguaje de programación C*.

```
$ man pipe
```

PIPE(2)

PIPE(2)

NAME

pipe - create an interprocess channel

SYNOPSIS

```
int pipe (fildes)
int fildes[2];
```

DESCRIPTION

Pipe creates an I/O mechanism called a pipe and returns two file descriptors, `fildes[0]` and `fildes[1]`. `fildes[0]` is opened for reading and `fildes[1]` is opened for writing.

A read-only file descriptor `fildes[0]` accesses the data written to `fildes[1]` on a first-in-first-out (FIFO) basis. For details of the I/O behavior of pipes see `read(2)` and `write(2)`.

EXAMPLES

The following example uses pipe to implement the command string "ls | sort":

```
#include <sys/types.h>
pid_t pid;
int pipefd[2];

/* Assumes file descriptor 0 and 1 are open */
pipe (pipefd);

if ((pid = fork()) == (pid_t)0) {
    close(1);          /* close stdout */
    dup (pipefd[1]);
    execlp ("ls", "ls", (char *)0);
}
else if (pid > (pid_t)0) {
    close(0); /* close stdin */
    dup (pipefd[0]);
    execlp ("sort", "sort", (char *)0);
}
```

RETURN VALUE

Upon successful completion, a value of 0 is returned. Otherwise, a value of -1 is returned and `errno` is set to indicate the error.

ERRORS

Pipe fails if one or more of the following is true:

[EMFILE]	NFILE - 1 or more file descriptors are currently open.
[ENFILE]	The system file table is full.
[ENOSPC]	The file system lacks sufficient space to create the pipe.

SEE ALSO

sh(1), read(2), write(2), popen(3S).

STANDARDS CONFORMANCE

pipe: SVID2, XPG2, XPG3, POSIX.1, FIPS 151-1

La forma de averiguar si la llamada a la función ha fallado es analizar el valor que devuelve. Este valor es -1 de forma estándar para todas las llamadas cuando se produce un error. Para averiguar cuál es el error producido, hemos de consultar el valor que toma la variable externa **errno**. En el fichero de cabecera **<errno.h>** hay una descripción de todos los valores que puede tomar **errno**. Los códigos de error tendrán un significado u otro según la llamada que los haya generado, por ello es recomendable consultar el manual para clarificar su significado.

A continuación se muestra un programa que visualiza por pantalla todos los códigos de error del sistema y una descripción asociada a cada uno de ellos.

Programa 1.1: Listado de códigos de error (**errno.c**)

```
#include <stdio.h>

main ()
{
    int i;

    /* En el array sys_errlist hay una descripción corta asociada a cada número
    de error, sys_nerr es el total de elementos del array sys_errlist. */
    for (i = 0; i < sys_nerr; i++)
        printf ("%d: %s\n", i, sys_errlist [i]);
}
```

Existen dos formas de obtener el mensaje de error asociado a la variable **errno**: la primera es usar **errno** como índice para acceder al array **sys_errlist** y obtener una cadena de caracteres descriptiva del error; la segunda es usar la función **perror** cuya declaración es:

```
#include <stdio.h>
void perror (char *str);
```

y la forma de invocarla es

```
perror ("Mi mensaje");
```

si, por ejemplo, **errno** vale 2, por pantalla se desplegará el mensaje,

```
    Mi mensaje: No such file or directory
```

donde **No such file or directory** es el mensaje asociado al código de error número 2.

1.4 Ejercicios

- 1.1** Escribir un programa que presente en el fichero estándar de salida una lista con todos los códigos de error que manejan las llamadas al sistema UNIX. Por cada código debe aparecer su número y la cadena descriptiva que el sistema le asocia. El nombre del programa será `ej1_1.c` y al ejecutarlo debe producir una salida como la siguiente:

```
1: Operation not permitted
2: No such file or directory
3: No such process
4: Interrupted system call
5: I/O error
6: No such device or address
7: Arg list too long
8: Exec format error
9: Bad file number
10: No child processes
11: Try again
12: Out of memory
.
.
.
120: Is a named type file
121: Remote I/O error
```

- 1.2** Escribir una función de tratamiento genérico de errores para ser utilizada en los programas que se van a escribir en los capítulos posteriores. La interfaz de esta función debe ser:

```
void error (char nfichero, int nro_linea, char mensaje);
```

- **nfichero** es el nombre del fichero fuente desde donde se ha llamado a la función **error**. Utilizando la constante `__FILE__` que genera el preprocesador podemos hacer que este nombre se determine automáticamente.
- **nro_linea** es el número de línea del fichero fuente desde donde se ha llamado a la función **errno**. Utilizando la constante `__LINE__` generada por el preprocesador podemos hacer que este número se calcule automáticamente.
- **mensaje** es una cadena de caracteres que contiene un mensaje escrito por el usuario.

Como ejemplo, vamos a suponer que la línea 19 del fichero `prueba.c` es la siguiente:

```
error (__FILE__, __LINE__, "Error al abrir un fichero");
```

Si el último error producido es `ENOENT`, entonces la función escribirá el mensaje siguiente en el fichero estándar de salida de errores:

```
prueba.c (19). ERROR: ENOENT, No such file or directory. Error al
abrir un fichero
```

La cadena `ENOENT` es el identificador del código de error. Este identificador, y no el número de error, es el empleado en el manual de UNIX cuando se describen los códigos de error que devuelven las llamadas al sistema. Por ello, es más significativo imprimir este identificador que el número de error.

Los identificadores asociados a cada error están definidos en el fichero de cabecera `/usr/include/errno.h`. Estos identificadores se definen como constantes para el preprocesador por lo que no podemos utilizarlos directamente en el programa para imprimirlos. Para imprimirlos es necesario que sean tratados como cadenas de caracteres. Esto se puede conseguir declarando un array de la forma siguiente:

```
char errores [] = {"", "EPERM", "ENOENT", "ESRCH", "EINTR", "EIO",
                  "ENXIO", "E2BIG", ..., "EREMOTEIO", "EDQUOT"};
```

Hacemos notar que en esta declaración cada cadena ocupa la posición que le corresponde según el número de error que tiene asociado.

La cadena `No such file or directory` es la cadena descriptiva del último error producido. Esta cadena se debe determinar consultando las variables `errno` y `sys_errlist`.

Como queremos que esta función se pueda utilizar en otros programas, se aconseja crear:

- (a) El fichero de cabecera `$HOME/include/mic.h` que contenga la declaración de prototipo de `error` y de otras funciones que se creen más adelante.
- (b) La biblioteca⁵ `$HOME/lib/libmic.a` que contenga la función `error` y otras que se creen más adelante.

Para probar el correcto funcionamiento de la función podemos ejecutar el programa siguiente:

Programa 1.2: Ejemplo para probar la función `error` (ej1-2.c)

```
#include "mic.h"

main ()
{
    extern int sys_nerr;
    int i;
    char mensaje [50];

    for (i = 1; i <= sys_nerr; i++) {
        sprintf (mensaje, "Error nro. %d", i);
```

⁵Si el lector no está familiarizado con las herramientas de desarrollo del sistema UNIX es aconsejable que lea primero el *Apéndice B. Desarrollo de aplicaciones en el entorno UNIX*.


```

        errno = i;
        error (__FILE__, __LINE__, mensaje);
    }
}

```

La forma de compilar el programa será:

```
$ cc ej1_2.c -I$HOME/include -L$HOME/lib -lmic
```

La salida que produce el programa es:

```
$ a.out
```

```

ej1_2.c (19). ERROR: EPERM, Operation not permitted. Error nro. 1
ej1_2.c (19). ERROR: ENOENT, No such file or directory. Error nro. 2
ej1_2.c (19). ERROR: ESRCH, No such process. Error nro. 3
ej1_2.c (19). ERROR: EINTR, Interrupted system call. Error nro. 4
...

```

1.3 Añadir a la biblioteca libmic.a la función `error_fatal` que se declara a continuación:

```
void error_fatal (char mensaje, int código_salida);
```

Esta función debe imprimir en el fichero estándar de salida de errores la información siguiente:

- Identificador asociado al código del último error producido.
- Cadena que asocia el sistema al último error producido.
- `mensaje` del usuario.

Ejemplo, si `errno` vale 2, la llamada

```
error_fatal ("No se puede abrir datos.dat", -1);
```

produce la salida:

```

ERROR FATAL: ENOENT, No such file or directory. No se puede
abrir datos.dat

```

La función `error_fatal` termina llamando a la función `exit` y pasándole a ésta el código de salida `código_salida`. Por lo tanto, `error_fatal` se debe utilizar para terminar un programa y devolverle al sistema un código numérico elegido por el usuario. La interpretación de este código es responsabilidad del programador. Para probar esta función escribir el programa `ej1_3.c` donde se debe hacer una llamada a `error_fatal`. La forma de compilar el programa será:

```
$ cc ej1_3.c -I$HOME/include -L$HOME/lib -lmic
```

¿Dónde guarda el sistema operativo los códigos que le envía la función `error_fatal` a través de `exit`?

Parte I

El sistema de ficheros

2	Arquitectura del sistema de ficheros	13
3	Manejo de ficheros ordinarios	51
4	Manejo de directorios y ficheros especiales	117

Capítulo 2

Arquitectura del sistema de ficheros

«Aquella noche quemó y abrasó el ama cuantos libros había en el corral y en toda la casa, y tales debieron de arder que merecían guardarse en perpetuos archivos...»

(Cervantes, Quijote I-7)

2.1	Características del sistema de ficheros	13
2.2	Estructura del sistema de ficheros	14
2.3	Tipos de ficheros en UNIX	23
2.4	Extensiones del sistema BSD	29
2.5	Tablas de control de acceso a los ficheros	31
2.6	Administración de los sistemas de ficheros	35
2.7	Ejercicios	44

2.1 Características del sistema de ficheros

Un sistema de ficheros es un mecanismo de abstracción de los dispositivos físicos de almacenamiento que nos permite manejarlos a un nivel lógico sin la necesidad de conocer su arquitectura hardware particular. El sistema de ficheros de UNIX se caracteriza por:

- Poseer una estructura jerárquica.
- Realizar un tratamiento consistente de los datos de los ficheros.
- Poder crear y borrar ficheros.
- Permitir un crecimiento dinámico de los ficheros.
- Proteger los datos de los ficheros.

- Tratar los dispositivos y periféricos —terminales, unidades de cinta, discos, etc.— como si fuesen ficheros.

El sistema de ficheros está organizado, a nivel lógico, en forma de árbol invertido, con un nodo principal conocido como nodo raíz `/`. Cada nodo dentro del árbol es un directorio y puede contener a su vez otros nodos —subdirectorios—, ficheros normales o ficheros de dispositivos.

Los nombres de los ficheros se especifican mediante la ruta `—path name—`, que describe cómo localizar un fichero dentro de la jerarquía del sistema. La ruta de un fichero puede ser absoluta —referida al nodo raíz— o relativa —referido al directorio de trabajo actual - *CWD current work directory*—.

Los programas que se ejecutan en UNIX no conocen el formato interno con el que el núcleo almacena los datos. Cuando accedemos al contenido de un fichero mediante una llamada al sistema `—read—`, el sistema nos lo va a presentar como una secuencia de bytes sin formato. Nuestro programa es el encargado de interpretar la secuencia de bytes y darle significado según sus necesidades. Por lo tanto, la sintaxis `—forma—` del acceso a los datos de un fichero viene impuesta por el sistema y es la misma para todos los programas, y la semántica `—significado—` de los datos es responsabilidad del programa que trabaja con el fichero.

2.2 Estructura del sistema de ficheros

Los sistemas de ficheros suelen estar situados en dispositivos de almacenamiento modo bloque, tales como cintas o discos.

Las cintas tienen un tiempo de acceso mucho más alto que los discos, por ello es poco práctico instalar un sistema de ficheros sobre ellas. Sin embargo, son muy útiles para realizar copias de seguridad `—backup—` de un sistema ya instalado en disco.

Lo normal es que un sistema UNIX se arranque `—boot—` desde cinta o CDROM cuando queremos instalarlo por primera vez sobre un disco o cuando, tras producirse una quiebra `—crash—` del sistema, queremos restaurarlo a su situación anterior —proceso conocido como *recovery system*—. En el resto de nuestra discusión vamos a considerar que los sistemas de ficheros están instalados sobre discos.

Un sistema UNIX puede manejar uno o varios discos físicos, cada uno de los cuales puede contener uno o varios sistemas de ficheros. Los sistemas de ficheros son particiones lógicas del disco.

Hacer que un disco físico contenga varios sistemas de ficheros permite una administración más segura, ya que si uno de los sistemas de ficheros se daña, perdiéndose la información que hay en él, este accidente no se habrá transmitido al resto de los sistemas de ficheros que hay en el disco y podremos seguir trabajando con ellos para intentar una restauración o una reinstalación.

El núcleo del sistema trabaja con el sistema de ficheros a un nivel lógico y no trata directamente con los discos a nivel físico. Cada disco es considerado como un dispositivo lógico que tiene asociados unos números de dispositivo `—minor number` y `major number`—. Estos números se utilizan para indexar, dentro de una tabla de funciones, la que tenemos que emplear para acceder al manejador del disco. El manejador del disco se va a

encargar de transformar las direcciones lógicas —núcleo— de nuestro sistema de ficheros a direcciones físicas del disco.

Un sistema de ficheros se compone de una secuencia de bloques lógicos, cada uno de los cuales tiene un tamaño fijo. El tamaño de cada bloque es el mismo para todo el sistema de ficheros y suele ser múltiplo de 512 bytes.

A pesar de que el tamaño de un bloque es homogéneo en un sistema de ficheros, puede variar de un sistema a otro dentro de una misma configuración UNIX con varios sistemas de ficheros. El tamaño elegido para el bloque va a influir en las prestaciones globales del sistema. Por un lado, interesa que los bloques sean grandes para que la velocidad de transferencia entre el disco y memoria sea grande. Sin embargo, si los bloques lógicos son demasiado grandes, la capacidad de almacenamiento del disco se puede ver desaprovechada cuando abundan los ficheros pequeños que no llegan a ocupar un bloque completo. Valores típicos en bytes para el tamaño de un bloque son: 512, 1024 y 2048.

En la figura 2.1 podemos ver, en un primer nivel de análisis, la estructura que tiene un sistema de ficheros en el UNIX System V.

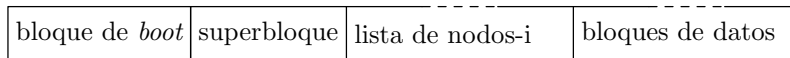


Figura 2.1: Primer nivel en la estructura de un sistema de ficheros.

En la figura podemos ver cuatro partes:

- El *bloque de arranque* —*boot*—. Ocupa la parte del principio del sistema de ficheros, típicamente el primer sector, y puede contener el código de arranque. Este código es un pequeño programa que se encarga de buscar el sistema operativo y cargarlo en memoria para inicializarlo.
- El *superbloque* describe el estado de un sistema de ficheros. Contiene información acerca de su tamaño, el número total de ficheros que puede contener, qué espacio queda libre, etc.
- La *lista de nodos índice* —nodos-i—. Se encuentra a continuación del superbloque. Esta lista tiene una entrada por cada fichero, donde se guarda una descripción del mismo: situación del fichero en el disco, propietario, permisos de acceso, fecha de actualización, etc. El administrador del sistema es quien especifica el tamaño de la lista de nodos índice al configurar el sistema.
- Los *bloques de datos* empiezan a continuación de la lista de nodos índice y ocupan el resto del sistema de ficheros. En esta zona es donde se encuentra situado el contenido de los ficheros a los que hace referencia la lista de nodos-i. Cada uno de los bloques destinados a datos sólo puede ser asignado a un fichero, tanto si lo ocupa totalmente como si no.

2.2.1 El superbloque

Como hemos visto anteriormente, en el superbloque está la descripción del estado del sistema de ficheros. En el fichero de cabecera `<sys/filsys.h>` hay declarada una estructura

en C que describe el significado del contenido del superbloque. El superbloque contiene, entre otras, la siguiente información:

- Tamaño del sistema de ficheros.
- Lista de bloques libres disponibles.
- Índice del siguiente bloque libre en la lista de bloques libres.
- Tamaño de la lista de nodos-i.
- Total de nodos-i libres.
- Lista de nodos-i libres.
- Índice del siguiente nodo-i libre en la lista de nodos-i libres.
- Campos de bloqueo de elementos de las listas de bloques libres y de nodos-i libres. Estos campos se emplean cuando se realiza una petición de bloque o de nodo-i libre.
- Indicador que informa si el superbloque ha sido modificado o no.

Cada vez que, desde un proceso, se accede a un fichero, es necesario consultar el superbloque y la lista de nodos-i. Como el acceso a disco suele degradar bastante el tiempo de ejecución de un programa, lo normal es que el núcleo realice la E/S con el disco a través de un *buffer caché* y que el sistema tenga en memoria una copia del superbloque y de la lista de nodos-i. Esto puede plantear problemas de consistencia de los datos, ya que una actualización en memoria del superbloque y de la tabla de nodos-i no implica una actualización inmediata en disco. La solución a este problema consiste en realizar periódicamente una actualización en disco de los datos de administración que mantenemos en memoria. De esta tarea se encarga un *demonio*¹ que se arranca al inicializar el sistema. Naturalmente, antes de apagar el sistema también hay que actualizar el superbloque y las tablas de nodos-i del disco. El programa `shutdown` se encarga de esta tarea y es fundamental tener presente que no se debe realizar una parada del sistema sin haber invocado previamente al programa `shutdown`, porque de lo contrario el sistema de ficheros podría quedar seriamente dañado.

2.2.2 Nodos índice (*inodes*)

Cada fichero en un sistema UNIX tiene asociado un nodo-i. El nodo-i contiene la información necesaria para que un proceso pueda acceder al fichero. Esta información incluye: propietario, derechos de acceso, tamaño, localización en el sistema de ficheros, etc.

La *lista de nodos-i* se encuentra situada en los bloques que hay a continuación del superbloque. Durante el proceso de arranque del sistema, el núcleo lee la lista de nodos-i del disco y carga una copia en memoria, conocida como *tabla de nodos-i*. Las manipulaciones que haga el subsistema de ficheros —parte del código del núcleo— sobre los ficheros

¹La nomenclatura que se emplea en el sistema UNIX muchas veces está cargada de sentido del humor. Por *demonio* se debe entender todo programa del sistema que se está ejecutando en segundo plano —*background*— y que realiza tareas periódicas relacionadas con la administración del sistema.

van a involucrar a la tabla de nodos-i pero no a la lista de nodos-i. Mediante este mecanismo se consigue una mayor velocidad de acceso a los ficheros, ya que la tabla de nodos-i está cargada siempre en memoria. En el párrafo anterior vimos que hay un demonio del sistema —**syncer**— que se encarga de actualizar periódicamente el contenido de la lista de nodos-i con la tabla de nodos-i.

En el fichero de cabecera `<sys/ino.h>` se declara la estructura C `struct dinode` que describe la información de un nodo-i. Los campos que componen un nodo-i son los siguientes:

- *Identificador del propietario del fichero.* La posesión se divide entre un propietario individual y un grupo de propietarios y define el conjunto de usuarios que tienen derecho de acceso al fichero. El superusuario tiene derecho de acceso a todos los ficheros del sistema.
- *Tipo de fichero.* Los ficheros pueden ser ordinarios de datos, directorios, especiales de dispositivo —en modo carácter o en modo bloque— y tuberías —o *FIFO*—.
- *Tipo de acceso al fichero.* El sistema protege los ficheros estableciendo tres niveles de permisos: permisos del propietario, del grupo de usuarios al que pertenece el propietario y del resto de los usuarios —conocido también como el mundo—. Cada clase de usuarios puede tener habilitados o deshabilitados los derechos de lectura, escritura y ejecución. Para los directorios, el derecho de ejecución significa poder acceder o no a los ficheros que contiene.
- *Tiempos de acceso al fichero.* Dan información sobre la fecha de la última modificación del fichero, la última vez que se accedió a él y la última vez que se modificaron los datos de su nodo-i.
- *Número de enlaces del fichero.* Representa el total de los nombres que el fichero tiene en la jerarquía de directorios. Como veremos más adelante, un fichero puede tener asociados diferentes nombres que correspondan a diferentes rutas y a través de los cuales accedamos a un mismo nodo-i y por consiguiente a los mismos bloques de datos.
- *Entradas para los bloques de dirección* de los datos de un fichero. Si bien los usuarios ven los datos de un fichero como si fuesen una secuencia de bytes contiguos, el núcleo puede almacenarlos en bloques que no tienen por qué ser contiguos. En los bloques de dirección es donde se especifican los bloques de disco que contienen los datos del fichero.
- *Tamaño del fichero.* Los bytes de un fichero se pueden direccionar indicando un desplazamiento a partir de la dirección de inicio del fichero —desplazamiento 0—. El tamaño del fichero es igual al desplazamiento del byte más alto, incrementado en una unidad. Por ejemplo, si un usuario crea un fichero y escribe en él un solo byte en la posición 2.000, el tamaño del fichero es 2.001 bytes.

Hay que hacer notar que el nombre del fichero no queda especificado en su nodo-i. Como veremos más adelante, es en los ficheros de tipo directorio donde a cada nombre de fichero se le asocia su nodo-i correspondiente.

También hay que reseñar la diferencia entre escribir el contenido de un nodo-i en disco y escribir el contenido del fichero. El contenido del fichero —sus datos— cambia sólo cuando se escribe en él. El contenido de un nodo-i cambia cuando se modifican los datos del fichero o la situación administrativa del mismo —propietario, permisos, enlaces, etc.—

La tabla de nodos-i contiene la misma información que la lista de nodos-i, junto con la siguiente información adicional:

- El estado del nodo-i, que indica:
 - si el nodo-i está bloqueado;
 - si hay algún proceso esperando a que el nodo-i quede desbloqueado;
 - si la copia del nodo-i que hay en memoria difiere de la que hay en el disco;
 - si la copia de los datos del fichero que hay en memoria difiere de los datos que hay en el disco —caso de la escritura en el fichero a través del *buffer caché*—.
- El número de dispositivo lógico del sistema de ficheros que contiene al fichero.
- El número de nodo-i. Como los nodos-i se almacenan en el disco en un array lineal, al cargarlo en memoria, el núcleo le asigna un número en función de su posición en el array. El nodo-i del disco no necesita esta información.
- Punteros a otros nodos-i cargados en memoria. El núcleo enlaza los nodos-i sobre una cola dispersa —cola *hash*— y sobre una lista libre. Las claves de acceso a la cola dispersa nos las dan el número de dispositivo lógico del nodo-i y el número de nodo-i.
- Un contador que indica el número de copias del nodo-i que están activas —por ejemplo, porque el fichero está abierto por varios procesos—.

2.2.3 Los bloques de datos. Estructura de un fichero ordinario

Los bloques de datos están situados a partir de la lista de nodos-i. Como se mencionó anteriormente, cada nodo-i tiene unas entradas —bloques de direcciones— para localizar dónde están los datos de un fichero en el disco. Como cada bloque del disco tiene asociada una dirección, las entradas de direcciones consisten en un conjunto de direcciones de bloques del disco.

Si los datos de un fichero se almacenasen en bloques consecutivos, para poder acceder a ellos nos bastaría con conocer la dirección del bloque inicial y el tamaño del fichero. Sin embargo, esta política de gestión del disco produce a la larga un desaprovechamiento del mismo, ya que tenderán a proliferar áreas libres demasiado pequeñas para poder ser usadas. Por ejemplo, supongamos que un usuario crea tres ficheros, A, B y C, cada uno de los cuales ocupa 10 bloques en el disco. Supongamos que el sistema sitúa estos tres ficheros en bloques contiguos —figura 2.2 (a)—. Si el usuario necesita añadir 5 bloques al fichero central —fichero B— porque ha aumentado de tamaño, el sistema tendrá que buscar 15 bloques contiguos que se encuentren libres para copiar el fichero B en esa zona —figura 2.2 (b)—. Esto plantea dos inconvenientes, el primero es el tiempo que se pierde en copiar los 10 bloques de B que no varían de contenido, el segundo es que los 10

bloques que quedan libres sólo pueden contener ficheros con un tamaño inferior o igual a 10 bloques. Esto provoca a la larga una microfragmentación del disco que lo va a dejar inservible.

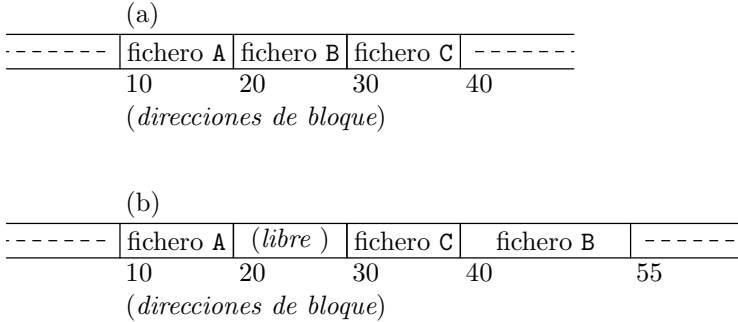


Figura 2.2: Reserva de espacio para ficheros y fragmentación del espacio libre.

El núcleo puede minimizar la fragmentación del disco ejecutando periódicamente procesos para compactarlo, pero esto produce una degradación de las prestaciones del sistema en cuanto a velocidad.

Para una mayor flexibilidad, el núcleo reserva los bloques para un fichero de uno en uno, y permite que los datos de un fichero estén esparcidos por todo el sistema de ficheros. Este esquema de reserva complica la tarea de localizar los datos.

Las entradas de direcciones del nodo-*i* consisten en una lista de direcciones de los bloques que contiene los datos del fichero. Un cálculo simple nos muestra que almacenar la lista de bloques del fichero en un nodo-*i* es algo difícil de implementar. Por ejemplo, si los bloques son de 1 Kbyte y el fichero ocupa 10 Kbytes —10 bloques—, vamos a necesitar almacenar 10 direcciones en el nodo-*i*. Pero si el fichero es de 100 Kbytes —100 bloques—, necesitaremos 100 direcciones para acceder a todos los datos. Así pues, esta gestión nos impone que el tamaño del nodo-*i* sea variable, ya que, si lo fijamos en un valor concreto, estamos fijando el tamaño máximo de los ficheros que podemos manejar. Debido a lo poco manejable que resulta, la idea de nodos-*i* de tamaño variable es algo que no implementa ningún sistema.

Para conseguir que el tamaño de un nodo-*i* sea pequeño y a la vez podamos manejar ficheros grandes, las entradas de direcciones de un nodo-*i* se ajustan al esquema que muestra la figura 2.3. En el UNIX System V, los nodos-*i* tienen una tabla con 13 entradas. Las 10 entradas marcadas como directas en la figura contienen direcciones de bloques en los que hay datos del fichero. La entrada marcada como indirecta simple direcciona un bloque de datos que contiene una tabla de direcciones de bloques de datos. Para acceder a los datos a través de una entrada indirecta, el núcleo debe leer el bloque cuya dirección nos indica la entrada indirecta y buscar en él la dirección del bloque donde realmente está el dato, para a continuación leer ese bloque y acceder al dato. La entrada marcada como indirecta doble contiene la dirección de un bloque cuyas entradas actúan como entradas indirectas simples, y la entrada indirecta triple direcciona un bloque cuyas entradas son indirectas dobles.

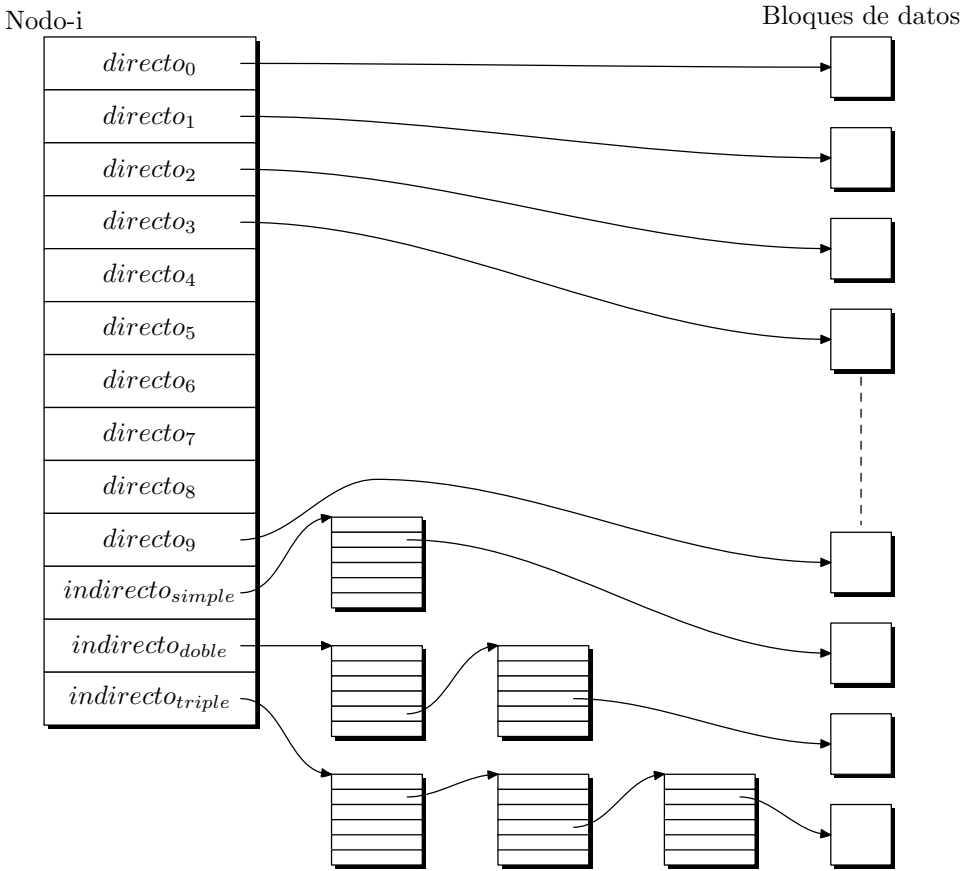


Figura 2.3: Tabla de direcciones de bloque de un nodo-i.

Este método puede extenderse para soportar entradas indirectas cuádruples, quintuples, etc., pero en la práctica tenemos más que suficiente con una indirección triple.

Vamos a ver el caso práctico en el que los bloques son de 1 Kbyte y el bloque se direcciona con 32 bits — $2^{32} = 4$ Gdirecciones posibles—. En esta situación, un bloque de datos puede almacenar 256 direcciones de bloques y un fichero podría llegar a tener un tamaño del orden de 16 Gbytes —consulte la tabla 2.1—, pero teniendo en cuenta que el campo *tamaño del fichero* del nodo-i es de 32 bits, el tamaño máximo de un fichero no es del orden de los 16 Gbytes, como indica la tabla, sino de 4 Gbytes. Este tamaño, en la práctica, resulta más que suficiente.

Los procesos acceden a los datos de un fichero indicando la posición, con respecto al inicio del fichero, del byte que queremos leer o escribir. El fichero es visto como una secuencia de bytes que empieza en el byte número 0 y llega hasta el byte cuya posición, con respecto al inicial, coincide con el tamaño del fichero menos uno. El núcleo se encarga de transformar las posiciones de los bytes, tal y como las ve el usuario, a direcciones de

Tabla 2.1: Capacidad de direccionamiento de los bloques de direcciones de un nodo-i. Suponemos bloques de 1 Kbyte.

Tipo de entrada	Total de bloques accesibles	Total de bytes accesibles
10 entradas directas	10 bloques de datos	10 Kbytes
1 entrada indirecta simple	1 bloque indirecto simple → 256 bloques indirectos	256 Kbytes
1 entrada indirecta doble	1 bloque indirecto doble → 256 bloques indirectos simples → 65.536 bloques de datos	64 Mbytes
1 entrada indirecta triple	1 bloque indirecto triple → 256 bloques indirectos dobles → 65.536 bloques indirectos simples → 16 ₁ 777.216 bloques de datos	16 Gbytes

los bloques de disco. Veamos un ejemplo para clarificar estos conceptos. Supongamos un fichero cuyos datos están en los bloques que nos indican las entradas de direcciones del nodo-i descrito en la figura 2.4. Vamos a seguir suponiendo que el bloque tiene un tamaño de 1.024 bytes. Si un proceso quiere acceder a un byte que se encuentra en la posición 9.125 del fichero, el núcleo calcula que ese byte está en el bloque número 8 del fichero —empezando a numerar los bloques lógicos del fichero desde 0—.

En efecto,

$$\begin{aligned}\text{bloque}_{\text{fichero}} &= \text{desplazamiento}_{\text{fichero}} / \text{tamaño}_{\text{bloque}} = \\ &= 9.125 \text{ bytes} / 1.024 \text{ (bytes/bloque)} = 8 \text{ bloques}\end{aligned}$$

La división la consideramos como división entera. Para ver a qué bloque del disco corresponde el bloque número 8 del fichero, hay que consultar el número de bloque que almacena la entrada número 8 —entrada de dirección directa— de la tabla de direcciones del nodo-i. En el caso de la figura, el bloque de disco buscado es el 412. Dentro de este bloque, el byte 9.125 del fichero se corresponde con el byte 933 con respecto al inicio del bloque —los bytes del bloque se numeran desde 0 a 1.023—.

En efecto,

$$\begin{aligned}\text{desplazamiento}_{\text{bloque}} &= \text{desplazamiento}_{\text{fichero}} \% \text{tamaño}_{\text{bloque}} \\ \text{desplazamiento}_{\text{bloque}} &= 9.125 \text{ bytes} \% 1.024 \text{ (bytes/bloque)} = 933 \text{ bytes}\end{aligned}$$

El operador % indica resto de la división entera. En este ejemplo el cálculo ha sido sencillo porque el byte buscado era accesible desde una entrada directa del nodo-i. Veamos qué ocurre si queremos localizar en el disco el byte que se encuentra en la posición 425.000 del fichero. Si calculamos su bloque lógico de fichero, veremos que se encuentra en el bloque 415,

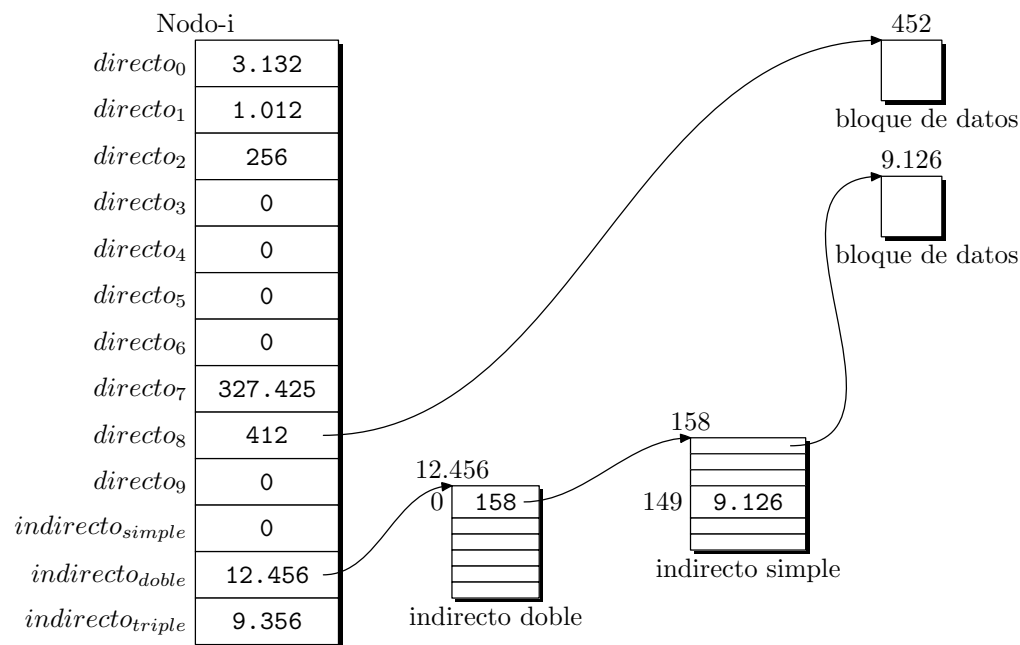


Figura 2.4: Ejemplo de tabla de direcciones de un nodo-i.

$$\begin{aligned} \text{bloque}_{\text{fichero}} &= \text{desplazamiento}_{\text{fichero}} / \text{tamaño}_{\text{bloque}} = \\ &= 425.000 \text{ bytes} / 1.024 \text{ (bytes/bloque)} = 415 \text{ bloques} \end{aligned}$$

El total de bloques al que podemos acceder con las entradas directas es de 10. Con la entrada indirecta simple podemos acceder a 256 bloques y, con la indirecta doble, a 65.536. Luego el byte buscado estará direccionado por la entrada indirecta doble.

A esta entrada pertenecen los bloques comprendidos entre los números lógicos de fichero 266 y 65.581 —ambos inclusive— y el bloque buscado es el 415. Si nos fijamos en la figura, el número de bloque que contiene la entrada indirecta doble es el 12.456, que es el bloque de disco donde están las direcciones de los bloques con entradas indirectas simples.

La entrada número 0 del bloque indirecto doble nos da acceso a los bloques de fichero comprendidos entre el 266 y el 521, ya que cada entrada actúa de indirección simple y da acceso a 256 bloques de datos. En la entrada 0 vemos que el número de bloque de disco del bloque indirecto simple que buscamos es 158. Dentro del bloque indirecto simple, la entrada que nos interesa es la diferencia entre 415 —bloque lógico del fichero— y 266 —bloque inicial al que da acceso el bloque indirecto doble—; luego es 149. Según la figura, la entrada 149 del bloque de disco 158 contiene el número 9.126. Es en el bloque de disco 9.126 donde se encuentra el dato que buscamos, y en el byte 40 de este bloque está el byte 425.000 de nuestro fichero,

$$\text{desplazamiento}_{\text{bloque}} = \text{desplazamiento}_{\text{fichero}} \% \text{tamaño}_{\text{bloque}}$$

$\text{desplazamiento}_{\text{bloque}} = 425.000 \text{ bytes} \% 1.024 \text{ (bytes/bloque)} = 40 \text{ bytes.}$

Si observamos la figura 2.4 con más detenimiento, vemos que hay algunas entradas del nodo-*i* que están a 0. Esto significa que no referencian a ningún bloque del disco y que los bloques lógicos correspondientes del fichero no tienen datos. Esta situación se da cuando se crea un fichero y nadie escribe en los bytes correspondientes a estos bloques, por lo que permanecen en su valor inicial 0. Al no reservar el sistema bloques de disco para estos bloques lógicos, se consigue un ahorro de los recursos del disco. Imaginemos que creamos un fichero y sólo escribimos un byte en la posición 1₁048.276, esto significa que el fichero tiene un tamaño de 1 Mbyte. Si el sistema reservase bloques de disco para este fichero en función de su tamaño y no en función de los bloques lógicos que realmente tiene ocupados, nuestro fichero ocuparía 1.024 bloques de disco en lugar de 1, como en realidad ocupa.

La conversión de un desplazamiento de byte de fichero a su dirección de disco es un proceso laborioso, sobre todo si se ve involucrada una indirección triple. En este caso, el núcleo necesita acceder a tres bloques de direcciones, además de al nodo-*i* y al bloque de datos. Aun en el caso de que el núcleo encuentre los bloques en el *buffer caché*, la operación sigue siendo costosa porque debe hacer varios accesos al *buffer* y puede que tenga que esperar a que algunos *buffers* queden desbloqueados. La efectividad de este algoritmo queda limitada en la práctica por la frecuencia de uso de los ficheros grandes.

2.3 Tipos de ficheros en UNIX

Como vimos en el epígrafe anterior, en cada nodo-*i* hay un campo que nos indica el tipo de fichero al que se refiere ese nodo-*i*. En el sistema UNIX hay cuatro tipos de ficheros: ordinarios —también llamados regulares o de datos—, directorios, ficheros de dispositivos —conocidos también como ficheros especiales— y tuberías —en inglés *pipes* o *FIFOS*—. Algunos autores [Rochkind, 1985] incluyen los ficheros de dispositivo y las tuberías en el grupo de los ficheros especiales.

2.3.1 Ficheros ordinarios

Los ficheros ordinarios contienen bytes de datos organizados como un array lineal. Esto no es nuevo para nosotros. Las operaciones que se pueden hacer con los datos de un fichero son:

- Leer o escribir cualquier byte del fichero.
- Añadir bytes al final del fichero, con lo que aumenta su tamaño.
- Truncar el tamaño de un fichero a cero bytes. Esto es como si borrásemos el contenido del fichero.

Se debe tener presente que las siguientes operaciones no están permitidas con los ficheros:

- Insertar bytes en un fichero, excepto al final. No hay que confundir la inserción de bytes con la modificación de los que ya existen —proceso de escritura—.

- Borrar bytes de un fichero. No hay que confundir el borrado de bytes con la puesta a cero de los que ya existen.
- Truncar el tamaño de un fichero a un valor distinto de cero.

Dos o más procesos pueden leer y escribir concurrentemente sobre un mismo fichero. Los resultados de esta operación dependerán del orden de las llamadas de entrada/salida individuales de cada proceso y de la gestión que el planificador haga de los procesos, y en general son impredecibles.

Hasta hace poco, UNIX no poseía mecanismos eficientes para controlar el acceso concurrente a los ficheros. Recientemente, algunas versiones de UNIX proporcionan facilidades de bloqueo de ficheros y de gestión de semáforos, para controlar el acceso a los ficheros.

Los ficheros ordinarios, como tales, no tienen nombre y el acceso a ellos se realiza a través de los nodos-i.

2.3.2 Directorios

Los directorios son los ficheros que nos permiten darle una estructura jerárquica a los sistemas de ficheros de UNIX. Su función fundamental consiste en establecer la relación que existe entre el nombre de un fichero y su nodo-i correspondiente.

Estructura de un directorio en el UNIX System V

En esta versión de UNIX, un directorio es un fichero cuyos datos están organizados como una secuencia de entradas, cada una de las cuales contiene un número de nodo-i y el nombre de un fichero que pertenece al directorio. Al par $[nodo-i] \leftrightarrow [nombre\ de\ fichero]$ se le conoce como enlace y puede haber varios nombres de ficheros —distribuidos por la jerarquía de directorios— que estén enlazados con un mismo nodo-i.

El tamaño de cada entrada del directorio es de 16 bytes; dos dedicados al nodo-i y 14 dedicados al nombre del fichero. En la figura 2.5 podemos ver la estructura típica de un directorio.

<i>desplazamiento</i>	<i>nodo-i</i>	<i>nombre del fichero</i>
0	45	.
16	2	..
32	2.395	inittab
48	371	mount
64	4.921	shutdown
⋮		

Figura 2.5: Ejemplo de estructura del directorio /etc para el UNIX System V.

Las dos primeras entradas de un directorio reciben los nombres . y ... El fichero . tiene asociado el nodo-i correspondiente al directorio actual y al fichero .. se le asocia el nodo-i del directorio padre del actual. Estas dos entradas están presentes en todo directorio, y en el caso del directorio raíz /, el programa `mkfs` —*make file system*,

programa mediante el cual se crea un sistema de ficheros— se encarga de que el fichero .. se refiera al propio directorio raíz.

El núcleo maneja los datos de un directorio con los mismos procedimientos empleados para acceder a los datos de ficheros ordinarios, usando la estructura nodo-i y los bloques de acceso directos e indirectos.

Los procesos pueden leer el contenido de un directorio como si se tratase de un fichero de datos, sin embargo no pueden modificarlos. El derecho de escritura en un directorio está reservado al núcleo. Esto es una medida de seguridad tomada para evitar que una manipulación incorrecta de un directorio pueda destruir un sistema de ficheros.

No hay razones estructurales para impedir la creación de múltiples enlaces a un directorio, sin embargo, esta posibilidad complicaría bastante la escritura de los programas que recorren el sistema de ficheros, por lo que el núcleo lo prohíbe.

Los permisos de acceso a un directorio tienen los siguientes significados:

- *Lectura*, permite que un proceso lea ese directorio.
- *Escritura*, permite a un proceso crear una nueva entrada en el directorio o borrar alguna ya existente. Esto deberá hacerlo a través de las llamadas: `creat`, `mknod`, `link` o `unlink`.
- *Ejecución*, autoriza a un proceso para buscar el nombre de un fichero dentro del directorio.

Estructura de un directorio en el sistema BSD

El concepto de directorio desempeña la misma función en el UNIX de Berkeley que en el de AT&T. La función principal es la de establecer los enlaces entre los nombres de los ficheros y los nodos-i. La diferencia entre los directorios de las versiones System V y BSD es que en ésta los nombres de los ficheros pueden ser más largos, hasta 255 caracteres, y no se reserva un espacio fijo de bytes para cada entrada del directorio.

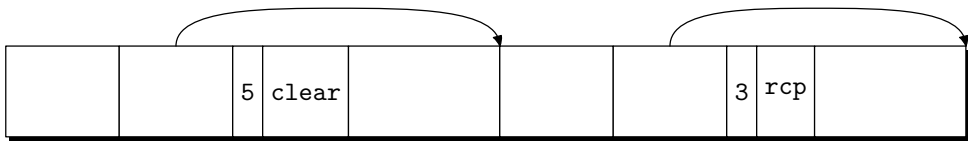
Los directorios se encuentran situados en unidades conocidas como *bloques de directorio* —*chunks* [Leffler *et al.*, 1989]— tal y como se muestra en la figura 2.6.

El tamaño de un bloque de directorio se elige de tal forma que pueda ser transferido en una sola operación con el disco. Esto permite actualizar los directorios en accesos atómicos —ningún proceso puede interrumpir para actualizar un directorio mientras haya otro que lo esté haciendo—.

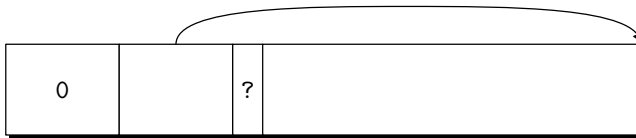
Cada bloque de directorio se compone de entradas de directorio de tamaño variable. No se permite que una entrada esté distribuida en más de un bloque. Los tres primeros campos de una entrada de directorio son de tamaño fijo y contienen:

1. El tamaño de la entrada.
2. La longitud del nombre del fichero al que se refiere la entrada.
3. El número del nodo-i asociado al fichero.

El resto de la entrada es un campo de longitud variable que contiene una cadena de caracteres terminada con el carácter nulo. Esta cadena es el nombre del fichero y su longitud máxima permitida es de 255 caracteres.



(a) Bloque de directorio con 2 entradas



(b) Bloque de directorio vacío

Figura 2.6: Formato de las entradas de un directorio en el sistema BSD.

El espacio libre en un directorio se registra bajo en una o varias entradas que lo acumulan en su campo *tamaño de la entrada*. Estas entradas se reconocen porque su tamaño es mayor que el necesario para almacenar sus campos de tamaño fijo más el campo *nombre del fichero*.

Cuando se borra una entrada de un directorio, el sistema añade el espacio que queda libre a la entrada anterior, si ésta pertenece al mismo bloque de directorio que la borrada, aumentado así el tamaño de la entrada. Si la primera entrada de un bloque de directorio está libre, el número de nodo-*i* que almacena esa entrada es cero, para indicar que no está reservada por ningún fichero.

Acceso al contenido de un directorio

Como ya hemos indicado, los procesos pueden leer el contenido de un directorio, pero no pueden modificarlo. Este privilegio está reservado exclusivamente al núcleo del sistema.

Para leer un directorio podemos utilizar las mismas llamadas que empleamos para los ficheros ordinarios —`open`, `read`, `lseek`, `close`, etc.— El sistema BSD ofrece una interfaz más cómoda para movernos por el interior de la jerarquía de directorios. Esta interfaz también ha sido adoptada por el UNIX System V, y sus funciones son: `opendir`, `readdir`, `rewindir`, `closedir`, `seekdir` y `tellidir`. Estas funciones no son parte del conjunto de llamadas al sistema, sino de la biblioteca estándar de E/S —consulte `directory(3C)` y el § 4.1.5 pág. 122—.

Las funciones anteriores pueden codificarse a partir de las llamadas de manejo de ficheros [Kernighan & Ritchie, 1988], lo cual ofrece la ventaja de poder ser emuladas sobre una red o incluso por un sistema no UNIX.

Conversión de ruta de acceso a nodo-*i*

Desde el punto de vista del usuario, los ficheros se sitúan en la jerarquía de directorios y se nombran mediante su ruta de acceso. Llamadas como `open`, `chdir` o `link` reciben como parámetro de entrada la ruta de acceso de un fichero y no su nodo-*i*. El núcleo es

quien se encarga de traducir la ruta de acceso de un fichero a su nodo-i correspondiente. El algoritmo que realiza la transformación —llamado **namei**— se encarga de analizar los componentes de la ruta de acceso y de leer los nodos-i intermedios necesarios para verificar que se trata de una ruta correcta y que el fichero realmente existe.

Si la ruta es absoluta, la búsqueda del nodo-i del fichero se iniciará en el directorio raíz. Si la ruta es relativa, la búsqueda se iniciará en el directorio de trabajo actual que tiene asociado el proceso que quiere acceder al fichero.

A medida que se van recorriendo los nodos-i intermedios, se verifican los permisos para comprobar si el proceso tiene derechos de acceso a los directorios intermedios.

Como ejemplo, vamos a suponer que un proceso quiere abrir el fichero **/etc/passwd** que tiene una entrada por cada usuario del sistema y donde, entre otra información, figura la contraseña del usuario cifrada. Cuando el núcleo inicia el análisis de la ruta, encuentra que empieza por **/** y pasa a leer el nodo-i asociado al directorio raíz. Este nodo-i pasa a ser el nodo-i de trabajo y el núcleo verifica si corresponde a un directorio y si el proceso tiene permiso para buscar ficheros dentro de él. Suponiendo que los permisos están en regla, el núcleo toma el siguiente elemento de la ruta, que es **etc**, y busca, dentro del directorio raíz, alguna entrada cuyo nombre de fichero sea **etc**. Cuando el núcleo la encuentra, libera el nodo-i del directorio raíz y lee el nodo-i del fichero **etc**. En este nodo-i se refleja que **etc** es otro directorio y por lo tanto en él podemos buscar el fichero **passwd**, tercer elemento de la ruta. El núcleo, tras verificar los permisos de acceso a **etc**, repite el mismo proceso de búsqueda para el nombre de fichero **passwd** y tras encontrarlo y verificar que es un fichero y no un directorio, como se esperaba a la vista de su ruta, y que tenemos permiso para acceder a él, nos habilita una estructura para poder trabajar con ese fichero. En el § 3.2.2 pág. 55 veremos mediante qué mecanismos podemos acceder a este fichero ya abierto.

2.3.3 Ficheros especiales

Los ficheros especiales, o ficheros de dispositivo, se utilizan para que los procesos se comuniquen con los dispositivos periféricos: discos, cintas, impresoras, terminales, redes, etc.

Hay dos familias de ficheros de dispositivo: ficheros modo bloque y ficheros modo carácter.

Los ficheros de dispositivo modo bloque se ajustan a un modelo concreto: el dispositivo contiene un array de bloques de tamaño fijo —generalmente múltiplo de 512 bytes— y el núcleo gestiona una memoria intermedia —*buffer caché*, también traducido como *antememoria*— que acelera la velocidad de transferencia de los datos. La transferencia de información entre el dispositivo y el núcleo se efectúa con mediación de la antememoria y el bloque es la unidad mínima que se transfiere en cada operación de entrada/salida. Esta memoria intermedia se implementa vía software y no hay que confundirla con las memorias *caché* de acceso rápido de que disponen los microprocesadores actuales. Ejemplos típicos de dispositivos modo bloque son los discos y las unidades de cinta.

En los ficheros de dispositivo modo carácter la información no se organiza según una estructura concreta y es vista por el núcleo, o por el usuario, como una secuencia lineal de bytes. En la transferencia de datos entre el núcleo y el dispositivo no participa la memoria intermedia y por lo tanto se realiza a menor velocidad. Ejemplos típicos de dispositivos

modo carácter son los terminales serie y las líneas de impresora. Un mismo dispositivo físico puede soportar los dos modos de acceso: bloque y carácter, y de hecho esto suele ser habitual en el caso de los discos.

Por otro lado, en el caso de los discos, si queremos realizar un acceso más organizado, debemos recurrir a las facilidades que brinda el sistema de ficheros y la estructura de directorios que hay creada sobre él. Esto no nos impide un acceso de más bajo nivel a través de los ficheros de dispositivo asociados al disco. Este tipo de acceso pasa por encima de la estructura del sistema de ficheros y trata directamente con el manejador del disco. Para Leffler [Leffler *et al.*, 1989], el sistema de ficheros es un apartado más dentro del subsistema de entrada/salida.

Los módulos del núcleo que gestionan la comunicación con los dispositivos se conocen como manejadores de dispositivo. Lo normal es que cada dispositivo tenga su manejador propio, aunque puede haber manejadores que controlen a toda una familia de dispositivos con características comunes, por ejemplo, el manejador que controla los terminales.

El sistema también puede soportar dispositivos software —o pseudodispositivos— que no tienen asociados un dispositivo físico. Por ejemplo, si una parte de la memoria del sistema se gestiona como un dispositivo, los procesos que quieran acceder a esa zona de memoria tendrán que usar las mismas llamadas al sistema que hay para el manejo de ficheros, pero sobre el fichero de dispositivo `/dev/mem` —fichero de dispositivo genérico para acceder a memoria—. En esta situación, la memoria es tratada como un periférico más.

Como ya hemos visto en epígrafes anteriores, los ficheros de dispositivo, al igual que el resto de los ficheros, tienen asociado un nodo-i. En el caso de los ficheros ordinarios o de los directorios, el nodo-i nos indica los bloques donde se encuentran los datos del fichero, pero en el caso de los ficheros de dispositivo no hay datos a los que referenciar. En su lugar, el nodo-i contiene dos números conocidos como *major number* y *minor number*. El *major number* indica el tipo de dispositivo de que se trata —disco, cinta, terminal, etc.— y el *minor number* indica el número de unidad dentro del dispositivo. En realidad, estos números los utiliza el núcleo para buscar dentro de unas tablas —*block device switch table* y *character device switch table*— una colección de rutinas que permiten manejar el dispositivo. Esta colección de rutinas constituyen realmente el manejador del dispositivo.

Cuando se invoca a una llamada al sistema para realizar una operación de entrada/salida sobre un fichero especial, el núcleo se encarga de llamar al manejador de dispositivo adecuado. Lo que ocurre a continuación es algo que compete al diseñador del manejador y es completamente transparente para el usuario.

Naturalmente, nosotros también podemos convertirnos en diseñadores de manejadores y añadir a una arquitectura hardware determinada los dispositivos que necesitemos.

2.3.4 Tuberías con nombre

Una tubería con nombre es un fichero con una estructura similar a la de un fichero ordinario. La diferencia principal con éstos es que los datos de una tubería son transitorios. Esto quiere decir que los datos desaparecen de la tubería a medida que son leídos.

Las tuberías se utilizan para comunicar procesos. Lo normal es que un proceso abra la tubería para escribir en ella y el otro para leer de ella. Los datos escritos en la tubería se leen en el mismo orden en el que fueron escritos, siguiendo la disciplina de una hilera:

el primer dato en entrar es el primero en salir². La sincronización del acceso a la tubería es algo de lo que se encarga el núcleo.

El almacenamiento de los datos en una tubería se realiza de la misma forma que en un fichero ordinario, excepto que el núcleo sólo utiliza las entradas directas de la tabla de direcciones de bloque del nodo-*i* de la tubería. Por lo tanto, una tubería con nombre podrá almacenar 10 Kbytes a lo sumo —suponiendo bloques de 1.024 bytes—. Esto no supone un problema grave, ya que los datos de la tubería son transitorios; sin embargo, si la velocidad a la que se introducen datos es mayor que la velocidad de extracción, la tubería podría rebosar y perderse información.

Los accesos de lectura/escritura en las tuberías se realizan con las mismas llamadas que empleamos para los ficheros ordinarios —`open`, `close`, `read`, `write`, etc.— Además, estos accesos son de tipo atómico³, con lo que se garantiza el sincronismo en el caso de que varios procesos compitan concurrentemente por el uso de la misma tubería.

En contraposición a las tuberías con nombre tenemos las tuberías sin nombre que no tienen asociado ningún nombre de fichero y que sólo existen mientras algún proceso está unido a ellas.

Las tuberías sin nombre también actúan como un canal de comunicación entre dos procesos y tienen asociado un nodo-*i* mientras existen. Un ejemplo típico de tubería sin nombre es la que se crea desde el intérprete de órdenes a través del carácter de tubería `|`. Por ejemplo,

```
$ ls | sort -r
```

La línea de órdenes anterior le indica al intérprete que debe arrancar dos procesos: uno para ejecutar `ls` —listar por pantalla el contenido del directorio actual— y otro para `sort -r` —filtro que lee líneas de texto de la entrada estándar y las presenta por pantalla en orden alfabético inverso—. Además, el intérprete debe crear una tubería sin nombre a la que se unirán los procesos creados para ejecutar `ls` y `sort`. `ls` dirigirá su salida hacia la tubería y `sort` leerá datos de la tubería.

Las tuberías sin nombre son creadas desde un proceso con la llamada `pipe`, mientras que las tuberías con nombre se crean con la orden `mknod` desde la línea de órdenes del sistema o con la llamada `mknod` desde un proceso.

2.4 Extensiones del sistema BSD

La arquitectura del sistema de ficheros y los tipos de ficheros tal y como los hemos descrito hasta ahora se ajustan al modelo del UNIX System V de AT&T y a las primeras versiones del UNIX de Berkeley.

En la organización de los discos de la versión 4.4 de BSD, al igual que en versiones anteriores, cada disco puede contener uno o más sistemas de ficheros. Cada sistema está definido mediante su superbloque, localizado al principio de la partición de disco

²Esta disciplina de manejo de datos se conoce también como *FIFO* —*first in first out*.

³Por acceso *atómico* entendemos acceso *indivisible*. Es decir, cuando un proceso está accediendo a la tubería para escribir, no se ve interrumpido por otro proceso que quiera realizar una operación de escritura sobre la misma tubería. Lo mismo ocurre con la operación de lectura. Naturalmente, cuando un proceso intenta leer datos de una tubería vacía, la operación puede quedar durmiendo mientras espera a que algún proceso escriba datos en la tubería.

dedicada al sistema. Dada la importancia de los datos contenidos en el superbloque, éste está duplicado para evitar pérdidas de información en situaciones límite. Generalmente, la copia del superbloque no es referenciada, a menos que se estropee el original.

Para asegurar que los ficheros de tamaño 2^{32} bytes se pueden manipular sin necesidad de recurrir a la entrada indirecta triple del nodo-i, el tamaño mínimo del bloque pasa a ser de 4.096 bytes. Como sabemos, el tamaño del bloque se especifica en el superbloque y es posible manejar simultáneamente sistemas de ficheros con diferentes tamaños de bloque. No obstante, el tamaño del bloque no puede cambiarse una vez que el sistema ha sido creado, a menos que se decida reinstalarlo de nuevo.

En el sistema BSD se introduce una mejora en la organización del disco basada en el concepto de grupo de cilindros.

2.4.1 Grupo de cilindros

En la organización de los sistemas de ficheros del BSD, cada disco físico se divide en una o más áreas, conocidas como grupo de cilindros. La figura 2.7 muestra un disco dividido en 4 grupos de cilindros, cada uno de los cuales puede agrupar a uno o más cilindros físicos del disco. Cada grupo de cilindros contiene información de control, que incluye una copia del superbloque, espacio para los nodos-i, un mapa de bits que refleja la ocupación de bloques dentro del grupo e información estadística que describe el uso de los bloques dentro del grupo de cilindros. El número de nodos-i que tiene ese grupo debe ser asignado cuando éste es creado.

El motivo de dividir el disco en grupos de cilindros es crear grupos de nodos-i que estén distribuidos por todo el disco, en lugar de colocarlos todos al principio del sistema, como ocurre en otras versiones de UNIX. Así, los nodos-i estarán en bloques situados físicamente cerca de los bloques de fichero a los que hacen referencia. Con esto ganamos en velocidad de acceso a los datos del fichero, ya que los desplazamientos de las cabezas lectoras para pasar de los bloques donde están situados los nodos-i a los bloques de datos serán más cortos. Además, la seguridad del sistema ante pérdidas accidentales de información se ve incrementada, ya que si se dañan los bloques físicos de algunos nodos-i, no se habrán perdido todos los nodos-i.

Toda la información de control sobre el grupo de cilindros puede situarse al principio del mismo; sin embargo, ésta no es la solución más segura, ya que la información de control de todos los grupos de cilindros estaría situada sobre el mismo plato del disco. Si se daña ese plato se perderá toda la información de configuración del disco y por lo tanto el resto de la información del mismo. Esto justifica que la información de control se almacene en una posición, con respecto al inicio del grupo de cilindros, diferente para cada grupo. La posición de cada grupo sucesivo se calcula para que esté alejada al menos un sector con respecto al grupo anterior. Así, la información del disco estará distribuida en diferentes platos y sectores, por lo que la pérdida de uno de ellos no va ocasiona el daño de todo el disco.

El espacio que hay entre el inicio del grupo de cilindros y el inicio de la información de control del grupo se aprovecha para almacenar datos. Sólo en el caso del primer grupo la información de control está situada al principio del mismo.

Como vemos, el diseño de los sistemas de ficheros del BSD responde a mejoras en tiempos de acceso y seguridad.

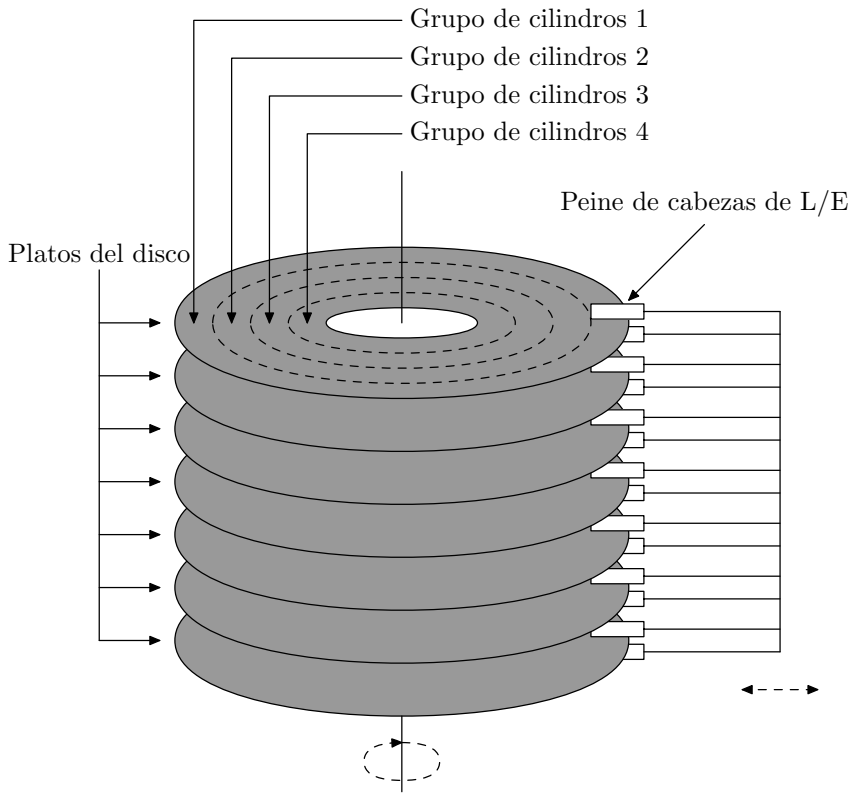


Figura 2.7: Organización del disco en el sistema BSD.

2.5 Tablas de control de acceso a los ficheros

Además de la tabla de nodos-i, el núcleo mantiene en memoria otras dos tablas que contienen información necesaria para poder manipular un fichero: la tabla de ficheros y la tabla de descriptores de fichero —consulte la figura 2.8—.

2.5.1 Tabla de nodos-i

Como ya hemos visto, la tabla de nodos-i es una copia en memoria de la lista de nodos-i que hay en el disco, a la que se le añade información adicional. Esta tabla se copia del disco para conseguir un acceso más rápido a sus componentes.

2.5.2 Tabla de ficheros

La tabla de ficheros es una estructura global del núcleo y en ella hay una entrada por cada fichero distinto que los procesos del núcleo o los procesos del usuario tienen abiertos. Cada vez que un proceso abre o crea un fichero nuevo, se reserva una nueva entrada en la tabla.

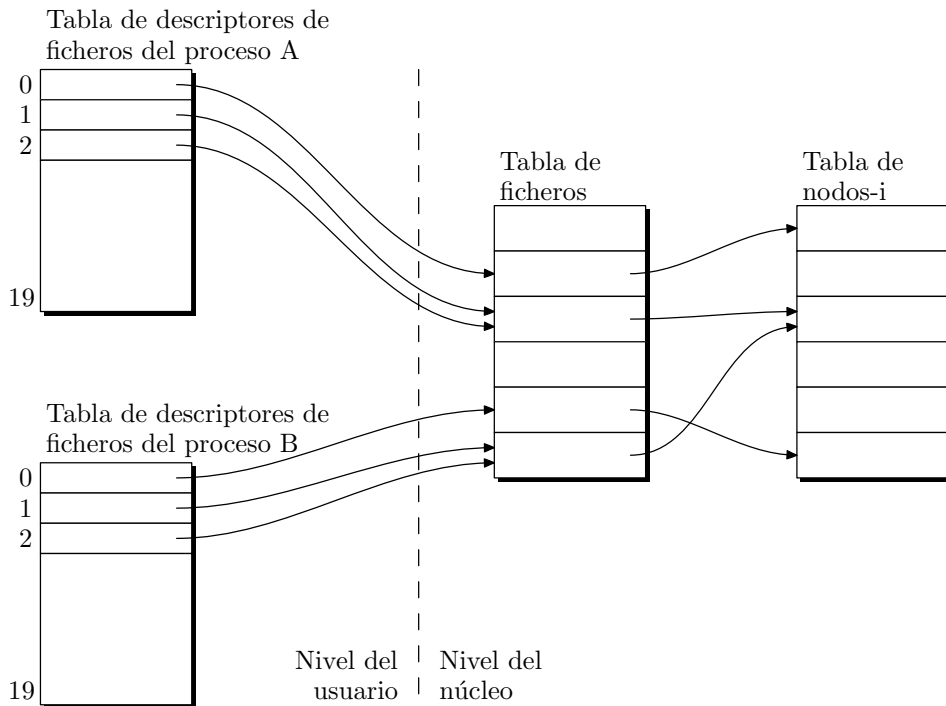


Figura 2.8: Tablas de control de acceso a los ficheros: tabla de descriptors, tabla de ficheros y tabla de nodos-i.

La tabla de ficheros es una estructura de datos orientada a objetos [Leffler *et al.*, 1989]. Cada entrada de la tabla contiene un bloque de datos y un array de punteros a funciones que traducen las operaciones genéricas que se pueden realizar con los ficheros: leer, escribir, situar el puntero de lectura/escritura, cerrar, posicionar, etc., en acciones concretas asociadas a cada tipo de fichero. Las operaciones que se deben implementar por cada tipo son:

- Funciones para leer —**read**— y escribir —**write**— en un fichero.
- Una función para realizar la multiplexación síncrona de la E/S —**select**—.
- Una función para controlar los modos de E/S —**ioctl**—.
- Una función para cerrar el fichero —**close**—.

Se puede observar que no hay ninguna rutina de apertura de ficheros en esta definición de objeto. Esto es debido a que el sistema sólo empieza a tratar los elementos de esta tabla como objetos a partir de que se haya abierto el fichero.

Cada entrada de la tabla de ficheros tiene también un puntero a una estructura de datos que contiene información del estado actual del fichero. Entre otros, se deben tener en cuenta los siguientes campos:

- Nodo-*i* asociado a la entrada de la tabla.
- Desplazamientos de lectura y escritura que indican sobre qué byte del fichero van a tener efecto las siguientes operaciones de lectura o escritura. Estos desplazamientos quedan actualizados cada vez que se realiza una de las operaciones anteriores.
- Permisos de acceso para el proceso que ha abierto el fichero.
- Indicadores del modo de apertura del fichero, que se verificarán cada vez que se realice una operación sobre el mismo para ver si es congruente. Por ejemplo, si un fichero se abre para ser leído solamente, el núcleo no permitirá que se realicen operaciones de escritura sobre él.
- Un contador para indicar cuántas entradas de la tabla de descriptores tiene asociadas esta entrada de la tabla de ficheros.

2.5.3 Tabla de descriptores de fichero

La tabla de descriptores de fichero es una estructura local a cada proceso. Esta tabla identifica todos los ficheros abiertos por un proceso. Cuando utilizamos las llamadas `open`, `creat`, `dup` o `link`, el núcleo devuelve un descriptor de fichero, que es un índice para poder acceder a las entradas de la tabla anterior. En cada una de las entradas de la tabla hay un puntero a una entrada de la tabla de ficheros del sistema.

Los procesos no manipulan directamente ninguna de las tablas anteriores —esto lo hace el núcleo—, sino que acceden a los ficheros a través de un descriptor asociado, que es un número. Cuando un proceso utiliza una llamada para realizar una operación sobre un fichero, le pasa al núcleo el descriptor de ese fichero. El núcleo usa este número para acceder a la tabla de descriptores de fichero del proceso y buscar en ella cuál es la entrada de la tabla de ficheros que le da acceso a su nodo-*i*.

Este mecanismo puede parecer artificioso, pero ofrece una gran flexibilidad cuando queremos que un proceso acceda simultáneamente a un mismo fichero en modos distintos, o que varios procesos compartan ficheros.

El tamaño de la tabla de descriptores tiene un valor limitado para cada proceso. Este valor depende de la configuración del sistema —consulte `limits`, constante `OPEN_MAX`—, aunque está bastante extendido que sea 20. Así, un proceso no puede tener abiertos más de `OPEN_MAX` ficheros distintos en un instante determinado. Si algún proceso necesita manejar más de `OPEN_MAX` ficheros, será obligatorio cerrar algunos —por ejemplo, los menos usados— para poder abrir otros, ya que todos no pueden estar abiertos simultáneamente. Esta situación se plantea, por ejemplo, cuando intentamos compilar un proyecto que consta de más de `OPEN_MAX` ficheros distribuidos entre ficheros de cabecera, ficheros fuente, ficheros objeto y bibliotecas. El compilador debe arreglárselas para generar el fichero ejecutable sin imponer límites al número de ficheros de nuestro proyecto.

Cuando se arranca un proceso en UNIX, el sistema le asigna tres ficheros que ocupan las tres primeras entradas de la tabla de descriptores. Estos ficheros se conocen como fichero estándar de entrada —por lo general es el teclado de un terminal— que tiene asociado el descriptor número 0, el fichero estándar de salida —la pantalla de un terminal— que tiene asociado el descriptor número 1, y el fichero estándar de salida de mensajes de

error —también suele estar asociado a la pantalla de un terminal— que tiene asociado el descriptor número 2 —consulte la figura 2.9—.

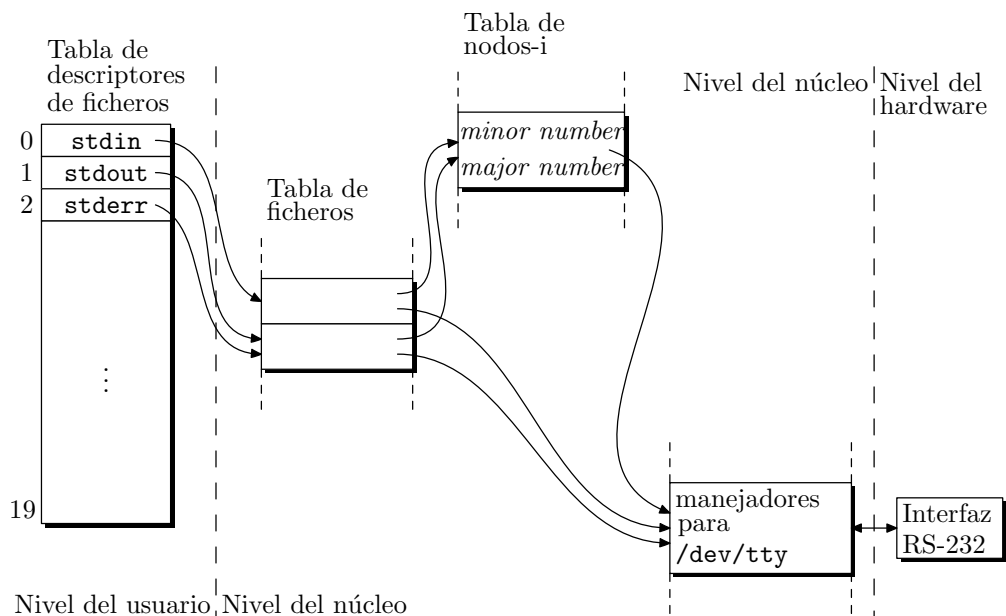


Figura 2.9: Ficheros estándar de entrada, salida y salida de mensajes de error de un proceso.

Estos ficheros están asociados por defecto a todo proceso y pueden cambiarse para que tomen otros valores. Uno de los procedimientos consiste en intercalar en el código del programa las llamadas necesarias para modificar la tabla de descriptors. Este procedimiento lo podremos emplear sólo cuando dispongamos del código fuente del programa y aun así no es muy recomendable. El otro procedimiento lo facilita el sistema a través de la redirección y las tuberías. Con los caracteres de redirección `<` y `>` podemos hacer que un programa codificado para trabajar con los ficheros estándar pase a trabajar con los ficheros que nos interesen. Por ejemplo, la orden

```
$ ls > directorio
```

hace que la salida estándar de `ls` —la pantalla— se redirija hacia el fichero `directorio`, y la orden

```
$ cc programa.c 2> errores
```

redirige la salida estándar de errores del compilador —descriptor 2 que por defecto tiene asociada la pantalla— hacia el fichero `errores`.

Como vimos en el párrafo dedicado a las tuberías con nombre, el carácter `|`, usado en la línea de órdenes, también permite modificar los ficheros estándar asociados a un programa. Así, la línea,

```
$ ls | grep root
```

hace que `ls` dirija su salida hacia una tubería sin nombre y que `grep` lea líneas de la misma tubería.

2.6 Administración de los sistemas de ficheros

Comentaré a continuación algunas de las tareas que se deben realizar para crear un sistema de ficheros y asegurar su correcto funcionamiento. Estos trabajos no corresponden al usuario del sistema, sino al administrador, pero es útil conocerlos para tener una idea más adecuada del funcionamiento del sistema UNIX. Sobre todo, el usuario neófito podrá obtener respuesta a algunas de las preguntas que se le plantean cuando se enfrenta al sistema por primera vez. Hay que indicar también que las órdenes y procesos que se describirán a continuación varían de un sistema a otro, por lo que la última palabra la tiene siempre la documentación técnica del sistema, a la que deberá acudir el lector en caso de duda.

2.6.1 Particiones del disco

La primera idea que se debe manejar es la posibilidad de que en un mismo disco físico coexistan varios sistemas operativos sin que interfieran unos con otros. Esto se consigue dedicando una partición del disco a cada uno de los sistemas. Un sistema puede ocupar una o varias particiones, dependiendo de cómo configuremos el disco. Las particiones son, por lo tanto, divisiones del disco independientes unas de las otras y compete al administrador del sistema decidir qué va a contener cada una de ellas.

Las particiones deben crearse antes de hacer ninguna instalación sobre el disco, ya que toda la información que éste contenga dejará de estar accesible. Cada sistema dispone de un programa que permite crear la tabla de particiones. En el caso del sistema DOS o de algunas versiones de UNIX para PC, el programa es **fdisk**. Este programa presenta un menú que permite definir el tamaño dedicado a cada partición, visualizar el total de particiones que tenemos definidas, definir una partición como partición activa, etc. No es necesario trocear todo el disco para poder trabajar con él. Si sólo necesitamos una parte, podemos crear una partición y posponer la creación de nuevas particiones para cuando tengamos necesidad de ellas. Igualmente, podemos dedicar la totalidad del disco a una sola partición.

Si en el disco hay instalados varios sistemas operativos, el usuario puede preguntarse cuál es el sistema que se carga al arrancar el ordenador. Para resolver este problema, **fdisk** permite definir una partición activa; es decir, una partición en la que se busca el sistema operativo en el momento de arranque. Naturalmente, para que una partición sea autoarrancable, debe tener un sector o bloque de *boot* y el archivo o archivos del sistema. Esto lo especificamos al dar formato a la partición.

2.6.2 Formato del disco

Una vez establecidas las particiones del disco, podemos pasar a instalar sistemas de ficheros sobre ellas. Dependiendo del sistema operativo que vaya a albergar cada partición, será necesario darle un tipo de formato u otro. Cada sistema tiene su propio programa para formatear; así, DOS dispone del programa **format** que se utiliza para dar formato a discos duros y disquetes. En UNIX no hay un programa estándar para formatear; cada fabricante suministra un conjunto de herramientas de administración del sistema y los aspectos de formateo pueden estar incluidos dentro de ellas. Así, por ejemplo, SUN ofrece

a sus usuarios el programa `diag`, que incluye la opción `format` para dar formato a una partición. `SCO` da formato a las particiones en el momento de instalar el sistema.

Otra de las verificaciones que debemos realizar antes de empezar a instalar sistemas de ficheros sobre el disco es revisar el estado de sus sectores y generar una tabla de sectores defectuosos. El programa que realiza esta operación también depende de la versión del sistema que estamos manejando. Algunos ejemplos son: `bad144` y `badsect` para el UNIX de Berkeley, `badblk` para AT&T, `badtrk` para SCO, etc.

2.6.3 Ficheros de dispositivo del disco

Con objeto de construir un sistema de ficheros sobre alguna de las particiones de disco, es necesario que en el directorio `/dev` esté presente el fichero de dispositivo que nos dé acceso a esa partición. Si estamos haciendo la primera instalación de UNIX, no es necesario llevar a cabo esta tarea, puesto que el programa de instalación se encarga de crear los ficheros de dispositivo necesarios. Sin embargo, en ampliaciones de una instalación ya realizada, hay que tener presentes estos aspectos.

La forma de crear los ficheros de dispositivo es mediante la orden `mknod`. Su sintaxis es la siguiente:

```
$ /etc/mknod nombre c|b major minor
```

`nombre` es el nombre del fichero de dispositivo que se va a crear; `c|b` indica el tipo de acceso: modo bloque o modo carácter, sólo uno debe estar presente; `major` es el *major number* del dispositivo y `minor` su *minor number*.

Para usar `mknod` debemos conocer los *major* y *minor number* de los dispositivos que puede manejar nuestro sistema. Esta numeración tampoco es algo estándar, por lo que debemos consultar la documentación técnica del fabricante. De forma estándar, en la sección 7 del manual se documentan los ficheros de dispositivo que maneja el sistema, así como el significado de sus números asociados.

De cara a los nombres de los dispositivos se suelen utilizar algunos convenios. Así, los ficheros de dispositivo de los discos duros se llaman `hd##`, donde `##` representa dos números que indican la unidad de disco y la partición a la que se refieren. Por ejemplo, `hd00` se refiere a la totalidad del primer disco físico, `hd01` a la primera partición del primer disco físico, `hd02` a la segunda partición, etc. Para referirnos a la totalidad del segundo disco, emplearemos el fichero de dispositivo `hd10`, la primera partición del segundo disco será `hd11`, y así sucesivamente.

Podemos ver los ficheros de dispositivo destinados a disco mediante la orden:

```
$ ls -al /dev/hd*
```

```
brw----- 2 sysinfo sysinfo 1,  0 Mar 14 1989 /dev/hd00
brw----- 2 sysinfo sysinfo 1, 15 Mar 14 1989 /dev/hd01
brw----- 2 sysinfo sysinfo 1, 23 Mar 14 1989 /dev/hd02
brw----- 2 sysinfo sysinfo 1, 31 Mar 14 1989 /dev/hd03
brw----- 2 sysinfo sysinfo 1, 39 Mar 14 1989 /dev/hd04
brw----- 2 sysinfo sysinfo 1, 47 Mar 14 1989 /dev/hd0a
brw-r----- 2 dos      sysinfo 1, 55 Mar 14 1989 /dev/hd0d
```

En las columnas 5 y 6 de la salida que produce la orden `ls` podemos ver cuáles son

los *major* y *minor number* de los distintos ficheros de dispositivo. En el ejemplo anterior, todos los ficheros de disco tienen el *major number* 1 y los *minor number* 0, 15, 23, 31, 39, 47 y 55, respectivamente.

También podemos ver que estos ficheros corresponden a dispositivos modo bloque, el carácter **b** de la primera columna así lo indica.

Como vimos en apartados anteriores, hay dispositivos que pueden ser referenciados a través de ficheros de dispositivo modo bloque o modo carácter. En concreto, los discos son de ese tipo de dispositivos, por lo que existe toda una familia de ficheros paralela a la anterior. Los nombres de estos ficheros responden al esquema **rhd##**, donde **##** representa dos números con el significado ya explicado. Para visualizar estos dispositivos podemos escribir:

```
$ ls -al /dev/rh*

crw----- 2 sysinfo sysinfo 1,  0 Mar 14 1989 /dev/rhd00
crw----- 2 sysinfo sysinfo 1, 15 Mar 14 1989 /dev/rhd01
crw----- 2 sysinfo sysinfo 1, 23 Mar 14 1989 /dev/rhd02
crw----- 2 sysinfo sysinfo 1, 31 Mar 14 1989 /dev/rhd03
crw----- 2 sysinfo sysinfo 1, 39 Mar 14 1989 /dev/rhd04
crw----- 2 sysinfo sysinfo 1, 47 Mar 14 1989 /dev/rhd0a
crw-r----- 2 dos      sysinfo 1, 55 Mar 14 1989 /dev/rhd0d
```

Se puede apreciar que los números asociados a estos ficheros son los mismos que los del modo bloque. La única diferencia es que el acceso a través de estos nuevos ficheros se va a realizar carácter a carácter, sin la intervención de la memoria intermedia, por lo que responderán de una forma más lenta.

Para los sistemas basados en el UNIX de AT&T, los ficheros de acceso al disco se encuentran, además, en los directorios **/dev/dsk** y **/dev/rdsk**. Ambos directorios contienen los mismos ficheros; el primero está dedicado a los ficheros de acceso al disco en modo bloque, y el segundo, a los ficheros de acceso en modo carácter. Los nombres de los ficheros responden al esquema **#s#**. El primer número indica el número de disco físico, y el segundo, la partición. Para algunos sistemas, estos números representan el manejador y la sección dentro del manejador. El significado real depende, como siempre, del sistema que estemos manejando.

Algunos de los ficheros que hay en el directorio **/dev/dsk** son:

```
$ ls -al /dev/dsk/*s*

brw----- 2 sysinfo sysinfo 1,  0 Mar 14 1989 /dev/dsk/0s0
brw----- 2 sysinfo sysinfo 1,15 Mar 14 1989 /dev/dsk/0s1
brw----- 2 sysinfo sysinfo 1,23 Mar 14 1989 /dev/dsk/0s2
brw----- 2 sysinfo sysinfo 1,31 Mar 14 1989 /dev/dsk/0s3
brw----- 2 sysinfo sysinfo 1,39 Mar 14 1989 /dev/dsk/0s4
brw----- 2 sysinfo sysinfo 1,47 Mar 14 1989 /dev/dsk/0sa
brw-r----- 2 dos      sysinfo 1,55 Mar 14 1989 /dev/dsk/0sd
```

Podemos ver que los *major* y *minor number* coinciden con los que presentan los ficheros con la forma **hd##**; en realidad, se trata de los mismos ficheros. En efecto, tal y como

vemos en la segunda columna, cada uno de los ficheros tiene un total de dos enlaces, uno para el fichero de la forma `/dev/hd##` y otro para el fichero de la forma `/dev/dsk/#s#`.

En lo que respecta a los ficheros de dispositivo de la forma `/dev/rdsk/#s#`, hay que decir que son los mismos estudiados en la forma `/dev/rhd##`.

Para crear un fichero especial de dispositivo de disco mediante `mknod`, habrá que escribir algo parecido a:

```
$ mknod /dev/hd02 b 1 23
```

La creación de ficheros de dispositivo con la ayuda de `mknod` implica el conocimiento de los números *major device number* y *minor device number*. Estos números están documentados en los manuales del fabricante y están asignados de una forma bastante arbitraria. Para que al usuario le resulte mas cómodo crear los ficheros sin necesidad de indagar cuáles son los números que debe pasar al programa `mknod`, se le suministra un programa del intérprete de órdenes que crea de forma automática todos los ficheros de una familia de dispositivos. Este programa es `mkdev` y está en el directorio `/etc` o en `/dev`. La forma de invocarlo será:

```
$ mkdev hd
```

para crear todos los ficheros del tipo `hd##` y

```
$ mkdev rhd
```

para crear todos los ficheros del tipo `rhd##`, y así para otras familias de ficheros, entre las que pueden estar ficheros de dispositivo de terminales serie, cintas, etc.

2.6.4 Construcción de un sistema de ficheros

En estos momentos, el disco está preparado para crear un sistema de ficheros sobre alguna de sus particiones, para ello debemos usar el programa estándar `mkfs`. `mkfs` da formato a la partición reservando el primer bloque como bloque de *boot*, crea el superbloque con la lista de bloques libres, la tabla de nodos-i y los mapas de direcciones de los bloques de datos. También crea los directorios raíz y `lost+found` para el sistema de ficheros. La sintaxis de la llamada al programa dependerá del sistema en el que trabajemos; en el caso más general, queda como sigue:

```
$ /etc/mkfs [-L|-S] special size [nsect ntrack blksize fragsize ncpgr  
minfree rps nbpi]
```

- **special** es el fichero de dispositivo de la partición o sección del disco donde se va a construir el sistema de ficheros.
- **size** es el total de bloques que tendrá el sistema. El tamaño del bloque se define mediante la constante `BSIZE` del fichero `<sys/param.h>`.
- **nsect** es el número de sectores por pista.
- **track** es el número de pistas por cilindro.
- **blksize** es el tamaño principal del bloque. Debe ser una potencia de 2 comprendida entre 4.096 y 8.192.

- **fragsize** representa la menor cantidad de disco que se le puede asignar a un fichero. Su valor debe ser una potencia de 2 comprendida entre **BSIZE** y **MAXBSIZE**.
- **ncp** es el número de cilindros de disco que tendrá cada grupo de cilindros. Aplicable sólo a sistemas basados en BSD.
- **minfree** es el porcentaje mínimo de disco libre permitido. Cuando el porcentaje de disco libre es inferior a esta cantidad, sólo el superusuario puede ocupar bloques de datos. Esto garantiza que el sistema no se bloquee por falta de espacio libre. El valor por defecto de este campo suele ser del 10%.
- **rps** es la velocidad angular del disco especificada en revoluciones por segundo.
- **nbpi** es el número mínimo de bytes de la zona de datos que referencia cada nodo-i. El total de nodos-i se calcula en función del tamaño del sistema de ficheros.

Las opciones **-L** y **-S** le indican a **mkfs** qué tipo de fichero de directorio debe crear: **L** para directorios con el formato BSD, directorios que pueden contener nombres de fichero de hasta 255 caracteres; **S** para ficheros con el formato AT&T, donde el nombre de fichero ocupa sólo 14 caracteres. Si no se especifica, **mkfs** crea directorios del mismo tipo que los del directorio raíz que ya esté instalado.

Otra forma de invocar a **mkfs** es la siguiente:

```
$ /etc/mkfs [-L|-S] special proto [nsect ntrack blksize fragsize ncp
minfree rps nbpi]
```

donde aparece como novedad el campo **proto**. **proto** es un fichero prototipo con datos para **mkfs** y se compone de elementos separados por espacios en blanco o por caracteres de nueva línea. A continuación mostramos un ejemplo de fichero prototipo:

```
/etc/boot
4872 110
d--777 3 1
usr d--777 3 1
    sh ---755 3 1 /bin/sh
    ken d--755 6 1
        $
    b0 b--644 3 1 0 0
    c0 c--644 3 1 0 0
        $
$
```

La primera línea es el nombre del fichero que se va a copiar en el bloque número cero como programa de *boot* —**/etc/boot**—. Si el nombre es "", no se instala ningún programa de *boot*. El segundo elemento es el tamaño en bloques del sistema —4.872 bloques—. A continuación viene el tamaño de la lista de nodos-i especificado en bloques —110 bloques dedicados a nodos-i—. La línea tercera contiene la especificación del directorio raíz. Las especificaciones para ficheros se componen de elementos que informan sobre el modo del fichero, el identificador de usuario —*UID*—, identificador de grupo —*GID*—

y el contenido inicial del fichero. La sintaxis del campo contenido depende del modo del fichero.

El campo de modo está formado por una cadena de 6 caracteres. El primer carácter especifica el tipo de fichero —los caracteres `-bcd` indican que se trata de un fichero: normal, de dispositivo modo bloque, de dispositivo modo carácter o directorio, respectivamente—. El segundo carácter es una `u` o un `-` para indicar si está o no activo el bit *cambiar el identificador de usuario en ejecución*. El tercer carácter vale `g` o `-`, según esté o no activo el bit *cambiar el identificador de grupo en ejecución*. Los restantes tres caracteres del campo modo son tres dígitos en octal que indican los permisos de lectura, escritura y ejecución para el propietario del fichero, el grupo al que pertenece el propietario y el resto de los usuarios. Para más información sobre el campo modo, consultar la página `chmod(2)` del manual y el § 3.5.2 pág. 80.

Los dos números decimales que siguen al campo modo son el identificador de usuario y de grupo del propietario del fichero.

Así, por ejemplo, en el fichero anterior, la línea `usr d-777 3 1` indica que se debe crear en el nuevo sistema de ficheros un directorio `usr` que depende del directorio raíz, que no tiene inactivos los bits *cambiar el UID* y *cambiar el GID*, sobre el que todos los usuarios tienen permisos de lectura, escritura y ejecución, y cuyo propietario tiene el identificador de usuario número 3 e identificador de grupo número 1.

Si el fichero es de tipo ordinario, en la misma línea debe aparecer la ruta del fichero cuyo contenido se va a copiar en el nuevo sistema de ficheros. Así, en el ejemplo anterior, la línea `sh --755 3 1 /bin/sh` le indica a `mkfs` que en el directorio `/usr` del nuevo sistema de ficheros debe crearse el fichero `sh`, que es una copia del fichero `/bin/sh` del sistema actual.

Si el fichero es de dispositivo en modo bloque o carácter, en la misma línea deben aparecer el *major number* y el *minor number* de dispositivo correspondiente.

Si el fichero a crear es un directorio, el programa `mkfs` reserva en él las dos primeras entradas para los ficheros `.` y `..`, y a continuación lee de forma recursiva la lista de ficheros que se deben crear en ese directorio y sus subdirectorios. Para indicar que en un directorio no hay más entradas, se utiliza el carácter `$`.

Ejemplos de llamadas a `mkfs` son los siguientes:

```
$ mkfs /dev/hd03 4872 48 20 8192 1024 16 10 66 2048
```

La orden anterior crea un sistema de ficheros en la sección tres del primer disco y lo configura con los siguientes parámetros: 4.872 bloques, 48 sectores por pista, 20 pistas por cilindro, 8.192 bytes por bloque, 1.024 bytes de fragmento mínimo manejable, 16 cilindros por grupo de cilindros, 10% mínimo libre, 66 rps velocidad de giro del disco y 2.048 bytes mínimo direccionable por cada nodo-i.

Si queremos crear un sistema de ficheros con las características anteriores y con una configuración de ficheros como la del fichero prototipo anterior, haríamos una llamada como la siguiente:

```
$ mkfs /dev/hd03 /stand/diskboot 48 20 8192 1024 16 10 66 2048
```

donde `/stand/diskboot` es el fichero prototipo.

2.6.5 Comprobación del estado de un sistema de ficheros

Nada más crear el sistema de ficheros, debemos revisarlo para verificar su consistencia y asegurar que todos sus bloques son accesibles. Esto se consigue con el programa **fsck**. Su sintaxis es la siguiente:

```
$ fsck [opciones] [sistema ...]
```

fsck revisa y repara de forma interactiva las posibles inconsistencias que encuentra en los sistemas de ficheros de UNIX. Si el sistema no presenta inconsistencias, el programa **fsck** informa sobre el número de ficheros, número de bloques usados y número de bloques libres de que dispone el sistema. Si el sistema es inconsistente, **fsck** proporciona mecanismos para corregirlo.

fsck revisa los sistemas de ficheros que se le indican en la línea de órdenes. Si no se especifica ningún sistema, **fsck** revisa los sistemas que se especifican en el fichero `/etc/checklist` —AT&T— o en el `/etc/fstab` —BSD—. Las inconsistencias que revisa **fsck** son las siguientes:

- Bloques reclamados por más de un nodo-i o la lista de bloques libres.
- Bloques reclamados por un nodo-i o la lista de bloques libres, pero que están fuera del rango del sistema.
- Contadores de enlaces incorrectos.
- Revisión de tamaños: número de bloques demasiado grande, tamaño de directorios inadecuado.
- Formato inadecuado para los nodos-i.
- Bloques no registrados por nadie —nodos-i, lista de bloques libres, etc.—
- Revisión en directorios: ficheros que apuntan a nodos-i no asignados, números de nodo-i fuera de rango.
- Revisión en el superbloque: más bloques para nodos-i de los que hay en el sistema de ficheros.
- Formato incorrecto de la lista de bloques libres.
- Total de bloques libres o contador de nodos-i libres incorrecto.

Si **fsck** encuentra un fichero o directorio cuyo directorio padre no puede determinarse, colocará al fichero huérfano en el directorio **lost+found** perteneciente al sistema que se está revisando. Puesto que el nombre del fichero se registra en su directorio correspondiente y éste es desconocido, a la hora de guardarlo en **lost+found** no podemos hacerlo con su verdadero nombre, por lo que se nombrará con su número de nodo-i.

fsck no sólo se utiliza para revisar un sistema de ficheros recién creado. Su misión principal es revisar sistemas estropeados por alguna causa accidental como una parada incontrolada del sistema. Como sabemos, el sistema mantiene copias en memoria tanto del superbloque como de la tabla de nodos-i. Además, el acceso a disco se realiza a través de la memoria intermedia. Esto crea inconsistencias entre el disco y la memoria,

inconsistencias que se corrigen periódicamente con la intervención de los demonios **syncer** o **update** que se encargan de realizar una llamada a **sync** para sincronizar el disco con la memoria. Si por cualquier circunstancia el sistema deja de funcionar antes de que se produzca una sincronización, la próxima vez que se intente utilizar ese sistema de ficheros será necesario repararlo en la medida de lo posible con la ayuda de **fsck**.

Para usuarios expertos, UNIX suministra un depurador manual del sistema conocido como **fsdb**. Con este programa se puede examinar el estado del sistema de ficheros y también se puede intentar una reparación de inconsistencias. No es aconsejable trabajar con este depurador debido a su dificultad y a lo delicado del sistema de ficheros. Un usuario inexperto podría dañar seriamente el sistema si intenta manipularlo con el programa **fsdb**.

Un ejemplo de utilización de **fsck** es el siguiente:

```
$ fsck /dev/root
```

La llamada anterior revisará el sistema de ficheros **/dev/root** y reparará los daños que encuentre en él. Hay que indicar que **/dev/root** es el fichero de dispositivo que nos da acceso al disco de arranque del sistema y sobre el que se monta el directorio raíz.

2.6.6 Montaje y desmontaje de un sistema de ficheros

Hasta ahora hemos visto que la vía de acceso a un sistema de ficheros es el fichero de dispositivo que da acceso al disco o a la partición del disco donde se ha creado ese sistema. Sin embargo ésta no es una forma cómoda ni operativa de manejar los ficheros que hay en el sistema. Esta vía de acceso se utiliza sólo para realizar las tareas de más bajo nivel con el sistema de ficheros —tareas tales como revisarlo para reparar posibles daños—. Lo normal es que un sistema de ficheros sea accesible a través de la jerarquía de directorios de UNIX, para lo cual hay que llevar a cabo una operación conocida como montar el sistema de ficheros. Montar un sistema consiste en asociarle un directorio que será el punto de acceso a los ficheros y directorios. La orden que se utiliza es **mount** y su sintaxis es la siguiente:

```
$ /etc/mount sistema_de_ficheros directorio
```

mount le indica al sistema operativo que un sistema de ficheros será unido al árbol de directorios en la entrada **directorio**.

directorio debe ser la ruta absoluta de un directorio que esté presente en el árbol. Este directorio pasará a ser el directorio raíz de sistema de ficheros que se está montando. Si el directorio tuviese ficheros, éstos no aparecerán, dará la impresión de que se han borrado; en realidad no se han borrado pero no serán accesibles hasta que no se desmonte el sistema de ficheros. Esto es así porque los ficheros visibles a través del directorio pasan a ser los del nuevo sistema de ficheros que se ha montado sobre él.

sistema_de_ficheros es el nombre del fichero de dispositivo que da acceso al nuevo sistema que vamos a montar.

El programa **mount** mantiene en el fichero **/etc/mount** una tabla de sistemas de ficheros montados. Si invocamos a **mount** sin argumentos, podemos ver el contenido de esta tabla:

```
$ /etc/mount
```

```
/dev/root on / read/write on Tue Oct 13 16:16:02 1992
```

```
/dev/fd0135ds18 on /mnt read/write on Tue Oct 13 17:22:07 1992
```

Como vemos, el sistema `/dev/root` está montado sobre el directorio `/`, y el sistema `/dev/fd0135ds18`, sobre el directorio `/mnt`. Estos dos sistemas han sido montados con las órdenes:

```
$ /etc/mount /dev/root /
```

```
$ /etc/mount /dev/fd0135ds18 /mnt
```

Para demontar un sistema de ficheros usaremos la orden `umount`,

```
$ /etc/umount sistema_de_ficheros
```

Por ejemplo,

```
$ /etc/umount /dev/fd0135ds18
```

desmonta el sistema `/dev/fd0135ds18` y sus ficheros dejarán de estar accesibles.

Por lo general, el usuario no debe preocuparse de ir montando y desmontando sistemas de ficheros, ya que ésta es una tarea que se realiza de forma automática al arrancar la máquina y al apagarla mediante el programa `shutdown`. Es importante tener presente que antes de desconectar un ordenador, todos los sistemas de ficheros deben estar desmontados para que no se produzcan inconsistencias. El sistema `/dev/root` se monta desde el programa `rc` —programa invocado por `init` al arrancar el ordenador— sobre el directorio raíz.

Los programas `mount` y `umount` sólo pueden ser invocados por el superusuario.

2.6.7 Monitorización y contabilidad del sistema de ficheros

UNIX suministra de forma estándar una serie de programas que permiten hacer un seguimiento del estado de uso de los distintos sistemas de ficheros que hay montados.

El programa `df` informa sobre el nivel de ocupación de un sistema de ficheros. Su sintaxis es:

```
$ df [opciones] [sistema_de_ficheros]
```

Sus opciones más comunes son:

- `-t` informa sobre el total de bloques ocupados.
- `-f` informa sobre el total de bloques que hay en la lista de bloques libres.
- `-v` informa sobre el porcentaje de bloques ocupados, así como del número de bloques usados y libres.
- `-i` informa sobre el porcentaje de nodos-i ocupados, así como del número de nodos-i usados y libres.

Ejemplos de uso de `df` son:

```
$ df -t
```

```
/      (/dev/root ): 138150 blocks 19552 i-nodes
                    ( 178230 total blocks, 2788 for i-nodes)
```

```
$ df -f
```

```
Mount Dir Filesystem blocks used    free % used
/        /dev/root  178230 40082 138148    22%
```

El programa `quot` informa sobre el grado de ocupación del disco que corresponde a cada usuario del sistema. A continuación vemos la sintaxis y un ejemplo de uso de esta orden:

```
$ quot [opciones] [sistema_de_ficheros]
```

```
$ quot -f
```

```
/dev/root:
29144      2030  bin
3896       474  root
1876        65  uucp
600         47  lp
414         40  sysinfo
350         8   dos
```

La primera columna presenta los bloques que está ocupando cada usuario, la segunda indica el total de ficheros que tiene ese usuario y la tercera es el nombre del usuario.

El programa `du` informa sobre el uso de disco que se realiza en un nodo de la jerarquía de directorios. Su sintaxis es:

```
$ du [opciones] [path_name]
```

En el siguiente ejemplo vemos un listado de los bloques de disco que ocupa cada uno de los directorios que hay en `/usr/sys`.

```
$ du /usr/sys
```

```
208      /usr/sys/h
1732     /usr/sys/conf
32       /usr/sys/ml
6        /usr/sys/xnet
250     /usr/sys/sys
300     /usr/sys/mddep
688     /usr/sys/io
3218    /usr/sys
```

2.7 Ejercicios

- 2.1** (Linux) El nombre y contenido de los ficheros de cabecera que describen la estructura del sistema de ficheros UNIX es muy dependiente de cada implementación particular. En el caso de Linux, ¿cómo se llaman y dónde se encuentran los ficheros equivalentes a `<sys/filsys.h>` y `<sys/ino.h>`?

- 2.2** (Linux) En el sistema Linux, ¿cuál es el nombre equivalente de la estructura `struct dinode` donde se define la composición de un nodo-i? ¿En qué fichero de cabecera se encuentra esta estructura? Enumere alguno de los campos de esta estructura.
- 2.3** (Linux) ¿Cuál es la estructura que define la composición del superbloque de los sistemas de ficheros de Linux? Enumerar alguno de los campos más importantes de esta estructura.
- 2.4** En algunos sistemas UNIX los bloques de disco tienen un tamaño de 8Kb. ¿Cuál es el tamaño del fichero más grande que se puede almacenar en estos sistemas? Para responder a esta pregunta vamos a suponer que las entradas de direcciones de bloque de un nodo-i son las siguientes:
- 10 entradas de direcciones directas,
 - 1 entrada indirecta simple,
 - 1 entrada indirecta doble.

Las direcciones de bloque constan de 32 bits.

- 2.5** ¿Qué problemas nos encontramos si en el ejercicio anterior suponemos que también hay una entrada de dirección indirecta triple? ¿Cómo se solucionaría este problema? ¿Cuál sería el tamaño óptimo de las direcciones de bloque una vez solucionado el problema?
- 2.6** Complete la tabla siguiente donde figura el rango de bloques y bytes a los que se puede acceder con cada uno de los elementos de dirección de un nodo-i —suponga que cada bloque es de 1.024 bytes—.

Tipo dirección	Rango Bloques	Rango Bytes	Total Bloques
Entradas de dirección directas (0...9)	0..9	0...10.239	10
Dirección indirecta simple			
Dirección indirecta doble		272.384...671381.247	
Dirección indirecta triple			$256^3 = 16_1777.216$

- 2.7** Supongamos un sistema de ficheros donde el tamaño del bloque es 1.024 bytes. ¿En qué parte del sistema de ficheros está definida esta característica?
- Un programa crea un fichero de datos y escribe sendos bytes en las posiciones 5.243 y 90₁72.475. ¿Cuántos bloques de datos ocupará el fichero?
- 2.8** ¿Cuál es el tamaño máximo de una tubería con nombre y por qué? ¿Qué problemas puede plantear esta limitación de tamaño?

2.9 El fichero de dispositivo `/dev/tty` representa el terminal que da servicio a cada usuario del sistema UNIX. Con la función estándar `fopen` podemos abrir este fichero y asociarle un flujo para manejarlo desde un programa. Escriba un programa que:

- (a) Declare un array de seis flujos —`FILE *f[6];`—
- (b) Abra el fichero `/dev/tty` tres veces en modo lectura y le asocie los tres primeros flujos del array.
- (c) Abra el fichero `/dev/tty` tres veces en modo escritura y le asocie los tres últimos flujos del array.
- (d) Escriba en el fichero estándar de salida el descriptor asociado a cada uno de los flujos anteriores.
- (e) Usando la función `fprintf` escriba sobre cada uno de los flujos de salida el mensaje:
 Mensaje dirigido a al flujo nro. #
 donde # debe ser sustituido por el número del flujo —posición que ocupa en el array—.

¿Sobre qué dispositivo físico se produce la salida de los mensajes mencionados en el punto 5?

¿Cómo podemos redirigir la salida que se produce sobre los flujos `f[3]`, `f[4]` y `f[5]` para que se escriba en los ficheros `f3.txt`, `f4.txt` y `f5.txt` respectivamente?

- 2.10** (Linux) ¿Cómo se llaman los ficheros de dispositivo que controlan el acceso a las distintas particiones de los discos rígidos del sistema Linux?
- 2.11** (Linux) ¿Cómo se llaman los ficheros de dispositivo que controlan el acceso a los discos flexibles en el sistema Linux?
- 2.12** (Linux) ¿De qué tipo es el fichero `/dev/hda1`? ¿Cuál es su *major number* y cuál su *minor number*?
- 2.13** ¿Cuál es el *major number* de los ficheros de dispositivo que controlan el acceso a los terminales?
- 2.14** (Linux) ¿Cuál es la orden equivalente a `mkdev` en el sistema Linux? ¿En qué directorio se encuentra y qué tipo de fichero es?
- 2.15** (Linux) ¿Qué tipo de dispositivo es controlado por los ficheros `/dev/sda`, `/dev/sdb`, `/dev/sdc`, `/dev/sdd` y `/dev/sde`?
- 2.16** ¿Qué significan las siglas *SCSI*? ¿Qué es un disco *CDROM-SCSI*? ¿Cuáles son los ficheros de dispositivo para acceder a los discos *CDROM-SCSI*?
- 2.17** ¿Cómo podemos ver el número del nodo-i asociado a un fichero, usando la orden `ls`? ¿Qué nodo-i ocupa el directorio raíz? ¿Qué nodo-i ocupa el directorio padre del directorio raíz? ¿Qué nodo-i ocupa nuestro directorio de arranque?

2.18 Cree el directorio `tmp.1` en el directorio de arranque. Ejecutar la orden:

```
$ ln tmp.1 tmp.2
```

¿Qué ocurre? ¿Cómo podemos enlazar el directorio `tmp.2` con `tmp.1`?

2.19 ¿Cuántos enlaces duros tiene el programa `/usr/bin/cc`? ¿Con qué otro nombre aparece en el sistema de ficheros?

2.20 (Linux) ¿Cuántos enlaces duros tiene el fichero `/usr/bin/mread`? Escribir una orden que busque en todo el disco y muestre en pantalla todos los ficheros que están enlazados con `mread`.

2.21 (Linux) ¿Cuál es la orden Linux para crear un sistema de ficheros?

Inserte un disquete de alta densidad en la unidad de $3\frac{1}{2}$ " del ordenador y crear en él un sistema de ficheros con un tamaño de 1.440 bloques —use para ello el fichero de dispositivo `/dev/fd0`—.

¿Cuánto valen los siguientes parámetros del sistema recién creado?

- Número de nodos-i.
- Bloques reservados para el sistema.
- Tamaño del bloque.
- Tamaño del fragmento de bloque.

2.22 (Linux) Compruebe el estado del sistema de ficheros creado en el disco flexible en el ejercicio anterior.

2.23 (Linux) Cree el directorio `mnt` en el directorio de arranque y ejecute la orden:

```
$ echo > $HOME/mnt/prueba1
```

Al ejecutar la orden `ls mnt` vemos que el directorio `mnt` tiene un fichero `prueba1` de 1 byte. Monte el sistema `/dev/fd0` sobre el directorio `$HOME/mnt`.

2.24 (Linux) Visualice el contenido del fichero `/etc/mntab` y responda a las preguntas siguientes:

¿Se puede leer y escribir en el sistema de ficheros montado sobre `$HOME/mnt`?

¿Cómo podríamos montar un sistema de ficheros y deshabilitar las operaciones de escritura sobre el mismo?

2.25 (Linux) Desmonte el sistema de ficheros que hay en el disquete y vuelva a montarlo en el directorio `mnt` del directorio de arranque. En esta ocasión se debe montar sin permisos de escritura. Ejecute la orden siguiente y explique lo que ocurre:

```
$ echo > $HOME/mnt/prueba2
```

2.26 (Linux) Si ejecutamos la orden `ls $HOME/mnt` vemos que el fichero `prueba1` ha desaparecido. ¿Dónde se encuentra este fichero? ¿Ha sido borrado? ¿Se puede recuperar?

- 2.27** (Linux) Use la orden `df` para ver los nodos-i usados y libres que tiene el sistema de ficheros montado sobre `$HOME/mnt`.
- 2.28** (Linux) ¿Qué porcentaje de la partición Linux del disco rígido hay ocupada? ¿Cuántos bloques hay ocupados y cuántos libres?
- 2.29** (Linux) Si dividimos el total de bloques ocupados por el total de bloques y el resultado lo multiplicamos por 100, obtendremos el % usado del sistema de ficheros:

$$\% \text{ (disco usado)} = (\text{Total bloques usados} / \text{Total bloques}) \times 100$$

Calcule el porcentaje libre de los sistemas montados sobre el directorio raíz y sobre el directorio `$HOME/mnt`. ¿Detecta alguna diferencia entre los resultados calculados y los que muestra el sistema? En caso afirmativo, ¿a qué se debe esa diferencia?

- 2.30** (Linux) Repita la secuencia de órdenes siguiente:

- (a) Monte el disquete de $3\frac{1}{2}$ " en el directorio `$HOME/mnt`. En el caso de que ya esté montado, desmóntelo y vuelva a montarlo.
- (b) `$ cd`
- (c) `$ echo >mnt/prueba3`
- (d) `$ ls mnt`
- (e) Extraiga el disquete de la unidad de $3\frac{1}{2}$ ".
- (f) Desmonte el sistema que hay en `mnt`.
- (g) Vuelva a montar el disquete de $3\frac{1}{2}$ " en el directorio `$HOME/mnt`.
- (h) Al ejecutar la orden `ls mnt` vemos que el fichero `prueba3` ha desaparecido del disquete. Sin embargo, al ejecutar la misma orden en el punto 3, el fichero `prueba3` estaba en el directorio. ¿A qué se debe este comportamiento?

- 2.31** Escriba un programa que reciba a través de la línea de órdenes un número entero que representa el tamaño del bloque de un sistema de ficheros. El programa debe generar una tabla con los rangos de representación de cada una de las entradas de direcciones de un nodo-i. Para realizar estos cálculos se debe calcular previamente el tamaño de cada dirección tomando como unidad mínima el byte. Es decir, si el tamaño óptimo es de 45 bits, tenemos que redondear por arriba y la dirección será de 48 bits —6 bytes—.

La salida que produce el programa debe tener un aspecto similar a la tabla generada en el ejercicio de la pág. 45:

`$ tabla 1024`

Tipo dir.	Rango bloques	Rango bytes	Total bloques
directa 0	0	0..1023	1
directa 1	1	1023..2047	1

```

...
indirecta      10..265      ?      ?
indirecta 2    ?           272384..67381247  ?
indirecta 3    ?           ?           16777216

```

2.32 Escriba un programa para simular la gestión de los bloques de direcciones del sistema operativo UNIX. El objetivo del ejercicio es ilustrar la forma de acceso a los distintos bytes de un fichero a través de los mecanismos de direccionamiento que hay en los nodos-i. El programa debe trabajar con la estructura siguiente:

```

typedef struct {
    char *directos[10]; // Entradas con las direcciones de los bloques directos.
    char **simple; // Entrada con la dirección del bloque indirecto simple.
    char ***doble; // Entrada con la dirección del bloque indirecto doble.
    char ****triple; // Entrada con la dirección del bloque indirecto triple.
}DIRECCIONES;

```

Para trabajar con esta estructura se deben escribir las funciones siguientes:

- `void escribir_byte (DIRECCIONES *dir, char byte, long posición);`
 - `dir` es un puntero a las entradas de dirección de un nodo-i. Este puntero se habrá inicializado previamente con una llamada a `malloc`.
 - `byte` es el byte que queremos escribir en el fichero.
 - `posición` indica la posición donde se debe escribir el byte dentro del fichero.

Cuando escribimos en el fichero habrá que acceder a sus bloques de datos para escribir el byte indicado. Como estos bloques no existen cuando se hace la primera escritura, será necesario reservar memoria para los mismos. La reserva se realizará con llamadas a `malloc` suponiendo que cada bloque de datos es de 1.024 bytes.

Cuando sea necesario escribir en una posición a la que se acceda a través de las direcciones indirectas, hay que tener presente que la reserva de bloques para escribir el byte implica la reserva de bloques que van a contener direcciones intermedias.

En el caso de escribir en una posición para la que ya hay reservado un bloque de datos por haber escrito anteriormente en otra posición que necesitaba ese bloque, no será necesario, como es lógico, volver a reservar memoria para ese bloque.

- `char leer_byte (DIRECCIONES *dir, long posición);`
 - `dir` es un puntero a las entradas de dirección de un nodo-i. Este puntero se habrá inicializado previamente con una llamada a `malloc`.
 - `posición` indica la posición que ocupa, dentro del fichero, el byte que queremos leer.

La función devuelve el byte leído del fichero.

Para probar estas funciones podemos escribir una sección de programa como la siguiente:

```
main ()
{
    DIRECCIONES *dir;

    // Reserva de memoria para dir
    escribir_byte (dir, 45, 1025L); // Dirección directa.
    escribir_byte (dir, 255, 56356L); // Implica indirección doble
    printf ("%d\n", leer_byte (dir, 136256L)); // Escribe 0
    printf ("%d\n", leer_byte (dir, 56356L)); // Escribe 255
}
```

Capítulo 3

Manejo de ficheros ordinarios

«Bien es verdad que el segundo autor desta obra no quiso creer que tan curiosa historia estoviese entregada a las leyes del olvido, ni que hubiesen sido tan poco curiosos los ingenios de la Mancha, que no tuviesen en sus archivos o en sus escritorios algunos papeles que deste famoso caballero tratasen...»

(Cervantes, Quijote I-8)

3.1	Introducción	51
3.2	Entrada/salida sobre ficheros ordinarios	53
3.3	Biblioteca estándar de funciones de entrada/salida	59
3.4	Control de ficheros abiertos (<code>fcntl</code>)	72
3.5	Administración de ficheros	77
3.6	Compartición y bloqueo de ficheros	87
3.7	Ejercicios	94

3.1 Introducción

En el capítulo anterior hemos delineado la estructura interna del sistema de ficheros de UNIX. En este capítulo vamos a estudiar la interfaz que ofrece el sistema para comunicarnos con el núcleo y poder acceder a los recursos del sistema de ficheros. Más concretamente, la comunicación se va a realizar con una parte del núcleo, el subsistema de ficheros o subsistema de entrada/salida. Trabajaremos con los ficheros aprovechando la estructura de alto nivel que ofrece el sistema de ficheros y que permite realizar una abstracción de lo que es el soporte físico de la información —los discos—.

Para empezar, vamos a ver un ejemplo sencillo que ilustra cómo se comunica un programa con el subsistema de ficheros. El programa siguiente es una versión simplificada de la orden `cp`, que permite copiar ficheros. Aquí lo he llamado `copiar`.

Programa 3.1: Copiar ficheros. Primera versión (copiar.c)

```

/**
    $ copiar origen destino
    Retorna 0 si se ejecuta satisfactoriamente o -1 en caso contrario
***/
#include <stdio.h> /* Fichero de cabecera de la biblioteca estándar de E/S */
#include <fcntl.h> /* Fichero de cabecera para llamadas al sistema de ficheros */

char buffer [BUFSIZ]; /* Memoria intermedia para manipular los datos. */

main (int argc, char *argv [])
{
    int fd_origen, fd_destino;
    int nbytes;

    /* Análisis de los argumentos que aparecen en la línea de órdenes. */
    if (argc != 3) {
        fprintf (stderr, "Forma de uso: %s origen destino\n", argv [0]);
        exit (-1);
    }
    /* Apertura del fichero origen en modo sólo lectura. */
    if ((fd_origen = open (argv [1], O_RDONLY)) == -1) {
        perror (argv [1]);
        exit (-1);
    }
    /* Apertura o creación del fichero destino en modo sólo escritura. */
    if ((fd_destino=open(argv[2],O_WRONLY|O_TRUNC|O_CREAT,0666)) == -1) {
        perror (argv [2]);
        exit (-1);
    }
    /* Copiamos el fichero origen en el fichero destino. */
    while ((nbytes = read (fd_origen, buffer, sizeof buffer)) > 0)
        write (fd_destino, buffer, nbytes);
    /* Cierre de ficheros. */
    close (fd_origen);
    close (fd_destino);

    exit (0);
}

```

Este programa tan sencillo muestra el esquema general que se debe seguir para trabajar con ficheros. La secuencia recomendada es:

- Abrir los ficheros —llamada `open`— y comprobar si se produce algún error en la apertura. En caso de error, `open` devuelve el valor `-1`.
- Manipular los ficheros de acuerdo con nuestras necesidades. En este ejemplo leemos

del fichero origen y escribimos en el fichero destino. Hay que hacer notar que la lectura/escritura se realiza en bloques de tamaño `BUFSIZ` —constante definida en `<stdio.h>`—. La forma de detectar que hemos leído todo el fichero origen es analizando el valor devuelto por `read`. Si este valor es igual a cero, significa que ya no quedan más datos por leer del fichero origen.

- Cerrar los ficheros una vez que hemos terminado de trabajar con ellos.

En el programa podemos ver que la forma de referirse a un fichero u otro es mediante los descriptores devueltos por `open`. Cada fichero abierto por nuestro programa va a tener asociado un descriptor que tendremos que utilizar siempre que queramos servirnos de alguna llamada al sistema.

Suele ser un convenio bastante extendido que un programa devuelva el valor 0 al sistema operativo cuando termina satisfactoriamente. Este valor es almacenado en la variable de entorno `$?`, que puede ser analizada por otro proceso para ver el código de error devuelto por el último proceso que la modificó. Algunas utilidades, como `make`, emplean estos códigos de error para determinar si deben proseguir su ejecución o deben detenerse. Desde la línea de órdenes podemos ver el contenido de `$?` escribiendo,

```
$ echo $?
```

Para los códigos devueltos en caso de error no existe ningún criterio concreto, salvo el de que sean distintos de 0.

3.2 Entrada/salida sobre ficheros ordinarios

En los párrafos siguientes vamos a estudiar las llamadas al sistema necesarias para realizar entrada/salida sobre ficheros ordinarios de datos.

3.2.1 Apertura de un fichero (`open`)

`open` es la llamada que utilizaremos para indicarle al núcleo que habilite las estructuras necesarias para trabajar con un fichero que especificaremos mediante una ruta. El núcleo devolverá un descriptor de fichero con el que podremos referenciar el fichero en llamadas posteriores. La declaración de `open` es:

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int open (char *path, int flag [, mode_t mode]);
```

`path` es un puntero a la ruta del fichero que queremos abrir. Puede ser una ruta absoluta o relativa, y su longitud no debe exceder de `PATH_MAX` bytes.

`flags` es una máscara de bits —varias opciones pueden estar presentes usando el operador `OR` a nivel de bits |— que le indica al núcleo el modo en que queremos que se abra el fichero. Uno de los bits, `O_RDONLY`, `O_WRONLY` u `O_RDWR`, y sólo uno, debe estar presente al componer la máscara; de lo contrario, el modo de apertura quedaría indefinido. Los `flags` más significativos que hay disponibles son:

- `O_RDONLY`, abrir en modo sólo lectura.
- `O_WRONLY`, abrir en modo sólo escritura.
- `O_RDWR`, abrir para leer y escribir.
- `O_NDELAY`, este indicador afectará a futuras llamadas de lectura/escritura.

En relación con `O_NDELAY`, cuando abrimos una tubería con nombre y activamos el modo `O_RDONLY` u `O_WRONLY`:

- Si `O_NDELAY` está activo:
Un `open` en modo sólo lectura regresa inmediatamente. Un `open` en modo sólo escritura devuelve un error si en el instante de apertura no hay otro proceso que tenga abierta la tubería en modo sólo lectura.
- Si `O_NDELAY` no está activo:
Un `open` en modo sólo lectura no devuelve el control hasta que un proceso no abre la tubería para escribir en ella. Un `open` en modo sólo escritura no devuelve el control hasta que un proceso no abre la tubería para leer de ella.

Si el fichero que queremos abrir está asociado con una línea de comunicaciones:

- Si `O_NDELAY` está activo, `open` regresa sin esperar por la portadora —llamada no bloqueante—.
- Si `O_NDELAY` está inactivo, `open` no regresa hasta que detecta la portadora —llamada bloqueante—.
- `O_APPEND`, el puntero de lectura/escritura del fichero se sitúa al final del mismo antes de empezar la escritura. Así garantizamos que lo escrito se añade al final del fichero.
- `O_CREAT`, si el fichero que queremos abrir ya existe, este indicador no tiene efecto, excepto en lo que se indicará para el indicador `O_EXCL`. El fichero es creado en caso de que no exista y se creará con los permisos indicados en el parámetro `mode`.
- `O_EXCL`, si está presente el indicador `O_CREAT`, `open` devuelve un código de error cuando el fichero ya existe.
- `O_TRUNC`, si el fichero existe, trunca su longitud a cero bytes, incluso si el fichero se abre para leer.

Estos son los indicadores más corrientes para `open`. Dependiendo del fabricante del sistema, pueden estar disponibles otros, por ello se aconseja consultar la entrada `open(2)` del manual.

`mode` es el tercer parámetro de `open` y sólo tiene significado cuando está activo el indicador `O_CREAT`. Le indica al núcleo qué permisos tendrá el fichero que va a crear. `mode` es también una máscara de bits y se suele expresar en octal mediante un número de 3 dígitos. El primero de los dígitos hace referencia a los permisos de lectura, escritura y ejecución para el propietario del fichero; el segundo se refiere a los mismos permisos para el grupo de usuarios al que pertenece el propietario, y el tercero se refiere a los permisos del resto de usuarios. Así, por ejemplo, `0644 —110 100 100—` indica

permisos de lectura y escritura para el propietario, y permiso de lectura para el grupo y para el resto de usuarios.

Si el núcleo realiza satisfactoriamente la apertura del fichero, **open** devolverá un descriptor de fichero. En caso contrario, devolverá -1 y en la variable **errno** pondrá el valor del tipo de error producido.

Los siguientes son ejemplos de apertura de ficheros:

```
int fd;
...
fd = open ("mifichero", O_RDONLY); // Abre un fichero para leer datos de él.

fd = open ("mifichero", O_WRONLY | O_TRUNC | O_CREAT, 0600);
// Abre un fichero para escribir datos en él. Si el fichero existe, trunca su tamaño
// a 0 bytes. Si el fichero no existe, lo crea con permiso de lectura y escritura para
// el propietario y ningún permiso para el grupo y demás usuarios.

fd = open ("mifichero", O_RDWR | O_APPEND);
// Abre un fichero en modo lectura/escritura y fuerza a que el puntero de
// lectura/escritura se sitúe al final del fichero.
```

3.2.2 Lectura de datos de un fichero (read)

read es la llamada que emplearemos para leer datos de un fichero. Su declaración es la siguiente:

```
#include <unistd.h>
int read (int fildes, char *buf, unsigned nbytes);
```

read lee **nbyte** bytes del fichero asociado al descriptor **fildes** y los coloca en la memoria intermedia apuntada por **buf**. Si la lectura se lleva a cabo correctamente, **read** devuelve el número de bytes realmente leídos y copiados en la memoria intermedia. Este número puede ser menor que **nbyte** en el caso de que el fichero esté asociado a una línea de comunicaciones, o de que quedasen menos de **nbyte** bytes por leer.

Cuando se intenta leer más allá del final del fichero, **read** devuelve el valor 0. Sólo en el caso de que **read** falle, devuelve el valor -1 y **errno** contendrá el tipo de error que se ha producido.

En los ficheros con capacidad de acceso aleatorio, la lectura empieza en la posición indicada por el puntero de lectura/escritura del fichero. Este puntero queda actualizado después de efectuar la lectura. En los ficheros asociados a dispositivos sin capacidad de acceso aleatorio —por ejemplo, líneas serie—, **read** siempre lee de la misma posición y el valor del puntero no tiene significado.

Los siguientes ejemplos muestran algunas formas de invocar a **read**. En estos ejemplos suponemos que **fd** es el descriptor de un fichero correctamente abierto.

```
char mem [4096];
int nbytes, fd;
...
```

```
nbytes = read (fd, mem, sizeof (mem));
// Lee 4.096 bytes que se almacenan en mem
```

La lectura no tenemos por qué hacerla siempre sobre un array de caracteres, también se puede hacer sobre una estructura. Supongamos que queremos leer 40 registros con un formato concreto de un fichero de datos. Si la composición de cada registro la tenemos definida en una estructura de nombre `REGISTRO`, una secuencia de código para efectuar esta lectura puede ser:

```
struct REGISTRO mem [40];
int nbytes, fd;
...
nbytes = read (fd, mem, 40 * sizeof (REGISTRO));
```

3.2.3 Escritura de datos en un fichero (write)

Utilizaremos la llamada `write` para escribir datos en un fichero. Su declaración es muy parecida a la de `read`:

```
int write (int fildes, char *buf, unsigned nbytes);
```

`write` escribe `nbyte` bytes de la memoria referenciada por `buf` en el fichero asociado al descriptor `fildes`. Si la escritura se lleva a cabo correctamente, `write` devuelve el número de bytes realmente escritos; en caso contrario, devuelve `-1` y `errno` contendrá el tipo del error producido.

En los ficheros con capacidad de acceso aleatorio, la escritura se realiza en la posición indicada por el puntero de lectura/escritura del fichero. Después de la escritura, el puntero queda actualizado. En los ficheros sin capacidad de acceso aleatorio, la escritura siempre tiene efecto sobre la misma posición.

Si el indicador `O_APPEND` estaba presente al abrir el fichero, el puntero se situará al final del mismo para que las llamadas de escritura añadan información al fichero.

En los ficheros ordinarios, la escritura se realiza a través del *buffer caché*, por lo que una llamada a `write` no implica una actualización inmediata del disco. Este mecanismo acelera la gestión del disco, pero presenta problemas de cara a la consistencia de los datos. Si no ocurre ningún imprevisto, no hay nada que temer, pero en el caso de fallo no previsto —un corte de la alimentación del equipo, por ejemplo— es posible que se pierdan datos del *buffer caché* que no habían sido actualizados. Si al abrir el fichero estaba presente el indicador `O_SYNC`, forzamos que las llamadas a `write` no devuelvan el control hasta que se escriban los datos en el disco, asegurando así la consistencia. Naturalmente, este modo de trabajo está penalizado con un mayor tiempo de ejecución de nuestro proceso. Algunos ejemplos de uso de `write` son:

```
char *str = "En un lugar de la Mancha...";
int nbytes, fd;
...
nbytes = write (fd, str, strlen (str));
```

y para escritura con un formato concreto:

```
struct REGISTRO reg;  
int nbytes, fd;  
...  
nbytes = write (fd, &reg, sizeof (reg));
```

3.2.4 Cierre de un fichero (close)

Utilizaremos la llamada `close` para indicarle al núcleo que dejamos de trabajar con un fichero previamente abierto. El núcleo se encargará de liberar las estructuras que había montado para trabajar con el fichero. La declaración de `close` es:

```
#include <unistd.h>  
int close (int fildes);
```

Si `fildes` es un descriptor de fichero correcto devuelto por una llamada a `creat`, `open`, `dup`, `fcntl` o `pipe`, `close` cierra su fichero asociado y devuelve el valor 0; en caso contrario, devuelve -1 y `errno` contendrá el tipo de error producido. El único error que se puede producir en una llamada a `close` es que `fildes` no sea un descriptor válido.

Al cerrar un fichero, la entrada que ocupaba en la tabla de descriptores de ficheros del proceso queda libre para que la pueda utilizar una llamada posterior a `open`. Por otro lado, el núcleo analiza la entrada correspondiente en la tabla de ficheros del sistema y, si el contador que tiene asociado este fichero es 1 —esto quiere decir que no hay más procesos que estén unidos a esta entrada—, esa entrada también se libera.

Si un proceso no cierra los ficheros que tiene abiertos, al terminar su ejecución el núcleo analiza la tabla de descriptores y se encarga de cerrar los ficheros que aún estén abiertos.

3.2.5 Creación de un fichero (creat)

La llamada `creat` permite crear un fichero ordinario o reescribir sobre uno existente. Su declaración es:

```
#include <fcntl.h>  
int creat (char *path, mode_t mode);
```

`path` es un puntero al nombre del fichero que queremos crear.

`mode` es una máscara de bits con el mismo significado que vimos para la llamada `open`.

En esta máscara se especifican los permisos de lectura, escritura y ejecución para el propietario, grupo al que pertenece el propietario y el resto de los usuarios.

Si `creat` funciona correctamente, devuelve un descriptor de fichero y el fichero es abierto en modo *sólo escritura*, incluso si `mode` no permite este tipo de acceso. Si el fichero ya existe, su tamaño es truncado a 0 bytes y su puntero de escritura se sitúa al principio. Si la llamada a `creat` falla, por ejemplo, si no tenemos permiso para crear un fichero en el directorio en el que intentamos hacerlo, la función devolverá -1 y en `errno` estará el código del tipo de error producido.

La llamada a `creat` tiene la misma funcionalidad que una llamada a `open` con los indicadores `O_WRONLY | O_CREAT | O_TRUNC` activos. Así, las siguientes llamadas tienen la misma funcionalidad:

```
fd = creat ("mifichero", 0666);  
fd = open ("mifichero", O_WRONLY | O_CREAT | O_TRUNC, 0666);
```

3.2.6 Duplicado de un descriptor (`dup`)

La llamada `dup` duplica un descriptor de fichero que ya ha sido asignado y que está ocupando una entrada en la tabla de descriptores de fichero. Su declaración es:

```
#include <unistd.h>  
int dup (int fildes);
```

`fildes` es un descriptor obtenido a través de una llamada previa a `creat`, `open`, `dup`, `fcntl` o `pipe`.

La llamada a `dup` recorre la tabla de descriptores y va a marcar como ocupada la primera entrada que encuentre libre, pasando a devolvernos el descriptor asociado a esa entrada. Si falla en su ejecución, devolverá el valor `-1`, indicando a través de `errno` el error producido.

Los dos descriptores —original y duplicado— tienen en común que comparten el mismo fichero, por lo que a la hora de leer o escribir podemos usarlos indistintamente. Cuando estudiemos las tuberías sin nombre, veremos la utilidad de esta llamada.

3.2.7 Acceso aleatorio (`lseek`)

Con la llamada `lseek` podremos modificar el puntero de lectura/escritura de un fichero. Su declaración es la siguiente:

```
#include <sys/types.h>  
#include <unistd.h> // Para las constantes simbólicas.  
off_t lseek (int fildes, off_t offset, int whence);
```

`lseek` modifica el puntero de lectura/escritura del fichero asociado a `fildes` de la siguiente forma:

- Si `whence` vale `SEEK_SET`, el puntero avanza `offset` bytes con respecto al inicio del fichero.
- Si `whence` vale `SEEK_CUR`, el puntero avanza `offset` bytes con respecto a su posición actual.
- Si `whence` vale `SEEK_END`, el puntero avanza `offset` bytes con respecto al final del fichero.

Si `offset` es un número positivo, los avances deben entenderse en su sentido natural; es decir, desde el inicio del fichero hacia el final del mismo. Sin embargo, también se puede conseguir que el puntero retroceda pasándole a `lseek` un desplazamiento negativo.

Cuando `lseek` se ejecuta satisfactoriamente, devuelve un número entero no negativo, que es la nueva posición del puntero de lectura/escritura medida con respecto al principio del fichero. Si `lseek` falla, devuelve `-1` y en `errno` estará el código del error producido.

Hay que tener presente que en algunos ficheros no está permitido el acceso aleatorio y por lo tanto la llamada a `lseek` no tiene sentido. Ejemplos de estos ficheros son las tuberías con nombre y los ficheros de dispositivo en los que la lectura se realice siempre a través de un mismo registro o posición de memoria.

3.2.8 Consistencia de un fichero

La entrada/salida con el disco se realiza a través del *buffer caché* para agilizar la transferencia de datos. Sin embargo, hay aplicaciones cuyas especificaciones obligan a que se prescinda del *buffer caché* y que las escrituras en un fichero se reflejen de forma inmediata en el disco. UNIX ofrece dos soluciones para este tipo de aplicaciones. La primera consiste en abrir el fichero indicándole a `open` que toda operación posterior de escritura debe ir acompañada de una actualización en el disco. Esto se consigue pasándole a `open`, dependiendo del sistema, alguno de los indicadores `O_SYNCW`, `O_SYNC`, etc. Otra solución es hacer llamadas a `fsync` allí donde queremos asegurar que se produzca una escritura en el disco. La declaración de `fsync` es la siguiente:

```
#include <unistd.h>
int fsync (int fildes);
```

`fildes` es el descriptor del fichero que queremos sincronizar con el disco. La sincronización consiste en escribir en el disco todas las memorias intermedias asociadas al fichero.

3.3 Biblioteca estándar de funciones de entrada/salida

La biblioteca estándar de funciones de entrada/salida, que forma parte de la definición del C estándar ANSI, hace uso de las llamadas al sistema para presentarnos una interfaz de alto nivel que permite al programador trabajar con los ficheros desde un punto de vista más abstracto. Por otro lado, con esta biblioteca cada acceso al disco se va a gestionar de una forma más eficiente, ya que las funciones manejan memorias intermedias para almacenar los datos y se esperará a que estas memorias estén llenas antes de transferirlas al disco o al *buffer caché*.

3.3.1 Interfaz de la biblioteca estándar

Las funciones de la biblioteca referencian los ficheros mediante punteros a estructuras de tipo `FILE`. Tanto el tipo `FILE` como las definiciones prototipo se encuentran en el fichero de cabecera `<stdio.h>`. Las cuatro primeras funciones que vamos a estudiar son `fopen`, `fread`, `fwrite` y `fclose`.

Apertura (`fopen`)

Su declaración es:

```
#include <stdio.h>
FILE *fopen (const char *file_name, const char type);
```

Si todo funciona correctamente, `fopen` abre el fichero cuyo nombre está apuntado por `file_name` y le asocia un *flujo*¹, que no es más que una estructura de tipo `FILE`. La función, acto seguido, devuelve un puntero a este flujo. Si la llamada falla, `fopen` devuelve un puntero a `NULL` y `errno` tendrá el código del error producido.

`type` es una cadena de caracteres que le indica a `fopen` el modo de acceso al fichero. Entre otros, puede tomar los siguientes valores:

- "r" abrir para leer.
- "w" Abrir para escribir.
- "a" Abrir para escribir al final del fichero o crear el fichero, si no existe, para escribir en él.
- "r+" Abrir para actualizar el fichero —leer y escribir—.
- "w+" Abrir el fichero para leer y escribir, pero truncando primero su tamaño a 0 bytes. Si el fichero no existe, se crea para leer y escribir en él.

`FILE` es un tipo definido a partir de la estructura `_iobuf`:

```
typedef struct _iobuf {
    int __cnt; // Nro. de bytes que quedan en la memoria intermedia de E/S.
    unsigned char *__ptr; // Ptr. al sig. carácter de la memoria intermedia.
    unsigned char *__base; // Puntero a la memoria intermedia de E/S.
    short __flag; // Máscara con el modo de acceso al fichero y los errores
                  // que se producen al acceder a él.
    char __file; // Descriptor del fichero.
}FILE;
```

Los valores que puede tomar el campo `__flag` se definen a continuación:

```
#define _IOFBF      0000
#define _IOREAD     0001 Fichero abierto en modo lectura.
#define _IOWRT      0002 Fichero abierto en modo escritura.
#define _IONBF      0004 Fichero abierto sin memoria intermedia.
#define _IOMYBUF    0010 Modificación en el tamaño de la memoria intermedia.
#define _IOEOF      0020 Detectado final de fichero.
#define _IOERR      0040 Error de entrada/salida.
#define _IOLBF      0200 Memoria intermedia de salida gestionada línea a línea.
#define _IORW       0400 Fichero abierto en modo lectura/escritura.
```

Cada proceso que se ejecuta en UNIX y que está enlazado con la biblioteca estándar C, tiene asociado una tabla de flujos que se define de la siguiente forma:

```
FILE __iob[OPEN_MAX]; // Tabla de flujos
```

¹Algunos autores traducen *stream* por *corriente*, pero está más extendido el término *flujo*.

El tamaño de esta tabla depende de la configuración del sistema, y tiene tantas entradas como la tabla de descriptores de ficheros asociada al proceso.

Al igual que ocurre con la tabla de descriptores, las tres primeras entradas de la tabla de flujos están ocupadas por los ficheros estándar —entrada, salida y salida de errores—. Asociados con estos tres ficheros hay tres identificadores que se definen en `<stdio.h>` como sigue:

```
#define stdin (&__iob [0])    // Flujo para el fichero estándar de entrada,
#define stdout (&__iob [1])   // para el fichero estándar de salida y
#define stderr (&__iob [2])   // para el fichero estándar de salida de errores.
```

Para que las tres primeras entradas de `__iob` se correspondan con los ficheros estándar tal y como figuran en la tabla de descriptores, `__iob` debe ser correctamente inicializada. Esto se puede conseguir en el momento de declararla, ya que C permite inicializar las variables en el mismo instante en que se declaran. Así, la definición correcta de `__iob` es:

```
FILE __iob [OPEN_MAX] ={
    {0, (char *)0, (char *)0, _IOREAD, 0}, // stdin
    {0, (char *)0, (char *)0, _IOWRITE, 1}, // stdout
    {0, (char *)0, (char *)0, _IOWRITE | _IONBF, 2} // stderr
};
```

Sólo nos preocupamos de la inicialización de los tres primeros elementos de `__iob`. El resto de los elementos quedarán inicializados a 0, ya que `__iob` es una variable global y esa zona de memoria se inicializa a 0 por defecto².

Lectura de datos (fread)

Se utiliza para leer datos de un fichero a través de su flujo asociado. Su declaración es:

```
#include <stdio.h>
size_t fread (char *ptr, size_t size, size_t nitems, FILE *stream);
```

`fread` copia en el array apuntado por `ptr` un total de `nitems` bloques de datos procedentes del fichero apuntado por `stream`. Cada bloque, o ítem, de datos leído tiene un tamaño de `size` bytes.

`fread` termina su lectura cuando encuentra el final del fichero, se da una condición de error o ha leído el total de bloques que se le pide. Si la lectura se realiza correctamente, `fread` devuelve el número de bloques leídos. Si el valor devuelto es 0, significa que hemos encontrado el final del fichero.

Escritura de datos (fwrite)

`fwrite` permite escribir datos en un fichero a través de su flujo asociado. Su declaración es:

²En alguna ocasión he oído la rectificación *por omisión* para la expresión *por defecto*. Veamos qué dice la vigésima segunda edición del diccionario de la RAE al respecto: **defecto**.... ||2. loc. adv. *Inform.* Dicho de seleccionar una opción: Automáticamente si no se elige otra.

```
#include <stdio.h>
size_t fwrite (const char *ptr, size_t size, size_t nitems, FILE *stream);
```

`fwrite` copia en el fichero apuntado por `stream` el número de bloques indicado en `nitems`, cada uno de tamaño `size` bytes. Los bloques a escribir se encuentran en la zona de memoria apuntada por `ptr`.

Si la llamada se ejecuta correctamente, devuelve el total de bloques escritos. Si se da una condición de error, el número devuelto por `fwrite` será distinto del número de bloques que se le pasó como parámetro.

Cierre (`fclose`)

`fclose` cierra un fichero que ha sido abierto con `fopen`. Su declaración es:

```
#include <stdio.h>
int fclose (FILE *stream);
```

`fclose` hace que toda memoria intermedia de datos asociada a `stream` sea escrita en el disco, que el espacio de memoria reservado para las memorias intermedias sea liberado y que el flujo sea cerrado.

La operación de cerrado se realiza de forma automática sobre todos los ficheros abiertos cuando llamamos a la función `exit`.

`fclose` devuelve 0 si la llamada funciona correctamente y EOF —final de fichero— en caso de que se produzca algún error.

Como ejemplo de aplicación de estas cuatro funciones estándar, vamos a implementar una nueva versión del programa para copiar ficheros. Su listado se puede ver a continuación.

Programa 3.2: Copiar ficheros. Segunda versión (`fcopiar.c`)

```
/**
$ fcopiar origen destino
Retorna 0 si se ejecuta satisfactoriamente o -1 en caso contrario
***/
#include <stdio.h> /* Fichero de cabecera de la biblioteca estándar de E/S */

main (int argc, char *argv [])
{
    FILE *forigen, *fdestino;
    char c;

    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc != 3) {
        fprintf (stderr, "Forma de uso: %s origen destino\n", argv [0]);
        exit (-1);
    }

    /* Apertura del fichero origen en modo sólo lectura. */
    if ((forigen = fopen (argv [1], "r")) == NULL) {
```

```
        perror (argv [1]);
        exit (-1);
    }
    /* Apertura o creación del fichero destino en modo sólo escritura. */
    if ((fdestino = fopen (argv [2], "w")) == NULL) {
        perror (argv [2]);
        exit (-1);
    }
    /* Copiamos el fichero origen en el fichero destino. */
    while (fread (&c, sizeof c, 1, forigen) > 0)
        fwrite (&c, sizeof c, 1, fdestino);
    /* Cierre de ficheros. */
    fclose (forigen);
    fclose (fdestino);

    exit (0);
}
```

En este ejemplo, la lectura se realiza sobre una memoria intermedia de 1 byte; sin embargo, la función `fread` gestiona otra memoria intermedia interna para que la lectura se haga por bloques. El motivo por el que leemos sobre una memoria intermedia de un byte y no de tamaño `BUFSIZ` —como en el programa `copiar.c`—, es que `fread` no devuelve el total de bytes leídos, sino el de bloques. Si el tamaño de nuestro fichero no es múltiplo de `BUFSIZ`, al leer el último bloque `fread` devolverá 0 bloques leídos y, por lo tanto, este último bloque no será copiado.

3.3.2 Entrada/salida de caracteres con la biblioteca estándar

En la biblioteca estándar hay dos funciones para leer y escribir caracteres —bytes— sobre un fichero asociado a un flujo. Estas funciones son `fgetc` y `fputc`:

```
#include <stdio.h>
int fgetc (FILE *stream); // Lectura de caracteres.
int fputc (int c, FILE *stream); // Escritura de caracteres.
```

`fgetc` devuelve el carácter siguiente al último leído del fichero asociado a `stream`. Aunque `fgetc` lee caracteres —bytes— del fichero, devuelve un entero, con lo que se consiguen dos objetivos: el primero es que el byte leído se devuelve como un carácter sin signo, y el segundo, que cuando se detecta el final del fichero se puede devolver `EOF` (-1) sin que haya lugar a confundirlo con un dato válido.

En `fputc`, `c` es el byte que se quiere escribir en el fichero. Cuando se produce un error, `fputc` devuelve `EOF`. Si la escritura se produce correctamente, `fputc` devuelve el valor escrito.

Estas dos funciones tienen dos macros equivalentes³: `getc` y `putc`, que se sirven de la memoria intermedia asociada a `stream` y de las funciones `__fillbuf` y `__flsbuf` para agilizar el proceso de lectura/escritura.

Las funciones `getchar` y `putchar`, que actúan sobre la entrada estándar y la salida estándar, pueden codificarse como macros a partir de `getc` y `putc`:

```
#define getchar() getc (stdin)
#define putchar(c) putc ((c), stdout)
```

3.3.3 Implementación de la biblioteca estándar de entrada/salida

Una vez conocida la interfaz que tienen las funciones de la biblioteca estándar, vamos a implementarla ajustándonos lo máximo posible a su operatividad. Para que nuestras funciones no interfieran con las que ya existen, las nombraremos de igual forma que a las funciones estándar pero anteponiendo en carácter `m` a cada nombre —por ejemplo, `fopen` pasará a llamarse `mfopen`—.

La primera tarea a realizar es crear un nuevo fichero de cabecera que sustituya a `<stdio.h>`. Este fichero, cuyo listado se muestra a continuación, lo hemos llamado `mstdio.h` y residirá en nuestro directorio de trabajo.

Fichero 3.3: Fichero de cabecera para la biblioteca estándar de E/S (`mstdio.h`)

```
#ifndef _MSTDIO_
#define _MSTDIO_

#define NULL 0
#define EOF (-1)
/* Constantes del sistema. */
#define BUFSIZE 1024
#define OPEN_MAX 20

/* Definición de la estructura de un flujo. */
typedef struct _iobuf {
    int cnt; /* Caracteres que quedan en la memoria intermedia. */
    char *ptr; /* Puntero al carácter siguiente la memoria intermedia. */
```

³En [Kernighan & Pike, 2000, pág. 105] podemos encontrar el siguiente consejo respecto al diseño de interfaces: «Una interfaz debe ofrecer toda la funcionalidad necesaria, pero no más, y las funciones no deben traslapar excesivamente su capacidad. Tener muchas funciones puede facilitar el uso de la biblioteca —lo que uno requiere está allí disponible—. Sin embargo, una interfaz grande es más difícil de escribir y mantener; además, el tamaño puede dificultar su aprendizaje y uso[...] La biblioteca estándar de E/S de C ofrece al menos cuatro funciones diferentes para escribir un solo carácter en un flujo de salida:

```
char c;
putc(c, fp);
fputc(c, fp);
fprintf(fp, "%c", c);
fwrite(&c, sizeof(char), 1, fp);
```

Si el flujo es `stdout`, hay varias posibilidades más. Son adecuadas, pero no son todas necesarias. Las interfaces angostas son preferibles a las anchas, al menos hasta tener evidencia clara de que se requieren más funciones.»

```

    char *base; /* Puntero al inicio de la memoria intermedia. */
    int flag; /* Modo de acceso al fichero y estado del mismo. */
    int fd; /* Descriptor del fichero al que queremos acceder. */
}FILE;

/* Valores posibles del campo flag. */
enum _flags {
    _READ = 0x01, /* Fichero abierto en modo sólo lectura. */
    _WRITE = 0x02, /* Fichero abierto en modo sólo escritura. */
    _UNBUF = 0x04, /* Fichero sin memoria intermedia asociada. */
    _EOF = 0x08, /* Detectado final de fichero. */
    _ERR = 0x10, /* Detectado algún error de entrada/salida. */
};

/* Tabla de flujos. */
extern FILE _iob [];

/* Flujos estándar. */
#define stdin (&_iob[0])
#define stdout (&_iob[1])
#define stderr (&_iob[2])

/* Macros. */
/* Control del flujo. */
#define feof(p) (((p)->flag & _EOF) != 0) /* Detectado final de fichero. */
#define ferror(p) (((p)->flag & _ERR) != 0) /* Detectado algún error. */
#define fileno(p) ((p)->fd) /* Devuelve el descriptor del fichero. */

/* Entrada/salida de bytes sobre ficheros en general. */
#define getc(p) (--(p)->cnt >= 0 ? (unsigned char) *(p)->ptr++:\
    m_fillbuf(p))
#define putc(x,p) (--(p)->cnt >= 0 ? *(p)->ptr++ = (x) :\
    m_flushbuf ((x),(p)))

/* Entrada/salida de bytes sobre los ficheros asociados a la consola. */
#define getchar() getc(stdin)
#define putchar(x) putc((x), stdout)

/* Prototipo de las funciones públicas de esta biblioteca. */
FILE *mfopen ();

#endif

```

El código de esta biblioteca está distribuido entre los ficheros fuente: `mfopen.c`, `mfio.c` y `mfflush.c`. En el fichero `mfopen.c` están las funciones `mfopen` y `mfclose`. Su listado es:

Fichero 3.4: Biblioteca estándar de E/S: implementación de `mfdopen` y `mfclose` (`mfdopen.c`)

```

#include "mstdio.h"
#include <fcntl.h>

/* Tabla de flujos. Los tres primeros se inicializan. */
FILE _iob [OPEN_MAX] ={
    {0, (char *)0, (char *)0, _READ, 0}, /* stdin */
    {0, (char *)0, (char *)0, _WRITE, 1}, /* stdout */
    {0, (char *)0, (char *)0, _WRITE|_UNBUF, 2} /* stderr */
};

/* Permisos por defecto al crear un fichero: rw-rw-rw- */
#define PERMISOS 0666

/**
 * mfdopen: abre un fichero y devuelve un flujo.
 */
FILE *mfdopen (char *nombre, char *modo)
{
    int fd;
    FILE *fp;

    if (*modo != 'r' && *modo != 'w' && *modo != 'a')
        return NULL;

    /* Recorremos _iob buscando una entrada libre. */
    for (fp = _iob; fp < _iob + OPEN_MAX; fp++)
        if ((fp->flag & (_READ | _WRITE)) == 0)
            break; /* Se ha encontrado una entrada libre. */
    if (fp >= _iob + OPEN_MAX)
        return NULL; /* Llegamos al final de _iob sin encontrar entradas libres. */

    /* Proceso de apertura. */
    /* Apertura en modo sólo escritura */
    if (*modo == 'w') {
        if ((fd =open (nombre,
                      O_WRONLY|O_CREAT|O_TRUNC,
                      PERMISOS))== -1)
            return NULL;
    }
    /* Apertura en modo sólo escritura y añadiendo al final */
    else if (*modo == 'a') {
        if ((fd = open (nombre, O_WRONLY|O_CREAT, PERMISOS)) == -1)
            return NULL;
        lseek (fd, 0L, 2);
    }
}

```

```

/* Apertura en modo sólo lectura. */
else if (*modo == 'r') {
    if ((fd = open (nombre, O_RDONLY)) == -1)
        return NULL;
}
else
    return NULL;

/* Inicialización del flujo. */
fp->fd = fd;
fp->cnt = 0;
fp->base = NULL;
fp->flag = (*modo == 'w') ? _WRITE: _READ;

return fp;
};

/**
 mfclose: libera el flujo asociado a un fichero.
 Hace una llamada a mfflush para limpiar la memoria intermedia del flujo.
 ***/
mfclose (FILE *fp)
{
    if(fp) {
        if (fp->base) {
            /* Volcamos la memoria intermedia asociada al flujo. */
            mfflush (fp);
            /* Liberamos la memoria intermedia asociada al flujo. */
            free (fp->base);
        }
        fp->flag = 0;
        /* Cerramos el fichero asociado al flujo. */
        close (fp->fd);
        return 0;
    }
    else
        return EOF;
}

```

En el módulo `mfio.c` están las funciones `m_fillbuf` y `m_flushbuf`, que son invocadas por las macros `getc` y `putc` para gestionar la transferencia de datos entre la memoria intermedia asociada a cada flujo y el disco. Su listado se muestra a continuación.

Fichero 3.5: Biblioteca estándar de E/S: implementación de `m_fillbuf` y `m_flushbuf` (`mfio.c`)

```
#include "mstdio.h"
```

```

#include <fcntl.h>

/****
    m_fillbuf: llena la memoria intermedia asociada a un flujo y devuelve el primer
    byte leído.
****/
m_fillbuf (FILE *fp)
{
    int bufsize;

    /* Comprobamos si el fichero esta abierto en modo lectura. */
    if ((fp->flag & (_READ | _EOF | _ERR)) != _READ)
        return EOF;
    /* Calculamos el tamaño de la memoria intermedia. */
    bufsize = (fp->flag & _UNBUF) ? 1 : BUFSIZE;

    /* Comprobamos si hay una memoria intermedia asociada al flujo. */
    if (fp->base == NULL)
        if ((fp->base = (char *)malloc (bufsize)) == NULL)
            return EOF; /* No se le puede asignar una memoria intermedia. */
    fp->ptr = fp->base;

    /* Leemos un bloque, con un tamaño igual al de la memoria intermedia, del
    fichero asociado al flujo. Esta lectura se realiza con la llamada read. */
    fp->cnt = read (fp->fd, fp->ptr, bufsize);
    /* Comprobación de la lectura por si se ha producido algún error. */
    if (--fp->cnt < 0) {
        if (fp->cnt == -1)
            fp->flag |= _EOF;
        else
            fp->flag |= _ERR;
        fp->cnt = 0;
        return EOF;
    }

    /* Devolvemos el primer byte leído. */
    return (unsigned char) *fp->ptr++;
}

/****
    m_flushbuf: vacía la memoria intermedia asociada a un flujo y escribe en
    la memoria intermedia el byte que se le pasa.
****/
m_flushbuf (int x, FILE *fp)
{
    int bufsize, nbytes;

```

```

/* Comprobamos si el fichero está abierto en modo lectura. */
if ((fp->flag & (_WRITE | _EOF | _ERR)) != _WRITE)
    return EOF;
/* Calculamos el tamaño de la memoria intermedia. */
bufsize = (fp->flag & _UNBUF) ? 1 : BUFSIZE;

/* Comprobamos si hay una memoria intermedia asociada al flujo. */
if (fp->base == NULL) {
    if ((fp->base = (char *)malloc (bufsize)) == NULL)
        return EOF; /* No se le puede asignar una memoria intermedia. */
} else {
    /* Volcamos la memoria intermedia en el disco. */
    nbytes = write (fp->fd, fp->base, bufsize);
    /* Comprobamos los posibles errores de entrada/salida. */
    if (nbytes != bufsize) {
        if (nbytes == -1)
            fp->flag |= _ERR;
        else
            fp->flag |= _EOF;
        fp->ptr = fp->base;
        fp->cnt = bufsize;
        return EOF;
    }
}
fp->ptr = fp->base;
fp->cnt = bufsize - 1;
*fp->ptr++ = (unsigned char)x;

/* Devolvemos el carácter introducido en la memoria intermedia. */
return (unsigned char)x;
}

```

En el módulo `mfflush.c` está la función `mfflush`, con la que tendremos que simular el comportamiento de `fflush` para vaciar la memoria intermedia asociada a un flujo. `mfflush` es invocada por `mfclose` antes de cerrar el flujo asociado a un fichero. El listado de `mfflush.c` es el siguiente:

Fichero 3.6: Biblioteca estándar de E/S: implementación de `mfflush` (`mfflush.c`)

```

#include "mstdio.h"
#include <fcntl.h>

/**
    mfflush: vacía la memoria intermedia asociada a un flujo.
***/
mfflush (FILE *fp)

```

```

{
    int bufsize, nbytes, total;

    /* Comprobamos que no se han producido errores al acceder al fichero. */
    if (feof (fp) || ferror (fp))
        return EOF;
    /* Calculamos el tamaño de la memoria intermedia del flujo. */
    bufsize = (fp->flag & _UNBUF) ? 1 : BUFSIZE;

    /* Caso de lectura. */
    if (fp->flag & _READ) {
        /* Perdemos el contenido la memoria intermedia. */
        fp->cnt = 0;
        fp->ptr = fp->base;
        return 0;
    }
    /* Caso de escritura. */
    else if (fp->flag & _WRITE) {
        /* Volcamos el contenido de la memoria intermedia en el disco. */
        total = (int)fp->ptr - (int)fp->base;
        nbytes = write (fp->fd, fp->base, total);
        fp->ptr = fp->base;
        fp->cnt = bufsize;
        /* Revisamos errores. */
        if (nbytes != total) {
            if (nbytes == -1)
                fp->flag |= _ERR;
            else
                fp->flag |= _EOF;
            return EOF;
        }
    }

    return 0;
}

```

Estos tres módulos se archivan en la biblioteca `mstdio.a`, que podemos enlazar con programas que utilicen la interfaz alternativa de alto nivel que hemos diseñado. Como ejemplo de aplicación de esta interfaz, incluimos el listado de una tercera versión del programa para copiar ficheros. Este fichero es `mfcopiar.c`.

Programa 3.7: Copiar ficheros. Tercera versión (`mfcopiar.c`)

```

/****
$ mfcopiar origen destino
Retorna 0 si se ejecuta satisfactoriamente o -1 en caso contrario
****/

```

```
#include "mstdio.h" /* Fichero de cabecera de la biblioteca alternativa de E/S. */

main (int argc, char *argv [])
{
    FILE *forigen, *fdestino;
    char c;

    /* Revisión de los argumentos de la línea de órdenes. */
    if (argc != 3) {
        printf ("Forma de uso: %s origen destino\n", argv [0]);
        exit (-1);
    }
    /* Apertura del fichero origen en modo sólo lectura. */
    if ((forigen = mfopen (argv [1], "r")) == NULL) {
        perror (argv [1]);
        exit (-1);
    }
    /* Apertura o creación del fichero destino en modo sólo escritura. */
    if ((fdestino = mfopen (argv [2], "w")) == NULL) {
        perror (argv [2]);
        exit (-1);
    }
    /* Copiamos el fichero origen en el fichero destino. */
    c = getc (forigen);
    while (!feof (forigen)) {
        putc (c, fdestino);
        c = getc (forigen);
    }
    /* Cierre de ficheros. */
    mfclose (forigen);
    mfclose (fdestino);
    exit (0);
}
```

Por último utilizaremos el programa `make`⁴ para compilar esta aplicación y generar tanto la biblioteca como el fichero ejecutable. Este programa se sirve del fichero `makefile` para determinar cómo tiene que compilar los distintos módulos fuentes y cómo debe combinarlos. El fichero `makefile` empleado es el siguiente:

Fichero 3.8: Compilación de la biblioteca estándar de E/S (`makefile`)

```
# Ficheros que forman parte de la biblioteca mstdio.a
OBJETOS = mfopen.o mfio.o mfflush.o

# Creación de la biblioteca
```

⁴Si el lector no está familiarizado con la forma de trabajo del programa `make` puede consultar primero el Apéndice B. *Desarrollo de aplicaciones en el entorno UNIX.*

```

mstdio.a: makefile $(OBJETOS)
        ar -rv mstdio.a $(OBJETOS)
        make mfcopiar

# Creación del programa ejecutable mfcopiar
mfcopiar: makefile mfcopiar.o mstdio.a
        cc -o mfcopiar mfcopiar.o mstdio.a

# Dependencias entre los módulos fuente y los módulos cabecera
mfcopiar.o $(OBJETOS): mstdio.h

```

Para compilar esta aplicación basta con escribir:

\$ make

3.4 Control de ficheros abiertos (fcntl)

Con la llamada `fcntl` se puede controlar un fichero abierto mediante una llamada previa a `open`, `creat`, `dup`, la propia `fcntl` o `pipe`. Este control consiste en las posibilidades de cambiar los modos permitidos de acceso al fichero, y de bloquear el acceso a parte del mismo o a su totalidad. El bloqueo tiene especial importancia cuando varios procesos trabajan simultáneamente con un fichero, y es imprescindible que los accesos a determinados registros del mismo sean atómicos. Imaginemos el caso de dos procesos que acceden a una base de datos común, uno para actualizar sus registros y otro para leer esos mismos registros. Si no implementamos ningún mecanismo de sincronización, puede darse el caso de que el proceso lector lea una información parcialmente actualizada. Esto ocurrirá cuando el proceso que actualiza interrumpa al proceso lector en mitad de una operación de consulta de la base de datos. La declaración de `fcntl` es la siguiente:

```

#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>
int fcntl (int fildes, int cmd, union {int val; struct flock *lockdes;} arg);

```

`fildes` es el descriptor de un fichero previamente abierto, `arg` es un entero o un puntero, dependiendo del valor que tome `cmd`. Los siguientes son valores permitidos para `cmd`:

- **F_DUPFD**, la llamada devuelve el descriptor de un fichero que se encuentra libre en este instante y que reúne las siguientes características:
 - Es el menor descriptor de valor mayor o igual a `arg.val`.
 - Tiene asociado el mismo fichero que el descriptor `fildes`.
 - Tiene asociado el mismo puntero a fichero que `fildes`.
 - El modo del fichero referenciado por el nuevo descriptor es el mismo que el de `fildes`.

- Los indicadores de estado del fichero de ambos descriptores serán los mismos.
- El descriptor se heredará de padres a hijos en las llamadas a `exec`.
- `F_GETFD`, la función devuelve el valor del indicador *close-on-exec* asociado al descriptor `fildes`. Si este indicador está activo, el fichero no se cerrará después de ejecutar una llamada a `exec`. Del valor devuelto por `fcntl` sólo tendrá validez el bit menos significativo, que valdrá 0 si el indicador no está activo y 1 en caso contrario.
- `F_SETFD`, fija el indicador *close-on-exec* asociado a `fildes` de acuerdo con el bit menos significativo de `arg.val`. Si el bit está a 1, el indicador estará activo.
- `F_GETFL`, devuelve los indicadores de estado y modo de acceso del fichero referenciado por `fildes`: `O_RDONLY`, `O_WRONLY`, `O_RDWR`, `O_NDELAY`, `O_APPEND`, etc.
- `F_SETFL`, fija los indicadores de estado de `fildes` de acuerdo con el valor de `arg.val`.
- `F_GETLK`, devuelve el primer cerrojo que se encuentra bloqueando la región del fichero referenciado por `fildes` y descrito en la estructura de tipo `struct flock` apuntada por `arg.lockdes`. La información devuelta sobrescribe la información pasada a `fcntl` en `arg`. Si no se encuentra ningún cerrojo sobre esa región, la estructura es devuelta sin cambios, excepto en el campo `l_type`, donde se activa el bit `F_UNLCK`.
- `F_SETLK`, activa o desactiva un cerrojo sobre la región del fichero referenciado por `fildes` y descrita por la estructura de tipo `struct flock` referenciada por el argumento `arg.lockdes`. La orden `F_SETLK` se utiliza para establecer un cerrojo de lectura —`F_RDLCK`—, de escritura —`F_WRLCK`— o para eliminar uno existente —`F_UNLCK`—. Si no se puede fijar alguno de estos cerrojos, la función termina inmediatamente y devuelve el valor -1.
- `F_SETLKW`, esta orden es la misma que `F_SETLK`, con la diferencia de que si no se puede establecer algún cerrojo, porque lo impiden otros ya establecidos, el proceso se pondrá a dormir hasta que se den las condiciones que se lo permitan.

Un *cerrojo de lectura* indica que el proceso actual está leyendo del fichero, por lo que ningún otro proceso debe escribir en el área bloqueada. Puede haber varios cerrojos de lectura simultáneos sobre una misma región de un fichero.

Un *cerrojo de escritura* indica que el proceso actual está escribiendo en el fichero, por lo que ningún proceso debe leer o escribir del área bloqueada. Sólo puede haber un cerrojo de escritura sobre una misma área del fichero.

La estructura `struct flock` se define como sigue:

```
struct flock {
    short l_type; // Tipo de cerrojo:
                  // F_RDLCK - lectura
                  // F_WRLCK - escritura
                  // F_UNLCK - eliminar el cerrojo
    short l_whence; // Inicio de la región a bloquear:
                  // SEEK_SET - origen del fichero
```



```

        // SEEK_CUR — posición actual
        // SEEK_END — final del fichero
off_t l_start; // Posición relativa de inicio de la región a bloquear. Está
               // referida al punto indicado en l_whence
off_t l_len; // Longitud de la región a bloquear. Si vale 0, se bloquea
             // desde el punto indicado en l_start hasta el final del fichero
pid_t l_pid; // Identificador del proceso —PID— que tiene fijado el cerrojo.
             // Devuelto con la orden F_GETLK
long l_sysid; // Identificador del sistema que tiene fijado el cerrojo.
             // Devuelto con la orden F_GETLK
};

```

Los cerrojos fijados por un proceso sobre un fichero se borran cuando el proceso termina. Además, los cerrojos no son heredados por los procesos hijos tras la llamada a `fork`.

Si `fcntl` no se ejecuta satisfactoriamente, devuelve el valor `-1` y en `errno` estará codificado el tipo de error producido.

Como ejemplo, vamos a ver el empleo de `fcntl` para realizar bloqueos sobre un fichero y sincronizar el acceso al mismo por parte de dos procesos. El programa siguiente lee un número que se encuentra en un fichero y tras incrementarlo lo presenta por pantalla y lo vuelve a grabar en el disco. Esta operación la repite varias veces. La forma de invocar al programa es:

```
$ fcntl [opciones]
```

Si se invoca al programa sin opciones, la lectura/escritura se realizara sin bloquear. Si se le pasa al programa la opción `-b`, la lectura/escritura se realizara bloqueando. Para ejecutar n copias del programa y que accedan simultáneamente al mismo fichero, podemos escribir:

```
$ fcntl & fcntl &...fcntl &
```

Analizaremos después qué ocurre cuando son dos los procesos que acceden al fichero para realizar la misma operación. Dependiendo del tipo de acceso, con bloqueo o sin él, los resultados serán distintos. El listado del programa es el siguiente:

Programa 3.9: Uso de `fcntl` para realizar accesos con bloqueo a ficheros (`fcntl.c`)

```

#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#ifdef SEEK_SET
# define SEEK_SET 0
#endif

/* Macro para ver si dos cadenas de caracteres son iguales. */
#define EQ(str1, str2) (strcmp(str1, str2) == 0)

#define FICHERO "tmp"

```

```
/* Función que no realiza nada. */
sin_bloqueo () {}

/**
    con_bloqueo: bloquea y desbloquea, tanto en lectura como en escritura, la
    totalidad de un fichero dado.
***/
con_bloqueo (int fd, int orden)
{
    struct flock cerrojo;

    /* Inicializamos el cerrojo para que bloquee todo el fichero referenciado
    por fd. */
    cerrojo.l_type = orden;
    cerrojo.l_whence = SEEK_SET;
    cerrojo.l_start = 0;
    cerrojo.l_len = 0;
    /* Intentamos bloquear, si no se puede, el proceso se pone a dormir hasta que
    se pueda. */
    if (fcntl (fd, F_SETLKW, &cerrojo) == -1) {
        perror ("fcntl");
        exit (-1);
    }
}

main (int argc, char *argv [])
{
    int fd;
    int nro, i, j;
    int (*bloquear) (); /* Puntero a la función de bloqueo. */

    /* Análisis de parámetros de la línea de órdenes. */
    if (argc == 1)
        bloquear = sin_bloqueo;
    else if (argc == 2 && EQ(argv [1], "-b"))
        bloquear = con_bloqueo;
    else {
        fprintf (stderr, "Forma de uso: %s [-b]\n", argv [0]);
        exit (-1);
    }
    /* Apertura del fichero. */
    if ((fd = open (FICHERO, O_RDWR|O_CREAT, 0644)) == -1) {
        perror (FICHERO);
        exit (-1);
    }
}
```

```

/* Inicialización del contador a cero cuando se crea el fichero. */
if (read (fd, &nro, sizeof(nro)) != sizeof(nro)) {
    nro = 0;
    write (fd, &nro, sizeof(nro));
}
/* Bucle para contar. */
for (i = 0; i < 10; i++) {
    lseek (fd, 0L, SEEK_SET);
    /* Bloqueamos en escritura para que la segunda copia del programa no
    pueda leer el fichero. */
    (*bloquear) (fd, F_WRLCK);
    read (fd, &nro, sizeof(nro));
    ++nro;
    lseek (fd, 0L, SEEK_SET);
    /* Introducimos un retardo de 1 segundo entre la lectura y la escritura
    del número incrementado, así forzamos un cambio de contexto en el caso
    de que haya otros procesos solicitando la CPU. */
    sleep (1);
    write (fd, &nro, sizeof(nro));
    printf ("PID = %d, nro = %d\n", getpid (), nro);
    (*bloquear) (fd, F_UNLCK);
}
close (fd);
}

```

En la implementación del programa hemos empleado la función `getpid`, que devuelve el valor del identificador del proceso que la llama. Sobre esta función y el significado del *PID* insistiremos más adelante. La ejecución del programa arroja resultados como los siguientes:

```
$ fcntl & fcntl &
```

PID = 154, nro = 1	PID = 154, nro = 6
PID = 155, nro = 1	PID = 155, nro = 6
PID = 154, nro = 2	PID = 154, nro = 7
PID = 155, nro = 2	PID = 155, nro = 7
PID = 154, nro = 3	PID = 154, nro = 8
PID = 155, nro = 3	PID = 155, nro = 8
PID = 154, nro = 4	PID = 154, nro = 9
PID = 155, nro = 4	PID = 155, nro = 9
PID = 154, nro = 5	PID = 154, nro = 10
PID = 155, nro = 5	PID = 155, nro = 10

Como vemos, los procesos 154 y 155 presentan los mismos valores del contador. En este caso, la lectura y escritura en el fichero se realiza sin bloqueo, lo que ocasiona que cuando el proceso 155 lee el número, aún no haya sido actualizado por el proceso 154.

Si tanto la lectura como la escritura las realizamos con bloqueo, el resultado que se obtiene es el siguiente:

```
$ fcntl & fcntl &
```

PID = 154, nro = 1	PID = 154, nro = 11
PID = 155, nro = 2	PID = 155, nro = 12
PID = 154, nro = 3	PID = 154, nro = 13
PID = 155, nro = 4	PID = 155, nro = 14
PID = 154, nro = 5	PID = 154, nro = 15
PID = 155, nro = 6	PID = 155, nro = 16
PID = 154, nro = 7	PID = 154, nro = 16
PID = 155, nro = 8	PID = 155, nro = 18
PID = 154, nro = 9	PID = 154, nro = 19
PID = 155, nro = 10	PID = 155, nro = 20

En este caso, las acciones: leer el valor del número, incrementarlo y escribirlo en el fichero, se realizan de forma indivisible, por lo que cuando un proceso lee el número, lo va a encontrar actualizado por el último proceso que trabajó con él.

3.5 Administración de ficheros

Estudiaremos ahora una serie de llamadas al sistema que nos permitirán acceder y cambiar la información de tipo administrativo y estadístico de un fichero.

3.5.1 stat, lstat y fstat

Estas tres llamadas devuelven la información que se almacena en la tabla de nodos-i sobre el estado de un fichero concreto. La declaración de estas funciones es:

```
#include <sys/types.h>
#include <sys/stat.h>
int stat (char *path, struct stat *buf);
int lstat (int fildes, struct stat *buf);
int fstat (int fildes, struct stat *buf);
```

La diferencia entre **stat** y **fstat** es que la primera recibe como primer parámetro un puntero al nombre del fichero —**path**—, mientras que la segunda trabaja con un fichero ya abierto y le debemos pasar su descriptor —**fildes**—.

Ambas funciones devuelven, a través de la estructura apuntada por **buf**, la información estadística del fichero.

lstat trabaja de forma similar a **stat** excepto cuando el nombre del fichero corresponde a un enlace simbólico. En este caso, **lstat** devuelve la información correspondiente al fichero que sirve de enlace mientras que **stat** devuelve información correspondiente al fichero al cual apunta el enlace.

Si las llamadas se ejecutan correctamente, devuelven el valor 0; en caso contrario, devuelven -1 y en **errno** podremos consultar el tipo de error producido.

La información administrativa del fichero se almacena en una estructura de tipo `struct stat`. Este tipo está definido en el fichero de cabecera `<sys/stat.h>`. Según la versión de UNIX con la que trabajemos, la estructura `stat` tendrá unos campos u otros —consulte `stat(5)`—. A continuación se muestran algunos de los campos estándar de esta estructura junto con su tipo asociado:

- `dev_t st_dev`, número del dispositivo que contiene al nodo-*i*. Aquí están codificados el *minor number* y el *major number* del dispositivo.
- `ino_t st_ino`, número del nodo-*i*.
- `ushort st_mode`, 16 bits que codifican el modo del fichero —consulte la llamada `chmod` en el § 3.5.2 pág. 80—.
- `ushort st_nlink`, número de enlaces al fichero.
- `uid_t st_uid`, identificador de usuario —*UID*— del propietario del fichero.
- `gid_t st_gid`, identificador del grupo —*GID*— al que pertenece el propietario del fichero.
- `dev_t st_rdev`, identificador de dispositivo. Tiene significado únicamente para los ficheros especiales en modo carácter y en modo bloque.
- `off_t st_size`, tamaño, en bytes, del fichero.
- `time_t st_atime`, fecha del último acceso al fichero —lectura—.
- `time_t st_mtime`, fecha de la última modificación del fichero.
- `time_t st_ctime`, fecha del último cambio de la información administrativa del fichero, por ejemplo, cambio de propietario, permisos, etc. Todas las fechas se miden en segundos con respecto a una *Época* —origen de tiempos, consulte el § 7.7.1 pág. 281 para ver la definición de *Época* que se emplea en UNIX—.

Los tipos `dev_t`, `ino_t`, `ushort`, `uid_t`, `gid_t`, `off_t` y `time_t` están definidos en el fichero de cabecera `<sys/types.h>` y suelen ser alias de los tipos base `short`, `int` y `long`.

3.5.2 Modos de un fichero

Al estudiar la llamada `open` hemos visto que cada fichero tiene asociada una máscara de 9 bits que indica los permisos de lectura, escritura y ejecución que tienen el propietario, el grupo y el resto de usuarios sobre el fichero. En realidad, estos 9 bits forman parte de una máscara más amplia, conocida como *modo* del fichero. La máscara de modo se compone de 16 bits cuyo significado se muestra en la figura 3.1. En el fichero de cabecera `<sys/stat.h>` hay definidas unas constantes para acceder a los bits de modo y extraer su información. Estas constantes se muestran en la tabla 3.1.

Podemos usar estas constantes como filtros sobre el bit que nos interese. Hay que advertir que para determinar el tipo de un fichero se debe usar la máscara `S_IFMT`. Por ejemplo, si queremos saber si un fichero es un directorio o no, se debe usar una expresión como,

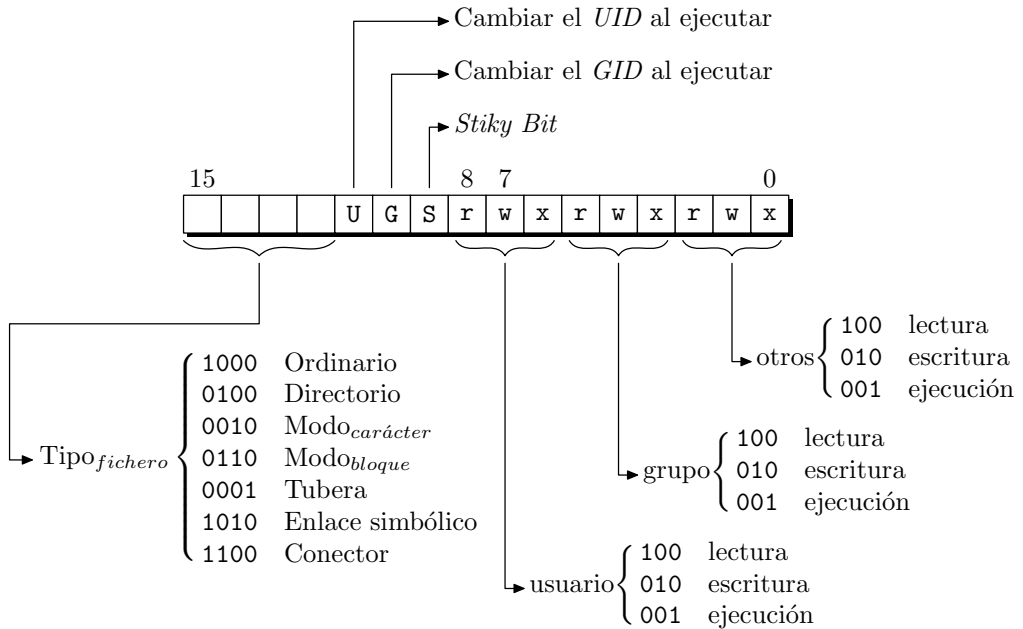


Figura 3.1: Máscara de modo de un fichero.

```
if ((mode & S_IFMT) == S_IFDIR)
```

porque si utilizamos,

```
if ((mode & S_IFDIR) == S_IFDIR)
```

nos dará también el valor lógico VERDAD cuando ese fichero sea de tipo especial modo bloque.

Hay tres bits cuyo significado no se ha definido de momento, son: **S_ISUID** —nº 11—, **S_ISGID** —nº 10— y **S_ISVTX** —nº 9—. Veamos qué significan:

- **S_ISUID** —cambiar el identificador del usuario en ejecución— le indica al núcleo que, cuando un proceso acceda a este fichero, cambie el identificador de usuario del proceso y le ponga el del fichero. Esto tiene aplicación cuando intentamos acceder a ficheros que son de otro usuario y no tenemos permiso para escribir en ellos. Como ejemplo vamos a mencionar la orden **passwd** —utilizada para cambiar la contraseña—. Al cambiar la contraseña, la nueva se debe guardar en el fichero **/etc/passwd**; sin embargo, este fichero sólo puede ser modificado por un usuario con privilegios de superusuario. Ante este impedimento, a un usuario normal le resultaría imposible cambiar de contraseña. No obstante, la realidad es distinta; como el programa **passwd** tiene activo el bit **S_ISUID**, al ejecutarlo nos convertimos momentáneamente en superusuarios, porque nuestro **UID** cambia y toma el valor del usuario **root** —propietario de **passwd**—, pudiendo así escribir sobre el fichero **/etc/passwd**, cuyo propietario es también **root**.

Tabla 3.1: Constantes definidas en `<sys/stat.h>` para el modo de un fichero.

Bits	Constante	Valor	Significado
15-12	S_IFMT	0170000	Tipo de fichero:
	S_IFREG	0100000	Ordinario
	S_IFDIR	040000	Directorio
	S_IFCHR	020000	Especial modo carácter
	S_IFBLK	060000	Especial modo bloque
	S_IFIFO	010000	Tubería
	S_IFSOCK	0140000	Conector
11	S_ISUID	04000	Activar ID del usuario al ejecutar
10	S_ISGID	02000	Activar ID del grupo al ejecutar
9	S_ISVTX	01000	<i>Sticky</i> bit
8-0			Bits de permisos

- **S_ISGID** —cambiar el identificador del grupo en ejecución— tiene un significado parecido al de **S_ISUID**, pero referido al grupo de usuarios al que pertenece el propietario del fichero. Así, cuando ejecutamos un programa que tiene activo este bit, nuestro GID —identificador de grupo— toma el valor del GID del propietario del programa.
- **S_ISVTX** —bit pertinaz, *sticky bit*— le indica al núcleo que este fichero es un programa con capacidad para que varios procesos compartan su segmento de código y que este segmento se debe mantener en memoria, aun cuando alguno de los procesos que lo utiliza deje de ejecutarse o pase al área de intercambio. La técnica de compartir el mismo código entre varios procesos permite gran ahorro de memoria en el caso de programas muy utilizados, como editores de texto, compiladores, etc.

Cambio de modo: `chmod` y `fchmod`

Las llamadas `chmod` y `fchmod` se utilizan para cambiar el modo de un fichero. Sus declaraciones son:

```
#include <sys/types>
#include <sys/stat.h>
int chmod (char *path, mode_t mode);
int fchmod (int fildes, mode_t mode);
```

Las dos llamadas cambian el modo de un fichero haciendo que tome el valor `mode`. En `chmod` especificamos el fichero por su ruta, `path`, y con `fchmod` actuamos sobre un fichero ya abierto y que tiene asociado el descriptor `fildes`. Si se ejecutan correctamente, estas funciones devuelven 0; en caso contrario, devuelven -1 y hacen que `errno` tome el código asociado al tipo de error producido.

Accesibilidad (access)

Determina la accesibilidad de un fichero por parte de un proceso. Su declaración es:

```
#include <unistd.h>
int access (char *path, int amode);
```

`path` es un puntero a la ruta del fichero al que queremos acceder, `amode` es una máscara que codifica el tipo de acceso por el que preguntamos. En `<unistd.h>` están definidos los siguientes valores para `amode`:

- `R_OK` permiso para leer.
- `W_OK` permiso para escribir.
- `X_OK` permiso para ejecutar —buscar en caso de directorios—.

En la llamada a `access` se emplean siempre el UID y el GID reales y no los efectivos. Si la petición de acceso se satisface, la función devuelve el valor 0, en caso contrario, devuelve -1 y `errno` contendrá el tipo de error producido.

Máscara de permisos (umask)

La usamos para definir la máscara de permisos que tendrá asociado un proceso a la hora de crear ficheros. Su declaración es:

```
#include <sys/types.h>
#include <sys/stat.h>
mode_t umask (mode_t cmask);
```

La nueva máscara por defecto se indica en `cmask` y `umask` devuelve el valor que tenía la máscara anterior. Se debe tener presente que los bits que estén activados —valor 1— en `cmask` se interpretarán como desactivados —valor 0— cuando creemos un fichero y, si las llamadas a `open` o `creat` intentan activar estos bits, no van a poder hacerlo. Así, por ejemplo, en la siguiente secuencia de código,

```
int fildes;
...
umask (0066);
fildes = creat ("mifichero", 0666);
```

`creat` intenta crear `mifichero` con permisos de lectura y escritura para el propietario, grupo y otros usuarios —`rw-rw-rw-`—, pero `umask` prohíbe expresamente la creación de ficheros con esos permisos, por lo que `mifichero` se crea con permiso 0600 —`rw----`—.

3.5.3 Cambio de la información estadística de un fichero

Hemos visto cómo cambiar el modo asociado a un fichero a través de la llamada `chmod`. Veamos ahora otras llamadas para cambiar parámetros tales como nombre del fichero, propietario, fechas y tamaño.

Cambio del nombre de un fichero (rename)

La declaración de `rename` es:

```
#include <stdio.h>
int rename (const char *source, const char *target);
```

El argumento `source` apunta al nombre inicial del fichero y `target` al nuevo nombre. En el caso de que `target` ya exista, es borrado previamente. `source` y `target` deben ser del mismo tipo y pueden ser ficheros ordinarios, directorios, ficheros especiales o tuberías con nombre.

Como siempre, si la llamada falla, devuelve `-1` y en caso de funcionar correctamente devuelve `0`.

Cambio del propietario y del grupo de un fichero: `chown` y `fchown`

Las llamadas `chown` y `fchown` sirven tanto para cambiar el identificador del propietario de un fichero como el identificador del grupo. Sus declaraciones son:

```
#include <sys/types.h>
int chown (char *path, uid_t owner, gid_t group);
int fchown (int fildes, uid_t owner, gid_t group);
```

La diferencia entre `chown` y `fchown` es que la primera trabaja con la ruta `—path—` de un fichero mientras que la segunda lo hace con el descriptor `—fildes—` de un fichero ya abierto.

`owner` debe ser el identificador de un usuario declarado en el sistema y `group` el identificador de un grupo. Si queremos dejar alguno de estos parámetros inalterados mientras cambiamos el otro, podemos usar las constantes `UID_NO_CHANGE` y `GID_NO_CHANGE` como parámetros `owner` y `group`, respectivamente.

Si la llamada funciona correctamente, devuelve `0`; en caso contrario, devuelve `-1` y en `errno` encontraremos el código del error producido.

Cambio de la fecha de un fichero (utime)

Para cambiar las fechas de último acceso y última modificación de un fichero tendremos que usar la llamada `utime`:

```
#include <sys/types.h>
#include <utime.h>
int utime (char *path, struct utimbuf *times);
```

`path` es el puntero al nombre del fichero cuyas fechas queremos cambiar, `times` es un puntero a una estructura de tipo `struct utimbuf` definida en `<utime.h>`. Si `times` vale `NULL`, las fechas de acceso y modificación toman el valor de la fecha y hora locales del ordenador —la hora actual—. Si un proceso tiene el mismo identificador de usuario que el fichero, o tiene permiso de escritura sobre el fichero, podrá modificar las fechas según este procedimiento. En el caso de que `times` no sea un puntero de valor `NULL`, las fechas se asignan de acuerdo con los campos de la estructura `struct utimbuf`,

```
struct utimbuf {
    time_t actime; // Fecha de acceso
    time_t modtime; // Fecha de modificación
};
```

Tanto `actime` como `modtime` se expresan en segundos con respecto a un origen de tiempos denominado *Época* —consulte el § 7.7.1 pág. 281—. Este procedimiento de cambio de fechas sólo lo pueden emplear el propietario del fichero y el superusuario.

Si la llamada a `utime` se ejecuta correctamente devuelve 0, en caso contrario, devuelve -1 y `errno` tendrá el código del tipo de error producido.

Longitud de un fichero: `truncate` y `ftruncate`

En el UNIX System V no es posible modificar el tamaño de un fichero salvo para reducirlo a 0 bytes o para incrementarlo añadiéndole bytes al final. En el sistema BSD se puede truncan la longitud de un fichero para que tome cualquier valor comprendido entre la longitud nula y la longitud actual del fichero. Con las llamadas `truncate` y `ftruncate` se puede realizar esta operación:

```
truncate (char *path, unsigned long length);
ftruncate (int fildes, unsigned long length);
```

`length` es la nueva longitud, en bytes, que tendrá el fichero. Mientras que `truncate` trabaja con un fichero especificado por su nombre —argumento `path`—, `ftruncate` lo hace con un fichero ya abierto en modo escritura y que tiene asociado `fildes` como descriptor.

Si las funciones se ejecutan correctamente, devuelven 0. En caso contrario devuelven -1, indicando el tipo de error a través de `errno`.

3.5.4 Ejemplo de utilización de la información estadística de un fichero

A continuación presentaremos un programa que se puede utilizar para mostrar por pantalla toda la información de un fichero a la que tenemos acceso con la llamada `stat`. Este programa se llama `estado` y la forma de usarlo es:

```
$ estado nombre_fichero
```

Algunos ejemplos de utilización son:

```
$ estado /dev/tty00
```

```
Fichero: /dev/tty00
Reside en el dispositivo: 2, 91
Nro. de nodo-i: 13446
Tipo: especial modo carácter.
Permisos: 0622 rw--w--w-
Enlaces: 1
User ID: 0; Nombre: root
Group ID: 1; Nombre: other
```

```

Números de dispositivo: 0, 4
Longitud: 0 bytes.
Último acceso: Fri Mar 27 16:32:08 1992
Última modificación: Fri Mar 27 16:35:54 1992
Último cambio de estado: Fri Mar 27 16:35:54 1992

```

```
$ estado /bin/passwd
```

```

Fichero: /bin/passwd
  Reside en el dispositivo: 2, 91
  Nro. de nodo-i: 33832
  Tipo: ordinario.
  Cambiar el ID del propietario en ejecución.
  Permisos: 0555 r-xr-xr-x
  Enlaces: 1
  User ID: 0; Nombre: root
  Group ID: 2; Nombre: bin
  Longitud: 75828 bytes.
  Último acceso: Wed Apr 29 16:30:30 1992
  Última modificación: Tue Jan 16 01:00:00 1990
  Último cambio de estado: Tue Mar 13 04:43:22 1990

```

El listado del programa es el siguiente:

Programa 3.10: Lectura y presentación del nodo-i de un fichero (estado.c)

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <pwd.h> /* Para getpwuid */
#include <grp.h> /* Para getgrid */

char permisos [] = {'x', 'w', 'r'};

main (int argc, char *argv [])
{
    int i;

    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc < 2) {
        fprintf (stderr, "Forma de uso: %s nombre_fichero.\n", argv [0]);
        exit (-1);
    }
    for (i = 1; i < argc; i++)
        estado (argv [i]);
    exit (0);
}

```

```
estado (char *fichero)
{
    struct stat buf;
    struct passwd *pw;
    struct group *gr;
    int i;

    if (stat (fichero, &buf) == -1) {
        perror (fichero);
        exit (-1);
    }

    printf ("Fichero: %s\n", fichero);
    printf (" Reside en el dispositivo: %d, %d\n",
            (buf.st_dev & 0xff00) >> 8, buf.st_dev & 0x00ff);
    printf (" Nro. de nodo-i: %d\n", buf.st_ino);
    printf (" Tipo: ");
    /* Análisis del tipo de dispositivo. */
    switch (buf.st_mode & S_IFMT) {
    case S_IFREG:
        printf ("ordinario.\n");
        break;
    case S_IFDIR:
        printf ("directorio.\n");
        break;
    case S_IFCHR:
        printf ("especial modo carácter.\n");
        break;
    case S_IFBLK:
        printf ("especial modo bloque.\n");
        break;
    case S_IFIFO:
        printf ("FIFO.\n");
        break;
    }
    if (buf.st_mode & S_ISUID)
        printf (" Cambiar el ID del propietario en ejecución.\n");
    if (buf.st_mode & S_ISGID)
        printf (" Cambiar el ID del grupo en ejecución.\n");
    if (buf.st_mode & S_ISVTX)
        printf (" Sticky bit activo.\n");

    /* Presentación de los permisos de lectura, escritura y ejecución; en la
    forma 0777 rwxrwxrwx */
    printf (" Permisos: 0%o ", buf.st_mode & 0777);
```

```

for (i = 0; i < 9; i++)
    if (buf.st_mode & (0400 >> i))
        printf ("%c", permisos [(8-i)%3]);
    else
        printf ("-");
printf ("\n");

/* Número de enlaces. */
printf (" Enlaces: %d\n", buf.st_nlink);

/* UID y GID. */
printf (" User ID: %d; Nombre: ", buf.st_uid);
if ((pw = getpwuid (buf.st_uid)) == NULL)
    printf ("???\n");
else
    printf ("%s\n", pw->pw_name);

printf (" Group ID: %d; Nombre: ", buf.st_gid);
if ((gr = getgrgid (buf.st_gid)) == NULL)
    printf ("???\n");
else
    printf ("%s\n", gr->gr_name);

/* Presentación de los números de dispositivo. */
switch (buf.st_mode & S_IFMT) {
case S_IFCHR:
case S_IFBLK:
    printf (" Números de dispositivo: %d, %d\n",
            (buf.st_rdev & 0xff00) >> 8, buf.st_rdev & 0x00ff);
}

/* Tamaño del fichero. */
printf (" Longitud: %d bytes.\n", buf.st_size);
/* Fechas del fichero. */
printf (" Último acceso: %s",
        asctime (localtime (&buf.st_atime)));
printf (" Última modificación: %s",
        asctime (localtime (&buf.st_mtime)));
printf (" Último cambio de estado: %s",
        asctime (localtime (&buf.st_ctime)));
}

```

En este programa hemos usado dos funciones de la biblioteca estándar que merecen ser comentadas con detalle. Se trata de `getpwuid` y `getgrgid`.

La función `getpwuid` se emplea para leer la información relativa al propietario del fichero; esta información se encuentra en el fichero `/etc/passwd`:

```
#include <pwd.h>
struct passwd *getpwuid (uid_t uid);
```

Si el usuario cuyo identificador indica `uid` existe, la función devuelve un puntero a una estructura de tipo `struct passwd` que contiene información más detallada sobre el usuario. De cara al programa, interesa el campo `pw_name` de la estructura `passwd`, ya que contiene el nombre del usuario. Para más información sobre esta función, podemos consultar la página `getpwent` del manual de UNIX.

La función `getgrgid` la empleamos para buscar en el fichero `/etc/group` más información sobre el grupo al que pertenece el propietario del fichero:

```
#include <grp.h>
struct group *getgrgid (gid_t gid);
```

Si el grupo indicado en `gid` existe, la función devuelve un puntero a una estructura de tipo `struct group` que contiene, entre otros, el campo `gr_name` que da el nombre del grupo. Para más información sobre esta función, podemos consultar `getgrent(3C)`.

3.6 Compartición y bloqueo de ficheros

En el § 3.4 pág. 72 hemos visto cómo la llamada `fcntl` puede servirnos para bloquear el acceso a la totalidad o parte de un fichero. El bloqueo se hace especialmente útil cuando varios procesos usan concurrentemente un mismo fichero.

A la hora de bloquear un recurso —ya sea este recurso un fichero o de otra naturaleza— podemos distinguir dos implementaciones o formas de trabajo distintas. Por un lado tenemos el bloqueo consultivo y, por otro, el bloqueo obligatorio.

Mediante el *bloqueo consultivo*, el sistema operativo conoce en cada momento qué recursos se encuentran bloqueados y qué procesos los bloquean, pero no prohíbe a ningún otro proceso que haga uso de esos recursos. Las medidas que se deben tomar son consultar el estado del recurso antes de usarlo y, si se encuentra libre, trabajar con él; en caso contrario habrá que esperar a que quede libre. La decisión de usarlo o no depende del usuario, ya que el sistema operativo se limita solamente a informar del estado del recurso. Este tipo de bloqueo es adecuado para procesos cooperativos. Estos son procesos diseñados para consultar el estado del recurso que comparten y esperar que quede libre. La inclusión de un proceso que no respete estas reglas puede tener resultados desagradables para todos los procesos que trabajan con un recurso compartido.

Con el *bloqueo obligatorio*, el sistema operativo comprueba cada uno de los accesos al recurso compartido con objeto de denegarle el acceso a aquellos procesos no autorizados en ese instante. Con este tipo de bloqueo no es necesario consultar el estado del recurso, ya que aunque intentemos acceder indebidamente, el sistema lo va a impedir.

En UNIX System V versión 3 están presentes los dos tipos de bloqueo para ficheros. La llamada `lockf` está diseñada para bloquear la totalidad o parte de un fichero, tanto en bloqueo consultivo como obligatorio. La declaración de esta llamada es:

```
#include <unistd.h>
int lockf (int fildes, int function, long size);
```

`lockf` bloquea la región deseada del fichero con descriptor `fd`, impidiendo que otros procesos accedan a esa región y puede bloquearse más de una región de un fichero. Cuando un proceso termina su ejecución o cierra alguno de los ficheros que tiene bloqueados, se borran todos los cerrojos que tenía definidos sobre ese fichero.

Para definir cerrojos sobre un fichero es necesario abrirlo en modo *sólo escritura o lectura/escritura*. El parámetro `function` especifica el tipo de acción para definir el cerrojo. Sus posibles valores son:

- `F_ULOCK`, desbloquear una región previamente bloqueada.
- `F_LOCK`, bloquear una región para uso exclusivo del proceso que invoca a `lockf`. Si la región no está disponible, porque ha sido bloqueada por otro proceso, el proceso actual se pondrá a dormir hasta que la región esté disponible.
- `F_TLOCK`, comprobar si la región a bloquear está disponible; si lo está, bloquear la región para uso exclusivo del proceso que llama a `lockf`. Si la región no está disponible, `lockf` devuelve `-1` y en `errno` estará el código del error producido.
- `F_TEST`, comprobar si la región especificada está bloqueada por otro proceso o no. Si la región está accesible, `lockf` devolverá el valor `0`; en caso contrario, devolverá `-1` y en `errno` el código `EAGAIN`⁵.

`size` es el total de bytes contiguos que se van a bloquear o desbloquear. El segmento a bloquear empieza en la posición actual del puntero de lectura/escritura y ocupa tantos bytes como indique `size`. Este parámetro puede tomar un valor negativo, con lo que el segmento a bloquear es el situado antes del puntero de lectura/escritura, sin que éste quede incluido. Si `size` vale `0`, el segmento bloqueado se extiende hasta el final de fichero, incluso aunque el fichero crezca de tamaño. Esto quiere decir que los bytes que se añadan en el futuro también van a estar bloqueados.

Conviene destacar que la diferencia entre los modos de bloqueo `F_LOCK` y `F_TLOCK` es la actuación a seguir en el caso de que el recurso a bloquear ya esté ocupado. Con `F_LOCK` el proceso dormirá hasta que el recurso quede libre, con `F_TLOCK` la llamada a `lockf` falla y devuelve `-1`. El modo `F_TLOCK` es equivalente en su funcionamiento al de la instrucción *test-and-set*. Lo importante es que las operaciones de comprobar y bloquear se realizan de forma atómica. Imaginemos que no disponemos de esta opción; la forma de comprobar y bloquear, en caso de que el fichero esté libre, sería la mostrada a continuación:

```
if (lockf (fd, F_TEST, size) == 0) // Comprobación
    lockf (fd, F_LOCK, size); // Bloqueo
```

pero la operación anterior no es indivisible, porque después de la comprobación, el planificador puede cambiar de contexto y pasarle el control a otro proceso que bloquee el fichero, ocasionando que falle la segunda llamada a `lockf`. Este tipo de errores se soluciona empleando el modo `F_TLOCK`. A los cerrojos fijados con `F_TLOCK` se les conoce como *cerrojos no bloqueantes*, puesto que si el proceso que lo utiliza no puede seguir adelante con el bloqueo, no se queda durmiendo en espera de poder fijar el cerrojo. Ejemplo de

⁵Algunos sistemas advierten que `errno` contendrá el valor `EACCES`, pero que en el futuro contendrá `EAGAIN`. Por ello, para escribir software transportable a futuras versiones del mismo sistema, hay que contemplar los dos posibles valores de `errno`.

programas que hacen uso de este tipo de cerrojos son aquellos que impiden que haya más de una copia de sí mismos en memoria. Estos programas declaran un cerrojo no bloqueante sobre un fichero y si la declaración falla es porque ya hay una copia cargada en memoria; en caso contrario, se carga la primera copia y el fichero queda bloqueado impidiendo la ejecución de futuras copias. Los demonios que dan servicios de red, impresora, sincronismo con el disco, etc., son programas de este tipo.

Existe el peligro de que el uso de bloqueos nos lleve a la situación indeseada de *abrazo mortal*. Pensemos por un momento en dos procesos que tienen bloqueados sendos ficheros, y que cada uno de ellos está esperando a que el otro desbloquee su fichero para continuar. En el escenario descrito, ambos procesos dormirán eternamente. Para prevenir esta situación, el sistema revisa las llamadas a `fcntl` y `lockf`, y en caso de que se dé un abrazo mortal, las llamadas fallarán, devolviendo el valor `-1` y haciendo que `errno` valga `EDEADLK`.

En el ejemplo siguiente se muestran los dos modos de bloqueo que permite `lockf`. El programa está implementado para bloquear la totalidad de un fichero durante 30 segundos. La primera vez que se ejecuta, el bloqueo se efectúa sin problemas. Si mientras el fichero está bloqueado intentamos bloquearlo con el mismo programa, veremos que el bloqueo no se puede efectuar, y dependiendo del tipo de bloqueo que utilicemos, sin comprobación previa o con ella, el programa se pondrá a dormir o nos devolverá el control. En el caso de que duerma, el programa despertará cuando el fichero quede desbloqueado y pasará inmediatamente a bloquearlo.

Programa 3.11: Bloqueo de ficheros, con comprobación previa y sin ella (`bloquear.c`)

```

/****
$ bloquear opciones fichero
Opciones:
    -a Intenta un bloqueo de tipo consultivo. El programa intenta bloquear
        el fichero; si no puede, recupera el control y termina su ejecución.
    -m Intenta un bloqueo de tipo obligatorio. El programa intenta bloquear
        el fichero; si no puede, se pone a dormir hasta que se restauren las
        condiciones que le permitan bloquear el fichero.

****/
#include <stdio.h>
#include <sys/types.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define EQ(str1, str2) (strcmp((str1), (str2)) == 0)

main (int argc, char *argv [])
{
    int fd;

    /* Análisis de los parámetros de la línea de órdenes. */
    if (argc != 3) {

```



```

        fprintf (stderr, "Forma de uso: %s modo fichero\n",
                    argv [0]);
        fprintf (stderr, " modo: -a [consultivo - advisory]\n");
        fprintf (stderr, " -m [obligatorio - mandatory]\n");
        exit (-1);
    }
    /* Apertura del fichero. */
    if ((fd = open (argv[2], O_RDWR|O_CREAT, 0744)) == -1) {
        perror (argv[2]);
        exit (-1);
    }

    /* Bloqueamos el fichero. */
    if (EQ (argv[1], "-a")) {
        if (lockf (fd, F_TLOCK, 0L) == -1) {
            fprintf (stderr,
                    "El fichero [%s] está bloqueado por otro proceso.\n",
                    argv[2]);
            exit (-1);
        }
    } else if (EQ (argv[1], "-m"))
        if (lockf (fd, F_LOCK, 0L) == -1) {
            perror (argv[2]);
            exit (-1);
        }

    /* El proceso duerme durante 30 segundos. */
    printf ("PID: %d. El fichero [%s] estará bloqueado\n \
        durante 30 segundos.\n", getpid (), argv[2]);
    sleep (30);

    /* Desbloqueamos el fichero. */
    if (lockf (fd, F_ULOCK, 0L) == -1) {
        perror ("lockf");
        exit (-1);
    } else
        printf ("\007PID: %d. El fichero [%s] ha sido desbloqueado.\n",
            getpid (), argv[2]);

    close (fd);
}

```

Para ver qué ocurre cuando ejecutamos el programa en sus diferentes modos, tenemos que ejecutar varias copias del mismo en segundo plano:

```
$ bloquear -a tmp &
```

PID:89. El fichero [tmp] estará bloqueado durante 30 segundos

```
$ bloquear -a tmp
```

El fichero [tmp] está bloqueado por otro proceso

```
$ bloquear -m tmp
```

La segunda vez que ejecutamos `bloquear`, éste devuelve el control, ya que se ha empleado el modo `F_TLOCK` y el fichero `tmp` estaba bloqueado. La tercera vez, el proceso se pone a dormir hasta que el fichero quede desbloqueado. En esta ocasión hemos usado el modo `F_LOCK`.

3.6.1 Implementación de `lockf` a partir de `open`

Aunque el documento *SVID*⁶ incluye la definición de `lockf` como una función estándar del conjunto de llamadas al sistema UNIX System V, no todas las versiones de UNIX poseen llamadas para bloquear ficheros. Algunos sistemas brindan funciones propias de bloqueo, pero no es aconsejable utilizarlas, porque se perdería la transportabilidad del código fuente.

Versiones anteriores de UNIX implementaban las técnicas de bloqueo haciendo uso de las propiedades de las llamadas `creat` y `open`. Para ello, a la hora de bloquear un fichero se recurre a crear un fichero auxiliar. Sabemos que la llamada `creat` falla si se intenta crear un fichero que ya existe, e igual ocurre cuando se invoca a la llamada `open` con los parámetros `O_CREAT|O_EXCL`. Además, en ambas llamadas, la comprobación de la existencia de un fichero y su posterior creación constituyen una operación indivisible. Podemos así implementar un mecanismo de bloqueo consistente en intentar crear un fichero auxiliar, de tal forma que si las llamadas a `creat` u `open` fallan lo tomaremos como indicador de que el fichero ha sido bloqueado por otro proceso y, si no fallan, el fichero quedará bloqueado por nuestro proceso. Para liberar el fichero basta con borrar el fichero auxiliar.

En el módulo siguiente se implementa una versión simplificada de la función `lockf`, construida a partir de las funciones `open` y `unlink`. El nombre del fichero auxiliar se construye respondiendo al esquema `mlockf#`, donde `#` es una cadena de dígitos construida a partir del número de nodo-`i` del fichero. Con esto se pretende que no se repitan los nombres de los ficheros auxiliares asociados a dos ficheros distintos. El código del módulo es el siguiente:

Fichero 3.12: Implementación de una versión simplificada de `lockf` (`lockf.c`)

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <errno.h>
```

```
/* No se incluye <unistd.h> para no generar un conflicto con el prototipo
```

⁶*System V Interface Definition.*

de `lockf` definido en ese fichero de cabecera, por eso es necesario que definamos aquí los valores de las constantes asociadas a los modos de bloqueo. Las siguientes cuatro definiciones han sido copiadas literalmente del fichero `<unistd.h>`. */

```
#define F_ULOCK 0
#define F_LOCK 1
#define F_TLOCK 2
#define F_TEST 3
```

```
extern int errno;
```

```
/**
```

`lockf`: función con una operatividad similar a la función estándar del mismo nombre. A diferencia de ésta, el bloqueo se implementa sobre la totalidad del fichero sin posibilidad de bloquear segmentos.

Parámetros:

`fil-des`: descriptor de un fichero ya abierto.

`function`: tipo de cerrojo a establecer.

`F_ULOCK` Desbloquear.

`F_LOCK` Bloquear todo el fichero. Si el fichero ha sido bloqueado por otro proceso, esperar a que quede libre.

`F_TLOCK` Comprobar y bloquear. Si el fichero ya ha sido bloqueado, la función devuelve el valor -1.

```
*/
```

```
lockf (int fil-des, int function)
```

```
{
```

```
    char faux [256];
```

```
    struct stat buf;
```

```
    char aux [2];
```

```
    int fdiaux, i;
```

```
    static int debug = 1; /* Variable usada para informar que lockf ha sido
                           escrita por el usuario. */
```

```
    if (debug) {
```

```
        printf ("Función lockf implementada por el usuario en %s.\n",
                __FILE__);
```

```
        debug = 0;
```

```
    }
```

```
    if (fstat (fil-des, &buf) == -1)
```

```
        return (-1);
```

```
    else { /* Generación del nombre del fichero auxiliar. */
```

```
        faux [0] = 0;
```

```
        strcat (faux, "mlockf");
```

```
        i = buf.st_ino;
```

```
        aux [1] = 0;
```

```
        /* Conversión del número de nodo—i a cadena de caracteres. */
```

```
        while (i/10) {
```

```

        aux [0] = i%10 + '0';
        strcat (faux, aux);
        i /= 10;
    }
    aux [0] = i%10 + '0';
    strcat (faux, aux);
}

switch (function) {
case F_TEST:
    if ((fdaux = open (faux, O_RDWR)) != -1) {
        /* El fichero auxiliar existe y por tanto el original está
        bloqueado. */
        close (fildes);
        errno = EAGAIN;
        return (-1);
    }
    break;
case F_TLOCK:
    if ((fdaux = open (faux, O_RDWR|O_CREAT|O_EXCL, 0600)) == -1) {
        /* El fichero auxiliar existe y por tanto el original se encuentra
        bloqueado. */
        errno = EAGAIN;
        return (-1);
    }
    break;
case F_LOCK:
    while ((fdaux = open (faux, O_RDWR|O_CREAT|O_EXCL, 0600)) == -1)
        sleep (1);
    break;
case F_ULOCK:
    return (unlink (faux));
default:
    errno = EINVAL;
    return (-1);
}
return 0;
}

```

Para enlazar el programa **bloquear** desarrollado anteriormente con la nueva función **lockf**, podemos emplear la orden siguiente:

```
$ cc -o bloquear bloaquear.c lockf.c
```

Hay que decir que la versión de **lockf** que se ha implementado es compatible con todas las versiones de UNIX que admitan la llamada a **open** con los parámetros **O_CREAT|O_EXCL**. Si nuestro sistema no la admite, puede sustituirse por una llamada a **creat**. Por otro lado, hay que señalar algunos inconvenientes que presenta esta versión de **lockf**:

1. Cuando se produce una caída del sistema, los ficheros auxiliares que indican bloqueo no se borran, por lo que hay que borrarlos manualmente cuando se restaura el sistema. Una posible solución es situar estos ficheros en el directorio `/usr/tmp` para que no estén distribuidos aleatoriamente y evitar así que ensucien la jerarquía de directorios.
2. Cuando el fichero está bloqueado, si un segundo proceso quiere bloquearlo en modo `F_LOCK` deberá esperar a que quede libre. En principio no sabemos cuánto tiempo esperar. En el ejemplo se ha programado 1 segundo de espera entre cada intento, pero la mejor solución consiste en poner a dormir al proceso hasta que algún mecanismo le notifique que despierte.
3. El problema más grave consiste en que el proceso que tiene bloqueado un fichero puede terminar su ejecución sin desbloquearlo, con lo que el fichero quedará permanentemente bloqueado a menos que se borre manualmente su fichero auxiliar. Una posible solución a este problema consiste en escribir en el fichero auxiliar el *PID* del proceso que bloquea, de tal forma que, cuando otro proceso intente bloquearlo y no pueda, consulte en el fichero auxiliar cuál es el proceso que se lo impide y, mediante una llamada a `kill` —consulte `kill(2)` y el § 7.3.1 pág. 243—, determinar si ese proceso existe. En el caso de que no exista, será porque ha terminado sin desbloquear adecuadamente y podremos pasar a desbloquear por nuestra cuenta.

Debido a los problemas que presenta implementar una función satisfactoria de bloqueo, es aconsejable utilizar la que suministra el sistema.

3.7 Ejercicios

- 3.1** Escriba un programa para copiar ficheros. El programa se debe llamar `copiar1` y su forma de uso será:

```
$ copiar1 fichero_origen fichero_destino
```

En el caso de que no se puedan copiar los ficheros, el programa le devolverá al sistema operativo los códigos de error siguientes:

- 1 – no se ha especificado fichero origen ni fichero destino
- 2 – no se ha especificado el fichero destino
- 3 – el fichero origen no existe o no se puede abrir
- 4 – el fichero destino no existe o no se puede abrir.

En el caso de que se produzca alguno de estos errores se debe terminar la ejecución del programa y devolver el control al sistema operativo. Se imprimirá también un mensaje de error usando la función `error_fatal` implementada en el ejercicio 1.3.

Si el programa funciona correctamente devolverá el código 0.

Para escribir este programa se deben usar las llamadas al sistema: `open`, `read`, `write...`; y no la biblioteca estándar de entrada/salida: `fopen`, `fread`, `fwrite...`

3.2 Cuando copiamos ficheros con el programa `copiar1` ¿qué podemos decir con respecto a los permisos de los ficheros `origen` y `destino`, ¿coinciden o son distintos?

3.3 Ejecute las órdenes siguientes:

```
$ copiar1
```

```
$ copiar1 /usr/bin/cc
```

```
$ copiar1 /usr/bin/cc $HOME
```

```
$ copiar1 /usr/bin/cc $HOME/compilador
```

¿Cuánto vale la variable de entorno ? después de ejecutar cada una de las órdenes anteriores?

¿Se ejecuta correctamente la orden tercera? En caso negativo, ¿por qué?

3.4 Escriba un programa para unir ficheros. El nombre de este programa debe ser `fusionar` y la forma de utilizarlo se muestra en los ejemplos siguientes:

- Unir dos ficheros y crear un tercer fichero suma de los dos anteriores:

```
$ fusionar f1 f2 fusión
```

- Unir todos los ficheros que tengan extensión `.c` y almacenar el resultado en `programas`:

```
$ fusionar *.c programas
```

- Unir todos los ficheros que empiecen por `ej_` y almacenar el resultado en `ejercicios`:

```
$ fusionar ej_* ejercicios
```

3.5 Queremos escribir un programa para ordenar ficheros binarios indexados que contienen información sobre los empleados de una empresa determinada. La estructura de estos ficheros viene descrita en el fichero de cabecera siguiente:

```
// FICHERO: empleados.h
#if !defined(_EMPLEADOS_H_)
#define _EMPLEADOS_H_
```

```
#define MAX 61
```

```
// EMPLEADO: en el fichero de empleados hay un registro por cada empleado.
```

```
typedef struct {
    char nombre [MAX]; // Nombre del empleado
    float sueldo; // Sueldo del empleado
}EMPLEADO;
```

```
// Estructura que figura al final del fichero de empleados.
```

```

typedef struct {
    long nro_índices; // Total de registros de tipo EMPLEADO
    long *índices; // Puntero a los índices que indican el orden en que deben
                  // leerse los registros del fichero. En el pie del fichero habrá tantos
                  // índices como registros tenga el fichero.
}PIE;

// Estructura que se debe crear en memoria cuando se abra un fichero
typedef struct {
    int descriptor; // Puntero a la estructura de control del fichero.
    PIE pie; // Pie del fichero. Se debe leer del fichero referenciado por el
            // campo descriptor anterior
}FICHERO;
#endif

```

Para explicar mejor la estructura de este fichero analicemos el ejemplo siguiente. Supongamos que una empresa tiene cuatro empleados:

```

Tirante el Blanco, 23.000 reales
Felixmarte de Hircania, 21.500 reales
Palmerín de Oliva, 19.300 reales
Olivante de Laura, 35.000 reales

```

Esta información puede aparecer en el fichero distribuida según se muestra en la figura 3.2.

Obsérvese cómo al recorrer los índices del pie del fichero antes de ser ordenado, la secuencia que se obtiene es:

```

Tirante el Blanco, 23.000 reales
Palmerín de Oliva, 19.300 reales
Olivante de Laura, 35.000 reales
Felixmarte de Hircania, 21.500 reales

```

Sin embargo, al recorrer los índices del fichero ordenado, la secuencia que obtenemos es la siguiente:

```

Felixmarte de Hircania, 21.500 reales
Olivante de Laura, 35.000 reales
Palmerín de Oliva, 19.300 reales
Tirante el Blanco, 23.000 reales

```

Por lo tanto, el proceso de ordenación consiste en cambiar los índices que figuran en el pie del fichero. Los registros de datos no sufren ninguna alteración.

Para escribir el programa se aconseja realizar los trabajos siguientes:

- (a) Escribir una función con la declaración siguiente:

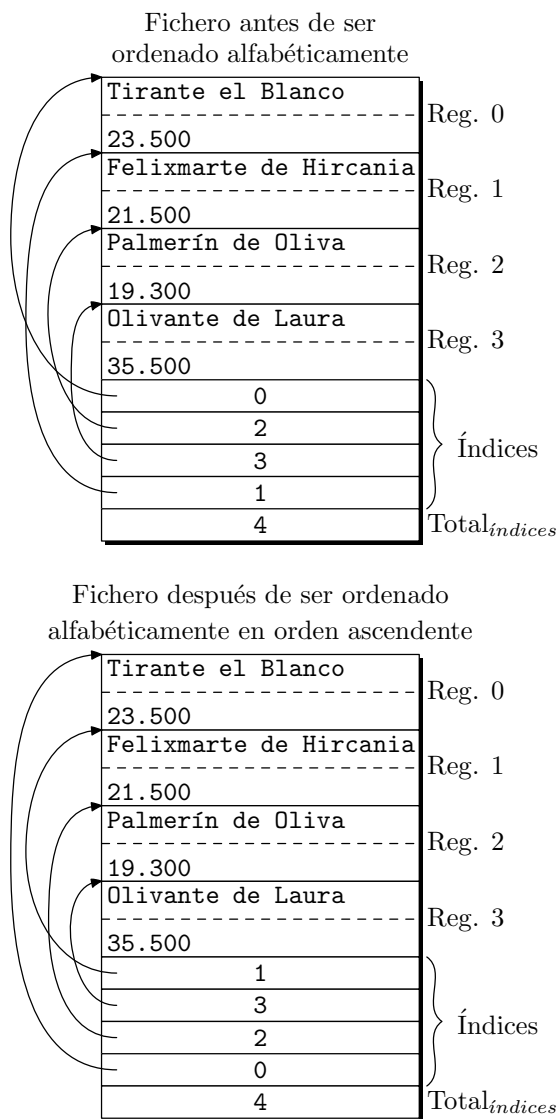


Figura 3.2: Estructura de los ficheros a ordenar.

```
FICHERO *abrir_fichero (char *nombre);
```

Esta función debe abrir un fichero con la estructura descrita y cuyo nombre recibe a través de la cadena de caracteres **nombre**.

Una vez abierto el fichero, se debe crear en memoria una estructura con el formato **FICHERO** que contenga:

- i. Un puntero a la estructura de control del fichero abierto.

- ii. Una copia en memoria del pie del fichero, es decir, el total de registros del fichero y la secuencia de índices.

La función debe devolver un puntero a la estructura **FICHERO** creada para poder acceder al fichero.

- (b) Escribir una función con la declaración siguiente:

```
void cerrar_fichero (FICHERO *f);
```

Esta función debe cerrar el fichero referenciado por **f** y que previamente habrá sido abierto por la función **abrir_fichero**.

Como los índices que indican el orden en que deben ser leídos los registros pueden haber cambiado desde que se abrió el fichero, será necesario que esta función actualice dichos índices antes de cerrar el fichero.

La función debe también liberar las estructuras de memoria que se hayan asignado dinámicamente durante la apertura del fichero.

- (c) Escribir una función con la declaración siguiente:

```
char *leer_nombre (FICHERO *f, long n);
```

Esta función debe devolver un puntero a una cadena de caracteres con el nombre del empleado que ocupa la posición referenciada por el índice número **n** del fichero controlado a través de **f**.

Téngase en cuenta que **n** no es el número del registro que queremos leer, sino el número del índice que contiene el número del registro.

La dirección que se debe devolver corresponderá a una cadena de caracteres local y estática de la propia función.

- (d) Escribir una función con la declaración siguiente:

```
void qsr (FICHERO *f, long inf, sup);
```

Esta función debe modificar los índices del fichero controlado por **f** para que, al recorrerlos según la forma descrita anteriormente, los registros aparezcan ordenados en orden alfabético ascendente según el campo **nombre**.

Se recomienda utilizar el algoritmo de ordenación *quicksort* [Hoare, 1961] y [Knuth, 1973].

Los parámetros **inf** y **sup** son los índices inferior y superior que intervienen en el proceso de ordenación. Es decir, los índices a modificar son **inf**, **sup** y todos los comprendidos entre ellos.

- (e) Utilizando las funciones anteriores, escribir un programa que reciba el nombre de un fichero a través de la línea de órdenes y lo ordene en orden alfabético ascendente según el campo **nombre** de los registros **EMPLEADO**. El nombre del programa será **ordenar**.

La ordenación se debe hacer modificando los índices que figuran en el pie del fichero. Nunca se deben modificar los registros de datos.

Para comprobar que el programa **ordenar** funciona correctamente será necesario escribir otros dos programas:

- **crear**, para crear un fichero que contenga registros desordenados,
- **leer**, para mostrar por pantalla el contenido de un fichero con la estructura indicada anteriormente.

Como ejemplo, mostramos a continuación el código del programa **crear**:

```
#include "empleados.h"
#include <stdio.h>
#include <fcntl.h>

#define modo (O_WRONLY|O_TRUNC|O_CREAT)

FICHERO *abrir_fichero (char *nombre)
{
    FICHERO *f;

    if ((f = (FICHERO *)malloc (sizeof (FICHERO))) == NULL) {
        fprintf (stderr, "ERROR reservando memoria\n");
        exit (-1);
    }
    if ((f->descriptor = open (nombre, modo, 0666)) == -1) {
        perror (nombre);
        exit (-1);
    }
    f->pie.nro_índices = 0;
    f->pie.índices = NULL;
    return (f);
}

void cerrar_fichero (FICHERO *f)
{
    long i;

    f->pie.índices=(long *)malloc(f->pie.nro_índices*sizeof(long));
    if (f->pie.índices == NULL) {
        fprintf (stderr, "ERROR reservando memoria\n");
        exit (-1);
    }
    for (i = 0; i < f->pie.nro_índices; i++)
        f->pie.índices [i] = i;
    write (f->descriptor, f->pie.índices,
        sizeof(long)*f->pie.nro_índices);
    write (f->descriptor, &f->pie.nro_índices, sizeof(long));

    close (f->descriptor);
}
```

```

EMPLEADO *leer_registro ()
{
    static EMPLEADO e;

    printf ("Nombre: ");
    if (gets (e.nombre) == NULL)
        return (NULL);
    printf ("Sueldo: ");
    if (scanf ("%f", &e.sueldo) == EOF)
        return NULL;
    getchar (); // Eliminar retorno de carro
    return (&e);
}

escribir_registro (FICHERO *f, EMPLEADO e)
{
    write (f->descriptor, &e, sizeof(e));
    ++f->pie.nro_índices;
}

main (int argc, char *argv[])
{
    EMPLEADO *e;
    FICHERO *f;

    if (argc != 2) {
        fprintf(stderr, "ERROR. Forma de uso: %s fichero\n", argv[0]);
        exit (-1);
    }
    f = abrir_fichero (argv [1]);
    while ((e = leer_registro ()) != NULL)
        escribir_registro (f, *e);
    cerrar_fichero (f);
}

```

La forma de crear un fichero de empleados es:

```
$ crear empleados
```

```

Nombre: Tirante el Blanco
Sueldo: 23000
Nombre: Palmerín de Oliva
Sueldo: 19300
Nombre: Ctrl+D

```

Los programas leer y ordenar deben ser escritos por el lector.

3.6 El objetivo de este ejercicio es ilustrar la necesidad de crear una biblioteca de funciones de entrada/salida que ofrezca una interfaz de más alto nivel que las llamadas al sistema.

Tal y como hemos visto en los ejercicios y ejemplos anteriores, la entrada/salida con `open`, `read`, `write` y `close` es más que suficiente para manipular ficheros. Sin embargo, hay algunos detalles de gestión que están en manos del programador y que pueden llevar a una baja prestación si los programas no se escriben correctamente. Uno de estos detalles es la gestión de memorias intermedias de entrada/salida.

Para explicar la importancia que tiene realizar una entrada/salida con memorias intermedias de tamaño adecuado, vamos a compilar una nueva versión del programa `copiar`. En esta versión vamos a trabajar con una memoria intermedia de tamaño 1 byte. El programa se muestra a continuación:

```
#include <stdio.h>
#include <fcntl.h>
// Memoria intermedia de 1 byte para manipular los datos
char mem[1];

main (int argc, char *argv [])
{
    int fd_origen, fd_destino;
    int nbytes;

    if (argc != 3) {
        fprintf(stderr, "Forma de uso: %s origen destino\n", argv[0]);
        exit (-1);
    }
    if ((fd_origen = open (argv [1], O_RDONLY)) == -1) {
        perror (argv [1]);
        exit (-1);
    }
    if ((fd_destino =
        open (argv [2], O_WRONLY|O_TRUNC|O_CREAT, 0666)) == -1) {
        perror (argv [2]);
        exit (-1);
    }
    while ((nbytes = read ( fd_origen, mem, sizeof mem)) > 0)
        write (fd_destino, mem, nbytes);
    close (fd_origen);
    close (fd_destino);

    exit (0);
}
```

El tiempo que emplean los programas `copiar` y `copiar2` en realizar copias se puede medir con la orden `time`.

```
$ time copiar /usr/bin/cc cc1
```

```
$ time copiar2 /usr/bin/cc cc2
```

En la tabla siguiente podemos ver un resumen de los tiempos empleados por cada programa copiando un mismo fichero `—/usr/bin/cc—`.

Programa	T. usuario (s)	T. sistema (s)
<code>copiar</code> (con memoria intermedia)	0,01	0,05
<code>copiar2</code> (sin memoria intermedia)	0,25	5,46

Se puede comprobar que una mala gestión de la entrada/salida degrada enormemente las prestaciones del programa.

Para forzar el uso de memorias intermedias es útil escribir una biblioteca que haga abstracción de estos detalles y se los oculte al programador. Este principio de programación se conoce como encapsulamiento y consiste en habilitar mecanismos de programación donde se oculten los detalles que no están directamente relacionados con las aplicaciones. Así, por ejemplo, cuando un programador escribe una aplicación de gestión de nóminas, no tiene por qué preocuparse en estudiar detalles del sistema operativo que le permitan aumentar la velocidad de su aplicación. Si esos detalles existen, es más cómodo que estén encapsulados en las herramientas y bibliotecas de desarrollo y que el programador se centre únicamente en la aplicación.

La biblioteca que se pide desarrollar debe tener las siguientes funciones públicas:

- `FICHERO *abrirF (char *nfichero, char *modo);`

donde,

- `nfichero` es una cadena de caracteres que contiene el nombre de un fichero.
- `modo` es una cadena de caracteres que codifica el tipo de acceso al fichero. Sus posibles valores serán: `"l"` para abrir el fichero en modo lectura y `"e"` para abrirlo en modo escritura.

Si el fichero se puede abrir, la función devuelve un puntero a una estructura de tipo `FICHERO`. Esta estructura se define en el fichero de cabecera `Fio.h` y tiene los campos siguientes:

- `int descriptor`, descriptor de fichero.
- `char modo`, modo de apertura: `'l'` —lectura—, `'e'` —escritura—.
- `int total`, total de bytes que hay en la memoria intermedia.
- `int sig`, próximo byte a leer de la memoria intermedia o a escribir en ella.
- `char mem [TAMAÑO]`, memoria intermedia que contiene los datos.

La estructura `FICHERO` se debe reservar dinámicamente mediante una llamada a `malloc`.

En el caso de que la función no pueda abrir el fichero indicado devolverá el puntero `NULL` y la variable `errno` tomará alguno de los valores siguientes:

- `EINVAL`, el parámetro `modo` no tiene un valor adecuado.

- ENOMEM, no hay suficiente memoria para crear la estructura **FICHERO**.

Para poder abrir el fichero, la función **abrirF** deberá hacer una llamada a **open**.

- **int leercF (FICHERO *f);**

Esta función devuelve un byte leído del fichero referenciado por **f**. Como la lectura se hace a través de una memoria intermedia, la función intenta leer primero del campo **f->mem**. El byte que se debe devolver es el que indique el índice **f->sig**. El campo **f->total** se actualizará para que indique siempre el número de bytes que aún no han sido leídos de la memoria intermedia.

En el caso de que la memoria intermedia esté vacía, **leercF** hará una llamada a la función **LeerMem** que se describe más abajo. Con esta llamada se consigue recargar la memoria intermedia leyendo datos del fichero.

Cuando el fichero referenciado por **f** no tenga más datos, **leercF** debe devolver **EOF**.

- **BOOLEAN escribircF (FICHERO *f, char c);**

Esta función escribe el byte **c** en el fichero referenciado por **f**. Como la escritura se hace a través de una memoria intermedia, la función intenta escribir primero en el campo **f->mem**. La posición donde se debe escribir es la que indique el índice **f->sig**.

En el caso de que la memoria intermedia esté llena, **escribircF** hará una llamada a la función **EscribirMem** que se describe más abajo. Con esta llamada se consigue vaciar la memoria intermedia escribiendo los datos en el fichero.

La función devuelve el valor lógico **ÉXITO** si consigue escribir satisfactoriamente. En caso contrario devuelve el valor lógico **FALLO**. Estos valores lógicos se definen en el fichero de cabecera **Fio.h**.

- **BOOLEAN cerrarF (FICHERO *f);**

Esta función cierra el fichero referenciado por **f** y libera la zona de memoria a la que apunta **f**. Si el fichero está abierto en modo escritura, antes de cerrarlo la función debe llamar a **EscribirMem** para actualizar el fichero con los últimos datos escritos en la memoria intermedia y que aún no habían sido escritos en el fichero.

cerrarF devolverá **ÉXITO** si el fichero se cierra satisfactoriamente. En caso contrario devolverá **FALLO**. Los motivos que pueden hacer que el fichero no se cierre bien son:

- Que el campo **f->descriptor** no corresponda a un descriptor válido.
- Que se produzca un fallo al actualizar los datos de la memoria intermedia.

Junto a las funciones públicas, hay que escribir las siguientes funciones privadas:

- **static BOOLEAN LeerMem (FICHERO *f);**

Esta función carga la memoria intermedia referenciada por el campo **f->mem** con datos procedentes del fichero cuyo descriptor es **f->descriptor**. La operación se debe hacer mediante una llamada a **read**.

Si se produce algún fallo en la lectura la función devolverá **FALLO**. En caso contrario devolverá **ÉXITO**.

- `static BOOLEAN EscribirMem (FICHERO *f);`

Esta función escribe el contenido de la memoria intermedia referenciada por `f->mem` en el fichero cuyo descriptor es `f->descriptor`. La operación de escritura se debe realizar con el menor número posible de llamadas a `write`.

Si la lectura se realiza correctamente, `EscribirMem` devolverá `ÉXITO`. En caso contrario devolverá `FALLO`.

Como podemos observar, ambas funciones se declaran con clase de almacenamiento `static`. Esto impide que las funciones sean visibles fuera del módulo donde han sido codificadas. Se dice que estas funciones son privadas a este módulo.

Tanto las funciones públicas como las privadas pertenecen al módulo `Fio.c`. En el fichero `Fio.h` tenemos las declaraciones necesarias para poder utilizar la biblioteca de entrada/salida. El contenido de `Fio.h` es el siguiente:

```
// Fio.h: definición de los datos para la biblioteca de entrada/salida con
// memorias intermedias.
#ifndef _Fio_h
#define _Fio_h
// Tamaño de la memoria intermedia
#define TAMAÑO 1024

typedef enum {FALLO, ÉXITO} BOOLEAN;

// Definición de la estructura FICHERO
typedef struct {
    int descriptor; // Descriptor de fichero
    char modo; // Modo de apertura: 'l' -> lectura, 'e' -> escritura
    int total; // Total de bytes que hay en la memoria intermedia
    int sig; // Próximo byte a leer o escribir en la memoria intermedia
    char mem [TAMAÑO]; // Memoria intermedia
}FICHERO;

// Prototipo de las funciones
FICHERO *abrirF (char *nfichero, char *modo);
BOOLEAN cerrarF (FICHERO *f);
int leercF (FICHERO *f);
BOOLEAN escribircF (FICHERO *f, char c);

#endif
```

Para comprobar el correcto funcionamiento de la biblioteca creada escribiremos una nueva versión del programa que copia ficheros.

```
#include <stdio.h>
#include "Fio.h"

main (int argc, char *argv [])
```

```
{
    FICHERO *origen, *destino;
    char byte;

    if (argc != 3) {
        fprintf(stderr, "Forma de uso: %s origen destino\n", argv[0]);
        exit (-1);
    }
    if ((origen = abrirF (argv [1], "l")) == NULL) {
        perror (argv [1]);
        exit (-1);
    }
    if ((destino = abrirF (argv [2], "e")) == NULL) {
        perror (argv [2]);
        exit (-1);
    }
    while ((byte = leerF (origen)) != EOF)
        escribirF (destino, byte);
    cerrarF (origen);
    cerrarF (destino);
    exit (0);
}
```

Una vez escrito el programa, podemos evaluar sus prestaciones y compararlo con las dos versiones anteriores —copiar y copiar2—. Si la biblioteca está bien escrita, con copiar3 se deben conseguir velocidades de copiado parecidas a las de la primera versión —copiar—.

3.7 Con este ejercicio se pretende ilustrar la forma de uso de `fcntl` para bloquear ficheros e implementar una versión del problema de los lectores-redactores. Este problema se enuncia como sigue:

- (a) Dos conjuntos de usuarios —lectores y redactores— tienen que coordinarse para acceder a unos datos comunes —el recurso que comparten.
- (b) Los lectores sólo inspeccionan y, por lo tanto, pueden acceder simultáneamente a los datos junto con otros lectores.
- (c) Los redactores modifican los datos y, por ello, todo redactor debe trabajar en exclusión mutua con el resto de usuarios —lectores y redactores—.

Como ya sabemos, la función `fcntl` se puede utilizar para establecer cerrojos sobre segmentos de un fichero. El inconveniente de estos cerrojos es que carecen de carácter vinculante. Esto quiere decir que los procesos pueden consultar los cerrojos antes de acceder a un fichero y decidir a continuación si los respetan o no. Los procesos también pueden prescindir de consultar el valor de los cerrojos, con lo que la capacidad coercitiva de los mismos es nula.

Para ilustrar la forma de fijar cerrojos con `fcntl` y el significado de los mismos, se pide escribir un programa controlado por menú que le presente la siguiente interfaz al usuario:

```
$ lec-red notas
```

```
GESTOR DE BASE DE DATOS
0 --> Escribir registros
1 --> Leer registros
2 --> Fijar bloqueo tipo lectura
3 --> Fijar bloqueo tipo escritura
4 --> Desbloquear registros
5 --> Fin
Opción:
```

`lec-red` es el nombre del programa y `notas` es el nombre de una base de datos con las notas de un conjunto de alumnos. Los registros de este fichero se definen con la estructura siguiente:

```
typedef struct {
    char nombre [81];
    float nota;
}ALUMNO;
```

El programa le presenta al usuario un menú con 5 opciones cuyos significados son:

- **Escribir registros** pide un rango de registros y a continuación lee del fichero estándar de entrada los datos correspondientes a esos registros:

```
Rango de registros: 1 4
Registro número 1
Nombre: Isaías
Nota: 10
...
Registro número 4
Nombre: Daniel
Nota: 10
```

A medida que se leen los registros se deben ir escribiendo en la base de datos en su posición correspondiente de acuerdo con el número de registro. El acceso de escritura se realizará sin consultar los cerrojos que haya definidos sobre esos registros, es decir, haremos caso omiso de esos cerrojos.

- **Leer registros** pide un rango de registros y a continuación muestra en el fichero estándar de salida los datos correspondientes a esos registros:

```
Rango de registros: 2 3
2 - Miqueas 10
3 - Ezequiel 10
```

Los datos que se muestran deben leerse de la base de datos. El acceso de lectura se realizará sin consultar los cerrojos que haya definidos sobre esos registros.

- **Fijar bloqueo tipo lectura** pide un número de registro y a continuación intenta fijar un cerrojo de lectura sobre el mismo.

El primer intento para fijar el cerrojo se hará de forma no bloqueante —llamada `fcntl(descriptor, &cerrojo, F_SETLK)`—. Si esta llamada falla será porque ya existe un cerrojo de escritura sobre el fichero. En este caso se debe proceder de la forma siguiente:

- (a) Determinar el *PID* del proceso que ha fijado ese cerrojo de escritura y presentarlo en el fichero estándar de salida.
- (b) Intentar fijar el cerrojo de lectura de forma bloqueante. Es decir, llamar a `fcntl` con los parámetros necesarios para que no devuelva el control hasta que todos los cerrojos de escritura del registro, fijados por otros procesos, hayan sido borrados.

Ejemplo:

```
Número de registro: 1
Registro 1 bloqueado por el proceso 201
Esperando para bloquear...
    Espera condicionada por el resto de procesos que han
    fijado cerrojos de escritura sobre el registro número 1.
Registro 1 con bloqueo tipo lectura
```

- **Fijar bloqueo tipo escritura** pide un número de registro y a continuación intenta fijar un cerrojo de escritura sobre el mismo.

El primer intento para fijar el cerrojo se debe hacer de forma no bloqueante —llamada `fcntl(descriptor, &cerrojo, F_SETLK)`—. Si esta llamada falla será porque ya existe un cerrojo de escritura sobre el fichero. En este caso se debe proceder de la forma siguiente:

- (a) Determinar el *PID* del proceso que ha fijado ese cerrojo de escritura y presentarlo en el fichero estándar de salida.
- (b) Intentar fijar el cerrojo de escritura de forma bloqueante. Es decir, llamar a `fcntl` con los parámetros necesarios para que no devuelva el control hasta que todos los cerrojos de escritura del registro, fijados por otros procesos, hayan sido borrados.

Ejemplo:

```
Número de registro: 2
Registro 2 bloqueado por el proceso 201
Esperando para bloquear...
    Espera condicionada por el resto de procesos que han
    fijado cerrojos de escritura sobre el registro número 2.
Registro 2 con bloqueo tipo escritura
```

- **Desbloquear registro** pide un número de registro y a continuación borra los cerrojos que el proceso actual haya fijado en él.

Ejemplo:

```
Número de registro: 2
Registro 2 desbloqueado
```

- **Fin** cierra la base de datos y termina el proceso.

Para escribir este programa será necesario implementar las funciones siguientes:

```
void escribir (fd);
void leer (fd);
void bloqueo_de_lectura (int fd);
void bloqueo_de_escritura (int fd);
void desbloquear (int fd);
void fin (int fd);
```

Cada función da servicio a una de las opciones de menú. El parámetro `fd` es un descriptor de fichero para acceder a la base de datos. Este descriptor lo habrá devuelto la llamada `open`. Para presentar el menú podemos escribir la función siguiente:

```
int menu (void);
```

Esta función devuelve en cada instante el número de la opción seleccionada. Este número lo podemos utilizar para indexar un array de punteros a funciones, que se habrá declarado como sigue:

```
// Tipo puntero a función que no devuelve nada
typedef void (*PFUNCIÓN) (int fd);
PFUNCIÓN funciones [] = { // Array de punteros a funciones.
    escribir,
    leer,
    bloqueo_de_lectura,
    bloqueo_de_escritura,
    desbloquear,
    fin
};
```

La función principal se encarga de analizar los parámetros de la línea de órdenes, abrir la base de datos e invocar a las opciones de menú como muestra el código siguiente:

```
while (!0) funciones[menu ()] (fd);
```

- 3.8** Escriba un programa de nombre `fechas.c` que sirva para mostrar por pantalla las fechas que se almacenan en los nodos-*i* de un conjunto de ficheros. La forma de invocar al programa será:

```
$ fechas ficheros
```

A continuación se muestran algunos ejemplos de invocación al programa y las salidas que produce:

```
$ fechas
```

```
ERROR. Forma de uso: fechas ficheros
```

```
$ fechas mover.c
```

```
mover.c
Último acceso: Mon Apr 3 21:00:17 1995
Última modificación: Mon Apr 3 12:17:14 1995
Última modificación del nodo-i: Mon Apr 3 12:17:14 1995
```

```
$ fechas /u*
```

```
/user
Último acceso: Tue Apr 4 10:50:21 1995
Última modificación: Sat Feb 18 23:10:11 1995
Última modificación del nodo-i: Sat Feb 18 23:10:11 1995
/usr
Último acceso: Mon Apr 3 11:48:10 1995
Última modificación: Sat Feb 18 23:07:27 1995
Última modificación del nodo-i: Sat Feb 18 23:07:27 1995
/usrlocal
Último acceso: Sat Mar 25 11:52:38 1995
Última modificación: Fri May 13 15:08:43 1994
Última modificación del nodo-i: Fri May 13 15:08:43 1994
```

Hay que hacer notar que en este último ejemplo se ha usado el metacarácter `*` para referirnos a varios ficheros, en concreto, todos aquellos ficheros del directorio raíz que empiezan por `u`.

- 3.9** Escriba un programa de nombre `modo.c` que sirva para visualizar en pantalla los permisos y el tipo de uno o varios ficheros. La máscara de modo que hay en el nodo-`i` de los ficheros se debe traducir de formato binario a formato *ASCII* tal y como hace la orden `ls` con el parámetro `-l`. Así, por cada fichero se debe presentar una cadena de caracteres con la forma `-rwxrwxrwx` o alguna de sus variantes, dependiendo de los permisos del fichero. La forma de invocar al programa será:

```
$ modo ficheros
```

A continuación se muestran algunos ejemplos de uso de este programa:

```
$ mkdir tmp
```

```
$ modo tmp
```

```
tmp: drwxr-xr-x

$ echo >temp.1

$ chmod 0666 temp.1 Permisos de lectura y escritura para todos los usuarios.

$ modo temp.1

temp.1: -rw-rw-rw-

$ chmod 0666 temp.1 Se activan los bits Cambiar el UID y Cambiar el GID.

$ modo temp.1

temp.1: -rwsrwsrw-

$ chmod 07707 temp.1 Se activa el Sticky bit y se desactivan los permisos de lectura, escritura y ejecución para el grupo.

$ modo temp.1

temp.1: -rws--srwt

$ ln temp.1 temp.2 Creación de un enlace duro entre temp.1 y temp.2

$ modo temp.2

temp.2: -rws--srwt Los permisos de temp.1 coinciden con los de temp.2

$ ln -s temp.1 temp.3 Creación de un enlace blando entre temp.1 y temp.3

$ modo temp.3

temp.3: lrwxrwxrwx Los permisos de temp.3 no coinciden con los de temp.1 ya que ocupan nodos—i distintos.

$ modo temp.* Para indicar varios nombres de fichero se pueden utilizar metacaracteres.

temp.1: -rws--srwt
temp.2: -rws--srwt
temp.3: lrwxrwxrwx

$ modo /dev/cua0 /dev/fd0 /dev/log /dev/printer
```

```

/dev/cua0: crw-rw-rw- Fichero de dispositivo modo carácter
/dev/fd0: brw-rw-rw- Fichero de dispositivo modo bloque
/dev/log: srw-rw-rw- Conector
/dev/printer: prwxrwxrwx Tubería con nombre

```

3.10 Escriba un programa de nombre `permisos.c` que sirva para cambiar los permisos de lectura, escritura y ejecución de un fichero o directorio —compare con la orden `chmod`—. La forma de invocar al programa será:

```
$ permisos máscara ficheros
```

- `máscara` es un patrón de caracteres que se ajusta al formato ya conocido `rwrxrwxrwx` donde se indican los permisos del propietario, el grupo al que pertenece el propietario y el resto de propietarios.
- `ficheros` es el nombre del fichero o ficheros cuyos permisos se van a cambiar. Para expresar más de un fichero se pueden utilizar los metacaracteres del intérprete de órdenes `*` y `?`.

A continuación podemos ver algunos ejemplos de uso:

```
$ permisos rw-r-r- fichero
```

```
$ permisos rw-r-r- *.c
```

```
$ permisos rwxr-xr-x *.c
```

Un asterisco `*` indica que ese permiso no debe cambiar. En el ejemplo siguiente el permiso de lectura para el propietario y los permisos de lectura y escritura para el resto de usuarios —excluido el grupo al que pertenece el propietario— no deben cambiar.

```
$ permisos *wx--*x *.c
```

Los permisos que no se indican en la máscara no se deben modificar en el fichero. Por ejemplo:

```
$ permisos -xr-x *.c
```

indica que la máscara debe ser `****-xr-x`. Es decir, los permisos de lectura, escritura y ejecución del propietario no deben cambiar, y el permiso de lectura del grupo al que pertenece el propietario tampoco cambia.

3.11 Escriba un programa de nombre `mover.c` que sirva para cambiar el nombre de un fichero o directorio —compare con la orden `mv`—. La forma de invocar al programa será:

```
$ mover origen destino
```

La orden servirá para mover uno o más ficheros. En el caso de que se vayan a mover varios ficheros, `destino` debe ser un directorio. El nombre de los ficheros

que se copien en el interior de **destino** coincidirán con los nombres de los ficheros en origen. Para especificar varios ficheros se pueden utilizar los metacaracteres del intérprete de órdenes.

A continuación se muestran algunos ejemplos de ejecución del programa **mover**:

```
$ mover a.out
```

```
ERROR. Forma de uso: mover origen destino
```

```
$ mover a.out programa
```

```
a.out --> programa
```

```
$ mover *.bak ..
```

```
Fio.c.bak --> ../Fio.c.bak
Fio.h.bak --> ../Fio.h.bak
copiar.c.bak --> ../copiar.c.bak
copiar1.c.bak --> ../copiar1.c.bak
copiar2.c.bak --> ../copiar2.c.bak
copiar3.c.bak --> ../copiar3.c.bak
```

```
$ mover ../*.bak .
```

```
../Fio.c.bak --> ./Fio.c.bak
../Fio.h.bak --> ./Fio.h.bak
../copiar.c.bak --> ./copiar.c.bak
../copiar1.c.bak --> ./copiar1.c.bak
../copiar2.c.bak --> ./copiar2.c.bak
../copiar3.c.bak --> ./copiar3.c.bak
```

```
$ mover a.out programa
```

```
No se puede mover 'a.out' a 'programa': No such file or directory
```

```
$ mover programa /usr/bin
```

```
No se puede mover 'programa' a '/usr/bin/programa': Permission denied
```

3.12 Escriba un programa de nombre **usuarios.c** que muestre en el fichero estándar de salida una lista de los usuarios que hay declarados en el sistema. El formato de la información que se debe desplegar se puede ver en el ejemplo siguiente:

```
$ usuarios
```

```
Usuario: root
  UID: 0
  GID: 0 --> root
  Directorio de arranque: /
  Programa de arranque: /bin/sh
Usuario: adm
  UID: 4
  GID: 4 --> adm
  Directorio de arranque: /
  Programa de arranque:
Usuario: anonymous
  UID: 403
  GID: 1 --> other
  Directorio de arranque: /home/ftp
  Programa de arranque: /bin/sh
Usuario: ftp
  UID: 404
  GID: 1 --> other
  Directorio de arranque: /home/ftp
  Programa de arranque: /bin/sh
Usuario: Francisc
  UID: 406
  GID: 6 --> users
  Directorio de arranque: /home/Francisco
  Programa de arranque: /bin/sh
```

Para escribir este programa será necesario consultar los ficheros `/etc/passwd` y `/etc/group`. Para ayudarnos en esta tarea podemos utilizar las siguientes funciones estándar:

- Para consultar las entradas de `/etc/passwd`: `getpwent`, `setpwent`, `endpwent`, `getpwnam` y `getpwuid`.
- Para consultar las entradas de `/etc/group`: `getgrent`, `setgrent`, `endgrent`, `getgrnam` y `getgrgid`.

3.13 Escriba un programa de nombre `grupos.c` que muestre en el fichero estándar de salida una lista con los grupos que hay declarados en el sistema, así como los usuarios que pertenecen a cada grupo. El formato de la información que se debe desplegar se muestra en el ejemplo siguiente:

```
$ grupos
```

```
Grupo: root
  GID: 0
  Miembros: root,
Grupo: bin
  GID: 2
```



```

    Miembros: root, bin, daemon,
Grupo: sys
    GID: 3
    Miembros: root, bin, sys, adm,
Grupo: adm
    GID: 4
    Miembros: root, adm, daemon,
Grupo: daemon
    GID: 12
    Miembros: root, daemon,
Grupo: staff
    GID: 51
    Miembros:

```

Para escribir este programa será necesario leer las entradas del fichero `/etc/group` y para ayudarnos en esta tarea podemos utilizar las funciones mencionadas en el ejercicio anterior.

- 3.14** Escriba un programa de nombre `propietario.c` que sirva para cambiar el propietario de un fichero o un grupo de ficheros —compare con la orden `chown`—. La forma de invocar al programa será:

```
$ propietario usuario ficheros
```

Para especificar varios ficheros se pueden utilizar los metacaracteres del intérprete de órdenes. A continuación mostramos algunos ejemplos de uso del programa:

```
$ propietario Francisc a.out
```

```
$ propietario curso0 /home/curso0
```

```
$ propietario root /usr/bin/*
```

```
$ propietario Francisc $HOME/*.c
```

- 3.15** Escriba un programa de nombre `grupo.c` que sirva para cambiar el grupo al que pertenece un fichero o un grupo de ficheros —compare con la orden `chgrp`—. La forma de invocar al programa será:

```
$ grupo nombre_grupo ficheros
```

Para especificar varios ficheros se pueden utilizar los metacaracteres del intérprete de órdenes. A continuación se muestran algunos ejemplos de uso del programa:

```
$ grupo users programa.c
```

```
$ grupo staff *.c
```

- 3.16** Recodifique el programa `bloquear.c` —ejemplo pág. 89— usando la llamada `flock`. La interfaz de esta función es la siguiente:

```
#include <fcntl.h>
int flock (int descriptor, int operación);
```

- **descriptor** es el descriptor de un fichero abierto con una llamada a `open`, `creat`, `fcntl`, etc.
- **operación** es el tipo de cerrojo que se va a definir sobre el fichero. Puede tomar los valores:
 - `LOCK_SH`, cerrojo compartido —es equivalente a los cerrojos de lectura—.
 - `LOCK_EX`, cerrojo exclusivo —es equivalente a los cerrojos de escritura—.
 - `LOCK_NB`, fijar el cerrojo de forma no bloqueante. Es decir, si el cerrojo no se puede fijar porque el fichero ha sido bloqueado por otro proceso, devolver el control. Esta constante se puede utilizar para formar una expresión *OR* con `LOCK_SH` y `LOCK_EX`.
 - `LOCK_UN`, borrar el cerrojo.

3.17 Escriba la función `lockf` a partir de `fcntl`. Para ello se deben editar los módulos `lockf.h` y `lockf.c`. En `lockf.h` figurarán:

- Las definiciones de las constantes: `F_LOCK`, `F_ULOCK`, `F_TLOCK` y `F_TEST`.
- La declaración del prototipo de `lockf`.

En `lockf.c` se implementará el cuerpo de la función `lockf` para que se ajuste a su funcionalidad estándar.

A continuación recompile el programa `bloquear.c` —ejemplo de la pág. 89— usando la función `lockf` recién creada.

3.18 Escriba un programa que realice las operaciones siguientes:

- (a) Abra el fichero `tmp` en modo escritura. Si no existe el fichero, se debe crear con permisos `0666`. En caso de que exista, su longitud se debe truncar a 0 bytes.
- (b) Escriba el valor 255 en la posición 5.243 del fichero.
- (c) Escriba el valor 255 en la posición 90_172.475 del fichero.
- (d) Cierre el fichero.
- (e) Lea la información del nodo-*i* del fichero.
- (f) Presente el tamaño del fichero, expresado en bytes, en pantalla.

Antes de ejecutar el programa responda a la pregunta siguiente: ¿cuánto espacio hay libre en el disco? A continuación ejecute el programa y responda a las preguntas siguientes:

- (a) ¿Cuál es el tamaño del fichero `tmp`, expresado en bytes?
- (b) Suponiendo que el espacio libre del disco medido anteriormente fuese inferior a 90 Mb, ¿cómo es posible crear un fichero con ese tamaño?

- (c) ¿Cuántos bloques de disco ocupa realmente el fichero `tmp`?
- (d) ¿Concuerda el número de bloques ocupados con el tamaño del fichero?, en caso negativo ¿a qué se debe esa diferencia?
- (e) Si sólo se hubiera escrito en la posición 90172.475 y no se hubiera escrito el byte de la posición 5.233, ¿cuál sería el tamaño del fichero y cuántos bloques ocuparía?

3.19 Escriba una función con la siguiente declaración:

```
off_t ltell (int fildes);
```

que reciba como parámetro `fildes` el descriptor de un fichero correctamente abierto, y devuelva la posición actual del puntero de lectura/escritura del fichero asociado a `fildes`. Esta posición se debe medir con respecto al inicio del fichero.

3.20 Implemente una función con la siguiente declaración:

```
size_t fsize (char *path);
```

que devuelva el tamaño, en bytes, del fichero cuyo nombre está apuntado por `path`. Utilice las llamadas `open`, `lseek` y `close`.

3.21 Escribir un programa con una funcionalidad similar a la del programa estándar `touch`.

3.22 Modifique el programa `lec-red` escrito en el ejercicio de la pág. 105 para que los accesos de lectura y escritura realizados en las funciones `leer` y `escribir` tengan en cuenta los cerrojos que hay fijados sobre los registros. La semántica que se debe aplicar a los cerrojos es la siguiente:

- (a) Cuando un registro está bloqueado con un cerrojo de lectura, los procesos que no son propietarios de ese cerrojo no pueden escribir en ese registro, sin embargo, esos procesos sí pueden leer los datos del registro.
- (b) Cuando un registro está bloqueado con un cerrojo de escritura, ningún proceso excepto el propietario del cerrojo puede leer o escribir en el registro.
- (c) Como los cerrojos se definen por registros, el resto de registros que se encuentran desbloqueados son accesibles en cualquiera de los dos modos.

Capítulo 4

Manejo de directorios y ficheros especiales

«El cual autor no pide a los que la leyeren, en premio del inmenso trabajo que le costó inquerir y buscar todos los archivos manchegos por sacarla a luz, sino que le den el mismo crédito que suelen dar los discretos a los libros de caballerías, que tan validos andan en el mundo, que con esto se tendrá por bien pagado y satisfecho y se animará a sacar y buscar otras, si no tan verdaderas, a lo menos de tanta invención y pasatiempo.»

(Cervantes, Quijote I-52)

4.1	Acceso a directorios	117
4.2	Acceso a ficheros especiales	129
4.3	Administración del sistema de ficheros	137
4.4	Ejercicios	144

4.1 Acceso a directorios

Los directorios son ficheros que le proporcionan al sistema una estructura jerárquica de árbol invertido. Estos ficheros tienen una estructura de datos que es interpretada y mantenida por el núcleo. Los directorios, como cualesquiera otros ficheros, se pueden abrir mediante una llamada `open` y pueden ser leídos mediante la llamada `read`, pero a ningún usuario le está permitido escribir directamente sobre ellos mediante llamadas a `write`.

Para leer la información de un directorio, debemos conocer su estructura. Como la organización de los datos de un directorio depende del sistema —las diferencias fundamentales se dan entre UNIX System V y BSD—, los programas que escribamos para manejarlos no van a ser transportables. Para solucionar este inconveniente y hacer que la estructura del directorio sea transparente al programador de aplicaciones, se ha propuesto una biblioteca estándar de funciones de manejo de directorios. Esta biblioteca la estudiaremos más adelante —consulte `directory(3C)` y el § 4.1.5 pág. 122—.

Aunque no está permitido modificar directamente la estructura de un directorio, hay una serie de llamadas que permiten crear o borrar directorios y añadirles nuevas entradas, lo cual es una vía indirecta para modificarlos.

4.1.1 Creación de un directorio (`mknod` y `mkdir`)

Hemos visto que para crear un fichero ordinario disponemos de dos llamadas, una de ellas es `open` con el indicador `O_CREAT` activo, y la otra es `creat`; pero en UNIX hay otros tres tipos de ficheros: directorios, ficheros especiales y tuberías con nombre. Para crearlos se utiliza la llamada `mknod`:

```
#include <sys/types.h>
#include <sys/stat.h>
int mknod (char *path, mode_t mode, int dev);
```

`mknod` crea un fichero nuevo cuya ruta se indica en el parámetro `path`. El modo del fichero viene especificado en el parámetro `mode`. El significado de este parámetro es el mismo que vimos para la llamada `chmod` —consulte el § 3.5.2 pág. 80—. En el fichero de cabecera `<sys/stat.h>` hay definidas unas constantes simbólicas que podemos utilizar para construir el parámetro `mode` mediante el operador `OR` | a nivel de bits —consulte la tabla 4.1—.

El parámetro `dev` tiene aplicación únicamente cuando vamos a crear un fichero de dispositivo —modo bloque o modo carácter—, siendo ignorado en el resto de los casos. `dev` codifica el *major number* y el *minor number* del dispositivo y es dependiente de la implementación y de la configuración del sistema. `dev` puede ser creado con la macro `makedev`, definida en `<sys/sysmacros.h>`.

`mknod` sólo puede ser invocada por un usuario con privilegios de superusuario, excepto a la hora de crear una tubería con nombre, situación en la que cualquier usuario puede usar `mknod`. Por este motivo, `mknod` no es muy útil para crear nuevos directorios.

Si `mknod` se ejecuta satisfactoriamente, devuelve el valor 0; en caso contrario, devuelve -1 y en `errno` estará el código de error producido.

Para superar los problemas que plantea `mknod` a la hora de crear directorios, algunos sistemas suministran la llamada `mkdir`:

```
#include <sys/types.h>
#include <sys/stat.h>
int mkdir (char *path, mode_t mode);
```

Esta llamada crea un nuevo fichero de directorio cuya ruta es la indicada en el parámetro `path`. Los bits de permiso del nuevo directorio son inicializados de acuerdo con el parámetro `mode` y son modificados por la máscara de modo de creación del proceso —consulte `umask(2)` y el § 3.5.2 pág. 81—. Cada bit activado en la máscara de modo de creación aparecerá desactivado en la máscara de modo del directorio recién creado.

Las constantes simbólicas definidas en `<sys/stat.h>` y referidas a los bits de permiso —desde `S_IRUSR`, permiso de lectura por el propietario, hasta `S_IXOTH`, permiso de ejecución por otros— pueden utilizarse para crear el parámetro `mode` que le pasamos a `mkdir`.

Tabla 4.1: Modos de un fichero.

Constante	Significado
S_IFMT	Máscara de tipo de fichero
S_IFIFO	Tubería con nombre
S_IFCHR	Fichero especial modo carácter
S_IFDIR	Directorio
S_IFBLK	Fichero especial modo bloque
S_IFREG	Fichero regular
S_ISUID	Cambiar el identificador del usuario en ejecución
S_ISGID	Cambiar el identificador del grupo en ejecución
S_ISVTX	Mantener el segmento de texto en memoria después de ejecutar
S_IRWXU	Máscara de permiso para el propietario
S_IRUSR	Permiso de lectura para el propietario
S_IWUSR	Permiso de escritura para el propietario
S_IXUSR	Permiso de ejecución (búsqueda) para el propietario
S_IRWXG	Máscara de permiso para el grupo
S_IRGRP	Permiso de lectura para el grupo
S_IWGRP	Permiso de escritura para el grupo
S_IXGRP	Permiso de ejecución (búsqueda) para el grupo
S_IRWXO	Máscara de permiso para otros
S_IROTH	Permiso de lectura para otros
S_IWOTH	Permiso de escritura para otros
S_IXOTH	Permiso de ejecución (búsqueda) para otros

Si `mkdir` se ejecuta satisfactoriamente, devuelve el valor 0; en caso contrario, devolverá -1 y en `errno` estará el código del error producido.

`mkdir` puede escribirse a partir de `mknod` y cualquier usuario con los privilegios del superusuario podrá ejecutarlo de esta forma. Una posible secuencia de código para `mkdir` es:

```
#include <sys/types.h>
#include <sys/stat.h>

int mkdir (char *path, mode_t mode)
{
    return (mknod (path, S_IFDIR | mode, 0));
}
```

Desde la línea de órdenes del sistema podemos crear directorios invocando al programa estándar `mkdir`.

4.1.2 Borrado de un directorio (`rmdir`)

Si queremos borrar un directorio de la jerarquía del sistema, usaremos la llamada `rmdir`:

```
#include <unistd.h>
int rmdir (char *path);
```

donde **path** es un puntero a la ruta del directorio que queremos borrar. Para que la llamada se ejecute satisfactoriamente, se debe cumplir: que el directorio esté vacío, es decir, que sus dos únicas entradas sean **.** y **..**; y que no sea el directorio de trabajo de ningún proceso.

rmdir devuelve 0 si se ejecuta correctamente; en caso contrario, devuelve -1 y en **errno** colocará el código del tipo de error producido.

4.1.3 Creación de nuevas entradas en un directorio (**link**)

Ya hemos visto que **open**, **creat** y **mknod** permiten crear nuevos ficheros, lo que se traduce en crear nuevas entradas para los directorios donde se encuentran esos ficheros. Los directorios, en realidad, se utilizan para asignarle un nombre de fichero a un nodo-i y puede haber varios nombres conectados con un mismo nodo-i. Este mecanismo se conoce como enlace y cada nodo-i tiene un campo para llevar la cuenta del número de enlaces que tiene. Para enlazar un nombre con un nodo-i se emplea la llamada **link**:

```
#include <unistd.h>
int link (char *path1, char *path2);
```

path1 debe apuntar a la ruta de un fichero existente, a cuyo nodo-i se enlazará el nombre de fichero que viene indicado por **path2**; **path2** es añadido como nueva entrada en el directorio; así, por ejemplo, la llamada:

```
link ("/bin/ls", "/usr/local/bin/dir");
```

hace que el programa **ls** que está en el directorio **/bin** pueda ser accedido como **dir** desde el directorio **/usr/local/bin**. Así, si en la variable de entorno **PATH** figura el directorio **/usr/local/bin**, se podrá ejecutar **ls** escribiendo **dir**.

link devuelve 0 cuando se ejecuta satisfactoriamente y -1 cuando se produce algún error; en este caso, **errno** tendrá el código del tipo de error producido.

Con **unlink** podremos deshacer el enlace existente entre un nombre de fichero y su nodo-i asociado, eliminando así una entrada de directorio. La declaración de **unlink** es:

```
#include <unistd.h>
int unlink (char *path);
```

donde **path** es un puntero a la ruta del fichero que queremos eliminar del directorio. Cuando todos los enlaces de un fichero son borrados y no hay ningún proceso que tenga abierto el fichero, el espacio ocupado en el disco es liberado y el fichero deja de existir. Si uno o más procesos tienen abierto el fichero cuando se rompe el último enlace, el borrado del fichero se pospone hasta que se cierren todas las referencias al fichero.

unlink devuelve 0 si se ejecuta satisfactoriamente, y -1 en caso contrario.

4.1.4 Directorios asociados a un proceso (chdir y chroot)

Cuando se arranca un proceso en UNIX, se le asignan una serie de directorios sobre los que trabajar. El primer directorio recibe el nombre de *directorio de trabajo actual* y es el punto de partida para construir las rutas de partida de los ficheros. El nombre que tiene asociado el directorio de trabajo actual es un punto ..

Otro directorio importante es el *directorio raíz*, que le indica al proceso cuál es el fichero de directorio al que se le asigna el nombre /. Por lo general, el directorio raíz es el de arranque del sistema y no se suele modificar.

Para cambiar el directorio de trabajo actual de un proceso se utiliza la llamada `chdir`, que se declara como sigue:

```
#include <unistd.h>
int chdir (char *path);
```

donde `path` es la ruta del nuevo directorio de trabajo actual. `chdir` devuelve 0 si se ejecuta bien, y -1 si se produce algún error.

En la biblioteca estándar hay una función que devuelve la ruta absoluta del directorio de trabajo actual (consulte `getcwd(3C)`). La interfaz de esta función es:

```
#include <unistd.h>
char *getcwd (char *buf, int size);
```

`buf` es un puntero a la zona de memoria donde se guarda la ruta del directorio de trabajo actual y `size` es el tamaño de `buf`. Si se ejecuta correctamente, `getcwd` devuelve el mismo puntero que le pasamos a través de `buf`; en caso contrario, devuelve NULL. La siguiente secuencia de código hace una llamada a `getcwd`,

```
char cwd [256];
...
getcwd (cwd, sizeof (cwd) - 1);
// El último byte de cwd lo reservamos para el carácter '\0'
```

Para cambiar el directorio raíz asociado a un proceso se emplea la llamada `chroot`:

```
#include <unistd.h>
int chroot (char *path);
```

El parámetro `path` apunta a una cadena con la ruta del directorio que actuará como nuevo directorio raíz. Para poder ejecutar esta llamada hay que tener los privilegios del superusuario. Después de ejecutarse correctamente, `chroot` devuelve el valor 0; en caso contrario, devuelve -1 y en `errno` coloca el código del error producido.

Un ejemplo de utilización de `chroot` se da cuando estamos desarrollando alguna aplicación que actúa sobre los ficheros de configuración del sistema. Imaginemos que estamos desarrollando un programa para añadir nuevos usuarios al sistema. Este programa deberá modificar, entre otros, el fichero `/etc/passwd`. Como habrá otros usuarios trabajando en el sistema mientras nosotros desarrollamos el programa, no podremos tocar los ficheros de configuración, ya que se producirían interferencias molestas y, si por error dañamos alguno de los ficheros de configuración, el sistema podría quedar inutilizado. La solución es cambiar el directorio raíz para nuestros procesos y engañarlos haciendo que cuando

modifiquen `/etc/passwd` estén modificando realmente otro fichero. Así, si hacemos que el nuevo directorio raíz pase a ser el directorio `/users/desarrollo`, por ejemplo, cuando el programa modifique `/etc/passwd`, en realidad estará modificando el fichero `passwd` situado en el directorio `/users/desarrollo/etc`.

4.1.5 Biblioteca estándar de acceso a directorios

Vamos a estudiar ahora una serie de funciones para leer el contenido de un directorio sin que tengamos que preocuparnos por la estructura del mismo. Esta biblioteca fue implementada para el sistema BSD, que introducía una estructura de directorio más compleja que la del UNIX System V, pero no tardó en ser adoptada por casi todos los sistemas UNIX.

La filosofía de esta biblioteca es muy parecida a la de cualquier biblioteca de alto nivel pensada para realizar entrada/salida sobre ficheros. Así, va a tener funciones para abrir y cerrar ficheros de directorio: `opendir` y `closedir`, para leer entradas de un directorio: `readdir`, y para controlar la posición del puntero de lectura: `seekdir`, `telldir` y `rewinddir`.

Apertura de un directorio (`opendir`)

La función para abrir un fichero de directorio es `opendir` y su declaración es la siguiente:

```
#include <sys/types.h>
#include <dirent.h>
DIR *opendir (char *dirname);
```

donde `dirname` es un puntero a la ruta del directorio que queremos abrir. Después de ejecutarse correctamente la llamada, `opendir` devuelve un puntero a una estructura del tipo `DIR`. Esta estructura la llamaremos *flujo de directorio* —compare con la forma de trabajo de `fopen` en el § 3.3.1 pág. 59— y la utilizarán otras funciones de la familia para identificar al directorio sobre el que deben trabajar.

Desde `opendir` se hace una llamada a `malloc` —consultar `malloc(3C)` y el § 6.5.1 pág. 202— con la que reserva memoria para el flujo de directorio que devuelve. Si se produce un error en la apertura del directorio, `opendir` devolverá el valor `NULL` y en la variable `errno` colocará el código del tipo de error producido.

El tipo `DIR` está definido en `<dirent.h>` y su estructura es la siguiente:

```
typedef struct __dirdesc {
    int dd_fd;
    long dd_loc;
    long dd_size;
    long dd_bbase;
    long dd_entno;
    long dd_bsize;
    char *dd_buf;
} DIR;
```

La secuencia de código siguiente muestra cómo abrir un directorio:

```
#include <dirent.h>
...
DIR *dir;
char *directorio = "/usr/include/sys";
...
if ((dir = opendir (directorio)) == NULL) {
    perror (directorio);
    exit (-1);
}
...
```

Lectura de las entradas de un directorio (readdir)

Para leer las entradas de un directorio abierto con `opendir`, emplearemos la función `readdir`:

```
#include <sys/types.h>
#include <dirent.h>
struct dirent *readdir (DIR *dirp);
```

donde `dirp` es el puntero a un flujo de directorio ya abierto. Si funciona correctamente, `readdir` lee la entrada donde se encuentre situado el puntero de lectura de ese directorio y actualiza el puntero a la siguiente entrada para permitir así una lectura de tipo secuencial. `readdir` devuelve la entrada leída a través de un puntero a una estructura de tipo `struct dirent` —también definida en el fichero de cabecera `<dirent.h>`—, y devuelve `NULL` cuando llega al final del directorio o cuando se produce algún error de lectura.

Internamente, `readdir` no hace ninguna reserva de memoria para la estructura `dirent`, ya que la dirección que devuelve corresponde a una variable local estática. Así, cuando escribamos código con `readdir`, no debemos preocuparnos por llamar a `free` para liberar las estructuras devueltas. La composición del tipo `struct dirent` es la siguiente:

```
struct dirent {
    ino_t d_ino; // Nodo—i asociado a esta entrada de directorio
    short d_reclen; // Longitud de esta entrada
    short d_namlen; // Longitud de la cadena de caracteres que hay
    en d_name
    char d_name[_MAXNAMLEN + 1]; // Nombre del fichero
};
```

Cierre de un directorio (closedir)

La declaración de `closedir` es:

```
#include <sys/types.h>
#include <dirent.h>
int closedir (DIR *dirp);
```

donde `dirp` es el flujo de directorio que queremos cerrar. `closedir` se encarga de llamar a `free` para liberar la memoria reservada por `malloc`, y de llamar a `close` para cerrar el fichero abierto por la llamada a `open` realizada internamente en `opendir`. Si se realiza satisfactoriamente, `closedir` devuelve 0 y, en caso contrario, -1, indicando el tipo de error producido a través de `errno`.

Control del puntero de lectura de un directorio (`seekdir`, `telldir` y `rewinddir`)

La función `seekdir` permite situar el puntero de lectura de un directorio, `telldir` devuelve la posición de ese puntero y `rewinddir` sitúa el puntero al principio del directorio y deja su flujo asociado tal y como quedó después de la llamada a `opendir`. Las declaraciones de estas funciones se muestran a continuación:

```
#include <sys/types.h>
#include <dirent.h>
void seekdir (DIR *dirp, long loc);
long telldir (DIR *dirp);
void rewinddir (DIR *dirp);
```

Todas estas funciones reciben como primer parámetro `dirp`, el flujo del directorio sobre el que queremos trabajar; `seekdir` recibe, además, un segundo parámetro `loc`, que es el valor de la entrada donde se situará el puntero de lectura. Después de ejecutarse correctamente, `telldir` devuelve la posición actual del puntero de lectura, si falla, devuelve -1 y en `errno` tendremos el tipo de error producido. Las otras dos funciones no devuelven nada.

Ejemplo de aplicación, el programa `tree`

Como ejemplo de aplicación de las funciones para el manejo de directorios, vamos a escribir el programa `tree`. Este programa se invoca de la siguiente forma,

```
$ tree [opciones] directorio
```

y lo vamos a utilizar para recorrer una jerarquía de directorios cuya raíz es `directorio`; `opciones` representa los parámetros que se le pueden pasar a `tree`. Si `tree` no recibe ninguna opción, nos mostrará por pantalla únicamente los nombres de los directorios que forman parte de la jerarquía analizada, por ejemplo:

```
$ tree /users/francisc/cursos
```

```
unix-iii
  prog
  fsc.dir
    estado.dir
    tree.dir
    mstdio.dir
  doc
```

Si `tree` recibe la opción `-f`, no sólo mostrará los directorios y subdirectorios, sino que también mostrará los ficheros que lo componen. Además, junto con el nombre del fichero,

se presentará entre paréntesis su tipo, según la lista siguiente:

- (d) directorio
- (o) fichero ordinario
- (b) fichero especial modo bloque
- (c) fichero especial modo carácter
- (p) tubería con nombre
- (x) fichero ejecutable —cuando sea ejecutable por el propietario, por el grupo o por otros—.

El resultado de la ejecución de `tree` con este parámetro activo será:

```
$ tree /users/francisc/cursos -f
```

```
unix-iii (d)
  prog (d)
  fsc.dir (d)
    (o) copiar.c
    (o) fcopiar.c
  estado.dir (d)
    (o) estado.c
    (o) core
    (x) estado
  tree.dir (d)
    (o) tree.c
    (o) out
    (x) tree
    (x) copiar
    (x) fcopiar
  mstdio.dir (d)
    (o) makefile
    (o) mfcopiar.c
    (o) mfcopiar.o
    (o) mfflush.c
    (o) mfflush.o
    (o) mfio.c
    (o) mfio.o
    (o) mfoopen.c
    (o) mfoopen.o
    (o) mstdio.a
    (o) mstdio.c
    (o) mstdio.h
    (o) mstdio.o
    (x) mfcopiar
```

```

doc (d)
    (o) pipe.man
    (o) errno.h
    (x) prog_1
    (o) prog_1.c
    (o) errno.man
    (o) inode.man
    (o) fs.man

```

El código del programa `tree` es el siguiente:

Programa 4.1: Recorrido de directorios, orden `tree` (`tree.c`)

```

/****
$ tree [opciones] nombre_directorio
[opciones]:
    -f mostrar los ficheros que hay dentro de un directorio añadiendo:
        (d) directorio
        (o) fichero ordinario
        (b) fichero especial modo bloque
        (c) fichero especial modo carácter
        (p) tubería con nombre
        (x) fichero ejecutable
    Retorno: 0 si se ejecuta correctamente, -1 si se produce algún error.
****/
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <dirent.h>
#include <string.h>

#define EQ(str1, str2) (strcmp ((str1), (str2)) == 0)

struct opciones {
    unsigned mostrar_ficheros:1;
};

enum boolean {NO, SI};

/****
main: se encarga de analizar los argumentos de la línea de órdenes y de invocar
a la función tree.
****/
main (int argc, char *argv [])
{
    struct opciones opciones = {NO};
    int i, j;

```

```

/* Análisis de los argumentos de la línea de órdenes. */
if (argc < 2) {
    fprintf (stderr, "Forma de uso: %s [-f] directorio\n", argv [0]);
    exit (-1);
}
for (i = 1; i < argc; i++)
    if (argv[i][0] == '-')
        for (j = 1; argv [i][j] != '\0'; j++)
            switch (argv [i][j]) {
                case 'f':
                    opciones.mostrar_ficheros = SI;
                    break;
                default:
                    fprintf (stderr, "Opción [-%c] desconocida\n",
                            argv [i][j]);
                    exit (-1);
            }

/* Análisis de la estructura en árbol de directorios para cada uno de los
argumentos que aparecen en la línea de órdenes y que no son opciones de tree. */
for (i = 1; i < argc; i++)
    if (argv [i][0] != '-')
        tree (argv [i], opciones);
}

/**
tree: recibe la ruta de un directorio y se encarga de analizarla. Si en el directorio
hay subdirectorios, la función se llama de forma recursiva para analizar ese
subdirectorio. Según las opciones que estén activas, la función desplegará por
pantalla un tipo de información u otro.
***/
tree (char *path, struct opciones opciones)
{
    DIR *dirp;
    struct dirent *dp;
    static nivel = 0;
    struct stat buf;
    int ok, i;
    char fichero [256], tipo_fichero;

    /* Apertura del directorio. */
    if ((dirp = opendir (path)) == NULL) {
        perror (path);
        return;
    }

```

```

/* Leemos, una por una, las entradas del directorio. */
while ((dp = readdir (dirp)) != NULL) {
    /* Saltamos las entradas referidas al directorio actual (.) y al
    directorio padre (..) */
    if (EQ (dp->d_name, ".") || EQ (dp->d_name, ".."))
        continue;
    /* Formamos la ruta correspondiente al fichero de la entrada de directorio que
    estamos procesando. */
    sprintf (fichero, "%s/%s", path, dp->d_name);
    /* Lectura del nodo-i del fichero. */
    ok = stat (fichero, &buf);
    /* Si el fichero es un subdirectorio, llamamos nuevamente a tree. */
    if (ok != -1 && (buf.st_mode & S_IFMT) == S_IFDIR) {
        for (i = 0; i < nivel; i++)
            printf ("\t");
        printf ("%s %s\n", dp->d_name,
            opciones.mostrar_ficheros ? "(d)": "");
        ++nivel;
        tree (fichero, opciones);
        --nivel;
    }
    /* Si el fichero no es un directorio y está activa la opción mostrar_ficheros
    (-f), presentamos por pantalla el nombre del fichero y su tipo. */
    else if (ok != -1 && opciones.mostrar_ficheros == SI) {
        for (i = 0; i <= nivel; i++)
            printf ("\t");
        switch (buf.st_mode & S_IFMT) {
            case S_IFREG:
                if (buf.st_mode & 0111)
                    tipo_fichero = 'x';
                else
                    tipo_fichero = 'o';
                break;
            case S_IFCHR:
                tipo_fichero = 'c';
                break;
            case S_IFBLK:
                tipo_fichero = 'b';
                break;
            case S_IFIFO:
                tipo_fichero = 'p';
                break;
            default:
                tipo_fichero = '?';
        }
    }
}

```

```
        printf("(%c) %s\n", tipo_fichero, dp->d_name);
    }
}
closedir (dirp);
}
```

4.2 Acceso a ficheros especiales

Otro de los grupos de ficheros que puede manipular UNIX es el constituido por los ficheros especiales. Dentro de este grupo se engloban prácticamente todos los periféricos que hay conectados a un ordenador. El gran acierto en la concepción del sistema UNIX reside en el esfuerzo realizado para no darle trato especial a los periféricos, y conseguir que puedan ser manipulados como cualquier otro fichero.

Un dispositivo periférico es accesible a través de su fichero de dispositivo asociado. Este fichero de dispositivo se encuentra en el directorio `/dev`. Para leer o escribir datos en un periférico, procederemos de igual forma que con un fichero ordinario: abriremos su fichero de dispositivo con `open`, leeremos datos de él con `read` y escribiremos datos en él con `write`. Los datos que maneja un dispositivo dependerán de la naturaleza de éste y será responsabilidad del programador interpretar qué es lo que está leyendo del dispositivo, y enviarle datos que tengan significado para él.

4.2.1 Entrada/salida sobre terminales

Los terminales son dispositivos especiales que trabajan en modo carácter. Todo proceso que se ejecuta en UNIX tiene asociados 3 descriptores de fichero que le dan acceso a su terminal de control. Estos descriptores son: 0 para la entrada estándar, 1 para la salida estándar y 2 para la salida estándar de errores. El fichero de dispositivo que permite a un proceso acceder a su terminal de control es `/dev/tty`. Si el sistema no reservase de forma automática los descriptores anteriores, podríamos hacerlo nosotros con las llamadas siguientes:

```
close (0);
open ("/dev/tty", O_RDONLY); // Reserva del descriptor 0
close (1);
open ("/dev/tty", O_WRONLY); // Reserva del descriptor 1
close (2);
open ("/dev/tty", O_WRONLY); // Reserva del descriptor 2
```

En el sistema hay un terminal especial llamado *consola* —dispositivo `/dev/console`— que es empleado para presentar los mensajes que se producen durante la puesta en marcha del sistema.

Cada usuario que inicia una sesión de trabajo interactiva lo hace a través de un terminal. Este terminal tiene asociado un fichero de dispositivo que localmente se puede abrir como `/dev/tty`, pero que visto por otros usuarios tiene la forma `/dev/tty##`, donde `##` representa dos dígitos.

Como ejemplo para ilustrar el acceso a terminales, vamos a escribir una versión simplificada de la orden `write`. Esta orden se emplea para enviar mensajes a los usuarios que hay conectados al sistema. Su forma de uso es:

```
$ write usuario

Línea de texto 1
Línea de texto 2
...
Línea de texto n
^D [Fin de fichero]
```

Esta secuencia hará que `usuario` reciba en su terminal las `n` líneas de texto que le enviamos. Para poder enviar el mensaje, tenemos que saber si el usuario existe y si ha iniciado una sesión de trabajo. También hay que conocer cuál es el fichero de dispositivo que tiene asociado su terminal. Para obtener respuesta a estas dos preguntas, hay que consultar el fichero `/etc/utmp`. Este fichero es gestionado por el sistema y contiene información administrativa de los procesos que hay ejecutándose en un instante determinado. En la página `utmp` del manual podemos ver una explicación detallada de la estructura de este fichero. Para hacer que la lectura de `utmp` sea independiente de su estructura, vamos a emplear la función estándar `getutent`. Su declaración es:

```
#include <sys/types.h>
#include <sys/utmp.h>
struct utmp *getutent ();
```

Con cada llamada a `getutent` se lee un registro del fichero `/etc/utmp`. Si el fichero no está abierto, la llamada se encarga de abrirlo. Después de leer un registro, la función devuelve un puntero a una estructura del tipo `utmp`, definida en el fichero de cabecera `<sys/utmp.h>`. Cuando `getutent` llega al final del fichero devuelve un puntero `NULL`. La definición de la estructura `utmp` es la siguiente:

```
struct utmp {
    char ut_user[8]; // Nombre del usuario
    char ut_id[4]; // Identificador de /etc/inittab
    char ut_line[12]; // Nombre del fichero de dispositivo asociado (console,
                        // tty##, ln##, etc...)
    pid_t ut_pid; // Identificador del proceso
    short ut_type; // Tipo de entrada:
                    // EMPTY
                    // RUN_LVL
                    // BOOT_TIME
                    // OLD_TIME
                    // NEW_TIME
                    // INIT_PROCESS
                    // LOGIN_PROCESS
                    // USER_PROCESS
                    // DEAD_PROCESS
```

```

// ACCOUNTING
struct exit_status {
    short e_terminatio; // Estado de terminación del proceso
    short e_exit; // Estado de salida del proceso
} ut_time; // Se aplica a las entradas cuyo tipo es DEAD_PROCESS
unsigned short ut_reserved1; // Reservada para usos futuros
char ut_host[16]; // Nombre del ordenador
unsigned long ut_addr; // Dirección internet del ordenador
};

```

La forma de averiguar si un usuario está o no conectado al sistema es buscar una entrada del fichero `utmp` cuyo campo `ut_user` coincida con el nombre del usuario que buscamos. Para saber cuál es el fichero de dispositivo que tiene asociado su terminal, tenemos que utilizar el campo `ut_line`.

A continuación mostramos el código del programa `mensaje-para`, que es equivalente a la orden estándar `write`.

Programa 4.2: Envío de mensajes a un usuario (`mensaje-para.c`)

```

/****
$ mensaje-para usuario
    Línea de texto 1
    Línea de texto 2
    ...
    Línea de texto n
    Ctrl+D [Final de fichero]
****/
#include <stdio.h>
#include <fcntl.h>
#include "utmp.h"

main (int argc, char *argv [])
{
    int tty;
    char terminal [20], mensaje [256], *logname;
    struct utmp *utmp, *getutent ();

    if (argc != 2) {
        fprintf (stderr, "Forma de uso: %s usuario\n", argv[0]);
        exit (-1);
    }

    /* Consultamos si el usuario ha iniciado una sesión. */
    while ((utmp = getutent ()) != NULL &&
        strcmp (utmp->ut_user, argv[1], sizeof (utmp->ut_user)) != 0);
    if (utmp == NULL) {
        printf ("EL USUARIO %s NO HA INICIADO SESIÓN.\n", argv[1]);
        exit (0);
    }
}

```

```

}

/* Apertura del terminal del usuario. */
sprintf (terminal, "/dev/%s", utmp->ut_line);
if ((tty = open (terminal, O_WRONLY)) == -1) {
    perror (terminal);
    exit (-1);
}

/* Lectura de nuestro nombre de usuario. */
logname = getenv ("LOGNAME");

/* Aviso al usuario que recibe nuestro mensaje. */
/* El carácter \07 produce un pitido en su terminal. */
sprintf (mensaje,
    "\n\t\tMENSAJE PROCEDENTE DEL USUARIO %s\07\07\07\n", logname);
write (tty, mensaje, strlen (mensaje));

/* Envío del mensaje. */
while (fgets (mensaje, sizeof (mensaje), stdin) != NULL) {
    write (tty, mensaje, strlen (mensaje));
    //sprintf (mensaje, "\n");
    //write (tty, mensaje, strlen (mensaje));
}
sprintf (mensaje, "\n<FIN DEL MENSAJE>\n");
write (tty, mensaje, strlen (mensaje));
close (tty);

exit (0);
}

```

En algunos sistemas, como FreeBSD, no se dispone de la llamada `getutent` y en su defecto hay que consultar directamente el fichero `/var/run/utmp` donde se almacena, a modo de array, una entrada por cada usuario conectado al sistema. Cada entrada de `/var/run/utmp` se corresponde con una estructura `struct utmp` que se define en `<utmp.h>`. Los campos de esta estructura para el sistema FreeBSD son:

```

#define UT_NAMESIZE 16
#define UT_LINESIZE 8
#define UT_HOSTSIZE 16

struct utmp {
    char ut_line[UT_LINESIZE]; // Nombre del terminal
    char ut_name[UT_NAMESIZE]; // Nombre del usuario
    char ut_host[UT_HOSTSIZE]; // Nombre del nodo
    time_t ut_time; // Hora y día en el que el usuario se conectó al sistema
};

```

Utilizando esta información veamos la forma de codificar, en el sistema FreeBSD, parte de la funcionalidad de `getutent`. En primer lugar hay que escribir un nuevo fichero de cabecera `utmp.h` como el que se muestra a continuación, donde definimos una versión reducida de `struct utmp` que satisface las necesidades del programa `mensaje-para`.

Fichero 4.3: Nueva versión de `utmp.h` para implementar la función `getutent` (`utmp.h`)

```
/* Fichero de cabecera para la implementación de la función getutent. */
#ifndef _UTMP_
#define _UTMP_

struct utmp {
    char ut_user[16];
    char ut_line[8];
    char ut_host[16];
};

struct utmp *getutent ();
#endif
```

A continuación mostramos el código de la función `getutent` donde podemos ver que las entradas del fichero `/var/run/utmp` se leen con el formato de la estructura `struct utmp_original`, que se corresponde con la estructura del fichero, mientras que la función devuelve estructuras con la forma de `struct utmp`, definida en nuestro fichero local `utmp.h`. Conseguimos así adaptar la forma de las entradas de `/var/run/utmp` a la forma de las estructuras que se manejan en el programa `mensaje-para`.

Fichero 4.4: Implementación de `getutent` (`getutent.c`)

```
/**
 * Implementación de la función getutent para el sistema FreeBSD. Sólo se
 * implementa la funcionalidad necesaria para rellenar los campos ut_user,
 * ut_line y ut_host de la estructura struct utmp.
 */
#include <stdio.h>
#include <fcntl.h>
#include "utmp.h"

/* Definición de la estructura del fichero /var/run/utmp. No utilizamos la definición
 * estándar que aparece en /usr/include/utmp.h para no entrar en conflicto con el fichero
 * utmp.h que aportamos en esta implementación. Las definiciones siguientes han sido
 * tomadas del fichero /usr/include/utmp.h. */
#define _PATH_UTMP "/var/run/utmp"

#define UT_NAMESIZE 16
#define UT_LINESIZE 8
#define UT_HOSTSIZE 16
```

```

struct utmp_original {
    char ut_line[UT_LINESIZE];
    char ut_name[UT_NAMESIZE];
    char ut_host[UT_HOSTSIZE];
    time_t ut_time;
};

struct utmp *getutent ()
{
    struct utmp_original ent_org;
    static struct utmp ent;
    static int fd = -1;

    /* Apertura del fichero en el caso de que no haya sido abierto con una llamada
    previa a getutent. */
    if ((fd == -1) && (fd = open (_PATH_UTMP, O_RDONLY)) == -1) {
        perror (_PATH_UTMP);
        exit (-1);
    }

    /* Lectura de una entrada del fichero. */
    if (read (fd, &ent_org, sizeof (ent_org)) != sizeof (ent_org))
        return NULL;
    else {
        strncpy (ent.ut_user, ent_org.ut_name, sizeof (ent.ut_user));
        strncpy (ent.ut_line, ent_org.ut_line, sizeof (ent.ut_line));
        strncpy (ent.ut_host, ent_org.ut_host, sizeof (ent.ut_host));
        return &ent;
    }
}

```

Para compilar el programa `mensaje-para` disponemos de dos posibilidades:

1. Nuestro sistema operativo suministra la función `getutent`. En este caso hacemos visible el fichero `utmp.h` del sistema con la línea `#include <utmp.h>` y compilamos de la forma habitual:

```
$ cc -o mensaje-para mensaje-para.c
```

2. Nuestro sistema no suministra la función `getutent`. En este caso hacemos visible el fichero de cabecera `utmp.h` local con la línea `#include "utmp.h"` y compilamos el programa junto con el fichero `getutent.c`:

```
$ cc -o mensaje-para mensaje-para.c getutent.c
```

La línea `#include "utmp.h"` hace que el preprocesador de C busque el fichero `utmp.h` en el directorio de trabajo actual y, en caso de no encontrarlo ahí, lo busca en los directorios estándar para ficheros de cabecera —consulte el § B.7 pág. 524—. Por ello podemos

escribir de entrada la versión de `mensaje-para` que haga visible el fichero `utmp.h` con la línea `#include "utmp.h"` y compilar con la primera orden o la segunda dependiendo de que nuestro sistema disponga o no de la función `getutent`.

4.2.2 Control de terminales

Para programar la forma de trabajo de un dispositivo, UNIX dispone de la llamada `ioctl`. Esta llamada se puede aplicar a todos los dispositivos que soportan el acceso en modo carácter, y su declaración es:

```
#include <sys/ioctl.h>
int ioctl (int fildes, int request, arg);
```

donde `fildes` es el descriptor de un fichero previamente abierto o creado, `request` codifica el tipo de operación que vamos a realizar y los valores que puede tomar dependen del tipo de dispositivo con el que estemos trabajando, `arg` contiene los parámetros con los que vamos a programar el dispositivo. Si la operación es de lectura del estado del dispositivo, `arg` contendrá los valores con los que ya está programado.

Tanto `request` como `arg` dependen del tipo de dispositivo que queremos controlar. La sección 7 del manual de UNIX contiene una descripción detallada de todos los dispositivos que el fabricante del sistema pone a nuestra disposición. Para el caso de terminales asíncronos, la página que se debe consultar es `termio(7)`. Esta entrada es muy extensa, por lo que no vamos a detenernos en considerar todos y cada uno de los pormenores relativos al control de un terminal, sobre todo teniendo en cuenta que muchos detalles apenas si se usan en la mayoría de los programas. El lector interesado en más detalles puede encontrar una descripción muy detallada en la referencia [Stevens, 1992, págs. 325–361].

Como ejemplo de aplicación de `ioctl`, mostraré la forma de programar la lectura del teclado de un terminal para darle solución a dos necesidades que pueden plantearse en muchas aplicaciones. En concreto, se trata de la lectura en *modo no canónico* y *sin eco local*. Sabemos que la lectura del teclado se realiza con la intervención de una memoria intermedia, de tal forma que, si una aplicación utiliza las funciones estándar de entrada/salida, no va a recibir datos de entrada hasta que no se pulsa la tecla **Enter**. De cara a determinadas aplicaciones, puede interesar que la lectura sea de efecto inmediato, y que no sea necesario pulsar **Enter** para que el contenido de la memoria intermedia del teclado sea enviada al programa. Por otro lado, también puede interesar que no aparezca por pantalla lo que se está escribiendo. La lectura de la contraseña para iniciar una sesión de trabajo es un ejemplo típico donde hay que suprimir el eco del terminal.

En el programa siguiente se muestra la forma de programar el terminal para suprimir estos dos efectos: lectura del terminal en forma canónica y eco local.

Programa 4.5: Forma de usar `ioctl` (entrada.c)

```
#include <stdio.h>
main ()
{
    char *str;
    int longitud;
```

```

char *leer_cadena ();

printf ("Longitud máxima de la cadena a leer: ");
scanf ("%d", &longitud);

str = leer_cadena (longitud);
printf ("\n%s\n", str);
}

/****
leer_cadena: lee de la entrada estándar una cadena cuya longitud máxima no
debe superar los n caracteres.
****/
char *leer_cadena (int n)
{
#include <sys/ioctl.h>
#include <termios.h>
/* Carácter de retroceso. Normalmente tiene asociado el código ASCII 8. */
#define BS 127
    char *mem, c;
    int i = 0;
    struct termios parm_ant, parm;

    /* Lectura de los parámetros actuales del fichero estándar de entrada. */
    ioctl (0, TIOCGETA, &parm_ant);

    /* Programación del fichero estándar de entrada con nuevos parámetros. */
    parm = parm_ant;
    /* Suprimimos la entrada en modo canónico y el eco local. */
    parm.c_lflag &= ~(ICANON | ECHO);
    /* Devolver el control después de leer un carácter. */
    parm.c_cc [4] = 1;
    ioctl (0, TIOCSETA, &parm);

    /* Reserva de memoria para la cadena de caracteres. */
    if ((mem = (char *) malloc (sizeof(char) *(n+1))) == NULL)
        return NULL;

    /* Lectura de la entrada estándar. */
    do {
        read (0, &c, 1);
        /* Tratamiento del carácter de retroceso. Este carácter provoca que
        se borre, tanto de la memoria intermedia como de la pantalla, el último
        carácter leído. */
        if (c == BS) {
            if (i > 0) {

```

```

        --i;
        putchar ('\b');
        putchar (' ');
        putchar ('\b');
    }
}
else {
    mem[i++] = c;
    putchar (c);
}
fflush (stdout);
} while (i < n && c != '\n');
mem [i] = 0; /* Final de la cadena de caracteres. */

/* Restauración de los parámetros del fichero estándar de entrada. */
ioctl (0, TIOCSETA, &parm_ant);

return (mem);
}

```

4.3 Administración del sistema de ficheros

Vamos a estudiar ahora una serie de llamadas diseñadas para realizar las tareas de administración de los sistemas de ficheros. Muchas de las órdenes de administración vistas en el capítulo 2 están construidas a partir de estas llamadas.

4.3.1 Montar y desmontar un sistema de ficheros (`mount` y `umount`)

Sabemos que un sistema de ficheros es accesible a través del fichero de dispositivo asociado al disco o sección del disco sobre la que está montado. También es accesible a través de la jerarquía de directorios, pero para ello hay que montarlo previamente sobre un directorio de entrada que se va a convertir en el directorio raíz del sistema montado. Podemos montar un sistema de ficheros desde un programa mediante la llamada `mount`:

```
int mount (char *spec, char *dir, int rwflag);
```

donde `spec` es un puntero a la ruta del fichero de dispositivo del disco donde se encuentra el sistema de ficheros que vamos a montar, `dir` es un puntero a la ruta del directorio sobre el que se va a montar el sistema de ficheros. Si la llamada se ejecuta satisfactoriamente, las nuevas referencias a `dir` darán acceso al directorio raíz del sistema montado.

El bit menos significativo de `rwflag` se utiliza para revisar los accesos de escritura sobre el sistema de ficheros. Si vale 1, la escritura estará prohibida, por lo que sólo se podrán hacer accesos de lectura; en caso contrario, la escritura estará permitida, pero de acuerdo con los permisos individuales de cada fichero.

Si la llamada se ejecuta satisfactoriamente, devolverá el valor 0; en caso contrario, devolverá -1 y en `errno` se encontrará el código del error producido.

El siguiente es un ejemplo de llamada a `mount` para montar la sección 2 del disco número 1 sobre el directorio `/usr`. El sistema se monta en modo lectura/escritura:

```
mount ("/dev/dsk/0s2", "/usr", 0);
```

El resultado de la ejecución de esta llamada se puede ver en la figura 4.1

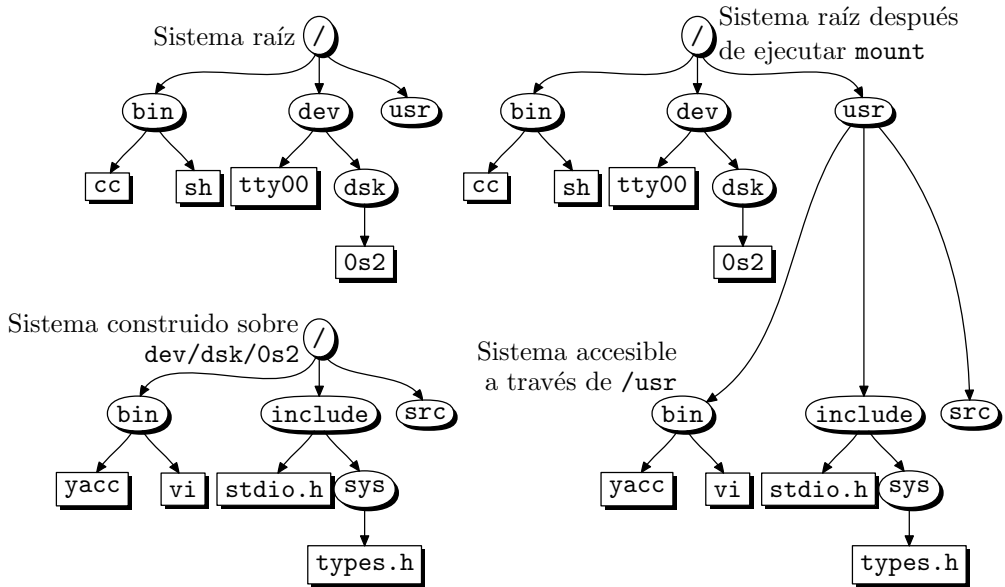


Figura 4.1: Estructura lógica de directorios después de montar el sistema de ficheros `/dev/dsk/0s2` sobre el directorio `/usr` del sistema raíz.

Cuando un sistema de ficheros deja de ser utilizado, puede ser desmontado con la llamada `umount`:

```
int umount (char *name);
```

donde `name` es un puntero a la ruta del fichero de dispositivo que da acceso al sistema de ficheros que queremos desmontar.

Las llamadas `mount` y `umount` no actualizan el fichero `/etc/mnttab`, que contiene la tabla de todos los sistemas de ficheros montados. Por lo tanto, si decidimos montar un sistema desde un programa, habrá que actualizar también la tabla desde ese programa.

4.3.2 Consistencia del sistema de ficheros (sync)

La llamada `sync` produce una sincronización entre la memoria intermedia que gestiona el núcleo y el disco, forzando así la consistencia del sistema. Los demonios `syncer` y `update` realizan periódicamente llamadas a `sync`. La interfaz de esta función es la siguiente:

```
#include <unistd.h>
void sync (void);
```

`sync` actualiza también el superbloque y los nodos-i que hayan sufrido modificaciones.

4.3.3 Estado de un sistema de ficheros

La información administrativa y estadística de un sistema de ficheros se encuentra en su superbloque. Para acceder a los aspectos más relevantes de esta información podemos usar la llamada `ustat`. Su declaración es la siguiente:

```
#include <sys/types.h>
#include <ustat.h>
int ustat (dev_t dev, struct ustat *buf);
```

donde `dev` es el número de dispositivo de la sección del disco donde se encuentra el sistema de ficheros —este número codifica los *major* y *minor number* del dispositivo—, y `buf` es un puntero a una estructura de tipo `struct ustat` donde la llamada devolverá los datos más importantes del sistema de ficheros; esta estructura se define como sigue:

```
struct ustat {
    daddr_t f_tfree; // Número de bloques libres
    ino_t f_tinode; // Número de nodos-i libres
    char f_fname [6]; // Nombre del sistema de ficheros
    char f_fpack [6]; // Nombre del paquete del sistema de ficheros
};
```

La llamada trabaja con el parámetro `dev` —número de dispositivo—, que es incómodo de manejar. A través de la llamada `stat` podemos obtener el número de dispositivo asociado a un fichero de dispositivo especificado a través de su ruta, por lo que la forma más cómoda de trabajar con `ustat` es combinar esta llamada con `stat`. En las siguientes líneas de código se muestra la forma de llamar a `ustat` para leer la información relativa al sistema de ficheros que hay en `/dev/root`.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <ustat.h>
...
struct stat sbuf;
struct ustat ubuf;
...
if (stat ("/dev/root", &sbuf) == -1) {
    // Fallo
    // Tratamiento del fallo
} else if (ustat (sbuf.st_dev, &ubuf) == -1) {
    // Fallo
    // Tratamiento del fallo
} else
    // Tratamiento de los datos leídos
```

La información que devuelve `ustat` es pobre en comparación con la que se almacena en el superbloque. La forma de acceder al resto de la información es tratando directa-

mente con el fichero de dispositivo que da acceso al sistema de ficheros. Para ello, hay que abrir ese fichero mediante una llamada a `open`. Es conveniente abrir el fichero en modo sólo lectura, para evitar posibles daños debidos a una manipulación inadecuada del superbloque. Una vez abierto el fichero de dispositivo, hay que situarlo en su segundo bloque físico mediante `lseek`. Recordemos que el primer bloque está dedicado al programa de *boot*. El contenido del superbloque se debe leer con `read`, recogiendo los datos en una estructura adecuada. El fabricante del sistema incluye en el fichero de cabecera `<sys/filsys.h>` la definición de la estructura del superbloque. Lamentablemente, esta estructura no está estandarizada en su definición ni en su nombre, por ello los programas que traten directamente con el superbloque no serán transportables a nivel de código fuente.

Como ejemplo, veremos un programa que le presenta al usuario información sobre el estado de todos los sistemas de ficheros que hay montados en una instalación UNIX. Este programa está escrito para el UNIX de la firma SCO —XENIX— y deberá ser modificado para compilarlo en otra versión. He decidido incluirlo aún a pesar de no ser transportable, porque puede ser de utilidad a los usuarios de XENIX y porque los procedimientos de tratamiento del superbloque que figuran en él sí son de aplicación general.

El programa trabaja con el fichero `/etc/mnttab`. Este fichero contiene una descripción de todos los sistemas de ficheros que hay montados en un instante determinado, y es actualizado por los programas `mount` y `umount`. La estructura de este fichero tampoco está normalizada y mientras en algunos sistemas es un fichero de texto, en otros es un fichero binario de datos. Para leer sus entradas, cada fabricante de UNIX brinda un conjunto de funciones de biblioteca. En el caso de SCO, la lectura se realiza mediante la llamada estándar `read`, recogiendo los datos sobre una estructura del tipo `struct mnttab` que se define en el fichero de cabecera `<mnttab.h>` como sigue:

```
struct mnttab {
    char mt_dev[LFNMAX]; // Nombre del fichero de dispositivo que da
                        // acceso al sistema de ficheros
    char mt_filsys[LPNMAX]; // Nombre del directorio que actúa como
                        // directorio raíz del sistema de ficheros
    short mt_ro_flg; // Permisos de lectura/escritura con los que se
                        // ha montado el sistema
    time_t mt_time; // Fecha en la que se ha montado el sistema
};
```

A través del campo `mt_dev` podemos acceder al fichero de dispositivo asociado al sistema de ficheros. Tras abrir este fichero y posicionarnos en su segundo bloque físico, se realiza una lectura del superbloque. Los datos se recogen en una estructura de tipo `struct filsys` que se define en `<sys/filsys>` y que contiene, entre otros, los siguientes campos:

- `s_isize`, tamaño en bytes de la lista de nodos-i.
- `s_fsize`, número de bloques del sistema.
- `s_time`, fecha de última actualización del superbloque.
- `s_tfree`, número de bloques libres.

- `s_tinode`, número de nodos-i libres.
- `s_fname`, nombre del sistema de ficheros.
- `s_fpack`, nombre del paquete del sistema de ficheros.

Como se puede ver, a partir de los campos `s_tfree`, `s_tinode`, `s_fname` y `s_fpack` se puede construir la estructura devuelta por `ustat`. El listado del programa es el siguiente:

Programa 4.6: Estado de un sistema de ficheros – lectura del superbloque (`estado-sf.c`)

```

/**
$ estado-sf
Este programa ofrece información sobre un sistema de ficheros. El programa abre
el fichero /etc/mnttab para buscar en él los nombres de los sistemas que hay
montados. La información sobre el estado del sistema se busca en el superbloque.
El superbloque es accesible a través del fichero de dispositivo que da acceso al
disco. Es el segundo bloque del fichero. El primer bloque está ocupado por el
programa de boot.
***/
#include <stdio.h>
#include <sys/types.h>
#include <time.h>
#include <fcntl.h>
#include <mnttab.h>
#include <sys/param.h>
#include <sys/filsys.h>
#include <sys/ino.h>

/* Número de bloques de 512 bytes que contiene un bloque del sistema. */
#define BLK512 (BSIZE/512)
/* Tamaño de un nodo-i. */
#define ISIZE (sizeof (struct dinode))

main (int argc, char *argv [])
{
    struct mnttab mntbuf; /* Estructura para leer del fichero mnttab. */
    struct filsys sbbuf; /* Estructura para leer el superbloque. */
    int mnttabfd, sbfd;
    char sbdev [50];

    /* Análisis de los parámetros de la línea de órdenes. */
    if (argc != 1) {
        fprintf (stderr, "%s: demasiados parámetros\n", argv [0]);
        exit (-1);
    }

    /* Apertura del fichero /etc/mnttab. */

```

```

if ((mnttabfd = open ("/etc/mnttab", O_RDONLY)) == -1) {
    perror ("/etc/mnttab");
    exit (-1);
}

/* Lectura y procesamiento de las entradas de /etc/mnttab. */
while (read (mnttabfd, &mntbuf, sizeof (mntbuf)) == sizeof (mntbuf))
    if (strcmp (mntbuf.mnt_dev, "") != 0) {
        printf ("Sistema /dev/%s\n", mntbuf.mnt_dev);
        printf (" Montado en %s\n", mntbuf.mnt_filsys);
        printf (" Modo: %s\n", mntbuf.mnt_ro_flg == 1 ?
                "sólo lectura": "lectura/escritura");
        printf (" Fecha de montaje: %s",
                asctime (localtime (&mntbuf.mnt_time)));

        /* Apertura del fichero de dispositivo asociado al sistema de ficheros. */
        sprintf (sbdev, "/dev/%s", mntbuf.mnt_dev);
        if ((sbfd = open (sbdev, O_RDONLY)) == -1) {
            printf (
                "\007!!! ACCESO AL SISTEMA [%s] DENEGADO !!!\007\n",
                sbdev);
            printf ("Pulse ENTER para continuar.");
            while (putchar (getchar ()) != 10);
        }

        /* El puntero de lectura/escritura se sitúa en el segundo bloque.
        El primer bloque lo ocupa el programa de 'boot'. */
        else if (lseek (sbfd, BSIZE, SEEK_SET) == -1)
            perror (sbdev);

        /* Lectura del superbloque. */
        else if (read (sbfd, &sbbuf, sizeof (sbbuf)) != sizeof (sbbuf))
            perror (sbdev);
        else {
            /* Información sobre bloques. */
            /* Número de bloques de 512 bytes. */
            printf ("\tBloques: %ld = %ld bytes\n",
                    sbbuf.s_fsize*BLK512, sbbuf.s_fsize*BSIZE);
            /* Número de bloques usados. */
            printf ("\tBloques usados: %ld bloques = %ld bytes = %g%%\n",
                    (sbbuf.s_fsize - sbbuf.s_tfree)*BLK512,
                    (sbbuf.s_fsize - sbbuf.s_tfree)*BSIZE,
                    (sbbuf.s_fsize-sbbuf.s_tfree)*100.0/sbbuf.s_fsize);
            /* Número de bloques libres. */
            printf ("\tBloques libres: %ld bloques = %ld bytes\n",
                    sbbuf.s_tfree*BLK512, sbbuf.s_tfree*BSIZE);

            /* Información sobre nodos—i. */

```

```

/* Número de nodos-i. */
printf ("\tNodos-i = %ld bloques = %ld nodos-i\n",
        sbbuf.s_isize*BLK512, sbbuf.s_isize*BSIZE/ISIZE);
/* Número de nodos-i usados. */
printf ("\tNodos-i usados: %ld nodos-i = %g%%\n",
        sbbuf.s_isize*BSIZE/ISIZE - sbbuf.s_tinode,
        (sbbuf.s_isize*BSIZE/ISIZE - sbbuf.s_tinode)*100.0/
        (sbbuf.s_isize*BSIZE/ISIZE));
/* Número de nodos-i libres. */
printf ("\tInodes libres: %ld inodes\n",sbbuf.s_tinode);

/* Fecha de última actualización del superbloque. */
printf ("\tÚltima actualización superbloque: %s",
        asctime (localtime (&sbbuf.s_time)));
printf ("\n");
close (sbfd);
    }
}

close (mnttabfd);
exit (0);
}

```

Para que el programa pueda abrir todos los ficheros de dispositivo, es necesario tener atributos de superusuario. De lo contrario, puede ofrecer resultados como el siguiente:

```
$ estado-sf
```

```

Sistema /dev/root
Montado en /
Modo: lectura/escritura
Fecha de montaje: Sun Oct 25 17:41:01 1992
!!! ACCESO AL SISTEMA [/dev/root] DENEGADO !!!
Pulse ENTER para continuar.

```

donde se observa que no podemos acceder al sistema `/dev/root`. Para evitar estos inconvenientes, podemos activar el bit *Cambiar el UID en ejecución*. Esto se consigue ejecutando las siguientes órdenes como superusuario:

```
$ chown root estado-sf
```

```
$ chmod u+s estado-sf
```

Podemos ver que efectivamente el bit se activa:

```
$ ls -al estado-sf
```

```
-rwsr-xr-x 1 root users 23613 Oct 25 18:47 estado-sf
```

En estas condiciones el programa ofrece salidas como la siguiente:

\$ estado-sf

```
Sistema /dev/root
  Montado en /
  Modo: lectura/escritura
  Fecha de montaje: Sun Oct 25 17:41:01 1992
  Bloques: 178230 = 91253760 bytes
  Bloques usados: 44802 bloques = 22938624 bytes = 25.1372%
  Bloques libres: 133428 bloques = 68315136 bytes
  Nodos-i = 2788 bloques = 22304 nodos-i
  Nodos-i usados: 2772 nodos-i = 12.4283%
  Nodos-i libres: 19532 nodos-i
  Última actualización superbloque:Sun Oct 25 19:10:10 1992
```

```
Sistema /dev/fd0135ds18
  Montado en /mnt
  Modo: lectura/escritura
  Fecha de montaje: Sun Oct 25 17:45:23 1992
  Bloques: 2880 = 1474560 bytes
  Bloques usados: 260 bloques = 133120 bytes = 9.02778%
  Bloques libres: 2620 bloques = 1341440 bytes
  Nodos-i = 48 bloques = 384 nodos-i
  Nodos-i usados: 49 nodos-i = 12.7604%
  Inodes libres: 335 nodos-i
  Última actualización superbloque:Sun Oct 25 18:28:34 1992
```

4.4 Ejercicios

- 4.1 Escriba un programa de nombre `crear` que se pueda utilizar para crear ficheros especiales, tuberías con nombre y directorios. La forma de invocar al programa será:

\$ `crear -p fifo` crea una tubería de nombre *fifo*

\$ `crear -d directorio` crea un directorio de nombre *directorio*

\$ `crear -b dispositivo mayor minor` crea un fichero de dispositivo modo bloque con nombre *dispositivo* donde *mayor* y *minor* son sus *mayor* y *minor* number.

\$ `crear -c dispositivo mayor minor` crea un fichero de dispositivo modo carácter con nombre *dispositivo* donde *mayor* y *minor* son sus *mayor* y *minor* number.

Todos los nombres indicados: *fifo*, *directorio* y *dispositivo* podrán ser rutas absolutas o relativas.

Para que los usuarios sin privilegios de *root* puedan usar la orden `crear` en todos los casos, se debe activar el bit *Cambiar el UID en ejecución*:

```
$ chmod 04755 crear
```

Para codificar el número de dispositivo que se le debe pasar a la llamada `mknod` cuando crea un fichero especial, se puede usar la macro `makedev`:

```
número_dispositivo = makedev (major_number, minor_number);
```

4.2 (Linux) Aprovechando el programa `crear` del ejercicio anterior, emita la orden siguiente:

```
$ crear -b disquete 2 0
```

Inserte un disquete en la unidad de disco flexible del ordenador y repita la secuencia de órdenes siguientes:

- (a) Cree un sistema de ficheros sobre el disquete:

```
$ mke2fs disquete
```

- (b) Compruebe el estado del sistema de ficheros del disquete:

```
$ e2fsk disquete
```

- (c) Monte el sistema de ficheros que hay en el disquete en el directorio `mnt`:

```
$ mount disquete mnt
```

- (d) Copie algún fichero al disquete:

```
$ cp /usr/bin/cc mnt
```

- (e) Desmonte el sistema de ficheros que hay en `mnt`:

```
$ umount mnt
```

4.3 Escriba un programa de nombre `cambiar` que sirva para cambiar el directorio de trabajo actual. La forma de usar el programa será:

```
$ cambiar directorio
```

Cuando el programa esté terminado, ejecute las órdenes siguientes:

```
$ cambiar /usr/bin
```

```
$ pwd
```

¿Cuál es el directorio de trabajo que muestra `pwd`? ¿Por qué no conseguimos cambiar de directorio con el programa `cambiar`? ¿Por qué con la orden `cd` sí conseguimos cambiar el directorio de trabajo actual?

4.4 Escriba un programa de nombre `dir1` que sirva para visualizar por pantalla el nombre de los ficheros que haya en un conjunto de directorios. La forma de uso del programa debe ser:

```
$ dir1 [directorios]
```


Ejemplo de uso:

```
$ dir1 directorio1 directorio2
```

```
    directorio1:
        .
        ..
        fichero1
        fichero2
        fichero3
    directorio2:
        .
        ..
        fichero_a
        fichero_b
        fichero_c
        fichero_d
```

Si no se pasan parámetros, `dir1` debe mostrar el contenido del directorio de trabajo actual, por ejemplo:

```
$ dir1
```

```
.:
.
..
fichero1
fichero2
fichero3
```

4.5 Escriba el programa `dir2` que deberá ser invocado de la siguiente forma,

```
$ dir2 [ruta]
```

Este programa debe presentar por pantalla la siguiente información:

```
CONTENIDO DE: ruta_absoluta
nombre_fichero_1    tamaño [bytes] DD-MM-AA  HH:MM:SS
nombre_fichero_2    tamaño [bytes] DD-MM-AA  HH:MM:SS
...
```

donde DD-MM-AA y HH:MM:SS son la fecha y la hora de la última modificación de cada uno de los ficheros.

4.6 Escriba un programa de nombre `dir3` que sirva para visualizar por pantalla el nombre de los ficheros que haya en un conjunto de directorios. Este programa será una mejora de `dir1`. La forma de uso del programa debe ser:

```
$ dir3 [opciones] [directorios]
```

La orden `dir3` sin opciones debe trabajar lo mismo que la orden `dir1`. Con la opción `-r`, la orden `dir3` debe recorrer descendentemente todos los subdirectorios que hay en los directorios de búsqueda que se han pasado como parámetros, y presentar los nombres de los ficheros que hay en esos subdirectorios.

- 4.7 Escriba un programa de nombre `quién` que muestre por pantalla una lista de todos los usuarios que han iniciado una sesión de trabajo en nuestra máquina UNIX. A continuación podemos ver el formato de los datos que aparecen al ejecutar este programa:

```
$ quién
```

USUARIO	TERMINAL
Francisc	tty2
root	tty1
curso0	tty4
Francisc	tty3

- 4.8 Escriba un programa de nombre `pregonar` que sirva para enviar un mensaje a todos los usuarios que han iniciado una sesión de trabajo en la misma máquina UNIX que nosotros. Este programa debe funcionar de forma parecida a la orden estándar `wall`. La sintaxis de nuestro programa será:

```
$ pregonar [fichero]
```

donde `fichero` contiene el texto del mensaje que se enviará a todos los usuarios; en el caso de que no se indique el fichero que se va a pregonar, el mensaje se leerá del flujo estándar de entrada. Por ejemplo:

```
$ pregonar Don_Quijote.II.XXXII
```

El mensaje que recibirán los usuarios es:

```
MENSAJE PROCEDENTE DEL USUARIO Francisc
```

```
Capítulo XXXII
```

```
De la respuesta que dio don Quijote a su reprehensor,  
con otros graves y graciosos sucesos.
```

```
Levantado, pues, en pie don Quijote, temblando de los  
pies a la cabeza como azogado, con presurosa y turbada  
lengua, dijo:
```

```
-El lugar donde estoy, y la presencia ante quien me hallo,  
y el respeto que siempre tuve y tengo al estado que vuesa  
merced profesa, tienen y atan las manos de mi justo enojo; y  
así por lo que he dicho como por saber que saben todos que  
las armas de los togados son las mismas que las de la mujer,  
que son la lengua, entraré con la mía en igual batalla con
```

vuesa merced, de quien se debía esperar antes buenos consejos que infames vituperios...

Cervantes.

"Segunda parte del Ingenioso Caballero don Quijote de la Mancha"

<FIN DEL MENSAJE>

4.9 En la definición de la función `fflush` se indica que produce un resultado indeterminado cuando trabaja sobre un flujo de entrada. Sería muy cómodo poder extender la funcionalidad de `fflush` para limpiar la memoria intermedia del teclado y eliminar los caracteres basura que quedan en ella al mezclar llamadas a funciones de la biblioteca estándar tales como `scanf`, `getchar` o `gets`. Para responder a esta necesidad, se pide escribir la función `limpiar_teclado` que tiene la declaración siguiente:

```
void limpiar_teclado (void);
```

El objetivo de esta función es limpiar todos los caracteres que haya en las memorias intermedias asociadas al flujo estándar de entrada. Recordemos que la entrada desde `stdin` involucra al menos dos memorias intermedias: la del teclado y la asociada al flujo `stdin`. Por ello, para que la llamada a `limpiar_teclado` trabaje correctamente, será necesario vaciar ambas memorias.

Con el programa siguiente podemos probar el funcionamiento de `limpiar_teclado`:

```
#include <stdio.h>

void limpiar_teclado (void)
{
    // Ésta es la función que se debe escribir.
}

main ()
{
    char nombre [256], dirección [256];
    int teléfono;
    int c;

    printf ("Nombre: ");
    gets (nombre);
    printf ("Teléfono: ");
    scanf ("%d", &teléfono);
    printf ("Dirección: ");
    limpiar_teclado ();
    gets (dirección);
    printf ("%s:\n", nombre);
    printf ("\tTeléfono: %d\n", teléfono);
```

```
    printf ("\tDirección: %s\n", dirección);
}
```

- 4.10** Los sistemas basados en el UNIX de Berkeley suministran funciones para trabajar con el fichero `/etc/mnttab`. Las más utilizadas son: `setmntent`, `getmntent` y `endmntent`. En este ejercicio se pretende implementar las funciones anteriores de acuerdo con la siguiente interfaz:

```
#include <stdio.h>
#include <mntent.h>
FILE *setmntent (char *filename, char *type);
struct mntent *getmntent (FILE *filep);
int endmntent (FILE *filep);
```

La función `setmntent` se utiliza para abrir el fichero que contiene la tabla de sistemas montados. Este fichero es `/etc/mnttab`, pero podemos especificar otro a través del parámetro `filename`. `type` indica el modo de apertura del fichero y su sintaxis es la misma que ya vimos para `fopen`. Si la función se ejecuta correctamente, devolverá un flujo asociado al fichero abierto.

La función `getmntent` lee una entrada del fichero asociado al flujo `filep`. La entrada leída se devuelve a través de un puntero a una estructura de tipo `struct mntent`. Su definición es la siguiente:

```
struct mntent {
    char *mnt_fsname; // Nombre del sistema de ficheros; es decir, el
                      // fichero de dispositivo asociado al sistema de ficheros
    char *mnt_dir; // Directorio sobre el que está montado el sistema
                  // de ficheros
    char *mnt_type; // Tipo de sistema de ficheros: hfs, nfs, swap, etc.
    char *mnt_opts; // Indicadores: sólo lectura, lectura/escritura, etc.
    int mnt_freq; // Frecuencia, medida en días, de realización de
                  // las copias de seguridad
    int mnt_passno; // Número de estados que se van a desarrollar en
                   // paralelo durante la ejecución de fsck
    long mnt_time; // Fecha en la que se montó el sistema
};
```

El puntero devuelto por `getmntent` referencia una zona de datos estática que se sobrescribe con cada llamada a la función.

La función `endmntent` cierra el fichero asociado al flujo `filep`.

En el fichero de cabecera `<mntent.h>` se definen las siguientes constantes:

```
#define MNTOPT_RO "ro" // Sistema montado en modo sólo lectura
#define MNTOPT_RW "rw" // Sistema montado en modo lectura/escritura
#define MNT_MNTTAB "/etc/mnttab"
```

Implemente las funciones anteriores atendiendo a la estructura del fichero `mnttab` con el que estemos trabajando.

- 4.11 (Linux) Escriba un programa de nombre `sdf1` que tenga una funcionalidad parecida al programa estándar `df`. Si recordamos, el programa `df` lee el superbloque de los sistemas de ficheros que hay montados e informa sobre el estado de ocupación de los mismos. A continuación se muestra un ejemplo de la información que debe presentar el programa `sdf1`:

```
$ sdf1
```

```
Sistema /dev/hda2
  Montado en /
  Tipo: ext2
  Modo: rw
  Fecha de montaje: Fri Apr 14 07:05:25 1995
  Total bloques: 102433 = 104891392 bytes
  Bloques reservados: 5121 = 5243904 bytes = 4.99937%
  Bloques usados: 89903 = 92060672 bytes = 92.3863%
  Bloques libres: 7409 = 3793408 bytes = 7.61366%
  Total nodos-i: 25688
  Nodos-i usados: 7625 = 29.6831%
  Nodos-i libres: 18063 = 70.3169%
  Última actualización superbloque: Fri Apr 14 14:18:54 1995
Sistema /dev/fd0
  Montado en /home/Francisco/mnt
  Tipo: ext2
  Modo: rw
  Fecha de montaje: Fri Apr 14 14:18:14 1995
  Total bloques: 1440 = 1474560 bytes
  Bloques reservados: 72 = 73728 bytes = 5%
  Bloques usados: 63 = 64512 bytes = 4.60526%
  Bloques libres: 1305 = 668160 bytes = 95.3947%
  Total nodos-i: 360
  Nodos-i usados: 11 = 3.05556%
  Nodos-i libres: 349 = 96.9444%
  Última actualización superbloque: Sat Mar 25 12:56:46 1995
```

Como podemos ver en el ejemplo, el programa `sdf1` presenta información sobre el grado de uso de dos sistemas de ficheros: `/dev/hda2` —segunda partición del primer disco rígido— montado en el directorio `/` y `/dev/fd0` —unidad de disco flexible— montado en el directorio `/home/Francisco/mnt`.

Para determinar los sistemas de ficheros que hay montados, se pueden leer directamente las entradas que figuran en el fichero `/etc/mntab`, sin embargo, resulta más cómodo leer esas entradas a través de las funciones estándar `setmntent`, `getmntent` y `endmntent`. Para ver el significado y forma de uso de estas funciones se pueden consultar las páginas del manual de UNIX y el ejercicio 4.10 pág. 149.

Una vez que se han determinado los sistemas de ficheros que se deben analizar, el programa `sdf1` tiene que leer el superbloque de cada uno de ellos. La estructura

del superbloque, como ya sabemos, depende de cada implementación de UNIX y no es transportable de un sistema a otro.

En el caso de Linux se contempla la posibilidad de montar diferentes tipos de sistemas de ficheros, incluso sistemas de ficheros creados por otros sistemas operativos, como Ms-DOS. Por lo tanto, hay que saber el tipo de sistema cuyo superbloque se va a leer. Si nos fijamos en el ejemplo, el tipo de los sistemas creados con Linux es `ext2`, por lo tanto, la estructura con la que debemos leer del superbloque es `ext2_super_block`. Esta estructura está definida en el fichero de cabecera `<linux/ext2_fs.h>` y su composición es la siguiente:

```
struct ext2_super_block {
    unsigned long s_inodes_count; // Contador de nodos-i
    unsigned long s_blocks_count; // Contador de bloques
    unsigned long s_r_blocks_count; // Contador de bloques reservados
    unsigned long s_free_blocks_count; // Contador de bloques libres
    unsigned long s_free_inodes_count; // Contador de nodos-i libres
    unsigned long s_first_data_block; // Primer bloque de datos
    unsigned long s_log_block_size; // Tamaño del bloque
    long s_log_frag_size; // Tamaño del fragmento
    unsigned long s_blocks_per_group; // Número de bloques por grupo
    unsigned long s_frags_per_group; // Número de fragmentos por grupo
    unsigned long s_inodes_per_group; // Número de nodos-i por grupo
    unsigned long s_mtime; // Fecha de montaje
    unsigned long s_wtime; // Fecha de escritura
    unsigned short s_mnt_count; // Contador de montajes
    short s_max_mnt_count; // 'Maximal mount count'
    unsigned short s_magic; // Número mágico
    unsigned short s_state; // Estado del sistema de ficheros
    unsigned short s_errors; // Comportamiento en caso de error
    unsigned short s_pad;
    unsigned long s_lastcheck; // Fecha de la última reparación
    unsigned long s_checkinterval; // Intervalo entre reparaciones
    unsigned long s_reserved[238]; // Relleno
};
```

Recordemos que para leer la cabecera del superbloque se deben seguir los pasos siguientes:

- (a) Abrir, en modo lectura, el fichero de dispositivo donde reside el sistema de ficheros.
- (b) Situar el puntero de lectura en el segundo bloque —el primero está ocupado por el programa de *boot*—.
- (c) Leer una estructura del tipo `ext2_super_block`.

4.12 Algunos sistemas ofrecen las llamadas `statfs` y `fstatfs`¹ para leer el estado de un sistema de ficheros. En este ejercicio se pide codificar dos funciones con la misma

¹`statfs` y `fstatfs` han sido desarrolladas por Sun Microsystems, Inc. El sistema BSD ha incorporado

operatividad que `statfs` y `fstatfs` considerando la estructura del superbloque y la forma de acceder a él. La declaración de estas funciones es la siguiente:

```
#include <sys/types.h>
#include <sys/vsf.h>
int statfs (char *path, struct statfs *buf);
int fstatfs (int fildes, struct statfs *buf);
```

donde `path` es la ruta del fichero de dispositivo asociado al sistema de ficheros, `fildes` es un descriptor que da acceso a un fichero de dispositivo abierto mediante un llamada previa a `open`, la estructura `struct statfs` se define como sigue:

```
struct statfs {
    long f_bavail; // Bloques libres para los usuarios distintos del
                  // superusuario
    long f_bfree; // Bloques libres
    long f_blocks; // Total de bloques del sistema de ficheros
    long f_ysize; // Tamaño del bloque (en bytes)
    long f_ffree; // Nro. total de nodos-i libres
    long f_files; // Nro. total de nodos-i
    long f_type; // Tipo de información. Cero para no indicar nada
    fsid_t f_fsid; // Identificador del sistema de ficheros
};
```

Dependiendo del sistema en el que trabajemos, alguno de los campos de la estructura anterior deberá quedar vacío.

4.13 Los programas que realizan el recorrido de un árbol de directorios se implementan de una forma muy cómoda escribiendo una función recursiva que se encargue de los trabajos siguientes:

- (a) Abrir el directorio que corresponde a la raíz del árbol.
- (b) Leer una por una todas las entradas del directorio.
 - i. Si la entrada leída corresponde a un fichero, procesarlo.
 - ii. Si la entrada leída corresponde a un directorio distinto de `.` y de `..`, volver a llamar a la función que recorre el árbol, pasándole como raíz el directorio correspondiente a la entrada que se está procesando.
- (c) Cerrar el directorio que corresponde a la raíz del árbol.

Queremos escribir un programa para calcular la distribución de ficheros según sus tamaños. El nombre del programa será `estadística`, y la forma de usarlo es la siguiente:

```
$ estadística [directorios]
```

estas llamadas en la versión 4.4, añadiendo campos adicionales a la estructura `struct statfs`. En este sistema las definiciones de los datos y los prototipos de las funciones aparecen en los ficheros de cabecera `<sys/param.h>` y `<sys/mount.h>`.

donde `directorios` contiene las rutas de los directorios que se van a analizar; en el caso de que no se indique ningún directorio, el programa calculará la distribución de ficheros del directorio de trabajo actual. La distribución se debe confeccionar en tramos de 10Kb según se muestra en el ejemplo:

```
$ estadística /usr
```

```
Árbol: /usr
Rango      Total    Porcentaje
[0K, 10K)   5105    82.7525%
[10K, 20K)   477     7.73221%
[20K, 30K)   191     3.09613%
[30K, 40K)    96     1.55617%
[40K, 50K)    74     1.19955%
[50K, 60K)    43     0.697034%
[60K, 70K)    34     0.551143%
[70K, 80K)    23     0.372832%
[80K, 90K)    13     0.210731%
[90K, 100K)   15     0.243151%
Mayores      98     1.58859%
```

Para escribir este programa podemos completar el siguiente fragmento de código:

```
#include <stdlib.h>
#include <fcntl.h>
#include <dirent.h>
#include <sys/stat.h>

// Constantes
#define MAX_CAMINO 4096

// Variables globales
int total_ficheros = 0;
int total_grandes = 0;
int tamaños [10];

// recorrer_árbol: función genérica para recorrer un árbol de directorios.
// Parámetros:
//   char *dir; Nombre de la raíz del árbol que se va a recorrer.
//   void (*f) (); Puntero a una función que realiza algún tratamiento con
//                 las entradas del directorio.
// Valor de retorno: Ninguno.
void recorrer_árbol (char *dir, void (*f) ()) {...}

// estadística: calcula el número total de ficheros que hay en un árbol de
//              directorios agrupándolos por tamaños de 10 Kb.
void estadística (char *camino, struct stat *estado) {...}
```



```
// presentar_resultados: presenta en pantalla una tabla con el resultado de la
// distribución calculada.
void presentar_resultados (void) {...}

main (int argc, char **argv)
{
    char directorio [MAX_CAMINO];

    // Análisis de los argumentos de la línea de órdenes
    strcpy (directorio, argc<2 ? "." : argv[1]);

    recorrer_árbol (directorio, estadística);
    // Presentación de resultados.
    printf ("Árbol: %s\n\n", directorio);
    presentar_resultados ();
}
```

- 4.14 Escriba un programa de nombre **buscar** que realice la función básica de la orden estándar **find**. La forma de usar el programa será:

```
$ buscar fichero directorio
```

donde **fichero** es el nombre del fichero que se va a buscar y **directorio** es la raíz del árbol donde se iniciará la búsqueda. Una vez localizado el fichero, se escribirá en pantalla su ruta, por ejemplo:

```
$ buscar cc /usr

/usr/bin/cc
```

Para escribir este programa se recomienda utilizar la función **recorrer_árbol** escrita en el ejercicio anterior.

- 4.15 El programa **sdf1** escrito en el ejercicio 4.11 pág. 150 sólo es válido para el sistema Linux. En este ejercicio se pide escribir un programa de nombre **sdf2** que sea independiente de la versión de UNIX en la que se compila. Para ello, hay que utilizar las funciones estándar **statfs** y **fstatfs**. Podemos encontrar información sobre estas funciones en las páginas del manual de UNIX y en el ejercicio 4.12 pág. 151. A continuación podemos ver un ejemplo con la información que debe mostrar el programa **sdf2**:

```
$ sdf2

Sistema /dev/hda2
Montado en /
Tipo: ext2
Modo: rw
Total bloques: 102433 = 104891392 bytes
```

```
Bloques reservados: 5121 = 5243904 bytes = 4.99937%
Bloques usados: 89903 = 92060672 bytes = 92.3863%
Bloques libres: 7409 = 3793408 bytes = 7.61366%
Total nodos-i: 25688
Nodos-i usados: 7625 = 29.6831%
Nodos-i libres: 18063 = 70.3169%
Sistema /dev/fd0
Montado en /home/Francisco/mnt
Tipo: ext2
Modo: rw
Total bloques: 102433 = 104891392 bytes
Bloques reservados: 5121 = 5243904 bytes = 4.99937%
Bloques usados: 89903 = 92060672 bytes = 92.3863%
Bloques libres: 7409 = 3793408 bytes = 7.61366%
Total nodos-i: 25688
Nodos-i usados: 7625 = 29.6831%
Nodos-i libres: 18063 = 70.3169%
```

- 4.16** Implemente la función de biblioteca `getpass`. Esta función se utiliza para leer la contraseña de un usuario desde el fichero `/dev/tty` y se declara como sigue:

```
char *getpass (char *prompt);
```

donde `prompt` es un mensaje que se presenta por pantalla para indicarnos que el programa está esperando la contraseña.

La función `getpass` devuelve un puntero a una zona de datos estática que contiene la contraseña leída. Esta palabra debe contener al menos 8 caracteres. No debemos olvidar que la contraseña hay que leerla sin eco local para evitar que alguien pueda verla mientras la escribimos.

- 4.17** Implemente una función con la operatividad de `ustat`. Para ello habrá que trabajar directamente con el superbloque.

Parte II

Procesos e hilos

5	Estructura de un proceso	159
6	Gestión de procesos e hilos	183
7	Señales y funciones de tiempo	239
8	Perfilado, contabilidad y depuración	309

Capítulo 5

Estructura de un proceso

«Con estas razones acabó don Quijote de cerrar el proceso de su locura, y más con las que añadió, diciendo:...»

(Cervantes, Quijote II-18)

5.1	Programas y procesos	159
5.2	Estados de un proceso	161
5.3	Tabla de procesos y área de usuario	164
5.4	Contexto de un proceso	165
5.5	Hilos y procesos ligeros	167
5.6	Distribución y planificación de procesos	171
5.7	Sistemas con requisitos de tiempo real	173

5.1 Programas y procesos

Un programa es una colección de instrucciones y de datos que se encuentran almacenados en un fichero ordinario. Este fichero tiene en su nodo-i un atributo que lo identifica como ejecutable. Puede ser ejecutado por el propietario, el grupo y el resto de los usuarios, dependiendo de los permisos que tenga el fichero.

Los usuarios pueden crear ficheros ejecutables de varias formas. Una de las más sencillas es mediante la escritura de programas para el intérprete de órdenes —ficheros *shell script* o baterías de órdenes—. Con este procedimiento se deben seguir dos pasos para obtener un programa: el primero es editar un fichero de texto que contenga una serie de líneas que puedan ser interpretadas por un intérprete de órdenes —**sh**, **csh** y **ksh** son ejemplos de este tipo de intérpretes—; el segundo es cambiar los atributos del fichero para indicar que es ejecutable; esto se consigue con la orden **chmod**. Para ejecutar un fichero así creado tendremos que arrancar un intérprete de órdenes, pasándole como parámetro el nombre de nuestra batería de órdenes. Como, por defecto, al iniciar una sesión en UNIX, se ejecuta un intérprete para dar servicio al usuario, bastará con escribir el nombre de la batería de órdenes para que empiece a ejecutarse.

Trabajar con ficheros de órdenes presenta grandes ventajas a la hora de realizar programas cortos que no sean de gran complejidad, pero es una seria limitación a la hora de afrontar el desarrollo de una aplicación de envergadura. Por ello, en la mayor parte de las ocasiones, vamos a generar ficheros ejecutables mediante la ayuda de lenguajes de alto o bajo nivel. En nuestros análisis vamos a emplear el compilador de lenguaje C.

Este mecanismo de generación ya ha sido estudiado, pero para ser exhaustivos lo vamos a exponer. Primero se debe crear un fichero de texto que contenga el código fuente de nuestro programa —este fichero tiene extensión `.c`—. El compilador de C se encarga de traducir el código fuente a código objeto que entiende nuestra máquina. El compilador crea un fichero de salida, que por defecto se llama `a.out`, y lo marca como ejecutable. El compilador puede recibir parámetros para crear el programa ejecutable con un nombre distinto de `a.out`.

La estructura de todo programa ejecutable creado por el compilador de C viene impuesta por el sistema. Para ver una descripción detallada de esta estructura, podemos consultar la página `a.out` del manual de UNIX. Grosso modo, un programa consta de las siguientes partes:

- Un conjunto de cabeceras que describen atributos del fichero.
- Un bloque donde se encuentran las instrucciones en lenguaje máquina del programa. Este bloque se conoce en UNIX como texto del programa.
- Un bloque dedicado a la representación en lenguaje máquina de los datos que deben ser inicializados cuando arranca la ejecución del programa. Aquí está incluida una indicación de cuánto espacio de memoria debe reservar el núcleo para estos datos. Tradicionalmente, este bloque se conoce como *bss* —seudo-operador del ensamblador del IBM 7090 que significa *block started by symbol*—. El núcleo inicializa, en tiempo de ejecución, esta zona a valor 0.
- Otras secciones, tales como tablas de símbolos.

Cuando un programa es leído del disco por el núcleo y es cargado en memoria para ejecutarse, se convierte en un proceso. En un proceso no sólo hay una copia del programa, también nos encontramos información de control añadida por el núcleo. Un proceso se compone de tres bloques fundamentales conocidos como segmentos:

- El *segmento de texto* contiene las instrucciones que entiende la CPU de nuestra máquina. Este bloque es una copia del bloque de texto del programa.
- El *segmento de datos* contiene los datos que deben ser inicializados al arrancar el proceso. Si el programa ha sido generado por un compilador de C, en este bloque estarán las variables globales y las estáticas. Se corresponde con el bloque *bss* del programa.
- El *segmento de pila*. Lo crea el núcleo al arrancar el proceso y su tamaño es gestionado dinámicamente por el núcleo. La pila se compone de una serie de bloques lógicos, llamados *marcos de pila*, que son introducidos cuando se llama a una función y son sacados cuando se vuelve de la función. Un marco de pila se compone

de los parámetros de la función, las variables locales de la función y la información necesaria para restaurar el marco de pila anterior a la llamada a la función —dentro de esta información se incluyen el contador de programa y el puntero de pila anteriores a la llamada a la función—. En los programas fuente no se incluye código para gestionar la pila —a menos que estén escritos en ensamblador—; es el compilador quien incluye el código necesario para controlarla.

Debido a que los procesos se pueden ejecutar en dos modos: usuario y supervisor —este último conocido también como modo *kernel*—; el sistema maneja sendas pilas por separado. La pila del modo usuario contiene los argumentos, variables locales y otros datos relativos a funciones que se ejecutan en modo usuario, y la pila del modo supervisor contiene los marcos de pila de las llamadas al sistema —estas funciones que se ejecutan en modo supervisor—.

UNIX es un sistema de tiempo compartido que permite multiproceso —ejecución de varios procesos concurrentemente—. El planificador es la parte del núcleo encargada de gestionar la CPU y determinar qué proceso pasa a ocupar la CPU en un determinado instante. Un mismo programa puede estar siendo ejecutado en un instante determinado por varios procesos a la vez.

Desde un punto de vista funcional, un proceso en UNIX es una entidad creada tras la llamada **fork**. Todos los procesos, excepto el primero —proceso número 0—, son creados mediante una llamada a **fork**. El proceso que llama a **fork** se conoce como proceso padre y el proceso creado es el proceso hijo. Todos los procesos tienen un único proceso padre, pero pueden tener varios procesos hijos. El núcleo identifica cada proceso mediante su *PID* —*process identification*—, que es un número asociado a cada proceso y que no cambia durante el tiempo de vida de éste.

El proceso 0 es especial: es creado cuando arranca el sistema, y después de hacer una llamada a **fork** se convierte en el proceso intercambiador —encargado de la gestión de la memoria virtual—. El proceso hijo creado se llama **init** y su *PID* vale 1. Este proceso es el encargado de arrancar los demás procesos del sistema según la configuración que se indica en el fichero `/etc/inittab`.

5.2 Estados de un proceso

El tiempo de vida de un proceso se puede dividir en un conjunto de estados, cada uno con unas características determinadas. En la figura 5.1 podemos ver, en un primer nivel de aproximación, los estados por los que evoluciona un proceso. Éstos son:

1. El proceso se está ejecutando en modo usuario.
2. El proceso se está ejecutando en modo supervisor.
3. El proceso no se está ejecutando, pero está listo para ser ejecutado cuando lo indique el planificador de tareas. Puede haber varios procesos simultáneamente en este estado.
4. El proceso está durmiendo. Un proceso entra en este estado cuando no puede proseguir su ejecución porque está esperando a que se complete una operación de entrada/salida.

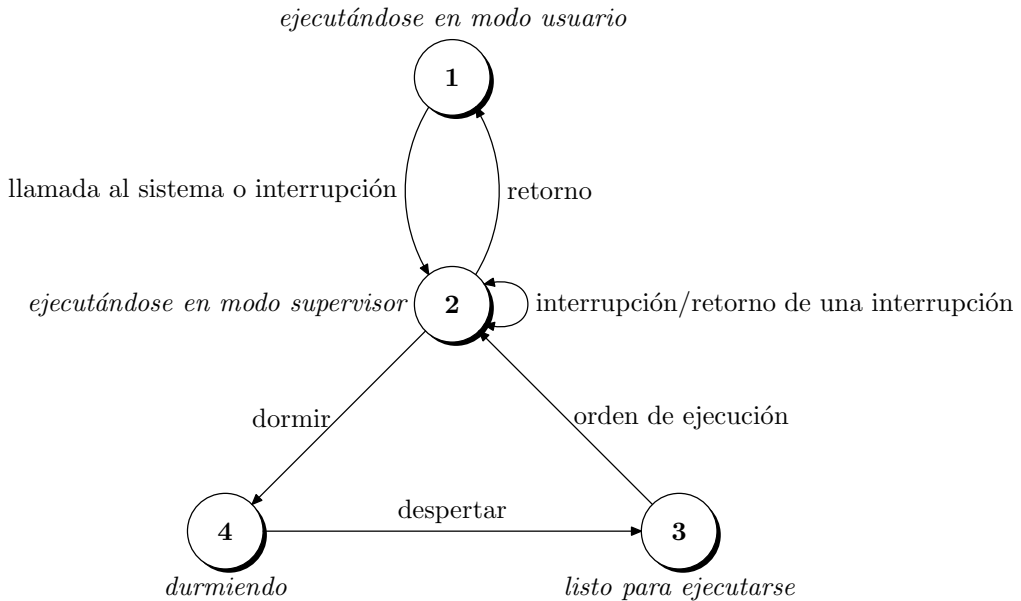


Figura 5.1: Estados de un proceso y transición entre estados.

En un sistema monoprocesador no puede haber más de un proceso ejecutándose a la vez. Así, en los dos modos de ejecución —usuario y supervisor: estados 1 y 2— sólo podrá haber un proceso en cada estado.

Un proceso no permanece siempre en un mismo estado, sino que está continuamente cambiando de acuerdo con unas reglas bien definidas. Estos cambios de estado vienen impuestos por la competencia que existe entre los procesos para compartir un recurso escaso como es la CPU. Las transiciones entre los cuatro estados descritos también quedan definidas en la figura 5.1, que en este sentido es un diagrama de transición de estados. Un diagrama de transición de estados es un grafo dirigido, cuyos nodos representan los estados que pueden alcanzar los procesos y cuyas ramas representan los eventos que hacen que un proceso cambie de un estado a otro.

Si bien la figura 5.1 muestra los cuatro estados básicos por los que puede pasar un proceso, el diagrama de estados para los procesos de UNIX es bastante más complejo. En la figura 5.2 podemos ver el diagrama completo, y los estados que en él se reflejan son:

1. El proceso se está ejecutando en modo usuario.
2. El proceso se está ejecutando en modo supervisor.
3. El proceso no se está ejecutando, pero está listo para ejecutarse tan pronto como el planificador lo ordene.
4. El proceso está durmiendo cargado en memoria.

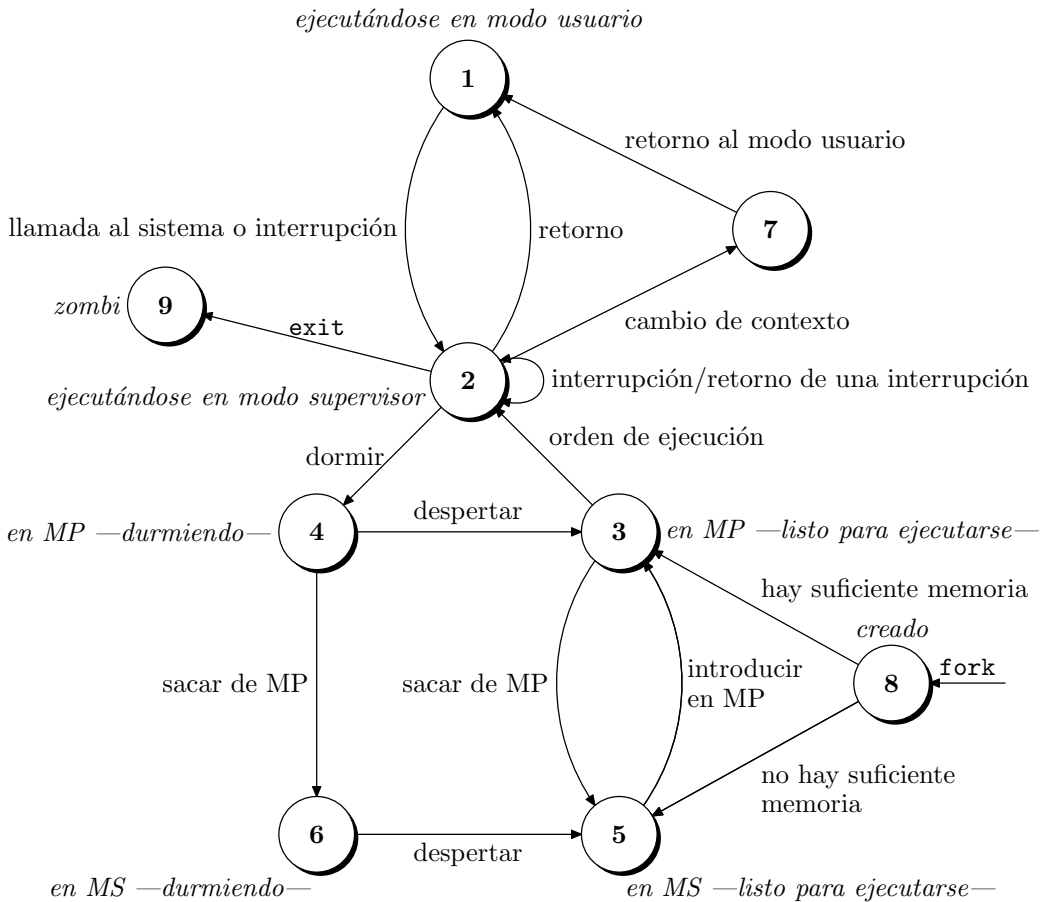


Figura 5.2: Diagrama completo de transición de estados de un proceso.

- El proceso está listo para ejecutarse, pero el intercambiador —proceso 0— debe cargar el proceso en memoria antes de que el planificador pueda ordenar que pase a ejecutarse.
- El proceso está durmiendo y el intercambiador ha descargado el proceso hacia una memoria secundaria —área de intercambio del disco— para crear espacio en la memoria principal donde poder cargar otros procesos.
- El proceso está volviendo del modo supervisor al modo usuario, pero el núcleo se apropia del proceso y hace un cambio de contexto, pasando otro proceso a ejecutarse en modo usuario.
- El proceso acaba de ser creado y está en un estado de transición; el proceso existe, pero ni está preparado para ejecutarse —estado 3—, ni durmiendo —estado 4—. Este estado es el inicial para todos los procesos, excepto el proceso 0.

9. El proceso ejecuta la llamada `exit` y pasa al estado zombi. El proceso ya no existe, pero deja para su proceso padre un registro que contiene el código de salida y algunos datos estadísticos tales como los tiempos de ejecución. El estado de zombi es el estado final de un proceso.

5.3 Tabla de procesos y área de usuario

Todo proceso tiene asociadas una entrada en la tabla de procesos y una área de usuario —*u area*—. Éstas son dos estructuras que describen el estado del proceso y que le facilitan al núcleo su control. La tabla de procesos tiene campos que son accesibles desde el núcleo, pero los campos del área de usuario sólo necesitan ser visibles por el proceso. Las áreas de usuario se reservan cuando se crea un proceso y no es necesario que una entrada de la tabla de procesos que no aloja a ningún proceso tenga reservada una área de usuario.

Los campos que tiene cada una de las entradas de la tabla de procesos son los siguientes:

- Campo de estado que identifica el estado del proceso.
- Campos para localizar el proceso y su área de usuario, tanto en memoria principal como en memoria secundaria. El núcleo usa esta información para realizar los cambios de contexto cuando el proceso pasa de un estado a otro. También utiliza esta información cuando traslada el proceso de la memoria principal al área de intercambio, y viceversa. En estos campos también hay información sobre el tamaño del proceso, para que el núcleo sepa cuánto espacio de memoria debe reservar para él.
- Algunos identificadores de usuario —*UID*— para determinar los privilegios del proceso. Por ejemplo, el campo *UID* delimita a qué procesos puede enviar señales el proceso actual y qué procesos pueden enviárselas a él.
- Identificadores de proceso —*PID*— que especifican las relaciones entre procesos. Estos identificadores son fijados cuando se crea el proceso mediante una llamada a `fork`.
- Descriptores de eventos, cuando el proceso está durmiendo y que serán utilizados al despertar.
- Parámetros de planificación que le permiten al núcleo determinar el orden en el que los procesos pasan del estado *ejecutándose en modo supervisor* a *ejecutándose en modo usuario*.
- Un campo de señales que enumera las señales recibidas, que no han sido tratadas todavía.
- Algunos temporizadores que indican el tiempo de ejecución del proceso y el tiempo de uso de los recursos del núcleo. Estos campos se usan para llevar la contabilidad del proceso y calcular la prioridad dinámica que se le asigna. Aquí también hay un temporizador que puede programar el usuario y que se usa para enviarle al proceso la señal `SIGALRM`.

El área de usuario contiene información que es necesaria sólo cuando el proceso se está ejecutando. Algunos de los campos de que consta esta zona son los siguientes:

- Puntero a la entrada de la tabla de procesos correspondiente al proceso al cual pertenece el área de usuario.
- Los identificadores de usuario real y efectivo, que determinan algunos privilegios del proceso, tales como el permiso de acceso a un fichero.
- Temporizadores que registran el tiempo empleado por el proceso ejecutándose en modo usuario y en modo supervisor.
- Un array que indica cómo va a responder el proceso a las señales que recibe.
- El terminal de inicio de sesión asociado al proceso, que indica cuál es, si existe, el terminal de control.
- Un registro de errores que indica los errores producidos durante alguna llamada al sistema.
- Un campo de valor de retorno que contiene el resultado de las llamadas efectuadas por el proceso.
- Parámetros de entrada/salida que describen la cantidad de datos a transferir, la dirección del array origen o destino de la transferencia y los punteros de lectura/escritura del fichero al que se refiere la operación de entrada/salida.
- El directorio de trabajo actual y el directorio raíz asociados al proceso —consulte `chdir` en el § 4.1.4 pág. 121—.
- La tabla de descriptores de fichero que identifica los ficheros que tiene abiertos el proceso —consulte `open` en el § 3.2.1 pág. 53—.
- Campos de límite que restringen el tamaño del proceso y de algún fichero sobre el que puede escribir.
- Una máscara de permisos que es usada cada vez que se crea un fichero —consulte `umask` en el § 3.5.2 pág. 81—.

5.4 Contexto de un proceso

El contexto de un proceso es su estado definido por: su código, los valores de sus variables de usuario globales y sus estructuras de datos, el valor de los registros de la CPU, los valores almacenados en su entrada de la tabla de procesos y en su área de usuario, y el valor de sus pilas de usuario y supervisor. El código del sistema operativo y sus estructuras de datos globales son compartidos por todos los procesos, pero no son considerados parte del contexto del proceso.

Cuando se ejecuta un proceso, se dice que el sistema se está ejecutando en el contexto de un proceso. Cuando el núcleo decide ejecutar otro proceso, realiza un cambio de

contexto y guarda la información necesaria para reanudar la ejecución del proceso interrumpido en el mismo punto donde la dejó. De igual manera, cuando el proceso cambia del modo usuario al modo supervisor, el núcleo guarda información para que el proceso pueda volver al modo usuario. Sin embargo, el cambio de modo no se contempla como un cambio de contexto.

Desde un punto de vista formal, el contexto de un proceso es la unión de su contexto del nivel de usuario, su contexto de registro y su contexto del nivel de sistema.

El contexto del nivel de usuario se compone de los segmentos de texto, datos y pila del proceso, así como de las zonas de memoria compartida que se encuentran en la zona de direcciones virtuales del proceso. Las partes del espacio de direcciones virtuales que periódicamente no residen en memoria principal debido al intercambio o a la paginación también son parte del contexto del nivel de usuario.

El contexto de registros se compone de las siguientes partes:

- El contador de programa que contiene la dirección de la siguiente instrucción que debe ejecutar la CPU. Esta dirección es una dirección virtual, tanto en modo usuario como en modo supervisor.
- El registro de estado del procesador —*PS*— que especifica el estado del hardware de la máquina. Las operaciones que causan la modificación del registro de estado son realizadas en un determinado proceso, por lo tanto este registro contiene el estado del hardware en relación con ese proceso.
- El puntero de pila apunta a la cima de la pila de usuario o de supervisor. La arquitectura de la máquina dicta si el puntero de pila debe apuntar a la siguiente entrada libre o a la última entrada usada. De igual forma, es la arquitectura de la máquina la que impone si la pila crece hacia direcciones altas o bajas de memoria.
- Los registros de propósito general contienen datos generados por el proceso durante su ejecución.

El contexto del nivel de sistema de un proceso tiene una parte estática —los tres primeros campos de la relación siguiente— y una parte dinámica —los dos últimos campos—. Todo proceso tiene una única parte estática del contexto del nivel de usuario, pero puede tener un número variable de partes dinámicas. La parte dinámica es vista como una pila de capas de contexto que el núcleo puede introducir y sacar según los eventos que se produzcan. Las partes del contexto del nivel de sistema son las siguientes:

- La entrada en la tabla de procesos. Define el estado del proceso y contiene información de control que es siempre accesible al núcleo.
- El área de usuario. Contiene información de control del proceso que necesita ser accedida sólo en el contexto del proceso.
- Entradas de la tabla de regiones por proceso —*region*—, tabla de regiones y tabla de páginas, que definen el mapa de transformación entre las direcciones del espacio virtual y las direcciones físicas. Si varios procesos comparten regiones comunes, estas regiones también son consideradas parte del contexto de cada proceso, ya que cada proceso las manipula independientemente. Es trabajo del módulo de gestión

de memoria indicar qué partes del espacio de direcciones virtuales de un proceso no están cargadas en memoria.

- La pila de supervisor, que contiene los marcos de pila de las funciones ejecutadas en modo supervisor. Si bien todos los procesos ejecutan el mismo código del núcleo, hay una copia privada de la pila del núcleo para cada uno de ellos que da cuenta de las llamadas que cada proceso hace a las funciones del núcleo. Por ejemplo, un proceso puede ejecutar la llamada `creat` y ponerse a dormir hasta que se le asigne un nodo-i, y otro proceso puede invocar a la llamada `read` y dormir hasta que se efectúe la transferencia entre el disco y la memoria. Ambos procesos están ejecutando funciones del núcleo, pero tienen pilas separadas que contienen la secuencia de llamadas que ha realizado cada uno. El núcleo debe ser capaz de recuperar el contenido de la pila del modo supervisor y la posición del puntero de pila para reanudar la ejecución de un proceso en modo supervisor. La pila del núcleo está vacía cuando el proceso se está ejecutando en modo usuario.
- La parte dinámica del contexto del nivel de sistema se compone de una serie de capas que se almacenan a modo de pila. Cada capa contiene la información necesaria para recuperar la capa anterior, incluyendo el contexto de registro de la capa anterior.

El núcleo introduce una capa de contexto cuando se produce una interrupción, una llamada al sistema o un cambio de contexto. Las capas de contexto son extraídas de la pila cuando el núcleo vuelve del tratamiento de una interrupción, cuando el proceso vuelve al modo usuario después de ejecutar una llamada al sistema o cuando se produce un cambio de contexto. La capa introducida es la del último proceso que se estaba ejecutando y la extraída es la del proceso que se pasará a ejecutar. Cada una de las entradas de la tabla de procesos contiene información suficiente para recuperar las capas de contexto.

La figura 5.3 muestra los componentes que forman el contexto de un proceso. En el lado izquierdo tenemos la parte estática del contexto y, en el lado derecho, la parte dinámica.

Un proceso se ejecuta en su contexto, o más exactamente, en su capa de contexto actual. El número de capas de contexto está limitado por el número de niveles de interrupción que soporta la máquina. Por ejemplo, si una máquina soporta 5 niveles de interrupción —interrupciones software, de terminales, de disco, de varios periféricos y del reloj—, un proceso podrá tener al menos 7 capas de contexto: una por cada nivel de interrupción, una para las llamadas al sistema y otra para el nivel de usuario. Estas 7 capas son suficientes para contener todas las capas posibles, incluso si las interrupciones se dan en la secuencia más desfavorable, ya que las interrupciones de un nivel determinado son bloqueadas —la CPU no las atiende— mientras que el núcleo está atendiendo una interrupción de nivel igual o superior.

5.5 Hilos y procesos ligeros

El modelo de procesos descrito hasta ahora presenta dos limitaciones importantes. La primera está relacionada con el hecho de que cada proceso es contemplado como una entidad dinámica que ejecuta una secuencia de instrucciones actuando sobre un conjunto de datos

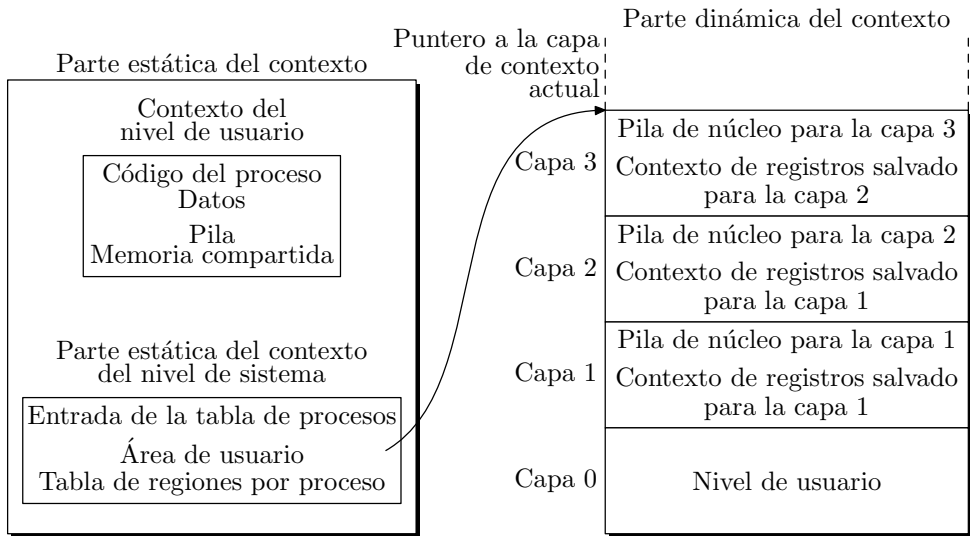


Figura 5.3: Componentes del contexto de un proceso.

locales y sin relación con el resto de procesos. Esta concepción dificulta la implementación de aplicaciones que necesitan compartir datos y recursos, y para cuya construcción, en las versiones tempranas de UNIX, se recurría a mecanismos de comunicación artificiosos. Esto se hace especialmente patente cuando las aplicaciones tienen que compartir espacios de direcciones comunes —consulte el § 10.3 pág. 379—.

La segunda limitación es la dificultad que tienen los procesos para aprovechar las ventajas de una arquitectura multiprocesador [Vahalia, 1996].

Aunque la concepción de una aplicación multiproceso no suele ser sencilla, para algunos problemas suele ser la solución más eficiente y elegante. Es por lo tanto deseable que el núcleo del sistema operativo encapsule los aspectos de creación, planificación, distribución y comunicación de los distintos flujos de ejecución secuencial de una aplicación y facilite así su creación. Las variantes modernas de UNIX abordan esta necesidad y ofrecen el mecanismo de los hilos¹ como herramienta para crear aplicaciones concurrentes de una forma más sencilla y ligera —desde el punto de vista computacional— que el mecanismo clásico de los procesos.

Los hilos no surgen para sustituir a los procesos sino para enriquecerlos y así, en aquellas variantes de UNIX que los incorporan, un proceso es una entidad compuesta por recursos y por hilos. Los recursos están relacionados con los datos del proceso y para ellos es aplicable todo lo expuesto con anterioridad. Los *hilos* son objetos dinámicos cada uno de los cuales ejecuta una secuencia de instrucciones y que comparten entre sí los recursos del proceso. En los sistemas UNIX tradicionales cada proceso se compone de un solo hilo mientras que los sistemas multihilo permiten más de un hilo por proceso. Adicionalmente, para posibilitar su planificación y distribución, cada hilo dispone de su propio contador

¹ *Hilo* es la traducción más extendida para *thread* en el campo de la informática aunque también es frecuente el uso de *hebra*.

de programa, su pila y su contexto de registros. La comunicación entre hilos a través de memoria compartida es de esta forma inmediata ya que los datos del proceso son comunes para todos ellos; solo será necesario disponer de mecanismos de sincronización para conseguir que los hilos hagan un uso adecuado de esos datos comunes.

Antes de continuar conviene establecer la diferencia entre concurrencia y paralelismo de una aplicación, para así entender mejor los diferentes tipos de hilos que implementan los sistemas operativos. La *concurrencia* de una aplicación es el número máximo de flujos de ejecución secuenciales —hilos— que podría estar ejecutando simultáneamente si dispusiera de un número ilimitado de procesadores y depende de cómo esté escrita la aplicación. El *paralelismo* es el número real de hilos que se están ejecutando simultáneamente en un momento determinado y por lo tanto está limitado por el número de procesadores y por la carga actual del sistema [Black, 1990]. Idealmente, una aplicación con concurrencia n podría alcanzar igual paralelismo si se ejecuta en una arquitectura con n o más procesadores, reduciendo además su tiempo de ejecución en un factor de $1/n$, aunque en la práctica no se alcanza nunca ese factor debido a la sobrecarga que supone crear, distribuir y sincronizar los hilos.

A la hora de implementar aplicaciones concurrentes se pueden considerar tres tipos de hilos: hilos del núcleo, procesos ligeros e hilos del usuario.

5.5.1 Hilos del núcleo

Los *hilos del núcleo* —*kernel thread*— no están asociados con ningún proceso de usuario; son creados y destruidos según las necesidades internas del núcleo y comparten el código y los datos de éste, si bien cada uno de ellos tiene su propia pila y contexto de registros. Este tipo de hilos es muy útil para atender a operaciones asíncronas como la entrada/salida o las interrupciones. Estas operaciones son tratadas de forma síncrona por parte del hilo que las gestiona pero son vistas como asíncronas por el núcleo, que se limita a crear un hilo cada vez que es necesario realizar una de ellas.

La concurrencia expresada a través de hilos ayuda así a simplificar la estructura del núcleo y por lo tanto se viene empleando desde hace bastante tiempo, de hecho, algunos de los demonios que ayudan a gestionar el sistema se comportan como hilos del núcleo.

Por otro lado, los hilos del núcleo son menos costosos de gestionar que los procesos ya que su creación no implica duplicar datos —salvo la pila y el contexto de registros— y su conmutación no exige mover datos ya que son compartidos por todos los hilos.

5.5.2 Procesos ligeros

Un *proceso ligero* —*LWP* — *lightweight process*— es un hilo del usuario sostenido por el núcleo. Desde el punto de vista del usuario, un proceso se puede componer de varios procesos ligeros, cada uno de los cuales se apoya en un hilo del núcleo. Como son visibles para el núcleo, éste se encarga de despacharlos y alojarlos en los procesadores que se encuentren libres, traduciendo así la concurrencia de una aplicación en paralelismo real. Las ventajas de los procesos ligeros son también vivibles en los sistemas monoprocesador ya que las operaciones bloqueantes, como el acceso a recursos ocupados o las operaciones de entrada/salida, bloquean hilos individuales y no al proceso en su totalidad, como ocurre con los hilos del usuario.

La planificación de los procesos ligeros es también poco costosa porque los datos que diferencian a unos de otros siguen siendo su pila y su contexto de registros y por lo tanto la conmutación de estos hilos no implica un movimiento grande de datos. No obstante, como las operaciones de creación, destrucción y sincronización de procesos ligeros se realizan mediante llamadas al sistema, y éstas son un poco más costosas en tiempo que las llamadas a funciones de bibliotecas de usuario, este tipo de hilos se ven ligeramente penalizados en tiempo frente a los hilos del usuario.

5.5.3 Hilos del usuario

Una aplicación puede también expresar su concurrencia a través de *hilos del usuario* que son gestionados por la aplicación, sin la intervención del núcleo. Esto se consigue a través de bibliotecas como *C-threads* de Mach [Cooper & Draves, 1990] o *pthread*s de POSIX [Mueller, 1993] que proporcionan las funciones necesarias para crear, sincronizar, planificar y distribuir hilos sin la intervención del núcleo.

La implementación de los hilos del usuario es posible gracias a que la pila y el contexto de registros de un hilo pueden ser salvados y restaurados sin la intervención del núcleo. Así, para desalojar un hilo y pasarle el control a otro basta con salvar la pila y el contexto de registros del primero y restaurar los del segundo; y para devolverle el control al primero repetiremos las operaciones invirtiendo el orden de los hilos.

El hecho de que los hilos del usuario no sean conocidos por el núcleo plantea dos inconvenientes. El primero es la imposibilidad de traducir la concurrencia de la aplicación en paralelismo real, ya que es el núcleo el encargado de alojar los hilos en los procesadores y esto sólo lo puede hacer con los hilos del núcleo y los procesos ligeros. Para solucionar este problema, algunos sistemas como Solaris [Powell *et al.*, 1991] implementan la posibilidad de asociar hilos del usuario con procesos ligeros, así, cuando el núcleo distribuye estos hilos también está distribuyendo sus hilos del usuario asociados.

El segundo inconveniente se plantea cuando el hilo del usuario hace una llamada al sistema. Ésta, al ser ejecutada en modo supervisor y por lo tanto por el núcleo, bloquea el proceso al que pertenece el hilo y por lo tanto bloquea todos los hilos de ese proceso. Este problema admite una doble solución: por un lado podemos asociar los hilos del usuario con procesos ligeros que, como hemos visto en el apartado anterior, no bloquean su proceso asociado cuando hacen una llamada al sistema; y por otro lado podemos limitarnos a realizar llamadas al sistema que admitan una implementación asíncrona. Éste es el caso de la entrada/salida asíncrona que permite realizar operaciones no bloqueantes. El modo de funcionamiento es el siguiente: el proceso que realiza la llamada continúa su ejecución sin esperar la respuesta y cuando termina la operación de entrada/salida es notificado a través de una señal. Esto realmente añade complejidad a las aplicaciones secuenciales pero es fácil de implementar en las aplicaciones multihilo ya que cada una de estas operaciones de entrada/salida asíncrona es vista, con la interfaz adecuada, como una operación síncrona por parte del hilo que la invoca, que se queda bloqueado al realizar la operación y que es despertado al terminar —aunque internamente, a nivel de proceso, esta operación se traduce en una operación asíncrona—. Durante el tiempo que el hilo está bloqueado, el resto de los hilos del proceso pueden seguir evolucionando.

La ventaja principal que presentan los hilos de usuario es la mejora en el rendimiento de la aplicación para conmutar de hilo, ya que no se consumen recursos del núcleo a menos que esté asociado a un proceso ligero [Stein & Shah, 1992].

5.6 Distribución y planificación de procesos

UNIX fue concebido como un sistema de tiempo compartido para ordenadores con un solo procesador. Las primeras versiones de UNIX ocupaban 42Kb y se ejecutaban, a principios de los años 70, en ordenadores DEC PDP-11/45 con un procesador de 16 bits y 144Kb de memoria [Ritchie & Thompson, 1974]. Con estas limitaciones del hardware el sistema ya era capaz de ofrecer servicios de multiprogramación y comunicación entre procesos. La evolución de UNIX no ha cesado desde sus inicios a la par de la evolución del hardware donde se ejecuta. Actualmente nos encontramos versiones de UNIX para sistemas monoprocesador, como los ordenadores personales, sistemas multiprocesador, sistemas distribuidos e incluso sistemas embebidos y con requisitos de tiempo real.

La teoría clásica sobre planificación de procesos e hilos, tanto para arquitecturas monoprocesador como multiprocesador, la podemos encontrar en obras clásicas sobre sistemas operativos como [Silberschatz *et al.*, 2001] y [Tanenbaum, 2001]. La implementación de estas técnicas en sistemas UNIX concretos aparecen descritas, entre otros, en [Bach, 1986], [McKusick *et al.*, 1996] y [Vahalia, 1996].

Aunque cada variante de UNIX presenta ligeras diferencias en la forma de distribuir los procesos, podemos decir que, en sistemas monoprocesador, la distribución se realiza generalmente con desalojo por prioridad circular —*round robin*— usando múltiples colas realimentadas. Esto quiere decir que el núcleo le asigna la CPU al primer proceso que haya en la cola de mayor prioridad y cuando la ejecución de este proceso supera un período de tiempo conocido como *rodaja* —*time quantum* o *time slice*— o realiza una operación bloqueante, es desalojado y se le concede la CPU al siguiente proceso que esté esperando en la cola de mayor prioridad. El proceso desalojado es introducido al final de la misma cola o en una nueva si el planificador decide cambiar su prioridad.

El sistema gestiona un conjunto de colas y cada una tiene una prioridad fija. A la hora de buscar un proceso para alojarlo en la CPU, el distribuidor las recorre por orden decreciente de prioridad y saca el primer proceso que encuentre.

Un parámetro de diseño importante es el valor de la rodaja temporal. Rodajas pequeñas mejoran la disponibilidad del sistema mientras que rodajas grandes mejoran el rendimiento ya que reducen las sobrecargas por cambio de contexto. A modo de ejemplo, en el sistema BSD la rodaja es de 0,1 s. desde hace 15 años. Este valor se obtuvo experimentalmente como el valor más alto de rodaja que se puede usar sin deteriorar la disponibilidad del sistema en trabajos interactivos como, por ejemplo, edición de textos.

El *planificador*² es la parte del núcleo encargada de asignar las prioridades y tiene en cuenta para ello el uso reciente de la CPU por parte de cada proceso con objeto de incrementar la prioridad de aquellos que llevan más tiempo esperando la CPU. Cada vez que un proceso cambia de prioridad, también es cambiado de cola para que el distribuidor le pueda asignar la CPU de acuerdo con su nueva prioridad.

La prioridad de un proceso se codifica a través de un número entero que varía entre 0

²Es frecuente encontrarse con el empleo del término *planificación* para referirse también a la distribución de procesos. No obstante en el *DRAE* vemos claramente que planificar algo es previo a ejecutarlo: «**planificar**. 1. tr. Trazar los planos para la ejecución de una obra», «**distribuir**. (*Del lat. distribuĕre.*) 1. tr. Dividir algo entre varias personas, designando lo que a cada una corresponde, según voluntad, conveniencia, regla o derecho.». Es el distribuidor quien reparte el tiempo de CPU entre los procesos de acuerdo con las indicaciones del planificador. Aunque también aparece en el diccionario el término *priorizar*, prefiero no emplearlo por no figurar los derivados *priorizador* y *priorización*.

—el más prioritario— y 127 —el menos prioritario—. El rango de valores reservado para la ejecución del proceso en modo supervisor es $[0,49]$ y para el modo usuario $[50,127]$; con esto se garantiza una rápida respuesta de las llamadas al sistema. Además, durante la ejecución en modo supervisor los procesos sólo son desalojados a petición propia o por bloqueo ya que los núcleos tradicionales de UNIX no son interrumpibles. Este modo de funcionamiento facilita la sincronización de procesos que comparten recursos del núcleo pero no es aceptable en sistemas con requisitos de tiempo real porque plantea el problema de la inversión de prioridades.

La prioridad en modo usuario se calcula con la expresión:

$$P_u = \min_{P_u} + \frac{t_{CPU}}{4} + 2 \times P_{nice}$$

donde P_u es el valor que estamos calculando; $\min_{P_u}$ es el valor más pequeño que puede tomar P_u , es decir, 50; t_{CPU} es una medida del uso reciente de CPU del proceso y P_{nice} ³ es un parámetro que elige el usuario en el rango $[0,39]$.

El valor de t_{CPU} es actualizado por el núcleo, incrementándose en 1 —hasta un valor máximo de 127— por cada pulso del reloj para el proceso que actualmente ocupa la CPU y decrementándose D unidades cada segundo para el resto de procesos. D se calcula con la expresión:

$$D = \frac{2 \times \overline{N_P}}{2 \times \overline{N_P} + 1}$$

donde $\overline{N_P}$ es el número promedio de procesos que están esperando la CPU durante el último segundo. Se consigue así que el valor de P_u se incremente —y por lo tanto disminuya su prioridad— para aquellos procesos que más han usado la CPU recientemente y por lo tanto se le ceda ese recurso a los que más tiempo llevan esperando.

Este esquema de asignación de prioridades dinámicas es sencillo y efectivo en sistemas de tiempo compartido donde nos encontramos con procesos interactivos con el usuario y procesos que se ejecutan en segundo plano; sin embargo, presenta algunos inconvenientes:

- Es ineficiente e introduce sobrecargas cuando hay que recalcular las prioridades de un número grande de procesos.
- No se puede reservar la CPU para un proceso determinado que así lo requiera.
- No se pueden garantizar los tiempos de respuesta de los procesos y por lo tanto no es aplicable a sistemas con requisitos de tiempo real.
- Produce inversión de prioridades entre los procesos que se ejecutan en modo supervisor porque el núcleo no es interrumpible.
- Los procesos tienen poco control sobre su prioridad: es el núcleo quien decide por ellos.

Todo esto hace que los sistemas UNIX tradicionales no puedan usarse con aplicaciones que tienen restricciones temporales estrictas y para las cuales hay que garantizar los tiempos de respuesta.

³Del inglés *nice* —amable— porque un usuario que disminuya la prioridad de sus procesos menos importantes incrementando el valor de este parámetro se mostrará amable y educado ante los demás.

5.7 Sistemas con requisitos de tiempo real

Para afrontar las necesidades de los sistemas de tiempo real ha sido necesario rediseñar el núcleo de los sistemas UNIX tradicionales y cambiar sus esquemas de distribución y planificación. Éste es el caso de SunOS [Khanna *et al.*, 1991] y del UNIX System V Release 4 —SVR4—. Algunas versiones de UNIX como RTLinux o Digital UNIX se han adaptado al estándar POSIX 1003.1b que define extensiones para sistemas de tiempo real de la interfaz básica 1003.1 —consulte la tabla 6.2 de la pág. 212—. En todas estas variantes la unidad de ejecución secuencial mínima es el hilo y se distingue entre hilos de tiempo compartido e hilos de tiempo real. Los primeros son distribuidos con desalojo por prioridad circular y planificados con prioridades dinámicas. Los segundos son distribuidos con desalojo por prioridad —incluso cuando el hilo se ejecuta en modo supervisor— y planificados con prioridad fija. Los valores de las prioridades pertenecen a conjuntos disjuntos en cada caso y garantizan una prioridad más alta para los hilos de tiempo real que para los de tiempo compartido; se minimiza así la posibilidad de que se produzca inversión de prioridades.

5.7.1 Planificación con prioridades fijas

La teoría sobre planificación de sistemas de tiempo real se ha venido desarrollando durante los últimos 30 años y, por lo que respecta a la planificación de sistemas monoprocesador, sus resultados más importantes han quedado fijados en [Liu & Layland, 1973], [Leung & Whitehead, 1982], [Sha & Lehoczky, 1986] y [Lehoczky *et al.*, 1989]. A continuación expondré un resumen de los mismos.

Vamos a considerar que un sistema de tiempo real está compuesto por un número fijo — N — de tareas⁴ periódicas independientes que se ejecutan en un sistema monoprocesador y son distribuidas con desalojo por prioridad y planificadas con prioridad fija. Cada tarea se define con unos atributos temporales que se muestran en la tabla 5.1.

Tabla 5.1: Atributos de la tarea τ_i .

Atributo	Definición
T_i	Periodo de la tarea
C_i	Tiempo de cómputo — <i>WCET</i> — <i>worst case execution time</i> —
D_i	Plazo de respuesta
P_i	Prioridad, a mayor P_i mayor prioridad
R_i	Tiempo de respuesta en el peor de los casos
U_i	Factor de utilización $U_i = T_i/C_i$

El objetivo de la planificación es asignar una prioridad distinta a cada tarea para que al distribuirlas con desalojo por prioridad se cumplan los requisitos temporales, es decir, $R_i \leq D_i$ para cada tarea.

En este escenario, el criterio de prioridades monótonas en frecuencia —también llamado prioridad al más frecuente o *RMS* —*rate monotonic scheduling*— es óptimo para

⁴En la literatura sobre sistemas de tiempo real se habla de tareas sin distinguir entre procesos e hilos

tareas periódicas independientes con plazos de respuesta iguales a su periodo $D_i = T_i$. Este criterio dice que las prioridades deben asignarse en orden inverso a los periodos, $T_i < T_j \rightarrow P_i > P_j$. El criterio es óptimo en el siguiente sentido [Sha & Lehoczky, 1986]:

«Si asignando prioridades con otro esquema garantizamos los plazos, con RMS también se garantizan. Si con RMS no se pueden garantizar los plazos, con cualquier otro esquema tampoco.»

La prueba del factor de utilización total del sistema presentada en [Liu & Layland, 1973] es condición suficiente para que todas las tareas cumplan sus requisitos temporales. Esta prueba dice que el sistema es planificable si:

$$\sum_{i=1}^N \frac{C_i}{T_i} < N \left(2^{1/N} - 1 \right)$$

Esta prueba es condición suficiente, pero no necesaria y, por lo tanto, si no se cumple no dice si el sistema es planificable o no. Podemos recurrir entonces al cálculo del tiempo de respuesta en el peor de los casos — R_i — de cada tarea. En [Lehoczky *et al.*, 1989] se demuestra que:

$$R_i = C_i + \sum_{j \in pM(i)} \left\lceil \frac{R_i}{T_j} \right\rceil \times C_j$$

donde:

$pM(i)$ es el conjunto de índices de todas las tareas cuya prioridad es mayor que la prioridad de τ_i , $pM(i) = \{j | P_j > P_i\}$.

$\lceil \bullet \rceil$ representa la función *redondeo al alza*⁵: si $m \in \mathbb{N}$ y $n \in \mathbb{N}$ entonces $c = \lceil m/n \rceil$ es el menor $c \in \mathbb{N}$ que cumple $c \times n \geq m$. Por ejemplo, $\lceil 4/2 \rceil = 2$ y $\lceil 5/2 \rceil = 3$.

Como no podemos despejar R_i de la ecuación anterior al intervenir el redondeo al alza, recurrimos al cálculo iterativo siguiente:

$$w_i^{n+1} = C_i + \sum_{j \in pM(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil \times C_j$$

tomando como valor inicial

$$w_i^0 = C_i + \sum_{j \in pM(i)} C_j$$

El cálculo termina cuando $w_i^{n+1} = w_i^n$ —decimos entonces que $R_i = w_i^n$ — o bien cuando $w_i^{n+1} > D_i = T_i$ con lo que la tarea τ_i incumple su plazo de respuesta y el sistema no es planificable.

El criterio de asignación de prioridades *RMS* deja de ser óptimo cuando alguna de las tareas cumple que $D_i < T_i$. En estas condiciones se emplea el criterio de prioridad al más urgente —*DMS—deadline monotonic scheduling*— que asigna la mayor prioridad a

⁵En muchos textos la llaman función *techo*, traducción directa de *ceiling*.

la tarea con menor plazo de respuesta, esto es: $D_i < D_j \rightarrow P_i > P_j$ y que es óptimo en este nuevo escenario [Leung & Whitehead, 1982].

La prueba del factor de utilización total ya no es aplicable para comprobar si el sistema es planificable y tenemos que recurrir nuevamente al cálculo del peor tiempo de respuesta de cada tarea. A modo de ejemplo veamos cómo planificar el conjunto de tareas descrito en la tabla 5.2.

Tabla 5.2: Sistema formado por 4 tareas con $D_i \leq T_i$.

Tarea	T(ms)	C(ms)	D(ms)
τ_1	150	5	20
τ_2	10	3	10
τ_3	50	8	50
τ_4	200	8	15

Las prioridades se asignan con criterio *DMS* y por lo tanto $P_1 = 2$, $P_2 = 4$, $P_3 = 1$ y $P_4 = 3$. Para determinar si el sistema es planificable calcularemos el peor tiempo de respuesta de cada tarea. Empezamos con τ_1 :

$$R_1 = C_1 + \sum_{j \in pM(1)} \left\lceil \frac{R_1}{T_j} \right\rceil \times C_j$$

$$w_1^0 = C_1 + C_2 + C_4 = 5 + 3 + 8 = 16$$

$$w_1^1 = C_1 + \left\lceil \frac{w_1^0}{T_2} \right\rceil \times C_2 + \left\lceil \frac{w_1^0}{T_4} \right\rceil \times C_4 = 5 + \left\lceil \frac{16}{10} \right\rceil \times 3 + \left\lceil \frac{16}{200} \right\rceil \times 8 = 5 + 3 \times 2 + 8 \times 1 = 5 + 6 + 8 = 19$$

$$w_1^2 = C_1 + \left\lceil \frac{w_1^1}{T_2} \right\rceil \times C_2 + \left\lceil \frac{w_1^1}{T_4} \right\rceil \times C_4 = 5 + \left\lceil \frac{19}{10} \right\rceil \times 3 + \left\lceil \frac{19}{200} \right\rceil \times 8 = 5 + 3 \times 2 + 8 \times 1 = 5 + 6 + 8 = 19$$

$$w_1^2 = w_1^1 = 19 < D_1 \rightarrow R_1 = w_1^2 = 19 \text{ ms}$$

Los restantes valores se calculan de igual forma. En la tabla 5.3 podemos ver un resumen de estos resultados y concluir que el sistema es planificable.

Con el programa **dms** que se muestra a continuación se puede generar una tabla con los tiempos de respuesta de sistemas como los descritos anteriormente. En este programa se implementa el algoritmo de cálculo de R_i por aproximaciones sucesivas ya mencionado.

Tabla 5.3: Resultado de la planificación de las tareas descritas en la tabla 5.2.

Tarea	T(ms)	C(ms)	D(ms)	P	R(ms)
τ_1	150	5	20	2	19
τ_2	10	3	10	4	3
τ_3	50	8	50	1	30
τ_4	200	8	15	3	14

Programa 5.1: Programa para planificar tareas con el criterio *DMS* (dms.c)

```

/****
$ dms

Planifica un sistema compuesto por  $N$  tareas y calcula sus peores tiempos de
respuesta  $R$ .
Cada tarea se define mediante sus atributos temporales:
     $T_i$  Periodo
     $C_i$  Tiempo de cómputo en el peor de los casos – WCET
     $D_i$  Plazo de respuesta
El objetivo del programa es asignarle una prioridad a cada tarea según el criterio
DMS y calcular su peor tiempo de respuesta para determinar si el sistema es
planificable o no. El sistema será planificable si se cumple  $R_i < D_i, \forall i \in \{1, \dots, N\}$ 
****/
#include <stdio.h>

typedef enum {NO, SI} boolean_t;

typedef struct {
    unsigned id, /* Identificador de la tarea. */
        T, /* Periodo. Todos los tiempos se expresan en ms. */
        C, /* Tiempo de cómputo. */
        D, /* Plazo de respuesta. */
        P, /* Prioridad. */
        R; /* Peor tiempo de respuesta. */
    boolean_t planificable;
} tarea_t;

void dms (tarea_t *tareas, int N)
{
    #define redondeo_al_alza(n,d) (((n)%(d) == 0) ? ((n)/(d)) : ((n)/(d)+1))
    int i, j;
    tarea_t aux;
    int w, w_ant, I;

```

```

/* Ordenación de las tareas según el campo D. */
for (i = 0; i < N-1; i++)
    for (j = i+1; j < N; j++)
        if (tareas[i].D < tareas[j].D) {
            aux = tareas[i];
            tareas[i] = tareas[j];
            tareas[j] = aux;
        }
/* Asignación de prioridades. */
for (i = 0; i < N; i++)
    tareas[i].P = i+1;

/* Cálculo de  $R_i$  con las expresiones:

$$w_i^0 = C_i + \sum_{j \in pM(i)} C_j$$


$$w_i^{n+1} = C_i + \sum_{j \in pM(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil \times C_j$$

*/
for (i = 0; i < N; i++) {
    w = w_ant = 0;
    /* Cálculo de  $w_i^0$  */
    for (j = i; j < N; j++)
        w += tareas[j].C;
    /* Cálculo de las aproximaciones sucesivas  $w_i^{n+1}$  */
    while (w != w_ant && w < tareas[i].D) {
        w_ant = w;
        /* Cálculo de la interferencia que producen las tareas con prioridad
        mayor que  $\tau_i$ :  $I = \sum_{j \in pM(i)} \left\lceil \frac{w_i^n}{T_j} \right\rceil \times C_j$ 
        */
        I = 0;
        for (j = i+1; j < N; j++)
            I += redondeo_al_alza(w_ant, tareas[j].T) * tareas[j].C;
        w = tareas[i].C + I;
    }
    tareas[i].R = w;
}

/* Comprobamos si se cumplen los plazos de respuesta  $R_i < D_i, \forall i \in \{1, \dots, N\}$  */
for (i = 0; i < N; i++)
    tareas[i].planificable = tareas[i].R <= tareas[i].D ? SI: NO;
}

main (int argc, char *argv [])
{
    tarea_t *tareas;
    int N; /* Número de tareas del sistema. */
    int nitems;

```



```

int i;

/* Lectura del número de tareas y reserva de memoria. */
nitems = scanf ("%d", &N);
if (nitems != 1) {
    fprintf (stderr, "%s: lectura incorrecta del número de tareas\n",
            argv [0]);
    exit (-1);
}
tareas = (tarea_t *) malloc (N * sizeof (tarea_t));
if (tareas == NULL) {
    perror (argv [0]);
    exit (-1);
}

/* Lectura de los atributos temporales de cada tarea. */
for (i = 0; i < N; i++) {
    tareas[i].id = i+1;
    nitems = scanf ("%d%d%d",&tareas[i].T,&tareas[i].C,&tareas[i].D);
    if (nitems != 3) {
        fprintf (stderr, "%s: lectura incorrecta de la tarea %d\n",
                argv [0], i+1);
        exit (-1);
    }
}

/* Asignación de prioridades y cálculo de los peores tiempos de respuesta. */
dms (tareas, N);

/* Presentación de resultados. */
printf ("-----+\n");
printf ("| Tarea----T----C----D----P----R--Planificable |\n");
printf ("-----+\n");
for (i = 0; i < N; i++)
    printf ("| %5d %5d %5d %5d %5d %5d %12s |\n",
            tareas[i].id,
            tareas[i].T,
            tareas[i].C,
            tareas[i].D,
            tareas[i].P,
            tareas[i].R,
            tareas[i].planificable ? "SI": "NO");
printf ("-----+\n");
}

```

Si ejecutamos dms para planificar el sistema de la tabla 5.2 descrito en el fichero

`dms.dat` cuyo contenido es:

```
4
150 5    20
10  3    10
50  8    50
200 8    15
```

```
$ dms < dms.dat
```

obtendremos la respuesta siguiente:

```
+-----+
| Tarea---T---C---D---P---R--Planificable |
+-----+
|      3    50    8    50    1    30          SI |
|      1   150    5    20    2    19          SI |
|      4   200    8    15    3    14          SI |
|      2    10    3    10    4     3          SI |
+-----+
```

5.7.2 Planificación con prioridades dinámicas

Todo lo expuesto anteriormente es aplicable a sistemas planificados con prioridades fijas —planificación estática—. En el caso de planificación dinámica se demuestra que la condición necesaria y suficiente para que el sistema sea planificable es que el factor de utilización total no supere la unidad, nuevamente [Liu & Layland, 1973]:

$$\sum_{i=1}^N \frac{C_i}{T_i} \leq 1$$

En estas condiciones, para conseguir un factor de utilización del 100% hay que asignar en cada instante de planificación la máxima prioridad a la tarea más urgente —criterio *EDF-earliest deadline first*— lo que exige recalcular constantemente el tiempo de cómputo consumido por la tarea para determinar cuánto falta para que concluya su plazo de respuesta.

5.7.3 Planificación de tareas dependientes

El escenario planteado en los apartados anteriores donde las tareas sólo compiten por la CPU como único recurso compartido —tareas independientes— es el menos frecuente en la realidad. Lo común es que las tareas de un sistema con requisitos de tiempo real se comuniquen entre sí y accedan a recursos compartidos, siendo esto una posible fuente de interferencias entre ellas. La *inversión de prioridades* es una de esas interferencias y afecta al tiempo de respuesta de las tareas. Se dice que un sistema presenta inversión de prioridades cuando una tarea es suspendida a la espera de que otra tarea de menor prioridad realice algún cálculo necesario y, como consecuencia, la tarea de prioridad mayor ve

alargado su tiempo de respuesta. Esta interferencia no se puede eliminar completamente, pero se han diseñado mecanismos de planificación dinámica que limitan su duración. Uno de los más utilizados es la *herencia de prioridades*, que consiste en hacer que una tarea herede la prioridad de otra tarea más prioritaria a la que bloquea. A estos mecanismos se les suele llamar *protocolos de herencia de prioridad* y algunos de ellos son:

- **Herencia simple.** Cuando una tarea está bloqueando a otra más prioritaria, hereda la prioridad de ésta y, así, la prioridad dinámica de una tarea es el máximo de su prioridad básica y las prioridades de todas las tareas bloqueadas por ella.
- **Herencia del techo de prioridad.** *CPP – ceiling priority protocol.* El *techo de prioridad* Tp_k de un recurso ρ_k es la máxima prioridad de las tareas que lo usan:

$$Tp_k = \max_{i \in \{1, \dots, N\}} usa(i, k) \times Pe_i$$

donde N es el total de tareas, Pe_i es la prioridad estática de la tarea τ_i y la función $usa(i, k)$ vale 1 si la tarea τ_i usa alguna vez el recurso ρ_k ó 0 si no lo usa nunca:

$$usa(i, k) = \begin{cases} 1, & \text{si } \tau_i \text{ usa alguna vez el recurso } \rho_k \\ 0, & \text{si no lo usa nunca} \end{cases}$$

Con este protocolo la prioridad dinámica Pd_i de una tarea τ_i es el máximo de su prioridad estática y las prioridades de las tareas a las que bloquea:

$$Pd_i = \max \left[\{Pe_i\} \cup \left(\bigcup_{k=1}^K usa(i, k) \times Tp_k \right) \right]$$

donde K es el número de recursos y $\bigcup_{k=1}^K usa(i, k) \times Tp_k$ es el conjunto de techos de prioridad de los recursos que usa la tarea τ_i , es decir, las prioridades de las tareas a las que bloquea.

En estas condiciones una tarea sólo puede bloquear un recurso si su prioridad dinámica es mayor que el techo de todos los recursos en uso por otras tareas. Conseguimos así que cada tarea se bloquee una vez como máximo en cada ciclo, se impiden los interbloqueos y también se impiden los bloqueos encadenados.

En [Sha *et al.*, 1990] se demuestra que, con el protocolo de herencia del techo de prioridad, el tiempo máximo de respuesta de cada tarea es:

$$R_i = C_i + \max_{j \in pm(i), k \in rb(i)} C_{j,k} + \sum_{j \in pM(i)} \left\lceil \frac{R_i}{T_j} \right\rceil \times C_j$$

donde $pm(i)$ es el conjunto de tareas menos prioritarias que τ_i , $rb(i)$ es el conjunto de recursos que pueden bloquear a la tarea τ_i y $C_{j,k}$ es el tiempo durante el cual la tarea τ_j accede al recurso ρ_k .

- **Herencia inmediata del techo de prioridad.** *ICCP – immediate ceiling priority protocol.* Es una variante del protocolo anterior que se utiliza por facilidad de implementación. Con este protocolo una tarea que accede a un recurso hereda inmediatamente el techo de prioridad del recurso. Se consigue así que si una tarea se bloquea lo haga al principio del ciclo. Por lo demás, las propiedades de este protocolo son idénticas a las del anterior y el tiempo de respuesta de las tareas se calcula con la misma expresión.

Capítulo 6

Gestión de procesos e hilos

«...me parece que el traducir de una lengua en otra, como no sea de las reinas de las lenguas, griega y latina, es como quien mira los tapices flamencos por el revés, que aunque se veen las figuras, son llenas de hilos que las escurecen y no se veen con la lisura y tez de la haz...»

(Cervantes, Quijote II-62)

6.1	Ejecución de programas mediante <code>exec</code>	183
6.2	Creación de procesos (<code>fork</code>)	188
6.3	Terminación de procesos (<code>exit</code> y <code>wait</code>)	189
6.4	Información sobre procesos	195
6.5	Control de la memoria asignada a un proceso	201
6.6	Creación de hilos	212
6.7	Tratamiento de los errores	215
6.8	Atributos de un hilo	217
6.9	Cancelación de hilos	220
6.10	Funciones seguras y reentrantes	225
6.11	Planificación de hilos	227
6.12	Ejercicios	233

Una vez visto el concepto de proceso y su estructura, abordaremos el estudio de las facilidades que brinda UNIX para manipular y gestionar los procesos. Lo primero que estudiaremos es la forma de invocar a un programa desde otro; esto lo conseguiremos con la llamada `exec`.

6.1 Ejecución de programas mediante `exec`

Existe toda una familia de funciones `exec` que podemos usar para ejecutar programas. Dentro de esta familia, cada función tiene su interfaz propia, pero todas tienen aspectos comunes y obedecen al mismo tipo de funcionamiento. El resultado que se consigue con

estas funciones es cargar un programa en la zona de memoria del proceso que ejecuta la llamada, sobrescribiendo los segmentos del programa antiguo con los del nuevo. El contenido del contexto del nivel de usuario del proceso que llama a `exec` deja de ser accesible y es reemplazado de acuerdo con el nuevo programa, es decir, el programa viejo es sustituido por el nuevo y nunca retornaremos a él para proseguir su ejecución, ya que es el programa nuevo el que pasa a ejecutarse. La declaración de la familia de funciones `exec` es la siguiente:

```
int execl (char *path, char *arg0, ... char *arg_n, (char *)0);
int execv (char *path, char *argv[]);
int execl_e (char *path, char *arg0, ... char *arg_n, (char *)0, char *envp[]);
int execve (char *path, char *argv[], char *envp[]);
int execlp (char *file, char *arg0, ... char *arg_n, (char *)0);
int execvp (char *file, char *argv[]);
```

En todas estas funciones, `path` apunta a la ruta —absoluta o relativa— de un fichero ordinario ejecutable y `file` apunta al nombre de un fichero ejecutable. La ruta del fichero se construye buscándolo en los directorios que se indican en la variable de entorno `PATH`. Tanto `path` como `file` se refieren a ficheros ejecutables o a ficheros de datos —batería de órdenes, también conocido como *shell script*— para un intérprete de órdenes. Si el fichero no tiene un *número mágico* que lo indentifique como directamente ejecutable, se le pasa a `/bin/sh` como un fichero de órdenes para el intérprete.

Los parámetros `arg0, ... arg_n` son punteros a cadenas de caracteres y constituyen la lista de argumentos¹ que se le pasa al nuevo programa. Por convenio, al menos `arg0` está presente siempre y apunta a una cadena idéntica a `path` o al último componente de `path`. A continuación de `arg_n` pasamos un puntero NULL —`(char *)0`— para indicar el final de los argumentos.

El parámetro `argv` es un array de cadenas de caracteres que constituyen la lista de argumentos que recibe el nuevo programa. Por convenio, `argv` debe tener al menos un elemento que apunta a una cadena idéntica a `path` o al último componente de `path`. El último elemento de `argv` es un puntero NULL que se debe interpretar como el final de `argv`.

El parámetro `envp` es un array de punteros a cadenas de caracteres que constituyen el entorno en el que se ejecutará el nuevo programa; `envp` termina también con un puntero NULL.

Si el nuevo programa que vamos a ejecutar ha sido escrito en C, entonces recibe los parámetros `arg0, ... arg_n` o `argv` y `envp` a través de la función principal `main`. Esta función se puede declarar de la forma siguiente:

```
main (int argc, char *argv, char *envp);
```

donde `argc` es el total de argumentos que recibe el programa —total de elementos de `argv`—, `argv` es un array de punteros a cada uno de los argumentos y `envp` tiene el mismo significado que hemos descrito antes. Si, por ejemplo, invocamos a un programa C mediante la siguiente línea:

¹`execl`, `execl_e` y `execlp` al igual que `printf` y `scanf` son ejemplos de funciones con un número variable de parámetros y para implementarlas es necesario emplear las funciones `va_start`, `va_arg` y `va_end` [Kernighan & Ritchie, 1988, págs. 155–159]

```
$ copiar fichero1 fichero2
```

`argc` toma el valor 3 y los contenidos de `argv` son `argv[0]="copiar"`, `argv[1]="fichero1"` y `argv[2]="fichero2"`. Como vemos, `argv[0]` contiene el nombre del programa. Esto es así porque el intérprete de órdenes se encarga de llamar a `exec` con `arg0="copiar"` o `argv[0]="copiar"`. En la figura 6.1 se puede ver una representación gráfica de la estructura de `argv`.

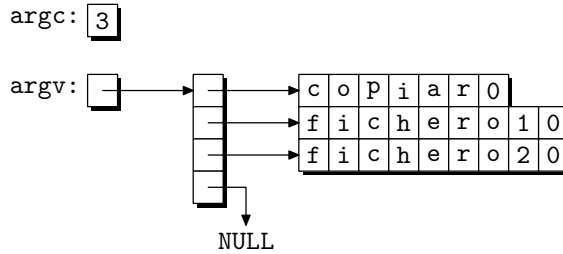


Figura 6.1: Representación de los argumentos `argc` y `argv`.

Si `exec` no se ejecuta correctamente devuelve el valor `-1` y en `errno` estará el código del tipo de error producido. En caso de que se ejecute correctamente, `exec` no le devuelve el control al programa que la llamó ya que éste es sustituido por un nuevo programa. A esta acción se la llama recubrimiento.

La estructura de un fichero ejecutable se define en la sección 4 del manual de UNIX —consulte `a.out(4)`—. Todo programa consta de una cabecera principal y una serie de secciones, cada una de las cuales se compone a su vez de una cabecera y una zona de datos. En la tabla 6.1 podemos ver la estructura lógica de todo programa. Los programas son generados por el enlazador —programa `ld`— y sus partes son:

1. La *cabecera principal*, que contiene el total de secciones del programa, la dirección de inicio de ejecución y el número mágico del programa que identifica qué tipo de fichero es.
2. *Cabeceras de sección* que describen cada una de las secciones del fichero. Contienen el tamaño de la sección, el total de direcciones virtuales que ocupará durante su ejecución y otra información.
3. Secciones con el *código* del programa y las *variables globales*.
4. Secciones con *tablas de símbolos*, información para el depurador, etc.

Como ejemplo de aplicación de dos de las funciones de la familia `exec`, vamos a escribir un programa que consulta el estado de un fichero cambiante y que ejecuta una acción determinada cuando el fichero sufre su última modificación. La forma de invocar a este programa será:

```
$ esperar-por nombre_fichero [-t timeout] [orden]
```

El programa lee el estado del fichero `nombre_fichero` y espera el tiempo indicado por `timeout`; si el fichero no ha sufrido ningún cambio, entonces ejecuta la secuencia de

Tabla 6.1: Composición de un fichero ejecutable.

Cabecera principal	Número mágico
	Número de secciones
	Valores iniciales de los registros
Cabecera de la sección 1	Tipo de sección
	Tamaño de la sección
	Espacio de direcciones virtuales
Cabecera de la sección 2	Tipo de sección
	Tamaño de la sección
	Espacio de direcciones virtuales
...	...
Cabecera de la sección n	Tipo de sección
	Tamaño de la sección
	Espacio de direcciones virtuales
Sección 1	Datos de la sección —por ejemplo, código—
Sección 2	Datos de la sección
...	...
Sección n	Datos de la sección
	Otra información

órdenes indicada en `orden`; en caso contrario, vuelve a esperar otros `timeout` segundos. Si no especificamos ningún valor para `timeout`, se deben tomar 60 segundos por defecto.

Programa 6.1: Espera por un acontecimiento (`esperar-por.c`)

```
/**
$ esperar-por nombre_fichero [-t timeout] [orden]
Ejemplo: esperar-por fichero -t 20 cat fichero
```

Los parámetros tienen que introducirse en el orden indicado.

```
***/
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

main (int argc, char *argv [])
{
    int fd, timeout;
    struct stat buf;
    time_t ultima_fecha = 0;
    char **orden;
```

```
/* Análisis de los parámetros de la línea de órdenes y apertura del fichero. */
if (argc < 2) {
    fprintf (stderr,
        "Forma de uso: %s nombre_fichero [-t timeout] [orden]\n",
        argv [0]);
    exit (-1);
}
if ((fd = open (argv [1], O_RDONLY)) == -1) {
    perror (argv [1]);
    exit (-1);
}
if (strcmp (argv [2], "-t") == 0)
    timeout = atoi (argv [3]);
else
    timeout = 60; /* Tiempo de espera por defecto. */

/* Observación de la fecha de última modificación del fichero. Si durante el tiempo
indicado en timeout esta fecha no varía, se considera que el fichero ha dejado de
sufrir modificaciones. */
fstat (fd, &buf);
while (buf.st_mtime != ultima_fecha) {
    ultima_fecha = buf.st_mtime;
    sleep (timeout);
    fstat (fd, &buf);
}

/* Determinación de la orden a ejecutar. */
if (argc == 3)
    orden = &argv [argc - 1];
else if (argc >= 5)
    orden = &argv [4];
else {
    /* Caso de que no se haya especificado ninguna orden. */
    printf ("Fichero %s estable\n", argv [1]);
    exit (0);
}

/* Ejecución de la orden. */
execvp (*orden, orden);

/* Esta línea se ejecuta sólo en el caso de que no se haya podido
ejecutar execvp. */
perror (argv [0]);
}
```

6.2 Creación de procesos (fork)

La forma de crear un proceso en el sistema UNIX es mediante la llamada `fork`. El proceso que invoca a `fork` se llama *proceso padre* y el proceso creado es el *proceso hijo*. La declaración de `fork` es:

```
#include <sys/types.h>
pid_t fork ();
```

y la forma de invocarla es `pid = fork ()`. La llamada a `fork` hace que el proceso actual se duplique. A la salida de `fork`, los dos procesos tienen una copia idéntica del contexto del nivel de usuario excepto el valor de `pid`, que para el proceso padre toma el valor de `PID` del proceso hijo y para el proceso hijo toma el valor 0. El proceso 0, creado por el núcleo cuando arranca el sistema, es el único que no se crea con una llamada a `fork`. Si la llamada a `fork` falla, devolverá el valor -1 y en `errno` estará el código del error producido. Una secuencia de código típica para manejar la llamada a `fork` es la siguiente:

```
int pid;
...
if ((pid = fork()) == -1)
    perror ("Error en la llamada a fork");
else if (pid == 0)
    // Código que ejecutará el proceso hijo
else
    // Código que ejecutará el proceso padre
```

Como primer ejemplo de uso de `fork` veamos un programa que crea un proceso hijo y a continuación tanto el padre como el hijo escribirán continuamente en pantalla:

```
Soy el proceso PADRE
    Soy el proceso HIJO
Soy el proceso PADRE
    Soy el proceso HIJO
...
```

Programa 6.2: Ejemplo de uso de `fork` (`fork.c`)

```
#include <sys/types.h>
main ()
{
    int i = 0;

    switch (fork()) {
    case -1:
        perror ("Error al crear procesos");
        exit (-1);
        break;
    case 0: /* Código para el hijo. */
        while (i<10) {
```

```
        sleep (1);
        printf ("\t\tSoy el proceso hijo: %d\n", i++);
    }
    break;
default: /* Código para el padre. */
    while (i<10) {
        printf ("Soy el proceso padre: %d\n", i++);
        sleep (2);
    }
};
exit (0);
}
```

Cuando llamamos a `fork`, el núcleo realiza las siguientes operaciones:

1. Busca una entrada libre en la tabla de procesos y la reserva para el proceso hijo.
2. Asigna un identificador de proceso —*PID*— para el proceso hijo. Este número es único e invariable durante toda la vida del proceso y es la clave para poder controlarlo desde otros procesos.
3. Realiza una copia del contexto del nivel de usuario del proceso padre para el proceso hijo. Las secciones que deban ser compartidas, como el código o las zonas de memoria compartida, no se copian, sino que se incrementan los contadores que indican cuántos procesos comparten esas zonas.
4. Las tablas de control de ficheros locales al proceso —como puede ser la tabla de descriptores de fichero— también se copian del proceso padre al proceso hijo, ya que forman parte del contexto del nivel de usuario. En las tablas globales del núcleo —tabla de ficheros y tabla de inodes— se incrementan los contadores que indican cuántos procesos tienen abiertos esos ficheros.
5. Retorna al proceso padre el *PID* del proceso hijo, y al proceso hijo le devuelve el valor 0.

En la figura 6.2 podemos ver el resultado de la ejecución de `fork`.

6.3 Terminación de procesos (`exit` y `wait`)

Una situación muy típica en programación concurrente es que el proceso padre espera a la terminación del proceso hijo antes de continuar su ejecución. Un ejemplo de esta situación es la forma de operar de los intérpretes de órdenes. Cuando escribimos una orden, el intérprete arranca un proceso para ejecutarla y no devuelve el control hasta que no se ha ejecutado completamente. Naturalmente, esto no es aplicable cuando la orden se ejecuta en segundo plano.

Para sincronizar los procesos padre e hijo se emplean las llamadas `exit` y `wait`. La declaración de `exit` es la siguiente:

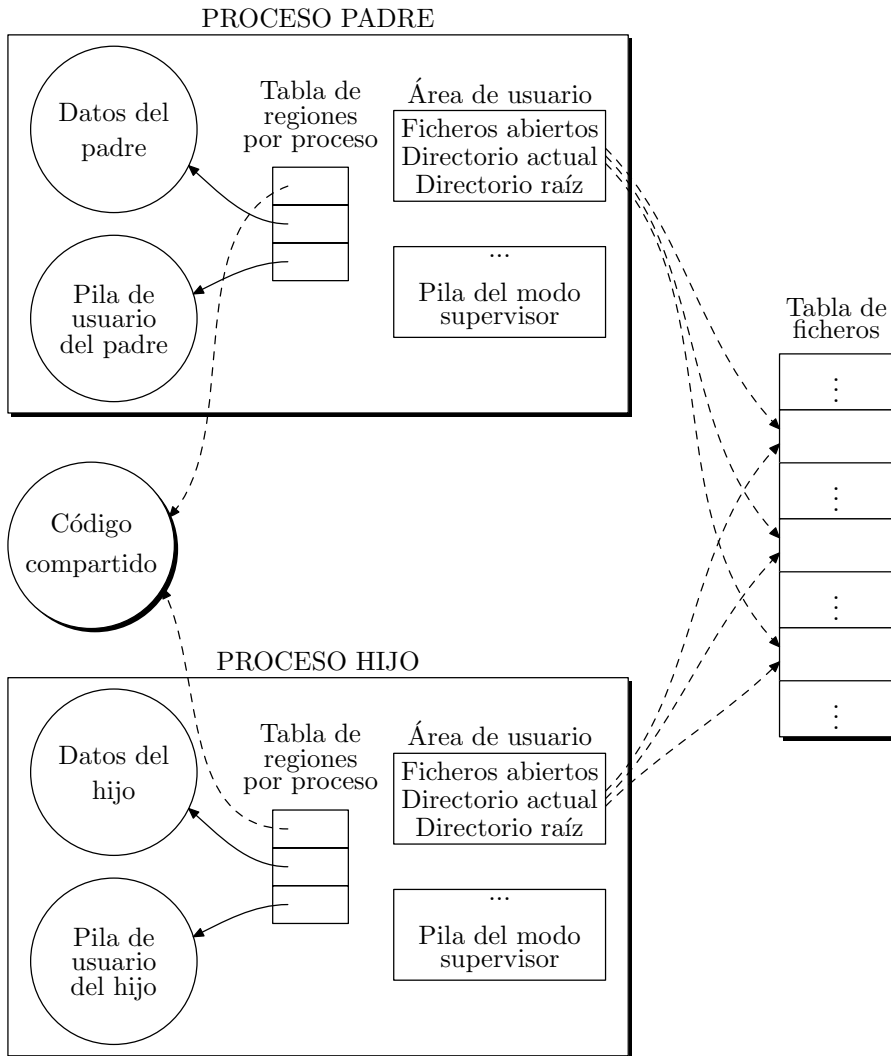


Figura 6.2: Creación de un nuevo contexto de proceso mediante `fork`.

```
#include <stdlib.h>
void exit (int status);
```

Esta llamada termina la ejecución de un proceso y le devuelve el valor de `status` al sistema. Este valor lo podemos consultar a través de la variable de entorno `?`. Un retorno efectuado desde la función principal —`main`— de un programa C tiene el mismo efecto que la llamada a `exit`. Si efectuamos el retorno sin devolver ningún valor en concreto, el resultado devuelto al sistema estará indefinido.

La llamada a **exit** tiene, además, las siguientes consecuencias:

- Las funciones registradas por **atexit** son invocadas en orden inverso a como fueron registradas. **atexit** permite indicarle al sistema las acciones que se deben ejecutar al producirse la terminación de un proceso.
- El contexto del proceso es descargado de memoria, lo que implica que la tabla de descriptores de ficheros es cerrada y sus ficheros asociados cerrados, si no quedan más procesos que los tengan abiertos.
- Si el proceso padre está ejecutando una llamada a **wait**, se le notifica la terminación de su proceso hijo y se le envían los 8 bits menos significativos de **status**. Con esta información, el proceso padre puede saber en qué condiciones ha terminado el proceso hijo.
- Si el proceso padre no está ejecutando una llamada a **wait**, el proceso hijo se transforma en un proceso *zombi*. Un proceso zombi sólo ocupa una entrada en la tabla de procesos del sistema y su contexto es descargado de memoria.

exit es uno de los pocos ejemplos de llamada que no devuelve ningún valor. Es lógico, ya que el proceso que la llama deja de existir después de haberla ejecutado.

La declaración de **wait** es la siguiente:

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait (int *stat_loc);
```

Esta llamada suspende la ejecución del proceso que la invoca hasta que alguno de sus procesos hijo termina. La forma de invocar a **wait** es:

```
pid_t pid;
int estado;
...
pid = wait (&estado);
```

```
// También vale
pid = wait (NULL);
```

donde **pid** es el identificador de alguno de los procesos hijo zombi y **estado** es la variable donde se almacena el valor que el proceso hijo le envía al proceso padre mediante la llamada a **exit** y que da idea de la condición de finalización del proceso hijo. Si queremos ignorar este valor, podemos pasarle a **wait** un puntero **NULL**.

Puede ocurrir que el proceso hijo termine de forma anormal. Para estos casos hay definidas, en el fichero **<sys/wait.h>**, unas macros que analizan el valor de **estado** para determinar la causa de terminación del proceso. Estas macros son:

- **WIFEXITED**, devuelve **VERDAD** —cualquier valor distinto de 0— cuando el proceso termina con una llamada a **exit** o a **_exit**.
- **WEXITSTATUS**, si **WIFEXITED** devuelve **VERDAD**, esta macro devuelve el valor de los 8 bits menos significativos que **exit** le pasa al proceso padre.

- **WIFSIGNALED**, devuelve VERDAD cuando el proceso termina debido a la acción por defecto de alguna señal —consulte `signal` y el § 7.3.2 pág. 245—.
- **WTERMSIG**, si **WIFSIGNALED** devuelve VERDAD, esta macro devuelve el número de la señal que ha causado la terminación del proceso.
- **WCOREDUMP**, devuelve VERDAD si se ha generado un fichero con un volcado de la memoria —*core image*— del proceso.
- **WIFSTOPPED**, devuelve VERDAD si el proceso está parado.
- **WSTOPSIG**, si **WIFSTOPPED** devuelve VERDAD, esta macro devuelve el número de la señal que ha causado la parada del proceso.

Si durante la llamada a `wait` se produce algún error, la función devuelve `-1` y en `errno` estará el código del tipo de error producido.

Como he indicado antes, `exit` y `wait` se suelen usar para conseguir que el proceso padre espere a la terminación de su proceso hijo. Una secuencia de código para realizar esta operación puede ser:

```
pid_t pid;
int estado;
...
if ((pid = fork()) == -1)
    // Error en la creación del proceso hijo
else if (pid == 0) {
    // Código del proceso hijo
    exit (10);
}
else {
    // Código del proceso padre
    pid = wait (&estado);
    // Cuando el proceso hijo llame a exit, le pasará al padre el valor 10,
    // que éste puede recibir a través de estado (estado=10)
}
```

La figura 6.3 ilustra la secuencia de código anterior.

En el programa siguiente se muestra un proceso principal que crea 5 hijos a los que se les pasa un código identificador y espera hasta su terminación. Cada hijo ejecuta un bucle donde se escribe 50 veces por la salida estándar un mensaje que muestra su código identificador `id`, el valor de la variable local `i` junto con el valor de la variable global `I`. Cada proceso tiene una copia de esas variables y al incrementarlas se incrementa cada una de las copias, por eso tanto `i` como `I` varían entre 0 y 49.

Programa 6.3: Creación y terminación de procesos hijo (`procesos.c`)

```
/**
 * $ procesos
 */
#include <sys/types.h>
```

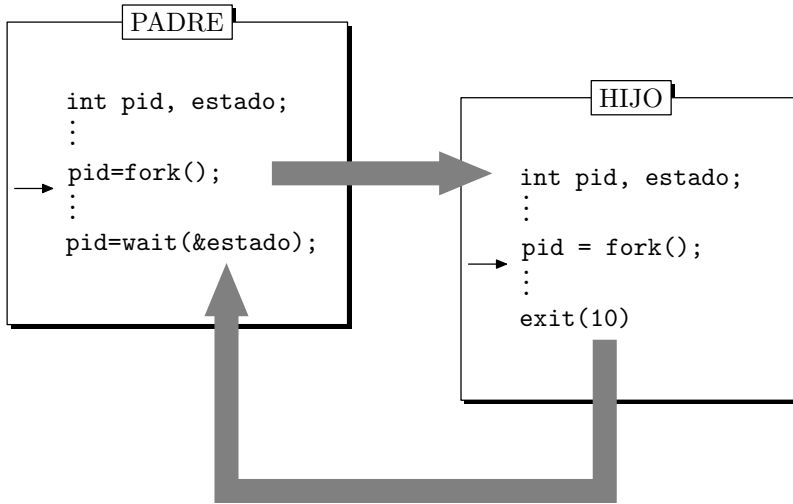


Figura 6.3: Sincronización entre los procesos padre e hijo.

```

#include <sys/wait.h>
#include <stdlib.h>

#define NUM_PROCESOS 5
int I = 0;

void codigo_del_proceso (int id)
{
    int i;

    for (i = 0; i < 50; i++)
        printf ("Proceso %d: i = %d, I = %d\n", id, i, I++);
    exit (id);
    /* El valor de id se almacena en los bits 8 al 15 antes de devolverlo al proceso
    padre. */
}

main()
{
    int p;
    int id [NUM_PROCESOS] = {1, 2, 3, 4, 5};
    int pid;
    int salida;

    for (p = 0; p < NUM_PROCESOS; p++) {
        pid = fork ();

```



```

        if (pid == -1){
            perror ("Error al crear un proceso: ");
            exit (-1);
        } else if (pid == 0) /* Código del proceso hijo. */
            codigo_del_proceso (id [p]);
    }
    /* Esta parte sólo la ejecuta el proceso padre. */
    for (p = 0; p < NUM_PROCESOS; p++) {
        pid = wait (&salida);
        printf ("Proceso %d con id = %x terminado\n", pid, salida >> 8);
        /* El id del proceso devuelto con la llamada a exit se almacena en los bits
        8 al 15 de salida, por eso lo extraemos desplazándolos con salida >> 8.*/
    }
}

```

Como ejemplo práctico de aplicación se muestra a continuación el código de la función estándar `system`. Esta función se declara de la forma:

```

#include <stdlib.h>
int system (const char *string);

```

donde `string` es un puntero a una cadena de caracteres que debe contener una orden para el intérprete de órdenes del sistema. Así, la llamada,

```
system ("ls -al");
```

despliega por pantalla el contenido del directorio actual. El código para esta función puede ser el siguiente:

Programa 6.4: Codificación de la función `system` (`sistema.c`)

```

#include <stdio.h>
#include <fcntl.h>

/**
 * sistema: función con la misma operatividad que system
 */
sistema (const char *orden)
{
    int pid, tty, estado, w;

    fflush (stdout);
    /* Apertura del terminal asociado al proceso. */
    if ((tty = open ("/dev/tty", O_RDWR)) == -1) {
        perror ("sistema");
        return (-1);
    }
    if ((pid = fork ()) == 0) {
        /* Código para el proceso hijo. */

```

```

/* Reasignamos los ficheros estándar de entrada, salida y errores del proceso
hijo para independizarlos de los del proceso padre. */
close (0); dup (tty);
close (1); dup (tty);
close (2); dup (tty);
close (tty);
/* Ejecución de la orden deseada. El parámetro -c le indica al intérprete sh
que orden no es un fichero del órdenes para el intérprete. */
execlp ("sh", "sh", "-c", orden, NULL);
/* Código de salida en caso de que no se pueda ejecutar execlp. */
exit (127);
}
/* Código para el proceso padre. */
close (tty);
/* Espera hasta que termine el proceso hijo. */
while ((w = wait (&estado)) != pid && w != -1);
/* El código que se recibe en estado es el de salida del programa sh. */
if (w == -1) estado = -1;
return estado;
}

/****
main: prueba la función sistema.
****/
main ()
{

    sistema ("clear");
    sistema ("ps -ef");
    sistema ("ls -al");
}

```

6.4 Información sobre procesos

En este apartado vamos a estudiar las llamadas necesarias para consultar y fijar algunos de los parámetros más importantes de un proceso, los que describen cómo se relaciona el proceso con el resto del sistema. Nos vamos a centrar en los siguientes aspectos: identificadores asociados a un proceso, usuarios asociados al proceso, variables de entorno y parámetros por defecto.

6.4.1 Identificadores de proceso

Todo proceso tiene asociados dos números desde el momento de su creación: el identificador de proceso y el identificador del proceso padre.

El identificador de proceso —*PID*— es un número entero positivo que actúa a modo de nombre del proceso. El identificador del proceso padre —*PPID*— es el *PID* del proceso que ha creado al actual. El *PID* de un proceso no cambia durante el tiempo de vida de éste; sin embargo, su *PPID* sí puede variar. Esta situación se da cuando el proceso padre muere, pasando el *PPID* del proceso hijo a tomar el valor 1 —*PPID* del proceso *init*—.

Para leer los valores de *PID* y *PPID* utilizaremos las llamadas `getpid` y `getppid`, respectivamente:

```
#include <types.h>
pid_t getpid ();
pid_t getppid ();
```

Las llamadas a estas funciones no van a fallar nunca.

En UNIX, los procesos están agrupados en conjuntos con alguna característica común —por ejemplo, tener un mismo proceso padre—. A estos conjuntos se les conoce como grupos de procesos y desde el sistema son controlados a través de un identificador de grupo de procesos. Para determinar a qué grupo pertenece un proceso utilizaremos la llamada `getpgrp`:

```
#include <sys/types.h>
pid_t getpgrp ();
```

El identificador de grupo de procesos es heredado por los procesos hijo después de una llamada a `fork`, pero también puede cambiarse creando un nuevo grupo de procesos. Esto lo conseguimos con la llamada `setpgrp`:

```
#include <sys/types.h>
pid_t setpgrp ();
```

Esta llamada hace que el proceso actual se convierta en el líder de un grupo de procesos. El identificador de este grupo coincide con el *PID* del proceso y es el valor devuelto por `setpgrp`. Si la llamada falla, devolverá el valor -1. Insistiremos más sobre los grupos de procesos cuando estudiemos las señales —consulte `signal(2)` y el § 7.3.2 pág. 245—.

El ejemplo siguiente muestra un programa que crea un proceso hijo. Padre e hijo van a mostrar su *PID*, *PPID* y su identificador de grupo de procesos. Pasados 5 segundos, el padre terminará, y el hijo mostrará sus nuevos *PID*, *PPID* e identificador de grupo de procesos. Pasados 10 segundos, el hijo se va a convertir en el líder de un nuevo grupo de procesos y volverá a mostrar sus identificadores.

Programa 6.5: Uso de las llamadas de identificación de procesos (`pid.c`)

```
#include <stdio.h>

main ()
{
    if (fork () == 0) {
        /* Código del proceso hijo. */
        printf ("Proceso hijo. PID = %d, PPID = %d, ID de grupo = %d\n",
                getpid (), getppid (), getpgrp ());
```

```

    sleep (10);
    /* Como el proceso padre ya ha terminado, el proceso hijo es adoptado
    por el proceso con PID=1 */
    printf ("Hijo. PID = %d, PPID = %d, ID de grupo = %d\n",
           getpid (), getppid (), getpgrp ());
    /* El proceso hijo se convierte en el líder de un nuevo grupo de procesos. */
    setpgrp ();
    printf ("Hijo. PID = %d, PPID = %d, ID de grupo = %d\n",
           getpid (), getppid (), getpgrp ());
    exit (0);
}
printf ("Proceso padre. PID = %d, PPID = %d, ID de grupo = %d\n",
       getpid (), getppid (), getpgrp ());
sleep (5);
printf ("Proceso padre terminado\n");
exit (0);
}

```

6.4.2 Identificadores de usuario y de grupo

El núcleo le asocia a cada proceso dos identificadores de usuario y dos identificadores de grupo. Los identificadores de usuario son el identificador del usuario real —*UID*— y el identificador del usuario efectivo —*EUID*—. Para el grupo, están el identificador del grupo real —*GID*— y el identificador del grupo efectivo —*EGID*—.

El *UID* identifica al usuario que es responsable de la ejecución del proceso y el *GID* al grupo al cual pertenece el usuario.

El *EUID* se usa para determinar el propietario de los ficheros recién creados, comprobar la máscara de permiso de acceso a ficheros y los permisos para enviar señales a otros procesos. El identificador de usuario efectivo permite acceder a ficheros de otros usuarios. Normalmente, el *UID* y el *EUID* coinciden, pero si un proceso ejecuta un programa que pertenece a otro usuario y que tiene activo el bit *S_ISUID* —*cambiar el identificador del usuario al ejecutar*—, el proceso cambia su *EUID* que toma el valor del *UID* del nuevo usuario. Es decir, a efectos de comprobación de permisos de usuario, tendrá los mismos permisos que tiene el usuario cuyo *UID* coincide con el *EUID* del proceso.

Con respecto al identificador de grupo efectivo, se aplica la misma norma, y el *EUID* es el *GID* del grupo al cual pertenece el usuario indicado en el *EUID*.

Para determinar qué valores toman estos identificadores, nos valemos de las llamadas: **getuid**: devuelve el identificador de usuario real; **geteuid**: devuelve el identificador del usuario efectivo; **getgid**: devuelve el identificador del grupo real; y **getegid**: devuelve el identificador del grupo efectivo. La declaración de cada una de estas funciones es:

```

#include <sys/types.h>
uid_t getuid ();
uid_t geteuid ();
gid_t getgid ();
gid_t getegid ();

```

Para cambiar los valores que toman estos identificadores, podemos usar las llamadas `setuid` y `setgid`, cuyas declaraciones son:

```
#include <sys/types.h>
int setuid (uid_t uid);
int setgid (gid_t gid);
```

Tanto `setuid` como `setgid` presentan un comportamiento dependiente del valor que tenga el *EUID* del proceso. En el caso de `setuid`, si el identificador de usuario efectivo es el del superusuario, el núcleo cambia el *UID* y el *EUID* del proceso para que tomen el valor del parámetro `uid`. Si el identificador del usuario efectivo no es el del superusuario, pero el valor del parámetro `uid` coincide con el del identificador del usuario real, *EUID* toma el valor `uid`. Esto se hace para restaurar el valor de *EUID* después de que el proceso ejecuta algún programa que tiene activo el bit *S_ISUID*. En estos casos, `setuid` devuelve 0 para indicar que no se ha producido ningún error. En cualquier otro caso devolverá el valor -1 y en *errno* estará el código del error producido.

Para `setgid` se aplica lo dicho en `setuid`, pero referido al *GID* y al *EGID*. En concreto, si el *EUID* del proceso es el del superusuario, entonces el *GID* y el *EGID* cambian para tomar el valor del parámetro `gid`, y si no lo es, pero `gid` tiene el valor *GID*, entonces el *EGID* toma el valor `gid`. En cualquier otro caso se producirá un error.

Un ejemplo típico de programa que usa estas llamadas es el programa `login`. Este programa se ejecuta con el *EUID* del usuario *root* —superusuario—. Después de preguntarnos nuestro nombre de usuario y nuestra contraseña, consulta en el fichero `/etc/passwd` cuáles son nuestro *UID* y *GID*, para hacer sendas llamadas a `setuid` y `setgid` y que los identificadores *UID*, *EUID*, *GID* y *EGID* pasen a ser los del usuario que quiere iniciar la sesión de trabajo. Luego llama a `exec` para ejecutar un intérprete de órdenes que nos dé servicio. Este intérprete se va a ejecutar con los identificadores de usuario y grupo —tanto reales como efectivos— del usuario que ha iniciado la sesión.

Los procesos hijo heredan el *UID*, *EUID*, *GID* y el *EGID* del padre.

6.4.3 Variables de entorno

Cuando un proceso empieza a través de una llamada a `exec`, el sistema pone a su disposición un array de cadenas de caracteres conocido como entorno —consulte `environ`—.

Para los programas que se ejecutan mediante una llamada a `execl`, `execv`, `execlp` o `execvp`, el entorno es accesible únicamente a través de la variable global `environ`, que se declara:

```
extern char **environ;
```

Para las llamadas `execle` y `execve`, el entorno es accesible también a través del tercer parámetro de la función `main`, el parámetro `envp`:

```
main (int argc, char **argv, char **envp);
```

Tanto `environ` como `envp` tienen estructura de array de cadenas de caracteres terminado con un puntero `NULL`. Por convenio, cada una de estas cadenas tienen la forma:

```
VARIABLE_ENTORNO=VALOR_VARIABLE
```

Algunas de las variables usadas por programas estándar son:

- **HOME**, directorio de inicio de sesión de un usuario.
- **LANG**, identifica el idioma del Soporte en Lengua Nativa.
- **PATH**, indica la secuencia de directorios en los que programas como **sh**, **time**, **nice**, **nohup**, etc. van a buscar cuando se especifica un fichero mediante su nombre y no mediante su ruta.
- **TERM**, identifica el tipo de terminal hacia el que se dirige la salida de los programas. Es usado por programas como **vi**, **ed**, **mm**, etc.

Podemos ver el valor de todas las variables de entorno asociadas a nuestra sesión de trabajo mediante la orden **env**. Un ejemplo de entorno puede ser:

```
$ env
_=/bin/env
CCOPTS=-g
HOME=/users/francisc
PWD=/users/francisc
SHELL=/bin/ksh
MAIL=/usr/mail/francisc
EDITOR=vi
LOGNAME=francisc
PS1=ws2:$PWD>
TERM=vt100
PATH=./bin:/usr/bin:/usr/contrib/bin:/usr/local/bin
TZ=MST7MDT
```

El entorno de un proceso es heredado por todos sus procesos hijo, después de la llamada **fork**.

El ejemplo siguiente muestra la forma que puede tener la orden **env**:

```
#include <stdio.h>

main ()
{
    extern char **environ;

    while (*environ)
        puts (*environ++);
}
```

Para preguntar por el valor de una variable de entorno determinada o para declarar nuevas variables, podemos usar las funciones que ofrece la biblioteca estándar. Estas funciones son **getenv** y **putenv**:

```
#include <stdlib.h>
char *getenv (char *name);
int putenv (char *string);
```

La función `getenv` busca en la zona de variables de entorno una cadena de caracteres que tenga la forma `name=value` y devuelve un puntero a `value` en esta zona. Si la cadena no existe, devuelve un puntero `NULL`. Ejemplos de llamadas a `getenv` son:

```
char *valor;
...
valor = getenv ("TERM"); // 1
valor = getenv ("TERM="); // 2
valor = getenv ("TERM=vt100"); // 3
```

En estos ejemplos, si la variable `TERM` vale `300h`, las líneas 1 y 2 devuelven `valor="300h"`, pero la 3 devuelve `NULL`. Sólo en el caso de que `TERM` valga `vt100`, la sentencia 3 devolverá `valor="vt100"`.

La función `putenv` permite declarar una nueva variable de entorno o modificar el valor de una ya existente. El parámetro `string` debe tener la forma `"name=value"`. Hay que tener presente que la zona de memoria a la que apunta `string` es añadida al entorno y que por tanto se producirá un error si `string` es una variable automática —local a una función—, ya que al abandonar la función dejará de existir. Hay que advertir también que `putenv` manipula el entorno referenciado por `environ`, pero no el referenciado en `envp` —tercer argumento de `main`—. Ejemplos de llamadas a `putenv` son:

```
char *var1 = "TERM=vt100";
char *var2="NOMBRE=Francisco";
...
putenv (var1);
putenv (var2);
```

Por último, indicamos que las variables de entorno son locales a cada proceso, de tal forma que las modificaciones que se realicen sobre ellas no se conservan cuando muere el proceso. Esto no es aplicable a las variables que han sido declaradas como globales con la orden `export`.

6.4.4 Parámetros relativos a ficheros

Todo proceso tiene asociado un directorio de trabajo y un directorio raíz. El directorio de trabajo indica dónde van a estar referidos los accesos a ficheros que se realicen mediante un nombre de fichero y no mediante una ruta. Así, por ejemplo, si un proceso tiene como directorio de trabajo actual el directorio `/usr/local/bin` y abre el fichero `datos` con la llamada

```
fd = open ("datos", O_RDONLY);
```

el fichero que se está abriendo tiene `/usr/local/bin/datos` por ruta. Como hemos visto en los capítulos dedicados al sistema de ficheros —en el § 4.1.4 pág. 121—, la llamada `chdir` puede cambiar el directorio de trabajo asociado a un proceso.

El directorio raíz indica cuál es, dentro del sistema de ficheros, el directorio que considera el proceso como su directorio raíz. Por lo general coincide con /, pero lo podemos cambiar con la llamada `chroot`. Así, la secuencia de código siguiente permite acceder al fichero `/users/local/bin/prueba` indicando `/bin/prueba` como ruta:

```
chroot ("/users/local");  
fd = open ("/bin/prueba", O_WRONLY);  
// Realmente está abriendo /users/local/bin/prueba
```

Relacionado con la creación de ficheros, cada proceso tiene una máscara que indica qué bits, dentro de los bits de permiso en el modo del fichero, deben estar inactivos. La llamada `umask` —consulte el § 3.5.2 pág. 81— permite fijar esta máscara. Así, por ejemplo, en la secuencia de código siguiente vamos a intentar crear un fichero con los permisos `rw-rw-rw-` —lectura, escritura y ejecución para el propietario, el grupo y otros—, pero el fichero se va a crear realmente con los permisos `rw-r--r--`, ya que la máscara de creación de ficheros se fija previamente al valor `000 011 011 (0033)`,

```
umask (0033);  
fd = open ("datos", O_WRONLY | O_CREAT, 0777);
```

Los procesos tienen limitado el tamaño máximo de los ficheros que pueden crear y el tamaño máximo de memoria que pueden tener asignada. Para consultar estos parámetros se emplea la llamada `ulimit`:

```
#include <ulimit.h>  
long ulimit (int cmd, ...); // El segundo parámetro es opcional
```

donde `cmd` le indica a `ulimit` el tipo de operación que queremos realizar. Sus posibles valores son:

- `UL_GETFSIZE`, `ulimit` devuelve el tamaño máximo —expresado en bloques de 512 bytes— de los ficheros que puede escribir el proceso. De cara a la lectura de ficheros no hay impuesto ningún límite.
- `UL_SETFSIZE`, `ulimit` fija el tamaño máximo de los ficheros que el proceso puede escribir. El tamaño se indica a través del segundo parámetro de la función, este parámetro es de tipo `long` y se expresa en unidades de bloques de 512 bytes.
- `UL_GETMAXBRK`, `ulimit` devuelve el tamaño máximo de memoria que puede ocupar el proceso —consulte `brk` en el § 6.5 pág. 202—.

6.5 Control de la memoria asignada a un proceso

Para conocer las direcciones de memoria donde se encuentran los segmentos de código y de datos de un proceso, están disponibles los siguientes 6 símbolos:

```
extern _end;  
extern end;  
extern _etext;  
extern etext;
```



```
extern _edata;  
extern edata;
```

La dirección de los símbolos `_etext` y `etext` es la primera dirección que hay después del segmento de código —también conocido como segmento de texto—, la dirección de `_edata` y `edata` es la primera que hay después de la región de datos que se inicializan al arrancar el programa y la dirección de `_end` y `end` es la primera que hay después de la región de datos que no necesitan inicialización.

Para acceder desde un programa C al valor de estos símbolos, hemos de utilizar el operador de dirección `&`. Así, `&end` será una forma correcta de acceder a la dirección donde está `end`.

La dirección asociada a `end` se conoce también como *program break*, y puede ser modificada con las llamadas `brk` y `sbrk`:

```
int brk (char *endds);  
char *sbrk (int incr);
```

Ambas llamadas se emplean para cambiar dinámicamente la cantidad de memoria reservada para el segmento de datos del proceso que las invoca.

La llamada `brk` hace que `end` tome el valor indicado en `endds` y `sbrk` añade `incr` bytes al segmento de datos a continuación del *program break*. Si `incr` toma un valor negativo, conseguiremos decrementar el tamaño dedicado al segmento de datos. Las dos llamadas inicializan a 0 la zona de datos añadida.

Si la llamada a `brk` no produce ningún error, la función devuelve 0; `sbrk` devuelve el antiguo valor del *program break* cuando se ejecuta satisfactoriamente. Ambas funciones devuelven -1 en caso de error.

6.5.1 Asignación dinámica de memoria

Como ejemplo de aplicación de las llamadas anteriores, vamos a implementar dos funciones de la biblioteca estándar para asignación dinámica de memoria. Estas funciones son `malloc` y `free`. La declaración de `malloc` es:

```
#include <stdlib.h>  
void *malloc (size_t size);
```

y se usa para reservar memoria para un bloque de al menos `size` bytes. El espacio reservado no es inicializado. Si `malloc` se ejecuta con éxito, devolverá un puntero a la zona reservada; en caso contrario —cuando no pueda reservar memoria— devolverá `NULL`.

La función complementaria de `malloc` es `free` y se declara como sigue:

```
#include <stdlib.h>  
void free (void *ptr);
```

Con esta función se libera la zona de memoria apuntada por `ptr`. Para que `free` funcione correctamente, `ptr` debe ser un puntero previamente inicializado por `malloc`. La zona liberada por `free` se añade a la lista de memoria libre del proceso y podrá ser utilizada por `malloc` en llamadas futuras. La liberación de una zona de memoria no

afecta para nada a los datos que en ella hay. Si `ptr` vale `NULL`, `free` no realiza ninguna acción.

Las funciones que vamos a implementar se llaman `rmem` —reservar memoria, será equivalente a `malloc`— y `lmem` —liberar memoria, será equivalente a `free`—. Las llamadas a `rmem` y `lmem` se pueden intercalar en cualquier orden, siempre que tengamos la precaución de que el puntero que le pasamos a `lmem` haya sido correctamente inicializado con una llamada previa a `rmem`.

El espacio libre de almacenamiento que mantiene `rmem` se gestiona como una lista circular, en la que cada elemento es un bloque de memoria libre. Cada bloque se compone de información de control —el tamaño del bloque y un puntero al bloque siguiente— y la zona de datos del bloque, que será devuelta al usuario por `rmem`. Los bloques son mantenidos en orden ascendente de dirección de almacenamiento, y el último bloque apunta al primero. Cuando se pide memoria, `rmem` recorre la lista hasta que encuentra un bloque suficientemente grande. Si éste es exactamente del tamaño solicitado, `rmem` lo saca de la lista. Si es más grande de lo solicitado, `rmem` lo divide en dos y saca de la lista un bloque del tamaño solicitado. Si no hay en la lista ninguno del tamaño deseado, `rmem` pide más memoria al sistema y la añade a la lista de bloques libres. La petición de memoria se hace mediante una llamada a `sbrk` que incrementa el tamaño del segmento de datos del proceso, y de ello se encarga la función `masmem`.

La liberación de memoria se realiza buscando en la lista de bloques libres cuál es el lugar adecuado para insertar el bloque que queremos liberar. Como al reservar el bloque mediante `rmem` se conservó junto con la zona de datos una cabecera que almacenaba el tamaño del bloque solicitado, `lmem` conocerá en todo momento cuál es el tamaño del bloque que se desea liberar; `lmem` también consulta si el bloque insertado es adyacente a alguno de los que ya había, para así crear un bloque mayor y evitar los problemas que provocaría la microfragmentación de la memoria. Determinar la adyacencia es fácil, porque la lista se mantiene en orden ascendente de direcciones.

En la figura 6.4 podemos ver la estructura que tiene cada uno de los bloques de memoria y la estructura de lista circular que se emplea para gestionarlos.

A continuación mostramos el código correspondiente a las funciones `rmem`, `lmem` y `masmem`.

Fichero 6.6: Gestión dinámica de memoria (`rmem.c`)

```
#include <stdio.h>

/* Tipo más restrictivo en cuanto a alineación. */
typedef long _alineacion;

/* Estructura para montar la lista circular de bloques de memoria libres. */
typedef union cabecera {
    struct {
        /* Tamaño de este bloque en múltiplos del tamaño de Cabecera. */
        unsigned longitud;
        /* Puntero al siguiente bloque libre. */
        union cabecera *sig;
    } cab;
};
```

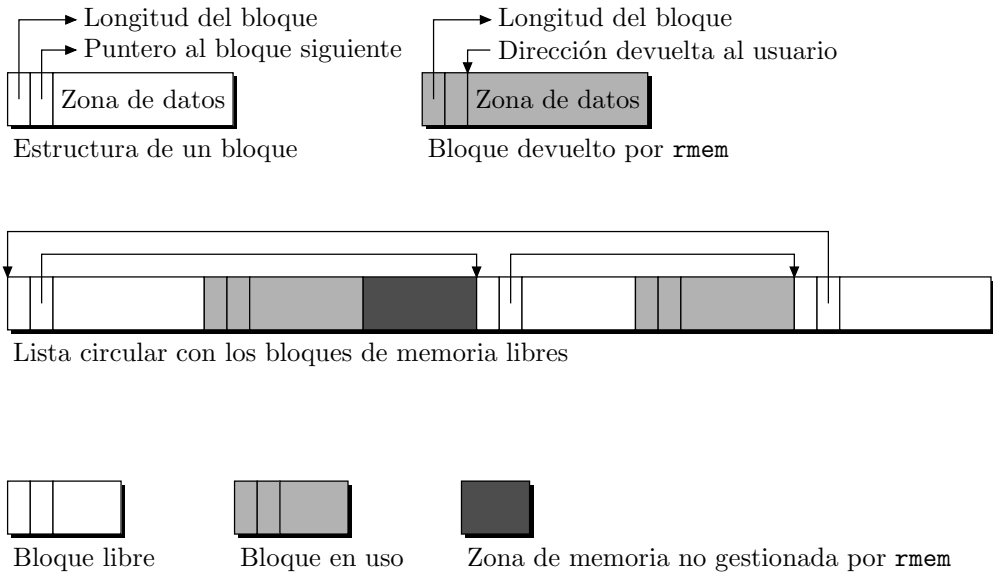


Figura 6.4: Estructuras empleadas por `rmem` y `lmem` para gestionar la memoria.

```

/* Este campo fuerza la alineación de cabecera al tipo más restrictivo. */
_alineacion _alineacion;
}Cabecera;

/* Variables para el control de la memoria libre. */
static Cabecera base; /* Lista de bloques libres. Inicialmente está vacía. */
static Cabecera *mlibre; /* Inicio de la lista de bloques libres. */

/* Prototipo de funciones locales a este módulo. */
static Cabecera *masmem (unsigned);

/**
 * rmem: esta función devuelve un puntero a una zona de memoria de longitud
 * nbytes. Si no puede reservar memoria, devuelve un puntero NULL.
 * Como rmem es de tipo void *, el puntero devuelto debe ser convertido
 * explícitamente al tipo deseado.
 */
void *rmem (unsigned nbytes)
{
    Cabecera *p, *p_ant = mlibre;
    unsigned n_unidades;

    /* Inicialización de la lista de bloques libres. */
    if (p_ant == NULL) {

```

```

    base.cab.sig = mlibre = p_ant = &base;
    base.cab.longitud = 0;
}

```

/* Transformación de unidades. `rmem` recibe el total de bytes que hay que reservar, pero previamente hay que transformarlos al total de estructuras del tipo `Cabecera`. Esta fórmula calcula el mejor redondeo sin que se pierdan bytes:

$$n_{unidades} = \frac{n_{bytes} + tamaño_{Cabecera} - 1}{tamaño_{Cabecera}} + 1$$

```

*/
n_unidades = (nbytes+sizeof(Cabecera)-1)/sizeof(Cabecera)+1;

/* Recorrido de la lista buscando un bloque de la longitud adecuada. */
for (p = p_ant->cab.sig; ; p_ant = p, p = p->cab.sig) {
    /* Caso de encontrar un bloque que satisface los requisitos. */
    if (p->cab.longitud >= n_unidades) {
        /* Caso de ser de longitud exacta. */
        if (p->cab.longitud == n_unidades)
            p_ant->cab.sig = p->cab.sig;
        else {
            /* Partición del bloque en dos para aprovechar el sobrante. */
            p->cab.longitud -= n_unidades;
            p += p->cab.longitud;
            p->cab.longitud = n_unidades;
        }
        mlibre = p_ant;
        return (void *) (p + 1);
    }
    /* En caso de volver al principio de la lista sin haber encontrado un bloque
    de la longitud adecuada, pedimos más memoria al sistema. */
    if (p == mlibre)
        if ((p = masmem (n_unidades)) == NULL)
            return NULL;
}

}

/**
    masmem: con esta función le pedimos más memoria al sistema, en caso de ser
    concedida la insertamos en la lista de memoria libre, n_unidades viene expresado
    en unidades del tamaño de Cabecera.
***/
static Cabecera *masmem (unsigned n_unidades)
{
    # define MIN_MEMORIA 1024

```

```

Cabecera *p;
void lmem ();

if (n_unidades < MIN_MEMORIA)
    n_unidades = MIN_MEMORIA;

if((p=(Cabecera *)sbrk(n_unidades*sizeof(Cabecera)))==(Cabecera *)-1)
    /* El sistema no puede incrementar la zona de datos del proceso en la
    cantidad pedida. */
    return NULL;

p->cab.longitud = n_unidades;
/* Insertamos el bloque de memoria concedido por el sistema en la lista de
memoria libre. */
lmem ((void *) (p+1));

return mlibre;
}

/****
lmem: inserta un bloque en la lista de memoria libre. Si hay dos o más bloques
contiguos, se unifican en un solo bloque, evitando así la microfragmentación de
la memoria.
****/
void lmem (void *mem)
{
    Cabecera *bloque = (Cabecera *)mem-1, *p;

    /* Recorremos la lista de memoria buscando la posición donde insertar el bloque.
    Si no hiciésemos esta búsqueda e insertásemos el bloque en cualquier posición, no
    sería posible unir bloques contiguos para constituir un bloque mayor. */
    for (p = mlibre; bloque < p || bloque > p->cab.sig; p = p->cab.sig)
        if (p >= p->cab.sig)
            break;

    /* Unificación con el bloque superior, si es posible. */
    if (bloque + bloque->cab.longitud == p->cab.sig) {
        bloque->cab.longitud += p->cab.sig->cab.longitud;
        bloque->cab.sig = p->cab.sig->cab.sig;
    }
    else
        bloque->cab.sig = p->cab.sig;

    /* Unificación con el bloque inferior, si es posible. */
    if (p + p->cab.longitud == bloque) {
        p->cab.longitud += bloque->cab.longitud;

```

```

    p->cab.sig = bloque->cab.sig;
}
else
    p->cab.sig = bloque;

mlibre = p;
}

```

Como ejemplo de programa que se vale de las funciones de reserva de memoria, se escribirá una aplicación que gestiona una lista simplemente enlazada con estructura de cola de mensajes. En la figura 6.5 podemos ver una representación gráfica de la cola de mensajes que vamos a gestionar. Cada elemento de la cola se compone de un puntero a la zona de datos donde está el mensaje —cadena de caracteres— y un puntero al elemento siguiente de la cola. El código para este programa se muestra a continuación:

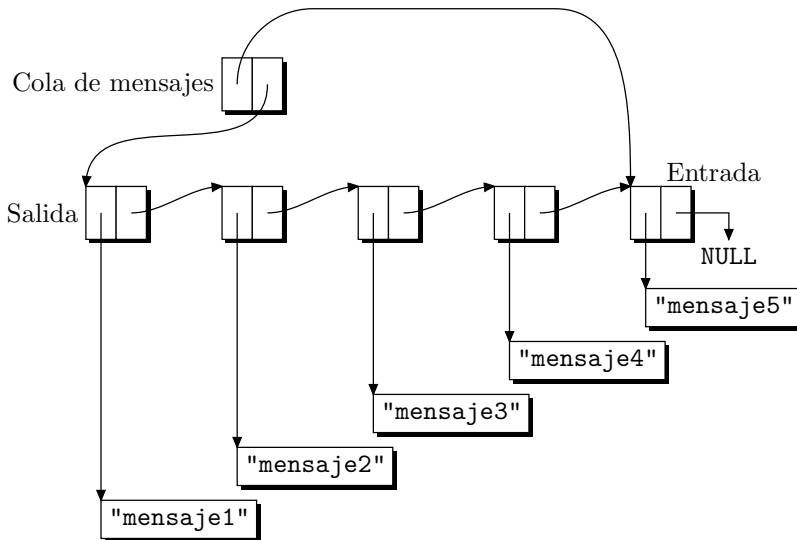


Figura 6.5: Estructura de una cola.

Programa 6.7: Aplicación de la gestión dinámica de memoria (mensajes.c)

```

/**
$ mensajes
Forma de uso:
< enviar mensaje (Orden para insertar un mensaje en la cola)
< recibir (Orden para extraer un mensaje de la cola)
> mensaje (Mensaje que se extrae)
< salir (Orden para salir del programa)
***/
#include <stdio.h>

```

```

#include <ctype.h> /* Para strcmp y strcpy. */

#define MAX 256
/* Macro para comparar dos cadenas de caracteres. */
#define EQ(str1, str2) (strcmp ((str1), (str2)) == 0)

/* Definición de los elementos de la cola de mensajes. */
struct mensaje {
    char *contenido;
    struct mensaje *siguiente;
};

/* Definición de la estructura de control de la cola. */
struct cola_mensajes {
    struct mensaje *entrada, *salida;
};

/* Prototipo de las funciones. */
void enviar_mensaje ();
struct mensaje *recibir_mensaje ();
struct mensaje *crear_mensaje ();
void liberar_mensaje ();

/**
    main: se encarga de la interactividad con el usuario.
***/
main ()
{
    char str [MAX];
    struct mensaje *mensaje;
    struct cola_mensajes cola = {NULL, NULL};

    /* Bucle de lectura y ejecución de órdenes. */
    while (!0) {
        printf("< ");
        scanf ("%s", str);
        if (EQ (str, "enviar")) { /* Caso de enviar un mensaje. */
            scanf ("%s", str);
            enviar_mensaje (&cola, str);
        } else if (EQ (str, "recibir")) { /* Caso de extraer un mensaje. */
            mensaje = recibir_mensaje (&cola);
            if (mensaje) {
                printf("> %s\n", mensaje->contenido);
                liberar_mensaje (mensaje);
            } else
                printf("No hay más mensajes.\n");
        }
    }
}

```

```
        } else if (EQ (str, "salir")) /* Salir del programa. */
            exit (0);
        else
            fprintf (stderr, "[%s] desconocido.\n", str);
    }
}

/**
    enviar_mensaje: esta función introduce una cadena de caracteres en una cola
    de mensajes. Antes de introducir la cadena, se debe crear una estructura del
    tipo mensaje, que es la que vamos a introducir en la cola.
***/
void enviar_mensaje (struct cola_mensajes *cola, char *str)
{
    struct mensaje *m;

    m = crear_mensaje (str);

    /* Caso de que la cola esté vacía. */
    if (cola->entrada == NULL)
        cola->entrada = cola->salida = m;
    else {
        cola->entrada->siguiente = m;
        cola->entrada = m;
    }
}

/**
    recibir_mensaje: esta función devuelve un puntero a la cadena de caracteres que
    está ocupando la posición de salida de la cola.
***/
struct mensaje *recibir_mensaje (struct cola_mensajes *cola)
{
    struct mensaje *m = cola->salida;

    if (cola->salida == NULL)
        cola->entrada = NULL;
    else
        cola->salida = cola->salida->siguiente;

    return m;
}

/**
    crear_mensaje: esta función reserva memoria para una estructura del tipo
    mensaje e inicializa su zona de datos con la cadena de caracteres que formará
    parte del mensaje.
***/
```



```

struct mensaje *crear_mensaje (char *str)
{
    void *rmem ();
    struct mensaje *m = (struct mensaje *) rmem (sizeof (struct mensaje));

    if (m == NULL) {
        perror ("enviar_mensaje");
        exit (-1);
    }
    if((m->contenido=(char *)rmem((strlen(str)+1)*sizeof(char)))==NULL) {
        perror ("enviar_mensaje");
        exit (-1);
    }
    strcpy (m->contenido, str);
    m->siguiente = NULL;

    return m;
}

/**
    liberar_mensaje: función encargada de liberar la memoria asignada a un mensaje.
***/
void liberar_mensaje (struct mensaje *m)
{
    void lmem ();

    if (m) {
        lmem (m->contenido);
        lmem (m);
    }
}

```

Este módulo y el anterior los podemos compilar y enlazar con la orden:

```
$ cc -o mensajes mensajes.c rmem.c
```

6.5.2 *Sticky bit*

Del *sticky bit* ya hemos hablado en secciones anteriores, pero quizá no quedó totalmente claro su significado, ya que aún no habíamos introducido los conceptos relativos a procesos. Este bit es uno de los bits de la palabra de modo de un fichero. Sólo es aplicable a los ficheros ejecutables —programas— e indica que el segmento de código de este programa puede ser compartido por varios procesos. Así, cuando el núcleo tiene que liberar memoria asignada al proceso —bien porque ha terminado su ejecución, bien porque debe pasar a la zona de intercambio—, si hay otros procesos que estén compartiendo el segmento de código, este segmento no es descargado de memoria. Sólo el superusuario puede modificar el *sticky bit* de un programa.

En el caso de programas muy utilizados —como pueden ser editores o compiladores— la activación de este bit supone una optimización del trasiego de información entre el disco y la memoria.

En el sistema 4.4BSD se delega en el módulo de gestión de la memoria virtual la función de registrar los programas recientemente ejecutados y de dejar residentes en memoria su segmento de código, previendo su uso en un futuro cercano. En este sistema el *stiky bit* se utiliza con un significado distinto, en concreto indica que un usuario sin los privilegios adecuados no puede borrar o renombrar los ficheros de otros usuarios que haya en un directorio cuyo *stiky bit* esté activo [McKusick *et al.*, 1996].

6.5.3 Bloqueo de memoria

Además del *stiky bit*, hay otros mecanismos para dejar residentes otros segmentos de un programa. La llamada `plock` se utiliza con esa idea. Su declaración es:

```
#include <sys/lock.h>
int plock (int op);
```

Con `plock` podemos dejar residentes en memoria el segmento de código de un programa, su segmento de datos o ambos segmentos. Dependiendo del valor que tome `op`, indicaremos una acción u otra. En el fichero de cabecera `<sys/lock.h>` están definidos los siguientes valores para `op`:

- `PRLOCK`, para dejar en memoria los segmentos de código y de datos, *proces lock*.
- `TXTLOCK`, para dejar en memoria el segmento de código, *text lock*.
- `DATLOCK`, para dejar en memoria el segmento de datos, *data lock*.
- `UNLOCK`, para desbloquear y liberar la memoria ocupada.

Si la llamada a `plock` se realiza satisfactoriamente, la función devuelve 0; en caso contrario, devuelve -1 y en `errno` estará el código del error producido. Sólo el superusuario tiene privilegios para emplear la llamada `plock`.

Los segmentos que queden residentes en memoria son inmunes al intercambio con la memoria secundaria y por ello restan capacidad del total de memoria libre del sistema. Como la memoria es un recurso escaso, caro y codiciado por todos los procesos, conviene no abusar del bloqueo de memoria, ya que puede bajar el rendimiento del sistema.

La técnica de dejar residente, total o parcialmente, un programa, mejora mucho la velocidad de ejecución en el caso de que un programa sea invocado muchas veces. Imaginemos, por ejemplo, una batería de órdenes que hace repetidas llamadas a un programa. Si ese programa se deja residente en memoria la primera vez que se ejecuta y se extrae de ella la última vez que se le invoca, habremos suprimido, del tiempo total de ejecución de la batería, el tiempo que se emplea en llevar el programa desde el disco a la memoria. Esto puede suponer una mejora sustancial en cuanto a rendimiento.

Cuando más adelante estudiemos la memoria compartida, veremos que también hay posibilidad de bloquear una zona de memoria compartida para que no esté intercambiándose entre la zona de intercambio y la memoria principal —consulte `shmctl(2)` y el § 10.3.2 pág. 380—.

6.6 Creación de hilos

Entre las interfaces propuestas para trabajar con hilos, la conocida como *pthread*s de POSIX [IEEE, 1995] es la adoptada mayoritariamente para implementar bibliotecas de hilos en entornos UNIX. Conviene explicar qué es POSIX. La norma *IEEE Std 1003.1-1988* convertida en norma internacional *ISO/IEC 9945-1:1990* y reafirmada por el IEEE como *IEEE Std 1003.1-1990* se denomina legalmente *IEEE Std. 1003.1-1990 Standard for Information Technology—Portable Operating System Interconnection (POSIX)—PART 1: System Application Programming Interface (API) [C Language]* y es más conocida por las siglas POSIX [Lewine, 1994]. Esta norma define una interfaz entre las aplicaciones y el sistema operativo que las ejecuta y está basada en los sistemas UNIX System V y BSD. La norma no define un sistema operativo y tampoco indica cómo deben escribirse las aplicaciones, solo define la interfaz entre ambos. A este primer documento se han ido añadiendo otros que extienden la norma base, por ejemplo *IEEE Std 1003.1c*, o que definen subconjuntos de servicios para aplicaciones concretas —perfiles—, por ejemplo *IEEE Std 1003.13*. Todos ellos constituyen la familia de normas *IEEE Std 1003* que podemos ver resumida en la tabla 6.2. La última edición del documento donde se define la interfaz en lenguaje C está publicada conjuntamente por el *Institute of Electrical and Electronics Engineers Inc.* y *The Open Group* con el título *1003.1TM Standard for Information Technology — Portable Operating System Interface (POSIX®) – System Interfaces, Issue 6* [IEEE, 2001b] y puede ser consultada en www.UNIX-systems.org.

Tabla 6.2: Normas POSIX y sus extensiones.

Número	Descripción
1003.1, 1a	Interfaz entre aplicaciones y el sistema operativo
1003.1b, 1d, 1j	Extensiones para sistema de tiempo real
1003.1c	Concurrencia mediante hilos
1003.1e	Mejoras relacionadas con la seguridad
1003.1f	<i>NFS—Network File System</i>
1003.1g	Servicios de red —conectores, <i>XTI</i> —
1003.1h	Tolerancia de fallos
1003.2, 2a, 2b	Intérprete de órdenes, se inspira en <i>sh</i> , <i>csh</i> y <i>ksh</i>
1003.2d	Extensiones de seguridad para el intérprete de órdenes
1003.3	Requisitos para los bancos de prueba de las normas POSIX
2003.#	Pruebas específicas para las normas 1003.#
1003.5, 5a, 5b	Interfaces en lenguaje Ada
1003.9	Interfaces en lenguaje FORTRAN-77
1003.10	Perfiles para aplicaciones de supercomputación
1003.13	Perfiles para sistemas de tiempo real
1003.14	Perfiles para sistemas multiprocesador
1003.19	Interfaces en lenguaje Fortran 90
1003.20	Comunicaciones de tiempo real
1387	Administración del sistema

La biblioteca de hilos POSIX tiene un enfoque orientado a objetos donde cada hilo es un objeto que representa un flujo de control secuencial dentro de un proceso y al que se le asocia un identificador de tipo `pthread_t` y unos atributos de tipo `pthread_attr_t`, ambos definidos en el fichero de cabecera `<pthread.h>`, donde también aparecen los prototipos de las funciones de esta biblioteca. Las funciones para crear un hilo, obtener su identificador, reclamar los recursos que ocupa, ordenar su terminación, esperar su terminación y comparar dos hilos son:

```
#include <pthread.h>

int pthread_create (pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine)(void*), void *arg);
pthread_t pthread_self (void);
int pthread_detach (pthread_t thread);
void pthread_exit (void *value_ptr);
int pthread_join (pthread_t thread, void **value_ptr);
int pthread_equal (pthread_t t1, pthread_t t2);
```

La función `pthread_create` crea un nuevo hilo con los atributos `attr` dentro del proceso que la invoca y devuelve a través de `thread` su identificador. Si `attr` vale `NULL` se pasan los atributos por defecto. En caso de fallo la función devuelve un número distinto de 0 que indica el error producido. El hilo ejecuta el código de la rutina referenciada por `start_routine` a la que se le pasan los argumentos referenciados por `arg`.

La función `pthread_self` devuelve el identificador del hilo que la invoca.

La función `pthread_detach` le indica al sistema que los recursos de almacenamiento ocupados por el hilo con identificador `thread` tienen que ser liberados cuando el hilo termine. La llamada a esta función no produce la terminación del hilo y múltiples llamadas a la misma producen un resultado indefinido, así que no conviene llamarla más de una vez por cada hilo.

La función `pthread_exit` termina la ejecución del hilo que la invoca y le devuelve los datos referenciados por `value_ptr` al hilo que esté esperando su terminación. Esta función es también llamada de forma implícita cuando termina la rutina `start_routine` indicada al crear el hilo. El valor devuelto por esta rutina hace las funciones de valor de salida del hilo.

La función `pthread_join` suspende la ejecución del hilo que la invoca y espera hasta la terminación del hilo con identificador `thread`. El puntero devuelto por este hilo con una llamada a `pthread_exit` es almacenado en la dirección referenciada por `value_ptr`. Si un hilo intenta ejecutar `pthread_join` sobre sí mismo se detecta un abrazo mortal y la llamada a la función falla devolviendo el valor `EDEADLK`.

La función `pthread_equal` devuelve 0 si los identificadores `T1` y `T2` no son iguales y cualquier valor distinto de 0 si coinciden.

En el ejemplo siguiente se muestra cómo emplear las funciones anteriores para crear 5 hilos dentro de un proceso. El código de la función `main` es también otro hilo —a partir de ahora lo llamaré *hilo principal*—, así que en realidad el proceso se compone de 6 hilos, los 5 hilos que se crean en `main` y el hilo principal que se encarga de esperar la terminación de los otros.

Programa 6.8: Creación de hilos y espera hasta su terminación (hilos.c)

```

/**
$ hilos

Compilación:
cc -o hilos hilos.c -pthread (-lpthread en Linux)
***/
#include <pthread.h>
#include <stdio.h>

#define NUM_HILOS 5
int I = 0;

void *codigo_del_hilo (void *id)
{
    int i;

    for (i = 0; i < 50; i++)
        printf ("Hilo %d: i = %d, I = %d\n", *(int *)id, i, I++);
    pthread_exit (id);
}

main()
{
    int h;
    pthread_t hilos[NUM_HILOS];
    int id[NUM_HILOS] = {1, 2, 3, 4, 5};
    int error;
    int *salida;

    for (h = 0; h < NUM_HILOS; h++) {
        error = pthread_create(&hilos[h], NULL, codigo_del_hilo, &id[h]);
        if (error){
            fprintf (stderr, "Error %d: %s\n", error, strerror (error));
            exit (-1);
        }
    }
    for (h = 0; h < NUM_HILOS; h++) {
        error = pthread_join (hilos [h], (void **)&salida);
        if (error)
            fprintf (stderr, "Error %d: %s\n", error, strerror (error));
        else
            printf ("Hilo %d terminado\n", *salida);
    }
}

```

Cada uno de los hilos creados con `pthread_create` ejecuta la función `codigo_del_hilo` que consta de un bucle que escribe 50 veces el identificador del hilo, el valor de la variable local `i` junto con el valor de la variable global `I`. Al ser `i` una variable local varía entre 0 y 49 ya que cada hilo tiene una copia de ella y por lo tanto no es accesible desde los otros hilos. No ocurre lo mismo con `I` que es una variable global y varía entre 0 y 249 ya que es compartida por todos los hilos. Observamos aquí una de las diferencias que hay entre los hilos y los procesos. En el ejemplo de procesos creados con `fork` —consulte el § 6.3 pág. 192— veíamos que las variables globales también se duplican y cada uno de ellos tiene sus propias variables locales y globales.

6.7 Tratamiento de los errores

Una de las diferencias que presenta la biblioteca de hilos *threads* frente a las llamadas al sistema convencionales es la forma de indicar los errores que se producen durante su ejecución. Las llamadas tradicionales devuelven 0 si se ejecutan satisfactoriamente y -1 en caso de error; adicionalmente guardan el código del error en la variable global `errno`. Este mecanismo no es apropiado para el nuevo escenario multihilo descrito ya que esta variable sería compartida por todos los hilos y cada uno de ellos genera sus propios errores independientemente. Por lo tanto al diseñar la interfaz de la biblioteca se ha optado por una representación alternativa más eficaz.

Las nuevas funciones devuelven 0 si se ejecutan satisfactoriamente y un valor distinto de 0 para indicar que se ha producido un error. Adicionalmente, si la función necesita devolver algún valor para indicar su estado de salida —es el caso de `pthread_join`— lo hará a través de una referencia que se pasa como argumento. Los códigos de error devueltos están definidos en `<errno.h>` y podemos obtener una descripción de los mismos con la función `strerror`;

```
#include <string.h>
char *strerror (int errnum);
```

donde `errnum` es el código del error. Las líneas siguientes muestran la forma de usar la función para imprimir un posible error tras una llamada para crear un hilo:

```
error = pthread_create(...);
if (error)
    fprintf (stderr, "Error %d: %s\n", error, strerror (error));
```

Por motivos de compatibilidad con las funciones tradicionales del sistema que usan la variable `errno`, la biblioteca también gestiona una variable `errno` local por cada hilo. En este caso, para hacer transparente al usuario los detalles de implementación, `errno` se define como una macro con la forma siguiente:

```
extern int *__errno ();
#define errno (*__errno ())
```

Utilizar `errno` conlleva llamar a la función `__errno` que devuelve un puntero a un campo dentro de la estructura local de control de cada hilo. Se consigue así compatibilizar la interfaz tradicional de las llamadas al sistema con el nuevo diseño orientado a objetos de la biblioteca de hilos.

Es una buena práctica de programación comprobar los códigos de error que devuelven las llamadas y actuar en consecuencia para que el hilo se recupere de sus errores, no obstante esto puede llevar a la escritura de un código oscuro donde el flujo de funcionamiento normal quede enmascarado por el código de tratamiento de los errores. Como alivio para esta situación se pueden utilizar funciones o macros que se encarguen de procesar los errores y nos permitan escribir un código *más limpio*. En el fichero de cabecera `errores.h` se muestra la macro `error_fatal` concebida con ese propósito.

Fichero 6.9: Macro para facilitar la gestión de errores (`errores.h`)

```
#ifndef __ERRORES__
#define __ERRORES__
#include <stdio.h>

#define error_fatal(codigo, texto) do {\
    fprintf (stderr, "%s:%d: ERROR: %s - %s\n",\
        __FILE__, __LINE__, texto, strerror (codigo));\
    abort ();\
} while (0)

#endif
```

Si, por ejemplo, recodificamos el programa `hilos.c` e introducimos un error en la llamada a `pthread_join`:

```
error = pthread_join (&hilos [h], (void **)&salida);
if (error)
    error_fatal (error, "pthread_join");
else
    printf ("Hilo %d terminado\n", *salida);
```

en primer lugar recibiremos un aviso al compilar el programa,

```
hilos.c:36: warning: passing arg 1 of 'pthread_join' from incompatible
pointer type
```

pero si aún así no hacemos caso del aviso e intentamos ejecutar el programa nos encontraremos con el siguiente mensaje de error:

```
hilos.c:38: ERROR: pthread_join - Invalid argument
Abort (core dumped)
```

La macro `error_fatal` se encarga de escribir el nombre del fichero donde se ha detectado el error —`__FILE__`—, el número de línea dentro de ese fichero —`__LINE__`—, un texto explicativo aportado por el usuario —`texto`—, un texto explicativo asociado al código del error y de abortar la ejecución del hilo.

El motivo por el cual `error_fatal` se ha implementado como macro y no como función es poder acceder al valor actualizado de las constantes `__FILE__` y `__LINE__` que son gestionadas por el preprocesador de C. Sin embargo, para permitir que el programador

la use sintácticamente como si de una función se tratase, se recurre al truco de encerrar las instrucciones de la macro en una estructura de control `do{...}while(0)` que no hace absolutamente nada. Si nos hubiésemos limitado a encerrar las instrucciones entre llaves, la expansión de una llamada como la siguiente:

```
if (error)
    error_fatal (...);
else
    ...
```

daría lugar al código:

```
if (error)
    {...};
else
    ...
```

que presenta un error en el `;` que hay antes del `else`, mientras que la implementación correcta da lugar a la siguiente expansión:

```
if (error)
    do {...} while (0);
else
    ...
```

que es correcta sintácticamente y no modifica el código semánticamente.

6.8 Atributos de un hilo

En la biblioteca *threads* los hilos son considerados objetos con una implementación oculta para el usuario, por lo tanto, la definición y modificación de los atributos de estos objetos es posible únicamente a través de las funciones definidas al efecto. POSIX define los siguientes atributos para los hilos: vinculación *—detachstate—*, tamaño de la pila *—stacksize—*, dirección de la pila *—stackaddr—*, ámbito de contienda *—scope—*, herencia de la planificación *—inheritsched—*, política de distribución *—schedpolicy—* y parámetros de planificación *—schedparam—*.

Los atributos de un hilo se almacenan en objetos del tipo `pthread_attr_t`, son inicializados con los valores por defecto llamando a la función `pthread_attr_init` y borrados con `pthread_attr_destroy`:

```
#include <pthread.h>
int pthread_attr_init (pthread_attr_t *attr);
int pthread_attr_destroy (pthread_attr_t *attr);
```

La condición de separable o no separable de un hilo se modifica con la función `pthread_attr_setdetachstate` y es consultada con `pthread_attr_getdetachstate`:

```
#include <pthread.h>
int pthread_attr_getdetachstate (pthread_attr_t *attr, int *detachstate);
int pthread_attr_setdetachstate (pthread_attr_t *attr, int detachstate);
```


donde `attr` es un puntero al objeto que almacena los atributos y `detachstate` puede tomar los valores `PTHREAD_CREATED_JOINABLE` o `PTHREAD_CREATED_DETACHED`. El primero es el valor por defecto con el que son creados los hilos e indica que otro hilo puede esperar hasta su terminación con una llamada a `pthread_join`. El segundo indica lo contrario y el sistema reclama los recursos de un hilo con este atributo activo nada más terminar su ejecución. La función `pthread_detach` se emplea para activar este atributo con posterioridad a la creación del hilo.

En aquellas implementaciones que definen el símbolo `_POSIX_THREAD_ATTR_STACKSIZE` en `<unistd.h>` se pueden emplear la función `pthread_attr_setstacksize` para indicar el tamaño máximo de los hilos creados con los atributos referenciados por `attr`, y la función `pthread_attr_getstacksize` para consultar el valor de ese tamaño.

```
#include <pthread.h>
#ifdef _POSIX_THREAD_ATTR_STACKSIZE
int pthread_attr_getstacksize(const pthread_attr_t *attr, size_t *stacksize);
int pthread_attr_setstacksize(pthread_attr_t *attr, size_t stacksize);
#endif
```

El símbolo `PTHREAD_STACK_MIN` define el tamaño mínimo de pila permitido para un hilo y es conveniente que el parámetro `stacksize` sea múltiplo de ese valor.

En aquellas implementaciones que definen el símbolo `_POSIX_THREAD_ATTR_STACKADDR` se pueden emplear las funciones `pthread_attr_setstackaddr` para indicar la dirección de inicio de la pila del hilo creado con el atributo `attr` y `pthread_attr_getstackaddr` para consultar el valor de esa dirección.

```
#include <pthread.h>
#ifdef _POSIX_THREAD_ATTR_STACKADDR
int pthread_attr_getstackaddr(const pthread_attr_t *attr, void **stackaddr);
int pthread_attr_setstackaddr(pthread_attr_t *attr, void *stackaddr);
#endif
```

Hay que poner especial atención al usar estas funciones porque el puntero de pila puede variar según direcciones crecientes o decrecientes y este aspecto puede hacer que nuestro programa no sea transportable de un sistema a otro. Por otro lado también hay que impedir que dos hilos compartan una misma pila ya que esto produciría resultados inesperados.

En el ejemplo siguiente veremos la forma de usar estas funciones para inicializar un objeto con los atributos por defecto de un hilo, activar el atributo `PTHREAD_CREATE_DETACHED`, definir una pila de tamaño $3 \times \text{PTHREAD_STACK_MIN}$ y crear un hilo con esos atributos.

Programa 6.10: Atributos de un hilo (atributos.c)

```
/**
```

```
$ atributos
```

```
Compilación:
```

```
$ cc -o atributos atributos.c -pthread (-lpthread en Linux)
```

```
***/  
#include <pthread.h>  
#include "errores.h"  
  
void *codigo_del_hilo (void *arg)  
{  
    pthread_attr_t *atributos = arg;  
    int detachstate;  
    int tam_pila;  
    int error;  
  
    error = pthread_attr_getdetachstate (atributos, &detachstate);  
    if (error)  
        error_fatal (error, "pthread_attr_getdetachstate");  
    else if (detachstate == PTHREAD_CREATE_DETACHED)  
        printf ("HILO separado.\n");  
    else  
        printf ("HILO no separado.\n");  
  
    error = pthread_attr_getstacksize (atributos, &tam_pila);  
    if (error)  
        error_fatal (error, "pthread_attr_getstacksize");  
    else  
        printf ("HILO. Tamaño de la pila: %d bytes = %d x %d\n",  
                tam_pila, tam_pila/PTHREAD_STACK_MIN, PTHREAD_STACK_MIN);  
    return NULL; /* Equivale a pthread_exit (NULL).*/  
}  
  
int main ()  
{  
    pthread_t hilo;  
    pthread_attr_t atributos;  
    size_t tam_pila;  
    int error;  
  
    /* Inicialización de los atributos. */  
    error = pthread_attr_init (&atributos);  
    if (error)  
        error_fatal (error, "pthread_attr_ini");  
  
    /* Activación del atributo PTHREAD_CREATE_DETACHED. */  
    error = pthread_attr_setdetachstate (&atributos,  
                                         PTHREAD_CREATE_DETACHED);  
    if (error)  
        error_fatal (error, "pthread_attr_setdetachstate");  
}
```

```

/* Manipulación del tamaño de la pila. */
error = pthread_attr_getstacksize (&atributos, &tam_pila);
if (error)
    error_fatal (error, "pthread_attr_getstacksize");
else {
    printf("Tamaño de pila por defecto: %d bytes\n", tam_pila);
    printf("Tamaño mínimo de la pila: %d bytes\n",PTHREAD_STACK_MIN);
}
error = pthread_attr_setstacksize (&atributos, 3*PTHREAD_STACK_MIN);
if (error)
    error_fatal (error, "pthread_attr_setstacksize");

/* Creación del hilo con los atributos anteriores. */
error = pthread_create (&hilo,&atributos,codigo_del_hilo,&atributos);
if (error)
    error_fatal (error, "pthread_create");

printf ("Fin del hilo principal.\n");
pthread_exit (NULL);
}

```

El resto de atributos los estudiaremos en el apartado sobre la planificación de hilos —consulte el § 6.11 pág. 227—.

6.9 Cancelación de hilos

En la mayor parte de las aplicaciones los hilos terminan por sí mismos con una llamada a `pthread_exit` una vez concluido su trabajo. Sin embargo puede haber aplicaciones donde los hilos no terminen nunca —es el caso de las aplicaciones de control de procesos industriales— o donde se permita que sean otros hilos quienes den la orden de terminarlos. Pensemos, por ejemplo, en una interfaz gráfica para navegar por el sistema de ficheros y que el usuario arranca un hilo para realizar una búsqueda; si ésta tarda demasiado, al usuario le puede interesar dar la orden de cancelar esa búsqueda, lo que se traducirá en la cancelación del hilo que está realizando ese trabajo. Las llamadas para cancelar hilos se utilizan para indicarle a un hilo que termine su ejecución. La cancelación no es una orden externa de terminación abrupta sino una petición de terminación controlada y las funciones que ofrece POSIX para llevarla a cabo son:

```

#include <pthread.h>
int pthread_cancel (pthread_t thread);
int pthread_setcancelstate (int state, int *oldstate);
int pthread_setcanceltype (int type, int *oldtype);
void pthread_testcancel (void);

void pthread_cleanup_push (void (*routine)(void *), void *arg);
void pthread_cleanup_pop (int execute);

```

Con la función `pthread_cancel` se hacen peticiones de cancelación del hilo indicado en `thread` y ésta se hará efectiva en un instante que está determinado por el *tipo* de cancelación definida —*diferida* o *asíncrona*— y por el *estado* —*habilitada* o *deshabilitada*—.

La función `pthread_setcancelstate` se emplea para cambiar el estado de cancelación del hilo actual. En el parámetro `state` se indica el nuevo estado y en el parámetro `oldstate` se devuelve el estado anterior. El valor que le pasaremos a `state` para permitir que un hilo sea interrumpido es `PTHREAD_CANCEL_ENABLE` y para impedirlo es `PTHREAD_CANCEL_DISABLE`.

La función `pthread_setcanceltype` se emplea para cambiar el tipo de cancelación del hilo actual. En el parámetro `type` se indica el nuevo tipo y en el parámetro `oldtype` se devuelve el tipo anterior. Los posibles valores de `type` son `PTHREAD_CANCEL_ASYNC` para indicar cancelación asíncrona y `PTHREAD_CANCEL_DEFERRED` para indicar cancelación diferida.

Una petición de *cancelación asíncrona* es aceptada inmediatamente y origina la terminación del hilo que se desea cancelar. Esto puede originar que la aplicación quede en un estado inconsistente si en ese momento se encontraba actualizando algún recurso compartido y esa actualización queda incompleta. Por ello no es aconsejable habilitar este tipo de cancelación a menos que tengamos la seguridad de no comprometer la consistencia de la aplicación.

En oposición, las peticiones de *cancelación diferida* sólo se hacen efectivas en los *puntos de cancelación*. La función `pthread_testcancel` crea un punto de cancelación en el hilo actual. Cuando se efectúa la llamada a esta función el hilo actual comprueba si tiene alguna petición de cancelación pendiente y en caso afirmativo procede con la misma. Existen otras funciones que también se comportan como puntos de cancelación. Es obligatorio que las funciones de la tabla 6.3, donde figuran algunas llamadas que ya hemos estudiado y otras que veremos en próximos apartados, sean puntos de cancelación en toda implementación de la biblioteca POSIX.

Tabla 6.3: Puntos de cancelación.

<code>accept</code>	<code>mq_timedsend</code>	<code>putmsg</code>	<code>sigpause</code>
<code>aio_suspend</code>	<code>msgrcv</code>	<code>putpmsg</code>	<code>sigsuspend</code>
<code>clock_nanosleep</code>	<code>msgsnd</code>	<code>pwrite</code>	<code>sigtimedwait</code>
<code>close</code>	<code>msync</code>	<code>read</code>	<code>sigwait</code>
<code>connect</code>	<code>nanosleep</code>	<code>readv</code>	<code>sigwaitinfo</code>
<code>creat</code>	<code>open</code>	<code>recv</code>	<code>sleep</code>
<code>fcntl(F_SETLCKW)</code>	<code>pause</code>	<code>recvfrom</code>	<code>system</code>
<code>fsync</code>	<code>poll</code>	<code>recvmsg</code>	<code>tcdrain</code>
<code>getmsg</code>	<code>pread</code>	<code>select</code>	<code>usleep</code>
<code>getpmsg</code>	<code>pselect</code>	<code>sem_timedwait</code>	<code>wait</code>
<code>lockf</code>	<code>pthread_cond_timedwait</code>	<code>sem_wait</code>	<code>waitid</code>
<code>mq_receive</code>	<code>pthread_cond_wait</code>	<code>send</code>	<code>waitpid</code>
<code>mq_send</code>	<code>pthread_join</code>	<code>sendmsg</code>	<code>write</code>
<code>mq_timedreceive</code>	<code>pthread_testcancel</code>	<code>sendto</code>	<code>writew</code>

El estado y tipo de cancelación que se establece por defecto al crear un nuevo hilo, incluido el hilo principal, es `PTHREAD_CANCEL_ENABLE` y `PTHREAD_CANCEL_DEFERRED` respectivamente, lo cual quiere decir que cuando se da la orden de cancelar un hilo, ésta queda pendiente hasta que se alcanza un punto de cancelación.

Una vez que se hace efectiva la cancelación el hilo llama a los *manejadores de limpieza* —*clean-up handler*— y termina su ejecución. Estos manejadores son definidos por el usuario y su objetivo es realizar las acciones pertinentes para garantizar la consistencia del sistema. Con la función `pthread_cleanup_push` se introduce la función *routine* en la pila de manejadores de limpieza del hilo actual. Esta rutina será invocada con el parámetro `arg` durante la cancelación del hilo, cuando el hilo ejecute una llamada a `pthread_exit` o cuando llame a la función `pthread_cleanup_pop` con el argumento `execute≠0`. Si `pthread_cleanup_pop` es invocada con `execute=0`, entonces es extraída pero no es invocada la función que haya en la cima de la pila.

Un detalle importante que se debe tener en cuenta al emplear las dos funciones anteriores es que en algunos sistemas están implementadas con forma de macros donde `pthread_cleanup_push` contiene la apertura de llave { y `pthread_cleanup_pop` el cierre de llave }. Esto obliga a que las llamadas se escriban emparejadas, dentro de una misma función y dentro de un mismo bloque.

En el ejemplo siguiente se muestra el uso de las funciones anteriores para implementar un programa donde se realiza un cálculo en dos pasos. En el primero se ejecuta rápidamente una parte obligatoria que calcula una primera aproximación del resultado que se considera buena aunque imprecisa. En el segundo paso se itera sobre una parte opcional del cálculo que va mejorando el resultado mientras haya tiempo.

Programa 6.11: Cálculo con refinamientos progresivos y limitación temporal. (exp.c)

```

/****
$ exp x plazo (Ejemplo: $ exp 2.35 1)

Ejemplo de la aplicación de la cancelación de hilos para la implementación de un
algoritmo de cálculo con refinamientos progresivos y limitado en tiempo.

Compilación:
$ cc -o exp exp.c -pthread (-lpthread en Linux)
****/
#include <stdlib.h>
#include <pthread.h>
#include "errores.h"

/****
    fin_del_calculo: presenta el resultado en la salida estándar.
****/
void fin_del_calculo (void *arg)
{
    double resultado = *(double *)arg;

    printf ("Resultado final: %g\n", resultado);

```

```
}
```

```
/***
```

calculo: en esta función se calcula e^x a través de su desarrollo en serie de Taylor:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!} = 1 + \frac{x}{1} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

En primer lugar se ejecuta una parte obligatoria que calcula los 10 primeros sumandos de la serie y en segundo lugar se calculan los restantes sumandos mediante refinamientos progresivos hasta que el hilo es cancelado. */

```
void *calculo (void *arg)
{
    int error;
    int estado_ant, tipo_ant;
    double x = *(double *)arg;
    double resultado = 1, sumando = 1;
    long i, j;

    /* Parte obligatoria del cálculo. */
    /* Deshabilitamos la posibilidad de cancelar el hilo. */
    error = pthread_setcancelstate (PTHREAD_CANCEL_DISABLE, &estado_ant);
    if (error)
        error_fatal (error, "pthread_setcancelstate");
    for (i = 1; i < 10; i++) {
        sumando *= x/i;
        resultado += sumando;
    }
    printf ("Primera aproximación de exp(%g) = %g\n", x, resultado);

    pthread_cleanup_push (fin_del_calculo, &resultado);

    /* Una vez ejecutada la parte obligatoria habilitamos la posibilidad de cancelar el
    hilo en modo diferido. */
    error = pthread_setcanceltype (PTHREAD_CANCEL_DEFERRED, &tipo_ant);
    if (error)
        error_fatal (error, "pthread_setcanceltype");
    error = pthread_setcancelstate (PTHREAD_CANCEL_ENABLE, &estado_ant);
    if (error)
        error_fatal (error, "pthread_setcancelstate");

    /* En esta parte de refinamiento del cálculo se aceptan peticiones de
    cancelación. */
    printf ("Refinamiento del cálculo.\n");
    for (;;) {
        pthread_testcancel (); /* Punto de cancelación. */
    }
}
```

```
/* Si no hay peticiones de cancelación pendientes se ejecuta un nuevo
refinamiento del cálculo. */
for (j = 0; j < 10; j++) {
    sumando *= x/i++;
    resultado += sumando;
}
}
/* La llamada siguiente no llega a ejecutarse nunca, la ponemos por criterios de
transportabilidad para aquellos sistemas que implementan pthread_cleanup_push
y pthread_cleanup_pop con forma de macros. */
pthread_cleanup_pop (1);
}

main (int argc, char *argv [])
{
    pthread_t hilo;
    int error;
    double x;
    int plazo;

    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc != 3) {
        fprintf (stderr, "Forma de uso: %s x plazo\n", argv [0]);
        exit (-1);
    } else {
        x = atof (argv [1]);
        plazo = atoi (argv [2]);
    }

    /* Creación del hilo que realiza los cálculos. */
    error = pthread_create (&hilo, NULL, calculo, &x);
    if (error)
        error_fatal (error, "pthread_create");

    /* Una vez concluido el plazo se cancela el hilo que calcula. */
    sleep (plazo);
    error = pthread_cancel (hilo);
    if (error)
        error_fatal (error, "pthread_cancel");

    /* Esperamos hasta que la cancelación se haga efectiva. */
    error = pthread_join (hilo, NULL);
    if (error)
        error_fatal (error, "pthread_join");
}
```

6.10 Funciones seguras y reentrantes

El hecho de que todos los hilos de un mismo proceso compartan sus datos globales nos lleva a plantearnos qué ocurre si varios hilos acceden concurrentemente a una misma variable. Lo habitual es que se produzcan *condiciones de carrera* —*race condition*— si no se introduce algún mecanismo explícito de sincronización o de control de acceso a los datos y que el resultado final dependa del orden de ejecución de los hilos [Netzer & Miller, 1992].

6.10.1 Funciones seguras para un uso concurrente

Se dice que una función es *segura* para su uso concurrente —*thread-safe*— si puede ser invocada por varios hilos sin que se produzcan condiciones de carrera. Una función escrita en lenguaje C, que sólo utilice variables locales automáticas y que no invoque a funciones inseguras, es segura por definición, ya que las variables locales automáticas son gestionadas dinámicamente a través de la pila local de cada hilo.

Para escribir funciones seguras que accedan a variables globales, variables locales estáticas, recursos compartidos —por ejemplo ficheros o terminales— o que llamen a funciones inseguras es necesario controlar los accesos y las llamadas para garantizar que se realicen en exclusión mutua. El seudocódigo siguiente muestra una función insegura que incrementa una variable local estática y devuelve el resultado:

```
int incrementar ()
{
    static int contador = 0;

    ++contador;
    return contador;
}
```

Esta función se puede hacer segura para su uso concurrente si protegemos el acceso a `contador` con un cerrojo:

```
int incrementar ()
{
    static int contador = 0;
    static cerrojo_t cerrojo = DESBLOQUEADO;
    int valor;

    lock (cerrojo);
    ++contador;
    valor = contador;
    unlock (cerrojo);

    return valor;
}
```

La manipulación de cerrojos en POSIX se realiza con objetos de tipo `pthread_mutex_t` y la estudiaremos en capítulos posteriores —consulte el § 10.6 pág. 400—. En el § 3.6

pág. 87 hemos visto estos conceptos aplicados para el control del acceso concurrente a ficheros con la llamada `lockf`.

Si nos vemos obligados a emplear una función insegura en nuestra aplicación concurrente podemos recurrir también a los cerrojos para garantizar que al llamarla se comporte como una función segura:

```
cerrojo_t cerrojo = DESBLOQUEADO;
...
lock (cerrojo);
función_insegura ();
unlock (cerrojo);
```

Esta situación puede introducir cuellos de botella en nuestra aplicación pero siempre será preferible una pérdida de prestaciones antes que comprometer su seguridad.

6.10.2 Funciones reentrantes

Se dice que una función es *reentrante* cuando carece de un estado interno asociado. Si la función está escrita en lenguaje C esto ocurre siempre que la función no tenga variables estáticas locales, no acceda a variables globales y sólo llame a funciones reentrantes.

Aunque puedan parecer conceptos parecidos, no hay que confundir las funciones reentrantes con las seguras. Una función puede ser segura, reentrante, ambas cosas o ninguna. Como ejemplo tomemos la segunda versión de la función `incrementar` del apartado anterior, es segura pero no es reentrante: cada vez que la llamamos devuelve un valor distinto y eso es debido a que en `contador` se almacena el valor devuelto en la última llamada; la función se comporta como si tuviese memoria, es decir, tiene un estado interno asociado. Pensemos ahora, por ejemplo, en la función `sqrt` de la biblioteca matemática que devuelve la raíz cuadrada de un número: esta función devuelve el mismo resultado siempre que le pasemos el mismo argumento; carece de memoria.

La biblioteca de funciones C estándar de UNIX ofrece muchas funciones no reentrantes que plantean problemas al usarlas concurrentemente. Veamos un ejemplo: la función `readdir` —consulte el § 4.1.5 pág. 123— devuelve un puntero a una estructura de tipo `struct dirent` local a la función y en la que hay información sobre la entrada de directorio que se acaba de leer; si esta función es invocada en primer lugar por el hilo H_1 y en segundo lugar por el hilo H_2 , como la dirección devuelta en ambos casos es la misma ya que se corresponde con una variable estática local, la segunda llamada modifica los datos con los que está trabajando H_1 . Este problema admite varias soluciones. Una de ellas es recodificar la función para que haga una reserva dinámica de memoria y no devuelva la dirección de una variable estática local. En este caso, el hilo que invoca a la función tiene que encargarse de liberar esa memoria cuando deje de necesitarla y esto puede ser una fuente de errores que produzca fugas de memoria si se nos olvida hacer esa liberación. La ventaja de esta solución es que no supone un cambio en la interfaz de la función aunque introduce un pequeño cambio en cuanto a su uso como acabo de comentar. Otra solución es cambiar la interfaz de `readdir` y que sea el hilo que la llama quien suministre la dirección donde se almacenarán los datos que devuelve la función. Ésta es la solución adoptada en POSIX y, así, la función `readdir` reentrante pasa a llamarse `readdir_r` con la siguiente declaración:

```
#include <sys/types.h>
#include <dirent.h>
int readdir_r (DIR *dirp, struct dirent *entry, struct dirent **result);
```

La función lee, al igual que `readdir`, la entrada donde se encuentre situado el puntero de lectura del flujo de directorio `dirp` y actualiza dicho puntero. Si se ejecuta correctamente, la función devuelve 0, copia la información leída en la estructura referenciada por `entry` y hace que `result` apunte a la misma dirección que `entry`. Cuando se alcanza el final del directorio devuelve 0 y `result` toma el valor NULL. En caso de fallo devuelve un código de error distinto de 0.

Como vemos, `readdir_r` no almacena nada en estructuras locales estáticas, todo se almacena en zonas de memoria suministradas por el usuario. Se comporta así como una función sin memoria. Por otro lado, el tratamiento de los errores es similar al que ya hemos visto para las funciones POSIX de la biblioteca de hilos: devolver 0 si todo va bien y un código de error en caso de fallo.

Podríamos pensar que `readdir_r` no es realmente reentrante ya que cada vez que la llamamos devuelve una entrada de directorio distinta y esto parece estar en contradicción con la afirmación de que *una función reentrante devuelve un mismo resultado siempre que se le pasen los mismos parámetros*; no conserva información entre llamadas. En realidad esto es también válido para `readdir_r` y el hecho de que devuelva entradas distintas en llamadas consecutivas se debe a que se actualiza el puntero de lectura del flujo `dirp`, por lo tanto, cuando llamamos a la función varias veces pasándole un mismo flujo le estamos pasando un mismo puntero a ese flujo, pero los datos referenciados por ese puntero han variado de una llamada a otra.

6.11 Planificación de hilos

La biblioteca de hilos POSIX ha sido concebida para adaptarse a sistemas con diferentes niveles de exigencia y para ello se han definido subconjuntos de requisitos de la norma base conocidos como perfiles. La norma POSIX 1003.13 define 4 perfiles para sistemas de tiempo real:

- PSE51 Sistema de tiempo real mínimo.* Está pensado para sistemas embebidos de tamaño pequeño que carecen de gestión de memoria, ficheros y terminales —por ejemplo, el controlador de un horno—.
- PSE52 Controlador de tiempo real.* Carece de controlador de memoria pero tiene un pequeño sistema de ficheros residente en disco y un terminal —por ejemplo, el controlador de un robot industrial—.
- PSE53 Sistema de tiempo real dedicado.* Pensado para sistemas embebidos grandes que requieren protección de memoria y comunicaciones en red pero sin almacenamiento en disco —por ejemplo, sistemas de aviónica y nodos de conmutación de telefonía móvil—.
- PSE54 Sistema de tiempo real generalizado.* No impone restricciones. Se requiere para sistemas de tiempo real grandes con todo tipo de servicios: entorno de

desarrollo, comunicaciones en red, sistema de ficheros en disco, terminales, interfaces gráficas de usuario, etc. —por ejemplo, sistemas de control de tráfico aéreo—.

En la tabla 6.4 se muestran con más detalle las características de cada uno de los perfiles.

Tabla 6.4: Perfiles definidos en POSIX 1003.13 para sistemas de tiempo real.

Característica	PSE51	PSE52	PSE53	PSE54
E/S básica	✓	✓	✓	✓
Señales	✓	✓	✓	✓
E/S síncrona	✓	✓	✓	✓
Reloj y temporizadores	✓	✓	✓	✓
Semáforos	✓	✓	✓	✓
Colas de mensajes	✓	✓	✓	✓
Hilos	✓	✓	✓	✓
Ficheros y directorios		✓	✓	✓
E/S asíncrona		✓	✓	✓
Tuberías			✓	✓
Múltiples procesos			✓	✓
Comunicaciones en red			✓	✓
Usuarios y grupos				✓
Intérprete de órdenes				✓
Enlace dinámico				✓
Atributos para ficheros				✓

Si la implementación de POSIX con la que estamos trabajando define la constante `_POSIX_THREAD_PRIORITY_SCHEDULING` entonces podremos definir la política de distribución de los hilos y asignarle prioridades de acuerdo con alguno de los esquemas descritos en el § 5.7 pág. 173 para sistemas con requisitos de tiempo real. Para modificar y consultar el esquema de distribución de nuestra aplicación se emplean las funciones `pthread_attr_setschedpolicy` y `pthread_attr_getschedpolicy` respectivamente:

```
#include <sched.h>
#include <pthread.h>
int pthread_attr_setschedpolicy (pthread_attr_t *attr, int policy);
int pthread_attr_getschedpolicy (const pthread_attr_t *attr, int *policy);
```

Con `pthread_attr_setschedpolicy` indicamos que `policy` será la política de distribución para los hilos que se creen con los atributos referenciados por `attr`. En el fichero de cabecera `<sched.h>` se definen los siguientes valores para `policy`:

SCHED_FIFO Distribución con desalojo por prioridad. Con esta política se le concede la CPU al hilo de mayor prioridad que será desalojado cuando voluntariamente lo decida, cuando quede bloqueado por acceder a un recurso ocupado o cuando se active otro hilo más prioritario.

- SCHED_RR** Distribución con desalojo por prioridad circular una vez concluida la rodaja temporal asignada. Con esta política se le concede la CPU al hilo de mayor prioridad que será desalojado cuando voluntariamente lo decida, cuando quede bloqueado por acceder a un recurso ocupado, cuando se active otro hilo más prioritario o cuando se active otro hilo de igual prioridad y concluya la rodaja temporal asignada al hilo actual.
- SCHED_OTHER** Cualquier otra política de distribución. Habitualmente coincide con **SCHED_FIFO**

Un proceso comunica que quiere ser voluntariamente desalojado de la CPU haciendo una llamada a `sched_yield`:

```
#include <sched.h>
int sched_yield (void);
```

POSIX permite también definir el *ámbito de contienda* entre los hilos que comparten una misma CPU. Para cambiar este parámetro se utiliza la función `pthread_attr_setscope` y para leerlo `pthread_attr_getscope`:

```
#include <pthread.h>
int pthread_attr_setscope (pthread_attr_t *attr,
                           int contentionscope);
int pthread_attr_getscope (const pthread_attr_t *attr,
                           int *contentionscope);
```

donde `attr` es una referencia a los atributos que estamos cambiando o consultando y `contentionscope` toma el valor `PTHREAD_SCOPE_PROCESS` para indicar que los hilos compiten por la CPU con otros del mismo proceso o `PTHREAD_SCOPE_SYSTEM` para indicar que compiten con todos los hilos del sistema.

Para cambiar y leer los parámetros de planificación de unos atributos empleamos las funciones `pthread_attr_setschedparam` y `pthread_attr_getschedparam`:

```
#include <sched.h>
#include <pthread.h>
int pthread_attr_getschedparam (const pthread_attr_t *attr,
                                struct sched_param *param);
int pthread_attr_setschedparam (pthread_attr_t *attr,
                                const struct sched_param *param);
```

Con `pthread_attr_setschedparam` indicamos que los parámetros de planificación, para los hilos que se creen con los atributos referenciados por `attr`, serán los indicados en la estructura de tipo `struct sched_param` referenciada por `param`. Esta estructura se define en `<sched.h>` y para las políticas de distribución **SCHED_FIFO** y **SCHED_RR** consta del campo entero `sched_priority` donde se almacena la prioridad.

Si queremos cambiar o consultar los atributos de distribución y planificación de hilos concretos que ya han sido creados emplearemos las funciones `pthread_setschedparam` y `pthread_getschedparam`:

```
#include <sched.h>
#include <pthread.h>
int pthread_setschedparam (pthread_t thread, int policy,
                           const struct sched_param *param);
int pthread_getschedparam (pthread_t thread, int *policy,
                           struct sched_param *param);
```

La función `pthread_setschedparam` cambia la política de distribución y la prioridad del hilo `thread` de acuerdo con los argumentos `policy` y `param` explicados anteriormente. Para consultar esos mismos atributos usamos `pthread_getschedparam`.

Para determinar el rango de prioridades permitidas en nuestro sistema para cada una de las políticas de distribución, se emplean las llamadas `sched_get_priority_min` y `sched_get_priority_max`:

```
#include <sched.h>
int sched_get_priority_max (int policy);
int sched_get_priority_min (int policy);
```

Ambas funciones devuelven el valor de la prioridad solicitada si se ejecutan satisfactoriamente o -1 si fallan, en este caso se almacena en `errno` el error producido.

Por último, para indicar si los nuevos hilos heredarán los atributos de distribución y planificación del hilo que los crea o tendrán atributos nuevos se utiliza:

```
#include <pthread.h>
int pthread_attr_setinheritsched (pthread_attr_t *attr, int inheritsched);
int pthread_attr_getinheritsched (const pthread_attr_t *attr,
                                  int *inheritsched);
```

Si `inheritsched` toma el valor `PTHREAD_INHERIT_SCHED` los atributos se heredan pero si toma el valor `PTHREAD_EXPLICIT_SCHED` se utilizan los que figuran en `attr`. Con la función `pthread_attr_getinheritsched` consultamos el valor del atributo de herencia.

En el programa siguiente se muestra la forma de definir los atributos de distribución y planificación de 4 hilos que una vez creados ejecutan un bucle sin fin hasta que el hilo principal decide cancelarlos. La forma de invocar al programa es:

```
$ plan arg
```

Si `arg` vale 0 los 4 hilos tendrán la misma prioridad y serán distribuidos con una política `SCHED_RR`. En caso de que valga 1 los hilos tendrán prioridades distintas y serán distribuidos con una política `SCHED_FIFO`.

Programa 6.12: Planificación y distribución de hilos (plan.c)

```
/**
```

```
$ plan 0 Distribuir hilos de igual prioridad con desalojo por prioridad circular.
$ plan 1 Distribuir hilos de distinta prioridad con desalojo por prioridad.
```

Compilación:

```
$ cc -o plan plan.c -pthread (-lpthread en Linux)
```

```

***/
#include <sched.h>
#include <pthread.h>
#include <errno.h>
#include "errores.h"

typedef struct {
    int id;
    pthread_t hilo;
    pthread_attr_t atributos;
} hilo_t;

#define NUM_HILOS 4

void *codigo_del_hilo (void *arg)
{
    hilo_t *h = (hilo_t *)arg;
    int distrib;
    struct sched_param param;
    int error;
    int i;

    for (i = 0; ; i++) {
        pthread_testcancel ();
        if (i%1000000 == 0) {
            error = pthread_getschedparam (h->hilo, &distrib, &param);
            if (error)
                error_fatal (error, "pthread_getschedparam");
            else
                printf ("%d: %s, %d\n", h->id,
                    distrib == SCHED_FIFO ? "FIFO":
                    distrib == SCHED_RR ? "RR": "OTROS",
                    param.sched_priority);
            sched_yield ();
        }
    }
    pthread_exit (NULL);
}

int main (int argc, char *argv[])
{
    hilo_t hilos [NUM_HILOS];
    struct sched_param param;
    int prioridad_min;
    int distrib;
    int error;

```

```

int i;

/* Análisis de los argumentos de la línea de órdenes. */
if (argc != 2) {
    fprintf (stderr, "Forma de uso: %s 0|1\n", argv [0]);
    exit (-1);
}
else
    distrib = atoi (argv [1]);

prioridad_min = sched_get_priority_min (SCHED_FIFO);
if (prioridad_min == -1)
    error_fatal (errno, "sched_get_min_priority");

/* Inicialización de los atributos. */
for (i = 0; i < NUM_HILOS; i++) {
    hilos[i].id = i + 1;
    error = pthread_attr_init (&hilos[i].atributos);
    if (error)
        error_fatal (error, "pthread_attr_ini");
        error = pthread_attr_init (&hilos[i].atributos);
    error = pthread_attr_setscope (&hilos[i].atributos,
                                   PTHREAD_SCOPE_PROCESS);

    if (error)
        error_fatal (error, "pthread_attr_setscope");
    /* argv[1]=0→policy=SCHED_RR y todos los hilos tendrán la misma
    prioridad; argv[1]=1→policy=SCHED_FIFO y cada hilo tendrá una
    prioridad distinta. */
    error = pthread_attr_setschedpolicy (&hilos[i].atributos,
                                         distrib?SCHED_FIFO:SCHED_RR);

    if (error)
        error_fatal (error, "pthread_attr_setschedpolicy");
    param.sched_priority = prioridad_min + i*distrib;
    error = pthread_attr_setschedparam (&hilos[i].atributos, &param);
    if (error)
        error_fatal (error, "pthread_attr_setschedparam");
    error = pthread_attr_setinheritsched (&hilos[i].atributos,
                                         PTHREAD_EXPLICIT_SCHED);

    if (error)
        error_fatal (error, "pthread_attr_setinheritsched");
}

/* Creación de los hilos. */
for (i = 0; i < NUM_HILOS; i++) {
    error = pthread_create (&hilos[i].hilo, &hilos[i].atributos,
                           codigo_del_hilo, (void *)&hilos[i]);

```

```

        if (error)
            error_fatal (error, "pthread_create");
    }

    /* Cancelación de los hilos en orden inverso al de su creación. */
    for (i = NUM_HILOS-1 ; i >= 0; i--) {
        sleep (1);
        error = pthread_cancel (hilos[i].hilo);
        if (error)
            error_fatal (error, "pthread_cancel");
        printf ("Hilo nro. %d terminado\n", hilos[i].id);
        error = pthread_join (hilos[i].hilo, NULL);
        if (error)
            error_fatal (error, "pthread_join");
    }

    printf ("Fin del hilo principal.\n");
    pthread_exit (NULL);
}

```

6.12 Ejercicios

6.1 El objetivo de este ejercicio es practicar con todas las posibles llamadas a las funciones de la familia `exec`. Para ello escriba un programa de nombre `ejecutar` en el que se invoque a la orden `ls -al /usr/bin` con las funciones `exec` tal y como se indica:

`$ ejecutar opción`

Las opciones válidas son:

- `-l`, invocar a la orden `ls` con la función `execl`.
- `-v`, invocar a la orden `ls` con la función `execv`.
- `-le`, invocar a la orden `ls` con la función `execl`.
- `-ve`, invocar a la orden `ls` con la función `execve`.
- `-lp`, invocar a la orden `ls` con la función `execlp`.
- `-vp`, invocar a la orden `ls` con la función `execvp`.

6.2 Las baterías de órdenes son ficheros que tienen permisos de ejecución, por lo tanto ¿pueden ser ejecutadas con las funciones de la familia `exec`? En caso afirmativo, escriba la batería de órdenes siguiente:

```

###
# Programa: batería

```



```
# Descripción. Ejemplo de batería de órdenes.
###
pwd
ls $HOME
echo Batería ejecutada satisfactoriamente
```

Para ejecutar este fichero con `exec` habrá que escribir un programa C parecido al siguiente:

```
main (int argc, char **argv)
{
    execlp ("batería", "batería", (char *)0);
    perror (argv [0]);
}
```

En caso de que no se consiga ejecutar la batería de órdenes, ¿a qué es debido? ¿cómo se puede solucionar este problema?

- 6.3** Escriba un programa de nombre `expiar` que ejecute una orden cuando un fichero sufra una modificación. La forma de invocar al programa será:

```
$ expiar fichero [-p período] [orden]
```

El programa debe consultar el estado del fichero con una periodicidad de `período` segundos. Si `fichero` sufre alguna modificación, entonces cargará en memoria y ejecutará el programa `orden`. El valor por defecto de los parámetros opcionales debe ser: 30 segundos para el `periodo` y `ls -l fichero` para la `orden`.

Un ejemplo de uso del programa `expiar` puede ser avisar al usuario de la recepción de correo, tarea que normalmente realiza el sistema. Para determinar el instante de la recepción de un mensaje, hay que expiar el buzón del correo. El nombre de este fichero es `/usr/spool/mail/nombre_usuario`. Cada usuario tiene su propio buzón. Por ejemplo, para expiar el buzón del usuario `curso0` se puede emitir la orden:

```
$ expiar /usr/spool/mail/curso0 -p 20 echo HA LLEGADO CORREO &
```

- 6.4** Escriba un programa que reciba a través de la línea de órdenes del sistema operativo el número de procesos hijo que debe crear. Cada proceso hijo deberá dormir un número aleatorio de segundos comprendido entre 0 y 30. El proceso padre debe esperar la terminación de cada hijo y, según vayan éstos terminando, presentará por pantalla el `PID` de cada uno de ellos y el número de segundos que ha estado durmiendo cada proceso hijo. El número de segundos que duerme cada hijo se debe generar internamente en el proceso hijo; es decir, el padre no tiene conocimiento de este dato hasta que el hijo no termina su ejecución y se lo comunica.

¿Qué ocurre si el número de procesos que debe crear el programa es muy grande?

- 6.5** Recodifique la función `sistema` —apartado 6.3, pág. 194— utilizando la llamada `execl` en lugar de `execlp` y `waitpid` en lugar de `wait`.

Esta función la vamos a probar escribiendo un programa que cree un proceso zombi y haciendo que su padre realice una llamada a `sistema` para ejecutar la orden `ps` y comprobar que el proceso hijo es realmente un zombi.

- 6.6** Escriba un programa de nombre `disfraz` que se comporte lo mismo que la orden `su` —cambiar de usuario—. La forma de invocar al programa debe ser:

```
$ disfraz [usuario]
```

La orden `disfraz` debe arrancar un nuevo intérprete de órdenes con los atributos del usuario indicado en `usuario`. En el caso de que no se indique este parámetro, el intérprete debe arrancarse con los atributos del usuario `root`. Las acciones que debe realizar el programa `disfraz` son las siguientes:

- (a) Analizar los parámetros de la línea de órdenes.
- (b) Comprobar si existe el usuario indicado —utilice la función `getpwnam`—.
- (c) Leer la contraseña del usuario. Para ello hay que reprogramar el terminal y conseguir que trabaje en modo no canónico —consulte el § 4.2.2 pág. 135—. Otra forma más sencilla es utilizar la función `getpass`.
- (d) Cifrar la contraseña. Para ello hay que utilizar la función `crypt`:

```
crypt(char *contraseña, char *semilla);
```

donde `contraseña` es una cadena de caracteres con la contraseña sin cifrar y `semilla` es una cadena elegida al azar. Para los sistemas UNIX, esta cadena se compone de dos caracteres que deben coincidir con los dos primeros caracteres de la clave cifrada.

- (e) En caso de que la contraseña que se acaba de cifrar no coincida con la que figura en el fichero `/etc/passwd`, sacar un mensaje de error y terminar la ejecución del programa.
- (f) Fijar el `UID` y el `GID` del proceso actual de acuerdo con el `UID` y el `GID` del nuevo usuario.
- (g) Programar las variables de entorno `HOME`, `LOGNAME` y `USER` de acuerdo con los valores correspondientes al nuevo usuario.
- (h) Cambiar al directorio de arranque del nuevo usuario —llamada `chdir`—.
- (i) Arrancar un nuevo intérprete de órdenes —`/bin/sh`— con alguna de las funciones de la familia `exec`.

La información que se necesita conocer del nuevo usuario —contraseña, `UID`, `GID`, directorio de arranque, etc.— se puede obtener del fichero `/etc/passwd` con la función `getpwnam`.

Para que el programa `disfraz` tenga una operatividad completa hay que cambiarle su propietario y activar su bit *cambiar el identificador de usuario al ejecutar*. Esto lo conseguimos con las órdenes:

```
$ chown root disfraz
```

```
$ chgrp root disfraz
```

```
$ chmod u+s disfraz
```

6.7 ¿Para qué sirve el programa estándar `login`? ¿Qué acciones tendría que realizar un programa de nombre `entrar` para que tuviese una funcionalidad similar a la del programa `login`? ¿Qué ficheros del sistema tendría que modificar este programa?

6.8 Utilizando las funciones de gestión de memoria implementadas en el módulo `rmem.c` de la pág. 203, escriba un programa que tenga la funcionalidad de una calculadora con notación postfija.

En la mayor parte de las calculadoras comerciales, las operaciones se introducen tal y como se escriben. Este tipo de calculadoras trabajan con la notación infija y necesitan el uso de paréntesis para modificar la prioridad de los operadores. Por ejemplo, la expresión $(a+b) \times c + \frac{b+d}{a}$ se introduce de la forma `(a+b)*c+(b+d)/a`. En las calculadoras con notación postfija —también llamada notación polaca inversa— primero se introducen los operandos y luego los operadores. La expresión anterior se escribiría de la forma `ab+c*bd+a/+`.

Para escribir este programa se aconseja utilizar una pila en la que se irán almacenando operandos y operadores. Como ya sabemos, una pila es un tipo abstracto de datos que se define por sus operaciones:

- Insertar elementos (`push`).
- Extraer elementos (`pop`).
- Consultar si hay elementos en la pila.

Lo realmente característico de una pila es que los datos de la misma se extraen en orden inverso al de inserción. Es decir, los últimos elementos insertados son los primeros en ser extraídos.

Las pilas se pueden implementar básicamente con arrays o con listas lineales simplemente enlazadas. Como en este ejercicio se pretende hacer uso de la gestión dinámica de memoria a través de las funciones `rmem` y `lmem`, se pide implementar las pilas mediante listas lineales.

A continuación se muestra un fragmento del programa `calculadora` que deberá completarse:

```
// $ calculadora
// Forma de uso. Por ejemplo, para evaluar la expresión 5*5 + 50 = 75
//      $ calculadora
//      < 5
//      < 5
//      < *
//      > 25
//      < 50
//      < +
//      > 75
```

```
//      < s Para salir
//      $

#include <stdio.h>
#include <math.h> // Para la función atof

// Definición de los datos.
struct operando {
    float valor;
    struct operando *siguiente;
};
// pila se maneja con las operaciones push y pop
struct operando *pila;

// Prototipo de las funciones.
void push (float);
float pop ();
struct operando *crear_elemento (float);

main ()
{
#   define MAX 256
    float op1, op2;
    char operador [MAX];

    do {
        printf("< ");
        gets (operador);
        switch (operador [0]) {
            case '+':
                op2 = pop ();
                op1 = pop ();
                printf("> %g\n", op1+op2);
                push (op1+op2);
                break;
            case '-':
                op2 = pop ();
                op1 = pop ();
                printf("> %g\n", op1-op2);
                push (op1-op2);
                break;
            case '*':
                op2 = pop ();
                op1 = pop ();
                printf("> %g\n", op1*op2);
                push (op1*op2);
```

```
        break;
    case '/':
        op2 = pop ();
        op1 = pop ();
        printf ("> %g\n", op1/op2);
        push (op1/op2);
        break;
    default:
        op1 = atof (operador);
        push (op1);
        break;
    }
}while (operador [0] != 's');
exit (0);
}

// Funciones que se deben escribir.
void push (float valor) {...}
float pop () {...}
struct operando *crear_elemento (float valor) {...}
```

Capítulo 7

Señales y funciones de tiempo

«En esto sucedió acaso que un porquero que andaba recogiendo de unos rastros una manada de puercos (que sin perdón así se llaman) tocó un cuerno, a cuya señal ellos se reconocen, y al instante se le representó a don Quijote lo que deseaba, que era que algún enano hacía señal de su venida...»

(Cervantes, Quijote I-2)

7.1	Concepto de señal	239
7.2	Tipos de señales	240
7.3	Señales en el UNIX System V	243
7.4	Señales en el sistema 4.3BSD	252
7.5	Gestión de señales en POSIX	262
7.6	Ejemplo de aplicación de las señales	276
7.7	Funciones de tiempo	281
7.8	Relojes, retardos y temporizadores en POSIX	294
7.9	Ejercicios	298

7.1 Concepto de señal

Las señales son interrupciones software que pueden ser enviadas a un proceso para informarle de algún evento asíncrono o situación especial. El término señal se emplea también para referirse al evento.

Los procesos pueden enviarse señales unos a otros a través de la llamada `kill` y es bastante frecuente que, durante su ejecución, un proceso reciba señales procedentes del núcleo. Cuando un proceso recibe una señal puede reaccionar de tres formas diferentes:

1. Ignorar la señal, con lo cual es inmune a la misma.
2. Invocar a la rutina de tratamiento por defecto. Esta rutina no la codifica el programador, sino que la aporta el núcleo. Según el tipo de señal, la rutina de tratamiento

por defecto realizará una acción u otra. Por lo general suele provocar la terminación del proceso mediante una llamada a `exit`. Algunas señales no sólo provocan la terminación del proceso, sino que además hacen que el núcleo genere, en el directorio de trabajo actual del proceso, un fichero llamado `core` que contiene un volcado de memoria del contexto del proceso. Este fichero podrá ser examinado con ayuda de un programa depurador —`adb`, `sdb`, `gdb`— para determinar qué señal provocó la terminación del proceso y en qué punto exacto de su ejecución se produjo. Este mecanismo es muy útil a la hora de depurar programas que contienen errores tales como: incorrecta manipulación de números en coma flotante, instrucciones ilegales, acceso a direcciones fuera de rango, etc.

3. Invocar a una rutina propia que se encarga de tratar la señal. Esta rutina es invocada por el núcleo en el supuesto de que esté montada y será responsabilidad del programador codificarla para que tome las acciones pertinentes como tratamiento de la señal. En estos casos, el programa no termina su ejecución a menos que la rutina de tratamiento indique lo contrario.

En la figura 7.1 vemos esquematizada la evolución temporal de un proceso y cómo, a lo largo de su ejecución, recibe varias señales procedentes del núcleo. La primera señal que recibe no provoca que el proceso cambie el curso de su ejecución, esto es debido a que la acción activada es ignorar la señal. El proceso prosigue su ejecución y recibe una segunda señal que lo fuerza a entrar en una rutina de tratamiento. Esta rutina, después de tratar la señal, puede optar por tres acciones: restaurar la ejecución del proceso al punto donde se produjo la interrupción, finalizar el proceso o restaurar alguno de los estados pasados del proceso —punto de repliegue— y continuar la ejecución desde ese punto —más adelante veremos que esto se consigue con las funciones estándar `setjmp` y `longjmp`, consulte `setjmp(3C)` en el § 7.3.4 pág. 250—. El proceso puede también recibir una señal que lo fuerce a entrar en la rutina de tratamiento por defecto.

En párrafos posteriores volveremos sobre estos conceptos y veremos cómo se especifica cuál de las tres formas es la elegida para tratar una señal.

7.2 Tipos de señales

Cada señal tiene asociado un número entero positivo y, cuando un proceso le envía una señal a otro, realmente le está enviando ese número. En el UNIX System V hay definidas 19 señales, y en 4.3BSD, 30. Las 19 señales del UNIX System V las tienen prácticamente todas las versiones de UNIX, y a éstas cada fabricante les añade las que considera necesarias. Podemos clasificar las señales en los siguientes grupos:

- Señales relacionadas con la terminación de procesos.
- Señales relacionadas con las excepciones inducidas por los procesos. Por ejemplo, el intento de acceder fuera del espacio de direcciones virtuales, los errores producidos al manejar números en coma flotante, etc.
- Señales relacionadas con los errores irreversibles originados en el transcurso de una llamada al sistema.

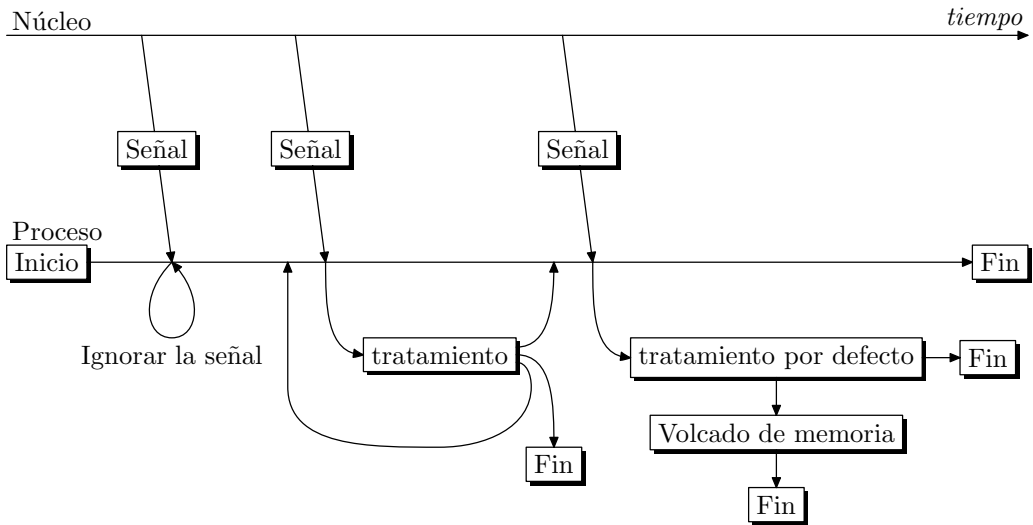


Figura 7.1: Tratamiento de las señales recibidas por un proceso.

- Señales originadas desde un proceso que se está ejecutando en modo usuario. Por ejemplo, cuando un proceso envía una señal a otro vía `kill`, cuando un proceso activa un temporizador y se queda en espera de la señal de alarma, etc.
- Señales relacionadas con la interacción con el terminal. Por ejemplo, pulsar las teclas `Ctrl+C`.
- Señales para ejecutar un proceso paso a paso. Son usadas por los depuradores.

En el fichero de cabecera `<signal.h>` están definidas las señales que puede manejar el sistema. Las 19 señales del UNIX System V son:

- SIGHUP** *Desconexión*. Es enviada cuando un terminal se desconecta de todo proceso del que es terminal de control. También se envía a todos los procesos de un grupo cuando el líder del grupo termina su ejecución. La acción por defecto de esta señal es terminar la ejecución del proceso que la recibe.
- SIGINT** *Interrupción*. Se envía a todo proceso asociado con un terminal de control cuando se pulsan la teclas de interrupción `Ctrl+c`. Su acción por defecto es terminar la ejecución del proceso que la recibe.
- SIGQUIT** *Salir*. Similar a **SIGINT**, pero es generada al pulsar la tecla de salida `Ctrl+\`. Su acción por defecto es generar un fichero `core` y terminar el proceso.
- SIGILL** *Instrucción ilegal*. Es enviada cuando el procesador detecta una instrucción que no forma parte de su repertorio. En los programas escritos en C suele producirse este tipo de error cuando manejamos punteros a funciones que

no han sido correctamente inicializados. Su acción por defecto es generar un fichero `core` y terminar el proceso.

- SIGTRAP** *Trace trap.* Cuando un proceso se está ejecutando *paso a paso*, esta señal es enviada después de ejecutar cada instrucción. Es empleada por los programas depuradores. Su acción por defecto es generar un fichero `core` y terminar el proceso.
- SIGIOT** *I/O trap instruction.* Se envía cuando se da un fallo de hardware. La naturaleza de este fallo depende de la máquina. También es enviada cuando llamamos a la función `abort`, que provoca el suicidio del proceso generando un fichero `core`.
- SIGEMT** *Emulator trap instruction.* También indica un fallo de hardware. Raras veces se utiliza. Su acción por defecto es generar un fichero `core` y terminar el proceso.
- SIGFPE** *Error en coma flotante.* Es enviada cuando el hardware detecta un error en coma flotante, como el uso de número en coma flotante con un formato desconocido, errores de desbordamiento, etc. Su acción por defecto es generar un fichero `core` y terminar el proceso.
- SIGKILL** *Terminación abrupta.* Esta señal provoca irremediamente la terminación del proceso. No puede ser ignorada y siempre que se recibe se ejecuta su acción por defecto, que consiste en generar un fichero `core` y terminar el proceso.
- SIGBUS** *Error de bus.* Se produce cuando se da un error de acceso a memoria. Las dos situaciones típicas que la provocan suelen ser intentar acceder a una dirección que físicamente no existe o intentar acceder a una dirección impar, violando así las reglas de alineación que impone el procesador. Su acción por defecto es generar un fichero `core` y terminar el proceso.
- SIGSEGV** *Violación de segmento.* Es enviada a un proceso cuando intenta acceder a datos que se encuentran fuera de su segmento de datos. Su acción por defecto es generar un fichero `core` y terminar el proceso.
- SIGSYS** *Argumento erróneo en una llamada al sistema.* No se usa.
- SIGPIPE** *Intento de escritura en una tubería de la que no hay nadie leyendo.* Esto suele ocurrir cuando el proceso de lectura termina de una forma anormal. Su acción por defecto es terminar el proceso.
- SIGALRM** *Despertador.* Es enviada a un proceso cuando alguno de sus temporizadores descendentes llega a cero. Su acción por defecto es terminar el proceso.
- SIGTERM** *Finalización controlada.* Es la señal utilizada para indicarle a un proceso que debe terminar su ejecución. Esta señal no es tajante como **SIGKILL** y puede ser ignorada. Lo correcto es que la rutina de tratamiento de esta señal se encargue de tomar las acciones necesarias para dejar al proceso en un estado coherente y a continuación finalizar su ejecución con una llamada a `exit`. Esta señal es enviada a todos los procesos cuando se emiten las órdenes `shutdown` o `reboot`. Su acción por defecto es terminar el proceso.

- SIGUSR1** *Señal número 1 de usuario.* Esta señal está reservada para uso del programador. Ninguna aplicación estándar va a utilizarla y su significado es el que le quiera dar el programador en su aplicación. Su acción por defecto es terminar el proceso.
- SIGUSR2** *Señal número 2 de usuario.* Su significado es idéntico al de **SIGUSR1**.
- SIGCLD** *Terminación del proceso hijo.* Es enviada al proceso padre cuando alguno de sus procesos hijo termina. Esta señal es ignorada por defecto.
- SIGPWR** *Fallo de alimentación.* Esta señal tiene diferentes interpretaciones. En algunos sistemas es enviada cuando se detecta un fallo de alimentación y le indica al proceso que dispone tan sólo de unos instantes antes de que se produzca la caída del sistema. En otros sistemas, esta señal es enviada, después de recuperarse de un fallo de alimentación, a todos aquellos procesos que estaban en ejecución y que se han podido rearmar. En estos casos, los procesos deben disponer de mecanismos para restaurar las posibles pérdidas producidas durante la caída de la alimentación.

En la tabla 7.1 podemos ver un resumen de las características de las señales mencionadas.

7.3 Señales en el UNIX System V

Vamos a iniciar ahora el estudio de la interfaz de manejo de señales que brinda el UNIX System V, posteriormente estudiaremos también la interfaz de BSD y la de POSIX, que son compatibles con la del System V y la mejoran en muchos aspectos.

7.3.1 Envío de señales (kill y raise)

Para enviar una señal desde un proceso a otro o a un grupo de procesos, emplearemos la llamada `kill`:

```
#include <signal.h>
int kill (pid_t pid, int sig);
```

donde `pid` identifica el conjunto de procesos al que queremos enviarle la señal. Este parámetro es un número entero y los distintos valores que puede tomar tienen los siguientes significados:

- `pid > 0`, es el *PID* del proceso al que le enviamos la señal.
- `pid = 0`, la señal es enviada a todos los procesos que pertenecen al mismo grupo que el proceso que la envía.
- `pid = -1`, la señal es enviada a todos aquellos procesos cuyo identificador real es igual al identificador efectivo del proceso que la envía. Si el proceso que la envía tiene identificador efectivo de superusuario, la señal es enviada a todos los procesos, excepto al proceso 0 —*swapper*— y al proceso 1 —*init*—.

Tabla 7.1: Señales del UNIX System V.

Nombre	Nº	Acción por defecto			No se puede ignorar	Restaura rutina por defecto
		Genera core	Termina	Ignora		
SIGHUP	1		✓			✓
SIGINT	2		✓			✓
SIGQUIT	3	✓	✓			✓
SIGILL	4	✓	✓			
SIGTRAP	5	✓	✓			
SIGIOT	6	✓	✓			✓
SIGEMT	7	✓	✓			✓
SIGFPE	8	✓	✓			✓
SIGKILL	9		✓		✓	✓
SIGBUS	10	✓	✓			✓
SIGSEGV	11	✓	✓			✓
SIGSYS	12	✓	✓			✓
SIGPIPE	13		✓			✓
SIGALRM	14		✓			✓
SIGTERM	15		✓			✓
SIGUSR1	16		✓			✓
SIGUSR2	17		✓			✓
SIGCLD	18			✓		✓
SIGPWR	19			✓		

- `pid < -1`, la señal es enviada a todos los procesos cuyo identificador de grupo coincide con el valor absoluto de `pid`.

En todos los casos, si el identificador efectivo del proceso no es el del superusuario o si el proceso que envía la señal no tiene privilegios sobre el proceso que la va a recibir, la llamada a `kill` falla.

El parámetro `sig` es el número de la señal que queremos enviar. Si `sig` vale 0 —señal nula— se efectúa una comprobación de errores, pero no se envía ninguna señal. Esta opción se puede utilizar para verificar la validez del identificador `pid`.

Si el envío se realiza satisfactoriamente, `kill` devuelve 0; en caso contrario, devuelve -1 y en `errno` estará el código del error producido.

En el ejemplo siguiente vemos cómo un proceso envía una señal a su proceso hijo para forzar su terminación.

Programa 7.1: Ejemplo del uso de `kill` (`kill-1.c`)

```
#include <signal.h>
main ()
```

```
{
    int pid;

    if ((pid = fork ()) == 0) {
        /* Código del proceso hijo. */
        while (1) {
            printf ("HIJO. PID = %d\n", getpid ());
            sleep (1);
        }
    }
    sleep (10);
    printf ("PADRE. Terminación del proceso %d\n", pid);
    kill (pid, SIGTERM);
    exit (0);
}
```

Si queremos que un proceso se envíe señales a sí mismo, podemos usar la llamada `raise`:

```
#include <signal.h>
int raise (int sig);
```

donde `sig` es el número de la señal que queremos enviar; `raise` se puede codificar a partir de `kill` de la siguiente forma:

```
int raise (int sig)
{
    return kill (getpid (), sig);
}
```

7.3.2 Tratamiento de señales (signal)

Para especificar qué tratamiento debe realizar un proceso al recibir una señal, se emplea la llamada `signal`:

```
#include <signal.h>
void (*signal (int sig, void (*action) ())) ();
```

La declaración de `signal` puede resultarnos extraña, pero es frecuente en los programas C. Como vemos, `signal` es del tipo *función que devuelve un puntero a una función void y recibe dos parámetros*:

- `sig` es el número de la señal cuya forma de tratamiento queremos especificar.
- `action` es la acción que se tomará al recibir la señal y puede tomar tres clases de valores:
 - `SIG_DFL`, indica que la acción a realizar cuando se recibe la señal es la acción por defecto asociada a la señal —manejador por defecto—. Por lo general,

esta acción consiste en terminar el proceso y en algunos casos también incluye generar un fichero `core`.

- `SIG_IGN`, indica que la señal se debe ignorar.
- `dirección`, es la dirección de la rutina de tratamiento de la señal —manejador suministrado por el usuario—; la declaración de esta función debe ajustarse al siguiente modelo:

```
#include <signal.h>
void handler (int sig [, int code, struct sigcontext *scp]);
```

Cuando se recibe la señal `sig`, el núcleo es quien se encarga de llamar a la rutina `handler` pasándole los parámetros `sig`, `code` y `scp`; `sig` es el número de la señal, `code` es una palabra que contiene información sobre el estado del hardware en el momento de invocar a `handler` y `scp` contiene información de contexto definida en `<signal.h>`. Tanto `code` como `scp` son parámetros opcionales y dependientes de la arquitectura de nuestra máquina.

La llamada a la rutina `handler` es asíncrona, lo cual quiere decir que puede darse en cualquier instante de la ejecución del programa. Esta rutina debe estar codificada para tratar las situaciones especiales que ocasionan que se produzca el envío de señales.

La llamada a `signal` devuelve el valor que tenía `action`, que puede servirnos para restaurarlo en cualquier instante posterior. Si se produce algún error, `signal` devuelve `SIG_ERR` y en `errno` estará el código del error producido.

Los valores de `SIG_DFL`, `SIG_IGN` y `SIG_ERR` son direcciones de funciones ya que los debe poder devolver `signal`. Sin embargo deben ser direcciones que sepamos que nunca van a estar ocupadas por otras funciones. Para darle solución a este problema, las constantes anteriores se definen de la forma siguiente:

```
#define SIG_DFL ((void (*) ())0)
#define SIG_IGN ((void (*) ())1)
#define SIG_ERR ((void (*) ())-1)
```

La conversión explícita de tipo que aparece delante de las constantes `-1`, `1` y `0` fuerza a que estas constantes sean tratadas como direcciones de inicio de funciones. Estas direcciones no contienen ninguna función, ya que en todas las arquitecturas UNIX son zona reservada para el núcleo. Además, la dirección `-1` ni siquiera tiene existencia física.

Como primer ejemplo de tratamiento de señales, vamos a codificar un programa que trata la señal `SIGINT`, generada al pulsar las teclas `Ctrl+C` —interrupción—.

Programa 7.2: Uso de `signal` (`signal-1.c`)

```
#include <stdio.h>
#include <signal.h>

/****
main: inicializa el manejador de la señal SIGINT y se pone en espera para
recibir la señal.
****/
main ()
```

```

{
    void manejador_SIGINT();

    if (signal (SIGINT, manejador_SIGINT) == SIG_ERR) {
        perror ("signal");
        exit (-1);
    }
    while (1) {
        printf ("En espera de Ctrl-C\n");
        sleep (999);
    }
}

/****
    manejador_SIGINT: rutina de tratamiento de la señal SIGINT.
****/
void manejador_SIGINT (int sig)
{

    printf ("Señal número %d recibida.\n", sig);
}

```

Al ejecutar este programa, la primera vez que se pulsa **Ctrl+C** aparece un mensaje por pantalla, pero la segunda termina la ejecución del proceso. Esto es debido a que cuando el núcleo llama a la rutina de tratamiento, se restaura la rutina por defecto como nueva rutina de tratamiento. Así, al salir de nuestra rutina y recibirse por segunda vez la señal, se invoca a la rutina por defecto, que se encarga de terminar el proceso. Para solventar este problema lo que tenemos que hacer es volver a llamar a **signal** dentro de nuestra rutina de tratamiento. La rutina de tratamiento debe codificarse como sigue:

```

void manejador_SIGINT (int sig)
{
    static cnt = 0;

    printf ("Señal número %d recibida.\n", sig);
    if (cnt < 20)
        printf ("Contador = %d\n", cnt++);
    else
        exit (0);
    if (signal (SIGINT, manejador_SIGINT) == SIG_ERR) {
        perror ("signal");
        exit (-1);
    }
}

```

Esto garantiza que la rutina de tratamiento sigue siendo la misma. Para las señales **SIGILL**, **SIGTRAP** y **SIGPWR** no es necesario tomar esta precaución, ya que no se restaura

la rutina por defecto, sino que sigue permaneciendo la que había. Con la rutina anterior el proceso termina después de pulsar 20 veces la teclas **Ctrl+C**.

Una pregunta que podemos hacernos es qué ocurre si se recibe una señal mientras se está tratando otra del mismo tipo. La respuesta depende de cómo esté codificada la rutina. En el ejemplo anterior esta situación producirá la terminación del proceso, ya que al recibirse la señal por primera vez la nueva rutina de tratamiento pasa a ser la rutina por defecto. Si queremos bloquear la recepción de señales de un tipo mientras se está tratando otra, tendremos que hacer una llamada a **signal** pasándole el parámetro **SIG_IGN**. Por ejemplo,

```
void manejador_SIGINT (int sig)
{
    static cnt = 0;

    if (signal (SIGINT, SIG_IGN) == SIG_ERR) {
        perror ("manejador_SIGINT");
        exit (-1);
    }
    printf ("Señal número %d recibida.\n", sig);
    if (cnt < 20)
        printf ("Contador = %d\n", cnt++);
    else
        exit (0);
    signal (SIGINT, manejador_SIGINT);
}
```

Otra posibilidad es permitir que una señal interrumpa a la rutina de tratamiento sin que esto cause la terminación del proceso. La siguiente secuencia de código permite este caso:

```
void manejador_SIGINT (int sig)
{
    static cnt = 0;

    signal (SIGINT, manejador_SIGINT);
    printf ("Señal número %d recibida.\n", sig);
    if (cnt < 20)
        printf ("Contador = %d\n", cnt++);
    else
        exit (0);
}
```

Esta técnica es desaconsejable, ya que se puede dar el caso de que la fuente de señales las genere a una velocidad mayor que la velocidad de tratamiento del manejador, con lo que el programa se vería desbordado. Lo normal es deshabilitar la recepción de una señal mientras está siendo tratada.

7.3.3 En espera de señales (pause)

En ocasiones puede interesar que un proceso suspenda su ejecución en espera de que ocurra algún evento exterior a él. Por ejemplo, al ejecutar una entrada/salida. Para estas situaciones nos valemos de la llamada a `pause`:

```
#include <unistd.h>
int pause (void);
```

Esta llamada hace que el proceso se pare en espera de la llegada de alguna señal. Cuando esto ocurre, y después de ejecutarse la rutina de tratamiento de la señal, `pause` devuelve el valor `-1` y en `errno` escribe el valor `EINTR` cuyo significado es que se ha producido una interrupción de la llamada. En otras llamadas al sistema, ésta es una condición de error, pero en el caso de `pause` es su forma correcta de operar. El programa continúa con la sentencia que sigue a `pause`.

El programa siguiente es un ejemplo en el que un proceso está esperando una señal y cuando recibe la señal `SIGUSR1` —nº 16— presenta por pantalla un número aleatorio.

Programa 7.3: Uso de `pause` (`pause-1.c`)

```
/**
$ pause-1
Forma de uso:
    $ pause_1 &
    kill -s USR1 pid
```

Por cada señal `SIGUSR1` que le enviemos al proceso, éste presentará por pantalla un número aleatorio.

```
***/
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

/**
main: arma los manejadores de SIGTERM y SIGUSR1 y se pone a
esperar señales.
***/
main ()
{
    void manejador_SIGUSR1 ();
    void manejador_SIGTERM ();

    signal (SIGTERM, manejador_SIGTERM);
    signal (SIGUSR1, manejador_SIGUSR1);
    while (1)
        pause ();
}
```



```

/****
    manejador_SIGTERM: saca un mensaje por pantalla y termina el proceso.
****/
void manejador_SIGTERM (int sig)
{
    printf ("Terminación del proceso %d a petición del usuario.\n",
            getpid ());
    exit (-1);
}

/****
    manejador_SIGUSR1: presenta un número aleatorio por pantalla.
****/
void manejador_SIGUSR1 (int sig)
{
    signal (sig, SIG_IGN);
    printf ("%d\n", rand ());
    signal (sig, manejador_SIGUSR1);
}

```

Para ejecutar este programa y que muestre los números aleatorios, debemos enviarle la señal SIGUSR1 con la orden kill,

```
$ kill -s USR1 pid
```

Podemos terminar la ejecución del proceso enviándole la señal número 15, SIGTERM.

7.3.4 Saltos globales (setjmp y longjmp)

En párrafos anteriores hemos indicado que la rutina de tratamiento de una señal puede hacer que el proceso vuelva a alguno de los estados por los que ha pasado con anterioridad. Esto no sólo es aplicable a las rutinas de tratamiento de señales sino que se puede extender a cualquier función. Para realizar esto nos valemos de las funciones estándar `setjmp` y `longjmp`:

```

#include <setjmp.h>
int setjmp (jmp_buf env);
void longjmp (jmp_buf env, int val);

```

La función `setjmp` guarda el entorno de pila en `env` para un uso posterior por parte de `longjmp`; `setjmp` devuelve el valor 0 en su primera llamada. El tipo de `env`, `jmp_buf`, está definido en el fichero de cabecera `<setjmp.h>`.

La función `longjmp` restaura el entorno guardado en `env` por una llamada previa a `setjmp`. Después de haberse ejecutado la llamada a `longjmp`, el flujo de la ejecución del programa vuelve al punto donde se hizo la llamada a `setjmp`, pero en este caso `setjmp` devuelve el valor `val` que hemos pasado mediante `longjmp`. Ésta es la forma de averiguar si `setjmp` está saliendo de una llamada para guardar el entorno o de una llamada de

`longjmp`; `longjmp` no puede hacer que `setjmp` devuelva 0, ya que en el caso de que `val` tome el valor 0, `setjmp` devolverá 1.

En la figura 7.2 podemos ver representada la forma de trabajar de `setjmp` y `longjmp`.

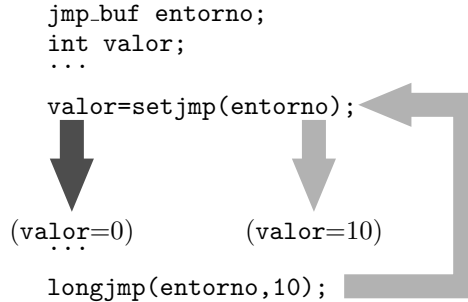


Figura 7.2: Saltos globales con `setjmp` y `longjmp`.

Las funciones `setjmp` y `longjmp` se pueden ver como una forma elaborada de implementar una sentencia `goto` no local, capaz de saltar desde una función a etiquetas que están en la misma o en otra función. Las etiquetas serían los entornos guardados por `setjmp` en la variable `env`.

Como ejemplo, vamos a ver un programa que cuenta desde 1 hasta 100 incrementando su valor cada 10 segundos. Además, cada 10 segundos se va a establecer un punto de repliegue de tal forma que si se recibe la señal `SIGUSR1` en algún instante, el programa reinicia su ejecución en ese punto.

Programa 7.4: Uso de `setjmp` y `longjmp` (`setjmp-1.c`)

```

/****
$ setjmp-1
Forma de uso:
    $ setjmp-1 &
    $ kill -s USR1 pid
Cada vez que le enviemos la señal SIGUSR1, el proceso reanuda su ejecución en
un punto de repliegue.
****/
#include <stdio.h>
#include <signal.h>
#include <setjmp.h>

jmp_buf env; /* Esta variable debe ser global para que la conozcan las rutinas de
               tratamiento de las señales. */

/****
main: se encarga de armar la señal SIGUSR1 y de establecer puntos de repliegue.
****/
main ()

```

```

{
    void manejador_SIGUSR1 ();
    int i;

    signal (SIGUSR1, manejador_SIGUSR1);

    for (i = 0; i < 100; i++) {
        if (setjmp (env) == 0)
            printf ("Punto de repliegue en el estado %d\n", i);
        else /* Esta parte se ejecuta cuando se efectúa una llamada a longjmp. */
            printf ("Regreso al punto de repliegue del estado %d\n", i);
        sleep (10);
    }
}

/****
    manejador_SIGUSR1: se encarga de resturar el proceso a alguno de sus estados
    previos guardados en la variable global env.
****/
void manejador_SIGUSR1 (int sig)
{

    signal (SIGUSR1, manejador_SIGUSR1);
    longjmp (env, 1);
}

```

7.4 Señales en el sistema 4.3BSD

La segunda interfaz para el manejo de señales que vamos a estudiar es la desarrollada por la Universidad de California en Berkeley. Las llamadas **kill**, **signal** y **pause** están disponibles aquí y se van a incorporar otras para hacer más versátil el manejo de las señales. Además, el UNIX de Berkeley añade nuevas señales a las existentes en la versión System V del UNIX de AT&T. Las señales nuevas son:

- SIGVTALRM** *Alarma de un temporizador en tiempo virtual.* Indica que un temporizador descendente en tiempo virtual ha llegado a cero. Su acción por defecto es terminar el proceso que la recibe.
- SIGPROF** *Alarma de un temporizador.* Indica que un temporizador descendente, que cuenta tanto en tiempo real como virtual, ha llegado a 0. Su acción por defecto es terminar el proceso que la recibe.
- SIGIO** *Señal de entrada/salida asíncrona.* Indica que un dispositivo o fichero está listo para una operación de entrada/salida. Su acción por defecto es ignorar la señal.

- SIGWINCH** *Cambio del tamaño de una ventana.* Se usa en las interfaces gráficas orientadas a ventanas como X-WINDOW. Su acción por defecto es ignorar la señal.
- SIGSTOP** *Señal de parada* de un proceso. Esta señal no proviene de un terminal de control. La señal no puede ser ignorada ni capturada y su acción por defecto es parar el proceso.
- SIGTSTP** *Señal de parada procedente de un terminal.* Es generada por el teclado. Su acción por defecto es parar el proceso.
- SIGCONT** *Continuar.* Señal para reanudar la ejecución de un proceso. Su acción por defecto es ignorar la señal.
- SIGTTIN** La reciben los procesos que se ejecutan en segundo plano y que intentan leer datos de un terminal de control. Su acción por defecto es parar el proceso.
- SIGTTOU** La reciben los procesos que se ejecutan en segundo plano y que intentan escribir en un terminal de control. Su acción por defecto es parar el proceso.
- SIGURG** Indica que ha llegado un *dato urgente* a través de un canal de entrada/salida. Su acción por defecto es ignorar la señal.
- SIGXCPU** Le indica al proceso que la recibe que ha superado su tiempo de CPU asignado. Su acción por defecto es terminar el proceso.
- SIGXFSZ** Le indica al proceso que la recibe que ha superado el tamaño máximo del fichero que puede manejar. Su acción por defecto es terminar el proceso.

De las señales que implementa el UNIX System V, SIGPWR no está implementada en 4.3BSD y SIGCLD pasa a llamarse SIGCHLD.

7.4.1 Tratamiento de señales (sigvec)

La llamada `signal` sigue disponible en el sistema 4.3BSD, pero además disponemos de otra llamada para especificar la forma de tratar una señal. Esta llamada es `sigvec` y su declaración es la siguiente:

```
#include <signal.h>
sigvec (int sig, struct sigvec *vec, struct sigvec *ovec);
```

donde `sig` es el número de la señal cuya forma de tratamiento queremos especificar. Podemos usar las constantes simbólicas que hay definidas en `<signal.h>` y que harán más legible el código.

Tanto `vec` como `ovec` son punteros a estructuras del tipo `sigvec`. Esta estructura está definida con los siguientes campos:

```
struct sigvec {
    void (*sv_handler) ();
    long sv_mask;
    long sv_flags;
};
```

El campo `sv_handler` es un puntero a una función que devuelve tipo `void` y tiene el mismo significado que el parámetro `action` de la llamada a `signal`. Este campo indica cuál es la rutina de tratamiento de la señal. Al igual que en el caso de `signal`, puede tomar tres tipos de valores con diferente significado:

- `SIG_DFL`, la rutina de tratamiento es la rutina de tratamiento por defecto.
- `SIG_IGN`, la señal será ignorada.
- `dirección`, la rutina de tratamiento empieza en `dirección` y ha sido codificada por el usuario.

El campo `sv_mask` codifica, en cada uno de sus bits, las señales que no deseamos que sean tratadas si son recibidas mientras se está ejecutando la rutina de tratamiento actual. Normalmente, este campo está a 0, lo que indica que mientras se está tratando una señal, cualquier otra señal puede interrumpir. Si alguno de los bits de `sv_mask` está a 1, vamos a impedir el anidamiento cuando se recibe esa señal. Para poner a 1 el bit correspondiente a la señal deseada, se puede usar la macro `sigmask` que se define de la forma:

```
#define sigmask(sig_nu) (1L << ((sig_nu) - 1))
```

En la figura 7.3 podemos ver el resultado de la actuación de `sigmask`.

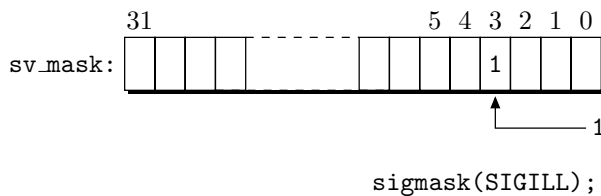


Figura 7.3: Acción de la macro `sigmask`.

El campo `sv_flags` codifica cuál será la semántica —significado— que se emplee en la recepción de la señal. Los siguientes bits están definidos para este campo:

- `SV_BSDSIG`, usar la semántica de Berkeley. Esto significa, entre otras cosas, que cuando se instala una rutina de tratamiento, permanecerá instalada hasta que se haga otra llamada a `sigvector` que instale una rutina nueva. Recordemos que una de las incomodidades que presentaban las señales en el UNIX System V es que, cuando se recibe una señal, la rutina de tratamiento volvía a ser la rutina por defecto y que, por lo tanto, debíamos llamar a `signal` para conservar la misma rutina de tratamiento. En 4.3BSD no se da esta situación si el bit `SV_BSDSIG` está activo.
- `SV_RESETHAND`, impone la misma semántica que la llamada `signal` de System V. Es decir, siempre se restaura la rutina de tratamiento por defecto antes de entrar en la rutina de tratamiento suministrada por el usuario.

Por lo general, el campo `sv_flags` estará a 0 en la llamada a `sigvector`.

Nos podemos referir a `vec` y `ovec` como los vectores de tratamiento de una señal o vectores de señal; `vec` apunta al que será el nuevo vector de señal y `ovec` devuelve un puntero al vector que había, por si deseamos restaurarlo posteriormente. Si `vec` es distinto de `NULL`, tiene el significado que hemos visto anteriormente; si vale `NULL`, la llamada a `sigvector` no cambiará nada en cuanto a la forma de tratar la señal, pero nos puede servir para preguntar por el vector de señal que está activo en este instante. `ovec` también puede ser un puntero `NULL`, en el caso de que no nos interese conocer cuál es el vector que había instalado. Si `vec` y `ovec` apuntan a una misma dirección, el valor de `vec` es leído antes de sobrescribir el valor de `ovec`.

La llamada a `sigvector` devuelve el valor 0 si se ejecuta satisfactoriamente, y -1 en caso de que se produzca algún error. En `errno` estará el código del tipo de error producido.

A la hora de escribir código que trate con señales, hemos de tener presente que se debe optar por hacer llamadas a `signal` o a `sigvector`, pero no se deben mezclar en un mismo programa llamadas a ambas.

A continuación se muestran algunos ejemplos de programas que ilustran las diferentes formas de uso de `sigvector`. En el primero vemos las tres formas de tratamiento de una señal: con un manejador —o rutina de tratamiento aportada por el usuario—, ignorando la señal o con la rutina de tratamiento por defecto. Este programa se envía a sí mismo en repetidas ocasiones la señal `SIGUSR1`; según la rutina de tratamiento que esté instalada, la respuesta será una u otra.

Programa 7.5: Uso de `sigvec` (`sigvec-1.c`)

```

/****
$ sigvec-1

Respuesta a una señal enviada desde un proceso a sí mismo. Vamos a probar
los modos:
    - rutina de tratamiento suministrada por el usuario,
    - ignorar la señal (SIG_IGN) y
    - rutina de tratamiento por defecto (SIG_DFL).
*/
#include <signal.h>

void manejador_SIGUSR1 (int sig)
{
    printf ("manejador_SIGUSR1: Señal recibida: %d.\n", sig);
}

main ()
{
    struct sigvec vec;

    /* Armamos el manejador de SIGUSR1 . */
    vec.sv_handler = manejador_SIGUSR1;

```

```

vec.sv_mask = 0;
vec.sv_flags = 0;
sigvec (SIGUSR1, &vec, 0);
printf ("Envío de SIGUSR1. Manejador activo.\n");
kill (getpid (), SIGUSR1);
printf ("Envío de SIGUSR1. Manejador activo.\n");
kill (getpid (), SIGUSR1);

/* Desactivamos el tratamiento de la señal SIGUSR1. */
vec.sv_handler = SIG_IGN;
sigvec (SIGUSR1, &vec, 0);
printf ("Envío de SIGUSR1. Ignorar activo.\n");
kill (getpid (), SIGUSR1);
printf ("Envío de SIGUSR1. Ignorar activo.\n");
kill (getpid (), SIGUSR1);

/* Armamos el manejador por defecto de SIGUSR1. */
vec.sv_handler = SIG_DFL;
sigvec (SIGUSR1, &vec, 0);
printf (
    "Envío de SIGUSR1. Rutina de tratamiento por defecto activa.\n");
kill (getpid (), SIGUSR1);

printf ("Este mensaje no debe aparecer en pantalla.\n");
}

```

En el ejemplo siguiente vamos a recibir una señal mientras se está manejando otra, pero al no estar permitido el anidamiento para la segunda señal, primero finalizará la rutina de tratamiento de la primera señal antes de tratar la segunda.

Programa 7.6: Uso de sigvec (sigvec-2.c)

```

/****
$ sigvec-2

En este ejemplo se recibe la señal SIGUSR1 mientras se está tratando
SIGALRM pero como está enmascarada, se espera a que termine la rutina de
SIGALRM antes de tratar SIGUSR1.
****/
#include <signal.h>

/****
manejador_SIGALRM: para simular que llega la señal SIGUSR1 mientras se
ejecuta esta rutina, en su código vamos a insertar una llamada a kill para
autoenviarse la señal SIGUSR1.
****/

```

```
void manejador_SIGALRM (int sig)
{

    printf ("manejador_SIGALRM. Señal recibida: %d\n", sig);
    kill (getpid (), SIGUSR1);
    printf("manejador_SIGALRM.Fin del tratamiento de la señal:%d.\n",sig);
}

/****
    manejador_SIGUSR1
****/
void manejador_SIGUSR1 (int sig)
{

    printf ("manejador_SIGUSR1. Señal recibida: %d.\n", sig);
    printf("manejador_SIGUSR1.Fin del tratamiento de la señal:%d.\n",sig);
}

main ()
{
    struct sigvec vec;

    /* Armamos el manejador de SIGALRM. */
    vec.sv_handler = manejador_SIGALRM;

    /* Deshabilitamos que la señal SIGUSR1 pueda interrumpir a la rutina de
    tratamiento de SIGALRM. */
    vec.sv_mask = sigmask (SIGUSR1);
    vec.sv_flags = 0;
    sigvec (SIGALRM, &vec, 0);

    /* Armamos el manejador de SIGUSR1. */
    vec.sv_handler = manejador_SIGUSR1;
    vec.sv_mask = 0;
    vec.sv_flags = 0;
    sigvec (SIGUSR1, &vec, 0);

    kill (getpid (), SIGALRM);
}
```

En el tercer ejemplo veremos un programa que permite el anidamiento en el tratamiento de señales, por lo que al recibir la señal SIGUSR1 cuando está tratando SIGALRM, suspenderá esta rutina para atender a la señal SIGUSR1. Reanudaremos al tratamiento de SIGALRM cuando hayamos terminado con SIGUSR1.

Programa 7.7: Uso de sigvec (sigvec-3.c)

```

/****
$ sigvec-3

En este ejemplo se recibe la señal SIGUSR1 mientras se está tratando
SIGALRM y como no está enmascarada, se suspende la ejecución de
manejador_SIGALRM para atender a la señal SIGUSR1. Al acabar
manejador_SIGUSR1, se reanuda la ejecución de la rutina de tratamiento
de SIGALRM.
****/
#include <signal.h>

/****
manejador_SIGALRM: para simular que llega la señal SIGUSR1 mientras se
ejecuta esta rutina, en su código vamos a insertar una llamada a kill para
autoenviarse la señal SIGUSR1.
****/
void manejador_SIGALRM (int sig)
{
    printf ("manejador_SIGALRM. Señal recibida: %d\n", sig);
    kill (getpid (), SIGUSR1);
    printf("manejador_SIGALRM.Fin del tratamiento de la señal:%d.\n",sig);
}

/****
manejador_SIGUSR1
****/
void manejador_SIGUSR1 (sig)
int sig;
{
    printf ("manejador_SIGUSR1. Señal recibida: %d.\n", sig);
    printf("manejador_SIGUSR1.Fin del tratamiento de la señal:%d.\n",sig);
}

main ()
{
    struct sigvec vec;

    /* Armamos el manejador de SIGALRM. */
    vec.sv_handler = manejador_SIGALRM;
    vec.sv_mask = 0;
    vec.sv_flags = 0;
    sigvec (SIGALRM, &vec, 0);

```

```

/* Armamos el manejador de SIGUSR1. */
vec.sv_handler = manejador_SIGUSR1;
sigvec (SIGUSR1, &vec, 0);

kill (getpid (), SIGALRM);
}

```

7.4.2 Protección de zonas críticas

En ocasiones puede interesarnos proteger determinadas zonas de código contra la llegada de alguna señal. Para realizar esto nos valdremos de las llamadas `sigsetmask` y `sigblock`:

```

#include <signal.h>
long sigsetmask (long mask);
long sigblock (long mask);

```

La función `sigsetmask` fija la máscara de señales actual. Esta máscara indica qué señales tienen bloqueada su recepción. La señal número *i* está bloqueada si el *i-ésimo* bit de `mask` vale 1. Este bit lo podemos fijar con la macro `sigmask(i)`. Naturalmente, las señales que no pueden ser ignoradas ni capturadas tampoco van a poder ser bloqueadas.

La función `sigblock` permite añadir a la máscara de señales actual aquellas señales que se especifiquen con el parámetro `mask`. Así, con `sigblock` podemos modificar la máscara fijada por `sigsetmask`.

Ambas llamadas, si se ejecutan satisfactoriamente, devuelven el valor que tenía la máscara de señales bloqueadas, en caso contrario devuelven el valor `-1` y en `errno` estará el código del error producido.

La diferencia entre `sigsetmask` y `sigblock` es que la primera fija la máscara de forma absoluta y, la segunda, de forma relativa. Las siguientes líneas de código aclaran este concepto.

```

long máscara_anterior;
...
// Bloqueamos la recepción de las señales SIGUSR1 y SIGUSR2
máscara_anterior = sigsetmask (sigmask (SIGUSR1) | sigmask (SIGUSR2));
...
// Añadimos la señal SIGINT, con lo que pasarán a ser tres las señales bloqueadas
sigblock (sigmask (SIGINT));
...
// Restauramos la máscara original
sigsetmask (máscara_anterior);

```

El programa siguiente muestra un ejemplo de uso del bloqueo de señales.

Programa 7.8: Uso de `sigsetmask` y `sigblock` (`sigmsk-1.c`)

```

#include <stdio.h>
#include <signal.h>

```

```

void manejador_SIGUSR1 (int sig)
{
    printf ("manejador_SIGUSR1. Señal recibida: %d.\n", sig);
    printf("manejador_SIGUSR1.Fin del tratamiento de la señal:%d.\n",sig);
}

main ()
{
    struct sigvec vec;
    long mascara_anterior;

    /* Armamos el manejador de SIGUSR1. */
    vec.sv_handler = manejador_SIGUSR1;
    vec.sv_mask = 0;
    vec.sv_flags = 0;
    sigvec (SIGUSR1, &vec, 0);

    /* Zona de código protegido. */
    mascara_anterior = sigblock (sigmask (SIGUSR1));
    printf ("Zona de código protegido.\n");
    kill (getpid (), SIGUSR1);
    printf ("En esta zona está deshabilitada la recepción de "
           "la señal SIGUSR1.\n");

    /* Restauramos la máscara anterior. */
    printf ("Fin de la zona de código protegido.\n\n");
    sigsetmask (mascara_anterior);
    printf ("Zona de código no protegido.\n");
    kill (getpid (), SIGUSR1);
}

```

7.4.3 En espera de señales (sigpause)

La llamada `sigpause` permite parar un proceso hasta que se reciba alguna de las señales deseadas. Si con `pause` podíamos parar un proceso en espera de la primera señal que se reciba, con `sigpause` podemos seleccionar la señal por la que esperamos. La declaración de esta función es:

```

#include <signal.h>
long sigpause (long mask);

```

La función `sigpause` bloquea la recepción de señales de acuerdo con el valor de `mask` de la misma forma que hace `sigsetmask`; a continuación se pone a esperar la llegada de alguna de las señales no bloqueadas. Si no queremos que `sigpause` bloquee ninguna señal, le pasaremos la máscara 0L.

Cuando **sigpause** termina su ejecución, restaura la máscara de señales que había antes de llamarla. La ejecución de **sigpause** termina cuando es interrumpida por una señal. Después de tratar la señal, **sigpause** hace que **errno** tome el valor **EINTR** y devuelve el valor **-1**.

En el programa siguiente bloqueamos una sección crítica contra la señal **SIGUSR1**. Cuando termina esta sección llamamos a **sigpause** para atender a las señales bloqueadas que hayan llegado mientras se ejecutaba la sección crítica. Para forzar la recepción de señales dentro del código de la sección crítica, vamos a incluir una llamada a **kill** para que el proceso se envíe una señal a sí mismo.

Programa 7.9: Protección de secciones críticas (**sigmsk-2.c**)

```
#include <stdio.h>
#include <signal.h>

void manejador_SIGUSR1 (int sig)
{
    printf ("manejador_SIGUSR1. Señal recibida: %d.\n", sig);
    printf ("manejador_SIGUSR1.Fin del tratamiento de la señal:%d.\n",sig);
}

main ()
{
    struct sigvec vec;
    long mascara_anterior;

    /* Armamos el manejador de SIGUSR1. */
    vec.sv_handler = manejador_SIGUSR1;
    vec.sv_mask = 0;
    vec.sv_flags = 0;
    sigvec (SIGUSR1, &vec, 0);

    /* Zona de código protegido. */
    mascara_anterior = sigblock (sigmask (SIGUSR1));
    printf ("Zona de código protegido.\n");
    kill (getpid (), SIGUSR1);
    printf ("En esta zona está deshabilitada la recepción de "
           "la señal SIGUSR1.\n");
    printf ("Pausa en espera de señales.\n");
    sigpause (mascara_anterior);
    printf ("Zona de código protegido.\n");
    kill (getpid (), SIGUSR1);

    /* Restauramos la máscara anterior. */
    printf ("Fin de la zona de código protegido.\n\n");
    sigsetmask (mascara_anterior);
}
```

```
printf ("Zona de código no protegido.\n");
kill (getpid (), SIGUSR1);
}
```

7.4.4 Concepto de máquina virtual en el sistema BSD

El manejo que hace el sistema BSD de las señales, junto con la idea de llamadas al sistema, lleva a muchos autores [Leffler *et al.*, 1989] a hablar de máquina virtual. En esta máquina virtual, las llamadas al sistema tienen el mismo papel que el conjunto de instrucciones de un procesador; las señales son la versión software de las interrupciones o de los *traps*, y las rutinas de tratamiento de señales son equivalentes a las rutinas de atención a interrupciones. Al igual que el hardware provee mecanismos para bloquear las interrupciones, los gestores de señales hacen posible el bloqueo de señales. Finalmente, las señales, al igual que las interrupciones, pueden manejar diferentes entornos de pila.

Estas equivalencias entre la máquina virtual UNIX y el hardware se muestran en la tabla 7.2.

Tabla 7.2: Comparación entre la máquina hardware y la máquina virtual UNIX.

Máquina hardware	Máquina virtual UNIX
Repertorio de instrucciones	Repertorio de llamadas al sistema
Interrupciones/ <i>Traps</i>	Señales
Manejador de interrupciones/ <i>traps</i>	Manejador de señales
Bloqueo de interrupciones	Bloqueo de señales
Pilas para interrupciones	Pilas para señales

7.5 Gestión de señales en POSIX

Además de los gestores de señales descritos, hay otros estándares adoptados por muchos fabricantes de sistemas UNIX. En concreto, el estándar POSIX 1003.1 propone un gestor de señales con una funcionalidad muy similar a la de 4.3BSD y que ha sido adoptada por Linux, 4.4BSD y FreeBSD.

La interfaz de señales de POSIX es uno de los aspectos más enrevesados de la norma debido al esfuerzo para hacerla compatible con la gestión tradicional de UNIX y no obligar así a modificar las aplicaciones ya existentes [Butenhof, 1997]. Esta complicación se hace especialmente patente cuando se mezclan las señales con la programación multihilo.

7.5.1 Tratamiento de señales (*sigaction*)

Con la llamada **sigaction** asociamos señales con acciones para indicar cómo responderá el proceso actual cuando reciba una señal. La declaración de **sigaction** es:

```
#include <signal.h>
int sigaction (int sig, const struct sigaction *act, struct sigaction *oact);
```

donde **sig** es la señal cuya acción queremos programar, **act** es la nueva acción que se le asocia y **oact** es la que tenía asociada. Si la llamada se ejecuta satisfactoriamente devuelve 0, en caso contrario devuelve -1, en **errno** se almacena el código del error producido y no se le asocia ninguna acción a la señal.

La norma POSIX establece que las señales de la tabla 7.3 estarán presentes en toda implementación de la biblioteca. Adicionalmente, para aquellos sistemas que implementen las extensiones de tiempo real habrá al menos **RTSIG_MAX** señales disponibles para el usuario con números en el rango **[SIGRTMIN, SIGRTMAX]**.

Tabla 7.3: Señales POSIX.

Señal	Acción	Descripción
SIGABRT		Abortar un proceso.
SIGALRM		Fin de un temporizador en tiempo real.
SIGBUS		Intento de acceso a una zona de memoria no definida.
SIGCHLD		Proceso hijo terminado o parado.
SIGCONT		Continuar con la ejecución, si está parado.
SIGFPE		Fallo en una operación aritmética.
SIGHUP		Desconexión.
SIGILL		Intento de ejecutar una instrucción ilegal.
SIGINT		Interrupción del terminal
SIGKILL		Matar el proceso.
SIGPIPE		Escritura en una tubería de la que nadie lee.
SIGQUIT		Salida del terminal.
SIGSEGV		Referencia no válida a memoria.
SIGSTOP		Parar la ejecución.
SIGTERM		Terminar.
SIGTSTP		Parar el terminal.
SIGTTIN		Intento de lectura en segundo plano.
SIGTTOU		Intento de escritura en segundo plano.
SIGUSR1		Señal número 1 de usuario.
SIGUSR2		Señal número 2 de usuario.
SIGPOLL		Evento interrogable.
Símbolos		Abortar la ejecución.
		Generar un volcado de memoria —fichero core —.
		Terminar la ejecución.
		Guardar el estado de terminación para su lectura con wait .
		Detener la ejecución.
		Reanudar la ejecución.
		Ignorar la señal.
		La señal no puede ser capturada ni ignorada.

(Continúa en la página siguiente...)

Tabla 7.3: Señales POSIX (continuación).

Señal	Acción	Descripción
SIGPROF	†⌚	Fin de un temporizador en tiempo virtual.
SIGSYS	⌚☐	Llamada al sistema errónea.
SIGTRAP	⌚☐	Punto de parada/traza alcanzado.
SIGURG	⏏	Dato urgente disponible en un conector.
SIGVTALRM	†⌚	Fin de un temporizador de usuario en tiempo virtual.
SIGXCPU	⌚☐	Tiempo de CPU excedido.
SIGXFSZ	⌚☐	Tamaño máximo de fichero excedido.
Símbolos	⚡	Abortar la ejecución.
	☐	Generar un volcado de memoria —fichero <code>core</code> —.
	†	Terminar la ejecución.
	⏏	Guardar el estado de terminación para su lectura con <code>wait</code> .
	⏸	Detener la ejecución.
	▶	Reanudar la ejecución.
	⏏	Ignorar la señal.
	⚡	La señal no puede ser capturada ni ignorada.

La estructura `struct sigaction` se define en el fichero de cabecera `signal.h` y tiene los campos siguientes: capturador de la señal —`sa_handler` o `sa_sigaction`—, máscara de señales —`sa_mask`— e indicadores —`sa_flags`—.

```

struct sigaction {
    // Si el indicador SA_SIGINFO no está activo.
    void (*sa_handler) (int signo);
    // Si el indicador SA_SIGINFO está activo.
    void (*sa_sigaction)(int signo, siginfo_t *info, void *context);
    sigset_t sa_mask; // Máscara de señales. Indica las señales adicionales
                      // que serán bloqueadas durante la ejecución del
                      // manejador.
    int sa_flags; // Indicadores.
}

```

Al definir un capturador de señal se optará por el prototipo de `sa_handler`, si no está activo el indicador `SA_SIGINFO` en `sa_flags`, o por el prototipo `sa_sigaction` si está activo; nunca se podrán definir los dos campos simultáneamente porque realmente se solapan. En cualquiera de los dos casos el capturador puede tomar los valores: `SIG_DFL` para asociarle a la señal el capturador por defecto, `SIG_IGN` para ignorar la señal, o un puntero a una función suministrada por el usuario y que se encargará de capturar y manejar la señal.

Para conseguir que `sa_handler` y `sa_sigaction` se solapen y aparezcan como dos campos distintos de una estructura, se definen apoyándose en una unión como se muestra a continuación:

```

union {
    void (*__sa_handler) (int);

```

```

    void (*__sa_sigaction) (int, siginfo_t *, void *);
} __sigaction_u;
...
#define sa_handler __sigaction_u.__sa_handler
#define sa_sigaction __sigaction_u.__sa_sigaction

```

Cuando la señal es capturada por `sa_handler`, a esta función se le pasa el argumento `signo` que contiene el número de la señal —compare con la llamada `signal` en el § 7.3.2 pág. 245—. Si la señal es capturada por `sa_sigaction` también se le pasa información referente al motivo que originó la señal —`info`— e información sobre el contexto de proceso interrumpido por la señal —`context`— que debe ser convertido explícitamente para que referencie un objeto de tipo `ucontext_t`.

El argumento `info` es un puntero a una estructura de tipo `siginfo_t` que contiene los campos siguientes:

```

typedef struct {
    int si_signo; // Número de la señal.
    int si_errno; // Número de error asociado con la señal.
    int si_code; // Código que especifica la causa de la señal.
    pid_t si_pid; // PID del proceso que envía la señal —señal SIGCHLD—.
    uid_t si_uid; // UID del proceso que envía la señal —señal SIGCHLD—.
    void *si_addr; // Dirección de la instrucción —señales SIGILL y SIGFPE—
                  // o del dato —señales SIGBUS y SIGSEGV— ilegal que
                  // origina la señal.
    int si_status; // Valor de salida o número de señal del proceso hijo
                  // —señal SIGCHLD—.
    long si_band; // Datos del evento —señal SIGPOLL—.
    union sigval si_value; // Datos de la señal —señales de
                          // tiempo real [SIGRTMIN, SIGRTMAX]—.
} siginfo_t;

```

Para consultar los valores del campo `si_code` se pueden usar las macros que se describen en la tabla 7.4.

Tabla 7.4: Macros para consultar el campo `si_code`.

Señal	Macro	Descripción
Todas	SI_USER	Señal enviada por <code>kill</code> .
	SI_QUEUE	Señal enviada por <code>sigqueue</code> .
	SI_TIMER	Expiración de un temporizador definido con <code>timer_settime</code> .
	SI_ASYNCIO	Fin de una operación de E/S asíncrona.
	SI_MSGQ	Llegada de un mensaje a una cola de mensajes vacía.
SIGILL	ILL_ILLOPC	Código de operación ilegal.
	ILL_ILLOPN	Operando ilegal.
	ILL_ILLADR	Modo de direccionamiento ilegal.

(Continúa en la página siguiente...)

Tabla 7.4: Macros para consultar el campo `si_code` (continuación).

Señal	Macro	Descripción
	ILL_ILLTRP	<i>Trap</i> ilegal.
	ILL_PRVOPC	Código de operación privilegiado.
	ILL_PRVREG	Registro privilegiado.
	ILL_COPROC	Error del coprocesador.
	ILL_BADSTK	Error interno de la pila.
SIGFPE	FPE_INTDIV	División entera por cero.
	FPE_INTOVF	Rebose con números enteros.
	FPE_FLTDIV	División por cero en coma flotante.
	FPE_FLOVF	Rebose con números en coma flotante.
	FPE_FLTUND	Pérdida de precisión en coma flotante.
	FPE_FLTRES	Resultado inexacto en coma flotante.
	FPE_FLTINV	Operación en coma flotante no válida.
	FPE_FLTSUB	Subíndice fuera de rango.
SIGSEGV	SEGV_MAPERR	Dirección no relacionada con una variable.
	SEGV_ACCERR	Permisos no válidos para el objeto.
SIGBUS	BUS_ADRALN	Alineación no válida de la dirección.
	BUS_ADRERR	Dirección física inexistente.
	BUS_OBJERR	Error de hardware concreto del objeto.
SIGTRAP	TRAP_BRKPT	Punto de parada del proceso.
	TRAP_TRACE	Punto de traza del proceso.
SIGCHLD	CLD_EXITED	Hijo terminado con una llamada a <code>exit</code> .
	CLD_KILLED	Terminación anormal del hijo sin creación de <code>core</code> .
	CLD_DUMPED	Terminación anormal del hijo con creación de <code>core</code> .
	CLD_TRAPPED	El proceso hijo se encuentra en un punto de traza.
	CLD_STOPPED	Proceso hijo parado.
	CLD_CONTINUED	El proceso hijo reanuda su ejecución.
SIGPOLL	POLL_IN	Datos de entrada disponibles.
	POLL_OUT	Zonas de memoria de salida disponibles.
	POLL_MSG	Mensaje de entrada disponible.
	POLL_ERR	Error de E/S.
	POLL_PRI	Entrada de alta prioridad disponible.
	POLL_HUP	Dispositivo desconectado.

El tipo `ucontext_t` aparece definido en el fichero de cabecera `<ucontext.h>` y contiene información sobre el contexto de usuario que puede ser manipulada con las funciones `getcontext`, `setcontext`, `makecontext` y `swapcontext`.

El campo `sa_mask` de la estructura `struct sigaction` codifica las señales adicionales, además de `sig`, que serán bloqueadas cuando ésta sea capturada. Por lo tanto, si se recibe alguna de estas señales durante el tratamiento de `sig`, serán almacenadas en una cola para su tratamiento posterior. Para definir la máscara `sa_mask` se emplean las funciones `sigprocmask` y `pthread_sigmask` como veremos en el § 7.5.3 pág. 268.

El campo `sa_flags` contiene indicadores que modifican el comportamiento de la señal `sig` y puede tomar los valores siguientes:

- SA_NOCLDSTOP** No generar la señal **SIGCHLD** cuando alguno de los procesos hijo se para o reanuda su ejecución.
- SA_ONSTACK** Enviar la señal sólo cuando el proceso se ejecute en un contexto de pila alternativo declarado con la función **sigaltstack**. Si no se ha definido un contexto alternativo entonces será enviada en el contexto actual.
- SA_RESETHAND** Restaurar el manejador por defecto —**SIG_DFL**— una vez que la señal ha sido capturada y manejada.
- SA_RESTART** Modifica el comportamiento de las funciones interrumpibles. Estas funciones devuelven **EINTR** en **errno** cuando reciben una señal pero se reinician sin fallar cuando **SA_RESTART** está activo.
- SA_SIGINFO** Indica el prototipo del capturador de la señal:
- ```
void func(int signo);
```
- cuando **SA\_SIGINFO** no está activo y,
- ```
void func(int signo, siginfo_t *info, void *context);
```
- cuando está activo.
- SA_NOCLDWAIT** No transformar los procesos hijo en zombis cuando terminan — consulte el § 5.2 pág. 162—.
- SA_NODEFER** La señal no será automáticamente bloqueada cuando se ejecute el manejador de la misma.

7.5.2 Envío de señales

En POSIX también podemos usar la llamada **kill** para enviar señales y su declaración coincide con la que hemos estudiado en el § 7.3.1 pág. 243, sin embargo su comportamiento presenta algunas variantes en procesos con varios hilos. Como regla general tendremos en cuenta que las señales afectan a todo el proceso, incluyendo sus hilos, y así, las señales que no pueden ser capturadas —**SIGKILL** y **SIGSTOP**— terminan o detienen la ejecución de todos los hilos de un proceso. No obstante hay señales que sólo afectan al hilo que las origina, se trata de las señales síncronas asociadas a un contexto hardware como **SIGFPE**, **SIGTRAP** o **SIGTRAP**.

Los sistemas que implementan las extensiones de tiempo real disponen de la llamada **sigqueue** para enviar señales acompañadas de información:

```
#include <signal.h>
int sigqueue (pid_t pid, int signo, const union sigval value);
```

donde **pid** es el identificador del proceso al que se le envía la señal número **signo** y **value** es el dato de tipo **union sigval** que la acompaña. La definición de este tipo la encontramos en el fichero de cabecera **<signal.h>**:

```
union sigval {
    int signal_int;
```

```
void *signal_ptr;
};
```

Recordemos que el manejador del proceso receptor podrá examinar los datos que acompañan a la señal sólo si tiene asociada una acción con el indicador `SA_SIGINFO` activo.

En el caso de que `signo` valga 0 —señal nula— se efectúan todas las comprobaciones necesarias para enviar la señal pero el envío no se realiza. Esto nos puede servir para comprobar si el proceso `pid` existe o no.

Para realizar envío de señales entre hilos de un mismo proceso se emplea la función `pthread_kill`:

```
#include <signal.h>
int pthread_kill (pthread_t thread, int sig);
```

donde `thread` es el hilo al que le enviamos la señal y `sig` es el número de la señal enviada. Nuevamente se tendrá en cuenta que `SIGKILL` afecta a todo el proceso y, por lo tanto, para terminar la ejecución de un hilo concreto utilizaremos `pthread_cancel`.

7.5.3 Máscara de señales

La máscara que indica las señales bloqueadas en un proceso o hilo determinado es un objeto de tipo `sigset_t` al que llamamos conjunto de señales y está definido en `<signal.h>`. Aunque `sigset_t` suele ser un tipo entero donde cada bit está asociado a una señal, no necesitamos conocer su estructura y estos objetos pueden ser manipulados con funciones específicas para activar y desactivar esos bits. Para manipular conjuntos de señales hay que empezar inicializándolos con `sigfillset` y `sigemptyset`:

```
#include <signal.h>
int sigfillset (sigset_t *set);
int sigemptyset (sigset_t *set);
```

La función `sigfillset` incluye en el conjunto referenciado por `set` todas las señales definidas en la norma y que aparecen en la tabla 7.3. Si utilizamos este conjunto como máscara todas las señales estarán bloqueadas. La función `sigemptyset` excluye del conjunto referenciado por `set` todas las señales definidas en la norma, desbloqueando así su recepción si utilizamos este conjunto como máscara.

Estas funciones devuelven 0 si se ejecutan satisfactoriamente o -1 en caso de error, guardando en `errno` el código del error producido.

Una vez que se ha inicializado un conjunto podemos manipularlo para incluir una señal —`sigaddset`—, excluirla —`sigdelset`— o preguntar si está incluida —`sigismember`—:

```
#include <signal.h>
int sigaddset (sigset_t *set, int signo);
int sigdelset (sigset_t *set, int signo);
int sigismember (const sigset_t *set, int signo);
```

En las tres funciones `set` es la referencia al conjunto y `signo` es el número de la señal. Las dos primeras devuelven 0 si se ejecutan satisfactoriamente o -1 en caso de error y la

tercera devuelve 1 si la señal **signo** pertenece al conjunto, 0 si no pertenece y -1 en caso de error.

A partir de un conjunto podemos modificar la máscara de señales bloqueadas en un proceso —**sigprocmask**— o en un hilo —**pthread_sigmask**—:

```
#include <signal.h>
int sigprocmask (int how, const sigset_t * set, sigset_t * oset);
int pthread_sigmask (int how, const sigset_t * set, sigset_t * oset);
```

El parámetro **set** es una referencia al conjunto que se utilizará para modificar la máscara de señales de acuerdo con las indicaciones de **how** que se muestran a continuación:

- SIG_BLOCK** La máscara resultante es el resultado de la unión de la máscara actual y el conjunto.
- SIG_SETMASK** La máscara resultante es la indicada en el conjunto.
- SIG_UNBLOCK** La máscara resultante es la intersección de la máscara actual y el complementario del conjunto. Es decir, las señales incluidas en el conjunto quedarán desbloqueadas en la nueva máscara de señales.

En la zona de memoria referenciada por **oset** se almacena la máscara de señales anterior y podremos usar este puntero para restaurarla con posterioridad.

Si el puntero **set** vale **NULL** las funciones anteriores se limitan a consultar el valor de la máscara actual sin modificarla.

La norma POSIX no especifica cuál es el resultado de ejecutar **sigprocmask** en un proceso multihilo, por lo tanto únicamente utilizaremos esta función con procesos tradicionales que se componen de un solo flujo de control.

Ambas funciones devuelven 0 si se ejecutan satisfactoriamente. En caso de fallo, **pthread_sigmask** devuelve el código del error producido mientras que **sigprocmask** devuelve -1 y almacena el código del error en la variable **errno**.

La secuencia de código siguiente muestra cómo definir un conjunto de señales para añadirlo a la máscara actual y bloquear también la recepción de las señales **SIGFPE** y **SIGSEGV** en el hilo actual:

```
sigset_t set, oset;
int error;
...
sigemptyset (&set);
sigaddset (&set, SIGFPE);
sigaddset (&set, SIGSEGV);
error = pthread_sigmask (SIG_BLOCK, &set, &oset);
if (error)
    // Tratamiento del error
```

Como ejemplo de aplicación de algunas de las funciones para manejo de señales de la biblioteca POSIX, veremos la forma de implementar una estructura de control parecida a la que ofrece Java para tratar errores. Cuando se ejecuta una aplicación Java, tanto la máquina virtual como la propia aplicación pueden detectar estados internos erróneos y lanzar avisos que así lo indiquen. Estos avisos se denominan *excepciones* y pueden

ser capturados y tratados síncronamente. Como ejemplo, la estructura de control para proteger la sección de código `Instruccionesprotegidas` frente a excepciones `TipoExcepción1` y `TipoExcepción2` es:

```
try {
    Instruccionesprotegidas
} catch (TipoExcepción1 e1) {
    Instrucciones1
} catch (TipoExcepción2 e2) {
    Instrucciones2
} finally {
    Instruccionesfinales
}
```

Si durante la ejecución de `Instruccionesprotegidas` es lanzada alguna excepción de tipo `TipoExcepción1` o `TipoExcepción2` entonces es capturada y manejada por el bloque de código `Instrucciones1` o `Instrucciones2` respectivamente. En cualquier caso el bloque de código `Instruccionesfinales` siempre se ejecuta independientemente de que fallen o no las `Instruccionesprotegidas` [Arnold *et al.*, 2000].

Para implementar en lenguaje C una estructura de control parecida a la anterior es necesario apoyarse en las facilidades que brinda el preprocesador de C. En el fichero de cabecera `exception.h` se definen las macros `try`, `catch`, `finally` y `throw` necesarias para simular la sintaxis del lenguaje Java.

Fichero 7.10: Macros para gestionar excepciones (exception.h)

```
/**
 * Ejemplo de uso de las funciones de salto global y de manejo de señales para
 * implementar la estructura de control try-catch-finally de manejo de
 * excepciones parecida a la de Java.
 */
***
#ifndef _EXCEPTION_
#define _EXCEPTION_
#include <signal.h>
#include <setjmp.h>

typedef int exception_t;

/* Excepciones predefinidas. */
extern exception_t
    Numeric_Error, /* Asociada a la señal SIGFPE. */
    Illegal_Instruction, /* Asociada a la señal SIGILL. */
    Segment_Violation, /* Asociada a la señal SIGSEGV. */
    Unknown_Error;

/* Lugar donde almacenamos el entorno para efectuar saltos globales. */
extern jmp_buf __jmpbuf;
```

```

/* Macros para formar estructuras de control del tipo:
    tray {
        Código.
    } catch (Excepción1) {
        Tratamiento de la Excepción1.
    } catch (Excepción2) {
        Tratamiento de la Excepción2.
    } finally {
        Tratamiento final del bloque. Se ejecuta en todos los casos.
    }
*/
#define try {exception_t current_exception;\
    sigset_t oset;\
    struct sigaction oact;\
    void bloquea_sig (sigset_t *oset, struct sigaction *oact);\
    void desbloquea_sig (sigset_t *oset, struct sigaction *oact);\
    bloquea_sig (&oset, &oact);\
    if ((current_exception = (exception_t) setjmp (__jmpbuf)) == 0)
#define catch(exception) else if (current_exception == (int)&exception)
#define finally desbloquea_sig (&oset, &oact);}
#define throw(exception) longjmp (__jmpbuf, (int)&exception)
#endif

```

Podemos ver que las macros se implementan a partir de las funciones `setjmp` y `longjmp` que nos permiten realizar saltos globales controlando el contexto de ejecución.

El fichero también exporta el tipo `exception_t` que nos permite definir nuevas excepciones y las variables `Numeric_Error`, `Illegal_Instruction`, `Segment_Violation` y `Unknown_Error` que son excepciones predefinidas asociadas a señales que indican fallos detectados por el sistema operativo y que éste comunica a través de las señales `SIGFPE`, `SIGILL` y `SIGSEGV`. Para que la recepción de alguna de estas señales se pueda traducir en la generación de una excepción se utiliza la función `manejador` que podemos ver en el fichero `exception.c`. Este manejador es instalado a través de la función `bloquear_sig` invocada por la macro `try`.

Fichero 7.11: Funciones invocadas por las macros `try-catch-finally` (`exception.c`)

```

/**
    Funciones invocadas por las macros try-catch-finally.
***/
#include <errno.h>
#include "errores.h"
#include "exception.h"

/* Variables globales. */
exception_t Numeric_Error,
            Illegal_Instruction,
            Segment_Violation,

```

```

    Unknown_Error;

/* Lugar donde almacenamos el entorno para efectuar saltos globales. */
jmp_buf __jmpbuf;

/**
 * manejador: se encarga de capturar las señales SIGFPE, SIGILL y SIGSEGV para
 * lanzar las excepciones asociadas a ellas.
 */
void manejador (int sig)
{
    switch (sig) {
    case SIGFPE:
        throw (Numeric_Error);
        break;
    case SIGILL:
        throw (Illegal_Instruction);
        break;
    case SIGSEGV:
        throw (Segment_Violation);
        break;
    default:
        throw (Unknown_Error);
    }
}

/**
 * bloquea_sig: instala el manejador de las señales a las que se les han asociado
 * excepciones y desbloquea la recepción de esas señales. Esta función es invocada
 * por la macro try.
 */
void bloquea_sig (sigset_t *oset, struct sigaction *oact)
{
    sigset_t set;
    struct sigaction act;
    int error;

    /* Desbloqueamos la recepción de las señales SIGFPE, SIGILL y SIGSEGV. */
    sigemptyset (&set);
    sigaddset (&set, SIGFPE);
    sigaddset (&set, SIGILL);
    sigaddset (&set, SIGSEGV);
    error = sigprocmask (SIG_UNBLOCK, &set, oset);
    if (error == -1)
        error_fatal (errno, "sigprocmask");
}

```

```

/* Instalación del manejador de las señales anteriores. */
act.sa_handler = manejador;
sigfillset (&act.sa_mask);
act.sa_flags = SA_RESETHAND;
error = sigaction (SIGFPE, &act, oact);
if (error == -1)
    error_fatal (errno, "sigaction");
error = sigaction (SIGILL, &act, oact);
if (error == -1)
    error_fatal (errno, "sigaction");
error = sigaction (SIGSEGV, &act, oact);
if (error == -1)
    error_fatal (errno, "sigaction");
}

/****
desbloquea_sig: restaura la máscara y los manejadores de señales que había
antes de entrar en el bloque try-catch-finally. Esta función es invocada por
la macro finally.
****/
void desbloquea_sig (sigset_t *oset, struct sigaction *oact)
{
    int error;

    error = sigprocmask (SIG_SETMASK, oset, NULL);
    if (error == -1)
        error_fatal (errno, "sigprocmask");

    error = sigaction (SIGFPE, oact, NULL);
    if (error == -1)
        error_fatal (errno, "sigaction");
    error = sigaction (SIGILL, oact, NULL);
    if (error == -1)
        error_fatal (errno, "sigaction");
    error = sigaction (SIGSEGV, oact, NULL);
    if (error == -1)
        error_fatal (errno, "sigaction");
}

```

En el programa de prueba `excep.c` se declara la excepción `Error_de_Lectura` y vemos cómo escribir un bloque protegido frente a algunas excepciones. En concreto, durante la ejecución de las instrucciones protegidas se fuerza una violación de segmento al intentar escribir una cadena en un puntero `NULL`, esto da lugar a que el sistema detecte el fallo y se lance automáticamente la excepción `Segment_Violation`. Mientras esta excepción está siendo tratada se fuerza un error numérico realizando la operación `0/0`, esto da lugar a

que el sistema detecte el fallo y se lance automáticamente la excepción `Numeric_Error`. Durante el tratamiento de esta excepción se lanza `Error_de_Lectura` con la macro `throw`. Por último, una vez tratadas todas las excepciones que se han ido capturando, se ejecuta el código asociado a la parte `finally`.

Programa 7.12: Prueba de la estructura de control `try-catch-finally` (`excep.c`)

```

/**
$ excep
Programa de prueba de la estructura try-catch-finally.

Compilación:
$ cc -o excep excep.c exception.c
***/
#include <stdio.h>
#include "exception.h"

main ()
{
    exception_t Error_de_Lectura;
    int x;

    try {
        printf ("Bloque de código protegido.\n");
        strcpy (NULL, "En un lugar de la Mancha...");
        printf ("Fin del bloque de código protegido.\n");
    } catch (Error_de_Lectura){
        printf ("Error de lectura.\n");
    } catch (Numeric_Error) {
        printf ("Error numérico.\n");
        throw (Error_de_Lectura);
    } catch (Illegal_Instruction) {
        printf ("Instrucción Ilegal.\n");
    } catch (Segment_Violation) {
        printf ("Violación de segmento.\n");
        x = 0/0;
    } finally {
        printf ("Código final.\n");
    }
}

```

7.5.4 Recepción síncrona de señales

Las señales son un mecanismo diseñado esencialmente para comunicar eventos asíncronos, sin embargo los procesos y los hilos pueden tratar las señales de forma síncrona impidiendo así que interrumpan en cualquier instante alguna zona de código crítica. Esto se consigue

programando la máscara de señales bloqueadas y consultando la cola de señales recibidas y pendientes de tratamiento. Para ello se emplea alguna de las funciones siguientes:

```
#include <signal.h>
int sigpending (sigset_t *set);
int sigsuspend (const sigset_t *sigmask);
int sigwait (const sigset_t *set, int *sig);
```

La función **sigpending** se utiliza para preguntar si hay señales enviadas al proceso o al hilo actual pero que al haber sido bloqueadas están aún pendientes de ser tratadas. En la zona referenciada por **set** se almacena el conjunto de señales pendientes. Si la función se ejecuta satisfactoriamente devuelve 0, en caso contrario devuelve -1 y en **errno** estará el código del error producido.

La función **sigsuspend** bloquea la recepción de señales de acuerdo con el valor de **sigmask** de la misma forma que hacen **sigprocmask** o **pthread_sigmask**; a continuación se pone a esperar la llegada de alguna de las señales no bloqueadas. Cuando **sigsuspend** termina su ejecución, restaura la máscara de señales que había antes de llamarla. La ejecución de **sigsuspend** termina cuando es interrumpida por una señal. Después de tratar la señal, **sigsuspend** hace que **errno** tome el valor **EINTR** y devuelve el valor -1.

La función **sigwait** se utiliza para esperar indefinidamente hasta que el hilo actual tenga alguna señal pendiente. Las señales por las que preguntamos están en el conjunto referenciado por **set** y el número de la señal que se retira de la cola de señales pendientes es devuelto a través de la referencia **sig**. Como es lógico, para que esta llamada tenga sentido y podamos esperar la llegada de una señal es necesario que esté bloqueada porque de lo contrario no entraría en la cola sino que sería capturada y activaría el manejador. Para evitar este tipo de situaciones indeseadas se suele considerar una práctica segura de programación bloquear todas las señales en el hilo principal de un proceso antes de la creación de los restantes hilos; así cuando éstos sean creados heredarán la máscara de señales del hilo principal y podrán usar **sigwait** de forma segura. Por otro lado, cuando varios hilos ejecutan **sigwait** para esperar una misma señal, sólo en uno de ellos devuelve el control la función, los restantes siguen esperando. Si la función se ejecuta satisfactoriamente devuelve 0, en caso contrario devuelve el código del error producido.

En aquellos sistemas que implementen las extensiones de tiempo real se dispone de dos funciones adicionales para esperar la llegada de señales de forma síncrona:

```
#include <time.h>
#include <signal.h>
int sigwaitinfo (const sigset_t *set, siginfo_t *info);
int sigtimedwait (const sigset_t *set, siginfo_t *info,
                  const struct timespec *timeout);
```

La función **sigwaitinfo** pregunta si alguna de las señales del conjunto referenciado por **set** está pendiente de tratamiento y si es así devuelve a través de la referencia **info** la información de la señal; en caso contrario detiene indefinidamente la ejecución del hilo hasta que llegue alguna de las señales esperadas. El argumento **info** referencia un objeto de tipo **siginfo_t** descrito en el § 7.5.1 pág. 265 donde se almacena el número de la señal —campo **si_signo**— y su causa —campo **si_code**—. Adicionalmente, si la señal

ha sido enviada con una llamada a `sigqueue`, también se almacenan los datos asociados a la señal —campo `si_value`—.

La función `sigtimedwait` tiene el mismo comportamiento que `sigwaitinfo` excepto que la espera hasta la llegada de alguna de las señales indicadas en `set` está acotada por el plazo indicado en `timeout`. El tipo `struct timespec` se define en el fichero de cabecera `<time.h>` y tiene los campos siguientes:

```
struct timespec {
    time_t tv_sec; // Segundos
    long tv_nsec; // Nanosegundos
}
```

En el caso de que el argumento `timeout` sea un puntero `NULL`, la función `sigtimedwait` se comportará lo mismo que `sigwaitinfo` por lo que ésta puede ser implementada como macro a partir de aquélla:

```
#define sigwaitinfo(set, info) sigtimedwait ((set), (info), NULL)
```

7.6 Ejemplo de aplicación de las señales

Presentaré a continuación algunos ejemplos prácticos donde se muestra la forma de aprovechar los recursos que brinda el manejo de señales para construir programas más robustos y versátiles.

7.6.1 Software tolerante a fallos

La forma habitual que tienen los programas de responder ante situaciones no previstas o condiciones de error es mediante una terminación abrupta. Este final suele estar provocado por una señal para la cual no había montado un manejador y, al ejecutarse la rutina por defecto, se produce la terminación del proceso. En programas sencillos o de prueba, esta respuesta puede no tener mayor trascendencia, pero en programas donde las especificaciones nos imponen que no se puede producir una terminación abrupta, hay que tener previstos mecanismos para tratar situaciones excepcionales. Imaginemos, por ejemplo, un programa que controla un proceso de producción; al programa le llegan datos procedentes de sensores y en función de ellos toma decisiones. Si el programa se para de forma anormal, por ejemplo por un error de división por cero, y el proceso físico queda descontrolado, puede suponer la pérdida de mucho dinero.

Un programa tolerante a fallos es aquel que puede recuperarse de situaciones de error que se producen de forma inesperada. Una técnica habitual para recuperarse de un fallo es hacer que el programa regrese a un estado previo cuando se llega a una situación en la que es imposible continuar con el flujo normal de control. Esta situación intermedia puede ser una reinicialización total del programa.

Como ejemplo, vamos a ver un programa que se reinicializa cada vez que se produce un error en coma flotante. Para simular este tipo de errores, vamos a invocar repetidas veces a una función que efectúa divisiones entre dos números enteros generados aleatoriamente. Cuando el denominador de la fracción sea 0, el programa recibirá la señal `SIGFPE` y

deberemos tratarla. El tratamiento va a consistir en un salto no local para reinicializar el programa.

Programa 7.13: Gestión de los fallos en coma flotante (sigfpe.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <setjmp.h>

/* Variable global para guardar el estado al que volveremos cuando se produzca
un fallo en coma flotante. */
jmp_buf entorno;

/**
manejador_SIGFPE: esta función realiza un salto al entorno guardado en la
variable global entorno.
***/
void manejador_SIGFPE (int sig)
{
    static int nro = 0;

    printf ("\nSeñal recibida: %d. Error nro. %d en coma flotante.\n",
            sig, ++nro);
    longjmp (entorno, 1);
}

/**
proceso_de_calculo: esta función simula un proceso de cálculo que genera
aleatoriamente fallos de coma flotante.
***/
proceso_de_calculo ()
{
    long n = random() % 10, d = random() % 10;

    sleep (1);
    printf ("%d / %d = ", n, d);
    printf ("%d\n", n/d);
}

main ()
{
    if (setjmp (entorno) == 0)
        printf ("Inicialización del programa.\n");
    else
        printf("Reinicialización del programa tras detectar un fallo.\n");
}
```

```

    signal (SIGFPE, manejador_SIGFPE);
    while (!0)
        proceso_de_calculo ();
}

```

La solución presentada aquí es un caso especial de recuperación inversa¹ donde el estado anterior considerado seguro es el estado inicial —arranque del proceso—. Se puede aprovechar la posibilidad de guardar entornos previos que brinda `setjmp` para guardar otros estados previos considerados seguros e implementar así una recuperación inversa más general.

7.6.2 Sincronización de procesos

Las señales pueden considerarse como una forma rudimentaria de comunicación entre procesos. Cuando un proceso le dice al núcleo que le envíe una señal a otro proceso, en cierto modo se está produciendo un intercambio de información entre los dos procesos. Naturalmente, como mecanismo de comunicación, las señales son algo muy restringido, pero para el caso de la sincronización entre procesos pueden ser más que suficiente.

Como ejemplo, veamos la forma de comunicar dos procesos a través de un fichero ordinario. Este mecanismo es bastante lento, ya que se ve involucrado el acceso a disco, pero puede ser un ejercicio interesante para introducir los conceptos de sincronización y para practicar con las señales.

El problema que tenemos que resolver es el de conseguir que un proceso escriba en un fichero y el otro lea de él. Naturalmente, para que el proceso receptor pueda sacar algo del fichero, primero tiene que haber sido escrito algo por el proceso emisor. La forma de indicarle al proceso receptor que hay algo disponible para leer es enviarle una señal. El receptor, a su vez, le devuelve otra señal al emisor, cuando ha efectuado la lectura, para indicarle que está listo para recibir más datos. En el diagrama de estados de la figura 7.4 podemos ver el protocolo del que se valen los dos procesos para sincronizarse.

El programa consta de una función principal donde se crean un proceso padre y otro hijo que pasan a comunicarse entre sí. El código es el siguiente:

Programa 7.14: Sincronización mediante el intercambio de señales (`sincro.c`)

```

#include <stdio.h>
#include <signal.h>

#define MAX 256

/* Identificador de proceso del emisor y del receptor. */
int pid_emisor, pid_receptor;

```

¹La *recuperación inversa* es un método de redundancia dinámica de recuperación ante fallos que consiste en hacer retroceder el proceso hacia un estado previo considerado seguro —punto de recuperación o repliegue—; en oposición nos encontramos con la *recuperación directa* donde al proceso se le hace avanzar desde el estado erróneo hacia un nuevo estado considerado seguro. Los métodos de recuperación directa se podrían implementar con manejadores de señales y en algunos lenguajes de programación, por ejemplo Ada, con los manejadores de excepciones [Burns & Wellings, 2001, cap. 5 y 6].

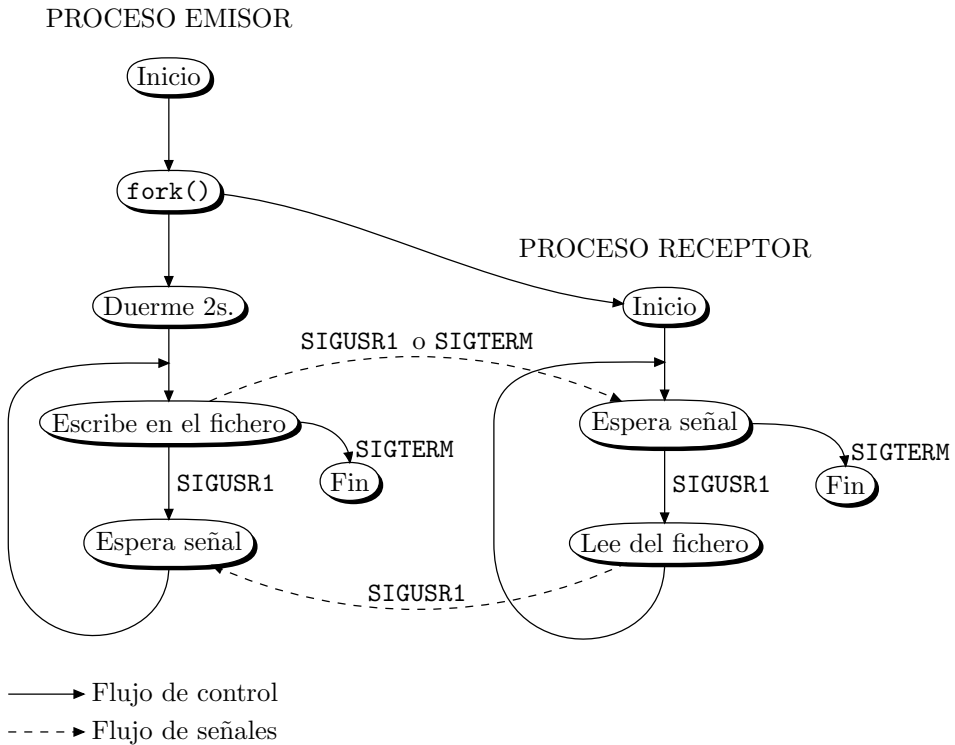


Figura 7.4: Sincronización de dos procesos mediante el intercambio de señales.

```

/**
  enviar: rutina de tratamiento de la señal SIGUSR1 para el proceso emisor.
  Este proceso se encarga de leer un mensaje del flujo estándar de entrada y
  escribirlo en el fichero buzón. Acto seguido le envía la señal SIGUSR1 al
  proceso receptor. Si se pulsa Ctrl+D, la rutina envía la señal SIGTERM al
  proceso receptor y termina su ejecución.
  */
void enviar (int sig)
{
    char str [MAX];
    FILE *fp;

    printf ("PROCESO EMISOR. MENSAJE: ");
    if (fgets (str, MAX, stdin) != NULL) {
        if ((fp = fopen ("buzón", "w")) == NULL) {
            perror ("enviar");
            kill (pid_receptor, SIGTERM);
        }
    }
}

```

```

    }
    fputs (str, fp);
    fclose (fp);
    signal (SIGUSR1, enviar);
    kill (pid_receptor, SIGUSR1);
} else {
    printf ("\n");
    kill (pid_receptor, SIGTERM);
    exit (0);
}
}

```

recibir: rutina de tratamiento de la señal SIGUSR1 para el proceso receptor. Este proceso se encarga de leer un mensaje del fichero buzón y escribirlo en el flujo estándar de salida. Acto seguido le envía la señal SIGUSR1 al proceso emisor.

****/

```

void recibir (int sig)
{
    char str [MAX];
    FILE *fp;

    if ((fp = fopen ("buzón", "r")) == NULL) {
        perror ("enviar");
        kill (pid_receptor, SIGTERM);
    }
    fgets (str, MAX, fp);
    fclose (fp);
    printf ("PROCESO RECEPTOR. MENSAJE: %s\n", str);
    signal (SIGUSR1, recibir);
    kill (pid_emisor, SIGUSR1);
}

```

main: se encarga de crear los procesos emisor y receptor y de iniciar la comunicación entre ambos.

****/

```

main (int argc, char *argv [])
{
    pid_emisor = getpid ();
    if ((pid_receptor = fork ()) == -1) {
        perror (argv [0]);
        exit (-1);
    } else if (pid_receptor == 0) {
        /* Código del proceso hijo. */
        signal (SIGUSR1, recibir);
    }
}

```

```
        while (!0)
            pause ();
    } else {
        /* Código del proceso padre. */
        /* Concedemos 2 segundos antes de iniciar la transmisión, así aseguramos
        que el proceso receptor está listo. */
        printf ("Inicializando...\n");
        sleep (2);
        enviar (SIGUSR1);
        while (!0)
            pause ();
    }
    exit (0);
}
```

7.7 Funciones de tiempo

En los párrafos siguientes vamos a estudiar cómo consultar y controlar los tiempos asociados a un proceso. Si bien UNIX no fue concebido como un sistema para aplicaciones en tiempo real, en condiciones de baja carga donde no está congestionado podremos imponerle a nuestros programas que se ejecuten con los tiempos de respuesta adecuados. Hay que tener presente que una respuesta en tiempo real no significa solamente que nuestro programa se ejecute muy rápido —para simular procesos físicos, por ejemplo—, sino también que seamos capaces de controlar el tiempo para dar la respuesta en el momento exacto y no antes. Imaginemos, por ejemplo, un programa para simular un reloj, si nuestra respuesta se tiene que dar cada segundo, de nada nos vale darla más deprisa.

La parte central de esta sección está dedicada a estudiar los mecanismos para dotar a nuestros programas de una temporización que estará impuesta por la naturaleza física del proceso que intentamos simular.

7.7.1 Fijación de la fecha del sistema (stime)

Si necesitamos fijar la fecha y hora locales de nuestro sistema, debemos usar la llamada `stime`:

```
#include <sys/time.h>
int stime (long *tp);
```

donde `tp` es un puntero a un número que contiene la hora actual medida en segundos con respecto a un origen de tiempos conocido como *Época*². La primera *Época* considerada fueron las 00:00:00 GMT —*Greenwich Mean Time*— del día 1 de enero de 1970. La hora GMT, basada en la duración media del año solar medida en el meridiano de Greenwich, ha sido considerada como referencia horaria desde 1884. Esta hora, corregida al tener

²**época.** (Del lat. *epōcha*, y este del gr. *ἐποχή*). f. Fecha de un suceso desde el cual se empiezan a contar los años. [RAE, 2001]

en cuenta el desplazamiento de los polos da lugar a la hora universal *UT1*. En 1967 la Décimo Tercera Conferencia General de Pesos y Medidas define la unidad de tiempo del Sistema Internacional de Unidades basándose en los relojes atómicos de cesio. Esto da lugar a la definición de la hora *TAI* —*Temps Atomique International*— que es calculada por el *BIPM* —*Bureau International des Poids et Mesures*— a partir de las lecturas realizadas en más de 200 relojes atómicos distribuidos en unos 30 países. La hora TAI está sincronizada con las 00:00:00 del día 1 de enero de 1958, desfasándose a partir de esa fecha debido a la duración irregular del año solar. Con objeto de definir una referencia horaria que armonice la hora TAI y la hora solar y que pueda servir como referencia oficial de la hora, la *Recomendación 460-4 (1986)* del *CCIR* define la hora *UTC* —*Tiempo Universal Coordinado*— para que se cumpla la desigualdad

$$|UT1 - UTC| < 0,9s$$

así, la hora UTC es la hora TAI a la que ocasionalmente se le añade algún segundo —preferiblemente al finalizar los meses de diciembre o junio— para poder cumplir la desigualdad anterior. El responsable de decidir cuándo deben añadirse estos segundos de ajuste es el *IERS* —*International Earth Rotation Service*— que publica semestralmente una nota conocida como *Boletín C* indicando la fecha y hora del próximo ajuste, así como la diferencia actual entre la hora oficial UTC y la hora TAI. El Boletín C junto el resto de información referente a la determinación de la hora oficial la podemos encontrar a través de la página oficial de la Oficina Internacional de Pesos y Medidas www.bipm.fr.

La definición de la *Época* se realiza actualmente con respecto a la hora UTC pero eso poco nos afecta porque al escribir programas que controlen tiempos de ejecución no vamos a trabajar con fechas absolutas sino con intervalos de tiempo y éstos son independientes de la *Época*. Por otro lado, cuando necesitemos conocer fechas absolutas, podemos usar funciones estándar como `ctime` para presentar la fecha en el formato estándar `hh:mm:ss dd:mm:aa` sin tener que preocuparnos por el origen de tiempos.

La fecha del sistema sólo la puede fijar un usuario con los privilegios del superusuario; por lo tanto, `stime` sólo se podrá ejecutar satisfactoriamente en aquellos procesos cuyo *EUID* —identificador de usuario efectivo— sea el de superusuario.

`stime` devuelve el valor 0 si se ejecuta satisfactoriamente y -1 en caso de error.

7.7.2 Lectura de la fecha del sistema (`time` y `gettimeofday`)

Si queremos leer la fecha actual que almacena el sistema, podemos usar la llamada `time`:

```
#include <time.h>
time_t time (time_t *tloc);
```

que devuelve el valor de la hora en segundos con respecto a la *Época*. Si `tloc` no es un puntero nulo, el valor que devuelve `time`, también se copia en la dirección apuntada por `tloc`.

Si la llamada a `time` falla, la función devolverá el valor `(time_t)(-1)` y en `errno` estará el código del tipo de error producido.

Los saltos de tiempo que se producen a intervalos irregulares por necesidades de ajuste del calendario —ya mencionados en el apartado anterior— no quedan reflejados en la hora

del sistema. Por citar un ejemplo, en el intervalo que va desde 1961 a 1999 se ha producido una variación de 32 segundos.

Si la resolución que ofrece `time` —segundos— no es suficiente y necesitamos realizar una medida más exacta, podemos usar la llamada `gettimeofday`:

```
#include <sys/time.h>
int gettimeofday (struct timeval *tp, struct timezone *tzp);
```

Esta función devuelve, en la estructura apuntada por `tp`, la hora UTC del sistema y, en la estructura apuntada por `tzp`, las correcciones con respecto a la hora UTC para poder calcular la hora local. Esta hora estará en función de la zona de la Tierra donde nos encontremos.

La estructura `timeval` está definida en `<sys/time.h>` y se declara de la forma siguiente:

```
struct timeval {
    // Segundos transcurridos desde la Época
    unsigned long tv_sec;
    // Microsegundos. Su rango está comprendido entre 0 y 999.999
    long tv_usec;
};
```

La estructura `timezone` también está definida en `<sys/time.h>` y sus campos son:

```
struct timezone {
    // Variación de la hora local, en minutos, con respecto a la hora UTC
    int tz_minuteswest;
    // Tipo de corrección a aplicar según las estaciones (horario de invierno o
    // de verano)
    int tz_dsttime;
};
```

En la mayor parte de los programas no se utiliza el valor devuelto por `tzp`. En el caso de que se haya definido la variable de entorno `TZ` no se debe emplear la corrección de hora indicada en `tzp`.

También se puede cambiar la hora del sistema mediante una llamada que maneja estructuras del tipo `timeval` y `timezone`. Esta llamada es `settimeofday`:

```
#include <sys/time.h>
int settimeofday (struct timeval *tp, struct timezone *tzp);
```

Para usar `settimeofday` hay que tener privilegios de superusuario.

Tanto `gettimeofday` como `settimeofday` devuelven el valor 0 si se han ejecutado satisfactoriamente y -1 en caso de error.

Hay que decir que la resolución del campo que mide los microsegundos —`tv_usec`— depende de la frecuencia del reloj del sistema, que suele ser bastante pequeña en comparación con la del reloj de la máquina.

Como ejemplo de uso de la estructura `timeval`, presentaré un programa para calcular el tiempo empleado en ejecutar una serie de operaciones. En concreto, vamos a

medir el tiempo empleado en calcular 1_000.000 de raíces cuadradas de números positivos aleatorios.

Programa 7.15: Medida del tiempo de ejecución de un programa (timeval.c)

```

/****
$ timeval

Forma de compilación:
$ cc -o timeval timeval.c -lm
****/
#include <stdio.h>
#include <sys/time.h>
#include <math.h>

main ()
{
    struct timeval t_inicio, t_fin, t_dif;
    struct timezone tz;
    unsigned long i;
    double x;

    gettimeofday (&t_inicio, &tz);

    /* Proceso de cálculo cuyo tiempo de ejecución queremos medir. */
    for (i = 0; i < 1000000; i++)
        x = sqrt ((double) rand ());

    gettimeofday (&t_fin, &tz);

    /* Cálculo del tiempo transcurrido. */
    if (t_inicio.tv_usec > t_fin.tv_usec) {
        t_fin.tv_usec += 1000000;
        --t_fin.tv_sec;
    }
    t_dif.tv_sec = t_fin.tv_sec - t_inicio.tv_sec;
    t_dif.tv_usec = t_fin.tv_usec - t_inicio.tv_usec;

    /* Presentación de resultados. */
    printf ("Tiempo de cálculo = %d s., %d us.\n",
        t_dif.tv_sec, t_dif.tv_usec);
}

```

El tiempo que presenta este programa depende de la carga del sistema. Si el programa no tiene otros procesos compitiendo por la CPU, tardará menos en ejecutarse que si tiene otros competidores. En el próximo apartado veremos cómo leer el tiempo real de CPU que el proceso emplea en su ejecución.

7.7.3 Tiempos de ejecución asociados a un proceso (times)

Para conocer el tiempo real empleado por un proceso en su ejecución, podemos usar la llamada `times`:

```
#include <sys/times.h>
clock_t times (struct tms *buffer);
```

Esta llamada rellena la estructura apuntada por `buffer` con la información estadística relativa a los tiempos de ejecución empleados por el proceso, desde su inicio hasta el momento de invocar a `times`. La estructura `tms` está definida en el fichero `<sys/times.h>`:

```
struct tms {
    // Tiempo de CPU en modo usuario
    clock_t tms_utime;
    // Tiempo de CPU en modo supervisor
    clock_t tms_stime;
    // Tiempo de CPU, en modo usuario, de los procesos hijo
    clock_t tms_cutime;
    // Tiempo de CPU, en modo supervisor, de los procesos hijo
    clock_t tms_cstime;
};
```

El tipo `clock_t` se define para contabilizar los pulsos de reloj. Cada segundo se compone de un total de `CLK_TCK` pulsos. El valor de `CLK_TCK` depende de la implementación del sistema y se puede leer mediante la llamada `sysconf`. Para calcular el tiempo en segundos que almacena una variable del tipo `clock_t`, tenemos que dividirla por `CLK_TCK`.

Los valores que aparecen en los campos de `buffer` se refieren al proceso que ejecuta la llamada a `times` y a todos aquellos procesos hijo para los cuales el proceso padre ejecuta una llamada a `wait`. El significado estos campos es el siguiente:

- `tms_utime` es el tiempo empleado por la CPU ejecutando el proceso en modo usuario.
- `tms_stime` es tiempo empleado por la CPU ejecutando el proceso en modo supervisor.
- `tms_cutime` es la suma de los campos `tms_utime` y `tms_cutime` para los procesos hijo. Es decir, tiempo empleado por la CPU ejecutando los procesos hijo, los hijos de los hijos, etc. en modo usuario.
- `tms_cstime` es la suma de los campos `tms_stime` y `tms_cstime` para los procesos hijo. Es decir, tiempo empleado por la CPU ejecutando los procesos hijo, los hijos de los hijos, etc. en modo supervisor.

En el cómputo de todos los tiempos no se tiene en cuenta el tiempo dedicado por los procesos del sistema a los procesos del usuario —por ejemplo, tiempo que emplea el sistema en hacer que un proceso de usuario cambie de contexto—. Los tiempos anteriores son tiempos reales de CPU, por lo que no se contabilizan los períodos en los que el proceso duerme.

Si `times` se ejecuta satisfactoriamente, devuelve el tiempo real transcurrido, en pulsos de reloj, contado a partir de un instante pasado arbitrario. Este instante puede ser el momento de arranque del sistema y no cambia de una llamada a otra. Si `times` falla, devolverá `-1` y en `errno` estará el código del error producido.

Como ejemplo de aplicación, vamos a implementar la orden `tiempo` —equivalente a la orden estándar `time`— que nos permite ver el tiempo empleado por un proceso en su ejecución. La forma de invocar a `time` es:

```
$ time línea_de_órdenes
```

donde `línea_de_órdenes` es cualquier otra orden o programa ejecutable. Un ejemplo de ejecución puede ser:

```
$ time ps -ef > /dev/nul
```

```
real 0m3.28s
user 0m0.08s
sys 0m0.54s
```

Programa 7.16: Tiempos de ejecución asociados a un proceso. (`tiempo.c`)

```
/**
 * tiempo línea_de_órdenes
 */
#include <stdio.h>
#include <fcntl.h>
#include <time.h>
#include <sys/times.h>
#include <signal.h>

main (int argc, char *argv[])
{
    struct tms bt1, bt2;
    clock_t t1, t2;
    int pid, w, estado, tty;

    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc < 2) {
        fprintf(stderr, "Forma de uso: %s línea_de_órdenes\n", argv [0]);
        exit (-1);
    }

    /* Apertura del terminal asociado al proceso. */
    if ((tty = open ("/dev/tty", O_RDWR)) == -1) {
        perror (argv [0]);
        exit (-1);
    }

    t1 = times (&bt1);
```

```
/* Creación de los procesos padre e hijo. */
if ((pid = fork ()) == -1) {
    perror (argv [0]);
    exit (-1);
} else if (pid == 0) {
    /* Código del proceso hijo. */
    /* Reasignamos los flujos estándar de entrada, salida y salida de errores del
    proceso hijo para independizarlos de los del proceso padre. */
    close (0); dup (tty);
    close (1); dup (tty);
    close (2); dup (tty);
    /* Aquí se ejecuta la línea de órdenes que recibe tiempo. */
    execvp (argv [1], &argv[1]);

    /* Caso de que falle la llamada a execvp. */
    exit (127);
} else {
    /* Código del proceso hijo. */
    close (tty);
    /* Deshabilitamos el tratamiento de las señales SIGINT y SIGQUIT, en el
    proceso padre. Esto es necesario para evitar que al pulsar las teclas Ctrl+C,
    que deben afectar solamente al hijo, el padre también se vea involucrado. */
    signal (SIGINT, SIG_IGN);
    signal (SIGQUIT, SIG_IGN);

    /* Esperamos a que termine el hijo. */
    while ((w = wait (&estado)) != pid && w != -1);
    t2 = times (&bt2);

    /* Habilitamos el tratamiento de las señales SIGINT y SIGQUIT, en
    el proceso padre. */
    signal (SIGINT, SIG_IGN);
    signal (SIGQUIT, SIG_IGN);

    /* Presentación de resultados. */
    printf ("Tiempos de ejecución:\n");
    printf ("\tReal = %g s.\n", (float)(t2-t1) / CLK_TCK);
    printf ("\tEn modo usuario = %g s.\n",
            (float)(bt2.tms_cutime - bt1.tms_cutime) / CLK_TCK);
    printf ("\tEn modo supervisor = %g s.\n",
            (float)(bt2.tms_cstime - bt1.tms_cstime) / CLK_TCK);

    exit (0);
}
}
```

7.7.4 Temporizadores

En determinadas aplicaciones puede interesarnos que el código se ejecute de acuerdo con una temporización determinada. Esto puede venir impuesto por las especificaciones del programa, donde una respuesta antes de tiempo puede ser tan perjudicial como una respuesta con retraso.

La primera llamada al sistema que nos ayuda a resolver las necesidades de temporización es `alarm`:

```
#include <unistd.h>
unsigned alarm (unsigned sec);
```

Esta llamada activa un temporizador que inicialmente toma el valor `sec` segundos y que se decrementará en tiempo real. Cuando hayan transcurrido los `sec` segundos, el proceso recibirá la señal `SIGALRM`. El valor de `sec` está limitado por la constante `MAX_ALARM`, definida en `<sys/param.h>`. En todas las implementaciones se garantiza que `MAX_ALARM` debe permitir una temporización de al menos 31 días. Si `sec` toma un valor superior al de `MAX_ALARM`, se trunca al de esta constante.

Llamadas sucesivas a `alarm` restauran el valor del temporizador al de `sec`. Así, para cancelar un temporizador previamente declarado, haremos la llamada `alarm(0)`.

Es importante tener presente que una alarma no es heredada por un proceso hijo después de una llamada a `fork`, pero sí es heredada por los programas cargados con `exec`.

La llamada `alarm` devuelve el tiempo que le quedaba al temporizador que había sido definido con anterioridad.

Como ejemplo de aplicación, vamos a implementar la función `dormir` —equivalente a la función estándar `sleep`— que permite parar la ejecución de un proceso durante una cantidad de segundos determinada. La declaración de `dormir` es:

```
void dormir (unsigned seg);
```

donde `seg` es la cantidad de segundos que el proceso estará parado.

Programa 7.17: Implementación de la función `sleep` (`dormir.c`)

```
#include <stdio.h>
#include <signal.h>
#include <setjmp.h>
#include <unistd.h>
static jmp_buf entorno;

/****
    manejador_SIGALRM
****/
static void manejador_SIGALRM (int sig)
{
    longjmp (entorno, 1);
}
```

```

/****
    dormir: versión de sleep.
    Parámetros: seg — segundos durante los que permanecerá parado el proceso.
****/
dormir (unsigned seg)
{
    unsigned alarma_ant;
    void (*manejador_SIGALRM_ant) ();

    if (seg == 0)
        return;
    manejador_SIGALRM_ant = signal (SIGALRM, manejador_SIGALRM);
    if (setjmp (entorno) == 0) {
        alarma_ant = alarm (seg); /* Activamos el temporizador. */
        pause (); /* Nos ponemos a esperar alguna señal. */
        /* Nos valemos de setjmp para evitar los problemas que podría dar invocar
        a dormir con un valor muy pequeño. Imaginemos que en el intervalo que
        transcurre desde que salimos de la llamada a alarm y entramos en la llamada
        a pause se recibe la señal. Primero se ejecutaría la rutina de tratamiento y
        luego la llamada a pause, de modo que la función quedaría parada por toda
        la eternidad. */
    } else {
        /* Cuando se reciba SIGALRM y se ejecute manejador_SIGALRM, el programa
        continuará por esta parte. */
        /* Restauramos las cosas como estaban para no interferir con otro
        temporizador que tuviese activo el programa. */
        signal (SIGALRM, manejador_SIGALRM_ant);
        alarm (alarma_ant);
    }
}

main ()
{
    int i;

    for (i = 0; i < 10; i++) {
        printf ("Durmiendo %d segundos.\n", i);
        dormir (i);
    }
}

```

La llamada `alarm` maneja una resolución de segundos, pero podemos necesitar temporizadores que controlen tiempos más pequeños. Para dar respuesta a estas necesidades, el UNIX de Berkeley define dos llamadas —una para declarar temporizadores y otra para leer su estado— que manejan resoluciones de tiempo mayores. Estos temporizadores responden al siguiente esquema de funcionamiento:

1. Se debe definir el temporizador. Dentro de la definición, entra declarar cuál será su valor inicial —en segundos y microsegundos— y cuál será su valor de recarga cuando llegue a 0.
2. Cuando el temporizador llega a cero, el proceso que lo declaró recibe alguna de las señales `SIGALRM`, `SIGVTALRM` o `SIGPROF`.
3. El contador se reinicializa con un valor predefinido y continúa con su cuenta regresiva, repitiéndose así todo el proceso.

Las funciones de BSD son `getitimer` y `setitimer`:

```
#include <sys/time.h>
getitimer (int wich, struct itimerval *value);
setitimer (int wich, struct itimerval *value, struct itimerval *ovalue);
```

El sistema permite que cada proceso tenga tres temporizadores, que se definen en `<sys/time.h>`. La llamada `getitimer` devuelve el valor actual del temporizador especificado en `wich`; `setitimer` declara el nuevo valor de un temporizador —el especificado en `wich`, y opcionalmente devuelve el valor anterior—; el valor del temporizador se especifica mediante una estructura del tipo `itimerval`, que se declara de la forma siguiente:

```
struct itimerval {
    struct timeval it_interval; // Intervalo del temporizador
    struct timeval it_value;   // Valor actual del temporizador
};
```

Si `it_value` no vale cero, indica el tiempo que falta para que el temporizador llegue a cero. Si `it_interval` no vale cero, indica el valor con el que se recargará el temporizador cuando llegue a cero. Poniendo `it_value` a cero, desactivamos el temporizador. Poniendo `it_interval` a cero, hacemos que el temporizador quede desactivado cuando llegue a cero.

El parámetro `wich` especifica el temporizador que vamos a usar, sus posibles valores son:

- `ITIMER_REAL`, el temporizador se decrementa en tiempo real. Cuando llega a cero, el proceso recibe la señal `SIGALRM`. Este temporizador también es modificado por `alarm`, por lo que el uso de esta llamada después de `setitimer` cambiará el valor del temporizador.
- `ITIMER_VIRTUAL`, el temporizador se decrementa sólo cuando el proceso se está ejecutando en modo usuario —tiempo virtual—. Mientras el proceso duerme, el temporizador está parado. Cuando el temporizador llega a cero, el proceso recibe la señal `SIGVTALRM`.
- `ITIMER_PROF`, el temporizador se decrementa cuando el proceso se está ejecutando en modo usuario y cuando se está ejecutando en modo supervisor. Cuando el temporizador llega a cero, el proceso recibe la señal `SIGPROF`. Puesto que esta señal puede llegar mientras se está ejecutando una llamada al sistema —recordemos que las llamadas al sistema se ejecutan en modo supervisor—, los procesos que la reciban deben estar programados para reiniciar la ejecución de las llamadas interrumpidas.

La resolución con la que trabajan estos temporizadores depende de nuestra máquina. La constante HZ, definida en `<sys/param.h>`, indica la frecuencia, en hercios, más alta que puede manejar un temporizador. Así, su resolución es de $\frac{1}{\text{HZ}}$ segundos. Los valores máximos de intervalo que pueden manejar los tres temporizadores son `MAX_ALARM`, `MAX_VTALARM` y `MAX_PROF`, respectivamente. En todas las implementaciones se garantiza un mínimo de 31 días para estos valores.

Tanto `getitimer` como `setitimer` devuelven el valor 0 si se ejecutan satisfactoriamente y -1 en caso contrario.

La siguiente secuencia de código declara un temporizador en tiempo real que tardará 10,5 segundos en expirar la primera vez, y después lo hará periódicamente cada 1,54 segundos:

```
struct itimerval temporizador, temporizador_ant;
...
temporizador.it_value.tv_sec = 10;
temporizador.it_value.tv_usec = 500000;
temporizador.it_interval.tv_sec = 1;
temporizador.it_interval.tv_usec = 540000;
...
setitimer (ITIMER_REAL, &temporizador, &temporizador_ant);
```

Como ejemplo de aplicación, vamos a codificar una versión reducida de la orden `at` que permite lanzar trabajos en segundo plano para que se ejecuten a una hora determinada. Nuestro programa se llama `a-las`, y la forma de invocarlo es:

```
$ a-las hh:mm:ss línea_de_órdenes
```

por ejemplo:

```
$ a-las 13:30:00 echo HORA DE COMER
```

Esta línea hace que a las 13:30:00 horas aparezca por pantalla el mensaje `HORA DE COMER`.

El programa `a-las`, tras crear un proceso hijo, le devuelve el control al sistema. El proceso hijo es el encargado de activar un temporizador en tiempo real que llegará a cero cuando sea la hora deseada.

Programa 7.18: Ejemplo de implementación de la orden `at` (`a-las.c`)

```
#include <stdio.h>
#include <sys/time.h>
#include <signal.h>

#define SEGUNDOS_POR_DIA 86400

/****
    hhmss_a_ss: macro que transforma horas, minutos y segundos a segundos.
****/
#define hhmss_a_ss(_hh, _mm, _ss) ((long)(_hh)*3600 + (_mm)*60 + (_ss))

/****
```

```

    ss_a_hhmmss: macro que transforma segundos a horas, minutos y segundos.
***/
#define ss_a_hhmmss(_seg, _hh, _mm, _ss) {\
    _hh = (int)((_seg)/3600);\
    _mm = (int)((((_seg) - (long)_hh*3600) / 60);\
    _ss = (int)((_seg) - (long)_hh*3600 - _mm*60);\
}

/****
    manejador_SIGALRM: rutina de tratamiento de la señal SIGALRM. Su cuerpo es nulo
    y no hace nada.
***/
void manejador_SIGALRM () {}

/****
    main: se encarga de analizar los argumentos de la línea de órdenes. Si son
    correctos, crea un proceso hijo y muere. El proceso hijo calcula el valor, en
    segundos, al que debe inicializar un temporizador en tiempo real, lo activa y se
    pone a esperar. Cuando recibe la señal SIGALRM, la rutina de tratamiento no hace
    nada, y al volver el proceso a su contexto hace una llamada a execvp para ejecutar
    la línea de órdenes.
***/
main (int argc, char *argv [])
{
    int hh, mm, ss, pid;

    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc < 3) {
        fprintf (stderr, "Forma de uso: %s hh:mm:ss línea_de_órdenes\n",
            argv [0]);
        exit (-1);
    }
    if (sscanf (argv[1], "%d:%d:%d", &hh, &mm, &ss) != 3) {
        fprintf (stderr, "Formato de hora erróneo: %s\n", argv[1]);
        exit (-1);
    } else if (hh<0 || hh>23 || mm<0 || mm>59 || ss<0 || ss>59) {
        fprintf (stderr, "Hora fuera de rango: %s\n", argv[1]);
        exit (-1);
    }

    /* Creación del proceso hijo que se ejecutará en segundo plano. */
    if ((pid = fork ()) == -1) {
        perror (argv [0]);
        exit (-1);
    } else if (pid == 0) { /* Código del proceso hijo. */
        long total_segundos;

```

```

time_t t;
struct tm *tm;
struct itimerval temporizador, temporizador_ant;

temporizador.it_value.tv_usec = 0;
temporizador.it_interval.tv_sec = 0;
temporizador.it_interval.tv_usec = 0;
signal (SIGALRM, manejador_SIGALRM );

/* Lectura de la fecha. */
time (&t);
tm = localtime (&t);

/* Cálculo del valor del temporizador. */
total_segundos = hhhmmss_a_ss (hh, mm, ss) -
    hhhmmss_a_ss (tm->tm_hour, tm->tm_min, tm->tm_sec);
if (total_segundos < 0)
    total_segundos += SEGUNDOS_POR_DIA;
temporizador.it_value.tv_sec = total_segundos;
setitimer (ITIMER_REAL, &temporizador, &temporizador_ant);
pause ();

/* Cuando expira el temporizador, ejecutamos la línea de órdenes. */
execvp (argv [2], &argv [2]);
} else /* Código del proceso padre. */
    exit (0);
}

```

En este programa hemos empleado la función estándar `localtime` que posibilita la manipulación de la fecha en diferentes formatos:

```

#include <time.h>
struct tm *localtime (const time_t *timer);

```

La función recibe un puntero a una fecha con formato `time_t` y devuelve un puntero a una estructura del tipo `tm`. Esta estructura se compone de los siguientes campos:

```

struct tm {
    int tm_sec; // Segundos que sobrepasan el minuto – [0, 59]
    int tm_min; // Minutos que sobrepasan la hora – [0, 59]
    int tm_hour; // Hora del día – [0, 23]
    int tm_mday; // Día del mes – [1, 31]
    int tm_mon; // Mes del año – [0, 11]
    int tm_year; // Año, contado a partir de 1900
    int tm_wday; // Día de la semana, contado a partir del domingo – [0, 6]
    int tm_yday; // Día del año – [0, 365]
    int tm_isdst; // Indicador de corrección de la fecha
};

```

Como vemos en el programa, sólo utilizamos los campos de la hora, los minutos y los segundos, y en el caso de que la hora indicada ya haya sido sobrepasada, la alarma se activa para la misma hora del día siguiente.

7.8 Relojes, retardos y temporizadores en POSIX

La norma POSIX define un *reloj* como «un objeto software o hardware que puede usarse para medir el transcurso real o aparente del tiempo» [IEEE, 2001a]. Las funciones para definir y consultar relojes son:

```
#include <time.h>
int clock_settime (clockid_t clock_id, const struct timespec *tp);
int clock_gettime (clockid_t clock_id, struct timespec *tp);
int clock_getres (clockid_t clock_id, struct timespec *res);
```

La función `clock_settime` se utiliza para poner en hora el reloj `clock_id` de acuerdo con la hora referenciada por `tp` y la función `clock_gettime` para leerla. El tipo `clockid_t` se define en el fichero de cabecera `<time.h>` y la norma establece como obligatorio el reloj de tiempo real `CLOCK_REALTIME` que se incrementa a medida que transcurre el tiempo —tanto si el proceso se está ejecutando como si está parado—; y como opcional el reloj monótono creciente `CLOCK_MONOTONIC` que se inicializa al arrancar el sistema y nunca retrocede —por ello no puede ser modificado con una llamada a `clock_settime`—. Algunos sistemas como FreeBSD implementan también los relojes `CLOCK_VIRTUAL`, que se incrementa sólo cuando el proceso se está ejecutando en modo usuario, y `CLOCK_PROF`, que se incrementa cuando el proceso se está ejecutando en modo usuario o en modo supervisor.

El tipo `struct timespec` se define también en el fichero de cabecera `<time.h>`, consta de los campos `tv_sec` —segundos— y `tv_nsec` —nanosegundos—; e indica la hora con respecto a un origen de tiempos que puede ser la *Época*.

La función `clock_getres` se utiliza para conocer la resolución del reloj `clock_id`, que es devuelta a través de la referencia `res`.

Las tres funciones devuelven 0 si se ejecutan satisfactoriamente y -1 en caso de error, escribiendo en `errno` el código del error producido.

7.8.1 Retardos

El reloj de tiempo real es utilizado para implementar retardos relativos o absolutos con las funciones `nanosleep` y `clock_nanosleep`:

```
#include <time.h>
int nanosleep (const struct timespec *rqtp, struct timespec *rmtp);
int clock_nanosleep (clockid_t clock_id, int flags,
                    const struct timespec *rqtp, struct timespec *rmtp);
```

La función `nanosleep` detiene el hilo actual hasta que haya transcurrido al menos el tiempo referenciado por `rqtp` o hasta que se reciba una señal. En este caso, la función devuelve el valor -1, `errno` valdrá `EINTR` y en la referencia `rmtp` se almacena el tiempo remanente —aquel que falta hasta que transcurra el plazo indicado en `rqtp`—. El plazo de

suspensión del hilo puede ser mayor que el solicitado si éste no es múltiplo de la resolución, ya que se redondea al alza.

El tipo de retardo que introduce la función `nanosleep` se denomina *relativo* porque el tiempo de espera está referido al instante en el que es invocada la función. En sistemas de tiempo real esto puede introducir imprecisiones en tareas que necesitan activarse en un instante concreto. Consideremos, como ejemplo, una tarea que tiene que despertar a una hora fija para atender a un dispositivo. La forma de hacer que la tarea duerma hasta ese instante puede ser:

1. Leer la hora actual con `clock_gettime`.
2. Calcular el retardo.
3. Llamar a `nanosleep` y pasarle el retardo calculado.
4. Atender al dispositivo cuando `nanosleep` devuelve el control.

Puede ocurrir que tras la ejecución del punto 2 el distribuidor le conceda la CPU a otra tarea y por lo tanto, al recuperar la CPU y reanudar la ejecución invoquemos a `nanosleep` con un retardo incorrecto. Por este motivo la función `clock_nanosleep` ofrece también la posibilidad de realizar retardos *absolutos* —aquellos que detienen el hilo hasta que llega un instante de tiempo concreto—. Si `flags` vale `TIMER_ABSTIME`, `clock_nanosleep` detiene el hilo que la invoca hasta que el reloj `clock_id` alcanza la hora referenciada en `rqtp` o recibe una señal, en caso contrario se comporta como la función `nanosleep`.

7.8.2 Temporizadores

Mientras los retardos introducen una *temporización síncrona* en nuestra aplicación —es decir, los hilos esperan a que el retardo concluya—, los temporizadores permiten que el hilo continúe su ejecución concurrentemente avisándole cuando han expirado —*temporización asíncrona*—.

POSIX define un *temporizador* como «*Un mecanismo para notificarle a un hilo que un reloj ha alcanzado una hora concreta o que ha transcurrido una determinada cantidad de tiempo*», y para manipular temporizadores se emplean las funciones:

```
#include <signal.h>
#include <time.h>
int timer_create (clockid_t clockid, struct sigevent *evp, timer_t *timerid);
int timer_settime (timer_t timerid, int flags,
                  const struct itimerspec *value, struct itimerspec *ovalue);
int timer_delete (timer_t timerid);
int timer_gettime (timer_t timerid, struct itimerspec *value);
```

Con la función `timer_create` creamos un nuevo temporizador asociado al proceso actual y que medirá el tiempo con el reloj `clockid`. El temporizador creado puede ser referenciado a través de la dirección devuelta por `timerid`. El argumento `evp` indica la forma de comunicarle al proceso que el temporizador ha expirado. Este argumento referencia una estructura de *notificación asíncrona* de eventos de tipo `struct sigevent` definida en `<signal.h>`:

```

struct sigevent {
    // Tipo de notificación
    int sigev_notify;
    // Número de la señal con la que se notifica
    int sigev_signo;
    // Datos asociados a la señal
    union sigval sigev_value;
    // Función que ejecuta el hilo creado para atender la notificación
    void (*sigev_notify_function) (union sigval);
    // Puntero a los atributos con los que se crea el hilo
    pthread_attr_t *sigev_notify_attributes;
};

```

Los tipos de notificación que define la norma son:

- SIGEV_NONE** No se realiza ninguna notificación cuando el evento se produce.
- SIGEV_SIGNAL** La notificación se realiza mediante el envío de la señal `sigev_signo` asociándole los datos `sigev_value`.
- SIGEV_THREAD** La notificación se realiza creando un hilo con los atributos referenciados por `sigev_notify_attributes` y que ejecuta el código de la función apuntada por `sigev_notify_function`.

Si el argumento `evp` vale `NULL`, el temporizador es creado con la estructura de notificación por defecto que tiene los valores: `sigev_notify=SIGEV_SIGNAL`, `sigev_signal=nº` de señal seleccionado por el sistema y `sigev_value.sigval_int=timerid`.

Los temporizadores no son heredados por los procesos hijo tras la llamada a `fork` y son borrados tras la llamada a `exec`.

Los temporizadores recién creados están parados —*desarmados*— y para ponerlos en marcha —*armarlos*— se utiliza la función `timer_settime` donde `timerid` identifica un temporizador creado con una llamada previa a `timer_create`, `value` referencia los nuevos valores de programación del temporizador y `ovalue` los valores con los que estaba programado. El tipo `struct itimerspec` se define en el fichero `<time.h>` con los campos:

```

struct itimerspec {
    struct timespec it_interval; // Periodo del temporizador
    struct timespec it_value; // Instante de la primera notificación
}

```

La definición de la estructura `struct timespec` la hemos visto con anterioridad en el § 7.5.4 pág. 276.

El campo `it_value` indica el instante en que el temporizador realiza la primera notificación y se puede interpretar como un retardo relativo o una hora absoluta dependiendo del valor que tome el argumento `flags` de la llamada:

- Si `flags=TIMER_ABSTIME`, `it_value` indica la hora concreta a la que se produce la primera notificación.
- Si `flags≠TIMER_ABSTIME`, `it_value` indica el tiempo que debe transcurrir hasta que se produzca la primera notificación.

Una vez que se efectúa la primera notificación, el temporizador se recarga con el valor del campo `it_interval` para realizar notificaciones periódicas hasta que decidamos pararlo. Esto se consigue llamando a `timer_settime` con un valor para `value` donde el campo `it_value`={0,0}. Las líneas de código siguientes muestran cómo crear un temporizador que mide el tiempo con el reloj `CLOCK_REALTIME`, envía la primera notificación transcurridos 10,5 segundos, las sucesivas cada 5 segundos y que crea un hilo para ejecutar la función `hilo_del_temporizador` con cada notificación:

```
void hilo_del_temporizador (void *arg) {...}
...
timer_t id_temporizador;
struct itimerspec et; // Especificación del temporizador
struct sigevent ev; // Evento asociado a la notificación
int error;
...
// Parámetros del evento
ev.sigev_notify = SIGEV_THREAD;
ev.sigev_value.sival_ptr = &id_temporizador;
ev.sigev_notify_function = hilo_del_temporizador;
ev.sigev_notify_attributes = NULL;
...
// Parámetros del temporizador
et.it_value.tv_sec = 10;
et.it_value.tv_nsec = 500000000; //  $5 \times 10^8$  ns = 0,5 s
ev.sigev_value.sigval_ptr = &id_temporizador;
...
// Creación del temporizador
error = timer_create (CLOCK_REALTIME, &ev, &id_temporizador);
if (error)
    error_fatal (error, "timer_create");
...
// Activación del temporizador
error = timer_settime (timer_id, ~TIMER_ABS, &et, NULL);
if (error)
    error_fatal (error, "timer_settime");
```

Para consultar el estado del temporizador `timerid` se utiliza la función `timer_gettime` que devuelve, en la dirección referenciada por `value`, el tiempo que resta hasta que el temporizador envíe la primera notificación —campo `it_value`— y su período —campo `it_interval`—.

En el momento en que una aplicación no necesita un temporizador, puede borrarlo y liberar los recursos que ocupa con la función `timer_delete`.

Las cuatro funciones anteriores devuelven 0 si se ejecutan satisfactoriamente y -1 en caso de error, escribiendo en `errno` el código del error producido.

Como hemos visto al explicar la función `timer_create`, el mecanismo de notificación que emplean los temporizadores puede ser el envío de una señal de tiempo real con unos datos asociados. Si la frecuencia de notificación es mayor que la frecuencia de captura de

esas señales, para evitar que se produzca el rebose de la cola donde son almacenadas, una misma señal no es enviada varias veces mientras esté pendiente de ser manejada, sino que se incrementa el contador de reenvíos asociado al temporizador. Este contador puede ser consultado con la función `timer_getoverrun` que devuelve el número de notificaciones no atendidas del temporizador `timerid`. Este número está limitado por la constante `DELAY_TIMER_MAX`.

```
#include <time.h>
int timer_getoverrun (timer_t timerid);
```

Si la función falla, devuelve `-1` y en `errno` indica la causa del fallo.

7.9 Ejercicios

7.1 Escriba un programa semejante a la orden `kill` para enviarle señales a un proceso desde la línea de órdenes. La forma de invocarlo será:

`$ enviar señal pid`

7.2 Escriba un programa de nombre `manejadores` que arme un manejador por cada una de las señales que se muestra en la tabla 7.5.

Tabla 7.5: Manejador asociado a cada señal.

Señal	Nombre del manejador
SIGINT	manejador_SIGINT
SIGILL	manejador_SIGILL
SIGTERM	manejador_SIGTERM
SIGUSR1	manejador_SIGUSR
SIGUSR2	manejador_SIGUSR

El programa, después de armar los manejadores, debe entrar en un bucle de espera de señales. Cuando reciba alguna de las señales anteriores, debe emitir un mensaje con la forma que se muestra en la tabla 7.6.

Tabla 7.6: Mensaje asociado a cada señal.

Señal	Mensaje
SIGINT	Proceso #: señal SIGINT recibida
SIGILL	Proceso #: señal SIGILL recibida
SIGTERM	Proceso #: señal SIGTERM recibida
SIGUSR1	Proceso #: señal SIGUSR1 recibida
SIGUSR2	Proceso #: señal SIGUSR2 recibida

Hay que observar que el manejador de las señales SIGUSR1 y SIGUSR2 es el mismo —`manejador_SIGUSR`—, sin embargo, los mensajes son distintos. Por lo tanto, la rutina de tratamiento debe diferenciar la señal recibida a través del primer parámetro que recibe.

El programa se puede probar con órdenes como las siguientes:

```
$ manejadores & produce la salida [1] 325
```

```
$ kill -16 325 produce la salida Proceso 325: señal SIGUSR1 recibida
```

```
$ kill -15 325 produce la salida Proceso 325: señal SIGTERM recibida
```

- 7.3** Escriba un programa de nombre `existe` que reciba como argumento el *PID* de un proceso y que le devuelva al intérprete de órdenes el valor 0 si el proceso existe, o el valor 1 si el proceso no existe.

Estos valores podrán ser consultados a través de la variable de entorno `?_como` se muestra a continuación:

```
$ sleep 1200 & produce la salida [1] 492
```

```
$ existe 492
```

```
$ echo $? produce la salida 0
```

```
$ existe 123456654
```

```
$ echo $? produce la salida 1
```

El programa `existe` se ha diseñado con esta funcionalidad para poder utilizarlo en instrucciones condicionales que formen parte de baterías de órdenes para el intérprete `sh`, por ejemplo:

```
###
#
# Bateria de órdenes escrita para probar el programa 'existe'
#
###
for PID in 12 45 148 155
do
    if (existe $PID)
    then
        echo El proceso $PID existe
    else
        echo El proceso $PID no existe
    fi
done
```

Al ejecutar la batería, y dependiendo de los procesos que haya en el sistema, se puede producir un resultado como el siguiente:

\$ batería

El proceso 12 no existe
 El proceso 45 no existe
 El proceso 148 existe
 El proceso 155 no existe

7.4 Escriba un programa de nombre **saltos** en el que se deben fijar varios puntos de parada. La secuencia de instrucciones del programa debe ser:

- (a) Al arrancar, fijar el punto de repliegue número 1.
- (b) Calcular e imprimir una tabla con los cuadrados de los cinco primeros números naturales.
- (c) Fijar el punto de repliegue número 2.
- (d) Calcular e imprimir una tabla con las raíces cuadradas de los cinco primeros números naturales.
- (e) Fijar el punto de repliegue número 3.
- (f) Calcular e imprimir una tabla con los logaritmos naturales de los cinco primeros números naturales.
- (g) Entrar en un ciclo de espera de señales.

Cuando se ejecuta el programa y se imprimen las tres tablas, el programa se pone a dormir en espera de señales. Si se quiere volver a alguno de los puntos de repliegue definidos, se le debe enviar al proceso alguna de las señales mostradas en la tabla 7.7.

Tabla 7.7: Punto de repliegue asociado a cada señal.

Señal	Punto de repliegue
1	1
2	2
3	3

Para atender a las señales anteriores se debe emplear el mismo manejador. En la tabla 7.8 podemos ver una sesión de trabajo con el programa.

7.5 Reescriba los programas de ejemplo 7.6, 7.8 y 7.9 utilizando la llamada **sigaction**.

7.6 El objetivo de este ejercicio es practicar con la sincronización de procesos mediante el empleo de señales. Escriba un programa de nombre **sincro** que reciba a través de la línea de órdenes el número de procesos hijo que debe crear. La forma de llamar al programa será:

\$ sincro n

Tabla 7.8: Sesión de trabajo con el programa saltos.

\$ saltos &	\$ kill -2 278
[1] 278	Tabla de raíces cuadradas
Tabla de cuadrados	Número Raíz cuadrada
Número Cuadrado	1 1
1 1	2 1.41421
2 4	3 1.73205
3 9	4 2
4 16	5 2.23607
5 25	
	Tabla de logaritmos naturales
Tabla de raíces cuadradas	Número Logaritmo natural
Número Raíz cuadrada	1 0
1 1	2 0.693147
2 1.41421	3 1.09861
3 1.73205	4 1.38629
4 2	5 1.60944
5 2.23607	
	\$ kill -3 278
Tabla de logaritmos naturales	Tabla de logaritmos naturales
Número Logaritmo natural	Número Logaritmo natural
1 0	1 0
2 0.693147	2 0.693147
3 1.09861	3 1.09861
4 1.38629	4 1.38629
5 1.60944	5 1.60944

El primer proceso debe encargarse de crear al primer proceso hijo, el primer proceso hijo debe crear al segundo proceso hijo —que será nieto del primer proceso—, el proceso nieto debe crear otro proceso hijo que será proceso biznieto del primer proceso. La creación de procesos se detendrá cuando se hayan creado un total de $n + 1$ procesos —el patriarca más sus n descendientes—.

Una vez que todos los procesos han sido creados empezarán a comunicarse entre sí. Para ello es necesario introducir un primer punto de sincronización mediante el cual los procesos que se van a comunicar indican que están disponibles para iniciar la comunicación. Para esta primera sincronización cada proceso debe realizar las siguientes operaciones:

- (a) Esperar a que su proceso hijo le comunique que ha sido creado.
- (b) Comunicarle a su proceso padre que él ya ha sido creado.

Para realizar esta comunicación, cada proceso esperará la recepción de la señal **SIGUSR1** y una vez recibida se la enviará a su proceso padre. La señal **SIGUSR1** actúa a modo de testigo único que se intercambian los distintos procesos.

En este punto todos los procesos están creados y listos para iniciar la comunicación. Será el proceso padre quien desencadene la comunicación entre ellos. La secuencia de sincronismo que debe realizar cada proceso es:

- (a) Esperar a que su proceso padre le envíe la señal **SIGUSR1**.
- (b) Una vez recibida la señal **SIGUSR1**, enviársela a su proceso hijo para pasarle el turno.

En esta secuencia hay dos excepciones:

- El proceso padre inicial debe recibir la señal desde el proceso hijo final.
- El último proceso hijo —proceso hijo final— no tiene descendencia, por lo tanto le pasará la señal al proceso padre inicial.

Se establece así una trayectoria circular en el paso de la señal que actúa de testigo, donde el último proceso creado se la pasa al primero.

Este circuito se debe repetir un total de cinco veces al final de las cuales se iniciará una secuencia de terminación. La terminación se debe realizar en orden inverso al de creación. Así, el último proceso creado debe ser el primero en terminar. La secuencia de sincronismo que se debe ejecutar es:

- (a) Cuando el ciclo de sincronismo anterior se haya realizado cinco veces, el proceso padre inicial le enviará la señal **SIGTERM** a su proceso hijo.
- (b) Cada proceso hijo debe esperar la recepción de la señal **SIGTERM** procedente de su proceso padre.
- (c) Una vez que reciba esta señal, se la enviará a su proceso hijo.

Con la secuencia anterior se consigue que la orden de terminar se transmita desde el proceso padre inicial hasta el hijo final, atravesando toda la estructura de procesos creada.

Cuando el último proceso hijo recibe la señal **SIGTERM**, se encarga de devolvérsela a su proceso padre, iniciándose así un retorno de la señal en sentido contrario. La secuencia de sincronismo que debe ejecutar cada proceso es:

- (a) Esperar a recibir la señal **SIGTERM**.
- (b) Una vez recibida la señal **SIGTERM**, enviársela a su proceso padre.
- (c) Terminar su ejecución con una llamada a **exit**.

Hay que tener cuidado de que el proceso padre inicial no le envíe la señal **SIGTERM** a su proceso padre, ya que al ser éste el intérprete de órdenes, el resultado que se obtiene es la terminación de la sesión de trabajo.

Como se puede observar, en la transmisión de las señales **SIGUSR1** y **SIGTERM** se pueden distinguir dos sentidos en el flujo de las mismas:

- Desde los procesos padre a los hijo. Este flujo está asociado a la transmisión de órdenes.
- Desde los procesos hijo a los padre. Este flujo está asociado a la comunicación de la aceptación de la orden.

A la hora de contemplar estos dos sentidos en el fluir de las señales resulta bastante cómodo utilizar dos manejadores distintos por cada señal. Uno de los manejadores se activa cuando las señales viajan en un sentido y el otro cuando las señales viajan en sentido opuesto.

Pero la pregunta que nos podemos hacer es ¿cómo distinguir un sentido de otro? Esa distinción la deben realizar los propios manejadores y está determinada por el enunciado del problema. Como ejemplo, veamos cómo se realiza esta distinción para la señal `SIGTERM`:

- El primer flujo de esta señal tiene como origen los procesos padre y como destino los procesos hijo. Por lo tanto, al crear los procesos se instalará un manejador —`manejador_1`— que realice las siguientes operaciones:
 - (a) Instalar el segundo manejador —`manejador_2`— de la señal `SIGTERM`.
 - (b) Enviar la señal `SIGTERM` a su proceso hijo.
 - (c) Enviar la señal a su proceso padre. Esto sólo lo realiza el último proceso hijo que es quien debe encargarse de hacer que la señal inicie su camino de vuelta.
- Si nos fijamos, la primera acción del `manejador_1` es instalar el `manejador_2`. Esto se hace porque sabemos que la próxima vez que se reciba la señal, el sentido de su viaje será el contrario. El `manejador_2` realiza las operaciones siguientes:
 - (a) Enviar la señal `SIGTERM` a su proceso padre.
 - (b) Terminar la ejecución del proceso mediante una llamada a `exit`.

Para determinar las acciones que deben realizar los manejadores de la señal `SIGUSR1` habrá que realizar un análisis parecido.

A continuación podemos ver un ejemplo de ejecución de este programa:

```
$ sincro 3
```

```
Hijo nro. 1 recién creado. Pid = 532
Hijo nro. 2 recién creado. Pid = 533
Hijo nro. 3 recién creado. Pid = 534
Último proceso creado
Proceso hijo nro. 3 listo
Proceso hijo nro. 2 listo
Proceso hijo nro. 1 listo
Proceso padre listo
Proceso padre. Contador = 5
Proceso hijo nro. 1
```

```

Proceso hijo nro. 2
Proceso hijo nro. 3
Proceso padre. Contador = 4
Proceso hijo nro. 1
Proceso hijo nro. 2
Proceso hijo nro. 3
Proceso padre. Contador = 3
Proceso hijo nro. 1
Proceso hijo nro. 2
Proceso hijo nro. 3
Proceso padre. Contador = 2
Proceso hijo nro. 1
Proceso hijo nro. 2
Proceso hijo nro. 3
Proceso padre. Contador = 1
Proceso hijo nro. 1
Proceso hijo nro. 2
Proceso hijo nro. 3
Proceso padre. Contador = 0
Inicio de la secuencia de terminación
Proceso hijo nro. 3 terminado
Proceso hijo nro. 2 terminado
Proceso hijo nro. 1 terminado
Proceso padre terminado

```

7.7 Escriba una nueva versión de la función `dormir` para que admita un argumento en coma flotante. La nueva declaración de `dormir` será:

```
dormir (float segundos);
```

donde `segundos` es el número de segundos que debe permanecer suspendido el proceso. Para implementar esta función hay que utilizar temporizadores con una resolución superior a 1 segundo, ya que la parte fraccionaria del argumento `segundos` es también significativa.

Con el programa siguiente se puede probar el correcto funcionamiento de `dormir`:

```

#include <stdio.h>
#include <math.h>

main ()
{
    int i;
    float s;

    for (i = 0; i < 10; i++) {
        s = (float)rand ()/RAND_MAX;

```

```

        s *= 5;
        printf ("Durmiendo %g segundos.\n", s);
        dormir (s);
    }
}

```

- 7.8** Escriba el programa **parar**, que suspende su ejecución durante un período de tiempo determinado. La forma de invocarlo debe ser:

```
$ parar tiempo
```

donde **tiempo** es un número real que expresa el total de segundos que el programa debe estar parado, por ejemplo:

```
$ parar 10.54
```

Utilice la función **atof** para convertir una cadena de caracteres numéricos en un número **float**.

- 7.9** Escriba una nueva versión de la función **sistema** para que admita un segundo argumento en coma flotante. La nueva declaración de **sistema** es:

```
sistema (const char *orden, float espera);
```

donde **orden** es una cadena de caracteres con la orden que debe ejecutar el intérprete **/bin/sh** y **espera** es un tiempo expresado en segundos que indica el plazo que se le otorga al intérprete para ejecutar la orden. En el caso de que este tiempo transcurra sin que haya terminado la ejecución de la orden, el proceso debe tomar el control y enviarle la señal **SIGTERM** al proceso que está ejecutando la orden. Acto seguido le devolverá el control a la función que llamó a **sistema**. Con esta nueva función conseguimos que un proceso no se quede bloqueado en una llamada a **sistema**.

Esta nueva función se puede probar con el programa siguiente:

```

main ()
{
    sistema ("clear", 1.0);
    sistema ("ls *.c", 0.5);
    sistema ("ps -eax | grep agetty", 0.1);
    sistema ("ps -eax | grep agetty", 0.9);
    sistema ("sleep 5", 5.67);
    sistema ("sleep 1200", 5.67);
}

```

La salida que se obtiene será algo parecido a lo siguiente:

```

sistema (clear, 1). Orden ejecutada satisfactoriamente.
dormir.c
existe.c
p7_6.c
p7_8.c

```



```

p7_9.c
saltos.c
sincro.c
sistema.c
sistema (ls *.c, 0.5). Orden ejecutada satisfactoriamente.
sistema (ps -eax | grep agetty, 0.1). Tiempo de espera concluido.
  74 v01 S      0:00 /sbin/agetty 9600 tty1
  78 v05 S      0:00 /sbin/agetty 9600 tty5
  79 v06 S      0:00 /sbin/agetty 9600 tty6
 787 v03 R      0:00 grep agetty
sistema (ps -eax | grep agetty, 0.9). Orden ejecutada satisfactoriamente.
sistema (sleep 5, 5.67). Orden ejecutada satisfactoriamente.
sistema (sleep 1200, 5.67). Tiempo de espera concluido.

```

7.10 Escriba un programa de nombre **agenda** que sirva para arrancar un proceso en una fecha y hora determinada. La forma de usar este programa será:

```
$ agenda [-h hora] -f fecha [-r repeticiones] [-t espera] -o "orden"
```

El significado de los argumentos es el siguiente:

- **orden**, orden que se debe ejecutar cuando se alcance la fecha y hora indicadas. La orden se debe indicar entre comillas dobles, por ejemplo "**ls -al**".
- **fecha**, día en el que se debe ejecutar la orden indicada. La forma de introducirlo será DD/MM/AA:
 - DD: día, rango=[1..31]
 - MM: mes, rango=[1..12]
 - AA: año, rango=[0..99]
- **hora**, hora a la que se debe ejecutar la orden dentro del día indicado. Su valor por defecto son las 0 horas. Se introduce de la forma hh:mm:ss:
 - hh: hora, rango=[1..23]
 - mm: minutos, rango=[0..59]
 - ss: segundos, rango=[0..59]
- **repeticiones**, número de veces que se ejecutará la orden una vez alcanzada la fecha indicada. Su valor por defecto es 1.
- **espera**, tiempo que se debe esperar antes de repetir la ejecución de la orden. Su valor por defecto debe ser 1 segundo.

En el caso de que no se introduzca alguno de los parámetros obligatorios **fecha** y **orden**, el programa debe sacar un mensaje de error.

Para que el uso de los parámetros sea más flexible, éstos no serán sensibles a la posición. Es decir, el parámetro **hora**, por ejemplo, puede ir en cualquier posición —primera, segunda,... última—. Lo mismo ocurre con el resto de parámetros.

Para realizar el análisis de los argumentos, cuando son muchos y no dependen de la posición, se suele emplear la función `getopt`. Esta función está documentada en las páginas del manual de UNIX. A continuación se muestra la forma de usarla para el análisis de los argumentos del ejemplo:

```
struct tm fecha;
int espera_entre_repeticiones = 1;
int repeticiones = 1;
int parámetros_obligatorios = 2;
char *orden;

char opción;
int errores = 0;
extern char *optarg;

memset (&fecha, 0, sizeof (fecha));
// Análisis de los argumentos de la línea de órdenes
while ((opción = getopt (argc, argv, "h:f:r:t:o:")) != -1)
    switch (opción) {
        case 'f':
            if (sscanf (optarg, "%d/%d/%d",
                        &fecha.tm_mday,
                        &fecha.tm_mon, &fecha.tm_year) != 3) {
                fprintf (stderr,
                        "%s: formato de fecha erróneo %s\n",
                        argv [0], argv [2]);
                exit (-1);
            }
            --fecha.tm_mon; // El rango de los meses es [0..11]
            --parámetros_obligatorios;
            break;
        case 'h':
            // Parecido al caso 'f'
            break;
        case 'r':
            repeticiones = atoi (optarg);
            break;
        case 't':
            // Parecido al caso 'r'
            break;
        case 'o':
            // Parecido al caso 'r'
            break;
        default:
            ++errores;
    }
}
```

```
if (errores)
    // Error en el uso de los parámetros.
if (parámetros_obligatorios)
    // Faltan parámetros obligatorios.
```

Una vez escrito el programa, se podrán hacer apuntes en la agenda como el siguiente:

```
$ agenda -t 60 -r 5 -h 13:30:00 -f 23/4/03 -o echo "HORA DE COMER"
```

Capítulo 8

Perfilado, contabilidad y depuración

«Tenga vuestra merced cuenta en las cabras que el pescador va pasando, porque si se pierde una de la memoria, se acabará el cuento, y no será posible contar más palabra dél.»

(Cervantes, Quijote I-20)

8.1	Perfil de un proceso	309
8.2	Contabilidad	315
8.3	Depuración de programas	322

8.1 Perfil de un proceso

Cuando se está desarrollando una aplicación, puede ser interesante realizar medidas estadísticas para conocer cómo se distribuye el tiempo total de ejecución entre sus distintas funciones. Estas medidas pueden servirnos para conocer cuáles son las partes más ejecutadas y tomar decisiones de desarrollo. Por ejemplo, si sabemos que una función ocupa el 50% del tiempo total de la aplicación, a la hora de diseñar una estrategia para mejorar la aplicación tendremos que dedicar un gran esfuerzo a mejorar esa función en concreto. La mejora irá encaminada hacia sus tiempos de respuesta, fiabilidad, etc. Estas consideraciones tienen aplicación cuando se da una limitación en el tiempo disponible para desarrollo y queremos diseñar una estrategia óptima.

La forma de elaborar la estadística anterior es creando un perfil de ejecución de la aplicación. En UNIX se pueden crear perfiles con la llamada `profil`. La parte del núcleo encargada de hacer el seguimiento de un proceso y crear su perfil se conoce como *perfilador* y su principio de funcionamiento está basado en la interrupción periódica del reloj. Cada vez que se produce una interrupción del reloj del sistema, el perfilador examina la dirección que contiene el contador de programa —*PC*— del proceso que se está ejecutando e

incrementa un contador asociado a esa dirección. El proceso que estamos perfilando tiene asociada una memoria intermedia que contiene un contador por cada dirección o grupo de direcciones de su segmento de texto. A medida que transcurre el tiempo, en la memoria intermedia se va creando un histograma de la ejecución del proceso. Como la memoria intermedia es un mapa de las direcciones del segmento de texto, consultando los valores de este mapa podemos saber qué partes del segmento de texto son las más ejecutadas. Si calculamos a qué funciones corresponden las distintas direcciones, podremos formar una estadística de tiempos consumidos por cada función.

En los programas de tipo determinista no es necesario recurrir a técnicas de perfilado para saber por dónde va a transcurrir su ejecución. En este tipo de programas se puede conocer, sin necesidad de realizar medidas, cuáles son las partes más ejecutadas y por lo tanto dónde se deben realizar los esfuerzos de depuración. Es en los programas de tipo estocástico donde la creación de perfiles tendrá utilidad.

La llamada para activar el dispositivo perfilador tiene la siguiente declaración:

```
#include <sys/param.h>
void profil (char *buff, int bufsiz, int offset, int scale);
```

donde **buff** apunta a una área de memoria cuya longitud —en bytes— viene dada por **bufsiz**. Una vez que se efectúa la llamada a **profil**, el contador de programa de usuario es examinado con cada interrupción del reloj; a la dirección que contiene el contador de programa se le resta el valor de **offset** y si el número que se obtiene corresponde a un índice dentro del rango de **buff**, se incrementa el valor de esa entrada de **buff**.

El parámetro **scale** se interpreta como una fracción sin signo en coma fija, con la coma decimal a la izquierda del bit más significativo. Si **scale** vale **0xffff** —valor hexadecimal—, el núcleo interpreta que por cada dirección del contador de programa dentro del rango a perfilar hay una entrada en **buff**; si **scale** vale **0x7fff**, por cada par de direcciones a perfilar, habrá una entrada en **buff**; el valor **0x2** hace que a todas las direcciones les corresponda la primera entrada de **buff**. En el caso de que **scale** valga 0 ó 1, la creación del perfil es detenida. También se detiene cuando se ejecuta una llamada a **exec** o cuando una actualización de **buff** puede provocar un fallo de memoria por intentar escribir fuera de sus límites. Un proceso hijo hereda la llamada a **profil** efectuada en el padre.

Como ejemplo, veamos un programa que codifica la forma de crear el perfil de uso de sus funciones. Consta de tres funciones de cálculo: **main**, **f** y **g**; y una función de presentación de resultados: **fin**. Esta última será tenida en cuenta a la hora de confeccionar el perfil. Las funciones **f** y **g** simulan sendos procesos de cálculo cuyo tiempo de ejecución está indeterminado. Para llevar a cabo esta simulación, ambas funciones constan de un bucle controlado por una variable que se inicializa —en cada llamada a la función— con números pseudoaleatorios uniformemente distribuidos. El rango de posibles valores iniciales para el índice es 5 veces superior en el caso de la función **g**. Podemos predecir así que la función **g** empleará, por término medio, el 86,66 % del tiempo del proceso y la función **f** empleará el 13,33% restante. La función **main** no emplea prácticamente nada del tiempo total, porque se limita a hacer llamadas sucesivas a **f** y **g**. Cabe esperar que los tiempos medios en el perfil del proceso se ajusten a los calculados teóricamente. A medida que transcurre el tiempo, se puede observar que los resultados experimentales se aproximan a los teóricos. De todas formas, lo importante no es la exactitud, sino observar cómo hay

una función que emplea la mayor parte del tiempo total del proceso y que, por tanto, los primeros esfuerzos de optimización y depuración deben ir dirigidos hacia esa función.

Programa 8.1: Creación del perfil de un proceso (`perfil.c`)

```
/**
```

```
$ perfil
```

Forma de compilación:

```
$ cc -o perfil perfil.c -lm
```

Este programa muestra cómo crear un perfil de ejecución de un proceso. El programa consta de tres funciones cuya ejecución queremos observar: `main`, `f`, y `g`; y una función de presentación de datos: `fin`. Mediante la llamada `profil` le indicamos al sistema que calcule el perfil de las tres funciones que nos interesen. Cada 10 segundos presentamos por la salida estándar el porcentaje de tiempo que cada función se ha estado ejecutando.

```
*/
```

```
#include <stdio.h>
```

```
#include <signal.h>
```

```
#include <math.h>
```

```
#define MAXF 1000 /* Constante asociada a la función f. */
```

```
#define MAXG 5000 /* Constante asociada a la función g. */
```

```
/* Estructura asociada al perfil de una función. */
```

```
struct func {
```

```
    /* Nombre de la función. */
```

```
    char *nombre;
```

```
    /* Direcciones de inicio y final de la función. */
```

```
    int inicio, fin;
```

```
    /* Contador con el número de veces que el dispositivo perfilador ha encontrado  
    que el contador de programa tenía una dirección correspondiente al código de la  
    función. */
```

```
    unsigned contador;
```

```
};
```

```
/* Prototipo de las funciones del módulo. */
```

```
int main (), f (), g();
```

```
void fin ();
```

```
/* Inicio de la zona de código a perfilar. */
```

```
#define FINICIO ((int)main)
```

```
/* Final de la zona de código a perfilar. */
```

```
#define FFIN ((int)fin)
```

```
/* Array de estructuras de perfilado. */
```

```
/* Cada elemento del array se inicializa en tiempo de compilación con:
```

```
    - Nombre de la función.
```

- Posición de inicio de la función (nombre de la función).
- Posición de final de la función (nombre de la función siguiente menos 1).
- Contador de ejecución. Su valor inicial es 0.

```

*/
struct func funciones [] = {
    "main", (int)main, (int)f - 1, 0,
    "f", (int)f, (int)g - 1, 0,
    "g", (int)g, (int)fin - 1, 0
};

/* Macros para acceder a las posiciones inicial y final de la función que se controla con
el elemento x del array funciones. */
#define inicio_func(x) ((funciones[(x)].inicio-FINICIO) / sizeof(int))
#define fin_func(x) ((funciones[(x)].fin - FINICIO) / sizeof (int))

#define total_funciones (sizeof (funciones) / sizeof (struct func))
unsigned *buf, bufsiz;

main (int argc, char *argv [])
{
    /* Asignación dinámica de memoria para el array de perfiles. */
    bufsiz = FFIN - FINICIO;
    if ((buf = (int *)calloc(bufsiz, sizeof (int))) == NULL) {
        perror (argv[0]);
        exit (-1);
    }

    /* Activamos el dispositivo de perfilado. El mapa de perfiles cumple una
relación 1:1 con la zona de código que se está analizando. */
    profil (buf, bufsiz, FINICIO, 0xffff);
    /* Instalación de la función fin como manejador de la señal SIGALRM. Cada 10
segundos, el proceso recibirá esta señal. */
    signal (SIGALRM, fin);
    alarm (10);

    /* Bucle principal de ejecución. */
    for (;;) {
        f ();
        g ();
    }
}

/**
f: esta función simula un tratamiento genérico de información. Como no nos
interesa tratar nada específico, la función consta solamente de un bucle con un

```

contador decreciente donde se calculan raíces cuadradas. El valor inicial del contador se programa de acuerdo con un número pseudoaleatorio que puede tomar un valor máximo MAXF.

```

***/
int f()
{
    long i = random() % MAXF;
    double aux;

    while (i--)
        aux = sqrt (i);
}

```

/*
g: esta función simula un tratamiento genérico de información. Como no nos interesa tratar nada específico, la función consta solamente de un bucle con un contador decreciente donde se calculan raíces cuadradas. El valor inicial del contador se programa de acuerdo con un número pseudoaleatorio que puede tomar un valor máximo MAXG.

```

***/
int g()
{
    long i = random() % MAXG;
    double aux;

    while (i--)
        aux = sqrt (i);
}

```

/*
fin: esta función es el manejador de la señal SIGALRM. Cada 10 segundos entra en funcionamiento y su misión es analizar el mapa de perfiles para actualizar los contadores de cada una de las funciones que hay en el array **funciones**.

```

***/
void fin ()
{
    int i, j;
    double total;
    static int cnt = 0;

    /* La forma de actualizar el mapa de perfiles es la siguiente:
        1 - Se recorre todo el mapa de perfiles.
        2 - Si el elemento del mapa que se analiza corresponde a alguna de las
           direcciones del código de las funciones que queremos perfilar, se
           incrementa el contador asociado a esa función.
    */

```



```

for (i = 0; i < bufsiz/sizeof (int); i++)
    if (buf[i])
        for (j = 0; j < total_funciones; j++)
            if (i >= inicio_func (j) && i <= fin_func (j))
                funciones[j].contador += buf[i];

/* Recorrido del array funciones para calcular totales y poder presentar
resultados en forma de porcentajes de ejecución. */
total = 0;
for (i = 0; i < total_funciones; i++)
    total += funciones[i].contador;
/* Presentación de resultados. */
for (i = 0; i < total_funciones; i++)
    printf ("%s = %d = %g%%\n",
            funciones [i].nombre,
            funciones[i].contador,
            funciones[i].contador/total*100);
printf ("\n");

/* Activación del manejador de SIGALRM. */
if (++cnt < 5)
    signal (SIGALRM, fin);
else
    exit (0);
alarm (10);
}

```

Como se puede ver en el listado, la función principal consta de un bucle infinito que llama sucesivamente a `f` y `g`. Cada 10 segundos se recibe la señal `SIGALRM`, que hace entrar en funcionamiento la rutina `fin`. Esta función analiza el mapa gestionado por el perfilador y presenta resultados por pantalla. Cuando `fin` se ejecuta 5 veces, termina la ejecución del proceso mediante una llamada a `exit`.

A continuación se presenta un ejemplo de la salida que produce el programa `perfil`, donde se puede apreciar que los resultados experimentales se aproximan a los teóricos:

\$ `perfil`

```

main = 0 = 0%
f = 2490402 = 15.3226%
g = 13762731 = 84.6774%

main = 1 = 1.97395e-06%
f = 6881373 = 13.5835%
g = 43778574 = 86.4165%

main = 3 = 2.86457e-06%
f = 13303980 = 12.7034%

```

```

g = 91423766 = 87.2966%

main = 5 = 2.82881e-06%
f = 22413595 = 12.6808%
g = 154339027 = 87.3192%

main = 7 = 2.61662e-06%
f = 33096103 = 12.3714%
g = 234424894 = 87.6286%

```

La inclusión de código por parte del programador para llevar a cabo la creación de perfiles es una solución no trivial e incómoda. Afortunadamente, podemos liberarnos de ese trabajo gracias a que el compilador de C puede generar automáticamente el código de perfilado. La forma de indicarlo es con la opción `-p`. El programa así compilado incorpora código que realiza una llamada a la función estándar `monitor`, que activa el perfilador. Esta función es una interfaz para la llamada `profil`. Al terminar la ejecución del programa, se realiza otra llamada a `monitor` para forzarla a generar el fichero `mon.out`, que contiene el perfil de ejecución de las distintas funciones. Los datos de `mon.out` pueden ser interpretados con la ayuda del programa `prof`. La secuencia de órdenes necesaria para la generación automática de perfiles es la siguiente:

```

$ cc -p -o perfil perfil.c    Compilación de perfil.c con inclusión auto-
                             mática de código para el perfilador.
$ perfil                     Ejecución del programa. Al terminar genera
                             el fichero mon.out.
$ prof perfil                Interpretación del contenido de mon.out. El
                             programa prof utiliza el fichero ejecutable
                             para leer las tablas de símbolos donde aparecen
                             los nombres de las funciones. En mon.out no
                             están esos nombres, sino sus direcciones vir-
                             tuales asociadas durante la ejecución.

```

8.2 Contabilidad

En ocasiones se hace necesario realizar la contabilidad de los usuarios que utilizan el sistema, qué órdenes ejecutan, qué tiempo, tanto real como de CPU, consume cada usuario, etc. La contabilidad puede ser útil por razones puramente estadísticas, de economía o seguridad.

La contabilidad por razones económicas se hace necesaria cuando un ordenador es alquilado para prestar servicios a usuarios de diferentes empresas. Imaginemos un superordenador vectorial de cálculo científico que presta servicio a empresas medianas que no tienen presupuesto para adquirir un equipo de esas magnitudes. A la hora de facturar, es necesario saber cuánto tiempo ha estado cada cliente empleando el ordenador.

La contabilidad por razones de seguridad es útil cuando sospechamos que algún usuario intenta adquirir privilegios para los que no está autorizado. Es común que en las universidades los estudiantes no estén conformes con los permisos que les asigna el admi-

nistrador del sistema y dediquen grandes cantidades de tiempo a la tarea de convertirse en superusuarios. Observando quién ejecuta la orden `su`, podemos estar al tanto de esas tentativas.

UNIX dispone de órdenes estándar para habilitar y deshabilitar la contabilidad. Se trata de `accton` para habilitar/deshabilitar la contabilidad, y `acctcom` para visualizar el contenido de un fichero de contabilidad. Estas órdenes se deben ejecutar con privilegios de superusuario.

También hay una llamada al sistema que se utiliza para habilitar y deshabilitar la contabilidad. Esta llamada es `acct` y su declaración es la siguiente:

```
int acct (char *path);
```

donde `path` es un puntero a la ruta del fichero sobre el que se escribirán los registros de contabilidad. Si `path` es distinto de `NULL`, la contabilidad se habilita; en caso contrario, se deshabilita. Para que la llamada se realice con éxito, es necesario que el proceso que la ejecuta tenga privilegios de superusuario y que el fichero referenciado en `path` exista y se pueda abrir para escribir en él.

Una vez que la contabilidad está habilitada, el núcleo se encarga de añadir un nuevo registro al fichero por cada una de las órdenes que ejecute cualquier usuario. Consultando el fichero de contabilidad podemos conocer la historia reciente de ejecución del sistema. El fichero irá creciendo hasta que se deshabilite la contabilidad. Los registros de este fichero tienen la estructura de `struct acct` definida en `<sys/acct.h>`:

```
struct acct {
    char    ac_flag; // Indicador
    char    ac_stat; // Estado de salida de la orden tras ejecutarse
    ushort  ac_uid;  // Identificador del usuario que ejecutó la orden
    ushort  ac_gid;  // Identificador del grupo
    dev_t   ac_tty;  // Terminal de control
    time_t  ac_btime; // Hora de inicio
    comp_t  ac_utime; // Tiempo total de ejecución en modo usuario,
                    // medido en pulsos de reloj
    comp_t  ac_stime; // Tiempo total de ejecución en modo supervisor,
                    // medido en pulsos de reloj
    comp_t  ac_etime; // Tiempo transcurrido, medido en pulsos de reloj
    comp_t  ac_mem;   // Uso de memoria
    comp_t  ac_io;    // Caracteres transferidos
    comp_t  ac_rw;    // Bloques leídos o escritos
    char    ac_comm[8]; // Nombre de la orden ejecutada
};
```

Las siguientes constantes tienen significado para el campo `ac_flag`:

```
#define AFORK 01 // La orden ha ejecutado fork, pero no exec
#define ASU 02  // La orden ha sido ejecutada con privilegios de superusuario
```

Como ejemplo de aplicación, veamos un programa que combina parte de las operaciones de `accton` y `acctcom`.

Programa 8.2: Contabilidad de procesos (cont.c)

/***

\$ cont [opciones] [fichero]

Programa de contabilidad del sistema. Aúna bajo un mismo programa las órdenes estándar **accton** y **acctcom**.

Opciones:

- e Habilita la contabilidad. Por defecto se escribirá en el fichero **\$HOME/adm/pacct**. Requiere permisos de superusuario.
- d Deshabilita la contabilidad. Requiere permisos de superusuario.
- r Muestra por pantalla el contenido del fichero de contabilidad. Por defecto, se lee el fichero **\$HOME/adm/pacct**. La información que se muestra es:

ORDEN Nombre de la orden ejecutada. Si se ejecutó con permisos de superusuario, aparece precedida por el carácter #.

USUARIO Nombre del usuario que ejecutó la orden.

TTY Nombre del terminal de control desde el que se ejecutó la orden.

HORA INICIO En formato **hh:mm:ss**

HORA FIN En formato **hh:mm:ss**

BLK L/E Número de bloques leídos o escritos por la orden.

CARACT L/E Número de caracteres leídos o escritos por la orden.

T. REAL Tiempo real, en segundos, invertido en la ejecución.

T. CPU Tiempo de CPU, en segundos, invertido en la ejecución.

Las tres opciones anteriores son autoexcluyentes. Sólo se realiza la primera que aparezca en la línea de órdenes.

- f fichero En el caso de que estén presentes **-e** o **-r**, sirve para especificar un fichero de contabilidad distinto de **\$HOME/adm/pacct**.

Compilación:

\$ cc -o cont cont.c**\$ cc -o cont cont.c -DFreeBSD** Para el sistema FreeBSD

***/*

#include <stdio.h>**#include <sys/types.h>****#include <sys/acct.h>****#include <pwd.h>****#include <time.h>****#include <sys/param.h>****#define PERMISOS 0644** /* Permisos con los que se crea el fichero de contabilidad. */

/* Opciones posibles en la línea de órdenes. */

char opciones[] = "derf:";**char error[] = "[-der] [-f fichero]";**

/* Nombre del programa. */

```

char *nombre_programa;

/****
    main: analiza los argumentos que hay en la línea de órdenes y actúa de acuerdo
    con esos parámetros.
****/
main (int argc, char *argv [])
{
    /* Declaración de variables necesarias para getopt. */
    int c;
    extern char *optarg;
    extern int optind;
    int errflg = 0;

    /* Declaración de variables y funciones prototipo necesarias para llevar a cabo la
    contabilidad. */
    char *home, acct_path [256];
    enum {NINGUNA, ACTIVAR, DESACTIVAR, LEER} accion = NINGUNA;
    FILE *f;
    struct acct act;
    comp_t bloques;
    struct passwd *pw, *getpwuid ();
    struct tm *tm;
    time_t tfin;
    char *getenv (), *nombre_tty ();

    nombre_programa = argv [0];
    /* Análisis de las opciones presentes en la línea de órdenes. */
    while ((c=getopt (argc, argv, opciones)) != EOF)
        switch (c) {
            case 'd':
                accion = DESACTIVAR;
                break;
            case 'e':
                accion = ACTIVAR;
                /* Formación de la ruta del fichero de contabilidad
                $HOME/adm/pacct. */
                home = getenv ("HOME");
                sprintf (acct_path, "%s/adm/pacct", home);
                break;
            case 'r':
                accion = LEER;
                /* Formación de la ruta del fichero de contabilidad
                $HOME/adm/pacct. */
                home = getenv ("HOME");
                sprintf (acct_path, "%s/adm/pacct", home);

```

```

        break;
    case 'f':
        /* El fichero de contabilidad es el suministrado por el usuario. */
        sprintf (acct_path, "%s", optarg);
        break;
    case '?':
        errflg++;
    }
    if (errflg) {
        fprintf (stderr, "Forma de uso: %s %s\n", argv [0], error);
        exit (-1);
    }

    /* Acciones a tomar en función del análisis de las opciones presentes en la
    línea de órdenes. */
    switch (accion) {
    case DESACTIVAR: /* Desactivar la contabilidad. */
        if (acct (NULL) == -1) {
            perror (argv[0]);
            exit (-1);
        } else {
            printf ("Contabilidad desactivada.\n");
            exit (0);
        }
    case ACTIVAR: /* Activar la contabilidad. */
        close (creat (acct_path, PERMISOS));
        if (acct (acct_path) == -1) {
            perror (acct_path);
            exit (-1);
        } else {
            printf ("Contabilidad activada sobre %s\n", acct_path);
            exit (0);
        }
    case LEER:
        /* Leer y presentar en pantalla el contenido del fichero de contabilidad. */
        if ((f = fopen (acct_path, "r")) == NULL) {
            perror (acct_path);
            exit (-1);
        }
        printf ("%9s %-8s %-8s %-8s %-8s %-5s %8s\t%-5s\t%-5s\n",
            "", "", "", "HORA", "HORA", "BLK", "CARACT", "T. REAL", "T. CPU");
        printf ("%9s %-8s %-8s %-8s %-8s %-5s %8s\t%-5s\t%-5s\n",
            "ORDEN", "USUARIO", "TTY", "INICIO", "FIN",
            "L/E", "L/E", "(s.)", "(s.)");
        while (fread (&act, sizeof (act), 1, f) == 1) {
            printf ("%c", (act.ac_flag & ASU) ? '#' : ' ');

```

```

printf ("%8s ", act.ac_comm);
/* Conversión del UID a nombre de usuario. */
pw = getpwuid (act.ac_uid);
printf ("%8s ", pw->pw_name);
printf ("%8s ", nombre_tty (act.ac_tty));
tm = localtime (&act.ac_btime);
printf ("%02d:%02d:%02d ",tm->tm_hour,tm->tm_min,tm->tm_sec);
#ifdef FreeBSD
    /* En algunos sistemas, es el caso de FreeBSD, la constante HZ se
    llama AHZ y la estructura struct acct carece del campo ac_rw,
    por ello utilizamos las directivas de compilación condicional. */
    #define HZ AHZ
    bloques = act.ac_io/BUFSIZ;
#else
    bloques = act.ac_rw;
#endif
tfin = act.ac_btime + act.ac_etime/HZ;
tm = localtime (&tfin);
printf ("%02d:%02d:%02d ",tm->tm_hour,tm->tm_min,tm->tm_sec);
printf ("%5d ", bloques);
printf ("%8d\t", act.ac_io);
printf ("%2.2f\t", (float)act.ac_etime/HZ);
printf ("%2.2f\n", (float)(act.ac_utime + act.ac_stime)/HZ);
}
fclose (f);
exit (0);
case NINGUNA: /* Caso de que se haya invocado mal al programa. */
    fprintf (stderr, "Forma de uso: %s %s\n", argv [0], error);
    exit (-1);
}
}

```

/**

nombre_tty: a partir de un número de dispositivo, devuelve un puntero a una cadena de caracteres que contiene el nombre del fichero de dispositivo al que pertenece el número.

La dirección devuelta corresponde a un array estático de la función.

Para buscar el nombre del fichero se procede de la siguiente forma:

- 1— Abrir el directorio /dev
- 2— Leer una entrada del directorio y comparar su número de dispositivo con el que buscamos.
- 3— Repetir el procedimiento hasta encontrar el número deseado.

```

char *nombre_tty (dev_t dev)
{
# include <sys/types.h>

```

```

# include <dirent.h>
# include <sys/stat.h>
    DIR *dir;
    struct dirent *dirent;
    char tty_path [256];
    static char n_tty [256];
    struct stat buf;

    /* Apertura del directorio /dev. */
    if ((dir = opendir ("/dev")) == NULL) {
        perror (nombre_programa);
        exit (-1);
    }

    /* Situamos el puntero de lectura en la tercera entrada. Las dos primeras (0 y 1)
    corresponden a los ficheros . y .. */
    seekdir (dir, 2);
    /* Leemos todas las entradas del directorio hasta que encontramos la buscada. */
    while ((dirent = readdir (dir)) != NULL) {
        sprintf (tty_path, "/dev/%s", dirent->d_name);
        /* Lectura del nodo-i asociado a la entrada. */
        if (stat (tty_path, &buf) == -1) {
            perror (tty_path);
            exit (-1);
        }
        /* El campo del nodo-i que interesa es st_dev, que contiene el número de
        dispositivo de un fichero especial modo bloque o carácter. */
        if (buf.st_rdev == dev) {
            sprintf (n_tty, "%s", dirent->d_name);
            break;
        }
    }
    closedir (dir);

    return (n_tty);
}

```

Para activar la contabilidad primero hay que adquirir privilegios de superusuario, después podemos escribir:

```
$ cont -e
```

```
Contabilidad activada sobre /home/Francisco/adm/pacct
```

Una vez activa, podemos visualizar el fichero con la orden:

```
$ cont -r
```


			HORA	HORA	BLK	CARACT	T.REAL	T.CPU
ORDEN	USUARIO	TTY	INICIO	FIN	L/E	L/E	(s.)	(s.)
#cont	root	tty01	22:36:00	22:36:00	1	54	0.04	0.02
#sh	root	tty01	22:35:31	22:36:01	0	1495	30.70	0.12
ls	Francisc	tty01	22:36:10	22:36:10	1	676	0.04	0.02
ps	Francisc	tty01	22:36:13	22:36:13	2	17827	0.08	0.06
cont	Francisc	tty01	22:36:21	22:36:21	0	7730	0.10	0.10

Para desactivar la contabilidad hay que volver a adquirir privilegios de superusuario y ejecutar el programa `cont`.

```
$ cont -d
```

Contabilidad desactivada.

8.3 Depuración de programas

El desarrollo de un programa o aplicación atraviesa varias fases desde su concepción hasta que el producto final está listo para ser explotado. Una de las fases que requiere más tiempo y también una de las más desagradables es la depuración de errores internos en la concepción e implementación de los algoritmos de tratamiento de la información. Para llevar a cabo esta tarea, es muy útil contar con la ayuda de herramientas de depuración tales como los programas depuradores.

Un *depurador* es un programa que permite controlar la ejecución de otro programa. Entre las posibilidades de un depurador están: detener la ejecución del programa depurado, marcar puntos de parada, ejecutar paso a paso, visualizar el valor que toman las variables del programa depurado, modificar el valor de esas variables, etc. Un depurador es, por lo tanto, una herramienta muy útil para el programador en la fase de puesta a punto de un programa.

Los lenguajes de alto nivel suelen tener opciones para incluir código especial en los programas ejecutables que generan. Este código será utilizado con posterioridad por el depurador. En el caso del compilador de C para UNIX, la opción `-g` cumple esa función.

El sistema UNIX suministra dos depuradores estándar: `adb` y `sdb`; en Linux disponemos también de `gdb`. El primero es un depurador de bajo nivel que permite depurar programas escritos en ensamblador; el segundo y el tercero se utilizan para programas escritos en C. Los tres depuradores están implementados en base a la llamada `ptrace`. Ésta es una llamada al sistema que permite la comunicación entre dos procesos: el proceso padre, que actúa como depurador, y el hijo, que actúa como proceso depurado. La declaración de esta llamada es la siguiente:

```
int ptrace (int request, int pid, int addr, int data);
```

El uso de esta llamada se muestra en el pseudocódigo siguiente:

```
if ((pid = fork ()) == 0) {
    // Proceso hijo
    ptrace (0, 0, 0, 0);
```

```
    exec ("Nombre del programa a depurar");
    exit ();
}
// Proceso padre, actúa como depurador
for (;;) {
    wait ((int *)0);
    leer (orden del depurador);
    ptrace (orden, pid, ...);
    if (orden == salir)
        break;
}
```

El depurador se encarga de crear un proceso hijo mediante una llamada a `fork`. El proceso hijo hace una llamada a `ptrace` antes de cargar en memoria el programa que se desea depurar. El objeto de esta llamada es que el núcleo active el *bit de traza* en la entrada de la tabla de procesos correspondiente al hijo. A continuación, el proceso hijo carga en memoria el programa que se desea depurar. Al terminar la ejecución de `exec`, y nada más pasarle el control al programa que se desea ejecutar, éste recibe la señal `SIGTRAP` que lo lleva a un estado de espera. Si el bit de traza no estuviera activo, la señal `SIGTRAP` ocasionaría que el proceso terminase su ejecución previa generación de un fichero `core`.

Mientras tanto, el proceso padre ha ejecutado una llamada a `wait` en espera de acontecimientos. Cuando el hijo entra en espera por recibir la señal `SIGTRAP`, el padre es notificado cuando `wait` le devuelve el control. En estos momentos el proceso depurador puede comunicarse con el depurado a través de la llamada a `ptrace`. La orden que le envía el padre al hijo está codificada en el parámetro `request`; `pid` es el identificador del proceso al que se le envía la orden. Lo normal es que un depurador trabaje con un solo hijo, pero podría hacerlo con varios. En este caso hay que tener en cuenta el valor devuelto por `wait`, ya que coincide con el `pid` del proceso que recibe la señal `SIGTRAP`. Los posibles valores de `request` son:

- 0 Esta petición debe ser usada sólo por el proceso hijo, si va a ser depurado por su padre. Activa el bit de traza que indica que el proceso debe ponerse en un estado de espera cuando reciba una señal. Recordemos que la recepción de una señal implicaba tres posibles acciones programables con `signal`: ignorar la señal, invocar a la rutina de tratamiento por defecto o invocar a una rutina de tratamiento suministrada por el usuario. Hay una cuarta acción, que es la descrita en el caso de que el bit de traza esté activo.

Los parámetros `pid`, `addr` y `data` son ignorados.

Los valores de `request` que vamos a describir a continuación sólo tienen validez cuando la llamada a `ptrace` la realiza el proceso padre —depurador—, y `pid` es el identificador de alguno de sus hijos.

- 1, 2 La palabra que se encuentra en la dirección `addr` del espacio de direcciones virtuales del hijo es devuelta al proceso padre. Si el código y los datos se encuentran en espacios separados, la petición 1 devuelve una palabra del espacio de código, y la petición 2, del espacio de datos. Si estos espacios no están separados, se pueden

utilizar ambas peticiones indistintamente. La llamada puede fallar si **addr** no es la dirección inicial de una palabra —problema de alineamiento—, en este caso, la función devuelve **-1** y **errno** contendrá el código **EIO**.

El parámetro **data** es ignorado.

- 3 La palabra que se encuentra en la dirección **addr** del área de usuario del hijo es devuelta al proceso padre. La estructura de esta zona de memoria se define en el fichero de cabecera `<sys/user.h>`. La llamada puede fallar si **addr** no es la dirección inicial de una palabra —problema de alineamiento—, en este caso, la función devuelve **-1** y **errno** contendrá el código **EIO**.

El parámetro **data** es ignorado.

- 4, 5 El valor que contiene el parámetro **data** es escrito en la dirección **addr** del espacio de direcciones virtuales del hijo. Si el código y los datos se encuentran en espacios separados, la petición 4 escribirá en el espacio de código, y la petición 5, en el espacio de datos. Si estos espacios no están separados, se pueden utilizar ambas peticiones indistintamente. Si la llamada funciona correctamente, la función devuelve el valor de **data**. La llamada puede fallar si **addr** no es la dirección inicial de una palabra —problema de alineamiento—, en este caso, la función devuelve **-1** y **errno** contendrá el código **EIO**.
- 6 Con esta petición se pueden modificar algunos de los campos del área de usuario, **data** aporta el valor a escribir y **addr** la dirección sobre la que se escribe. Los campos modificables son:
 - Los registros de propósito general.
 - Algunos registros de estado en coma flotante.
 - Algunos bits de estado del procesador.
- 7 Esta petición causa que el proceso hijo continúe con su ejecución. Si el argumento **data** vale 0, todas las señales pendientes de tratamiento, incluyendo la que causó que el proceso parara, son canceladas antes de continuar; **addr** debe valer 1 para esta petición. Si la llamada se ejecuta satisfactoriamente, devuelve el valor de **data**, en caso de fallo, devuelve **-1** y **errno** toma el valor **EIO**.
- 8 Esta petición causa que el proceso hijo termine de igual forma que si hiciera una llamada a **exit**.
- 9 La ejecución continúa como con la petición 7. Sin embargo, en la medida de lo posible, el proceso hijo va a recibir una señal **SIGTRAP** tras cada ejecución de una instrucción máquina. Esto es lo que se conoce como ejecución paso a paso y sólo será posible si el hardware del ordenador lo contempla.

Para manejar estos números de una forma independiente de la implementación de **ptrace**, algunos sistemas definen unas constantes simbólicas en el fichero de cabecera `<sys/ptrace.h>`:

```

#define PT_SETTRC 0
#define PT_RIUSER 1
#define PT_RDUSER 2
#define PT_RUAREA 3
#define PT_WIUSER 4
#define PT_WDUSER 5
#define PT_WUAREA 6
#define PT_CONTIN 7
#define PT_EXIT 8
#define PT_SINGLE 9

```

Como ejemplo de aplicación de la llamada `ptrace` y de las técnicas de depuración, veremos un programa que implementa las posibilidades básicas de un depurador de bajo nivel. Implementar un depurador no es algo sencillo, y menos aún si pensamos en un depurador de alto nivel para código escrito en C. Por ello este programa debe verse más como un ejemplo didáctico de uso de `ptrace` que como un intento serio de implementación de un depurador. A pesar de la modestia del programa, hay que decir que los depuradores comerciales de UNIX tienen un esquema de funcionamiento muy similar.

El depurador del ejemplo tiene funciones simples, como visualizar y modificar el contenido de una dirección de memoria, ejecutar paso a paso, ejecutar órdenes del sistema operativo y un pequeño texto de ayuda. Su código es el siguiente:

Programa 8.3: Depurador basado en `ptrace` (`dep.c`)

```

/****

```

Este programa implementa el núcleo de un depurador de bajo nivel. Las funciones que puede llevar a cabo son muy elementales:

- Ejecutar paso a paso.
- Examinar una zona de memoria.
- Modificar una zona de memoria.

Puesto que trabajar con la tabla de símbolos que genera el compilador no es sencillo, este depurador elemental no trabaja con los nombres simbólicos de las variables. Para examinar o modificar el valor de una variable es necesario conocer su dirección en el espacio de direcciones virtuales.

Forma de uso:

```
$ dep [nombre_programa]
```

Si no se especifica ningún programa, se intenta abrir `a.out`.

Una vez arrancado el depurador, podemos pedir ayuda escribiendo el carácter `?`.

```

****/

```

```

#include <stdio.h>
#include <signal.h>
#include <sys/types.h>

```

```

/* Constantes empleadas en la llamada a ptrace. */

```

```

#define PT_SETTRC 0
#define PT_RDUSER 2
#define PT_WDUSER 5
#define PT_CONTIN 7
#define PT_EXIT 8
#define PT_SINGLE 9

/* Texto de la ayuda. */
char *ayuda[] = {
    "\n",
    "OPCIONES POSIBLES\n",
    "p dirección\tMuestra el contenido de dirección\n",
    "s dirección valor\tEscribe valor en dirección\n",
    "t [#]\t\tEjecuta # instrucciones máquina\n",
    "c\t\tContinúa la ejecución del programa a depurar\n",
    "!\orden\t\tEjecuta una orden del sistema operativo\n",
    "q\t\tSalir del depurador\n",
    "?\t\tMuestra el menú de ayuda\n",
    "\n"
};

#define MAX_AYUDA (sizeof (ayuda) / sizeof (char *))

#define EQ(str1, str2) (strcmp ((str1), (str2)) == 0)

/**
    main: su flujo de control se puede esquematizar como sigue:
        1. Crea un proceso hijo, proceso a depurar.
        2. Espera a que el proceso hijo reciba la señal SIGTRAP.
        3. Lee una orden del flujo estándar de entrada.
        4. Se comunica con el proceso hijo a través de la llamada ptrace.
        5. Presenta los resultados devueltos por el proceso hijo.
        6. Repite desde el punto 2.
***/
***/
main (int argc, char *argv [])
{
    int pid;
    char prog [256];

    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc < 2)
        strcpy (prog, "a.out");
    else
        strcpy (prog, argv[1]);

    /* Creación del proceso hijo, proceso a depurar. */
    if ((pid = fork ()) == 0) {

```

```

/* Código del proceso hijo. */
/* Activación del bit de traza en el proceso hijo. */
ptrace (PT_SETTRC);
/* Cargamos el código del programa a depurar. Al terminar la ejecución de
exec, el núcleo le envía al proceso la señal SIGTRAP. */
execvp (prog, &argv[1], 0);
/* En caso de que falle exec. */
perror (prog);
exit (127);
}

/* Proceso padre. Se encarga de depurar. */
printf ("Proceso hijo: %d\n", pid);
for (;;) {
    char str_ant [256], str_act [256], orden;
    int total, valor, direc, i, estado;
    enum {NO, SI} repetir;

    /* Esperamos a que el proceso hijo reciba la señal SIGTRAP. */
    wait (NULL);

    /* El proceso padre lee de la entrada estándar una orden. */
    leer_orden:
    do {
        printf ("PID(%d) ? ==> ayuda< ", getpid());
        fflush (stdout);
        //gets (str_act);
        fgets (str_act, sizeof (str_act), stdin);
        /* Si se introduce como orden una línea vacía, se ejecutará la última
orden introducida. */
        if (EQ(str_act, "\n")) {
            strcpy (str_act, str_ant);
            repetir = SI;
        } else {
            strcpy (str_ant, str_act);
            total = sscanf (str_act, "%c %x %x %x",
                           &orden, &direc, &valor);
            repetir = NO;
        }
    } while (total <= 0);

    /* Análisis y ejecución de la orden introducida. */
    switch (orden) {

        /* Caso de consultar una zona de memoria del área de datos del
proceso hijo. */

```

```

case 'p':
    if (repetir)
        ++direc;
    else if (total != 2) {
        printf ("%s] incorrecto.\n", str_act);
        goto leer_orden;
    }
    if ((valor = ptrace (PT_RDUSER, pid, direc)) == -1)
        printf ("Dirección 0x%x incorrecta.\n", direc);
    else
        printf "(0x%x) = 0x%x = %d\n", direc, valor, valor);
    goto leer_orden;

/* Caso de modificar una zona de memoria del área de datos del
proceso hijo. */
case 's':
    if (total != 3) {
        printf ("%s] incorrecto.\n", str_act);
        goto leer_orden;
    }
    if (ptrace (PT_WDUSER, pid, direc, valor) != valor)
        printf ("Dirección 0x%x incorrecta.\n", direc);
    else
        printf "(0x%x) = 0x%x = %d\n", direc, valor, valor);
    goto leer_orden;

/* Activar el indicador de ejecución paso a paso. Como consecuencia, el
proceso hijo recibirá la señal SIGTRAP después de ejecutar una instrucción
máquina. */
case 't':
    if (total == 1)
        valor = 1;
    else
        valor = direc;
    for (i = 0; i < valor; i++)
        if (ptrace (PT_SINGLE, pid, 1, 0) == -1)
            perror (argv [0]);
    break;

/* Continuar la ejecución del proceso hijo hasta la finalización del mismo.
Una vez que haya terminado, el proceso hijo deja de existir. */
case 'c':
    if (ptrace (PT_CONTIN, pid, 1, 0) == -1)
        perror (argv [0]);
    break;

```

```

/* Ejecutar una orden del sistema operativo sin necesidad de salir del
programa. */
case '!':
    system (str_act+1);
    goto leer_orden;

/* Salir del programa de depuración. */
case 'q':
    if (kill (pid, 0) == -1)
        exit (0); /* Caso de que el proceso hijo no exista. */
    if (ptrace (PT_EXIT, pid) == -1)
        perror (argv[0]);
    exit (0);

/* Presenta el texto de ayuda. */
case '?':
    for (valor = 0; valor < MAX_AYUDA; valor++)
        printf (ayuda [valor]);
    goto leer_orden;

/* Tratamiento de las opciones erróneas. */
default:
    printf ("%s incorrecto\n", str_act);
    goto leer_orden;
}
}
}

```

Para mostrar la forma de manejo del programa anterior nos valdremos de un pequeño programa auxiliar que hará las funciones de programa a depurar. Su código es el siguiente:

Programa 8.4: Programa que depuraremos con dep (traza.c)

```

/**
Programa que utilizaremos para mostrar el uso del depurador. Los puntos de
parada son aquellos en los que el proceso recibe la señal SIGTRAP.
***/
#include <signal.h>

int i;

main ()
{
    int pid = getpid ();

    printf ("pid(%d) &i = 0x%x\n", pid, (int)&i);
    printf ("pid(%d) i = 0x%x\n", pid, i);
}

```



```

    i = 100;

    kill(getpid (), SIGTRAP); /* Punto de parada. */
    printf ("pid(%d) i = 0x%x\n", pid, i);
    kill(getpid (), SIGTRAP); /* Punto de parada. */
}

```

El resultado de la ejecución del programa **traza** es el siguiente:

\$ traza

```

pid(1344) &i = 0x11d0
pid(1344) i = 0x0
Trace/BPT trap(coredump)

```

Se puede apreciar que el programa deja de ejecutarse cuando recibe la señal **SIGTRAP**. Ahora veremos una sesión de trabajo en la que **traza** se ejecuta bajo el control del depurador **dep**:

\$ dep traza

Proceso hijo: 1346	<i>Pid del proceso hijo.</i>
PID(1345) ? ==> ayuda< c	<i>Orden de continuar con la ejecución.</i>
pid(1346) &i = 0x11d0	<i>Salida que produce el proceso hijo.</i>
pid(1346) i = 0x0	<i>Salida que produce el proceso hijo.</i>
PID(1345) ? ==> ayuda< p 11d0	<i>Orden para visualizar el contenido de i (dirección 0x11d0)</i>
(0x11d0) = 0x64 = 100	<i>Valor que almacena i, visualizado por el depurador.</i>
PID(1345) ? ==> ayuda< s 11d0 ffaa	<i>Orden para modificar el contenido de la variable i.</i>
(0x11d0) = 0xffaa = 65450	<i>Nuevo valor de i.</i>
PID(1345) ? ==> ayuda< c	<i>Orden para continuar con la ejecución.</i>
pid(1346) i = 0xffaa	<i>Salida que produce el proceso hijo, i contiene el valor modificado anteriormente.</i>
PID(1345) ? ==> ayuda< q	<i>Orden para salir del depurador.</i>

Parte III

Comunicación entre procesos

9	Comunicación mediante tuberías	333
10	Comunicación local entre procesos e hilos	369
11	Comunicaciones en red	423

Capítulo 9

Comunicación mediante tuberías

«Mas al darle de beber, no fue posible, ni lo fuera si el ventero no horadara una caña, y, puesto el un cabo en la boca, por el otro le iba echando el vino...»

(Cervantes, Quijote I-2)

9.1	Comunicación entre procesos	333
9.2	Tuberías sin nombre	334
9.3	Comunicación bidireccional	337
9.4	Tuberías en los intérpretes de órdenes	340
9.5	Tuberías con nombre	342
9.6	Comunicación dúplex	350
9.7	Ejercicios	359

9.1 Comunicación entre procesos

La comunicación entre procesos habilita mecanismos para que los procesos puedan intercambiarse datos y sincronizarse. Hasta ahora hemos visto tres formas bastante elementales para que dos procesos se comuniquen: el envío de señales para la sincronización, el uso de ficheros ordinarios para el intercambio de datos y la llamada a `ptrace` para que un proceso padre pueda manipular el espacio de direcciones virtuales de su hijo. Las señales no deben considerarse parte de la forma habitual de comunicar dos procesos y su uso debe restringirse a la comunicación de eventos o situaciones excepcionales. Los ficheros ordinarios tampoco son la forma más eficiente de comunicarse, ya que se ve involucrado el acceso a disco que suele ser 3 o 4 órdenes de magnitud más lento que el acceso a memoria. Por último, la comunicación a través de `ptrace` es unidireccional, sólo el padre puede leer datos del hijo, y su uso sólo tiene interés a la hora de implementar programas depuradores.

Los mecanismos que vamos a estudiar a continuación pretenden dar soluciones más eficientes a las necesidades de comunicación empleando como canal de transmisión la memoria principal. Esto supondrá una mayor velocidad de transferencia de datos.

A la hora de comunicar dos procesos, vamos a considerar dos situaciones claramente diferentes:

- Que los procesos se estén ejecutando bajo el control de una misma máquina.
- Que los procesos se estén ejecutando en máquinas separadas.

La primera situación no aporta especial dificultad, ya que el discurso que hemos venido realizando hasta ahora sobre UNIX es en esa línea. Así, para comunicar dos o más procesos a nivel local, vamos a estudiar un mecanismo clásico como son las tuberías, y después pasaremos a ver las facilidades *IPC* del UNIX System V. Estas facilidades están diseñadas con una misma filosofía y engloban tres mecanismos de comunicación: semáforos, memoria compartida y colas de mensajes.

El segundo escenario es más complejo, porque se ven involucradas las redes de ordenadores y la comunicación entre ellos. Hoy en día estos temas tienen mucho interés, porque el proceso distribuido se está convirtiendo en una tecnología con gran aceptación. La idea de tener varias máquinas interconectadas y que cualquier usuario pueda manejarlas como si se tratasen de una sola, pero con una potencia cercana a la suma de todas ellas, no sólo es interesante como ejercicio teórico, sino que ya tiene implementación real. El estudio de las redes de ordenadores está fuera de los objetivos de este libro. El lector interesado puede encontrar una exposición elegante, estructurada y detallada del mismo en [Tanenbaum, 2002].

Como introducción al tema de la comunicación entre máquinas remotas, vamos a exponer la interfaz que suministra el UNIX de Berkeley para comunicar procesos. Esta interfaz se compone de una serie de llamadas que manejan un nuevo tipo de fichero conocido como *conector* o *socket* y que actuará como canal de comunicación entre procesos. Aunque un conector es tratado sintácticamente como un fichero, semánticamente no lo es. Esto significa que no vamos a tener los problemas de velocidad inherentes al acceso a disco.

Las tuberías son una de las primeras formas de comunicación implantadas en UNIX y muchos sistemas se ofrecen hoy día con esta facilidad. Incluso sistemas monoproceso como DOS ofrecen posibilidad de montar tuberías desde el punto de vista del intérprete de órdenes.

Una tubería se puede considerar como un canal de comunicación entre dos procesos, y las hay de dos tipos: *tuberías con nombre* —*FIFOS*— y *tuberías sin nombre*.

9.2 Tuberías sin nombre

Las tuberías sin nombre se crean con la llamada `pipe` y sólo el proceso que hace la llamada y sus descendientes pueden utilizarla; `pipe` tiene la siguiente declaración:

```
#include <unistd.h>
int pipe (int fildes [2]);
```

Si la llamada funciona correctamente, devolverá el valor 0 y creará una tubería sin nombre; en caso contrario, devolverá -1 y en `errno` estará el código del error producido.

La tubería creada podrá ser manejada a través del array **fildes**. Los dos elementos de **fildes** se comportan como dos descriptors de fichero y los vamos a usar para escribir en la tubería y leer de ella. Al escribir en **fildes[1]** estamos escribiendo datos en la tubería, y al leer de **fildes[0]** extraeremos datos de ella. Naturalmente, **fildes[1]** se comporta como un fichero de sólo escritura y **fildes[0]** como un fichero de sólo lectura.

Como el núcleo trata la tubería igual que a un fichero del sistema, al crearla debe asignarle un nodo-i. También le asigna un par de descriptors de fichero —**fildes[0]** y **fildes[1]**— y reserva las correspondientes entradas en la tabla de ficheros del sistema y en la tabla de descriptors del proceso.

Todo esto facilita el manejo de la tubería, ya que al recibir el mismo tratamiento que un fichero, podremos leer y escribir en ella con las llamadas **read** y **write** que empleamos para los ficheros ordinarios, los directorios y los ficheros especiales.

Los descriptors de fichero se heredan de padres a hijos tras la llamada a **fork** o a **exec**. Así, para que se comuniquen padre e hijo mediante una tubería, la abriremos en el padre y tanto padre como hijo podrán compartirla.

La sincronización entre los accesos de escritura y lectura la lleva a cabo el núcleo, de tal manera que las llamadas a **read** para sacar datos de la tubería no devolverán el control hasta que no haya datos escritos por otro proceso mediante la correspondiente llamada a **write**. También es el núcleo el encargado de gestionar la tubería para dotarla de una disciplina de acceso en hilera y, así, el proceso receptor sacará los datos en el mismo orden en que los escriba el proceso emisor.

Los datos escritos en la tubería se gestionan en la memoria intermedia sin que lleguen al disco, por lo que al producirse la transferencia a través de memoria, las tuberías constituyen un mecanismo de comunicación mucho más rápido que el uso de ficheros ordinarios. El tamaño de una tubería, es decir, el bloque de datos más grande que podemos escribir en ella, depende del sistema, pero se garantiza que será inferior a 4.096 bytes.

Cuando la tubería está llena, las llamadas a **write** quedan bloqueadas hasta que no se saquen suficientes datos de la tubería como para escribir el bloque deseado.

Como ejemplo de aplicación, veamos la forma de enviar datos desde un proceso emisor a un proceso receptor a través de una tubería sin nombre —en la figura 9.1 se muestra la estructura de este programa—; este ejemplo es el mismo que vimos para ilustrar la sincronización de dos procesos mediante señales —consulte el § 7.6.2 pág. 278—; ahora veremos que sólo tenemos que preocuparnos por los aspectos de envío y recepción, ya que la sincronización es algo que resuelve el núcleo.

Programa 9.1: Envío de mensajes mediante tuberías sin nombre (pipe-1.c)

```
#include <stdio.h>
#define MAX 256
main ()
{
    int tuberia[2];
    int pid;
    char mensaje [MAX];

    /* Creación de la tubería sin nombre. */
    if (pipe (tuberia) == -1) {
```

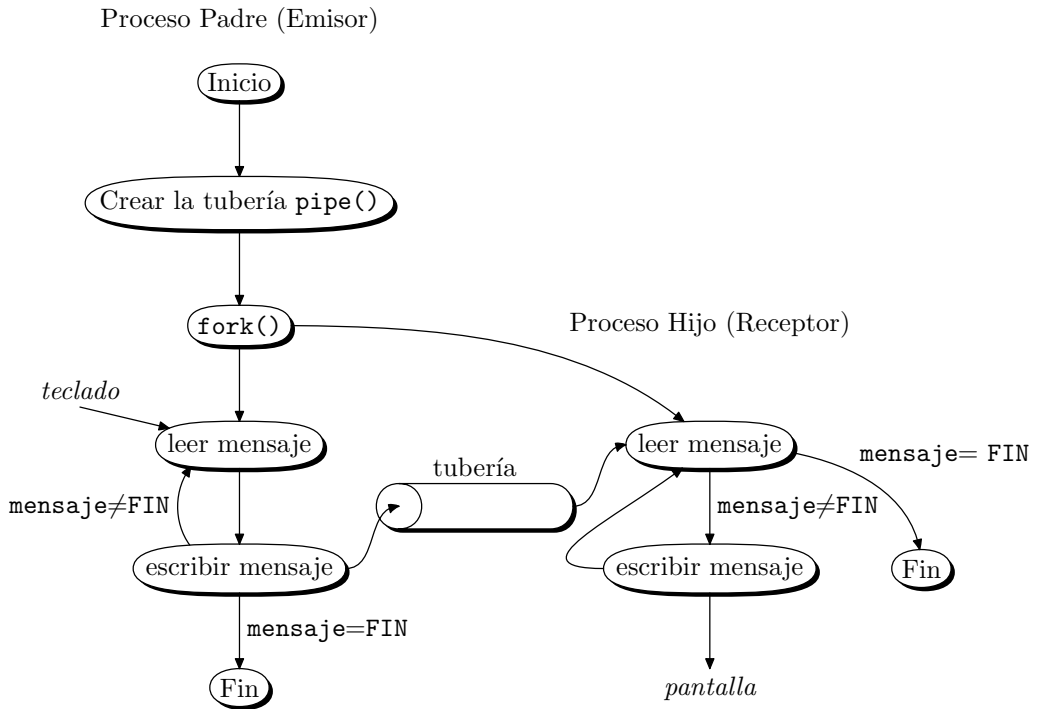


Figura 9.1: Primer ejemplo de comunicación a través de tuberías.

```

    perror ("pipe");
    exit (-1);
}
/* Creación del proceso hijo. */
if ((pid = fork ()) == -1) {
    perror ("fork");
    exit (-1);
} else if (pid == 0) {
    /* Código del proceso hijo. */
    /* El proceso hijo (receptor) se encarga de leer mensajes de la tubería y
    presentarlos en pantalla. El ciclo de lectura y presentación termina al
    leer el mensaje "FIN\n". */
    while (read (tuberia [0], mensaje, MAX) > 0 &&
           strcmp (mensaje, "FIN\n") != 0)
        printf ("PROCESO RECEPTOR. MENSAJE: %s\n", mensaje);
    close (tuberia [0]);
    close (tuberia [1]);
    exit (0);
} else {

```

```

/* Código del proceso padre. */
/* El proceso padre (emisor) se encarga de leer mensajes de la
entrada estándar y, acto seguido, escribirlos en la tubería para
que los reciba el proceso hijo. El ciclo de lectura de la entrada
estándar y escritura en la tubería terminará cuando se introduzca
el mensaje "FIN\n". */
while (printf ("PROCESO EMISOR. MENSAJE: ") != 0 &&
       fgets (mensaje, sizeof (mensaje), stdin) != NULL &&
       write (tuberia [1], mensaje, strlen(mensaje) + 1) > 0 &&
       strcmp (mensaje, "FIN\n") != 0);
close (tuberia [0]);
close (tuberia [1]);
exit (0);
}
}

```

9.3 Comunicación bidireccional

Uno de los problemas que presenta el programa del apartado anterior es la falta de comunicación entre el proceso hijo y el padre, de tal forma que el proceso emisor puede pedirnos que introduzcamos más mensajes antes de que el proceso receptor haya presentado el mensaje que acabamos de enviarle. Para solucionar este problema tenemos que implementar algún tipo de protocolo entre los procesos.

Para implementar esta comunicación necesitamos otra tubería que sirva de canal entre el proceso receptor y el emisor. Podríamos sentirnos tentados a aprovechar una sola tubería como canal bidireccional, pero esto plantea problemas de sincronismo y tendríamos que ayudarnos de señales o semáforos para controlar el acceso a la tubería. En efecto, si un proceso escribe en la tubería un mensaje para otro proceso y se pone a leer de ella la respuesta que le envía éste, puede darse el caso de que lea el mensaje que él mismo envió.

Lo mejor es valernos de dos tuberías —como muestra la figura 9.2—, una lleva los mensajes que van del proceso *A* al proceso *B*, y la otra lleva los mensajes en sentido contrario.

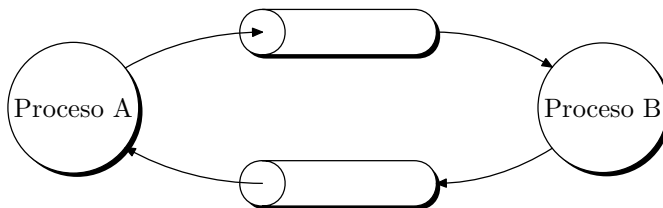


Figura 9.2: Comunicación bidireccional usando dos tuberías.

A continuación podemos ver el código correspondiente al mismo ejemplo del apartado anterior, pero con un protocolo entre los procesos emisor y receptor, de tal forma que el

proceso emisor no efectuará el envío de otro mensaje hasta que el receptor haya procesado totalmente el último mensaje que recibió —consulte la figura 9.3—.

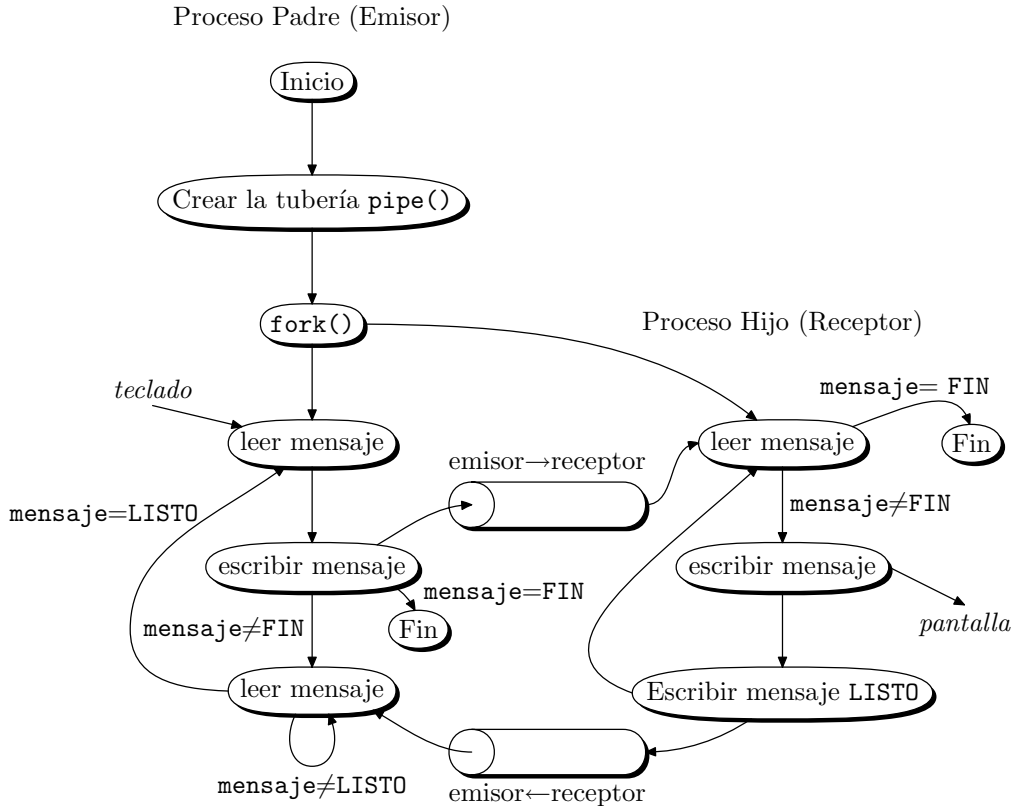


Figura 9.3: Comunicación bidireccional con protocolo.

Programa 9.2: Envío de mensajes entre dos procesos mediante dos tuberías sin nombre (pipe-2.c)

/**

Programa para ilustrar el envío de mensajes entre un proceso emisor y otro receptor a través de dos tuberías sin nombre. Estas tuberías permitirán implementar una comunicación bidireccional. El proceso emisor pedirá un mensaje que le enviará al proceso receptor. Cuando el proceso receptor haya presentado el mensaje por pantalla, solicitará al proceso emisor otro mensaje, indicándole así que está listo y que puede pedirle otro mensaje al usuario.

*/

#include <stdio.h>

#define MAX 256

main ()

{

```
int tuberia_em_re[2];
int tuberia_re_em[2];
int pid;
char mensaje [MAX];

/* Creación de las tuberías de comunicación. */
if (pipe (tuberia_em_re) == -1 || pipe (tuberia_re_em) == -1) {
    perror ("pipe");
    exit (-1);
}

/* Creación del proceso hijo. */
if ((pid = fork ()) == -1) {
    perror ("fork");
    exit (-1);
} else if (pid == 0) { /* Código del proceso hijo. */
    /* El proceso hijo (receptor) se encarga de leer un mensaje de la tubería y
    presentarlo en la pantalla. Al recibir el mensaje "FIN\n", termina el
    proceso. */
    while (read (tuberia_em_re [0], mensaje, MAX) > 0 &&
        strcmp (mensaje, "FIN\n") != 0) {
        printf ("PROCESO RECEPTOR. MENSAJE: %s\n", mensaje);
        /* Enviamos al proceso emisor un mensaje para indicar que estamos
        listos para recibir otro mensaje. */
        strcpy (mensaje, "LISTO");
        write (tuberia_re_em [1], mensaje, strlen(mensaje) + 1);
    }
    close (tuberia_em_re [0]);
    close (tuberia_em_re [1]);
    close (tuberia_re_em [0]);
    close (tuberia_re_em [1]);
    exit (0);
} else { /* Código del proceso padre. */
    /* El proceso padre (emisor) se encarga de leer un mensaje de la entrada
    estándar y acto seguido escribirlo en la tubería, para que lo reciba el
    proceso hijo. Al escribir el mensaje "FIN\n" acaban los dos procesos. */
    while (printf ("PROCESO EMISOR. MENSAJE: ") != 0 &&
        fgets (mensaje, sizeof (mensaje), stdin) != NULL &&
        write(tuberia_em_re[1], mensaje, strlen(mensaje)+1) > 0 &&
        strcmp (mensaje, "FIN\n") != 0)
        /* Nos ponemos a esperar el mensaje "LISTO" procedente del proceso
        receptor. */
        do {
            read (tuberia_re_em [0], mensaje, MAX);
        } while (strcmp (mensaje, "LISTO") != 0);
    close (tuberia_em_re [0]);
}
```

```

        close (tuberia_em_re [1]);
        close (tuberia_re_em [0]);
        close (tuberia_re_em [1]);
        exit (0);
    }
}

```

9.4 Tuberías en los intérpretes de órdenes

Hoy día, casi todos los intérpretes de órdenes ofrecen la posibilidad de redirigir los ficheros estándar —de salida y de entrada— asociados a un proceso.

Normalmente, los programas utilizan los ficheros estándar para efectuar la entrada/salida. Estos mismos programas pueden servirnos para trabajar con otros ficheros sin necesidad de modificar su código, pero para ello el intérprete de órdenes que nos comunica con el sistema operativo debe contemplar la redirección. La redirección hacia otros ficheros se le indica al intérprete mediante los caracteres `<` y `>` —redirección de la entrada y la salida, respectivamente—.

Otra forma de redirigir es mediante las tuberías. Con ellas, lo que conseguimos es que la salida de un programa se convierta en entrada para otro. Esto es importante a la hora de aprovechar programas estándar, que realizan funciones sencillas, para construir otros que realizan funciones más complejas.

Como ejemplo, veamos la forma de presentar en pantalla el listado de un directorio en orden alfabético inverso. Sabemos que la orden `ls` muestra por la salida estándar —pantalla— el contenido de un directorio. Sabemos también que la orden `sort -r` lee líneas de la entrada estándar y las muestra por la salida estándar en orden alfabético inverso. Mediante una tubería lograremos que la salida de `ls` se redirija hacia la entrada de `sort` y el resultado será un listado del directorio en el orden deseado. La orden a ejecutar es

```
$ ls | sort -r
```

Tanto `ls` como `sort` son dos programas que se ejecutan concurrentemente y que se comunican a través de una tubería sin nombre. Para ilustrar cómo se realiza esto veremos un programa que codifica la parte de un intérprete de órdenes encargada de crear tuberías y comunicar procesos a través de ellas —consulte la figura 9.4—; este programa se llama `tuberia` y la forma de invocarlo es,

```
$ tuberia programa_1 programa_2
```

El resultado de la ejecución de la línea anterior es el mismo que el de la línea,

```
$ programa_1 | programa_2
```

El código del programa `tuberia` es el siguiente:

Programa 9.3: Uso de las tuberías por parte del intérprete de órdenes (`tuberia.c`)

```
/**
```

```
Forma de uso: $ tuberia programa_1 programa_2
```

```
Esto es equivalente a: $ programa_1 | programa_2
```

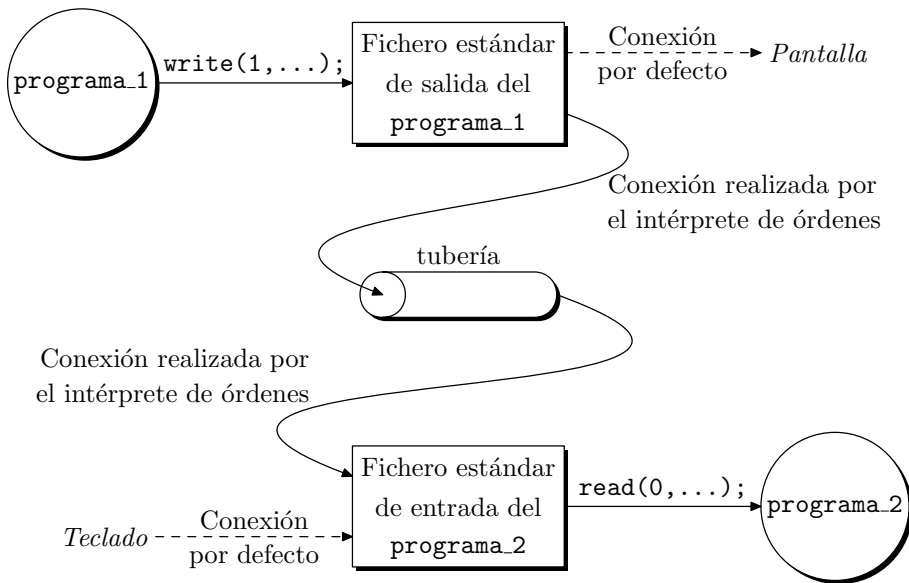


Figura 9.4: Uso de tuberías para redirigir los ficheros estándar de entrada y salida.

```

***/
#include <stdio.h>
main (int argc, char *argv[])
{
    int tuberia[2];
    int pid;
    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc < 3) {
        fprintf (stderr,
            "Forma de uso: %s programa_1 programa_2\n",argv[0]);
        exit (-1);
    }
    /* Creación de la tubería. */
    if (pipe (tuberia) == -1) {
        perror (argv[0]);
        exit (-1);
    }

    /* Creación de los procesos padre e hijo. */
    if ((pid = fork ()) == -1) {
        perror (argv[0]);
        exit (-1);
    } else if (pid == 0) { /* Código del proceso hijo. */

```

```

    close (0);
    dup (tuberia [0]);
    /* Al duplicar el descriptor asociado a tuberia[0], lo vamos a hacer sobre
    la primera entrada que haya libre en la tabla de descriptors de ficheros.
    Esta entrada es la número 0, ya que antes hemos cerrado el fichero de
    entrada (descriptor 0). Así, para el proceso hijo, el fichero estándar de
    entrada será el fichero asociado a la lectura de la tubería, y todas las
    funciones que trabajen con el fichero estándar de entrada en realidad van
    a trabajar con el fichero de lectura de la tubería. */
    close (tuberia [0]);
    close (tuberia [1]);

    /* Ejecución de programa_2. */
    execlp (argv [2], argv[2], 0);
    /* En la llamada a exec se heredan los descriptors asignados, por lo que
    el programa referenciado en argv[2] se ejecutará teniendo al fichero de
    lectura de la tubería como fichero estándar de entrada. */
} else { /* Código del proceso padre. */
    close (1);
    dup (tuberia [1]);
    close (tuberia [0]);
    close (tuberia [1]);

    /* Ejecución de programa_1. */
    execlp (argv [1], argv[1], 0);
}
exit (0);
}

```

Para que un programa pueda formar parte de una tubería tal y como las manejan los intérpretes de órdenes, ha de cumplir dos requisitos:

1. La entrada de datos al programa se debe realizar a través del fichero estándar de entrada —descriptor de fichero número 0—.
2. La salida de datos se debe realizar a través del fichero estándar de salida —descriptor de fichero número 1—.

9.5 Tuberías con nombre

Por medio de las tuberías sin nombre podemos comunicar procesos relacionados entre sí ya que el proceso que crea la tubería y sus descendientes tienen acceso a la misma. Para los procesos que no guardan ninguna relación de parentesco, no sirven los canales abiertos mediante tuberías sin nombre. Para comunicar este tipo de procesos tenemos que recurrir a las tuberías con nombre.

Una *tubería con nombre* es un fichero con una semántica idéntica a la de una tubería sin nombre, pero ocupa una entrada en un directorio y se accede a él a través de una ruta.

Un proceso puede abrir una tubería con nombre mediante una llamada a `open`, de la misma forma que abre un fichero ordinario. Así, para comunicar dos procesos mediante una tubería con nombre, uno de ellos debe abrir la tubería para escribir en ella y el otro para leer. La llamada a `open` tiene un comportamiento ligeramente distinto, según se trate de abrir una tubería con nombre o un fichero ordinario. Así, cuando un proceso abre una tubería con nombre para escribir en ella, se pone a dormir hasta que no haya otro proceso que la abra para leer de ella. Cuando es el proceso lector el primero en abrir la tubería, se pone a dormir hasta que algún proceso la abre para escribir. Esto tiene sentido, ya que no valdría para nada escribir en la tubería cuando nadie va a recoger esos datos.

Otra diferencia que hay entre las tuberías con nombre y los ficheros ordinarios es que, para las primeras, el núcleo sólo emplea los bloques directos de direcciones de su nodo-i, por lo que la cantidad total de bytes que se pueden enviar a una tubería con nombre en una sola operación de escritura está limitada.

Para controlar los accesos de escritura y lectura de la tubería, el núcleo emplea dos punteros. Los bloques directos de direcciones del nodo-i son manejados como si fuesen nodos de una cola circular, de tal forma que cuando el puntero de escritura llega al último de los bloques, empieza por el primero, y lo mismo ocurre para el puntero de lectura. Ambos punteros son gestionados de tal forma que el acceso a la tubería es del tipo *primero en entrar, primero en salir*, al igual que ocurría con las tuberías sin nombre —consulte la figura 9.5—.

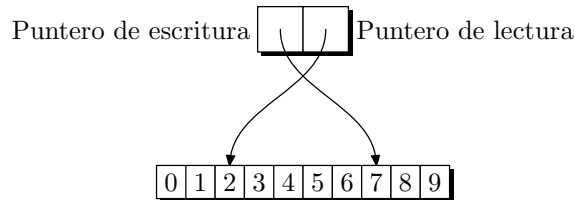


Figura 9.5: Lectura y escritura en una tubería con nombre.

Para poder abrir una tubería con nombre, ésta debe existir. Hay dos formas de crearla: desde la línea de órdenes, mediante una llamada a `mknod(1M)`, y desde programa, mediante una llamada a `mknod(2)`.

Para crear en nuestro directorio de trabajo actual una tubería de nombre `fifo_1`, usando `mknod(1M)`, debemos escribir:

```
$ mknod fifo_1 p
```

Para crear esa misma tubería con la llamada `mknod(2)`, debemos incluir en nuestro programa unas líneas parecidas a las siguientes:

```
if (mknod ("fifo_1", S_IFIFO | permisos, 0) == -1) {
    // Fallo al crear la tubería.
    // Tratamiento del fallo
}
```

donde `permisos` es la máscara, ya conocida, de permisos asociados a un fichero.

En algunos sistemas, por ejemplo 4.4BSD, la ejecución de la orden `mknod` requiere privilegios de superusuario. En estos sistemas se dispone de la orden `mkfifo` que le permite a los usuarios sin estos privilegios crear tuberías con nombre. La forma de invocar esta orden es:

```
$ mkfifo [-m mode] fifo_name
```

donde `mode` es la máscara de modo que codifica los permisos habituales de *lectura-escritura-ejecución* de la tubería y `fifo_name` es el nombre de la tubería.

De igual forma, allí donde se requieran privilegios de superusuario para ejecutar la llamada `mknod`, se dispone de la llamada `mkfifo` que le permite a los usuarios sin privilegios crear tuberías con nombre. La declaración de esta llamada es:

```
#include <sys/types.h>
#include <sys/stat.h>
int mkfifo (char *path, mode_t mode);
```

donde `path` es la ruta de la tubería con nombre que se va a crear y `mode` es la máscara de modo que codifica los permisos de *lectura-escritura-ejecución* de la tubería. La llamada devuelve el valor 0 o el valor -1 dependiendo de que se ejecute correctamente o no. Una posible causa de fallo es el intento de crear una tubería que ya existe. En este caso, el fallo de la llamada no significa que haya un error en nuestro programa, sólo significa que la tubería ya existe y por lo tanto no puede ser creada de nuevo; dependiendo del programa que estemos escribiendo, ante esta situación podemos proceder con normalidad pasando a abrir la tubería para establecer la comunicación y continuar con la ejecución normal del programa. En caso de que la llamada falle porque la tubería ya existe, `mkfifo` devuelve -1 y `errno` toma el valor `EEXIST`.

Como primer ejemplo de aplicación de las tuberías con nombre, vamos a escribir la pareja de programas `llamar-a` y `responder-a`. Estos programas permitirán que dos usuarios se comuniquen mediante el intercambio de mensajes. Supongamos que el usuario `usr1` desea comunicarse con `usr2`, entonces deberá escribir:

```
$ llamar-a usr2
```

y `usr2` recibirá por pantalla el mensaje:

```
LLAMADA PROCEDENTE DEL USUARIO usr1
RESPONDER ESCRIBIENDO: responder-a usr1
```

si `usr2` desea responder, tendrá que escribir:

```
$ responder-a usr1
```

para iniciar la comunicación.

Una vez iniciada la conversación, los dos usuarios se intercambiarán mensajes alternativamente, iniciando el envío `usr1`. Cada mensaje consta de una serie de líneas de texto, finalizando con la línea clave `cambio`. Esta línea servirá para pasarle el turno al otro usuario. Cuando alguno de los usuarios envíe la línea `corto`, la conversación terminará.

La comunicación se llevará a cabo a través de dos tuberías con nombre creadas por el programa `llamar-a` en el directorio `/tmp`. Estas tuberías tendrán los nombres: `/tmp/fifo_usr1_usr2` y `/tmp/fifo_usr2_usr1`. Una de ellas es para los mensajes que

vayan de `usr1` a `usr2`; la otra, para los mensajes que viajen en sentido contrario. El listado del programa `llamar-a` es el siguiente:

Programa 9.4: Simulación de conversaciones por radio (`llamar-a.c`)

```

/****
    El programa exige que la variable de entorno LOGNAME esté correctamente
    inicializada con el nombre del usuario. Esto se puede conseguir incluyendo las dos
    líneas siguientes en el fichero .profile de configuración de nuestro intérprete de
    órdenes:
        LOGNAME='logname'
        export LOGNAME
****/
#include <stdio.h>
#include <string.h>
#include <signal.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include "utmp.h"
#define MAX 256

/* Macro para comparar dos cadenas de caracteres. */
#define EQ(str1, str2) (strcmp ((str1), (str2)) == 0)

/* Descriptores de fichero de las tuberías con nombre mediante las cuales vamos a
comunicarnos. */
int fifo_12, fifo_21;
char nombre_fifo_12[MAX], nombre_fifo_21[MAX];

/* Array para leer los mensajes. */
char mensaje [MAX];

main (int argc, char *argv [])
{
    int tty;
    char terminal [MAX], *logname, *getenv ();
    struct utmp *utmp, *getutent ();
    void fin_de_transmision ();

    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc != 2) {
        fprintf (stderr, "Forma de uso: %s usuario\n", argv [0]);
        exit (-1);
    }

    /* Lectura de nuestro nombre de usuario. */
    logname = getenv ("LOGNAME");

```



```

/* Comprobación para que un usuario no se llame a sí mismo. */
if (EQ (logname, argv [1])) {
    fprintf (stderr, "Comunicación con uno mismo no permitida\n");
    exit (0);
}

/* Consultamos si el usuario ha iniciado una sesión. */
while ((utmp = getutent ()) != NULL &&
    strncmp (utmp->ut_user, argv [1], sizeof (utmp->ut_user)) != 0);
if (utmp == NULL) {
    printf ("EL USUARIO %s NO HA INICIADO SESIÓN.\n", argv [1]);
    exit (0);
}

/* Formación de los nombres de las tuberías de comunicación. */
sprintf (nombre_fifo_12, "/tmp/%s_%s", logname, argv[1]);
sprintf (nombre_fifo_21, "/tmp/%s_%s", argv[1], logname);
/* Creación y apertura de las tuberías. */
/* Primero borramos las tuberías, para que la llamada a mknod no falle. */
unlink (nombre_fifo_12);
unlink (nombre_fifo_21);
/* Cambiamos la máscara de permisos por defecto para este proceso. Esto
permitirá crear las tuberías con los permisos rw-rw-rw-=0666.*/
umask (~0666);
/* Creamos las tuberías para que la llamada a open no falle. */
if (mkfifo (nombre_fifo_12, 0666) == -1) {
    perror (nombre_fifo_12);
    exit (-1);
}
if (mkfifo (nombre_fifo_21, 0666) == -1) {
    perror (nombre_fifo_21);
    exit (-1);
}

/* Apertura del terminal del usuario. */
sprintf (terminal, "/dev/%s", utmp->ut_line);
if ((tty = open (terminal, O_WRONLY)) == -1) {
    perror (terminal);
    exit (-1);
}

/* Aviso al usuario con el que queremos comunicarnos. */
sprintf (mensaje,
    "\n\t\tLLAMADA PROCEDENTE DEL USUARIO %s\07\07\07\n"
    "\t\tRESPONDER ESCRIBIENDO: responder-a %s\n\n",
    logname, logname);

```

```

write (tty, mensaje, strlen (mensaje) + 1);
close (tty);

printf ("Esperando respuesta...\n");
/* Apertura de las tuberías. Una de ellas para escribir mensajes y la otra
para leerlos. */
if ((fifo_12 = open (nombre_fifo_12, O_WRONLY)) == -1 ||
    (fifo_21 = open (nombre_fifo_21, O_RDONLY)) == -1) {
    if (fifo_12 == -1)
        perror (nombre_fifo_12);
    else
        perror (nombre_fifo_21);
    exit (-1);
}
/* A este punto llegamos cuando nuestro interlocutor responde a nuestra
llamada. */
printf ("LLAMADA ATENDIDA. \07\07\07\n");

/* Armamos el manejador de la señal SIGINT. Esta señal se genera al
pulsar Ctrl+C. */
signal (SIGINT, fin_de_transmision);

/* Bucle de envío de mensajes. */
do {
    printf ("<== ");
    fgets (mensaje, sizeof (mensaje), stdin);
    write (fifo_12, mensaje, strlen(mensaje) + 1);
    /* Bucle de recepción de mensajes. */
    if (EQ(mensaje, "cambio\n"))
        do {
            printf ("==> "); fflush (stdout);
            read (fifo_21, mensaje, MAX);
            printf ("%s", mensaje);
        } while (!EQ(mensaje,"cambio\n") && !EQ(mensaje,"corto\n"));
    } while (!EQ(mensaje, "corto\n"));

/* Fin de la transmisión. */
printf ("FIN DE TRANSMISIÓN.\n");
close (fifo_12);
close (fifo_21);

exit (0);
}

/****
fin_de_transmision: rutina de tratamiento de la señal SIGINT. Al pulsar Ctrl+C

```

entramos en esta rutina, que se encarga de enviar el mensaje "corto\n" al usuario con el que estamos hablando.

```

***/
void fin_de_transmision (int sig)
{
    sprintf (mensaje, "corto\n");
    write (fifo_12, mensaje, strlen(mensaje) + 1);
    printf ("FIN DE TRANSMISIÓN.\n");
    close (fifo_12);
    close (fifo_21);
    exit (0);
}

```

En este programa se ha empleado la función `getutent`; para obtener más información sobre el uso y la implementación de esta función consulte el § 4.2.1 pág. 130.

El programa `responder-a` puede quedar como sigue:

Programa 9.5: Simulación de conversaciones por radio (`responder-a.c`)

```

/**/
El programa exige que la variable de entorno LOGNAME esté correctamente
inicializada con el nombre del usuario. Esto se puede conseguir incluyendo las dos
líneas siguientes en el fichero .profile de configuración de nuestro intérprete de
órdenes:
    LOGNAME='logname'
    export LOGNAME
***/
#include <stdio.h>
#include <signal.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#define MAX 256

/* Macro para comparar dos cadenas de caracteres. */
#define EQ(str1, str2) (strcmp ((str1), (str2)) == 0)

/* Descriptores de fichero de las tuberías mediante las que nos vamos a comunicar. */
int fifo_12, fifo_21;
char nombre_fifo_12 [MAX], nombre_fifo_21 [MAX];

/* Array para leer los mensajes. */
char mensaje [MAX];

main (int argc, char *argv [])
{

```

```

char *logname, *getenv ();
void fin_de_transmision ();

/* Análisis de los argumentos de la línea de órdenes. */
if (argc != 2) {
    fprintf (stderr, "Forma de uso: %s usuario\n", argv [0]);
    exit (-1);
}

/* Lectura del nombre del usuario. */
logname = getenv ("LOGNAME");
/* Comprobación para que un usuario no se responda a sí mismo. */
if (EQ (logname, argv [1])) {
    fprintf (stderr, "Comunicación con uno mismo no permitida\n");
    exit (0);
}

/* Formación del nombre de las tuberías de comunicación. */
sprintf (nombre_fifo_12, "/tmp/%s_%s", argv[1], logname);
sprintf (nombre_fifo_21, "/tmp/%s_%s", logname, argv[1]);

/* Apertura de las tuberías. */
if ((fifo_12 = open (nombre_fifo_12, O_RDONLY)) == -1 ||
    (fifo_21 = open (nombre_fifo_21, O_WRONLY)) == -1) {
    perror (nombre_fifo_21);
    exit (-1);
}

/* Armamos el manejador de la señal SIGINT. */
signal (SIGINT, fin_de_transmision);

/* Bucle de recepción de mensajes. */
do {
    printf ("==> "); fflush (stdout);
    read (fifo_12, mensaje, MAX);
    printf ("%s", mensaje);
    if (EQ(mensaje, "cambio\n"))
        do {
            printf ("<== ");
            fgets (mensaje, sizeof (mensaje), stdin);
            write (fifo_21, mensaje, strlen(mensaje) + 1);
        } while (!EQ(mensaje,"cambio\n") && !EQ(mensaje,"corto\n"));
} while (!EQ(mensaje, "corto\n"));

printf ("FIN DE TRANSMISIÓN.\n");
close (fifo_12);

```

```
    close (fifo_21);
    exit (0);
}

/****
fin_de_transmision: rutina de tratamiento de la señal SIGINT. Al pulsar Ctrl+C
entramos en esta rutina, que se encarga de enviar el mensaje "corto\n" al usuario
con el que estamos hablando.
****/
void fin_de_transmision (int sig)
{
    sprintf (mensaje, "corto\n");
    write (fifo_21, mensaje, strlen(mensaje) + 1);
    printf ("FIN DE TRANSMISIÓN.\n");
    close (fifo_12);
    close (fifo_21);
    exit (0);
}
```

Lo normal es que a la hora de usar una tubería con nombre sólo haya un proceso que escriba en ella y otro que lea de ella, pero esto no es una imposición. Puede haber más de un proceso escribiendo en la tubería y varios procesos leyendo de ella, y el número de procesos escritores no tiene por qué coincidir con el de lectores. En estas situaciones, todos los procesos que utilizan una misma tubería deben implementar mecanismos para coordinar su uso.

9.6 Comunicación dúplex

En los casos estudiados hasta ahora, la comunicación entre procesos es de tipo semidúplex. Esto se debe a que no hemos visto técnicas para conseguir que un proceso lea simultáneamente de dos o más dispositivos. Así, en el caso del programa que simula las conversaciones por radio, un programa actúa de hablante y el otro de oyente. Esto está impuesto por la secuencia del código, porque cuando el hablante lee un mensaje del teclado, no puede estar leyendo de su tubería asociada para ver si su interlocutor le envía algo. De igual forma, el oyente lee de la tubería correspondiente, pero en ese instante no atiende al teclado.

La comunicación *dúplex* soluciona estos inconvenientes, permitiendo que un proceso pueda actuar como emisor o receptor en cualquier instante, sin tener que sujetarse al protocolo de paso de la palabra. Un ejemplo de este tipo de comunicación es una conversación telefónica, en la que los interlocutores pueden hablar o escuchar cuando les apetezca e incluso pueden hablar ambos a la vez. Otra tema es que decidan respetar sus turnos para poder entenderse, pero esto no viene impuesto por el canal ni por la tecnología de la comunicación.

Para simular las comunicaciones dúplex tenemos que conseguir que un proceso pueda leer de forma simultánea datos procedentes de más de una fuente. Vamos a describir tres

técnicas que persiguen este objetivo:

- Interrogación periódica —*polling*—.
- Lectura conducida por eventos.
- Multiplexación mediante `select`.

A continuación comentaré con más detalle cada una de estas técnicas.

9.6.1 Interrogación periódica

La técnica de interrogación periódica consiste en hacer un recorrido periódico interrogando sobre el estado de las distintas fuentes. En el momento que se encuentra una con datos, se procede a leerlos y pasamos a interrogar sobre el estado de la siguiente. Esta técnica implica disponer de funciones que informen sobre el estado de la fuente; por ejemplo, si hay datos en una tubería o en un fichero. Estas funciones no suelen estar disponibles; por tanto, la solución está en interrogar con la propia función de lectura. Sabremos que no hay datos cuando la función no pueda leer.

Para impedir que la función de lectura se quede bloqueada en el caso de que no haya datos que leer —puede ser el caso de las tuberías vacías—, será necesario reprogramar el acceso al fichero o dispositivo para permitir una lectura no bloqueante. Esto se puede hacer abriendo el dispositivo con el indicador `O_NDELAY` activo o activándolo con una llamada a `fcntl`.

La periodicidad del interrogatorio debe calcularse lo suficientemente lenta como para que no sature de trabajo a la CPU, y lo suficientemente rápida como para que el usuario no note demoras en la comunicación. Habrá que llegar a una solución de compromiso que estará condicionada por la naturaleza estadística de las peticiones de comunicación.

Como ejemplo comentaré un programa que simula comunicaciones telefónicas. Este programa maneja dos fuentes de datos, el teclado y una tubería de comunicaciones. Como la comunicación es bidireccional, necesitaremos dos tuberías, una por cada dirección. La mecánica del programa consistirá en leer datos del teclado y escribir en una de las tuberías, o leer datos de la otra tubería y escribirlos en pantalla. Como la comunicación va a ser dúplex, no está preestablecido cuál es el dispositivo del que se lee primero. Se debe leer del primero que tenga datos disponibles. La periodicidad con la que se pregunta por la existencia de datos de entrada es de 1 segundo: primero vamos a preguntar por el teclado y a continuación por la tubería. El código del programa es el siguiente:

Program 9.6: Comunicación dúplex mediante interrogación periódica (`llamar.c`)

/***

Descripción: programa para simular conversaciones telefónicas, la comunicación es dúplex.

Forma de uso:

\$ `llamar -e usuario` Para efectuar una llamada.

\$ `llamar -r usuario` Para responder a una llamada.

El programa exige que la variable de entorno LOGNAME esté correctamente inicializada con el nombre del usuario. Esto se puede conseguir incluyendo las dos líneas siguientes en el fichero `.profile` de configuración de nuestro intérprete de órdenes:

```

    LOGNAME='logname'
    export LOGNAME
***/
#include <stdio.h>
#include <string.h>
#include <signal.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include "utmp.h"
#define MAX 256

/* Macro para comparar dos cadenas de caracteres. */
#define EQ(str1, str2) (strcmp ((str1), (str2)) == 0)

/* Descriptores de fichero de las tuberías mediante las que nos vamos a comunicar. */
int fifo_12, fifo_21;
char nombre_fifo_12[MAX], nombre_fifo_21[MAX];

/* Array para leer los mensajes. */
char mensaje [MAX];

/* Variable global para almacenar el estado inicial de acceso al terminal. */
int estado;

main (int argc, char *argv [])
{
    int tty;
    enum {LLAMAR, RECIBIR} tipo_llamada;
    enum {TECLADO, FIFO} origen;
    char terminal [MAX], *cadena, *logname, *getenv (), *leer_cadena ();
    struct utmp *utmp, *getutent ();
    void fin_de_transmision ();

    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc != 3 || (!EQ(argv[1], "-e") && !EQ(argv[1], "-r"))) {
        fprintf (stderr, "Forma de uso: %s -e|-r usuario\n", argv [0]);
        exit (-1);
    } else if (EQ(argv[1], "-e"))
        tipo_llamada = LLAMAR;
    else
        tipo_llamada = RECIBIR;

```

```

/* Lectura de nuestro nombre de usuario. */
logname = getenv ("LOGNAME");
/* Comprobación para que un usuario no se llame a sí mismo. */
if (EQ (logname, argv [2])) {
    fprintf (stderr, "Comunicación con uno mismo no permitida\n");
    exit (0);
}

/* Consultamos si el usuario llamado ha iniciado una sesión. */
while ((utmp = getutent ()) != NULL &&
    strncmp (utmp->ut_user, argv [2], sizeof (utmp->ut_user)) != 0);
if (utmp == NULL) {
    printf ("EL USUARIO %s NO HA INICIADO SESIÓN.\n", argv [2]);
    exit (0);
}

/* Formación de los nombres de las tuberías de comunicación. */
sprintf (nombre_fifo_12, "/tmp/%s_%s", logname, argv[2]);
sprintf (nombre_fifo_21, "/tmp/%s_%s", argv[2], logname);

/* Inicialización realizada por el proceso que efectúa la llamada. El proceso que
recibe la llamada no realiza esta inicialización. */
if (tipo_llamada == LLAMAR){
    /* Creación y apertura de las tuberías. */
    /* Primero borramos las tuberías, para que la llamada a mknod no falle. */
    unlink (nombre_fifo_12);
    unlink (nombre_fifo_21);
    /* Cambiamos la máscara de permisos por defecto para este proceso. Esto
    permitirá crear las tuberías con permisos rw-rw-rw==0666.*/
    umask (~0666);
    /* Creamos las tuberías para que la llamada a open no falle. */
    if (mkfifo (nombre_fifo_12, 0666) == -1) {
        perror (nombre_fifo_12);
        exit (-1);
    }
    if (mkfifo (nombre_fifo_21, 0666) == -1) {
        perror (nombre_fifo_21);
        exit (-1);
    }
    /* Apertura del terminal del usuario. */
    sprintf (terminal, "/dev/%s", utmp->ut_line);
    if ((tty = open (terminal, O_WRONLY)) == -1) {
        perror (terminal);
        exit (-1);
    }
}

```



```

/* Aviso al usuario con el que nos queremos comunicar. */
sprintf (mensaje,
        "\n\t\tLLAMADA PROCEDENTE DEL USUARIO %s\07\n"
        "\t\tRESPONDER ESCRIBIENDO: llamar -r %s\n\n",
        logname, logname);
write (tty, mensaje, strlen (mensaje) + 1);
close (tty);

printf ("Esperando respuesta...\n");
}

/* Apertura de las tuberías. Una de ellas para escribir mensajes y la otra para
leerlos. Es importante tener en cuenta el orden de apertura, porque si se invirtiera
podría producirse un abrazo mortal. */
if ((fifo_21 = open(nombre_fifo_21,O_RDONLY|O_NDELAY))==-1 ||
    (fifo_12 = open (nombre_fifo_12, O_WRONLY)) == -1 ) {
    if (fifo_12 == -1)
        perror (nombre_fifo_12);
    else
        perror (nombre_fifo_21);
    exit (-1);
}

/* A este punto llegamos cuando nuestro interlocutor responde a la llamada. */
if (tipo_llamada == LLAMAR)
    printf ("LLAMADA ATENDIDA.\07\n");

/* Reprogramación del fichero estándar de entrada para activar el
modo O_NDELAY. */
estado = fcntl (0, F_GETFL, 0);
if (fcntl (0, F_SETFL, estado | O_NDELAY) == -1)
    perror ("fcntl");

/* Armamos el manejador de la señal SIGINT. Esta señal se genera al
pulsar Ctrl+C. */
signal (SIGINT, fin_de_transmision);

/* Bucle de lectura (del teclado y de la tubería) y de envío de mensajes. */
printf ("<== "), fflush (stdout);
do {
    /* Lectura del teclado. */
    if ((cadena = leer_cadena ()) != NULL) {
        if (cadena [0] != 0) {
            strcpy (mensaje, cadena);
            write (fifo_12, mensaje, strlen(cadena) + 1);
        }
        printf ("<== "), fflush (stdout);
    }
}

```

```

        origen = TECLADO;
    }
    /* Lectura de la tubería de recepción. */
    if (read (fifo_21, mensaje, MAX) > 0) {
        printf ("==> %s\n", mensaje);
        printf ("<== "), fflush (stdout);
        origen = FIFO;
    }
    sleep (1); /* Cada segundo se repite el bucle. */
} while (!EQ(mensaje, "corto"));

/* Terminación de la comunicación */
/* Restauración del estado del terminal. */
fcntl (0, F_SETFL, estado);
/* Enviamos el mensaje corto a nuestro interlocutor. */
if (origen == TECLADO) {
    sprintf (mensaje, "corto");
    write (fifo_12, mensaje, strlen(mensaje) + 1);
}
printf ("FIN DE TRANSMISIÓN.\n");
close (fifo_12);
close (fifo_21);
exit (0);
}

/****
fin_de_transmision: rutina de tratamiento de la señal SIGINT. Al pulsar Ctrl+C
entramos en esta rutina, que se encarga de enviar el mensaje "corto" al usuario
con el que estamos hablando.
****/
void fin_de_transmision (int sig)
{
    /* Restauración del estado del terminal. */
    fcntl (0, F_SETFL, estado);
    sprintf (mensaje, "corto");
    write (fifo_12, mensaje, strlen(mensaje) + 1);
    printf ("FIN DE TRANSMISIÓN.\n");
    close (fifo_12);
    close (fifo_21);
    exit (0);
}

/****
leer_cadena: esta función lee caracteres de la entrada estándar y va formando un
mensaje. La lectura que realiza no es bloqueante; por tanto, si no se introduce
nada, devuelve NULL.

```

La función devuelve NULL siempre que no esté formado el mensaje completo. Se considera que el mensaje está completo cuando se pulsa la tecla **Enter**. En este caso se devuelve la dirección de una cadena estática sobre la que se han almacenado los valores leídos.

```

***/
char *leer_cadena ()
{
# define CR 10 /* Carácter Nueva Línea. */
    static char cadena [MAX];
    char cadena_aux [MAX];
    static int primera_vez = 1;
    int nbytes, i;

    if (primera_vez) {
        cadena [0] = '\0';
        --primera_vez;
    }

    if ((nbytes = read (0, cadena_aux, MAX)) == 0)
        return (NULL);
    for (i = 0; i < nbytes; i++)
        if (cadena_aux [i] == CR) {
            cadena_aux [i] = '\0';
            strcat (cadena, cadena_aux);
            ++primera_vez;
            return (cadena);
        }

    cadena_aux [i] = '\0';
    strcat (cadena, cadena_aux);
    return (NULL);
}

```

Uno de los inconvenientes que presenta la lectura por interrogación periódica es el tiempo que se pierde interrogando por el estado de las distintas fuentes de entrada. Por otro lado, las prestaciones del procedimiento van a estar estrechamente ligadas al período de muestreo que se seleccione. Si el régimen de generación de datos y el período de muestreo son muy altos, puede darse el caso de que alguna tubería o alguna memoria intermedia de entrada/salida se sature y se pierdan datos.

9.6.2 Lectura conducida por eventos

En una lectura conducida por eventos habrá varios procesos encargados de atender a las distintas fuentes de datos. En concreto, habrá un proceso por cada fuente. El proceso principal estará parado la mayor parte del tiempo y, sólo cuando alguno de sus procesos servidores le comunique que se ha producido un evento, tomará el control y leerá del dis-

positivo correspondiente. Una de las formas de comunicar eventos que tienen los procesos servidores es mediante el envío de señales al proceso principal.

Supongamos, como en los ejemplos anteriores, que un proceso recibe información de una tubería y del teclado. Este proceso puede crear dos procesos servidores, uno que leerá constantemente de la tubería y el otro que lo hará del teclado. Cuando alguna de las fuentes tiene datos, su proceso correspondiente procede a leerlos y acto seguido se lo comunica a su proceso principal mediante una señal. Por ejemplo, el proceso que lee de la tubería puede enviar la señal `SIGUSR1`, y el que lee del teclado, la señal `SIGUSR2`. Estos procesos, a su vez, escribirán los datos en otra tubería que los comunica con el principal. Las rutinas de tratamiento de las señales anteriores son las encargadas de leer de esta tubería, con lo que el proceso principal tiene así acceso a los datos.

El uso de señales distintas sirve para discernir las distintas fuentes, pero no resulta operativo cuando el número de fuentes es elevado. Esto es así porque el número de señales reservadas para el usuario es sólo de dos. Para eliminar este inconveniente, podemos recurrir a incluir una cabecera en el conjunto de datos que los procesos servidores le envían al proceso principal. Esta cabecera puede constar de dos campos: la procedencia del mensaje —tubería o teclado—, codificada mediante un número entero, y la longitud de la cadena de caracteres transmitida. Así, el proceso principal puede discernir los tipos de mensajes que se encuentran en su tubería de comunicación con los procesos servidores. Con este mecanismo, lo que hemos hecho ha sido implementar un protocolo de comunicaciones mediante mensajes.

En la figura 9.6 podemos ver la estructura que tendría el programa de llamadas telefónicas anterior. Los procesos *A* y *B* están asociados, respectivamente, a cada uno de los interlocutores. Dando servicio al proceso *A*, estarán los procesos *A*₁ y *A*₂. El primero de ellos se encarga de leer del teclado y el segundo lee de la salida de la tubería que comunica *B* con *A*. Tanto *A*₁ como *A*₂ escriben mensajes en una tubería que los comunica con el proceso *A*. Para el proceso *B*, la estructura es idéntica.

9.6.3 Multiplexación mediante select

La tercera técnica que estudiaremos para conseguir leer de varias fuentes simultáneamente es la multiplexación de la entrada/salida mediante la llamada `select`. Esta llamada permite interrogar al sistema sobre el estado de varios dispositivos y devuelve cuáles están listos para leer datos de ellos y cuáles lo están para escribir en ellos. Su declaración es la siguiente:

```
#include <time.h>
int select (int nfds, int readfds, int writefds, int exceptfds,
            struct timeval *timeout);
```

La llamada examina los descriptores de fichero especificados en las máscaras de bits `readfds`, `writefds` y `exceptfds`. Se examinan los bits comprendidos entre las posiciones 0 y `nfds-1`, ambas inclusive. El fichero de descriptor `fd` se codifica mediante el bit de la posición `1<fd` de la máscara correspondiente. El significado de cada máscara es el siguiente:

- `readfds` representa los descriptores de los ficheros de lectura por los que pregunta-

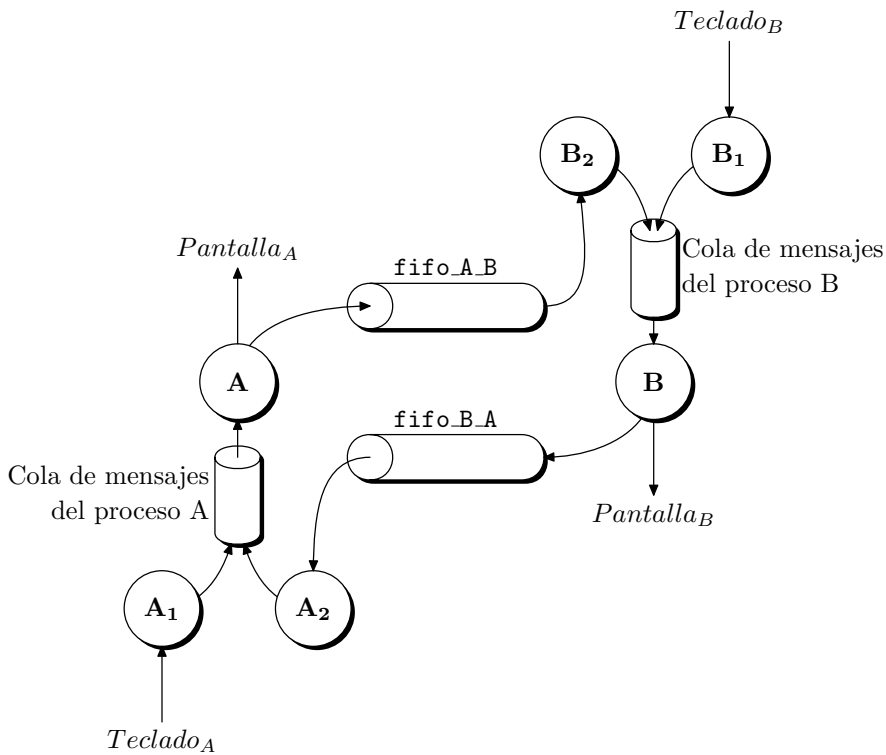


Figura 9.6: Estructura de los canales de comunicación de un programa conducido por mensajes.

mos. Cada bit activo significa que **select** debe interrogar sobre el estado del fichero asociado para ver si tiene datos que leer.

- **writelfds** representa los descriptores de los ficheros de escritura por los que preguntamos. Cada bit activo significa que **select** debe interrogar sobre el estado del fichero asociado para ver si se puede escribir en él.
- **exceptfds** representa los descriptores de los ficheros con alguna condición especial por los que preguntamos. Cada bit activo significa que **select** debe interrogar sobre el estado del fichero asociado para ver si se ha producido alguna condición especial.

Si alguna de las condiciones anteriores no nos interesa, lo indicaremos pasándole a **select** el argumento 0. Los descriptores que se analizan son los comprendidos entre 0 y **nfds**−1, ambos inclusive. Como podemos ver, los tres argumentos anteriores se pasan por referencia y no por valor. Esto significa que **select** los va a modificar de acuerdo con el estado de los ficheros que interroga. Así, cuando **select** devuelva el control, en **readfds**, **writelfds** y **exceptfds** aparecerán activados aquellos bits correspondientes a los descriptores de fichero que reúnen algún requisito sobre el que se preguntaba.

El parámetro **timeout** es un puntero no nulo que indica el tiempo máximo que se esperará desde que **select** entra en ejecución hasta que devuelve el control. La llamada

puede devolver el control bien porque alguno de los ficheros interrogados cumpla los requisitos pedidos o bien porque el tiempo de espera especificado en `timeout` expire. En el caso de que el tiempo de espera se agote, `select` devolverá el valor 0 y en las máscaras aparecerán todos los bits a 0. Si `timeout` es un puntero NULL, la llamada esperará a que se dé algún cambio de estado en alguno de los ficheros interrogados.

Las máscaras de bits que codifican los números de los descriptors agrupan 32 posibles descriptors por cada número entero. Si queremos interrogar por ficheros con un descriptor más alto, hay que utilizar un array de enteros. Así, por ejemplo, para interrogar por el descriptor número 45, hay que utilizar un array de dos enteros, y el bit número 13 del segundo entero deberá estar a 1. Para encapsular estos detalles, junto con `select` se facilita, ya construido, el tipo `fd_set` y un conjunto de 4 macros para manejar variables de ese tipo. Estas macros se definen con la siguiente interfaz:

```
// Pone a cero los bits de fdset
FD_ZERO (fd_set *fdset);
// Activa en fdset el bit correspondiente al descriptor fd
FD_SET (int fd, fd_set *fdset);
// Desactiva en fdset el bit correspondiente al descriptor fd
FD_CLEAR (int fd, fd_set *fdset);
// Comprueba en fdset el bit correspondiente al descriptor fd
FD_ISSET (int fd, fd_set *fdset);
```

Un ejemplo de uso de estas macros puede ser:

```
#include <sys/types.h>
#include <sys/time.h>
...
fd_set fd_var;
...
FD_ZERO (&fd_var); // Pone a cero todos los bits de fd_var
FD_SET (5, &fd_var); // Activa en fd_var el bit correspondiente al
                      // descriptor 5
...
```

9.7 Ejercicios

- 9.1** Escriba un programa de nombre `tuberías` que pueda utilizarse para arrancar tres procesos que se comuniquen a través de dos tuberías sin nombre. La forma de invocar al programa será:

```
$ tuberías -1 orden_1 -2 orden_2 -3 orden_3
```

El resultado de la ejecución del programa anterior debe ser equivalente a la siguiente orden del intérprete:

```
$ orden_1 | orden_2 | orden_3
```

Los parámetros `orden_1`, `orden_2` y `orden_3` del programa `tuberías` podrán componerse de una o más palabras expresadas entre comillas dobles. Así, se podrán

arrancar concurrentemente órdenes complejas como "ls -al /usr/bin" o "sort -r".

Por otro lado, el orden de los parámetros no debe afectar a la ejecución, por ello cada orden va precedida de un número que indica la posición que debe ocupar en el canal de comunicación formado por las tuberías. Así, los tres ejemplos siguientes constituyen la misma orden:

```
$ tuberías -1 "ls -al /usr/bin" -2 "sort -r" -3 more
```

```
$ tuberías -3 more -1 "ls -al /usr/bin" -2 "sort -r"
```

```
$ tuberías -2 "sort -r" -3 more -1 "ls -al /usr/bin"
```

Para facilitar el análisis de los argumentos se puede utilizar la función `getopt`. El fragmento de código siguiente muestra cómo utilizar `getopt` aplicado al programa tuberías:

```
char *orden [3];
int parámetros_obligatorios = 3;
char opción;
int errores = 0;
extern char *optarg;

// Análisis de los argumentos de la línea de órdenes.
while ((opcion = getopt (argc, argv, "1:2:3:")) != -1)
    switch (opcion) {
        case '1':
            orden [0] = optarg;
            --parámetros_obligatorios;
            break;
        case '2':
            orden [1] = optarg;
            --parámetros_obligatorios;
            break;
        case '3':
            orden [2] = optarg;
            --parámetros_obligatorios;
            break;
        default:
            ++errores;
    }

if (errores) {
    fprintf (stderr,
            "Forma de uso: %s -1 orden_1 -2 orden_2 -3 orden_3\n",
            argv [0]);
    exit (-1);
}
```

```

if (parámetros_obligatorios) {
    fprintf (stderr, "Faltan parámetros obligatotos\n");
    fprintf (stderr,
        "Forma de uso: %s -1 orden_1 -2 orden_2 -3 orden_3\n",
        argv [0]);
    exit (-1);
}

```

9.2 Todos los procesos que realicen su entrada/salida de datos a través de los ficheros estándar pueden intercambiarse información mediante tuberías y los mecanismos de redirección de la entrada/salida.

Una aplicación típica de la comunicación bidireccional es la constituida por programas que actúan como interfaz de otros programas. Es el caso del depurador **xxgdb** para X-WINDOW. Este programa no es realmente un depurador, se trata sólo de una interfaz gráfica con el depurador de línea **gdb**. Otro ejemplo lo constituyen los editores de pantalla, como **vi**, que tienen la capacidad de comunicarse con otros editores más rudimentarios como **ed**.

La idea que hay detrás de este tipo de comunicación es la de aprovechar al máximo los recursos del sistema. Cuando un usuario necesita un programa que le resuelva un problema concreto, puede optar por escribir desde el principio una aplicación que responda a sus necesidades. Una solución que resulta a veces más cómoda y rápida es aprovechar las aplicaciones ya existentes, pero que no responden exactamente a nuestras necesidades, y darles un lavado de cara que permita utilizarlas en nuestro servicio.

Como ejemplo se pide escribir un programa de nombre **buscar** que sea capaz de buscar cadenas de caracteres dentro de un fichero de texto. Las cadenas a buscar se indicarán mediante patrones conocidos como expresiones regulares. A continuación se muestra una sesión de trabajo con este programa:

\$ buscar

```

Fichero? listado.dat  Fichero en el que se realizarán las búsquedas
Patrón de búsqueda: ? cc Imprimir las líneas donde aparezca la cadena cc
yacc
gcc
cc
Patrón de búsqueda: ? ls Imprimir las líneas donde aparezca la cadena ls
mtools
ls
false.sh
false
Patrón de búsqueda: ? ^lp Imprimir las líneas que empiecen por lp
lpctest
lprm
lpr

```


lpq

lpf

lpc

Patrón de búsqueda: ? Ctrl+d *Finalizar*

Este programa puede codificarse por entero sin necesidad de utilizar tuberías ni comunicarse con otro programa que ya esté escrito. Sin embargo, para ilustrar el uso de la comunicación bidireccional y de los coprocesos —procesos que trabajan para otros—, el programa **buscar** debe actuar como interfaz con el editor de líneas **ed**. El editor **ed** es una herramienta primitiva en comparación con **vi** o con los editores gráficos del sistema X-WINDOW, sin embargo, tiene la ventaja de que puede trabajar como coproceso para otros programas que necesiten sus servicios.

El esquema de trabajo del programa **buscar** será el siguiente:

- (a) Abrir dos tuberías. Una transmitirá órdenes desde **buscar** hacia **ed**, la otra transmitirá resultados desde **ed** hacia **buscar**.
- (b) Crear un proceso hijo desde el que se invocará al programa **ed**. Este proceso hereda las tuberías creadas anteriormente, con lo que queda establecida la conexión entre **buscar** y **ed**.
- (c) El programa padre —**buscar**— leerá del teclado el nombre del fichero donde se debe buscar y le pedirá a **ed** que lo abra.
- (d) El programa **buscar** leerá patrones de búsqueda del teclado y le pedirá a **ed** que busque en el fichero indicado esos patrones.
- (e) Los resultados devueltos por **ed** serán presentados en pantalla por el programa **buscar**.

A continuación se muestran algunos fragmentos del programa **buscar**:

```
// Ejemplo de comunicación bidireccional con un programa estándar a través
// de dos tuberías.
// El programa se comunica con el editor de líneas ed que es el encargado
// de realizar los trabajos. El programa buscar actúa solo de interfaz.
#include <stdio.h>
#include <fcntl.h>

typedef enum {FALSO, VERDADERO} BOOLEAN;
...
void error_fatal (char *str)
{
    perror (str);
    exit (-1);
}

void invocar_a_ed ()
{
```

```

...
// Creación de las tuberías
...
switch (fork ()) {
case -1:
    // Error
    ...
case 0:
    // Proceso hijo
    // Redirección de los ficheros estándar
    ...

    // Invocación del editor
    execlp ("ed", "ed", "-p", "__EdItOr__\n", NULL);
    error_fatal ("execlp");
}
// Proceso padre
// Reasignación de los ficheros estándar
...
}

// enviar_a_ed: función para enviar órdenes al editor
void enviar_a_ed (char *orden) {...}

// recibir_de_ed: función para leer resultados procedentes del editor
// Devuelve VERDADERO cuando el mensaje leído es distinto de la
// cadena "__EdItOr__"
// La cadena "__EdItOr__" es el indicador con el que se ha programado
// el editor ed
BOOLEAN recibir_de_ed (char *mensaje, int tamaño) {...}

// recibir_todo_de_ed: función para leer todos los mensajes procedentes
// del editor
recibir_todo_de_ed ()
{
    char línea [256];

    ...
    while (recibir_de_ed (línea, sizeof (línea)))
        printf ("%s", línea);
}

// indicador: presenta el indicador del programa buscar
indicador (char *mensaje, char *resultado)
{

```

```

        printf ("\n%s? ", mensaje);
        return (gets (resultado) != NULL);
    }

    main ()
    {
        char línea [256];
        char orden_para_ed [256];

        invocar_a_ed ();
        recibir_todo_de_ed ();
        if (!indicador ("Fichero", línea))
            exit (0);
        sprintf (orden_para_ed, "e %s\n", línea);
        enviar_a_ed (orden_para_ed);
        recibir_todo_de_ed ();
        while (indicador ("Patrón de búsqueda: ", línea)) {
            sprintf (orden_para_ed, "g/%s/p\n", línea);
            enviar_a_ed (orden_para_ed);
            recibir_todo_de_ed ();
        }
        enviar_a_ed ("q\n");
    }
}

```

Sólo queda por comentar algunos aspectos sobre el funcionamiento del programa `ed`:

- La orden para abrir un fichero es `r`,
`r nombre_fichero`
- La orden para buscar cadenas es `g`,
`g/cadena/`
- Cada vez que el editor está listo para recibir una nueva orden aparece en pantalla el indicador de `ed`. Como en el ejercicio `ed` escribe en una tubería, este indicador no aparece en pantalla. Para distinguir entre el indicador y los datos generados por una búsqueda, el indicador se ha programado con el valor `__EdItOr__` que en principio es lo suficientemente extraño como para que no se confunda con los datos. Sin embargo, no tenemos la garantía de que en algún fichero de texto no haya una cadena con ese aspecto.

El aspecto del indicador se programa con el argumento `-p` de la orden `ed`,

```
$ ed -p indicador
```

Por eso, la llamada a `exec` tiene la forma siguiente:

```
execlp ("ed", "ed", "-p", "__EdItOr__\n", NULL);
```

9.3 Recodifique el programa `llamar` que simula conversaciones telefónicas dúplex empleando la técnica de lectura conducida por mensajes. El nuevo programa se llamará `llamar1`. Recordemos brevemente en qué consiste esta técnica:

- El proceso emisor dispone de dos procesos auxiliares que gestionan para él las distintas fuentes de información, consulte la figura 9.6 en la pág. 358.
 - El proceso *servidor de teclado* lee mensajes del teclado y los introduce en la tubería sin nombre que lo comunica con su proceso maestro.
 - El proceso *servidor de tubería* lee mensajes de la tubería *receptor-emisor* y los introduce en la tubería sin nombre que lo comunica con su proceso maestro. Esos mensajes habrán sido generados por el proceso receptor.
- El proceso emisor está leyendo continuamente mensajes procedentes de sus servidores. Estos mensajes tienen la composición siguiente:
 - Cabecera. A su vez se compone de:
 - * Tipo de mensaje. Número entero que codifica el tipo de mensaje. Se pueden considerar los siguientes:
 - `FIN`, mensaje para finalizar. Lo envía el servidor de teclado cuando el usuario introduce la cadena `corto`.
 - `TECLADO`, mensaje procedente del servidor de teclado.
 - `FIFO`, mensaje procedente del servidor de la tubería.
 - * Longitud del mensaje. Número entero que indica la longitud del mensaje que hay en la tubería sin nombre y que debe ser leído después de la cabecera.
 - Mensaje. Cadena de caracteres con el mensaje.

Para manejar estos datos se pueden utilizar las definiciones siguientes:

```
// Declaraciones relacionadas con los mensajes.
// Tipos de mensaje.
typedef enum {TECLADO = 1, FIFO, FIN} TIPOS_DE_MENSAJE;
// Estructura de la cabecera de los mensajes
typedef struct {
    TIPOS_DE_MENSAJE tipo;
    int longitud;
} CABECERA;
```

- Dependiendo del mensaje leído, el padre realizará una acción u otra. Por eso decimos que el padre es conducido por los mensajes. Las acciones son las siguientes:
 - `FIN`: enviarle al proceso receptor el mensaje `corto` para terminar la comunicación, enviarle a sus procesos servidores la señal `SIGTERM` para que estos terminen y terminar su ejecución.
 - `TECLADO`: mensaje procedente del servidor de teclado. Este mensaje se debe enviar al receptor a través de la tubería *emisor-receptor*.

- **FIFO**: mensaje procedente del servidor de la tubería. Este mensaje se debe presentar en pantalla. En caso de que su contenido sea la cadena **corto**, el proceso maestro debe terminar previo envío de la señal **SIGTERM** a sus procesos servidores.

El bucle de lectura y toma de decisiones del proceso maestro puede tener la forma siguiente:

```
if (tipo_llamada == LLAMAR)
    fifo = fifo_12;    // Caso de llamar
else
    fifo = fifo_21;    // Caso de responder a una llamada

// Bucle de lectura de los mensajes
while (read (tubería [0], &cabecera, sizeof (cabecera)) > 0)
    switch (cabecera.tipo) {
        case FIN:
            strcpy (mensaje, "corto");
            write (fifo, mensaje, strlen (mensaje) + 1);
            kill (pid_st, SIGTERM);
            kill (pid_sf, SIGTERM);
            exit (0);
        case TECLADO:
            read (tubería [0], mensaje, cabecera.longitud);
            write (fifo, mensaje, strlen (mensaje) + 1);
            break;
        case FIFO:
            read (tubería [0], mensaje, cabecera.longitud);
            if (!EQ (mensaje, "corto")) {
                printf ("\n==> %s\n<== ", mensaje);
                fflush (stdout);
            } else {
                printf ("\n==> %s\n", mensaje);
                kill (pid_st, SIGTERM);
                kill (pid_sf, SIGTERM);
                exit (0);
            }
    }
}
```

- El proceso receptor tiene una estructura idéntica y de hecho está codificado con el mismo programa. La diferencia radica en que la variable **tipo_llamada** tomará el valor **RECIBIR**, con lo que cambia el sentido de la comunicación a través de las tuberías y por lo tanto:
 - El servidor de la tubería debe leer mensajes de la tubería *emisor-receptor*.
 - El proceso maestro debe escribir mensajes en la tubería *receptor-emisor*.
- La estructura de la comunicación entre el receptor y sus servidores es la misma que para el proceso emisor.

Puesto que se aprovecha un mismo programa para ejecutar dos procesos con funciones distintas —emisor y receptor— conviene tener presentes los detalles siguientes:

- (a) A la hora de formar los nombres de las tuberías hay que distinguir si el proceso actual está en modo LLAMAR —es él quien efectúa la llamada— o en modo RECIBIR —es él quien la recibe—.

```
// Lectura de nuestro nombre de usuario
logname = getenv ("LOGNAME");
// Formación de los nombres de las tuberías de comunicación
if (tipo_llamada == LLAMAR) {
    sprintf(nombre_fifo_12, "/tmp/%s_%s", logname, argv[2]);
    sprintf(nombre_fifo_21, "/tmp/%s_%s", argv[2], logname);
} else {
    sprintf(nombre_fifo_12, "/tmp/%s_%s", argv[2], logname);
    sprintf(nombre_fifo_21, "/tmp/%s_%s", logname, argv[2]);
}
```

- (b) Esta misma distinción se debe realizar al abrir las tuberías.

```
// Apertura de las tuberías
if (tipo_llamada == LLAMAR) {
    fifo_12 = open (nombre_fifo_12, O_WRONLY);
    if (fifo_12 == -1)
        fatal (nombre_fifo_12);
    fifo_21 = open (nombre_fifo_21, O_RDONLY);
    if (fifo_21 == -1)
        fatal (nombre_fifo_21);
} else {
    fifo_12 = open (nombre_fifo_12, O_RDONLY);
    if (fifo_12 == -1)
        fatal (nombre_fifo_12);
    fifo_21 = open (nombre_fifo_21, O_WRONLY);
    if (fifo_21 == -1)
        fatal (nombre_fifo_21);
}
```

Por último, hay que decir que la forma de usar el programa sigue siendo la misma:

`$ llamar1 -e usuario para efectuar una llamada`

`$ llamar1 -r usuario para responder a una llamada`

9.4 Modifique el programa `llamar1` escrito en el ejercicio anterior para añadirle una funcionalidad mejorada. El nuevo programa tendrá por nombre `llamar2` y debe ser mejorado en los aspectos siguientes:

- (a) Instalar un manejador de la señal `SIGINT` que se genera al pulsar las teclas `Ctrl+C`. El objetivo es que el proceso principal capture la señal de interrupción

para poder iniciar una secuencia de finalización controlada que debe incluir: envío del mensaje **corto** al proceso con el que se comunica, envío de la señal **SIGTERM** a sus procesos servidores, cierre de ficheros y terminación del proceso.

- (b) En el caso de que el usuario de destino no responda a la llamada, el proceso reitera su petición de llamada seis veces con un período de repetición de 10 segundos. Si después de las seis llamadas el usuario de destino sigue sin responder, el proceso debe mostrar en el terminal del receptor el mensaje **DESCONEXIÓN POR FALTA DE RESPUESTA** y terminar.

Para controlar que, mientras no haya respuesta, se repite la llamada con un período de 10 segundos habrá que manejar temporizadores. Como sabemos, los temporizadores generan la señal **SIGALRM** cuando llegan a cero. Esto va a ocasionar un problema con las llamadas al sistema que se estuvieran efectuando en el instante de recepción de la señal, ya que esas llamadas interrumpen su ejecución y devuelven el valor -1.

Por lo tanto, será necesario controlar el valor devuelto por aquellas llamadas que se están haciendo en el instante de recepción de **SIGALRM**. En concreto, en ese instante se está abriendo la tubería que comunica al emisor con el receptor. Como la tubería aún no ha sido abierta por el receptor, la llamada de apertura efectuada por el emisor se encuentra durmiendo.

Las líneas de código siguientes muestran cómo controlar este tipo de eventos:

```
if (tipo_llamada == LLAMAR) {
    do {
        fifo_12 = open (nombre_fifo_12, O_WRONLY);
    } while (fifo_12 == -1 && errno == EINTR);
    if (fifo_12 == -1)
        fatal (nombre_fifo_12);
    do {
        fifo_21 = open (nombre_fifo_21, O_RDONLY);
    } while (fifo_21 == -1 && errno == EINTR);
    if (fifo_21 == -1)
        fatal (nombre_fifo_21);
}
```

Capítulo 10

Comunicación local entre procesos e hilos

«Si no os picáredes más de saber más menear las negras que lleváis que la lengua –dijo el otro estudiante–, vos llevarades el primero en licencias, como llevastes cola.»

(Cervantes, *Quijote II-19*)

10.1 Mecanismos <i>IPC</i> del UNIX System V	369
10.2 Semáforos	372
10.3 Memoria compartida	379
10.4 Colas de mensajes	387
10.5 Semáforos en POSIX	397
10.6 Cerrojos en POSIX	400
10.7 Monitores con interfaz de procedimiento	405
10.8 Ejercicios	415

10.1 Mecanismos *IPC* del UNIX System V

El paquete de comunicación entre procesos del UNIX System V se compone de tres mecanismos¹: los semáforos, que permiten sincronizar procesos; la memoria compartida, que permite que los procesos compartan su espacio de direcciones virtuales; y las colas de mensajes, que posibilitan el intercambio de datos con un formato determinado. Estos mecanismos están implementados como una unidad y comparten características comunes, entre las que se encuentran:

- Cada mecanismo tiene una tabla cuyas entradas describen el uso que se hace del mismo.

¹**mecanismo.** (Del lat. *mechanisma*, con adaptación del sufijo al usual *-ismo*). m. [...] ||3. Medios prácticos que se emplean en las artes.[RAE, 2001]

- Cada entrada de la tabla tiene una llave numérica elegida por el usuario.
- Cada mecanismo dispone de una llamada **get** para crear una entrada nueva o recuperar alguna ya existente. Dentro de los parámetros de esta llamada se incluye una llave y una máscara de indicadores. El núcleo busca dentro de la tabla alguna entrada que se ajuste a la llave suministrada. Si la llave toma el valor `IPC_PRIVATE`, el núcleo ocupa la primera entrada que se encuentre libre y ninguna otra llamada **get** podrá devolver esta entrada hasta que la liberemos. Si dentro de la máscara de indicadores está activo el bit `IPC_CREAT`, el núcleo crea una nueva entrada en caso de que no haya ninguna que responda a la llave suministrada. Si, además de `IPC_CREAT`, está activo el bit `IPC_EXCL`, el núcleo devuelve un error en caso de que ya exista una entrada para la llave suministrada. Si todo funciona correctamente, el núcleo devuelve un descriptor que se podrá usar en otras llamadas. Este descriptor cumple, para las llamadas relacionadas con las facilidades *IPC* —*Interprocess Communication*—, la misma finalidad que los descriptors de fichero para el caso de las llamadas relacionadas con el sistema de ficheros.
- Para cada mecanismo *IPC*, el núcleo aplica la fórmula siguiente a la hora de calcular el índice que da acceso a la tabla:

$$\text{índice}_{\text{tabla}} = \text{número}_{\text{descriptor}} \% (\text{número de entradas en la tabla})$$

Así, por ejemplo, si hay N procesos compartiendo la entrada número 3 de la tabla y la tabla consta de 100 entradas, cada uno de los procesos puede estar referenciando a la misma entrada a través de los descriptors 3, 103, 203, etc.

- Cada entrada de la tabla tiene un registro de permisos que incluye: identificador de usuario y de grupo del proceso que ha reservado la entrada, identificador de usuario y de grupo modificados por la llamada de control del mecanismo *IPC*, un conjunto de bits con los permisos de lectura, escritura y ejecución para el usuario, el grupo y otros usuarios.
- Cada entrada contiene información de estado, en la que se incluye el identificador del último proceso que ha utilizado la entrada.
- Cada mecanismo *IPC* tiene una llamada de control que permite leer y modificar el estado de una entrada reservada y también permite liberarla.

10.1.1 Formación de llaves

Una llave es una variable o constante del tipo `key_t` que usaremos para acceder a los mecanismos *IPC* previamente reservados o para reservar otros nuevos. Es normal que los mecanismos que están siendo utilizados como parte de un mismo proyecto compartan la misma llave.

Hay múltiples formas de crear llaves y es necesario que todo sistema defina algún procedimiento estándar para realizar esta función. Esto impedirá que procesos no relacionados, por no formar parte de un mismo proyecto, puedan interferirse involuntariamente debido a que usen la misma llave.

La biblioteca de C aporta la función `ftok` para crear llaves de una forma estándar. Esta función tiene la siguiente declaración:

```
#include <types.h>
#include <sys/ipc.h>
key_t ftok (char *path, char id);
```

y devuelve una llave basada en `path` y en `id`. Esta llave podrá usarse en llamadas futuras a `msgget`, `semget` y `shmget` para obtener un identificador que dé acceso a algún mecanismo *IPC*. El parámetro `path` es un puntero a la ruta de un fichero que debe existir dentro de nuestro sistema de ficheros y que debe ser accesible para el proceso que llama a `ftok`. El parámetro `id` es un carácter que identifica el proyecto. La función `ftok` devolverá la misma llave para rutas enlazadas a un mismo fichero, siempre que se utilice el mismo valor para `id`. Para el mismo fichero, `ftok` devolverá diferentes llaves para distintos valores de `id`.

Si el fichero no es accesible para el proceso, bien porque no existe, bien porque sus permisos lo impiden, `ftok` devolverá el valor `(key_t)(-1)`, indicando así que se ha producido un error en la creación de la llave.

Las líneas siguientes muestran la forma de llamar a `ftok` para crear una llave asociada al fichero "prueba" y al identificador 'A':

```
key_t llave;
...
if ((llave = ftok ("prueba", 'A')) == (key_t)-1) {
    // Error al crear la llave
    // Tratamiento del error
}
```

10.1.2 Control de las facilidades *IPC* desde la línea de órdenes

Los programas estándar `ipcs` e `ipcrm` nos permiten controlar los recursos *IPC* que gestiona el sistema; `ipcs` se utiliza para ver los mecanismos que están asignados y a quién, junto con información estadística; `ipcrm` se emplea para liberar un mecanismo asignado y dejar libre la entrada que ocupa. Estos dos programas son bastante útiles para depurar programas que se valen de los mecanismos *IPC*.

La forma de emplear `ipcs` es la siguiente:

```
$ ipcs [opciones]
```

Si no se especifica ninguna opción, `ipcs` muestra un resumen de la información administrativa que se almacena para los semáforos, memoria compartida y colas de mensajes que hay asignados. Las principales opciones son:

- `-q`, muestra información de las colas de mensajes que hay activas.
- `-m`, muestra información de los segmentos de memoria compartida que hay activos.
- `-s`, muestra información de los semáforos que hay activos.

- `-b`, muestra una información completa sobre los tres tipos de mecanismos *IPC* que hay activos.

Los siguientes son ejemplos de ejecución de `ipcs`:

```
$ ipcs -q
```

```
IPC status from /dev/kmem as of Tue Jun 30 12:05:58 1992
T      ID      KEY      MODE      OWNER      GROUP
Message Queues:
q       0 0x3c08204d -Rrw--w--w-      root      root
q       1 0x3e08204d --rw-r--r--      root      root
```

```
$ ipcs -b
```

```
IPC status from /dev/kmem as of Tue Jun 30 12:09:01 1992
T      ID      KEY      MODE      OWNER      GROUP QBYTES
Message Queues:
q       0 0x3c08204d -Rrw--w--w-      root      root  16384
q       1 0x3e08204d --rw-r--r--      root      root    264
T      ID      KEY      MODE      OWNER      GROUP SEG SZ
Shared Memory:
m       0 0x44082000 --rw-r-----      root      root  83992
T      ID      KEY      MODE      OWNER      GROUP NSEMS
Semaphores:
s       0 0x01090522 --ra-r--r--      root      root     1
```

La forma de emplear `ipcrm` es la siguiente:

```
$ ipcrm [ opciones ]
```

Algunas de las opciones que hay disponibles son:

- `-q msqid`, borra la cola de mensajes cuyo identificador coincide con `msqid`.
- `-m shmid`, borra la zona de memoria compartida cuyo identificador coincide con `shmid`.
- `-s semid`, borra el semáforo cuyo identificador coincide con `semid`.

10.2 Semáforos

Un semáforo es un mecanismo para prevenir la colisión que se produce cuando dos o más procesos solicitan simultáneamente el uso de un recurso que deben compartir.

10.2.1 Conceptos generales

En programación, al igual que ocurre con los semáforos de circulación, éstos sólo tienen un carácter informativo. Si aun a pesar de que un semáforo está prohibiendo el acceso a

un recurso decidimos saltarnos esa prohibición, el resultado de nuestro proceso puede ser inconsistente.

Dijkstra define un semáforo como un objeto de tipo entero sobre el que se pueden realizar dos operaciones: $\mathcal{P}$ y $\mathcal{V}$ ². La operación $\mathcal{P}$ decrementa el valor del semáforo y se utiliza para adquirirlo o bloquearlo. La operación $\mathcal{V}$ incrementa el valor del semáforo y se utiliza para liberarlo. Un semáforo no puede tomar un valor negativo y cuando vale 0 no se pueden realizar más operaciones $\mathcal{P}$ sobre él [Dijkstra, 1968a]. Si el semáforo sólo puede tomar dos valores, se dice que es binario.

Las operaciones $\mathcal{P}$ y $\mathcal{V}$ deben ser atómicas para que funcionen correctamente. Esto quiere decir que una operación $\mathcal{P}$ no puede ser interrumpida por otra operación $\mathcal{P}$ o $\mathcal{V}$, y lo mismo ocurre para la operación $\mathcal{V}$. Esto garantiza que, cuando varios procesos compitan por la adquisición de un semáforo, sólo uno de ellos podrá realizar la operación.

El mecanismo *IPC* de semáforos implementado en el UNIX System V es una generalización del concepto de semáforo descrito por Dijkstra, ya que permite manejar un conjunto de semáforos mediante el uso de un identificador asociado. También podremos realizar operaciones $\mathcal{P}$ y $\mathcal{V}$ que actualizan de forma atómica todos los semáforos asociados bajo un mismo identificador. Un semáforo en UNIX System V se compone de los siguientes elementos:

- El valor del semáforo.
- El identificador del último proceso que manipuló el semáforo.
- El número de procesos que hay esperando a que el valor del semáforo se incremente.
- El número de procesos que hay esperando a que el semáforo tome el valor 0.

Las llamadas relacionadas con los semáforos son: `semget`, para crear un semáforo o habilitar el acceso a uno ya existente; `semctl`, para realizar operaciones de control e inicialización; y `semop`, para realizar operaciones $\mathcal{P}$ y $\mathcal{V}$ sobre el semáforo.

10.2.2 Petición de semáforos (`semget`)

Con `semget` podemos crear o acceder a un conjunto de semáforos unidos bajo un identificador común:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
int semget (key_t key, int nsems, int semflg);
```

Si la llamada funciona correctamente, devolverá un identificador con el que podremos acceder a los semáforos en sucesivas llamadas. Si algo falla, `semget` devuelve el valor -1 y en `errno` estará el código del error producido.

²Las letras $\mathcal{P}$ y $\mathcal{V}$ provienen de los verbos holandeses *proberen* —intentar— y *verhogen* —incrementar—. Otras formas bastante extendidas de llamar a las operaciones con semáforos son *wait* o *down* para la operación $\mathcal{P}$ y *signal* o *up* para la operación $\mathcal{V}$. La primera opción —*wait-signal*— no es aconsejable en el entorno UNIX por ser éstos los nombres de dos llamadas al sistema. La segunda es aceptable. Sin embargo, por respeto a la memoria del Prof. Dijkstra voy a emplear los nombres originales.

El parámetro **key** es la llave que indica a qué grupo de semáforos queremos acceder. Esta llave puede crearse mediante una llamada a **ftok**. Si **key** toma el valor **IPC_PRIVATE**, la llamada crea un nuevo identificador sujeto a la disponibilidad de entradas libres en la tabla que gestiona el núcleo para los semáforos. El parámetro **nsems** es el total de semáforos que van a estar agrupados bajo el identificador devuelto por **semget** y **semflg** es una máscara de bits que indican el modo de adquisición del identificador. Si el bit **IPC_CREAT** está activo, la facilidad **IPC** se creará en el supuesto de que otro proceso no la haya creado ya. Si los bits **IPC_CREAT** e **IPC_EXCL** están activos simultáneamente, la llamada falla en el caso de que el conjunto de semáforos ya esté creado. Los 9 bits menos significativos de **semflg** indican los permisos del semáforo. Sus posibles valores son:

- 0400=100 000 000 Permiso de lectura para el usuario.
- 0200=010 000 000 Permiso de modificación para el usuario.
- 0060=000 110 000 Permiso de lectura y modificación para el grupo.
- 0006=000 000 110 Permiso de lectura y modificación para el resto de usuarios.

El identificador devuelto por **semget** es heredado por los procesos descendientes del actual.

Las líneas de código siguientes muestran cómo crear un nuevo identificador con 4 semáforos, asociado a la llave creada a partir del fichero **auxiliar** y la clave **K**. Este identificador se crea con permisos de lectura y modificación para el usuario.

```
int llave, semid;
...
if ((llave = ftok ("auxiliar", 'K')) == (key_t)-1) {
    // Error al crear la llave. Tratamiento del error.
}
if ((semid = semget (llave, 4, IPC_CREAT | 0600)) == -1) {
    // Error al crear el identificador. Tratamiento del error.
}
```

10.2.3 Control de las estructuras de semáforo (**semctl**)

Con **semctl** podremos acceder a la información administrativa y de control que dispone el núcleo sobre un semáforo:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
int semctl (int semid, int semnum, int cmd, arg)
union semun {
    int val;
    struct semid_ds *buf;
    ushort *array;
}arg;
```

La llamada actúa sobre el conjunto de semáforos que responden al identificador `semid` devuelto por una llamada previa a `semget`. El parámetro `semnum` indica cuál es el semáforo, de los que hay bajo `semid`, al que queremos acceder. La operación de control que se realizará sobre el semáforo viene indicada en `cmd`; los siguientes son valores válidos para `cmd`:

- `GETVAL`, se utiliza para leer el valor de un semáforo. El valor se devuelve a través del nombre de la función.
- `SETVAL`, permite inicializar un semáforo a un valor determinado que se especifica en `arg`.
- `GETPID`, se usa para leer el `PID` del último proceso que actuó sobre el semáforo. Este valor se devuelve a través del nombre de la función.
- `GETNCNT`, permite leer el número de procesos que hay esperando a que se incremente el valor del semáforo. Este número se devuelve a través del nombre de la función.
- `GETZCNT`, permite leer el número de procesos que hay esperando a que el semáforo tome el valor 0. Este número se devuelve a través del nombre de la función.
- `GETALL`, permite leer el valor de todos los semáforos asociados al identificador `semid`. Estos valores se almacenan en `arg`.
- `SETALL`, sirve para inicializar el valor de todos los semáforos asociados al identificador `semid`. Los valores de inicialización deben estar en `arg`.
- `IPC_STAT` e `IPC_SET`, permiten leer y modificar la información administrativa asociada al identificador `semid`. Para obtener más información puede consultar la página `semctl(2)` del manual de UNIX.
- `IPC_RMID`, le indica al núcleo que debe borrar el conjunto de semáforos agrupados bajo el identificador `semid`. La operación de borrado no tendrá efecto mientras haya algún proceso que esté usando los semáforos.

Si la llamada a `semctl` se realiza satisfactoriamente, la función devuelve un número cuyo significado dependerá del valor de `cmd`. Así, son significativos los valores devueltos cuando `cmd` vale `GETVAL`, `GETNCNT`, `GETZCNT` o `GETPID`. Para cualquier otro valor de `cmd`, la función devuelve 0 cuando se ejecuta satisfactoriamente y -1 en caso de error.

A continuación mostramos una secuencia de código que crea un identificador con 4 semáforos asociados. Los 2 primeros se inicializan con el valor 5 y los dos últimos con el valor 8. Luego preguntamos por el valor del semáforo número 3.

```
int semid, valor;
ushort semarray [4];
...
// Creación de los semáforos
semid = semget (ftok ("auxiliar", 'K'), 4, IPC_CREAT | 0600);
if (semid == -1) // Tratamiento del error
...
```

```
// Inicialización de los semáforos
semarray[0] = 5;
semarray[1] = 5;
semarray[2] = 8;
semarray[3] = 8;
semctl (semid, 0, SETALL, semarray);
...
// Preguntamos por el valor del semáforo número 3
valor = semctl (semid, 3, GETVAL, 0);
```

10.2.4 Operaciones $\mathcal{P}$ y $\mathcal{V}$ (semop)

Para realizar las operaciones $\mathcal{P}$ y $\mathcal{V}$ con los semáforos del UNIX System V, tenemos que usar la llamada `semop`:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
int semop (int semid, struct sembuf *sops, int nsops);
```

Esta llamada realiza operaciones atómicas sobre los semáforos que hay asociados bajo el identificador `semid`, `sops` es un puntero a un array de estructuras que indican las operaciones que llevarán a cabo sobre los semáforos y `nsops` es el total de elementos que tiene el array de operaciones. Cada elemento del array es una estructura del tipo `sembuf` que se define como sigue:

```
struct sembuf {
    ushort sem_num; // Número del semáforo
    short sem_op; // Operación: incrementar o decrementar
    short sem_flg; // Máscara de bits
};
```

El campo `sem_num` es el número del semáforo y se utiliza como índice para acceder a él, su valor está en el rango $0..N-1$, donde N es el total de semáforos que hay agrupados bajo el identificador; `sem_op` es la operación a realizar sobre el semáforo especificado en `sem_num`, si `sem_op` es negativo, el valor del semáforo se decrementará, lo que equivale a una operación $\mathcal{P}$, si `sem_op` es positivo, el valor del semáforo se incrementará, lo que equivale a una operación $\mathcal{V}$, si `sem_op` vale 0, el valor del semáforo no sufre ninguna alteración.

Las operaciones $\mathcal{V}$ siempre se ejecutan satisfactoriamente, ya que un semáforo puede tomar valores positivos; sin embargo, las operaciones $\mathcal{P}$ no siempre se pueden realizar. Si, por ejemplo, el semáforo tiene valor 2, no podemos restarle un número mayor que 2, porque el semáforo pasaría a tener valor negativo. Ante esta situación, `semop` responderá de diferentes formas, según el valor del campo `sem_flg`; sus bits significativos son:

- `IPC_NOWAIT`, la llamada a `semop` devuelve el control en caso de que no se pueda satisfacer la operación especificada en `sem_op`. La forma de trabajar por defecto es `IPC_WAIT`.

- **SEM_UNDO**, la operación se deshace cuando el proceso termina.

El bit **SEM_UNDO** previene contra el bloqueo accidental de semáforos. Supongamos que un proceso decrementa el valor de un semáforo y termina de forma anormal —por ejemplo, porque recibe una señal—, este semáforo quedará bloqueado para otros procesos. Este problema se previene activando el bit **SEM_UNDO** del campo **sem_flg**, ya que el núcleo se encarga de actualizar el valor del semáforo, deshaciendo las operaciones realizadas sobre él, cuando termina el proceso.

Si la llamada a **semop** no se realiza con éxito, la función devuelve **-1** y en **errno** estará el código del error producido.

Como ejemplo, veamos la forma de realizar una operación $\mathcal{P}$ y otra $\mathcal{V}$ sobre los semáforos número 1 y 3 de un identificador que agrupa un total de 4 semáforos:

```
struct sembuf operaciones[4];
...
operaciones[0].sem_num = 1; // Semáforo número 1
operaciones[0].sem_op = -1; // Operación  $\mathcal{P}$ 
operaciones[0].sem_flg = 0;
operaciones[1].sem_num = 3; // Semáforo número 3
operaciones[1].sem_op = 1; // Operación  $\mathcal{V}$ 
operaciones[1].sem_flg = 0;

semop (semid, operaciones, 2);
...
```

10.2.5 Ejemplo de aplicación. Sincronización de procesos

El ejemplo que vamos a implementar ilustra el uso de los semáforos para sincronizar dos procesos —padre e hijo—. Pretendemos sincronizar los procesos para que se vayan ejecutando alternativamente. Para conseguir esto, necesitamos al menos dos semáforos. Uno lo utiliza el padre para darle paso al hijo, y el otro lo utiliza el hijo para darle paso al padre. La secuencia de ejecución del padre es tomar su semáforo, realizar las operaciones que considere necesarias y abrir el semáforo del hijo cuando termine. Cuando el padre intenta tomar de nuevo su semáforo se da cuenta de que no puede porque no lo ha levantado, por lo que el proceso padre se pone a esperar. El proceso hijo realiza algo parecido, tomando primero su semáforo y levantando luego el del padre. Mediante estos mecanismos conseguimos que los procesos se vayan pasando el turno y se ejecuten alternativamente. El código que ilustra esto es el siguiente:

Programa 10.1: Sincronización de procesos mediante semáforos (**sem-1.c**)

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>

#define SEM_HIJO 0
#define SEM_PADRE 1
```



```
main (int argc, char *argv [])
{
    int i = 10, semid, pid;
    struct sembuf operacion;
    key_t llave;

    /* Petición de un identificador con dos semáforos. */
    llave = ftok (argv[0], 'K');
    if ((semid = semget (llave, 2, IPC_CREAT | 0600)) == -1) {
        perror ("semget");
        exit (-1);
    }
    /* Inicialización de los semáforos. */
    /* Cerramos el semáforo del proceso hijo. */
    semctl(semid, SEM_HIJO, SETVAL, 0);
    /* Abrimos el semáforo del proceso padre. */
    semctl(semid, SEM_PADRE, SETVAL, 1);

    /* Creación del proceso hijo. */
    if ((pid = fork()) == -1) {
        perror ("fork");
        exit (-1);
    } else if (pid == 0) { /* Código del proceso hijo. */
        while (i) {
            /* Cerramos el semáforo del proceso hijo. */
            operacion.sem_num = SEM_HIJO;
            operacion.sem_op = -1;
            operacion.sem_flg = 0;
            semop (semid, &operacion, 1);

            printf ("PROCESO HIJO: %d\n", i--);

            /* Abrimos el semáforo del proceso padre. */
            operacion.sem_num = SEM_PADRE;
            operacion.sem_op = 1;
            semop (semid, &operacion, 1);
        }

        /* Borrado del semáforo. */
        semctl (semid, 0, IPC_RMID, 0);
    } else { /* Código del proceso padre. */
        operacion.sem_flg = 0;
        while (i) {
            /* Cerramos el semáforo del proceso padre. */
            operacion.sem_num = SEM_PADRE;
            operacion.sem_op = -1;
```

```
semop (semid, &operacion, 1);

printf ("PROCESO PADRE: %d\n", i--);

/* Abrimos el semáforo del proceso hijo. */
operacion.sem_num = SEM_HIJO;
operacion.sem_op = 1;
semop (semid, &operacion, 1);
}

/* Borrado del semáforo. */
semctl (semid, 0, IPC_RMID, 0);
}
}
```

10.3 Memoria compartida

La forma más rápida de comunicar dos procesos es hacer que compartan una zona de memoria. Para enviar datos de un proceso a otro, sólo hay que escribir en memoria y automáticamente esos datos están disponibles para que los lea cualquier otro proceso.

La memoria convencional que puede direccionar un proceso a través de su espacio de direcciones virtuales es local a ese proceso y cualquier intento de direccionar esa memoria desde otro proceso provocará una violación de segmento.

Para solucionar este problema, UNIX System V brinda la posibilidad de crear zonas de memoria con la característica de poder ser direccionadas por varios procesos simultáneamente. El espacio de direcciones de esta memoria es virtual, por lo que sus direcciones físicas asociadas podrán variar con el tiempo. Esto no plantea problemas ya que los procesos no generan direcciones físicas, sino virtuales, y es el núcleo el encargado de traducir de unas a otras.

Las llamadas para manipular la memoria compartida son: **shmget**, para crear una zona de memoria compartida o habilitar el acceso a una ya creada; **shmctl**, para acceder y modificar la información administrativa y de control que el núcleo le asocia a cada zona de memoria compartida; **shmat**, para unir una zona de memoria compartida a un proceso; y **shmdt**, para separar una zona previamente unida.

10.3.1 Petición de memoria compartida (shmget)

Con **shmget** obtenemos un identificador con el que poder realizar futuras llamadas al sistema para controlar una zona de memoria compartida. Su declaración es:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmget (key_t key, int size, int shmflg);
```

donde **key** es una llave que tiene el mismo significado que vimos para la llamada **semget** —creación de semáforos—, ésta es una característica que tienen todas las llamadas para crear mecanismos *IPC*; **size** es el tamaño en bytes de la zona de memoria que queremos crear; y **shmflg** es una máscara de bits que tiene el mismo significado que vimos para la máscara **semflg** de la llamada **semget**. Si la llamada se ejecuta correctamente, devolverá el identificador —número entero no negativo— asociado a la zona de memoria; si falla, devolverá el valor **-1** y en **errno** estará el código del tipo de error producido. El identificador devuelto por **shmget** es heredado por los procesos descendientes del actual.

Las siguientes líneas muestran cómo crear una zona de memoria de tamaño 4.096 bytes, donde sólo el usuario tendrá permisos de lectura y escritura:

```
int shmid;
...
if ((shmid = shmget(IPC_PRIVATE, 4096, IPC_CREAT | 0600)) == -1) {
    // Error en la creación de la memoria compartida
    // Tratamiento del error
}
```

10.3.2 Control de una zona de memoria compartida (**shmctl**)

Con **shmctl** podremos realizar operaciones de control sobre una zona de memoria previamente creada por una llamada a **shmget**:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmctl (int shmid, int cmd, struct shmid_ds *buf);
```

El parámetro **shmid** es un identificador válido devuelto por una llamada previa a **shmget** y **cmd** indica el tipo de operación de control a realizar, sus posibles valores son:

- **IPC_STAT**, lee el estado de la estructura de control de la memoria y lo devuelve a través de la zona de memoria apuntada por **buf**.
- **IPC_SET**, inicializa algunos de los campos de la estructura de control de la memoria compartida. El valor de estos campos los toma de la estructura apuntada por **buf**.
- **IPC_RMID**, borra del sistema la zona de memoria compartida identificada por **shmid**. Si el segmento de memoria está unido a varios procesos, el borrado no se hace efectivo hasta que todos los procesos liberen la memoria.
- **SHM_LOCK**, bloquea en memoria el segmento identificado por **shmid**; esto quiere decir que no le afectarán las acciones del intercambiador —proceso **swapper**—. Sólo los procesos cuyo identificador de usuario efectivo —*EUID*— sea igual al del superusuario podrán realizar esta operación.
- **SHM_UNLOCK**, desbloquea el segmento de memoria compartida, con lo que los mecanismos de intercambio podrán trasladarlo de la memoria principal a la secundaria, y

viceversa, cada vez que sea necesario. Sólo los procesos cuyo identificador de usuario efectivo —*EUID*— sea igual al del superusuario pueden realizar esta operación.

La estructura `shmid_ds` se define como sigue:

```
struct shmid_ds {
    struct ipc_per shm_per; // Estructura de permisos
    int sh_segsz; // Tamaño del área de memoria compartida
    int pad1; // Usado por el sistema
    ushort shm_lpid; // PID del proceso que hizo la última operación con este
                    // segmento de memoria
    ushort shm_cpid; // PID del proceso creador del segmento
    ushort shm_nattach; // Número de procesos unidos al segmento de memoria
    short pad2; // Usado por el sistema
    time_t shm_atime; // Fecha de la última unión al segmento de memoria
    time_t shm_dtime; // Fecha de la última separación del segmento de memoria
    time_t shm_ctime; // Fecha del último cambio en el segmento de memoria
};
```

La siguiente línea muestra cómo borrar del sistema una zona de memoria compartida:

```
shmctl (shmid, IPC_RMID, 0);
```

10.3.3 Operaciones con la memoria compartida (`shmat` y `shmdt`)

Antes de usar una zona de memoria compartida tenemos que asignarle un espacio de direcciones virtuales de nuestro proceso. Esta acción se conoce como *unirse* o *atarse* al segmento de memoria compartida. Una vez que dejamos de usar un segmento de memoria, tenemos que desatarnos de él. Al realizar esta operación, el segmento deja de estar accesible para el proceso. Las llamadas al sistema para realizar estas operaciones son `shmat` —para atar— y `shmdt` —para desatar—; sus declaraciones son:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
char *shmat (int shmid, char *shmaddr, int shmflg);
int shmdt (char *shmaddr);
```

donde `shmid` es el identificador de una zona de memoria creada mediante una llamada previa a `shmget`; `shmaddr` es la dirección virtual donde queremos que empiece la zona de memoria compartida. Si la llamada a `shmat` funciona correctamente, devolverá un puntero a la dirección virtual a la que está unido el segmento de memoria compartida. Esta dirección puede coincidir o no con `shmaddr`, dependiendo de la decisión que tome el núcleo. Lo normal es que `shmaddr` valga 0, con lo cual se deja en manos del núcleo la elección de la dirección de inicio. Si `shmaddr` no vale 0, el núcleo intentará satisfacer la petición del usuario, pero no siempre se consigue. En el caso de `shmdt`, `shmaddr` es la dirección virtual del segmento de memoria compartida que queremos separar del proceso.

El parámetro `shmflg` es una máscara de bits que indica la forma de acceso a la memoria. Si el bit `SHM_RDONLY` está activo, la memoria será accesible para leer, pero no para escribir.

Si las llamadas funcionan correctamente, `shmat` devuelve la dirección a la que está unido el segmento de memoria compartida y `shmdt` devuelve 0. Si algo falla, ambas llamadas devuelven -1 y en `errno` estará el código del tipo de error producido.

Una vez que la memoria está unida al espacio de direcciones virtuales del proceso, el acceso a ella se realiza a través de punteros, como con cualquier otra memoria de datos asignada al programa.

A continuación mostramos la forma de crear una zona de memoria compartida en la que se almacenará un array unidimensional de números reales:

```
#define MAX 10

int shmid, i;
float *array;
key_t llave;
...
// Creación de una llave
llave = ftok ("prueba", 'K');
// Petición de una zona de memoria compartida
shmid = shmget (llave, MAX * sizeof(float), IPC_CREAT | 0600);

// Unión de la zona de memoria compartida a nuestro espacio de direcciones
// virtuales
array = shmat (shmid, 0, 0);

// Manipulación de la zona de memoria compartida
for (i = 0; i < MAX; i++)
    array [i] = i*i;
...
// Separación de la zona de memoria compartida de nuestro espacio de
// direcciones virtuales
shmdt (array);

// Borrado de la zona de memoria compartida
shmctl (shmid, IPC_RMID, 0);
...
```

10.3.4 Ejemplo: multiplicación concurrente de matrices

Como ejemplo, veamos la implementación de un algoritmo para multiplicar dos matrices concurrentemente. El programa principal se encarga de leer dos matrices y comprobar si se pueden multiplicar. Acto seguido se arrancan tantos procesos como le hayamos indicado en la línea de órdenes para multiplicar las matrices. Cada proceso se ocupa de generar una fila de la matriz producto mientras queden filas por generar. Naturalmente, las matrices

que intervienen en la operación deben estar en memoria compartida. Para controlar la fila que debe generar cada proceso, vamos a utilizar un semáforo que se inicializa con el total de filas de la matriz producto y se va decrementando por cada fila generada. El proceso principal se queda esperando a que todos los demás terminen para presentar el resultado.

Aunque en sistemas monoprocesador este método no resulta eficiente, es un buen ejercicio de sincronismo y comunicación entre procesos. El código de este programa es el siguiente:

Programa 10.2: Multiplicación concurrente de matrices (`matrices.c`)

```
/**
    $ matrices nro_de_procesos
    Programa para multiplicar matrices concurrentemente.
***/
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>

/* Definición de matriz. */
typedef struct {
    /* Identificador de la zona de memoria compartida donde estará la matriz. */
    int shmid;
    int filas, columnas; /* Número de filas y columnas de la matriz. */
    float **coef; /* Coeficientes de la matriz. */
}matriz;

/**
    crear_matriz: crea la memoria compartida donde se almacenará la matriz.
***/
matriz *crear_matriz (int filas, int columnas)
{
    int shmid, i;
    matriz *m;

    /* Petición de memoria compartida. */
    shmid = shmget (IPC_PRIVATE,
        sizeof (matriz) + filas*sizeof(float *) +
        filas*columnas*sizeof(float),
        IPC_CREAT | 0600);
    if (shmid == -1) {
        perror ("crear_matriz (shmget)");
        exit (-1);
    }
}
```

```

/* Nos atamos a la memoria. */
if ((m = (matriz *)shmat (shmid, 0, 0)) == (matriz *)-1) {
    perror ("crear_matriz (shmdt)");
    exit (-1);
}
/* Inicialización de la matriz. */
m->shmid = shmid;
m->filas = filas;
m->columnas = columnas;
/* Formateamos la memoria para poder direccionar los coeficientes de la matriz. */
m->coef = (float **)&m->coef + sizeof(float **);
for (i = 0; i < filas; i++)
    m->coef[i] = (float *)&m->coef[filas] + i*columnas*sizeof(float);
return m;
}

/****
    leer_matriz: lee una matriz del flujo estándar de entrada.
****/
matriz *leer_matriz ()
{
    int filas, columnas, i, j;
    matriz *m;

    printf ("Filas: ");
    scanf ("%d", &filas);
    printf ("Columnas: ");
    scanf ("%d", &columnas);
    m = crear_matriz (filas, columnas);
    for (i = 0; i < filas; i++)
        for (j = 0; j < columnas; j++)
            scanf ("%f", &m->coef[i][j]);
    return m;
}

/****
    multiplicar_matrices: multiplica dos matrices y crea una nueva para el
    resultado. El trabajo se distribuye entre el total de procesos que indique numproc.
****/
matriz *multiplicar_matrices (matriz *a, matriz *b, int numproc)
{
    int p, semid, estado;
    matriz *c;

    if (a->columnas != b->filas)
        return NULL;

```

```

c = crear_matriz (a->filas, b->columnas);
/* Creación de dos semáforos. Uno de ellos se inicializa con el total de filas de
la matriz producto. */
semid = semget (IPC_PRIVATE, 2, IPC_CREAT | 0600);
if (semid == -1) {
    perror ("multiplicar_matrices (semget)");
    exit (-1);
}
semctl (semid, 0, SETVAL, 1);
semctl (semid, 1, SETVAL, c->filas);
/* Creación de tantos procesos como indique numproc. */
for (p = 0; p < numproc; p++) { /*----- 1-for -----*/
    if (fork () == 0) { /*----- 2-if -----*/
        /* Código para los procesos hijo. */
        int i, j, k;
        struct sembuf operacion;

        operacion.sem_flg = SEM_UNDO;
        while (1) { /*----- 3-while -----*/
            /* Cada proceso hijo se encarga de generar una fila de la matriz
            producto. Para saber qué columna tiene que generar, consulta el
            valor del semáforo. */
            /* Operación  $\mathcal{P}$  sobre el semáforo 0. */
            operacion.sem_num = 0;
            operacion.sem_op = -1;
            semop (semid, &operacion, 1);
            /* Consultamos el valor del semáforo. */
            i = semctl (semid, 1, GETVAL, 0);
            if (i > 0) {
                /* Decrementamos el valor del semáforo 1 en una unidad. */
                semctl (semid, 1, SETVAL, --i);
                /* Operación  $\mathcal{V}$  sobre el semáforo 0. */
                operacion.sem_num = 0;
                operacion.sem_op = 1;
                semop (semid, &operacion, 1);
            } else
                exit (0);
            /* Cálculo de la fila i-ésima de la matriz producto. */
            for (j = 0; j < c->columnas; j++) {
                c->coef[i][j] = 0;
                for (k = 0; k < a->columnas; k++)
                    c->coef[i][j] += a->coef[i][k]*b->coef[k][j];
            }
        } /*----- 3-while -----*/
    } /*----- 2-if -----*/
} /*----- 1-for -----*/

```



```

/* Esperamos a que terminen todos los procesos. */
for (p = 0; p < numproc; p++)
    wait (&estado);
/* Borramos el semáforo. */
semctl (semid, 0, IPC_RMID, 0);
return c;
}

/****
    destruir_matriz: libera una zona de memoria compartida.
****/
destruir_matriz (matriz *m)
{
    shmctl (m->shmid, IPC_RMID, 0);
}

/****
    imprimir_matriz: presenta una matriz en el flujo estándar de salida.
****/
imprimir_matriz (matriz *m)
{
    int i, j;

    for (i = 0; i < m->filas; i++) {
        for (j = 0; j < m->columnas; j++)
            printf ("%g ", m->coef[i][j]);
        printf ("\n");
    }
}

/****
    Función principal.
****/
main (int argc, char *argv [])
{
    int numproc;
    matriz *a, *b, *c;

    /* Análisis de los argumentos de la línea de órdenes. */
    if (argc != 2 || (numproc = atoi (argv[1])) < 1)
        numproc = 2;
    /* Lectura de las matrices. */
    a = leer_matriz ();
    b = leer_matriz ();
    /* Procesamiento de las matrices. */

```

```
c = multiplicar_matrices (a, b, numproc);
if (c != NULL)
    imprimir_matriz (c);
else
    fprintf (stderr, "Las matrices no se pueden multiplicar.");
destruir_matriz (a);
destruir_matriz (b);
destruir_matriz (c);
}
```

Es importante aclarar la forma de organizar la memoria asignada a una matriz, ya que puede haber quedado algo oscura en el código. Recordemos que una matriz se define mediante la estructura:

```
typedef struct {
    int shmid;
    int filas, columnas;
    float **coef;
} matriz;
```

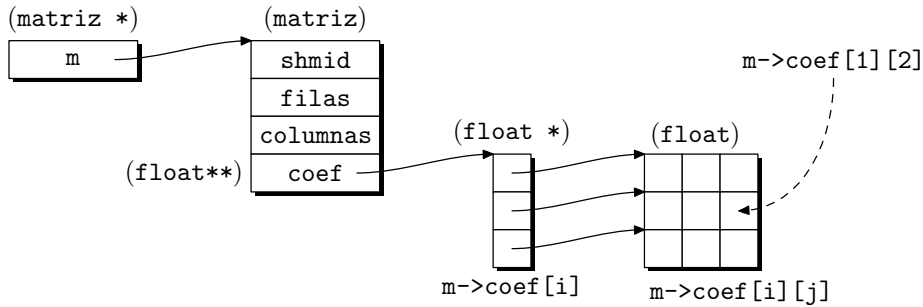
La matrices se almacenan en zonas de memoria compartida para que puedan ser manejadas por varios procesos. Estas zonas se crean mediante una llamada a `shmget`, y mediante `shmat` podemos unir las a un puntero del programa. La memoria creada por `shmget` sólo cumple el requisito de estar alineada, por lo que tenemos que darle formato de matriz.

Los campos `shmid`, `filas` y `columnas` no plantean ningún problema, ya que son accesibles a través de un puntero del tipo `matriz`. El problema lo plantean los coeficientes de la matriz, que como vemos se almacenan en el campo `coef`. Este campo es un doble puntero y para que pueda utilizarse para indexar una matriz, debe responder a una organización como la reflejada en la figura 10.1(a). Ésta es una representación gráfica para comprender el sentido de la doble indirección, pero la verdadera estructura de la memoria se muestra en la figura 10.1(b).

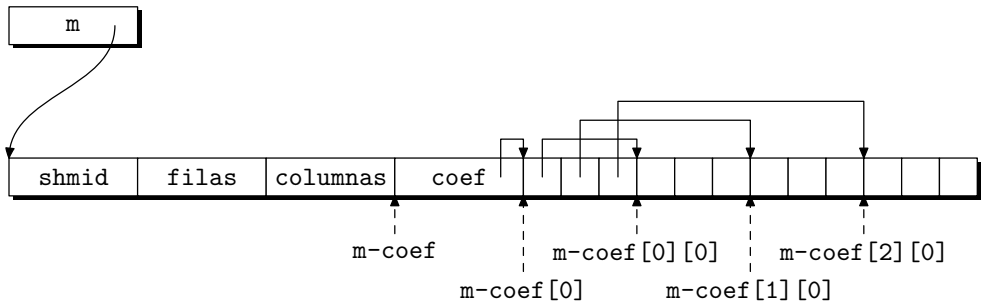
10.4 Colas de mensajes

Las colas de mensajes son otro de los mecanismos de comunicación entre procesos que brinda el UNIX System V. Su filosofía es parecida a la de las tuberías, pero con una mayor versatilidad. Una cola es una estructura de datos gestionada por el núcleo y donde podrán escribir varios procesos. Los mecanismos de sincronismo para que no se produzca colisión son responsabilidad del núcleo. Los datos que se escriben en la cola deben tener formato de mensaje y son tratados como un todo indivisible. A la vez, puede haber varios procesos leyendo de la cola y cada operación de lectura extrae un mensaje.

La cola se gestiona como un mecanismo con disciplina de hilera, donde el primer mensaje que entra es el primero que sale. Para dotar de más flexibilidad a las colas, se pueden hacer peticiones de lectura para extraer un mensaje de un tipo determinado, con lo que se rompe la gestión en hilera, aunque se sigue manteniendo para todos aquellos mensajes que son de un mismo tipo. Esta clasificación de los mensajes por tipos permite



a) Representación gráfica del doble uso de la indirección para indexar matrices.



b) Estructura real de la matriz.

Figura 10.1: Representación gráfica del tipo `matriz`.

distinguir cuáles son los mensajes que van destinados a cada uno de los procesos lectores. También se pueden hacer operaciones de lectura sin especificar el tipo de mensaje. Esto fuerza a que se lea el primer mensaje que haya en la cola. La figura 10.2 muestra la estructura que tienen las colas de mensajes.

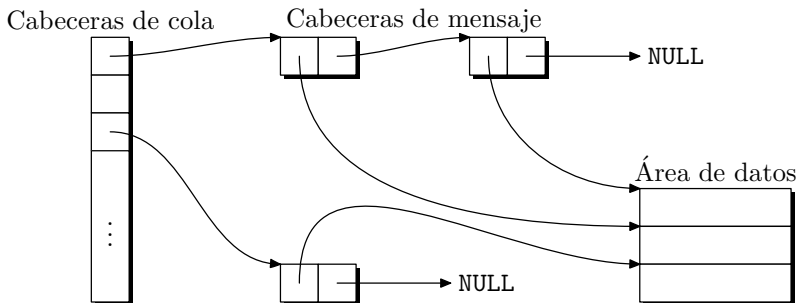


Figura 10.2: Estructura de una cola de mensajes.

Las llamadas que se emplean para manipular colas son: `msgget`, para crear una cola o habilitar el acceso a una ya existente; `msgctl`, para acceder y modificar la información administrativa y de control que el núcleo le asocia a cada cola de mensajes; `msgsnd`, para escribir un mensaje en la cola; y `msgrcv`, para sacar un mensaje de la cola.

10.4.1 Petición de una cola de mensajes (`msgget`)

La llamada que permite crear una cola de mensajes es `msgget` y su declaración es la siguiente:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgget (key_t key, int msgflg);
```

Si la llamada funciona correctamente, devuelve un identificador de la cola creada, en caso contrario, devuelve el valor `-1` y en `errno` estará el código del error producido.

El parámetro `key` es una llave que tiene el significado ya visto para los semáforos y la memoria compartida, `msgflg` es un mapa de bits con el significado visto para los parámetros `semflg` y `shmflg` de las llamadas de creación de semáforos y memoria compartida.

El identificador devuelto por `msgget` es heredado por los procesos descendientes del actual.

Las siguientes líneas muestran cómo crear una cola de mensajes:

```
int msqid;
key_t llave;
...
llave = ftok ("prueba", 'K');
if ((msqid = msgget (llave, IPC_CREAT | 0600)) == -1) {
    // Error en la creación de la cola
    // Tratamiento del error
}
```

10.4.2 Control de las colas de mensajes (`msgctl`)

La llamada para leer y modificar la información estadística y de control de una cola es `msgctl`:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgctl (int msqid, int cmd, struct msqid_ds *buf);
```

El parámetro `msqid` es un identificador creado mediante una llamada previa a `msgget` e indica la cola sobre la que vamos a actuar; `cmd` es el tipo de operación que queremos llevar a cabo y sus posibles valores son:

- `IPC_STAT`, lee el estado de la estructura de control de la cola y lo devuelve a través de la zona de memoria apuntada por `buf`. Podemos encontrar información sobre los campos de `buf` en `msgctl`.
- `IPC_SET`, inicializa los campos de la estructura de control de la cola. El valor de estos campos se toma de la estructura apuntada por `buf`.
- `IPC_RMID`, borra del sistema la cola de mensajes identificada por `msgqid`. Si la cola está siendo usada por otros procesos, el borrado no se hace efectivo hasta que todos los procesos terminan su ejecución.

A continuación podemos ver una línea de código que borra una cola de mensajes:

```
msgctl (msqid, IPC_RMID, 0);
```

10.4.3 Operaciones con colas de mensajes (`msgsnd` y `msgrcv`)

Podemos acceder a una cola para escribir y leer mensajes, las llamadas respectivas para estas operaciones son `msgsnd` y `msgrcv`:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgsnd (int msqid, void *msgp, int msgsz, int msgflg);
int msgrcv (int msqid, void *msgp, int msgsz, long msgtyp, int msgflg);
```

En ambas llamadas, `msqid` es el identificador de la cola sobre la que queremos trabajar; `msgp` apunta a la memoria intermedia con los datos que vamos a enviar o recibir, la composición de esta memoria la define el usuario, sólo es obligatorio que el primer campo sea de tipo `long` y se utiliza para identificar el tipo de mensaje, no existen tipos definidos por el sistema, por tanto, la clasificación de los mensajes depende del programador; `msgsz` es el tamaño, en bytes, del mensaje que queremos enviar o recibir, en este tamaño no se incluyen los bytes que ocupa el campo *tipo del mensaje*; `msgtyp` sólo aparece en la llamada de lectura y especifica el tipo del mensaje que queremos leer, puede tomar los siguientes valores:

- `msgtyp=0`, leer el primer mensaje que haya en la cola.
- `msgtyp>0`, leer el primer mensaje de tipo `msgtyp` que haya en la cola.
- `msgtyp<0`, leer el primer mensaje que cumpla que su tipo es menor o igual al valor absoluto de `msgtyp` y a la vez sea el más pequeño de los que hay.

El parámetro `msgflg` es un mapa de bits y tiene distintos significados según aparezca en la llamada `msgsnd` o `msgrcv`. Para la llamada `msgsnd` —escritura en la cola—, cuando la cola está llena:

- Si el bit `IPC_NOWAIT` está activo, la llamada devuelve el control inmediatamente y retorna el valor `-1`.

- Si el bit `IPC_NOWAIT` no está activo, el proceso suspende su ejecución hasta que haya espacio libre en la cola.

Para la llamada `msgrcv` —lectura de la cola—, cuando no hay ningún mensaje del tipo especificado:

- Si el bit `IPC_NOWAIT` está activo, la llamada devuelve el control inmediatamente y retorna el valor `-1`.
- Si el bit `IPC_NOWAIT` no está activo, el proceso suspende su ejecución en espera de que haya un mensaje del tipo deseado.

Si se ejecutan satisfactoriamente, `msgsnd` devuelve el valor `0` y `msgrcv` el total de bytes que realmente ha escrito en la memoria intermedia referenciada por `msgp`, este tamaño no incluye los bytes que ocupa el campo *tipo del mensaje*. Si las llamadas fallan durante su ejecución, devuelven el valor `-1` y en `errno` estará el código del error producido.

A continuación mostramos las líneas de código necesarias para enviar y recibir un mensaje del tipo `1`, que se compone de una cadena de `20` caracteres.

```
int msqid;
struct {
    long tipo;
    char cadena[20];
} mensaje;
int longitud = sizeof(mensaje) - sizeof (mensaje.tipo);
...
// Envío del mensaje
mensaje.tipo = 1;
strcpy (mensaje.cadena, "HOLA");
if (msgsnd (msqid, &mensaje, longitud, 0) == -1) {
    // Error en el envío
    // Tratamiento del error
}
...
// Recepción del mensaje
if (msgrcv (msqid, &mensaje, longitud, 1, 0) == -1) {
    // Error en la recepción
    // Tratamiento del error
}
```

10.4.4 Ejemplo: bases de datos centralizadas

Cuando una base de datos es utilizada simultáneamente por varios usuarios, conviene tomar precauciones para evitar colisiones de acceso. Una práctica habitual es encargar a un solo proceso la tarea de gestionar la base de datos. Este proceso gestor —*DBM* — *Data Base Manager*— recibe encargos de trabajo procedentes de otros procesos a los que se conoce como *clientes*. Así, el acceso a la base de datos se realiza siempre de forma indirecta, a través del gestor.

Para ilustrar estas ideas, vamos a escribir dos programas de ejemplo. Uno de ellos se comportará como gestor de una base de datos de personas. Esta base de datos se encuentra en el fichero `censo`, y cada uno de sus registros consta de los siguientes campos:

Nombre: 61 caracteres
 Dirección: 121 caracteres
 Teléfono: 11 caracteres

El otro programa será un cliente de la base de datos, y podrá realizar dos tipos de operaciones: visualizar el contenido de la base de datos y añadir nuevos registros.

Para comunicar el programa cliente con el gestor, se empleará una cola de mensajes. El *tipo* de cada mensaje coincidirá con el *PID* del proceso cliente. Así, cuando varios clientes soliciten un servicio al gestor, se podrá identificar a qué proceso pertenece cada mensaje. Los mensajes que viajen desde el gestor hacia el cliente se enviarán a través de otra cola, así evitamos que el gestor lea sus propios mensajes. La figura 10.3 muestra la interacción entre los procesos cliente y el gestor. Tanto el programa cliente como el gestor comparten el fichero de cabecera siguiente:

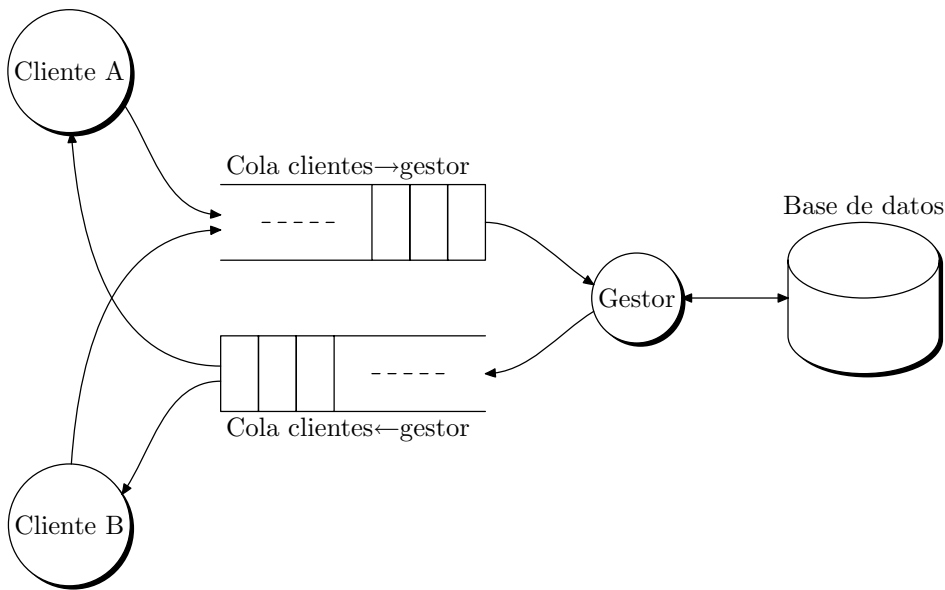


Figura 10.3: Flujo de mensajes entre los procesos cliente y el gestor de la base de datos.

Fichero 10.3: Cabecera para los clientes y el gestor de una base de datos centralizada (`censo.h`)

```

#ifndef _CENSO_
#define _CENSO_

#define MAX_NOMBRE 61
#define MAX_DIRECCION 121
    
```

```

#define MAX_TELEFONO 11
struct persona {
    char nombre [MAX_NOMBRE];
    char direccion [MAX_DIRECCION];
    char telefono [MAX_TELEFONO];
};

struct mensaje {
    long pid;
    int orden;
    union {
        struct persona persona;
    }datos;
};

#define LONGITUD (sizeof (struct mensaje) - sizeof (long))

/* Código de orden. */
#define LISTAR 1
#define ANNADIR 2
#define FIN 3
#define ERROR 4

#define FICHERO_LLAVE "censo.h"
#define CLAVE_CLIENTE_GESTOR 'K'
#define CLAVE_GESTOR_CLIENTE 'L'

#endif

```

El programa gestor queda como sigue:

Programa 10.4: Gestor de una base de datos centralizada (gestor.c)

```

/**
Programa que centraliza los accesos a la base de datos. Se encarga de prestar
servicio a otros programas clientes.
Forma de uso:
    $ gestor base_de_datos
***/
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include "censo.h"

main (int argc, char *argv [])
{

```



```

int cola_cg, cola_gc;
struct mensaje mensaje;
key_t llave;
FILE *pf;

/* Análisis de los argumentos de la línea de órdenes. */
if (argc != 2) {
    fprintf (stderr, "Forma de uso: %s fichero.\n", argv [0]);
    exit (-1);
}

/* Creación de las colas de mensajes. */
llave = ftok (FICHERO_LLAVE, CLAVE_CLIENTE_GESTOR);
if ((cola_cg = msgget (llave, IPC_CREAT | 0666)) == -1) {
    perror ("msgget");
    exit (-1);
}

llave = ftok (FICHERO_LLAVE, CLAVE_GESTOR_CLIENTE);
if ((cola_gc = msgget (llave, IPC_CREAT | 0666)) == -1) {
    perror ("msgget");
    exit (-1);
}

while (1) {
    msgrcv (cola_cg, &mensaje, LONGITUD, 0, 0);
    switch (mensaje.orden) {
        case LISTAR:
            if ((pf = fopen (argv [1], "r")) == NULL) {
                mensaje.orden = ERROR;
                msgsnd (cola_gc, &mensaje, LONGITUD, 0);
                break;
            }
            while (fread (&mensaje.datos.persona,
                sizeof (struct persona), 1, pf) == 1)
                msgsnd (cola_gc, &mensaje, LONGITUD, 0);
            fclose (pf);
            mensaje.orden = FIN;
            msgsnd (cola_gc, &mensaje, LONGITUD, 0);
            break;
        case ANNADIR:
            if ((pf = fopen (argv [1], "a")) == NULL) {
                mensaje.orden = ERROR;
                msgsnd (cola_gc, &mensaje, LONGITUD, 0);
                break;
            }
            fwrite (&mensaje.datos.persona, sizeof (struct persona), 1, pf);
            fclose (pf);
    }
}

```

```

        mensaje.orden = FIN;
        msgsnd (cola_gc, &mensaje, LONGITUD, 0);
        break;
    case FIN:
        msgctl (cola_cg, IPC_RMID, 0);
        msgctl (cola_gc, IPC_RMID, 0);
        printf("Programa gestor parado a petición de un cliente.\n");
        exit (0);
    default:
        mensaje.orden = ERROR;
        msgsnd (cola_gc, &mensaje, LONGITUD, 0);
    }
}
}

```

El programa cliente es el siguiente:

Programa 10.5: Cliente de una base de datos centralizada (cliente.c)

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>
#include "censo.h"

main (int argc, char *argv [])
{
    int cola_cg, cola_gc, pid;
    char opcion [256];
    int ncar;
    struct mensaje mensaje;
    key_t llave;
    enum {NO, SI} recibir = NO;

    /* Creación de las colas de mensajes. */
    llave = ftok (FICHERO_LLAVE, CLAVE_CLIENTE_GESTOR);
    if ((cola_cg = msgget (llave, 0666)) == -1) {
        perror ("msgget");
        exit (-1);
    }
    llave = ftok (FICHERO_LLAVE, CLAVE_GESTOR_CLIENTE);
    if ((cola_gc = msgget (llave, 0666)) == -1) {
        perror ("msgget");
        exit (-1);
    }
}

```

```

mensaje.pid = pid = getpid ();
while (1) {
    printf ("Orden: ");
    fflush (stdout);
    fgets (opcion, sizeof (opcion), stdin);
    switch (opcion[0]) {
        case 'l':
            recibir = SI;
            mensaje.orden = LISTAR;
            msgsnd (cola_cg, &mensaje, LONGITUD, 0);
            break;
        case 'a':
            recibir = SI;
            printf ("\tNombre: ");
            ncar = strlen (fgets (mensaje.datos.persona.nombre,
                                MAX_NOMBRE, stdin));
            mensaje.datos.persona.nombre [ncar-1] = '\0';
            printf ("\tDirección: ");
            ncar = strlen (fgets (mensaje.datos.persona.direccion,
                                MAX_DIRECCION, stdin));
            mensaje.datos.persona.direccion [ncar-1] = '\0';
            printf ("\tTeléfono: ");
            ncar = strlen (fgets (mensaje.datos.persona.telefono,
                                MAX_TELEFONO, stdin));
            mensaje.datos.persona.telefono [ncar-1] = '\0';
            mensaje.orden = ANNADIR;
            msgsnd (cola_cg, &mensaje, LONGITUD, 0);
            break;
        case 'f':
            recibir = NO;
            mensaje.orden = FIN;
            msgsnd (cola_cg, &mensaje, LONGITUD, 0);
        case 's':
            exit (0);
        default:
            recibir = NO;
            printf ("Opción [%c] errónea.\n");
            printf ("Opciones disponibles:\n");
            printf ("\ta - añadir registros.\n");
            printf ("\tf - parar el programa gestor.\n");
            printf ("\tl - visualizar todos los registros.\n");
            printf ("\ts - parar el programa cliente.\n");
    }
    if (recibir == SI)
        do {
            msgrcv (cola_gc, &mensaje, LONGITUD, pid, 0);

```

```

switch (mensaje.orden) {
case FIN:
    break;
case LISTAR:
    printf ("\n\tNombre: %s\n",
            mensaje.datos.persona.nombre);
    printf ("\tDirección: %s\n",
            mensaje.datos.persona.direccion);
    printf ("\tTeléfono: %s\n",
            mensaje.datos.persona.telefono);
    break;
case ERROR:
    printf ("Mensaje de error recibido.\n");
    break;
default:
    printf ("Tipo de mensaje desconocido [%d]\n",
            mensaje.orden);
}
} while (mensaje.orden != FIN && mensaje.orden != ERROR);
} /*----- while -----*/
} /*----- main -----*/

```

Conocida la estructura de los mensajes de comunicación con el programa gestor —el protocolo—, cualquier usuario puede escribir un programa cliente acorde con sus necesidades.

10.5 Semáforos en POSIX

La norma POSIX define un semáforo de la forma siguiente [IEEE, 2001a]:

«**Semáforo.** *Primitiva de sincronización mínima que pueden utilizar las aplicaciones para implementar mecanismos de sincronización más complejos[...]*Un semáforo se representa como un recurso compartido con un valor entero no negativo.

Bloqueo de un semáforo. *Si el semáforo vale cero, la operación produce el bloqueo del hilo que la llama y lo añade a la cola de hilos en espera; en caso contrario decrementa el valor del semáforo.*

Desbloqueo de un semáforo. *Si hay algún hilo bloqueado esperando en la cola, es extraído y desbloqueado, en caso contrario se incrementa el valor del semáforo.»*

Esta definición es muy parecida a la definición clásica de Dijkstra donde el bloqueo corresponde a la operación $\mathcal{P}$ y el desbloqueo a $\mathcal{V}$ —consulte el § 10.2.1 pág. 372—.

Si nuestro sistema soporta los semáforos POSIX entonces aparecerá definida la constante `_POSIX_SEMAPHORES` en el fichero de cabecera `<unistd.h>`. La norma distingue entre *semáforos nominales* y *semáforos anónimos*. Los primeros se utilizan para sincronizar procesos no relacionados y se les asocia un nombre en la jerarquía del sistema de ficheros; los segundos permiten sincronizar procesos que forman parte de una misma jerarquía o hilos de un mismo proceso.

Antes de utilizar un semáforo nominal es necesario abrirlo e inicializarlo con una llamada a `sem_open`:

```
#include <semaphore.h>
sem_t *sem_open (const char *name, int oflag, [mode_t mode, unsigned value]);
```

donde `name` es una ruta que sirve como nombre del semáforo y `oflag` es una máscara de bits que indica el modo de apertura: si está activo el bit `O_CREAT` el semáforo es creado en caso de que no exista, pero si además está activo el bit `O_EXCL` la función falla en caso de que el semáforo exista; esto impide que dos procesos intenten crear el semáforo a la vez. El argumento `mode` es una máscara con los permisos de lectura/escritura del semáforo y `value ≤ SEM_VALUE_MAX` es el valor de inicialización.

Si la función se ejecuta satisfactoriamente devuelve la dirección del semáforo creado que es de tipo `sem_t`. En caso de fallo devuelve `SEM_FAILED`, definido en `<semaphore.h>`, y escribe el código del error en la variable `errno`.

Para inicializar un semáforo anónimo utilizamos la función `sem_init`:

```
#include <semaphore.h>
int sem_init (sem_t *sem, int pshared, unsigned value);
```

donde `sem` es una referencia al semáforo, `value` es el valor de inicialización, `pshared ≠ 0` indica que el semáforo será compartido por procesos relacionados a través de una jerarquía padre-hijo y `pshared = 0` indica que el semáforo será compartido solo por los hilos del proceso que lo inicializa. Si la función no se ejecuta satisfactoriamente devuelve `-1` y en `errno` escribe el código del error producido.

Una vez creado e inicializado un semáforo podemos utilizarlo para sincronizar procesos o hilos realizando operaciones $\mathcal{P}$ y $\mathcal{V}$ sobre él. Para ello se emplean las funciones siguientes:

```
#include <semaphore.h>
int sem_post (sem_t *sem);
int sem_wait (sem_t *sem);
int sem_trywait (sem_t *sem);

#include <time.h>
int sem_timedwait (sem_t *sem, const struct timespec *abs_timeout);
```

La función `sem_post` desbloquea —operación $\mathcal{P}$ — el semáforo referenciado por `sem`. En el caso de que haya varios hilos esperando el desbloqueo del semáforo, se tendrá en cuenta la política de distribución definida —`SCHED_FIFO`, `SCHED_RR` u otra— a la hora de elegir el hilo a desbloquear.

La función `sem_wait` bloquea —operación $\mathcal{V}$ — el semáforo referenciado por `sem`. Como he descrito antes, el bloqueo del semáforo consiste en decrementarlo si no vale 0. En caso de que ya valga 0 el hilo detiene su ejecución indefinidamente hasta que otro hilo desbloquea el semáforo o hasta que una señal interrumpe la operación. La función `sem_trywait` también realiza una operación $\mathcal{V}$, pero si se encuentra el semáforo bloqueado devuelve el control inmediatamente sin detener al hilo que la ejecuta. Por último, la función `sem_timedwait` intenta realizar el bloqueo hasta que lo consigue o hasta que el reloj `CLOCK_REALTIME` alcanza la hora referenciada por `abs_timeout`.

Estas funciones devuelven 0 si se ejecutan satisfactoriamente y -1 en caso de error. Algunos códigos de error importantes que se deben tener en cuenta son:

EDEADLK Se ha detectado un posible interbloqueo o abrazo mortal.

EINTR La función termina por la recepción de una señal.

ETIMEDOUT No se ha podido bloquear el semáforo antes del plazo indicado.

El valor de un semáforo puede ser consultado con la función `sem_getvalue`:

```
#include <semaphore.h>
int sem_getvalue (sem_t *sem, int *sval);
```

Esta función devuelve en el entero referenciado por `sval` un valor positivo si el semáforo no está bloqueado, 0 cuando el semáforo está bloqueado y no hay hilos esperando a que quede desbloqueado, o un valor negativo cuyo valor absoluto representa el número de hilos que esperan el desbloqueo.

Una vez que hemos terminado de utilizar un semáforo y queremos liberar los recursos que ocupa recurriremos a alguna de las funciones siguientes:

```
#include <semaphore.h>
int sem_close (sem_t *sem);
int sem_unlink (const char *name);
int sem_destroy (sem_t *sem);
```

Con `sem_close` cerramos el semáforo nominal referenciado por `sem` que previamente hemos creado o abierto con `sem_open`. El nombre de este semáforo será borrado de la jerarquía de nombres con la función `sem_unlink`. Esta función no es de efecto inmediato ya que el sistema postpone la operación de borrado hasta que todos los procesos que utilizan el semáforo de nombre `name` dejan de usarlo. Si el semáforo es anónimo, lo eliminaremos con la función `sem_destroy`.

10.5.1 Críticas a los semáforos

Los semáforos son un mecanismo de sincronización de bajo nivel sencillo de utilizar pero propenso a errores porque una colocación inadecuada de las operaciones $\mathcal{P}$ y $\mathcal{V}$ puede bloquear nuestra aplicación. En los sistemas de tiempo real no sólo nos encontramos con requisitos respecto a la temporización de las aplicaciones sino también, y con igual importancia, con requisitos de fiabilidad. El objetivo que se persigue con los semáforos es conseguir *exclusión mutua* sobre una *sección crítica* y para ello se han propuesto aproximaciones más estructuradas que dejan los aspectos de sincronización en manos del sistema y no del programador. Una de estas aproximaciones son los monitores que estudiaremos en el § 10.7 pág. 405.

10.6 Cerrojos en POSIX

Si nuestro sistema soporta la biblioteca POSIX de hilos y nuestra aplicación se compone de un solo proceso con múltiples hilos, los cerrojos³ ofrecen las mismas posibilidades de sincronización que los semáforos binarios pero son menos costosos que éstos. La norma POSIX define un *cerrojo* como «*Un objeto de sincronización usado por los hilos para secuenciar sus accesos a los datos que comparten*». Tras esta definición están los conceptos de exclusión mutua y sección crítica. Como ya hemos visto en el § 5.5 pág. 167, los hilos de un proceso comparten un mismo espacio de direccionamiento —a excepción de la pila y el contexto de registros— y las variables globales pueden ser modificadas concurrentemente con el peligro de que se produzcan condiciones de carrera. Para evitar esta situación forzaremos que el acceso a los datos compartidos se haga en *exclusión mutua* y no pueda haber dos hilos modificando simultáneamente un mismo conjunto de datos. Las zonas de código donde se tienen que garantizar los accesos en exclusión mutua se denominan *secciones críticas*. Otra forma equivalente de definir estos objetivos es mediante el concepto de invariante. Una *invariante*⁴ es una relación que deben cumplir los datos de un proceso durante su ejecución; por ejemplo, el número de elementos de una cola de tamaño limitado será siempre menor o igual a su capacidad. Un proceso tiene que respetar sus invariantes, de lo contrario contendrá errores. Para garantizar las invariantes de un proceso, sus hilos tienen que manipular en exclusión mutua los datos implicados. Veamos ahora cómo construir secciones críticas con los cerrojos. Las funciones para inicializar, bloquear y desbloquear un cerrojo son:

```
#include <pthread.h>
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
int pthread_mutex_init (pthread_mutex_t *mutex,
                        const pthread_mutexattr_t *attr);
int pthread_mutex_destroy (pthread_mutex_t *mutex);
int pthread_mutex_lock (pthread_mutex_t *mutex);
int pthread_mutex_unlock (pthread_mutex_t *mutex);
int pthread_mutex_trylock (pthread_mutex_t *mutex);

#include <time.h>
int pthread_mutex_timedlock (pthread_mutex_t *mutex,
                             const struct timespec *abs_timeout);
```

Un cerrojo es un objeto de tipo `pthread_mutex_t` al que se le asocian unos atributos de tipo `pthread_mutexattr_t`. Para inicializar un cerrojo con los atributos por defecto podemos utilizar la macro `PTHREAD_MUTEX_INITIALIZER`.

La función `pthread_mutex_init` inicializa el cerrojo referenciado por `mutex` con los atributos referenciado por `attr`. Si `attr` vale `NULL` se utilizan los atributos por defecto. La biblioteca de hilos POSIX ha sido diseñada con un enfoque orientado a objetos, por lo

³También los podemos encontrar con el nombre *mutex* —contracción de *mutual exclusion*— o *exmut* para adaptarlo al español.

⁴**invariante.** f. *Mat.* Magnitud o expresión matemática que no cambia de valor al sufrir determinadas transformaciones; p. ej., la distancia entre dos puntos de un sólido que se mueve pero no se deforma. [RAE, 2001]

tanto, las únicas operaciones permitidas con los cerrojos son las que define la biblioteca y no tienen sentido operaciones como la copia de cerrojos mediante asignación. Entre las cosas que podemos hacer con los cerrojos están:

<code>pthread_mutex_lock</code>	Bloquea el cerrojo referenciado por <code>mutex</code> . Si el cerrojo ya estaba bloqueado por otro hilo entonces detiene el hilo actual y lo añade a la cola de hilos que esperan el desbloqueo del cerrojo.
<code>pthread_mutex_unlock</code>	Desbloquea el cerrojo referenciado por <code>mutex</code> . Si hay algún hilo esperando el desbloqueo del cerrojo, la política de distribución activa determina a quién se le concede el cerrojo.
<code>pthread_mutex_trylock</code>	Intenta bloquear el cerrojo referenciado por <code>mutex</code> pero si ya está bloqueado entonces no detiene al hilo que la llama sino que le devuelve el valor <code>EBUSY</code> .
<code>pthread_mutex_timedlock</code>	Intenta bloquear el cerrojo referenciado por <code>mutex</code> y si no lo consigue antes de que llegue la hora indicada en el parámetro <code>abs_timeout</code> —medida con respecto reloj <code>CLOCK_REALTIME</code> — devuelve el valor <code>ETIMEDOUT</code> .
<code>pthread_mutex_destroy</code>	Destruye el cerrojo referenciado por <code>mutex</code> y no se puede realizar sobre cerrojos bloqueados.

Todas estas operaciones devuelven 0 si se ejecutan satisfactoriamente o, en caso de fallo, el código del error producido.

La forma de transformar una secuencia de instrucciones en una sección crítica consiste esencialmente en protegerla con operaciones de bloqueo y desbloqueo de un cerrojo.

```
// Declaración de datos compartidos y el cerrojo que los protege
struct datos_compartidos {
    // Declaración de los datos
    Campos con la declaración de los datos
    // Declaración del cerrojo
    pthread_mutex_t cerrojo;
}datos;
...
void código_hilo_1 (void *arg)
{
    ...
    pthread_mutex_lock (&datos.cerrojo);
        // Sección crítica
        // Acceso a los datos compartidos
    pthread_mutex_unlock (&datos.cerrojo);
    ...
}
void código_hilo_2 (void *arg)
{
```



```

...
pthread_mutex_lock (&datos.cerrojo);
    // Sección crítica
    // Acceso a los datos compartidos
pthread_mutex_unlock (&datos.cerrojo);
...
}

void main ()
{
    pthread_t hilo_1, hilo_2;
    int error;
    ...
    // Inicialización del cerrojo
    error = pthread_mutex_init (&datos.cerrojo, NULL);
    if (error)
        // Tratamiento del error

    // Creación de los hilos
    error = pthread_create (&hilo_1, NULL, código_hilo_1, NULL);
    if (error) // Tratamiento del error
    error = pthread_create (&hilo_2, NULL, código_hilo_2, NULL);
    if (error) // Tratamiento del error
    ...
}

```

10.6.1 Atributos de un cerrojo

Al definir la interfaz de cerrojos se puso especial interés en facilitar una implementación mínima que fuese sencilla y eficiente. La sencillez se tradujo en asociarle a los cerrojos unos atributos por defecto con la semántica suficiente para realizar operaciones de bloqueo y desbloqueo nada más inicializarlos. La eficiencia dio lugar a eliminar la sobrecarga que supone dejar en manos del sistema la comprobación de errores. No obstante, aquellas aplicaciones que requieran un comportamiento más elaborado y seguro de los cerrojos pueden configurar sus atributos de acuerdo con sus necesidades.

Los atributos de un cerrojo se almacenan en objetos de tipo `pthread_mutexattr_t`, pueden ser inicializados a sus valores por defecto con la función `pthread_mutexattr_init` y destruidos con la función `pthread_mutexattr_destroy`:

```

#include <pthread.h>
int pthread_mutexattr_init (pthread_mutexattr_t *attr);
int pthread_mutexattr_destroy (pthread_mutexattr_t *attr);

```

Una vez inicializado un objeto con los atributos de un cerrojo podemos consultar su tipo con la función `pthread_mutexattr_gettype`. El tipo lo cambiaremos con la función `pthread_mutexattr_settype`:

```
#include <pthread.h>
int pthread_mutexattr_gettype (const pthread_mutexattr_t *attr, int *type);
int pthread_mutexattr_settype (pthread_mutexattr_t *attr, int type);
```

Los tipos definidos son:

- PTHREAD_MUTEX_NORMAL** Con este tipo de cerrojos el sistema no comprueba los interbloqueos. Así, si un hilo intenta bloquear un cerrojo que ya ha sido bloqueado por él mismo, se producirá un abrazo mortal. Además, los intentos para desbloquear un cerrojo bloqueado por otro hilo o un cerrojo desbloqueado producen un resultado indeterminado.
- PTHREAD_MUTEX_ERRORCHECK** Con este tipo de cerrojos el sistema comprueba los siguientes errores: intentos de bloquear un cerrojo que ya está bloqueado por el mismo hilo, intentos de desbloquear un cerrojo que ha sido bloqueado por otro hilo, e intentos de desbloquear un cerrojo que ya está desbloqueado.
- PTHREAD_MUTEX_RECURSIVE** Con este tipo se permite que un hilo realice varias operaciones de bloqueo sobre un mismo cerrojo sin que se produzca un abrazo mortal. Para que el cerrojo quede desbloqueado para otros hilos, habrá que emitir con posterioridad tantas operaciones de desbloqueo como operaciones de bloqueo se produjeron.
- PTHREAD_MUTEX_DEFAULT** Con este tipo el sistema no comprueba ninguno de los errores anteriores. Es el que se asigna por defecto a los atributos inicializados con `pthread_mutexattr_init`.

Los cerrojos encuentran su principal aplicación como herramienta de sincronización entre hilos de un mismo proceso. Sin embargo la norma también contempla la posibilidad de que hilos de diferentes procesos compartan cerrojos si así lo indicamos con la función `pthread_mutexattr_setpshared`:

```
#include <pthread.h>
int pthread_mutexattr_getpshared (const pthread_mutexattr_t *attr,
                                   int *pshared);
int pthread_mutexattr_setpshared (pthread_mutexattr_t *attr, int pshared);
```

Podremos determinar si nuestra biblioteca POSIX dispone de estas funciones consultando el símbolo `_POSIX_THREAD_PROCESS_SHARED` en el fichero de cabecera `<unistd.h>`. Los valores que puede tomar el argumento `pshared` son: `PTHREAD_PROCESS_SHARED` para indicar que el cerrojo puede ser compartido por hilos de diferentes procesos y `PTHREAD_PROCESS_PRIVATE` para indicar que el cerrojo sólo puede ser manipulado por los hilos de un mismo proceso.

Si utilizamos los cerrojos en la construcción de sistemas con requisitos de tiempo real es conveniente que prevengamos la inversión de prioridades y que acotemos su efecto con alguno de los protocolos de herencia estudiados en el § 5.7.3 pág. 180. Con este fin la norma define la función `pthread_mutexattr_setprotocol` para cambiar el protocolo de herencia de los atributos de un cerrojo y `pthread_mutexattr_getprotocol` para consultarlo:

```
#include <pthread.h>
int pthread_mutexattr_getprotocol (const pthread_mutexattr_t *attr,
                                   int *protocol);
int pthread_mutexattr_setprotocol (pthread_mutexattr_t *attr,
                                   int *protocol);
```

donde el argumento `protocol` referencia al protocolo y puede tomar los valores:

`PTHREAD_PRIO_NONE` Indica que no hay herencia de prioridades.
`PTHREAD_PRIO_INHERIT` Se emplea el protocolo de herencia simple.
`PTHREAD_PRIO_PROTEC` Se emplea el protocolo de herencia inmediata del techo de prioridad.

Para consultar o cambiar el techo de prioridad en los atributos de un cerrojo, en el caso de que se le haya asociado herencia inmediata del techo de prioridad, se emplean las funciones `pthread_mutexattr_getprioceiling` y `pthread_mutexattr_setprioceiling`:

```
#include <pthread.h>
int pthread_mutexattr_getprioceiling (const pthread_mutexattr_t *attr,
                                       int *prioceiling);
int pthread_mutexattr_setprioceiling (pthread_mutexattr_t *attr,
                                       int prioceiling);
```

Hay que recordar que los atributos de un cerrojo se definen antes de inicializarlo pero, en el caso del techo de prioridad, este atributo puede ser cambiado con posterioridad a la inicialización del cerrojo. Para ello se utiliza la función `pthread_mutex_setprioceiling`.

```
#include <pthread.h>
int pthread_mutex_setprioceiling (pthread_mutex_t *mutex,
                                  int prioceiling, int *old_ceiling);
int pthread_mutex_getprioceiling (const pthread_mutex_t *mutex,
                                  int *prioceiling);
```

Para realizar el cambio, la función `pthread_mutex_setprioceiling` bloquea el cerrojo referenciado por `mutex` y programa su techo de prioridad con el valor indicado en `prioceiling`. En la referencia `old_ceiling` se devuelve el valor anterior del techo. Si la función no puede bloquear el cerrojo por estarlo ya por otro hilo, entonces el hilo actual se detiene hasta que el cerrojo queda desbloqueado. Para consultar el valor del techo de prioridad emplearemos la función `pthread_mutex_getprioceiling`.

Todas las funciones que manipulan los atributos devuelven 0 si se ejecutan satisfactoriamente o un código de error en caso de que fallen.

Los cerrojos se pueden utilizar en solitario como primitivas de sincronización con una semántica idéntica a los semáforos binarios y por ello pueden recibir las mismas críticas que estos —consulte el § 10.5.1 pág. 399—, sin embargo encuentran su mayor utilidad como apoyo a las variables de condición para la construcción de monitores.

10.7 Monitores con interfaz de procedimiento

Los monitores se vienen empleando desde principios de los años 70 como herramientas para estructurar los sistemas operativos. Hoare define un monitor como «*una colección de datos administrativos y sus procedimientos asociados, diseñados para controlar el acceso a un recurso compartido*» [Hoare, 1974]. El monitor es un refinamiento del concepto de región crítica donde no sólo nos limitamos a asegurar la ejecución en exclusión mutua de la región sino que la entrada en la misma está controlada por predicados asociados al estado de los datos. Un *predicado* es una expresión lógica que tomará los valores VERDADERO o FALSO tras su evaluación y que en el caso de los monitores está formada por las variables de control del mismo. Si retomamos el ejemplo de la cola de tamaño limitado —consulte el § 10.6 pág. 400—, podemos definir un monitor con las operaciones **poner** y **quitar** para añadir o eliminar elementos de la cola; estas operaciones se tienen que realizar en exclusión mutua ya que modifican el estado de la cola y hay que garantizar las invariantes; además, la operación para **poner** un elemento sólo será aceptada cuando se cumpla el predicado «*hay sitio en la cola*» y la operación para **quitar** un elemento sólo se aceptará si «*hay elementos en la cola*». Si los predicados no son ciertos la operación no se interrumpe, simplemente queda suspendida hasta que cambie el estado de los datos y el predicado se cumpla. Esto requiere que se introduzca un nuevo mecanismo de sincronización que permita *esperar* cuando un predicado no se cumple y *señalar*⁵ que el estado de los datos cambia para comprobar de nuevo si se cumple o no el predicado. En el ejemplo de las colas que venimos empleando, la operación **poner** esperará cuando la cola esté llena, y cuando sea aceptada y añada un nuevo elemento, señalará que ya hay elementos. Con la operación **quitar** haremos algo parecido, primero esperará si la cola está vacía y, cuando sea aceptada y elimine un elemento, señalará que hay sitio para añadir. El pseudocódigo siguiente muestra la forma que tiene el monitor que gestiona una cola de tamaño limitado:

```
monitor cola {
    // Definición de los datos de control de la cola:
    Array donde se almacenan los elementos de la cola.
    unsigned número_elementos = 0; // Número de elementos de la cola.
    unsigned capacidadCola = Número máximo de elementos de la cola;
    Índices para controlar dónde se ponen y se quitan elementos.

    // Variables de condición asociadas a los predicados.
    condición hay_sitio, hay_elementos;

    // Operaciones
    operación poner {
```

⁵Empleo los términos *esperar* y *señalar* porque son los utilizados en las interfaces que definen variables de condición —*wait* y *signal*—.

```

    while (número_elementos == capacidadCola)
        esperar (hay_sitio);
    // Operaciones para manipular la cola y poner un nuevo elemento al final
    // de la misma.
    ...
    señalar (hay_elementos);
}
operación quitar {
    while (número_elementos == 0)
        esperar (hay_elementos);
    // Operaciones para manipular la cola y quitar el elemento que haya
    // al principio.
    ...
    señalar (hay_sitio);
}
}

```

Para la implementación práctica de los monitores es necesario disponer de un nuevo tipo de objetos de sincronización conocidos como *variables de condición* sobre las que podemos realizar las operaciones **esperar** y **señalar**. En el pseudocódigo anterior podemos observar que la entrada a cada operación se inicia con la evaluación de una condición o predicado que en el caso de no cumplirse nos lleva a **esperar** una variable de condición. Cuando alguien *señala* esa variable, la función **esperar** devuelve el control y nuevamente se procede a evaluar el predicado; si éste es cierto entramos en la sección crítica que manipula los datos. El sincronismo para garantizar el acceso en exclusión mutua a los datos está implícito en la definición de monitor y veremos que en algunos casos prácticos se emplea un cerrojo. Pasemos ahora a estudiar las variables de condición de POSIX.

10.7.1 Variables de condición en POSIX

Las *variables de condición* son objetos de tipo `pthread_cond_t` que «le permiten a los hilos suspender su ejecución repetidas veces hasta que sea cierto un predicado asociado» [IEEE, 2001a]. Para inicializar una variable de condición se puede utilizar la función `pthread_cond_init` o la macro `PTHREAD_COND_INITIALIZER`:

```

#include <pthread.h>
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
int pthread_cond_destroy(pthread_cond_t *cond);

```

Mientras que `pthread_cond_init` inicializa la variable de condición referenciada por `cond` con los atributos referenciados por `attr`, la macro `PTHREAD_COND_INITIALIZER` inicializa con los valores por defecto que están pensados para que pueda usarse la variable con la semántica descrita en el apartado anterior. La función `pthread_cond_destroy` la utilizaremos para liberar los recursos de una variable de condición una vez que dejemos de necesitarla.

Las operaciones básicas de *esperar* y *señalar* una variable son `pthread_cond_wait` y `pthread_cond_signal`:

```
#include <pthread.h>
int pthread_cond_wait (pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_signal (pthread_cond_t *cond);
```

Vemos que la operación `pthread_cond_wait` no sólo recibe como argumento la referencia `cond` a la variable por la que se espera, sino también la referencia `mutex` a un cerrojo, que es empleado para garantizar la exclusión mutua cuando se ejecuta alguna de las operaciones del monitor. La forma habitual de escribir estas operaciones es la siguiente:

```
pthread_mutex_t crj = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
...
tipooperación nombreoperación (argumentos)
{
    ...
    pthread_mutex_lock (&cerrojo);
    while (predicado)
        pthread_cond_wait (&cond, &crj);
    // Inicio de la sección crítica.
    ...
    // Señalar otras condiciones, si las hay.
    // Fin de la sección crítica.
    pthread_mutex_unlock (&crj);
    ...
    return valortipooperación;
}
```

La llamada `pthread_cond_wait(&cond,&crj)` desbloquea el cerrojo `crj` y se pone a esperar la señalización de la condición `cond`. Estas dos acciones se realizan de forma atómica y permiten que el cerrojo quede desbloqueado para que otra operación del monitor pueda bloquear el cerrojo, modificar los datos y señalar la condición. Cuando esto ocurre, la función `pthread_cond_wait` devuelve el control con el cerrojo bloqueado, lo que nos permite volver a evaluar el `predicado` dentro de una sección crítica. Si el `predicado` es cierto no se vuelve a llamar a `pthread_cond_wait` sino que se ejecuta el resto de las instrucciones de la sección crítica. Antes de terminar la operación, nada más salir de la sección crítica, no nos olvidaremos de desbloquear el cerrojo, de lo contrario el monitor quedaría bloqueado.

Como ejemplo de aplicación de los monitores veamos la forma de escribir un tipo abstracto de datos para gestionar colas genéricas con prioridades. Los tipos abstractos quedan definidos mediante sus operaciones, su estructura es oculta para el usuario. En este ejemplo, las colas son objetos del tipo `cola_t` y las operaciones permitidas son `INICIALIZAR_LA_COLA`, `poner_en_la_cola` y `quitar_de_la_cola`. En el fichero de cabecera `colas.h` podemos ver la definición del tipo y sus operaciones.

Fichero 10.6: Definición del tipo abstracto cola (colas.h)

```

#ifndef _COLAS_H_
#define _COLAS_H_
#include <sys/types.h>
#include <pthread.h>

/* Tipo de los elementos de la cola. Éste es el tipo que emplea el usuario.*/
typedef char elem_t;

/* Elementos de la cola. Éste es el tipo empleado internamente en la codificación de
las operaciones y, como vemos, se le asocia una prioridad a cada elemento que se
pone en la cola.*/
struct elem_c {
    unsigned prio;
    elem_t *elem;
};

typedef struct cola {
    unsigned capacidad;
    size_t tam_elem;
    struct elem_c *ptr_base;
    int primero;
    int ultimo;
    unsigned num_elem;
    pthread_mutex_t cerrojo;
    pthread_cond_t hay_elementos;
    pthread_cond_t hay_sitio;
}cola_t;

#define INICIALIZAR_LA_COLA(capacidad, tipo_elem) {\
    (capacidad),\
    sizeof (tipo_elem),\
    (struct elem_c *)reservar_memoria (capacidad, sizeof(tipo_elem)),\
    0,\
    0,\
    0,\
    PTHREAD_MUTEX_INITIALIZER,\
    PTHREAD_COND_INITIALIZER,\
    PTHREAD_COND_INITIALIZER\
}

int poner_en_la_cola (cola_t *cola, elem_t *elem, unsigned prio);
int quitar_de_la_cola (cola_t *cola, elem_t *elem, unsigned *prio);
#endif

```

Las colas son de tamaño fijo y pueden almacenar objetos de cualquier tipo. Así, para definir una cola con capacidad para 10 cadenas de caracteres de longitud máxima 20 escribiremos:

```
typedef char cadenas[21];
...
cola_t cola_cadenas = INICIALIZAR_LA_COLA (10, cadenas);
```

La cola está construida internamente como un monitor y permite la comunicación entre hilos, pero esos detalles se le ocultan al usuario. Lo único que necesita conocer el programador que emplee este tipo es que puede poner elementos al final de la cola con la operación `poner_en_la_cola` y quitar elementos del principio con la operación `quitar_de_la_cola`. Al ser una cola con prioridades, el elemento que se extrae es el de mayor prioridad y, en caso de igualdad, el más antiguo. En el fichero `colas.c` podemos ver el código de estas operaciones.

Fichero 10.7: Operaciones del tipo abstracto cola (`colas.c`)

```
#include "colas.h"
#include "errores.h"
#include <errno.h>

/****
    reservar_memoria: reserva memoria para la cola en función de su capacidad y
    del tamaño de los elementos.
****/
struct elem_c *reservar_memoria (unsigned capacidad, size_t tam_elem)
{
    int i;
    struct elem_c *ptr_elem;

    /* Reserva de memoria para la cola. */
    ptr_elem = (struct elem_c *) malloc (capacidad*sizeof(struct elem_c));
    if (ptr_elem == NULL)
        error_fatal (errno, "malloc");

    /* Reserva de memoria para los elementos. */
    for (i = 0; i < capacidad; i++) {
        ptr_elem[i].elem = (elem_t *) malloc (tam_elem);
        if (ptr_elem[i].elem == NULL)
            error_fatal (errno, "malloc");
    }
    return ptr_elem;
}

/****
    poner_en_la_cola: introduce un elemento en la cola.
****/
```



```

int poner_en_la_cola (cola_t *cola, elem_t *elem, unsigned prio)
{
    int error;

    error = pthread_mutex_lock (&cola->cerrojo);
    if (error)
        error (error, "pthread_mutex_lock");
    /* Esperamos a que haya sitio en la cola. */
    while (cola->num_elem == cola->capacidad) {
        error = pthread_cond_wait (&cola->hay_sitio, &cola->cerrojo);
        if (error)
            error (error, "pthread_cond_wait");
    }
    /* Ya hay sitio. Ponemos el elemento en la cola. */
    cola->ptr_base[cola->ultimo].prio = prio;
    memcpy (cola->ptr_base[cola->ultimo].elem, elem, cola->tam_elem);
    cola->ultimo = (cola->ultimo+1) % cola->capacidad;
    ++cola->num_elem;

    /* Indicamos que ya hay elementos en la cola. */
    error = pthread_cond_signal (&cola->hay_elementos);
    if (error)
        error (error, "pthread_cond_signal");
    /* Salimos de la sección crítica. */
    error = pthread_mutex_unlock (&cola->cerrojo);
    if (error)
        error (error, "pthread_mutex_unlock");
    return 0;
}

/****
quitar_de_la_cola: saca de la cola el elemento de mayor prioridad. Si hay
varios de igual prioridad, saca el más antiguo.
****/
int quitar_de_la_cola (cola_t *cola, elem_t *elem, unsigned *prio)
{
    int error;
    int i, j, max;
    struct elem_c aux;

    error = pthread_mutex_lock (&cola->cerrojo);
    if (error)
        error (error, "pthread_mutex_lock");
    /* Esperamos a que haya algún elemento en la cola. */
    while (cola->num_elem == 0) {
        error = pthread_cond_wait (&cola->hay_elementos, &cola->cerrojo);

```

```
        if (error)
            error (error, "pthread_cond_wait");
    }

    /* Ya hay algún elemento. Buscamos el elemento de mayor prioridad. */
    i = max = cola->primero;
    do {
        if (cola->ptr_base[i].prio > cola->ptr_base[max].prio)
            max = i;
        i = (i+1)%cola->capacidad;
    } while (i != cola->ultimo);

    /* Extraemos el elemento. */
    memcpy (elem, cola->ptr_base[max].elem, cola->tam_elem);
    *prio = cola->ptr_base[max].prio;

    if (max == cola->primero)
        cola->primero = (cola->primero+1) % cola->capacidad;
    else {
        /* Recolocamos el resto de elementos para que no queden huecos en la cola. */
        i = max;
        j = (max+1)%cola->capacidad;
        aux = cola->ptr_base[i];
        /* Esta recolocación está acotada en el tiempo y no depende del tamaño del
        elemento ya que no movemos elementos sino enlaces a elementos. */
        while (j != cola->ultimo) {
            cola->ptr_base[i] = cola->ptr_base[j];
            i = j;
            j = (j+1)%cola->capacidad;
        }
        cola->ptr_base[i] = aux;
        cola->ultimo = i;
    }
    --cola->num_elem;

    /* Indicamos que ya hay sitio libre en la cola. */
    error = pthread_cond_signal (&cola->hay_sitio);
    if (error)
        error (error, "pthread_cond_signal");
    /* Salimos de la sección crítica. */
    error = pthread_mutex_unlock (&cola->cerrojo);
    if (error)
        error (error, "pthread_mutex_unlock");
    return 0;
}
```

Todas las funciones que trabajan con variables de condición devuelven el valor 0 si se ejecutan satisfactoriamente o un código de error en caso de que fallen. En la implementación del módulo `colas.c` hemos empleado la macro `error` para imprimir un mensaje asociado a ese código y devolver el control sin abortar la ejecución del hilo; se deja en manos de éste emprender alguna acción para recuperarse del fallo o terminar su ejecución. El código de la macro `error` es una variante de la macro `error_fatal` explicado en el § 6.7 pág. 216:

```
#define error(codigo, texto) do {\
    fprintf (stderr, "%s:%d: ERROR: %s - %s\n",\
        __FILE__, __LINE__, texto, strerror (codigo));\
    return(codigo);\
} while (0)
```

Para probar el funcionamiento del módulo anterior podemos utilizar el programa `prueba-cola` donde se crean dos hilos —`emisor` y `receptor`— que se comunican a través de una cola. Los hilos se intercambian mensajes de igual prioridad que constan de un número de secuencia y una cadena de caracteres. Algunos mensajes —los urgentes— tienen una prioridad más elevada y por lo tanto el `receptor` los extrae antes de la cola.

Programa 10.8: Comunicación entre hilos (`prueba-colas.c`)

```
/**
$ prueba-colas

Programa para probar el módulo colas.c
Compilación:
cc -o prueba-colas prueba-colas.c colas.c -pthread (-lpthread en Linux)
***/
#include <stdio.h>
#include <errno.h>
#include <time.h>
#include "errores.h"
#include "colas.h"

typedef struct {
    int sec;
    char datos[11];
} mensaje_t;

void *cod_del_emisor (void *arg)
{
    cola_t *cola = (cola_t *)arg;
    struct timespec retardo = {0, 250000000}; /* 250 ms. */
    int i;
    int error;
    mensaje_t msg;
```

```

    for (i = 1; i <= 100; i++) {
        msg.sec = i;
        /* Uno de cada 10 mensajes lleva prioridad más alta. */
        if (i%10 == 0)
            strncpy (msg.datos, "URGENTE", sizeof (msg.datos));
        else
            strncpy (msg.datos, "normal", sizeof (msg.datos));
        error = poner_en_la_cola (cola, (elem_t *)&msg, i%10 == 0 ? 2:1);
        if (error)
            error_fatal (error, "poner_en_la_cola");
        error = nanosleep (&retardo, NULL);
        if (error == -1)
            error_fatal (errno, "nanosleep");
    }
    ++msg.sec;
    strncpy (msg.datos, "FIN", sizeof (msg.datos));
    poner_en_la_cola (cola, (elem_t *)&msg, 0);
    printf ("Fin del emisor.\n");
    pthread_exit (0);
}

void *cod_del_receptor (void *arg)
{
    cola_t *cola = (cola_t *)arg;
    struct timespec retardo = {1, 0}; /* 1000 ms. */
    int prio;
    int error;
    mensaje_t msg;

    do {
        quitar_de_la_cola (cola, (elem_t *)&msg, &prio);
        if (error)
            error_fatal (error, "quitar_de_la_cola");
        printf ("Mensaje %d %s con prioridad %d\n",
                msg.sec, msg.datos, prio);
        error = nanosleep (&retardo, NULL);
        if (error == -1)
            error_fatal (errno, "nanosleep");
    } while (strcmp (msg.datos, "FIN") != 0);
    printf ("Fin del receptor.\n");
    pthread_exit (0);
}

main (int argc, char *argv[])
{
    cola_t cola = INICIALIZAR_LA_COLA (32, mensaje_t);

```

```

pthread_t emisor, receptor;
int error;

/* Creación de los hilos. */
error = pthread_create (&emisor,NULL,cod_del_emisor,(void *)&cola);
if (error)
    error_fatal (error, "pthread_create");
error = pthread_create(&receptor,NULL,cod_del_receptor,(void *)&cola);
if (error)
    error_fatal (error, "pthread_create");

/* Esperamos la terminación de los hilos. */
error = pthread_join (emisor, NULL);
if (error)
    error_fatal (error, "pthread_join");
error = pthread_join (receptor, NULL);
if (error)
    error_fatal (error, "pthread_join");

printf ("Fin del hilo principal.\n");
}

```

Aparte de las operaciones básicas de espera y señalización, la norma POSIX define la función `pthread_cond_timedwait` para realizar una espera acotada en el tiempo y `pthread_cond_broadcast` para *difundir* el cambio de una variable de condición y despertar a todos los hilos que estén esperando por ella.

```

#include <pthread.h>
int pthread_cond_timedwait (pthread_cond_t *cond, pthread_mutex_t *mutex,
                           const struct timespec *abstime);
int pthread_cond_broadcast(pthread_cond_t *cond);

```

Es importante dejar clara la diferencia entre las dos funciones que indican el cambio en una variable de condición: mientras que `pthread_cond_signal` despierta al menos uno de los hilos que están esperando por la variable de condición, `pthread_cond_broadcast` los despierta a todos, pero sólo uno consigue bloquear el cerrojo y por lo tanto la evaluación del predicado sigue ejecutándose en exclusión mutua; no obstante todos los hilos compiten por el cerrojo al despertar y esto acarrea cambios de contexto —tal vez innecesarios—. Podríamos pensar que `pthread_cond_broadcast` es una generalización de `pthread_cond_signal`, de hecho siempre se puede sustituir la señalización por la difusión aunque con una pérdida de prestaciones porque tal vez se despierten hilos de forma innecesaria. Por lo general tendremos que acudir a la difusión cuando un cambio en la condición permita que varios hilos reinicien su actividad; es el caso, por ejemplo, del problema de los *lectores-redactores* donde sólo uno de los hilos puede escribir en los datos compartidos pero varios hilos pueden leer a la vez ya que la lectura no implica un cambio del estado de los datos [Ben-Ari, 1990].

10.7.2 Atributos de las variables de condición

Al definir la interfaz de variables de condición se puso especial interés en asociarles unos atributos por defecto con la semántica suficiente para realizar las operaciones *esperar*, *señalar* y *difundir* nada más inicializarlas. Sin embargo, podemos inicializarlas con unos atributos que modifiquen su comportamiento básico. Los atributos de una variable de condición son objetos de tipo `pthread_condattr_t` que se inicializan con la función `pthread_condattr_init` y cuyos recursos se liberan con `pthread_condattr_destroy`:

```
#include <pthread.h>
int pthread_condattr_init (pthread_condattr_t *attr);
int pthread_condattr_destroy (pthread_condattr_t *attr);
```

Las variables de condición encuentran su principal aplicación como herramienta de sincronización entre hilos de un mismo proceso. Sin embargo la norma también contempla la posibilidad de que hilos de diferentes procesos compartan variables de condición si así lo indicamos con la función `pthread_condattr_setpshared`:

```
#include <pthread.h>
int pthread_condattr_getpshared (const pthread_condattr_t *attr,
                                int *pshared);
int pthread_condattr_setpshared (pthread_condattr_t *attr, int pshared);
```

Podremos determinar si nuestra biblioteca POSIX dispone de estas funciones consultando el símbolo `_POSIX_THREAD_PROCESS_SHARED` en el fichero de cabecera `<unistd.h>`. Los valores que puede tomar el argumento `pshared` son: `PTHREAD_PROCESS_SHARED` para indicar que la variable puede ser compartida por hilos de diferentes procesos —esto requiere que los cerrojos que usemos con esta variable en operaciones de *espera* también tengan este atributo activo—; y `PTHREAD_PROCESS_PRIVATE` para indicar que la variable sólo puede ser manipulada por los hilos de un mismo proceso.

Por último, si nuestro sistema implementa las opciones avanzadas de tiempo real, podemos consultar o indicar el identificador del reloj con respecto al cual se miden los plazos de la función `pthread_cond_timedwait`. Para ello se emplean las funciones `pthread_condattr_getclock` y `pthread_condattr_setclock`:

```
#include <pthread.h>
int pthread_condattr_getclock (const pthread_condattr_t *attr,
                              clockid_t *clock_id);
int pthread_condattr_setclock (pthread_condattr_t *attr,
                              clockid_t clock_id);
```

Todas las funciones que manipulan los atributos devuelven 0 si se ejecutan satisfactoriamente o un código de error en caso de que fallen.

10.8 Ejercicios

10.1 Con las funciones de bloqueo de ficheros se puede implementar la operatividad de los semáforos, pero con el inconveniente de que el acceso a los mismos será lento.

Haciendo uso de la función `lockf` —consulte el § 3.6 pág. 87—, implemente las funciones `P` y `V` que nos permitan gestionar un semáforo binario construido a partir del sistema de ficheros. Sus declaraciones serán:

```
int P (char *ruta);
int V (char *ruta);
```

Estas funciones deben tener la operatividad descrita para las acciones $\mathcal{P}$ y $\mathcal{V}$ de los semáforos. El nombre de fichero apuntado por `ruta` se interpretará como el nombre del semáforo al que se refieren las acciones.

10.2 Haciendo uso de las funciones `P` y `V` del ejercicio anterior, implemente un programa en el que un proceso padre y otro hijo se pasen el turno a la hora de acceder al flujo estándar de salida, de tal forma que cada uno de ellos escriba de forma alternativa un mensaje por pantalla. Cada proceso debe completar un total de 10 mensajes. ¿Cuál es el número mínimo de semáforos necesario para que el programa funcione correctamente?

10.3 Modificar el programa del ejemplo que aparece en el § 10.2.5 pág. 377 y escriba el programa `sem2` donde se sincronicen `n` procesos. La forma de invocar al programa debe ser:

```
$ sem2 n
```

donde `n` es el número de procesos que se deben crear, incluido el padre. Cada proceso debe escribir en pantalla su *PID* y pasarle el control al proceso siguiente bloqueando y liberando los semáforos necesarios. La salida que se producirá debe ser parecida a la siguiente:

```
$ sem2 4
```

```
Proceso 1, pid = 245  La secuencia básica se repite 10 veces
Proceso 2, pid = 246
Proceso 3, pid = 247
Proceso 4, pid = 248
...
Proceso 1, pid = 245
Proceso 2, pid = 246
Proceso 3, pid = 247
Proceso 4, pid = 248
```

¿Cuál es el mínimo número de semáforos necesarios para escribir este programa?

10.4 Un problema típico que necesita mecanismos de sincronización para poder ser resuelto es el conocido como problema de los *productores-consumidores* con exclusión mutua. Su enunciado, resumido, es el siguiente:

- (a) Un conjunto de usuarios —productores— producen cierta clase de objetos que depositan en un almacén.

- (b) Un conjunto de usuarios —consumidores— retiran objetos del almacén para su posterior consumo.
- (c) El almacén tiene una capacidad máxima de N objetos.
- (d) El almacén tiene un único punto de carga y un único punto de descarga, por lo que dos o más productores no pueden depositar sus objetos simultáneamente en el almacén. De igual forma, dos o más consumidores no pueden retirar simultáneamente objetos del almacén.

Para modelar este tipo de sistemas concurrentes se suele emplear un grafo bipartido conocido como red de Petri. En la figura 10.4 se puede ver la estructura y la interpretación de una red de Petri que modela el sistema compuesto por dos productores, un consumidor y un almacén con una capacidad de 4 elementos.

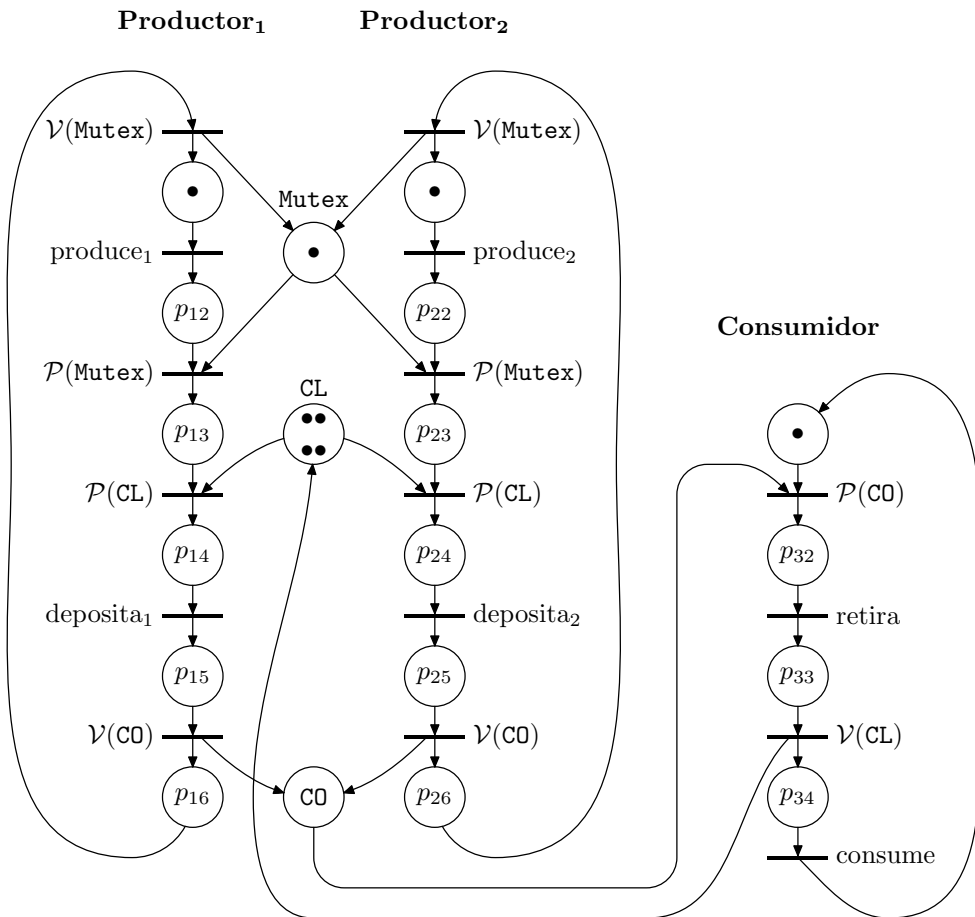


Figura 10.4: Red de Petri que modela el sistema productor-consumidor.

La interpretación de esta red es la siguiente:

- Lugares
 - **Mutex**, semáforo binario que garantiza la exclusión mutua de los productores al acceder al almacén.
 - **C0**, semáforo que modela el número de compartimentos ocupados en el almacén.
 - **CL**, semáforo que modela el número de compartimentos libres del almacén.
- Transiciones
 - **P(X)** Operación $\mathcal{P}$ sobre el semáforo **X**; **X** puede tomar los valores **Mutex**, **CL** o **C0**.
 - **V(X)** Operación $\mathcal{V}$ sobre el semáforo **X**; **X** puede tomar los valores **Mutex**, **CL** o **C0**.

La siguiente presentación informal puede servirle de ayuda a aquellos lectores que no estén familiarizados con la terminología de las redes de Petri, en las referencias [Murata, 1989] y [Silva, 1985] se puede encontrar una exposición detallada sobre la teoría y las aplicaciones de las redes de Petri. Una Red de Petri —en lo sucesivo RdP— es un grafo orientado con dos clases de nudos:

- *Lugares o plazas*: se representan con circunferencias y simbolizan los estados.
- *Transiciones*: se representan con barras y simbolizan los cambios de estado.

Los arcos unen lugares con transiciones y viceversa, pero nunca lugares entre sí o transiciones entre sí —se dice que las RdP son grafos bipartidos—. Para la descripción de sistemas se suele interpretar la red con las siguientes asociaciones:

- A los lugares se les asocian estados o acciones —salidas— del sistema.
- A las transiciones se les asocian eventos —funciones lógicas de las variables de entrada— y acciones.

En la figura 10.5 podemos ver un ejemplo de RdP.

La descripción anterior corresponde a la estructura de la red, veamos ahora su *comportamiento dinámico*. Un lugar puede contener un número natural o nulo de *marcas* que se representan con puntos. El conjunto de marcas de la red constituye el marcado. Cada marcado representa un estado interno del sistema. Así, si el número de marcados posibles es infinito, una RdP puede modelar un autómata no finito. El marcado evoluciona según las siguientes reglas:

- Una transición está sensibilizada si todos sus lugares de entrada están marcados.
- Una transición sensibilizada puede dispararse cuando se verifica el evento que tiene asociado.
- El disparo de una transición consiste en retirar una marca de cada lugar de entrada y añadir una marca a cada lugar de salida —véase la figura 10.6—.

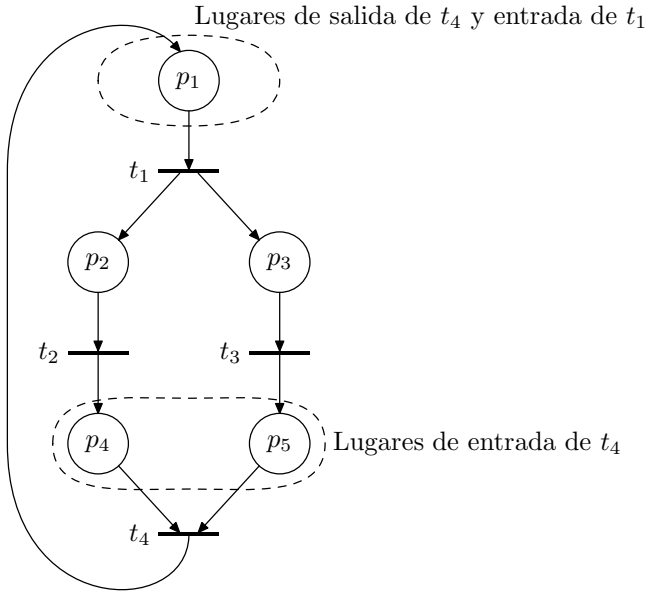


Figura 10.5: Estructura de una RdP.

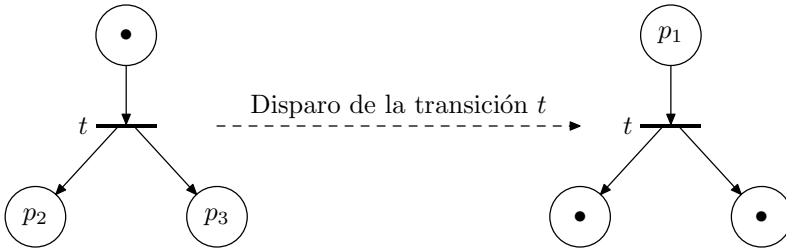


Figura 10.6: Disparo de una transición. Arcos con peso igual a la unidad.

Si los arcos tienen un peso superior a la unidad, la transición está sensibilizada cuando todos sus lugares de entrada tienen un número de marcas igual o superior al peso de su arco asociado. La transición se dispara quitando tantas marcas de los lugares de entrada como indique el peso de sus arcos y sumando tantas marcas a sus lugares de salida como indiquen sus arcos —véase la figura 10.7—.

Teniendo en cuenta la introducción anterior, se pide construir tres programas para modelar un sistema de *productores-consumidores*. Estos programas tendrán los nombres y forma de uso siguientes:

\$ almacén capacidad

\$ productor m período

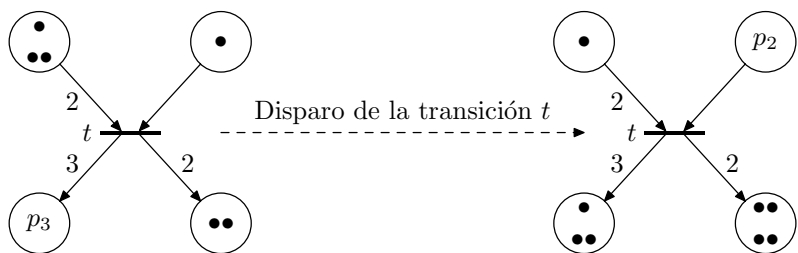


Figura 10.7: Disparo de una transición. Arcos con peso superior a la unidad.

\$ consumidor n período

El programa `almacén` debe crear un almacén con el número de compartimentos indicando en el parámetro `capacidad`. La operación de crear el almacén consistirá en declarar cuatro semáforos cuyos nombres y valores iniciales se muestra en la tabla 10.1.

Tabla 10.1: Semáforos de la aplicación *productores–consumidores*.

Semáforo	Valor	Descripción
Mutex_productores	1	Garantiza el acceso en exclusión mutua al almacén por parte de los productores.
Mutex_consumidores	1	Garantiza el acceso en exclusión mutua al almacén por parte de los consumidores.
CL	capacidad	Representa el número de compartimentos libres del almacén.
CO	0	Representa el número de compartimentos ocupados del almacén.

Cuando el programa termine su ejecución, no debe borrar estos semáforos, para que así los puedan utilizar los procesos *productor* y *consumidor*.

Los programas `productor` y `consumidor` sirven para arrancar procesos que realicen los ciclos básicos de producción y consumo. Estos ciclos los podemos ver en la tabla 10.2.

A medida que cada uno de estos procesos pase por estos estados, deberá sacar por pantalla un mensaje indicando su *PID* y el estado en el que se encuentra. El parámetro `m` del programa `productor` representa el número de objetos que se van a depositar en el almacén en cada operación de depósito. El parámetro `n` del programa `consumidor` representa el número de objetos que se van a extraer del almacén en cada operación de extracción. El parámetro `período` de los programas `productor` y `consumidor` representa el tiempo que transcurre en cada ciclo *producir–depositar*

Tabla 10.2: Ciclos básicos de los productores y los consumidores.

Productor	Consumidor
punto inicial; produce; P(Mutex_productores); P(CL); deposita; V(CO); V(Mutex_productores); volver al punto inicial;	punto inicial; P(Mutex_consumidores); P(CO); extrae; V(CL); V(Mutex_consumidores); consume; volver al punto inicial;

o *extraer-consumir*. A continuación podemos ver algunas formas de invocar a estos programas:

```
$ almacén 8
```

```
$ productor 2 15 &
```

```
$ productor 3 10 &
```

```
$ consumidor 1 10 &
```

```
$ consumidor 3 10 &
```

¿Qué forma tiene la RdP que modela un sistema de *productores-consumidores* como el que se ha creado con las órdenes anteriores?

Capítulo 11

Comunicaciones en red

«Paréceme, Sancho, que esto destas redes debe de ser una de las más nuevas aventuras que pueda imaginar.»

(Cervantes, Quijote II-58)

11.1 Mecanismos <i>IPC</i> del sistema BSD	423
11.2 Llamadas para el manejo de conectores	430
11.3 Ejemplos de servidores y clientes	439
11.4 Miscelánea de llamadas y funciones	464
11.5 Ejemplo de aplicación. Transferencia de ficheros	467
11.6 Ejercicios	483

11.1 Mecanismos *IPC* del sistema BSD

Este capítulo está destinado a dar una visión sobre el uso de la interfaz de comunicación entre procesos del sistema BSD. Esta interfaz permitirá comunicar procesos que se estén ejecutando bajo el control de una misma máquina o bajo el control de máquinas distintas. En el segundo caso es necesario que entren en juego redes de conexión entre ordenadores.

Siempre que se habla de una red de ordenadores hay que distinguir entre protocolos y servicios de la red. En realidad, un estudio exhaustivo de las redes debe empezar por el modelo de referencia *OSI* [ISO, 1984]. Éste es un modelo que estructura en siete capas los diferentes elementos que intervienen en las comunicaciones. Cada una de las capas ofrece una serie de servicios a la capa superior y utiliza los servicios que le brinda la capa inferior. Los servicios de una capa pueden verse como una interfaz de comunicación con la misma. Los protocolos, por su parte, definen la forma que tienen las tramas de bits que se intercambian las distintas capas y cómo se lleva a cabo ese intercambio. El objetivo que se persigue al separar protocolos de servicios es aislar los aspectos tecnológicos de los aspectos de uso de una red. Así, se procura definir servicios lo menos cambiantes posible con objeto de que los usuarios no tengan que estar modificando sus aplicaciones continuamente. Por otro lado, las mejoras en las tecnologías de las redes repercuten en

un diseño de protocolos que garanticen una transmisión más fiable, segura y rápida. Esto complicará los protocolos, pero no los servicios.

Para un estudio completo sobre las capas del modelo de referencia *OSI* y los protocolos y servicios que propone, se puede consultar [Tanenbaum, 2002]. En la figura 11.1 podemos ver las siete capas que propone el modelo. Éstas son:

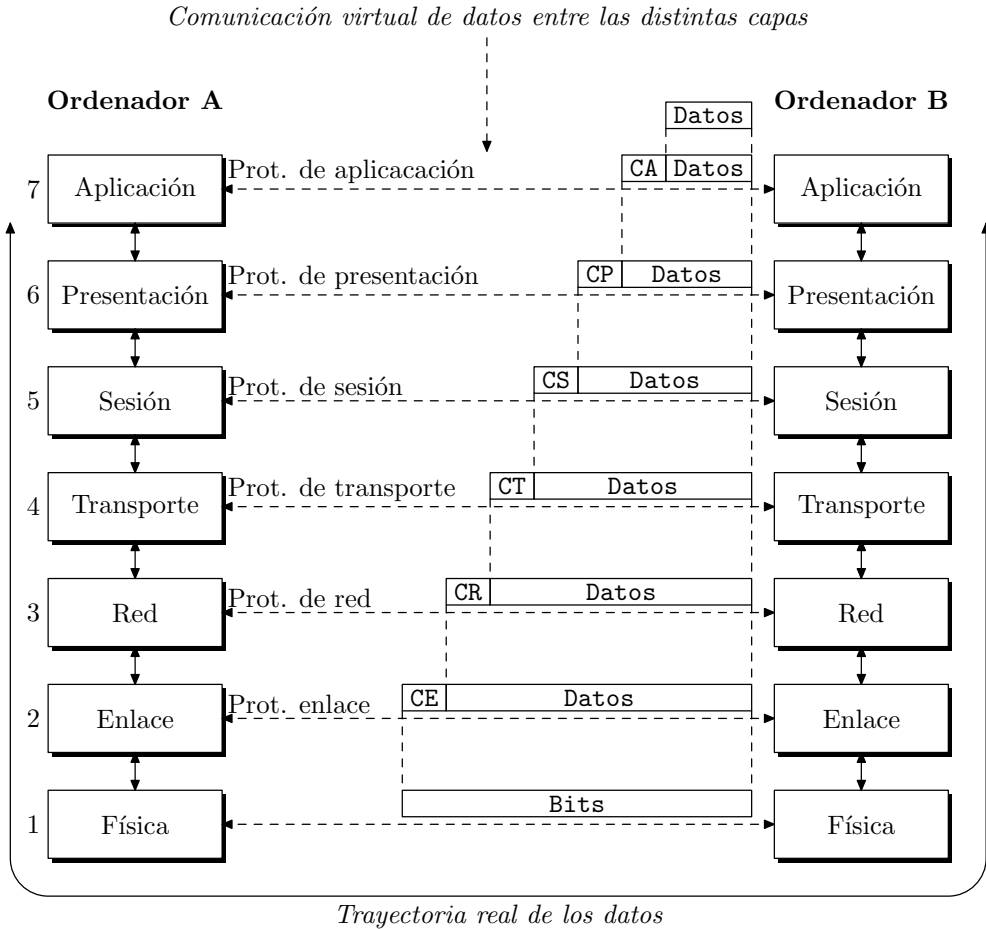


Figura 11.1: Estructura en capas del modelo *OSI* para comunicación entre ordenadores.

1. *Capa física.* Se encarga de la transmisión de bits a lo largo de un canal de comunicación.
2. *Capa de enlace.* La tarea principal de la capa de enlace consiste en transformar el medio de transmisión, que por lo general es ruidoso, en una línea sin errores, y por lo tanto fiable, para la capa de red —capa inmediatamente superior a ésta en la jerarquía—.

3. *Capa de red.* Se ocupa de la obtención de paquetes procedentes de la fuente y de encaminarlos por la red y las subredes hasta alcanzar su destino.
4. *Capa de transporte.* La función principal de la capa de transporte consiste en aceptar los datos de la capa de sesión, dividirlos, siempre que sea necesario, en unidades más pequeñas, pasarlos por la capa de red y asegurar que todos ellos lleguen correctamente al otro extremo.
5. *Capa de sesión.* Permite que los usuarios de diferentes máquinas puedan establecer sesiones de trabajo en máquinas remotas. Un ejemplo de sesión consiste en acceder a un sistema remoto en tiempo compartido y conectarnos a trabajar con él, tal y como hacemos con nuestro sistema local —`rlogin` o `telnet`—. Otro ejemplo puede ser transferir ficheros entre dos ordenadores —`rcp` bajo UNIX—.
6. *Capa de presentación.* Se ocupa de los aspectos de sintaxis y semántica de la información que se transmite. Un ejemplo típico de servicio de la capa de presentación es la codificación de los datos de acuerdo con unas normas. La información que un ordenador envía a otro ocupa unidades tales como caracteres, números enteros o números en coma flotante, y puede que en una red haya ordenadores que representen estas unidades de diferente forma. Por ejemplo, un ordenador envía un nombre codificado en caracteres *ASCII* mientras que el ordenador que lo recibe codifica los caracteres en *EBCDIC*; la capa de presentación se va a encargar de realizar los cambios necesarios de *ASCII* a *EBCDIC* para que el nombre recibido sea inteligible y no una secuencia de símbolos sin significado.
7. *Capa de aplicación.* La capa de aplicación contiene una variedad de protocolos que se necesitan frecuentemente. Un problema típico a resolver en esta capa es la compatibilización de los distintos terminales que hay en el mercado, con objeto de que los programas de aplicación no tengan que preocuparse de conocer las características hardware de los mismos. El sistema de ventanas X-WINDOW es un ejemplo de solución a este problema.

El modelo *OSI* fue aceptado inmediatamente en los ambientes académicos por su elegancia y estructuración, sin embargo no ocurrió lo mismo con la industria. Fue muy fácil llenar de contenidos los niveles inferiores del modelo, en gran parte porque ya existía un fuerte desarrollo en cuanto a medios de transmisión y redes de comunicación. Las dificultades vinieron a la hora de elaborar normas sobre protocolos y servicios para los niveles superiores. La industria necesita normas para abordar con ciertas garantías económicas el desarrollo de nuevos productos; en nuestro caso estos productos son casi siempre aplicaciones informáticas para la gestión y la explotación de las redes. La competitividad industrial exige ciclos de desarrollo cada vez más cortos y, ante una carencia normativa respecto a protocolos y servicios de los niveles superiores del modelo *OSI*, fueron surgiendo diferentes propuestas que, al margen del modelo de referencia, daban respuesta a las necesidades más acuciantes en cada momento para continuar con el desarrollo de redes cada vez más amplias y con mayores capacidades.

Éste fue el entorno en el que se produjo la aparición de *Internet*, una red de ámbito mundial que interconecta redes de diferentes tecnologías —cable, fibra óptica, inalámbr-

cas, etc.— y diferentes ámbitos —locales, metropolitanas, nacionales, etc.— a lo largo de todo el planeta.

Para crear esta red se definieron los protocolos *IP* —*Internet Protocol*— que permite interconectar diferentes redes, *TCP* —*Transmission Control Protocol*— y *UDP* —*User Datagram Protocol*— que permiten crear aplicaciones que usen la red.

El modelo de 4 capas definido para *Internet* —véase la figura 11.5— se ha convertido en el estándar de facto aceptado por la industria, en detrimento del modelo *OSI*. Se puede decir que la ambición del modelo *OSI* y su incapacidad de dar una respuesta normativa a tiempo produjeron que se optara por una solución práctica tal vez menos elegante pero más útil a corto plazo.

No obstante, el marco conceptual del modelo *OSI* sigue teniendo validez no sólo para el estudio de las redes de ordenadores sino también para el estudio de cualquier estructura compleja. La idea subyacente a este modelo —que también está presente en el modelo en 4 capas de *Internet*— es estructurar los sistemas complejos en diferentes niveles de abstracción y complejidad y estudiar en cada uno de ellos sus aspectos más relevantes. Esta idea está tomada de la teoría general de sistemas y es la forma habitual con la que la ciencia aborda el estudio de sistemas complejos. Por ello no es de extrañar que a lo largo del capítulo me refiera indistintamente al modelo *OSI* o al modelo *Internet*, teniendo presente que en cualquiera de los dos casos me estoy refiriendo a un modelo estructurado en capas concebido para el estudio de las redes de ordenadores.

Teniendo esto presente, hay que decir que la interfaz con la red que ofrece el sistema BSD corresponde al nivel 4 —capa de transporte— del modelo *OSI* que es la última capa de modelo de *Internet*. Por lo tanto, dos programas que se comuniquen usando esta interfaz se despreocuparán totalmente de aspectos tales como:

- Canal físico de comunicación, tipo de transmisión —analógica o digital—, tipo de modulación —*PSK*, *FSK*, ...—: todo esto corresponde a la capa física.
- Forma de codificar las señales para disminuir la probabilidad de error en la transmisión: corresponde a la capa de enlace.
- Nodos de la red por los que tienen que pasar las tramas de bits y gestión de las redes involucradas: corresponde a la capa de red.
- Formar paquetes a partir de la información a transmitir y buscar una ruta que una el ordenador origen con el destino: corresponde a la capa de transporte.

11.1.1 Protocolos y conexiones

Éste no es un libro sobre redes de ordenadores, por ello no voy a adentrarme en la descripción de las capas *OSI* o *Internet*, ni a nivel de servicios ni de protocolos. Sin embargo, no querría dejar pasar la oportunidad de aclarar algunos de los términos o acrónimos que aparecerán en discusiones posteriores.

Lo primero que me gustaría dejar claro es que la comunicación mediante conectores es una interfaz¹ con la capa de transporte —nivel 4— de la jerarquía *OSI*. Los protocolos de

¹*Interfaz y servicio* son dos términos que utilizo indistintamente para referirme al mismo concepto. Se trata del conjunto de funciones que cada capa del esquema *OSI* le ofrece al usuario.

los niveles inferiores no los vamos a estudiar; en primer lugar, porque su exposición sería demasiado larga y, en segundo lugar, porque no es necesario. La filosofía de la división por capas de un sistema es encapsular, dentro de cada una de ellas, detalles que conciernen sólo a la capa, y presentársela al usuario como una caja negra con unas determinadas entradas y salidas, de tal forma que el usuario pueda trabajar con ella sin necesidad de conocer sus detalles de implementación.

La interfaz de acceso a la capa de transporte del sistema BSD no está totalmente aislada de las capas inferiores, por lo que a la hora de trabajar con conectores es necesario conocer algunos detalles sobre esas capas. En concreto, a la hora de establecer una conexión, es necesario conocer la familia o dominio de la conexión y el tipo de conexión.

Una *familia* agrupa todos aquellos conectores que comparten características comunes, tales como protocolos, convenios para formar direcciones de red, convenios para formar nombres, etc. Más adelante mencionaremos algunas de las familias más empleadas.

El *tipo* de conexión indica el tipo de circuito que se va a establecer entre los dos procesos que se están comunicando. El circuito puede ser virtual² —orientado a conexión— o *datagrama* —no orientado a conexión—. Para establecer un circuito virtual se realiza una búsqueda de enlaces libres que unan los dos ordenadores a conectar. Es algo parecido a lo que hace la red telefónica conmutada para establecer una conexión entre dos teléfonos. Una vez establecida la conexión, se puede proceder al envío secuencial de los datos, ya que la conexión es permanente. Por el contrario, los *datagramas* no trabajan con conexiones permanentes. La transmisión por los *datagramas* es a nivel de paquetes, donde cada paquete puede seguir una ruta distinta y no se garantiza una recepción secuencial de la información.

11.1.2 Direcciones de red

A la hora de referirse a un nodo de la red, cada protocolo implementa un mecanismo de direccionamiento. La dirección distingue de forma inequívoca a cada nodo u ordenador y es utilizada para encaminar los datos desde el nodo origen al nodo destino. La forma de construir direcciones depende de los protocolos que se empleen en la capa de transporte y de red, por lo que no vamos a detenernos en ello. Sin embargo, hay muchas llamadas al sistema BSD que necesitan un puntero a una estructura de dirección de conector para trabajar. Esta estructura se define en el fichero de cabecera `<sys/socket.h>` y su forma es la siguiente:

```
struct sockaddr {
    u_short sa_family; // Familia de conectores. Se emplean las constantes
                        // de la forma AF_xxx que veremos más adelante
    char sa_data[14]; // 14 bytes que contiene la dirección. Su significado
                      // depende de la familia de conectores que se esté empleando
};
```

El contenido de los 14 bytes del campo `sa_data` depende de la familia de conectores que se esté usando. Así, si usamos una familia que emplea protocolos *Internet*, la forma de las direcciones de red será la definida en el fichero de cabecera `<netinet/in.h>`:

²Se hablará de conector —*socket*— del tipo corriente o flujo de datos —*stream*—.

```

struct in_addr {
    u_long s_addr; // 32 bits con la identificación de la red y del nodo
};

struct sockaddr_in {
    short sin_family; // AF_INET
    u_short sin_port; // 16 bits con el número de puerto
    struct in_addr sin_addr; // 32 bits con la identificación de
                            // la red y del nodo
    char sin_zero [8]; // 8 bytes no usados
};

```

Podemos ver que en la dirección *Internet* el campo `sin_family` es equivalente al campo `sa_family` de la dirección genérica y que los campos `sin_port`, `sin_addr` y `sin_zero` cumplen la misma función que el campo `sa_addr`.

La familia de conectores conocida como *dominio Unix* emplea direcciones con la forma definida en `<sys/un.h>`:

```

struct sockaddr_un {
    short sun_family; // AF_UNIX
    char sun_path [108]; // Ruta
};

```

Estas direcciones se corresponden en realidad con rutas de ficheros y su longitud —110 bytes— es superior a los 16 bytes que de forma estándar tienen las direcciones del resto de familias. Esto es posible debido a que esta familia de conectores se utiliza para comunicar procesos que se están ejecutando bajo el control de una misma máquina, por lo que no necesitan acceder a la red.

11.1.3 Modelo cliente-servidor

El modelo cliente-servidor es uno de los más empleados para construir aplicaciones en una red. Al hablar de las bases de datos centralizadas —consulte el § 10.4.4 pág. 391— vimos una aplicación de este modelo. En aquella ocasión, todos los procesos involucrados se ejecutaban bajo el control de una misma máquina y el canal de comunicación entre ellos eran las colas de mensajes. El modelo que veremos a continuación es más general, aunque las ideas esenciales son las mismas.

Un *servidor* es un proceso que se está ejecutando en un nodo³ de la red y que gestiona el acceso a un determinado recurso. Un *cliente* es un proceso que se ejecuta en el mismo o en diferente nodo y que realiza peticiones al servidor. Las peticiones están originadas por la necesidad de acceder al recurso que gestiona el servidor.

El servidor está continuamente esperando peticiones de servicio. Cuando se produce una petición, el servidor despierta y atiende al cliente. Cuando el servicio concluye, el servidor vuelve al estado de espera. De acuerdo con la forma de prestar el servicio, podemos considerar dos tipos de servidores:

³Por *nodo* entendemos cualquier ordenador o periférico que esté conectado a la red.

- *Servidores interactivos*. El servidor no sólo recoge la petición de servicio, sino que él mismo se encarga de atenderla. Esta forma de trabajo presenta un inconveniente: si el servidor es lento en atender a los clientes y hay una demanda de servicio muy elevada, se originarán unos tiempos de espera muy grandes.
- *Servidores concurrentes*. El servidor recoge cada una de las peticiones de servicio y crea otros procesos para que se encarguen de atenderlas. Este tipo de servidores sólo es aplicable en sistemas multiproceso, como es UNIX. La ventaja que tiene este tipo de servicio es que el servidor puede recoger peticiones a muy alta velocidad, porque está descargado de la tarea de atención al cliente. En aquellas aplicaciones donde los tiempos de servicio son variables, es recomendable implementar servidores del tipo concurrente.

11.1.4 Esquema general de un servidor y de un cliente

La forma general que tienen los diagramas de flujo de control, tanto de los procesos servidores como de los clientes, se puede ver en la figura 11.2. Las acciones que debe llevar a cabo el programa servidor son las siguientes:

1. Abrir el canal de comunicaciones e informar a la red tanto de la dirección a la que responderá como de su disposición para aceptar peticiones de servicio.
2. Esperar a que un cliente le pida servicio en la dirección que él tiene declarada.
3. Cuando recibe una petición de servicio, si es un servidor interactivo, atenderá al cliente. Los servidores interactivos se suelen implementar cuando la respuesta que necesita el cliente es sencilla e implica poco tiempo de proceso. Si el servidor es concurrente, creará un proceso mediante `fork` para que le dé servicio al cliente.
4. Volver al punto 2 para esperar nuevas peticiones de servicio.

El programa cliente, por su parte, llevará a cabo las siguientes acciones:

1. Abrir el canal de comunicaciones y conectarse a la dirección de red atendida por el servidor. Naturalmente, esta dirección debe ser conocida por el cliente y responderá al esquema de generación de direcciones de la familia de conectores que se esté empleando.
2. Enviar al servidor un mensaje de petición de servicio y esperar hasta recibir la respuesta.
3. Cerrar el canal de comunicaciones y terminar la ejecución.

Los esquemas anteriores son aplicables de forma general, independientemente de la interfaz con la capa de transporte que empleen el servidor y el cliente. Si nos fijamos en los servicios del sistema BSD, la secuencia de llamadas al sistema que deben realizar ambos procesos dependerá del tipo de conexión que se establezca entre ellos. En la figura 11.3 podemos ver las llamadas y la secuencia de las mismas que deben realizar el servidor y el cliente cuando la conexión es virtual.

En la figura 11.4 vemos la secuencia de llamadas necesarias para comunicar servidor y cliente cuando no hay conexión y el envío de datos es mediante *datagramas*.

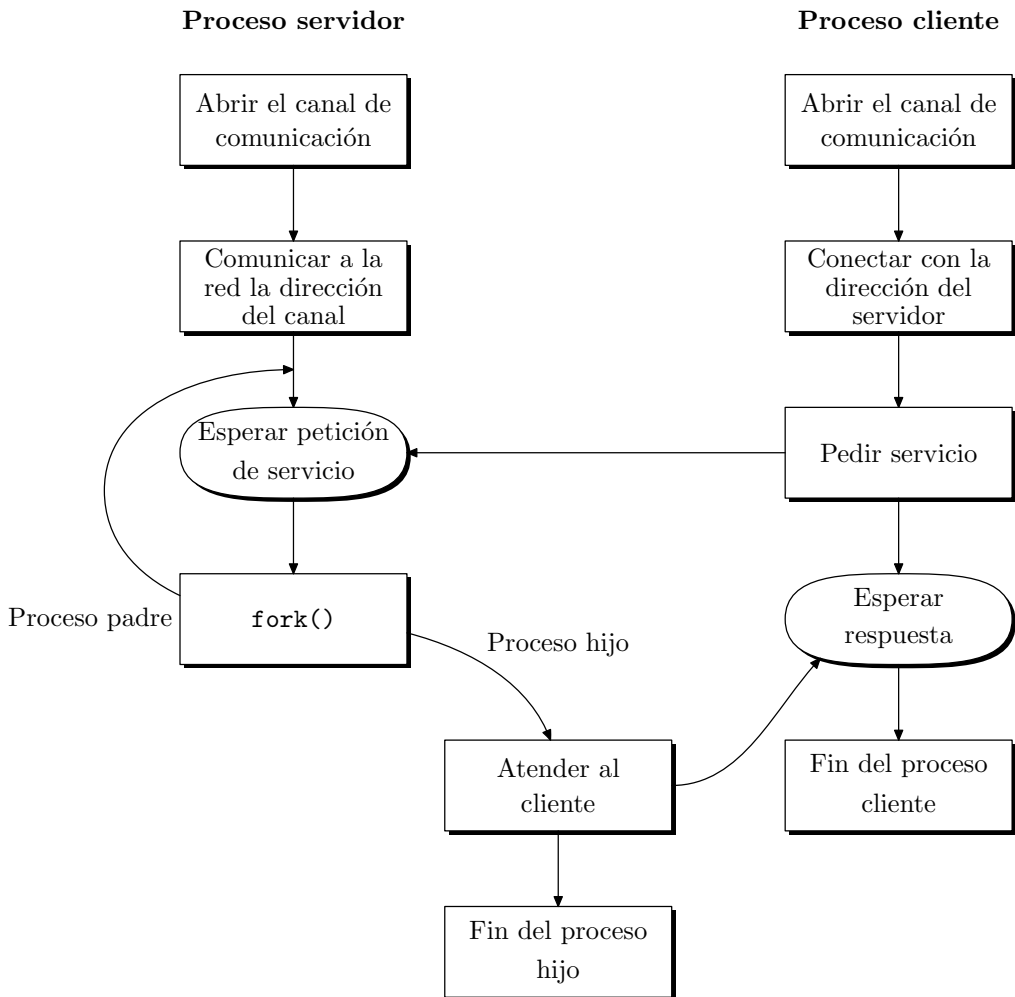


Figura 11.2: Estructura de los programas servidores y clientes.

11.2 Llamadas para el manejo de conectores

A continuación veremos detalladamente cada una de las llamadas al sistema que intervienen en la codificación de programas servidores y clientes que se comunican mediante conectores.

Hay que hacer notar que la interfaz de Berkeley se ha codificado para que sea muy similar a la del sistema de ficheros. Así, vamos a trabajar con descriptores de conectores y de ficheros, y algunas de las llamadas estudiadas para manejar ficheros estarán disponibles también para manejar conectores.

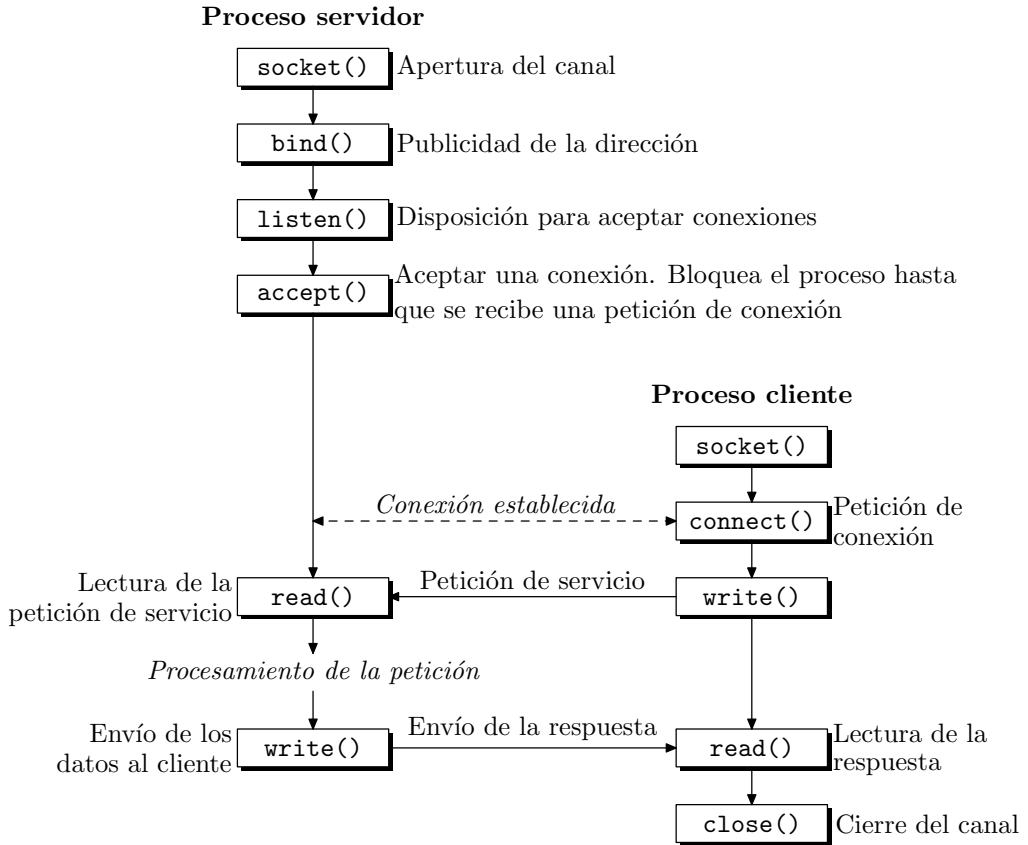


Figura 11.3: Secuencia de llamadas para una comunicación orientada a conexión.

11.2.1 Apertura de un punto terminal en un canal (socket)

La llamada para abrir un canal bidireccional de comunicaciones es `socket` y se declara como sigue:

```
#include <sys/types.h>
#include <sys/socket.h>
int socket (int af, int type, int protocol);
```

La llamada crea un punto terminal para conectarse a un canal y devuelve un descriptor. El descriptor del conector devuelto se usará en llamadas posteriores a funciones de la interfaz. El parámetro `af` —*address family*— especifica la familia de conectores o familia de direcciones⁴ que se desea emplear.

⁴En los sucesivos emplearé como sinónimos los términos *familia de conectores* y *familia de direcciones*. En algunos manuales del sistema también se refieren a este parámetro como *familia de protocolos*.

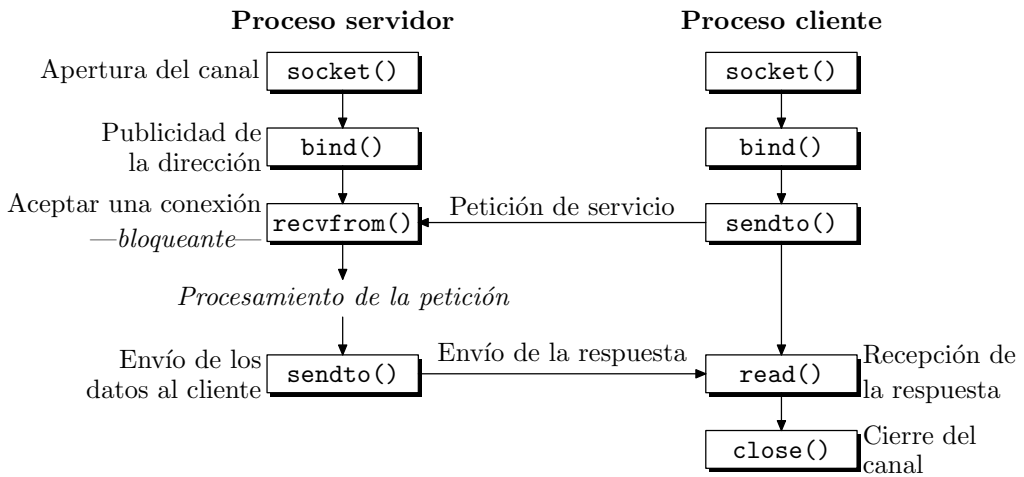


Figura 11.4: Secuencia de llamadas para una comunicación no orientada a conexión.

Las distintas familias están definidas en el fichero de cabecera `<sys/socket.h>` y dependerán del fabricante del sistema y de la configuración del hardware. Las dos familias siguientes suelen estar presentes en todos los sistemas:

- **AF_UNIX**⁵, protocolos internos UNIX. Es la familia de conectores empleada para comunicar procesos que se ejecutan en una misma máquina. Esta familia no requiere que esté presente un hardware especial de red puesto que, en realidad, no realiza accesos a ninguna red.
- **AF_INET**, protocolos *Internet*. Es la familia de conectores que se comunican mediante protocolos, tales como *TCP* —*Transmission Control Protocol*—, desarrollado por la Universidad de California en Berkeley para *DARPA* —*Defense Advance Research Projects Agency*— o *UDP* —*User Datagram Protocol*—.

En lo que resta de capítulo me voy a referir sólo a las familias **AF_UNIX** y **AF_INET**, no obstante, en la tabla 11.1 podemos ver una relación ampliada de familias.

El parámetro **type** indica la semántica de la comunicación para el conector y puede tomar los valores:

- **SOCK_STREAM**, conector con un protocolo orientado a conexión. Esto es lo que hemos estudiado como circuito virtual.
- **SOCK_DGRAM**, conector con un protocolo no orientado a conexión o *datagrama*.
- **SOCK_RAW**, conector que proporciona acceso directo a los protocolos internos de la red. Este tipo de conector ha sido concebido para los desarrolladores de protocolos que necesitan un acceso directo a los detalles internos de la red, por ello sólo pueden ser utilizados por usuarios con privilegios de superusuario.

⁵Estas constantes, al igual que las de la tabla 11.1, tienen la forma **AF_x**. También nos las podemos encontrar con la forma **PF_x** con el mismo significado. Por ejemplo, **PF_UNIX** es lo mismo que **AF_UNIX**.

Tabla 11.1: Familias de direcciones.

Nombre (antiguo)	Descripción
AF_LOCAL	es otro nombre para AF_UNIX
AF_INET	protocolos <i>Internet</i> DARPA — <i>TCP/IP</i> —
AF_IMPLINK	antigua interfaz de enlace 1822 <i>Interface Message Processor</i>
AF_PUP	antigua red Xerox
AF_CHAOS	red <i>Chaos</i> del MIT
AF_NS	arquitectura <i>Xerox Network System</i> — <i>XNS</i> —
AF_ISO	protocolos <i>OSI</i>
AF_ECMA	red <i>European Computer Manufactures</i>
AF_DATAKIT	red <i>Datakit</i> de AT&T
AF_CCITT	protocolos del CCITT, por ejemplo <i>X.25</i>
AF_SNA	<i>System Network Architecture</i> — <i>SNA</i> — de IBM
AF_DECnet	red DEC
AF_DLI	interfaz directa de enlace
AF_LAT	interfaz de terminales de red de área local
AF_HYLINK	<i>Network System Corporation Hyperchannel</i>
AF_APPLETALK	red <i>AppleTalk</i>
AF_ROUTE	comunicación con la capa de encaminamiento del núcleo
AF_LINK	acceso a la capa de enlace
AF_XTP	<i>eXpress Transfer Protocol</i>
AF_COIP	<i>Connection-oriented IP</i> — <i>ST II</i> —
AF_CNT	<i>Computer Network Tecnology</i>
AF_IPX	protocolo <i>Internet</i> de Novell

- **SOCK_SEQPACKET**, conector con un protocolo no orientado a conexión pero que proporciona un envío fiable y secuencial para *datagramas* con una longitud máxima fija. Actualmente sólo está implementado para los conectores de la familia **AF_NS**.
- **SOCK_RDM**, conector con un protocolo no orientado a conexión y envío fiable y secuencial de paquetes. Actualmente no está implementado pero puede ser simulado en el nivel de usuario [Leffler *et al.*, 1993].

El parámetro **protocol** especifica el protocolo particular que se va a usar con el conector. Normalmente, cada tipo de conector tiene asociado sólo un protocolo, pero si hubiera más de uno, se especificaría mediante este argumento. Si **protocol** toma el valor 0 la elección del protocolo se deja en manos del sistema.

Si la llamada se ejecuta satisfactoriamente devolverá un descriptor de fichero válido, en caso contrario devolverá -1 y en **errno** estará codificado el error producido.

11.2.2 Nombre de un conector (bind)

La llamada **bind** se utiliza para unir un conector con una dirección de red determinada, su declaración es la siguiente:


```
#include <sys/socket.h>
#include <sys/un.h> // Sólo para la familia AF_UNIX
#include <sys/netinet.h> // Sólo para la familia AF_INET
int bind (int sfd, const void *addr, int addrlen);
```

Cuando se crea un conector con la llamada `socket`, se le asigna una familia de direcciones, pero no una dirección particular, `bind` hace que el conector cuyo descriptor es `sfd` se una a la dirección de conector especificada en la estructura apuntada por `addr`, `addrlen` indica el tamaño de la dirección.

El formato de la dirección depende de la familia del conector, por lo que habrá que utilizar la estructura adecuada. Para la familia `AF_UNIX` se debe usar la estructura `struct sockaddr_un` y para la familia `AF_INET` la estructura `struct sockaddr_in`. El valor de `addrlen` se puede calcular con el operador `sizeof`.

La semántica utilizada en la unión del conector con la dirección depende de la familia. Así, por ejemplo, cuando se une un conector `AF_UNIX` a una ruta —por ejemplo, `/tmp/misocket`—, se crea un fichero con ese nombre. Cuando se cierra el conector, el fichero sigue existiendo hasta que se dé la orden de borrarlo. Si se une un conector `AF_INET` a una dirección, el campo `sin_port` puede contener el número de un puerto o 0. Si vale 0, el sistema le asigna el número de un puerto libre.

Si la llamada funciona correctamente devuelve el valor 0, en caso contrario, devuelve -1 y en `errno` estará el código del error producido.

11.2.3 Disponibilidad para recibir peticiones de servicio (`listen`)

Cuando se abre un conector orientado a conexión, el programa servidor indica que está disponible para recibir peticiones de conexión mediante la llamada a `listen`, que se declara como sigue:

```
int listen (int sfd, int backlog);
```

La llamada a `listen` la suele ejecutar el proceso servidor después de las llamadas a `socket` y `bind`; `listen` habilita una cola asociada al conector descrito por `sfd`, esta cola se utiliza para alojar peticiones de conexión procedentes de los procesos cliente, la longitud de esta cola es la especificada en el argumento `backlog`. Para que la llamada a `listen` tenga sentido, el conector debe ser del tipo `SOCK_STREAM` —conector orientado a conexión—.

La cola de conexiones es importante para los servidores de tipo interactivo, porque mientras están atendiendo a un cliente pueden llegar peticiones procedentes de otros. En los de tipo concurrente, el único instante en el que el servidor no atiende la recepción de peticiones es el que dedica a la llamada `fork`. En estos casos también es importante la cola de conexiones.

Si la llamada a `listen` funciona correctamente devuelve el valor 0, en caso contrario devuelve -1.

11.2.4 Petición de conexión (connect)

Para que un proceso cliente establezca una conexión con un servidor es necesario que haga una llamada a **connect**. Esta función se declara de la forma siguiente:

```
#include <sys/socket.h>
#include <sys/un.h> // Sólo para la familia AF_UNIX
#include <sys/netinet.h> // Sólo para la familia AF_INET
int connect (int sfd, const void *addr, int addrlen);
```

donde **sfd** es el descriptor del conector que da acceso al canal, **addr** es un puntero a una estructura que contiene la dirección del conector remoto al que queremos conectarnos y **addrlen** es el tamaño en bytes de la dirección. La estructura de la dirección dependerá de la familia de conectores con la que estemos trabajando.

Si el conector es del tipo **SOCK_DGRAM**, **connect** especifica la dirección del conector remoto al que se le enviarán los mensajes, pero no se conecta con él. La llamada devuelve el control inmediatamente. Además, a través del conector sólo se podrán recibir mensajes procedentes de la dirección especificada.

Si el conector es del tipo **SOCK_STREAM**, **connect** intenta contactar con el ordenador remoto con objeto de realizar una conexión entre el conector remoto y el conector local especificado en **sfd**. La llamada normalmente permanece bloqueada hasta que la conexión se completa. Por el contrario, si la conexión no se puede realizar de forma inmediata, pero el conector tiene activo el modo de acceso **O_NDELAY** —activado a través de una llamada a **fcntl**—, la llamada a **connect** devuelve el control inmediatamente indicando que se ha producido un error de conexión.

Si la conexión se realiza satisfactoriamente, la llamada devuelve el valor 0; en caso contrario, devolverá -1 y en **errno** estará el código de error producido.

11.2.5 Aceptación de una conexión (accept)

Los procesos servidores podrán leer peticiones de servicio mediante la llamada **accept**:

```
#include <sys/socket.h>
int accept (int sfd, void *addr, int *addrlen);
```

Esta llamada se usa con conectores orientados a conexión, como el tipo **SOCK_STREAM**. El argumento **sfd** es un descriptor del conector creado por una llamada previa a **socket** y unido a una dirección mediante **bind**. La llamada **accept** extrae la primera petición de conexión que hay en la cola de peticiones pendientes, creada con una llamada previa a **listen**. Una vez extraída la petición, **accept** crea un nuevo conector con las mismas propiedades que **sfd** y reserva un nuevo descriptor de fichero —**nsfd**— para él.

Si no hay peticiones de conexión pendientes y el conector no tiene activo el modo de acceso no bloqueante —**O_NDELAY**—, **accept** permanece bloqueada hasta que se reciba una nueva petición de conexión. Si el modo no bloqueante está activo, **accept** devolverá el control inmediatamente e indicará que se ha producido un error.

El conector original —**sfd**— permanece abierto y puede aceptar nuevas conexiones; sin embargo, el conector recién creado —**nsfd**— no puede usarse para aceptar más conexiones.

La llamada **select** —consulte el § 9.6.3 pág. 357— puede usarse para determinar si un conector tiene pendiente alguna petición de conexión.

El argumento **addr** debe apuntar a una estructura local con la dirección del conector. La llamada **accept** rellenará esa estructura con la dirección del conector remoto que pide la conexión. El formato de la estructura de dirección dependerá del tipo de conector que se esté manejando. El argumento **addrlen** debe ser un puntero a **int**. Inicialmente, debe contener el tamaño en bytes de la estructura de dirección. La función sobrescribirá en **addrlen** el tamaño real de la dirección leída de la cola de conexiones.

Si la llamada funciona correctamente, devolverá un número entero no negativo que se debe interpretar como un descriptor del conector aceptado, en caso de error devolverá el valor **-1**.

11.2.6 Lectura o recepción de mensajes de un conector

Una vez que el canal de comunicación entre los procesos servidor y cliente está correctamente inicializado y ambos procesos disponen de un conector con el canal, contamos con 5 llamadas al sistema para leer datos/mensajes de un conector y otras 5 llamadas para escribir datos/mensajes en él.

Las llamadas para leer datos son: **read**, **readv**, **recv**, **recvfrom** y **recvmsg**. La llamada **read** no la vamos a describir, porque ya vimos cuál era su interfaz al manejar el sistema de ficheros. De cara al manejo de conectores, la llamada **read** se comporta de la misma forma, pero con la salvedad de que el descriptor de fichero que utiliza es en realidad un descriptor del conector. Ésta es una de las ventajas que tiene la interfaz del BSD: que las llamadas para el manejo de ficheros siguen siendo válidas para el manejo de conectores.

La llamada **readv** es una generalización de **read** y puede utilizarse para leer datos de cualquier descriptor, ya sea fichero o conector. Su declaración es la siguiente:

```
#include <sys/uio.h>
ssize_t readv (int fildes, const struct iovec *iov, size_t iovcnt);
```

Esta llamada lee datos procedentes del fichero descrito por **fildes** y los distribuye entre las distintas zonas de memoria intermedia especificadas por el array **iov**. El total de elementos de **iov** viene indicado por el argumento **iovcnt**; así, **iov** describe un total de **iovcnt** memorias intermedias que son: **iov[0]**, **iov[1]**, ..., **iov[iovcnt-1]**.

Cada elemento del array **iov** es del tipo **struct iovec**, que se define con los siguientes campos:

```
struct iovec {
    caddr_t iov_base; // Dirección inicial de la memoria intermedia
    int iov_len; // Tamaño, en bytes, de la memoria intermedia
};
```

A medida que lee datos del fichero, **readv** irá rellenando las distintas memorias intermedias, empezando por la primera. Si llega al final del fichero o completa los bloques de memoria, termina su ejecución, devolviendo el número total de bytes leídos.

Si bien **read** y **readv** pueden leer de cualquier descriptor —descriptor de fichero o de conector—, las llamadas **recv**, **recvfrom** y **recvmsg** sólo pueden leer de un conector. Las declaraciones de estas funciones son las siguientes:

```
#include <sys/socket.h>
int recv (int sfd, void *buf, int len, int flags);
int recvfrom (int sfd, void *buf, int len, int flags, void *from, int fromlen);
int recvmsg (int sfd, struct msghdr msg [], int flags);
```

En todas las llamadas, `sfd` es el descriptor del conector del cuál se leen los datos, `buf` es un puntero a la memoria intermedia donde se escribirán los datos leídos y `len` es el número máximo de bytes que se pueden escribir en la memoria referenciada por `buf`.

En los conectores del tipo `SOCK_STREAM`, las llamadas pueden usarse sólo después de haber establecido una conexión previa llamada a `connect`. En los conectores no orientados a conexión —`SOCK_DGRAM`— las llamadas pueden usarse tanto si se ha establecido una conexión como si no.

La llamada `recvfrom` trabaja de la misma forma que `recv`, pero con la ventaja de que puede devolver la dirección del conector desde el que se enviaron los datos. En el caso de conectores *datagrama* en los que se ha establecido una conexión, la dirección devuelta por `recvfrom` siempre será la misma. Para conectores del tipo *flujo de datos* —*stream*—, `recvfrom` devuelve el mismo tipo de información que `recv` y no devuelve la dirección del conector origen.

Si `from` es un puntero distinto de `NULL`, la dirección del conector origen de los datos se escribirá en la estructura apuntada por `from`; `fromlen` es un parámetro de entrada/salida que, al llamar a la función, debe contener el tamaño de la estructura apuntada por `from` y, al salir de la función, contendrá el tamaño real de la dirección leída.

Para conectores basados en paso de mensajes, como los conectores *datagrama*, cada mensaje se debe leer en su totalidad en una sola operación. Si el mensaje es demasiado largo para caber en la memoria intermedia indicada, será truncado. En los conectores no basados en el paso de mensajes —flujos de datos—, los datos se devuelven a medida que están disponibles y no se maneja para nada el concepto de tamaño de un mensaje.

La llamada `recvmsg` es una generalización de las dos anteriores. La diferencia con ellas es que los datos leídos pueden distribuirse entre varios bloques de memoria. El argumento `msg` es un array de cabeceras de mensaje, cada una de las cuales tiene la siguiente estructura:

```
struct msghdr {
    caddr_t msg_name; // Dirección, opcional
    int msg_namelen; // Tamaño del campo msg_name
    struct iovec *msg_iov; // Array de bloques de memoria sobre los que
                          // distribuir los datos leídos
    int msg_iovlen; // Número de elementos del array msg_iov
    caddr_t msg_accrights; // Derechos de acceso
    int msg_accrightslen; // Tamaño de msg_accrights
};
```

Esta estructura se emplea tanto para leer mensajes de un conector como para escribirlos en él. Los campos `msg_name` y `msg_namelen` se usan para conectores no conectados cuando nos interesa conocer la dirección del conector origen o destino del mensaje. Si no necesitamos conocer esa dirección, el campo `msg_name` puede ser un puntero `NULL`. El campo `msg_iov` es el array de bloques de memoria sobre los que distribuir los datos

leídos, o de los que recoger los datos que se van a enviar. En `msg_accrights` se recogen los derechos de acceso enviados junto con los mensajes. Este campo sólo se utiliza con los conectores de la familia `AF_UNIX` cuando los procesos se intercambian descriptores de fichero. Este campo puede ser un puntero `NULL`, con lo cual no se leen del conector los derechos de acceso.

Para las tres llamadas, el argumento `flags` tiene el mismo significado y sus posibles valores son el valor 0 o una combinación de los bits siguientes:

- `MSG_PEEK`, cualquier dato leído del conector es tratado como si la lectura no se hubiese llevado a cabo, por lo que la siguiente operación de lectura leerá los mismos datos. Esta opción se puede usar para ojear si hay datos disponibles en el conector.
- `MSG_OOB`, se utiliza para leer datos que van fuera de banda. Éstos son datos a los que se les da carácter de urgentes, por lo que tienen preferencia en el envío y en la recepción.

En los conectores de la familia `AF_UNIX` no pueden estar activos ninguno de los bits anteriores.

Las tres funciones estudiadas devuelven el total de bytes leídos del conector.

11.2.7 Escritura o envío de mensajes a un conector

Las llamadas para escribir datos en un conector son: `write`, `writew`, `send`, `sendto` y `sendmsg`. La llamada `write` no la vamos a describir porque ya vimos cuál era su interfaz al manejar el sistema de ficheros. De cara al manejo de conectores, la llamada `write` se comporta de la misma forma, pero con la salvedad de que el descriptor de fichero que utiliza es en realidad un descriptor de conector.

La llamada `writew` es una generalización de `write` y puede utilizarse para escribir datos en cualquier descriptor, ya sea fichero o conector. Su declaración es la siguiente:

```
#include <sys/uio.h>
ssize_t writew (int fildes, const struct iovec *iov, size_t iovcnt);
```

Esta llamada escribe datos en el fichero descrito por `fildes` y toma esos datos de los distintos bloques de memoria especificados por el array `iov`. El significado del array `iov` y del argumento `iovcnt` es el mismo que vimos para la llamada `readv`.

Si bien `write` y `writew` pueden escribir en cualquier descriptor —descriptor de fichero o de conector—, las llamadas `send`, `sendto` y `sendmsg` sólo pueden escribir en un conector. Las declaraciones de estas funciones se muestran a continuación:

```
#include <sys/socket.h>
int send (int sfd, void *buf, int len, int flags);
int sendto(int sfd, void *buf, int len, int flags, void *to, int tolen);
int sendmsg (int sfd, struct msghdr msg [], int flags);
```

En todas las llamadas, `sfd` es el descriptor del conector en el cual se escriben los datos, `buf` es un puntero a la zona de memoria donde están los datos que se desean enviar y `len` es el número de bytes que hay en esa zona de memoria.

En los conectores orientados a conexión —`SOCK_STREAM`— las llamadas pueden usarse sólo después de haber establecido una conexión previa llamada a `connect`. En los conectores no orientados a conexión —`SOCK_DGRAM`—, debe usarse la función `sendto`, a menos que se especifique una dirección de destino con una llamada a `connect`. Si ya se ha especificado una dirección, al llamar a `sendto` se producirá un error si el parámetro `to` contiene una dirección. La dirección del conector destino está en la posición de memoria apuntada por `to` y su tamaño es `tolen`.

En los conectores del tipo *datagrama* para los que no se ha definido una dirección mediante la llamada correspondiente a `bind`, si se envía un mensaje con `sendto`, el sistema elegirá de forma automática una dirección local, pero no hay garantía de que esa dirección siga siendo la misma en sucesivas llamadas a `sendto`.

La llamada `sendmsg` es una generalización de las dos anteriores. La diferencia con ellas es que los datos a enviar pueden proceder de varios bloques de memoria. El argumento `msg` es un array de cabeceras de mensaje, cada una de las cuales tiene una estructura como la definida para la función `recvmsg` —`struct msghdr`—.

Para las tres llamadas, el argumento `flags` tiene el mismo significado y sus posibles valores son 0 o `MSG_OOB` —mensaje urgente—. En los conectores de la familia `AF_UNIX` no se pueden enviar mensajes urgentes.

Las tres funciones estudiadas devuelven el total de bytes escritos en el conector.

11.2.8 Cierre del canal (`close`)

Una vez que un proceso no necesita realizar más accesos a un conector, puede desconectarse del mismo. Para ello, aprovechando que un conector es tratado sintácticamente como si fuera un fichero, podemos usar la llamada `close`. Esta llamada cierra el conector en sus dos sentidos —servidor–cliente y cliente–servidor—. Veremos más adelante —en el § 11.4.4 pág. 465— que hay otra llamada que tiene un control más fino a la hora de cerrar y permite deshabilitar uno o los dos sentidos del conector.

11.3 Ejemplos de servidores y clientes

En este apartado estudiaremos una serie de programas escritos para mostrar la forma que tienen tanto los servidores como los clientes que manejan las distintas familias de conectores y sus protocolos asociados.

Estos ejemplos son unos programas cortos que no tienen pretensiones de aplicación, pero ilustran muy bien la estructura básica de un programa que trabaja con la red. Sobre todo, dejan muy claro cuál es la secuencia de llamadas necesaria para conseguir establecer la comunicación. Por otro lado, aprovecharemos para introducir los convenios que utilizan los distintos protocolos para formar las direcciones de los nodos. Veremos también algunas funciones útiles para manejar estas direcciones.

Comentaré ejemplos escritos para dos familias de conectores: `AF_UNIX` y `AF_INET`. El motivo de elegir estas dos familias es que la primera está presente en todas las instalaciones BSD, ya que no requiere hardware de red adicional, y la segunda es la familia más utilizada por redes de la norma IEEE-802.

En todos los ejemplos, tanto el servidor como el cliente se ajustan a una misma funcionalidad. El cliente enviará cadenas de caracteres al servidor y el servidor devolverá

esa misma cadena, pero acompañada del nombre que recibe la máquina donde se ejecuta, el nombre y versión del sistema donde se ejecuta el servidor, el *PID* del proceso que dio servicio al cliente, y la fecha y hora en la que se prestó servicio al cliente.

Para leer estos últimos datos se utilizarán las llamadas `uname` y `time`. La llamada `time` se explicó en capítulos anteriores —consulte el § 7.7.2 pág. 282—. La llamada `uname` se declara como sigue:

```
#include <sys/utsname.h>
int uname (struct utsname *name);
```

Esta llamada `uname` devuelve, en la estructura apuntada por `name`, información referente al nombre y versión del sistema operativo. La definición de `struct utsname` se encuentra en el fichero de cabecera `<sys/utsname.h>`,

```
struct utsname {
    char sysname [9]; // Nombre del sistema
    char nodename [9]; // Nombre del nodo
    char release [9]; // Edición del sistema
    char version [9]; // Versión del sistema
    char machine [9]; // Tipo de máquina en la que se ejecuta el sistema
};
```

11.3.1 Ejemplos con conectores de la familia AF_UNIX

Los primeros ejemplos que presentaré manejan conectores de la familia `AF_UNIX`. Ésta es una familia que se utiliza para comunicar procesos que se ejecutan bajo el control de una misma máquina, por lo que no necesitan una red de conexión.

Las direcciones que maneja esta familia tienen la misma forma que las rutas de los ficheros, pero la transferencia de los datos no se hace a través del sistema de ficheros, sino a través de memoria.

Vamos a plantear dos parejas de clientes-servidores, la primera trabajará con un protocolo orientado a conexión —flujos de datos—, mientras que la segunda lo hará con un protocolo no orientado a conexión —*datagramas*—.

Cliente-servidor con conectores AF_UNIX del tipo flujo

El programa servidor se compone de dos ficheros cabecera, `scomun.h` y `u-addr.h`, y tres ficheros fuente, `scomun.c`, `str-com.c` y `u-str-se.c`.

El fichero de cabecera `scomun.h` tiene declaraciones de tipos y funciones que serán usados por todos los programas de manejo de conectores que veamos en este apartado y en los siguientes. El contenido de este fichero es el siguiente:

Fichero 11.1: Fichero de cabecera (`scomun.h`)

```
/**
```

```
    Declaración de tipos y funciones utilizados por los programas de ejemplo de
    manejo de conectores.
```

```
***/
```

```

#ifndef _SCOMUN_
#define _SCOMUN_

#include <stdio.h>
#include <sys/types.h>
#include <sys/utsname.h>
#include <time.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <netinet/in.h>

#define CR 10 /* Carácter retorno de carro. */

extern char *nombre_prog;

/* Estructura que almacena información sobre el sistema. */
struct estad {
    char nodo [9];
    char sistema[9];
    char version [9];
    int pid;
    char fecha [25];
};

/* Declaración de funciones. */
struct estad leer_estad ();

/* Constantes de ayuda a la depuracion. */
#define DEP __FILE__, __LINE__
#endif

```

En `scomun.c` hay funciones que son usadas por los programas de ejemplo que veremos a continuación. Son funciones no relacionadas directamente con el manejo de conectores.

Fichero 11.2: Fichero con funciones comunes para los ejemplos de conectores (`scomun.c`)

```

#include "scomun.h"

char *nombre_prog;

/**
    leer_estad: lee datos referentes al proceso servidor para enviárselos al cliente.
***/
struct estad leer_estad()
{
    struct estad est;
    struct utsname sistema;

```



```

    time_t fecha;

    uname (&sistema);
    strncpy (est.nodo, sistema.nodename, 9);
    strncpy (est.sistema, sistema.sysname, 9);
    strncpy (est.version, sistema.release, 9);
    est.pid = getpid ();
    time (&fecha);
    strncpy (est.fecha, asctime(localtime(&fecha)), 24);
    est.fecha[24] = 0;
    return (est);
};

/****
    error: imprime mensajes de error.
****/
error (char *fichero, int linea, char *mensaje)
{
    extern int errno;

    fprintf (stderr, "%s:(%s - %d). %s - %s\n", nombre_prog, fichero,
             linea, mensaje, sys_errlist[errno]);
}

```

El fichero de cabecera `u-addr.h` contiene declaraciones de direcciones utilizadas por los programas de ejemplo que manejan conectores `AF_UNIX` del tipo `STREAM` —flujos de datos—. Llamamos la atención del lector sobre la longitud que tienen las rutas utilizadas como soporte para formar las direcciones de clientes y servidores. Aunque las direcciones pueden llegar a tener 118 bytes de longitud, sin embargo en las implementaciones actuales las rutas no pueden tener más de 14 caracteres. El contenido de `u-addr.h` es el siguiente:

Fichero 11.3: Fichero de cabecera (`u-addr.h`)

```

/****
    Fichero de cabecera para los ejemplos de servidores y clientes que emplean la
    familia de conectores AF_UNIX.
****/
#ifndef _UNIX_ADDR_
#define _UNIX_ADDR_
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/un.h>
#define UNIX_STR_ADDR "u_str_socket"
#define UNIX_DG_ADDR "u_dg_socket"
#define UNIX_DG_TMP "u_dgtmp.XXXXXX"
#endif

```

En el fichero `str-com.c` están las funciones que necesitan todos los servidores y clientes que se comunican mediante conectores del tipo `STREAM`, ya sean de la familia `AF_UNIX` o de la familia `AF_INET`.

Fichero 11.4: Funciones para comunicarse con conectores del tipo `STREAM` (`str-com.c`)

```
#include "scomun.h"
```

```
/**
```

```
    escribirn: función utilizada para escribir en un fichero referenciado por el
    descriptor fd. Esta función es una interfaz con write y es necesaria porque la
    escritura de bloques de datos muy grandes en un conector no se puede realizar
    mediante una sola llamada a write.
```

```
*/
```

```
escribirn (int fd, char *buf, int nbytes)
```

```
{
```

```
    int escritos, resto = nbytes;
```

```
    while (resto > 0) {
```

```
        escritos = write (fd, buf, resto);
```

```
        if (escritos == -1)
```

```
            return (-1);
```

```
        resto -= escritos;
```

```
        buf += escritos;
```

```
    }
```

```
    return (nbytes - resto);
```

```
}
```

```
/**
```

```
    leer_linea: función para leer líneas de un fichero de entrada.
```

```
*/
```

```
leer_linea (int fd, char *buf, int nbytes)
```

```
{
```

```
    int n, nb;
```

```
    char b;
```

```
    for (n = 1; n < nbytes; n++) {
```

```
        if ((nb = read (fd, &b, 1)) == -1)
```

```
            return (-1); /* Error de lectura. */
```

```
        if (b == '\n') /* Un mensaje termina con retorno de carro. */
```

```
            break;
```

```
        if (nb == 0)
```

```
            if (n == 1)
```

```
                return (0); /* Caso de encontrar EOF. */
```

```
            else
```

```

        break;
    *buf++ = b;
}
*buf = 0;

return (n);
}

#define MAXLINEA 256
/**
    recibir_mensaje_str: esta función la ejecutan los servidores que trabajen con
    conectores del tipo STREAM.
***/
recibir_mensaje_str (int sfd)
{
    int nbytes;
    char linea [MAXLINEA], mensaje [MAXLINEA];
    struct estad est;

    for (;;) {
        /* Lectura de los mensajes procedentes del cliente. */
        nbytes = leer_linea (sfd, linea, MAXLINEA);
        if (nbytes == 0)
            return; /* Fin de servicio. */
        else if (nbytes == -1) {
            error (DEP, "leer_linea");
            exit (-1);
        }
        printf ("Mensaje recibido: %s\n", linea);

        /* Formación del mensaje para el cliente. */
        est = leer_estad ();
        sprintf (mensaje, "%s(%s, %s), PID(%d), %24s, %s\n",
                est.nodo, est.sistema, est.version,
                est.pid, est.fecha, linea);

        /* Envío del mensaje al cliente. */
        nbytes = strlen (mensaje);
        if (escribirn (sfd, mensaje, nbytes) != nbytes) {
            error (DEP, "escribirn");
            exit (-1);
        }
        printf ("Mensaje devuelto: %s\n", mensaje);
    }
}

```

```

/****
    enviar_mensajes_str: esta función la ejecutan los clientes que trabajan con
    conectores del tipo STREAM.
****/
enviar_mensajes_str (int sfd)
{
    int nbytes;
    char linea [MAXLINEA];

    /* Leer líneas del flujo estándar de entrada. */
    printf ("<== "); fflush (stdout);
    while (fgets (linea, sizeof (linea), stdin) != NULL) {
        strcat (linea, "\n");
        /* Escribir cada línea en el conector para enviárselo al servidor. */
        if (escribirn (sfd, linea, strlen (linea)) == -1) {
            error (DEP, "escribirn");
            exit (-1);
        }

        /* Leer la respuesta procedente del servidor. */
        nbytes = leer_linea (sfd, linea, MAXLINEA);
        if (nbytes == -1) {
            error (DEP, "leer_linea");
            exit (-1);
        }

        /* Sacar el mensaje por el flujo estándar de salida. */
        printf ("==> %s\n", linea); fflush (stdout);

        printf ("<== "); fflush (stdout);
    }
}

```

El primer programa servidor que veremos es `u-str-se.c`. Este servidor es del tipo concurrente, su código es el siguiente:

Programa 11.5: Servidor que usa conectores `AF_UNIX` del tipo `SOCK_STREAM` (`u-str-se.c`)

```

#include "scomun.h"
#include "u-addr.h"

main (int argc, char *argv [])
{
    int sfd, nsfd, pid;
    struct sockaddr_un ser_addr, cli_addr;
    int ser_addr_len, cli_addr_len;

```

```

nombre_prog = argv [0];

/* Apertura de un conector orientado a conexión de la familia AF_UNIX. */
if ((sfd = socket (AF_UNIX, SOCK_STREAM, 0)) == -1) {
    error (DEP, "abrir conector");
    exit (-1);
}

/* Publicidad de la dirección del servidor. */
unlink (UNIX_STR_ADDR);
ser_addr.sun_family = AF_UNIX;
strcpy (ser_addr.sun_path, UNIX_STR_ADDR);
ser_addr_len = strlen(ser_addr.sun_path)+sizeof(ser_addr.sun_family);
if (bind (sfd, (struct sockaddr *) &ser_addr, ser_addr_len) == -1) {
    error (DEP, "bind");
    exit (-1);
}

/* Declaración de una cola con 5 elementos para peticiones de conexión. */
listen (sfd, 5);

/* Bucle de lectura de peticiones de conexión. */
for (;;) {
    cli_addr_len = sizeof (cli_addr);
    if ((nsfd = accept (sfd, (struct sockaddr *) &cli_addr,
                        &cli_addr_len)) == -1) {
        error (DEP, "accept");
        exit (-1);
    }

    /* Creación de un proceso hijo para atender al cliente. */
    if ((pid = fork ()) == -1)
        error (DEP, "fork");
    else if (pid == 0) { /* Código del proceso hijo. */
        close (sfd);
        recibir_mensaje_str (nsfd);
        close (nsfd);
        exit (0);
    }
    /* Código del proceso padre. */
    close (nsfd);
}
}

```

El programa cliente se encuentra en el fichero `u-str-cl.c` y su código fuente es el siguiente:

Programa 11.6: Cliente que usa conectores AF_UNIX del tipo SOCK_STREAM (u-str-cl.c)

```

#include "scomun.h"
#include "u-addr.h"

main (int argc, char *argv [])
{
    int sfd;
    struct sockaddr_un ser_addr;
    int ser_addr_len;

    nombre_prog = argv[0];
    /* Apertura de un conector orientado a conexión de la familia AF_UNIX. */
    if ((sfd = socket (AF_UNIX, SOCK_STREAM, 0)) == -1) {
        error (DEP, "apertura del conector");
        exit (-1);
    }
    /* Petición de conexión con el servidor. */
    ser_addr.sun_family = AF_UNIX;
    strcpy (ser_addr.sun_path, UNIX_STR_ADDR);
    ser_addr_len = strlen(ser_addr.sun_path)+sizeof(ser_addr.sun_family);
    if (connect (sfd, (struct sockaddr *)&ser_addr, ser_addr_len) == -1) {
        error (DEP, "conexión");
        exit (-1);
    }
    /* Comunicación con el servidor. */
    enviar_mensajes_str (sfd);

    close (sfd);
    exit (0);
}

```

Cliente-servidor con conectores AF_UNIX del tipo *datagrama*

Antes de implementar ejemplos con conectores no orientados a conexión hay que recordar que el protocolo de los *datagramas* no garantiza la distribución de los mensajes que se envían. Por otro lado, tampoco se garantiza que la secuencia de recepción de los mensajes coincida con la secuencia de emisión de los mismos. Lo único que tenemos garantizado es que los mensajes viajarán por la red sin fraccionarse, por lo que cuando se recibe un mensaje se tiene la garantía de que está íntegro. Para poder apreciar el orden en el que se distribuyen los mensajes, los vamos a acompañar de un número secuencial que indica el orden en que fueron generados por el cliente. Este número ya constituye en sí un protocolo rudimentario que puede servir para reconstruir la verdadera secuencia que deben seguir los mensajes, si bien en estos ejemplos sólo lo utilizaremos a título de información.

El programa servidor se compone de dos ficheros cabecera, `scomun.h` y `u-addr.h`, y tres ficheros fuente, `scomun.c`, `dg-com.c` y `u-dg-se.c`. Los ficheros `scomun.h`, `u-addr.h`

y `scomun.c` ya han sido comentados en el apartado anterior, por lo que no los vamos a repetir aquí. El módulo `dg-com.c` contiene las funciones que usan los servidores y clientes no orientados a conexión, tanto de este ejemplo como de los siguientes. El contenido de este módulo es el siguiente:

Fichero 11.7: Funciones para comunicarse con conectores del tipo *datagrama* (`dg-com.c`)

```

/****
    Este fichero contiene funciones que serán usadas por los programas que
    trabajan con conectores no orientados a conexión –datagramas–.
****/
#include "scomun.h"

#define MAXLINEA 256
/****
    recibir_mensaje_dg: esta función la utilizan los servidores que trabajen con
    conectores del tipo datagrama.
****/
recibir_mensaje_dg (int sfd, struct sockaddr *pcli_addr, int maxcli_len)
{
    int nbytes, cli_len = maxcli_len, nro_mensaje_rec;
    char linea [MAXLINEA], mensaje [MAXLINEA];
    struct estad est;

    for (;;) {
        /* Lectura de los mensajes procedentes del cliente. */
        nbytes = recvfrom (sfd, mensaje, MAXLINEA, 0, pcli_addr,
                           &cli_len);
        if (nbytes == -1) {
            error (DEP, "recibir_mensaje_dg (recvfrom)");
            exit (-1);
        }
        mensaje [nbytes] = 0;
        sscanf (mensaje, "%d %s", &nro_mensaje_rec, linea);
        imprimir_direccion_cliente (pcli_addr, cli_len);
        printf ("Nro. mensaje: %d\n", nro_mensaje_rec);
        printf ("Mensaje: %s\n", linea);

        /* Formación del mensaje para el cliente. */
        est = leer_estad ();
        sprintf (mensaje, "%s(%s, %s), PID(%d), %24s, %d %s\n",
                est.nodo, est.sistema, est.version, est.pid, est.fecha,
                nro_mensaje_rec, linea);

        /* Envío del mensaje al cliente. */
        nbytes = strlen (mensaje);
        if (sendto (sfd, mensaje, nbytes, 0, pcli_addr, cli_len) != nbytes) {

```

```

        error (DEP, "recibir_mensaje_dg (sendto)");
        exit (-1);
    }
    printf ("Mensaje devuelto: %s\n", mensaje);
}

}

/****
    enviar_mensajes_dg: esta función la utilizan los clientes que trabajan con
    conectores del tipo datagrama.
****/
enviar_mensajes_dg (int sfd, struct sockaddr *pser_addr, int maxser_len)
{
    int nbytes, nro_mensaje_env = 0;
    char linea [MAXLINEA], mensaje [MAXLINEA];

    /* Leer líneas del flujo estándar de entrada. */
    printf ("<== "); fflush (stdout);
    while (fgets (linea, sizeof (linea), stdin) != NULL) {
        /* Escribir cada línea en el conector para enviárselo al servidor. */
        sprintf (mensaje, "%d %s", ++nro_mensaje_env, linea);
        nbytes = strlen (mensaje);
        if (sendto(sfd,mensaje,nbytes,0,pser_addr,maxser_len)!=nbytes) {
            error (DEP, "enviar_mensajes_dg (sendto)");
            return (-1);
        }

        /* Leer la respuesta procedente del servidor. */
        nbytes = recvfrom (sfd, mensaje, MAXLINEA, 0, NULL, NULL);
        if (nbytes == -1) {
            error (DEP, "enviar_mensajes_dg (recvfrom)");
            return (-1);
        }

        /* Sacar el mensaje por el flujo estándar de salida. */
        mensaje[nbytes] = 0;
        printf ("==> %s\n", mensaje); fflush (stdout);
        printf ("<== "); fflush (stdout);
    }
}

/****
    imprimir_direccion_cliente: presenta por el flujo estándar de salida la
    dirección del conector al cual está conectado un proceso cliente. El tipo de
    presentación depende de la familia a la que pertenece el conector.
****/

```



```

imprimir_direccion_cliente (struct sockaddr *pcli_addr, int cli_len)
{
    struct sockaddr_un *u_addr;
    struct sockaddr_in *i_addr;

    switch (pcli_addr->sa_family) {
    case AF_UNIX:
        u_addr = (struct sockaddr_un *)pcli_addr;
        printf ("Dirección cliente: %14s\n", u_addr->sun_path);
        break;
    case AF_INET:
        i_addr = (struct sockaddr_in *)pcli_addr;
        printf ("Nodo del cliente: %s\n", inet_ntoa (i_addr->sin_addr));
        printf ("Puerto del cliente: %d\n", i_addr->sin_port);
        break;
    default:
        error (DEP, "imprimir_direccion_cliente");
    }
}

```

El programa servidor se encuentra en el fichero `u-dg-se.c` y es del tipo interactivo. Como no hay una conexión con el cliente, al leer los mensajes procedentes de éste es necesario conocer su dirección para saber a quién hay que dirigir la respuesta. El código del servidor se muestra a continuación:

Programa 11.8: Servidor que usa conectores AF_UNIX del tipo *datagrama* (`u-dg-se.c`)

```

/****
Este servidor se comunica a través de conectores de la familia AF_UNIX y del
tipo SOCK_DGRAM.
****/
#include "scomun.h"
#include "u-addr.h"

main (int argc, char *argv [])
{
    int sfd;
    struct sockaddr_un ser_addr, cli_addr;
    int ser_addr_len;
    nombre_prog = argv [0];

    /* Apertura de un conector no orientado a conexión de la familia AF_UNIX. */
    if ((sfd = socket (AF_UNIX, SOCK_DGRAM, 0)) == -1) {
        error (DEP, "abrir conector");
        exit (-1);
    }
}

```

```

/* Publicidad de la dirección del servidor. */
unlink (UNIX_DG_ADDR);
ser_addr.sun_family = AF_UNIX;
strcpy (ser_addr.sun_path, UNIX_DG_ADDR);
ser_addr_len = strlen(ser_addr.sun_path)+sizeof(ser_addr.sun_family);
if (bind (sfd, (struct sockaddr *) &ser_addr, ser_addr_len) == -1) {
    error (DEP, "bind");
    exit (-1);
}

/* Lectura y procesamiento de los mensajes del conector. */
recibir_mensaje_dg (sfd, &cli_addr, sizeof (cli_addr));
close (sfd);
exit (0);
}

```

El programa cliente se encuentra en el fichero `u-dg-cl.c`. Al no haber una conexión con el servidor, es necesario que cada cliente tenga una dirección distinta para que los mensajes se puedan encaminar correctamente dentro de la red. Las direcciones de los conectores de la familia `AF_UNIX` se forman a partir de rutas del sistema de ficheros. Esto fuerza a que cada cliente tenga asociada una ruta distinta. Para crear estas rutas podemos utilizar la función estándar `mktemp`, que se declara como sigue:

```

#include <stdlib.h>
char *mktemp (char *template);

```

Esta función reemplaza el contenido de la cadena apuntada por `template` por un nombre de fichero único y devuelve la dirección de `template`. La cadena `template` debe tener la forma de ruta terminada con seis caracteres `X` de la cadena `UNIX_TMP` definida en el fichero `u-addr.h`, `mktemp` reemplazará estos caracteres por una letra y el *PID* del proceso actual. La letra se elige para que no se dé repetición con un nombre de fichero ya existente.

El código del programa cliente es el siguiente:

Programa 11.9: Cliente que usa conectores `AF_UNIX` del tipo *datagrama* (`u-dg-cl.c`)

```

/****
Este cliente se comunica a través de conectores de la familia AF_UNIX y del
tipo SOCK_DGRAM.
****/
#include "scomun.h"
#include "u-addr.h"

main (int argc, char *argv [])
{
    int sfd;
    struct sockaddr_un ser_addr, cli_addr;
    int ser_addr_len, cli_addr_len;

```

```

nombre_prog = argv[0];

/* Apertura de un conector no orientado a conexión de la familia AF_UNIX. */
if ((sfd = socket (AF_UNIX, SOCK_DGRAM, 0)) == -1) {
    error (DEP, "apertura del conector");
    exit (-1);
}

/* Formación de la dirección del servidor. */
ser_addr.sun_family = AF_UNIX;
strcpy (ser_addr.sun_path, UNIX_DG_ADDR);
ser_addr_len = strlen(ser_addr.sun_path)+sizeof(ser_addr.sun_family);

/* Formación y publicidad de la dirección del cliente. Al tratarse de conectores
datagrama, cada cliente debe tener una dirección distinta de los demás. Por eso
formamos la dirección del cliente mediante un nombre de fichero temporal creado
con mktemp. */
cli_addr.sun_family = AF_UNIX;
strcpy (cli_addr.sun_path, UNIX_DG_TMP);
mktemp (cli_addr.sun_path);
cli_addr_len = strlen(cli_addr.sun_path)+sizeof(cli_addr.sun_family);
if (bind (sfd, (struct sockaddr *) &cli_addr, cli_addr_len) == -1) {
    error (DEP, "bind");
    exit (-1);
}

/* Comunicación con el servidor. */
enviar_mensajes_dg (sfd, &ser_addr, ser_addr_len);
close (sfd);
unlink (cli_addr.sun_path);
exit (0);
}

```

11.3.2 Ejemplos con conectores de la familia AF_INET

La familia `AF_INET` soporta los protocolos *DARPA* para las comunicaciones *Internet*: *TCP* —*Transmission Control Protocol*— y *UDP* —*User Datagram Protocol*—.

TCP es un protocolo orientado a conexión que proporciona a los procesos de usuario un canal dúplex fiable. *TCP* está basado en *IP* —*Internet Protocol*—, que es un protocolo de más bajo nivel —consulte la figura 11.5—. Aunque a veces se habla de *TCP/IP* como si se tratase de un solo protocolo, en el fondo no es correcto, ya que *UDP* también está construido sobre *IP*.

UDP es un protocolo no orientado a conexión, por lo que no hay garantía de que los mensajes —*datagramas*— siempre alcancen su destino. Si queremos realizar comunica-

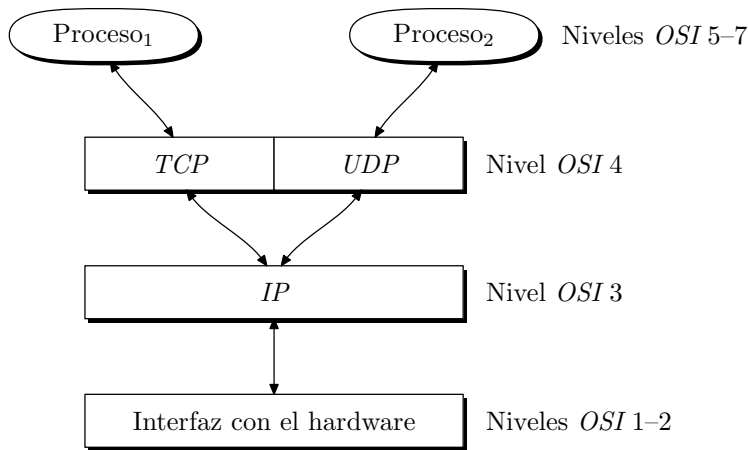


Figura 11.5: Protocolos *Internet*.

ciones fiables sobre conectores no conectados, es necesario recurrir a protocolos de más alto nivel, o que el usuario implemente, a partir de protocolos desarrollados por él, los mecanismos que hagan fiable al canal.

Con los conectores `AF_INET` podemos comunicar procesos que se estén ejecutando tanto en la misma máquina como en máquinas distintas. Esto acarrea la necesidad de una red que conecte físicamente las distintas máquinas que intervienen en la comunicación. No vamos a entrar en la descripción de las tecnologías de las redes ni de sus topologías, porque está fuera de lugar para nuestra discusión. Para nosotros, una red es un camino de comunicación entre ordenadores y, a través de unos protocolos y unos servicios, podemos hacer uso de la misma. Como ya hemos mencionado con anterioridad, a nosotros, como programadores usuarios de una red, lo único que nos interesa son las llamadas o conjunto de servicios con los que podemos usarla.

Antes de empezar a ver los ejemplos, es necesario que aclaremos algunas ideas sobre la forma de direccionar los distintos nodos y conectores dentro de la familia `AF_INET`. La forma de una dirección *Internet* es:

```

struct in_addr {
    u_long s_addr; // 32 bits que contienen la identificación de la red y
                  // del nodo
};

struct sockaddr_in {
    short sin_family; // AF_INET
    u_short sin_port; // 16 bits con el número de puerto
    struct in_addr sin_addr; // 32 bits con la identificación de la red y
                           // del nodo
    char sin_zero [8]; // 8 bytes no usados
};
  
```

En la estructura `sockaddr_in`, el campo `sin_family` debe valer `AF_INET`, por ser `AF_INET` la familia para la que tiene sentido una dirección con la estructura anterior.

El campo `sin_addr` contiene la dirección del nodo origen o destino del mensaje que circula por la red. Este campo es de 32 bits agrupados en 4 bytes. Estos 4 bytes se suelen representar de cara al usuario en la forma `ddd.ddd.ddd.ddd`, donde `ddd` representa un número decimal que varía en el rango 0-255. En el fichero de configuración `/etc/hosts` hay una relación de todos los nodos de que consta la red local y de sus direcciones asociadas. Cada línea de este fichero tiene la forma:

`ddd.ddd.ddd.ddd nombre1 nombre2 ... nombren`

donde `nombre1`, `nombre2`, ..., `nombren` son cadenas de caracteres que se utilizan como sinónimos de la dirección `ddd.ddd.ddd.ddd`. El siguiente es un ejemplo de fichero `hosts` para un nodo de una red que se compone de 3 ordenadores. Cada nodo de la red debe tener un fichero `hosts` parecido a éste:

```
# @(#) $Header: hosts,v 16.1 90/01/15 18:24:45 jmc Exp $
#
# The form for each entry is:
# <internet address>      <official hostname> <aliases>
#
# For example:
# 192.1.2.34   hpfcrm    loghost
#
# See the hosts(4) manual page for more information.
# Note: The entries cannot be preceded by a space.
#       The format described in this file is the correct format.
#       The original Berkeley manual page contains an error in
#       the format description.
#
127.0.0.1 localhost loopback
192.0.0.5 apolo
192.0.0.1 ws0   zeus
192.0.0.2 ws1   vulcano
192.0.0.3 ws2   marte
192.0.0.4 ws3   dionisios
```

El campo `sin_port` se utiliza para identificar los distintos procesos que, en un mismo nodo, están haciendo uso de la red. Este campo es necesario porque la dirección del nodo da acceso al ordenador con el que queremos comunicarnos, pero no especifica cuál es el proceso origen o destino de los mensajes. Si en un mismo nodo se están ejecutando varios clientes o servidores, el campo `sin_port` de la dirección especifica a cuál de ellos está referido el mensaje.

Esta estructura de dirección tiene aplicación sobre los protocolos *TCP* y *UDP*, por lo que los mensajes que se transmiten a través de la red, además de los datos del usuario, deben contener información de control dedicada a su encaminamiento. Esta información se compone de los campos:

- Protocolo (*TCP* o *UDP*).
- Dirección *Internet* del nodo origen (32 bits).
- Puerto asociado al proceso origen del mensaje (16 bits).
- Dirección *Internet* del nodo destino (32 bits).
- Puerto asociado al proceso destino del mensaje (16 bits).

Así, un ejemplo de cabecera de control del mensaje puede ser:

```
{tcp, 128.10.12.02, 1200, 128.10.12.11, 1350}
```

Antes de pasar a ver los ejemplos, comentaré un conjunto de funciones estándar que nos ayudarán a manejar las direcciones de nodo, los números de puerto y las posibles incompatibilidades entre distintos ordenadores.

Funciones para la conversión de direcciones

La página `inet` del manual de UNIX describe un conjunto de funciones estándar para manejar direcciones *Internet*. En los ejemplos vamos a usar `inet_addr` e `inet_ntoa`, que se declaran como sigue:

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
unsigned long inet_addr (const char *cp);
char *inet_ntoa (struct in_addr in);
```

La función `inet_addr` traduce la cadena de caracteres apuntada por `cp`, que contiene una dirección de nodo en el formato decimal estándar para el usuario, al formato binario definido por la estructura `struct in_addr`. El ejemplo siguiente muestra la traducción:

```
struct in_addr dirección;
...
dirección.s_addr = inet_addr ("192.12.10.03");
// dirección.s_addr contendrá el valor 0xc00c0a03 = 1074530818
```

La función `inet_ntoa` realiza la operación inversa de `inet_addr`; transforma una dirección de nodo con el formato binario `struct in_addr` al formato decimal de usuario. El puntero devuelto referencia una cadena de caracteres local a la función.

Reserva de puertos

A la hora de asignar un número de puerto a un proceso, tenemos dos posibilidades:

- Asignar un número fijo predeterminado, en cuyo caso hay que tener cuidado con los puertos que tiene reservados el sistema. Este tipo de reserva la suelen emplear los servidores que necesitan una dirección con un puerto perfectamente conocido por sus clientes.

- Pedir al sistema que le reserve al proceso alguno de los puertos que se encuentran libres en este instante. Esto se consigue poniendo el número de puerto 0 antes de hacer la llamada a `bind` para dar publicidad a una dirección.

En el caso de que el usuario fije un número de puerto, hay que tener presente que para los protocolos *Internet* —*TCP* y *UDP*— los números comprendidos entre 1 y 1.023 están reservados por el sistema, y por lo tanto ningún proceso puede utilizarlos. Los puertos reservados son utilizados por los servidores que se ejecutan con privilegios de superusuario. Así, los puertos del intervalo [1, 255] son utilizados por las aplicaciones *Internet* estándar, tales como `ftp`, `telnet`, `tftp`, etc. Los puertos del intervalo [256, 511] están reservados para futuras aplicaciones *Internet*. Por último, los puertos del intervalo [512, 1.023] no están reservados para aplicaciones estándar, sino para servidores de usuario que se ejecuten con privilegios de superusuario. Si queremos utilizar alguno de estos puertos, previamente hay que reservarlo con una llamada a `rresvport`. Esta función se define como sigue:

```
int rresvport (int *port);
```

Esta función crea un conector de la familia `AF_INET` y del tipo `SOCK_STREAM`, y reserva un puerto privilegiado para él. La función devuelve un descriptor del conector creado y, a través de la zona de memoria apuntada por `port`, el número del puerto reservado. El puerto reservado es el primero que se encuentre libre en el intervalo [512, 1.023]. La búsqueda de puerto libre se inicia con el número 1.023 y continúa en orden decreciente hasta 512. Si no se encuentra ningún puerto libre, la función devuelve el valor -1.

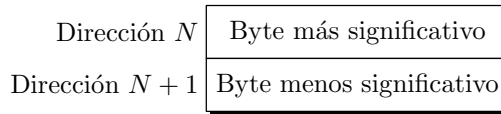
Los puertos libres para el usuario se encuentran en el intervalo [1024, 65.535]. De éstos, el sistema gestiona de forma automática los que se encuentran en [1.024, 5.000]. Por esto es conveniente que al fijar un número de puerto, desde un cliente o un servidor, éste sea superior a 5.000, para asegurar que no interferimos con ninguna operación de reserva llevada a cabo por el sistema.

Orden de los bytes

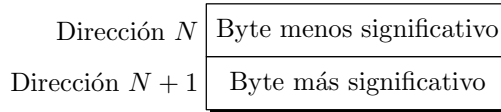
Al comunicar dos procesos que se ejecutan en máquinas diferentes se presenta un problema que no se da en las comunicaciones sobre una misma máquina. Se trata del orden con el que se almacenan en memoria las palabras multibyte. Se han acuñado los términos *extremista mayor* y *extremista menor*⁶ para referirse a las dos formas que hay de tratar palabras multibyte. Tomemos como ejemplo una palabra de 2 bytes; las máquinas que trabajan con formato *extremista mayor* almacenan el byte más significativo en la posición baja de memoria, y el menos significativo, en la posición alta —consulte la figura 11.6—. El formato *extremista menor* es justo el contrario del anterior.

El formato que emplean los protocolos *TCP* y *UDP* es *extremista mayor*, pero la máquina que los soporta puede manejar cualquier tipo de formato. Para solventar las potenciales diferencias de comunicación entre una arquitectura de ordenador y los protocolos *Internet*, en la página `byteorder` del manual de UNIX hay definidas 4 funciones estándar

⁶Es muy frecuente encontrar las expresiones *big endian* y *little endian* para referirse a *extremista mayor* y *extremista menor* respectivamente. En [Cohen, 1981] podemos encontrar el origen de estas expresiones aplicadas al orden de los bytes.



a) Formato *extremista mayor*



b) Formato *extremista menor*

Figura 11.6: Formatos de almacenamiento de palabras multibyte.

de conversión de formatos. Éstas son `htonl`, `htons`, `ntohl` y `ntohs`, y sus declaraciones son las siguientes:

```
#include <sys/types.h>
#include <netinet/in.h>
unsigned long htonl (unsigned long hostlong);
unsigned short htons (unsigned short hostshort);
unsigned long ntohl (unsigned long netlong);
unsigned short ntohs (unsigned short netshort);
```

- `htonl` convierte datos `unsigned long` del formato que maneja el ordenador al formato que maneja la red.
- `htons` convierte datos `unsigned short` del formato que maneja el ordenador al formato que maneja la red.
- `ntohl` convierte datos `unsigned long` del formato que maneja la red al formato que maneja el ordenador.
- `ntohs` convierte datos `unsigned short` del formato que maneja la red al formato que maneja el ordenador.

Cliente-servidor con conectores AF_INET del tipo flujo de datos

Los ejemplos que veremos aquí responden a la misma funcionalidad que ya hemos visto en los clientes-servidores de la familia AF_UNIX. En la familia AF_INET, *TCP* es el protocolo orientado a conexión. Por eso, el programa servidor se llama `i-tcp-se.c` y el cliente `i-tcp-cl.c`.

El programa servidor se compone de los ficheros de cabecera `scomun.h`, `i-addr.h`, y de los módulos fuente `scomun.c`, `str-com.c` e `i-tcp-se.c`. De todos estos ficheros, sólo `i-addr.h` y `i-tcp-se.c` son nuevos.

El fichero de cabecera `i-addr.h` contiene las declaraciones de direcciones necesarias para establecer conexiones y para enviar *datagramas*. El sistema en el que se han implementado los ejemplos consta de 5 ordenadores conectados mediante hardware *LAN* —*Local Area Network*— y sus direcciones *Internet*, así como sus nombres, figuran en el fichero `/etc/hosts` visto anteriormente. Este primer par de programas cliente-servidor está codificado para que el cliente sólo se pueda ejecutar en el nodo de dirección 192.0.0.5, tal y como muestran las constantes de fichero `i-addr.h`:

Fichero 11.10: Fichero de cabecera (`i-addr.h`)

```
/**
 * Fichero de cabecera para los ejemplos de servidores y clientes que emplean la
 * familia de conectores AF_INET.
 */
***
#ifndef _INTERNET_ADDR_
#define _INTERNET_ADDR_
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PUERTO_SERVIDOR_TCP 7000
#define PUERTO_SERVIDOR_UDP 7000
#define DIRECCION_NODO_SERVIDOR "192.0.0.5"

#endif
```

El programa servidor que se implementa es del tipo concurrente, al igual que el servidor orientado a conexión de la familia `AF_UNIX`. Su código se puede ver a continuación:

Programa 11.11: Serv. concurrente con conectores `AF_INET` del tipo `SOCK_STREAM` (`i-tcp-se.c`)

```
#include "scomun.h"
#include "i-addr.h"

main (int argc, char *argv [])
{
    int sfd, nsfd, pid;
    struct sockaddr_in ser_addr, cli_addr;
    int cli_addr_len;

    nombre_prog = argv [0];
    /* Apertura de un conector orientado a conexión de la familia AF_INET. */
    if ((sfd = socket (AF_INET, SOCK_STREAM, 0)) == -1) {
        error (DEP, "abrir socket");
        exit (-1);
    }
}
```

```

/* Publicidad de la dirección del servidor. */
ser_addr.sin_family = AF_INET;
ser_addr.sin_addr.s_addr=inet_addr(DIRECCION_NODO_SERVIDOR);
ser_addr.sin_port = htons (PUERTO_SERVIDOR_TCP);
if (bind (sfd,(struct sockaddr *)&ser_addr,sizeof(ser_addr)) == -1) {
    error (DEP, "bind");
    exit (-1);
}

/* Declaración de una cola con 5 elementos para peticiones de conexión. */
listen (sfd, 5);

/* Bucle de lectura de peticiones de conexión. */
for (;;) {
    cli_addr_len = sizeof (cli_addr);
    if ((nsfd = accept (sfd, (struct sockaddr *) &cli_addr,
                        &cli_addr_len)) == -1) {
        error (DEP, "accept");
        exit (-1);
    }

    /* Creación de un proceso hijo para atender al cliente. */
    if ((pid = fork ()) == -1)
        error (DEP, "fork");
    else if (pid == 0) {
        /* Código del proceso hijo. */
        close (sfd);
        recibir_mensaje_str (nsfd);
        close (nsfd);
        exit (0);
    }
    /* Código del proceso padre. */
    close (nsfd);
}
}

```

El código del programa cliente está en el fichero `i-tcp-cl.c`:

Programa 11.12: Cliente con conectores `AF_INET` del tipo `SOCK_STREAM` (`i-tcp-cl.c`)

```

#include "scomun.h"
#include "i-addr.h"

main (int argc, char *argv [])
{
    int sfd;
    struct sockaddr_in ser_addr;

```

```

nombre_prog = argv[0];
/* Apertura de un conector orientado a conexión de la familia AF_INET. */
if ((sfd = socket (AF_INET, SOCK_STREAM, 0)) == -1) {
    error (DEP, "apertura del conector");
    exit (-1);
}

/* Petición de conexión con el servidor. */
ser_addr.sin_family = AF_INET;
ser_addr.sin_addr.s_addr = inet_addr (DIRECCION_NODO_SERVIDOR);
ser_addr.sin_port = htons (PUERTO_SERVIDOR_TCP);
if (connect(sfd, (struct sockaddr *)&ser_addr, sizeof(ser_addr)) == -1) {
    error (DEP, "conexión");
    exit (-1);
}

/* Comunicación con el servidor. */
enviar_mensajes_str (sfd);

close (sfd);
exit (0);
}

```

Cliente-servidor con conectores AF_INET del tipo *datagrama*

En la familia AF_INET, el envío de *datagramas* se realiza con el protocolo UDP. Los programas cliente y servidor son *i-udp-cl.c* e *i-udp-se.c*, respectivamente. Ambos se apoyan en los ficheros de cabecera *scomun.h* e *i-addr.h*, así como en los módulos de código *scomun.c* y *dg-com.c*.

Podemos ver a continuación el código del programa servidor:

Programa 11.13: Servidor interactivo con conectores AF_INET del tipo SOCK_DGRAM (*i-udp-se.c*)

```

#include "scomun.h"
#include "i-addr.h"

main (int argc, char *argv [])
{
    int sfd;
    struct sockaddr_in ser_addr, cli_addr;

    nombre_prog = argv [0];
    /* Apertura de un conector no orientado a conexión de la familia AF_INET. */
    if ((sfd = socket (AF_INET, SOCK_DGRAM, 0)) == -1) {
        error (DEP, "abrir socket");
        exit (-1);
    }
}

```

```

/* Publicidad de la dirección del servidor. */
ser_addr.sin_family = AF_INET;
ser_addr.sin_addr.s_addr = inet_addr (DIRECCION_NODO_SERVIDOR);
ser_addr.sin_port = htons (PUERTO_SERVIDOR_UDP);
if (bind (sfd, (struct sockaddr *)&ser_addr, sizeof(ser_addr)) == -1) {
    error (DEP, "bind");
    exit (-1);
}

/* Lectura y procesamiento de los mensajes del conector. */
recibir_mensaje_dg (sfd, &cli_addr, sizeof (cli_addr));

close (sfd);
exit (0);
}

```

El programa cliente está disponible en el fichero `i-udp-cl.c`:

Programa 11.14: Cliente con conectores AF_INET del tipo SOCK_DGRAM (`i-udp-cl.c`)

```

#include "scomun.h"
#include "i-addr.h"

main (int argc, char *argv [])
{
    int sfd;
    struct sockaddr_in ser_addr, cli_addr;

    nombre_prog = argv[0];

    /* Apertura de un conector no orientado a conexión de la familia AF_INET */
    if ((sfd = socket (AF_INET, SOCK_DGRAM, 0)) == -1) {
        error (DEP, "apertura del conector");
        exit (-1);
    }

    /* Formación de la dirección del servidor. */
    ser_addr.sin_family = AF_INET;
    ser_addr.sin_addr.s_addr = inet_addr(DIRECCION_NODO_SERVIDOR);
    ser_addr.sin_port = htons (PUERTO_SERVIDOR_UDP);

    /* Formación y publicidad de la dirección del cliente. Al tratarse de conectores
    datagrama, cada cliente debe tener una dirección distinta de los demás, por eso
    dejamos que sea el sistema quien elija la direccion automáticamente. */
    cli_addr.sin_family = AF_INET;
    cli_addr.sin_addr.s_addr = htonl (INADDR_ANY);
    cli_addr.sin_port = htons (0); /* El sistema elige el número de puerto.*/
}

```

```

    if (bind (sfd,(struct sockaddr *)&cli_addr,sizeof(cli_addr)) == -1) {
        error (DEP, "bind");
        exit (-1);
    }

    /* Comunicación con el servidor. */
    enviar_mensajes_dg (sfd, &ser_addr, sizeof (ser_addr));

    close (sfd);
    exit (0);
}

```

Para compilar las cuatro parejas de servidores y clientes que hemos visto en este apartado de ejemplos conviene utilizar el programa `make`. A continuación podemos ver un fichero `makefile` para llevar a cabo la compilación de los ejemplos:

Fichero 11.15: Compilación de los distintos programas cliente – servidor (`makefile.1`)

```

#####
# Fichero con la descripción de los módulos que intervienen en la compilación de los
# distintos programas de ejemplo para conectores.
#####

#
# Servidor–cliente para conectores de la familia AF_UNIX y del
# tipo SOCK_STREAM.
#

u-str-se: makefile.1 u-str-se.o str-com.o scomun.o
    cc -o u-str-se u-str-se.o str-com.o scomun.o

u-str-cl: makefile.1 u-str-cl.o str-com.o scomun.o
    cc -o u-str-cl u-str-cl.o str-com.o scomun.o

u-str-se.o: u-addr.h scomun.h

u-str-cl.o: u-addr.h scomun.h

str-com.o: scomun.h

scomun.o: scomun.h

#
# Servidor–cliente para conectores de la familia AF_UNIX y del
# tipo SOCK_DGRAM.
#

u-dg-se: makefile.1 u-dg-se.o dg-com.o scomun.o

```

```
cc -o u-dg-se u-dg-se.o dg-com.o scomun.o

u-dg-cl: makefile.1 u-dg-cl.o dg-com.o scomun.o
cc -o u-dg-cl u-dg-cl.o dg-com.o scomun.o

u-dg-se.o: u-addr.h scomun.h

u-dg-cl.o: u-addr.h scomun.h

dg-com.o: scomun.h

#
# Servidor-cliente para conectores de la familia AF_INET y del
# tipo SOCK_STREAM.
#
i-tcp-se: makefile.1 i-tcp-se.o str-com.o scomun.o
cc -o i-tcp-se i-tcp-se.o str-com.o scomun.o

i-tcp-cl: makefile.1 i-tcp-cl.o str-com.o scomun.o
cc -o i-tcp-cl i-tcp-cl.o str-com.o scomun.o

i-tcp-se.o: i-addr.h scomun.h

i-tcp-cl.o: i-addr.h scomun.h

#
# Servidor-cliente para conectores de la familia AF_INET y del
# tipo SOCK_DGRAM.
#
i-udp-se: makefile.1 i-udp-se.o dg-com.o scomun.o
cc -o i-udp-se i-udp-se.o dg-com.o scomun.o

i-udp-cl: makefile.1 i-udp-cl.o dg-com.o scomun.o
cc -o i-udp-cl i-udp-cl.o dg-com.o scomun.o

i-udp-se.o: i-addr.h scomun.h

i-udp-cl.o: i-addr.h scomun.h

#
# Orden para compilar todas las aplicaciones
#
todo:
    make u-str-se -f makefile.1
    make u-str-cl -f makefile.1
    make u-dg-se -f makefile.1
```

```
make u-dg-cl -f makefile.1
make i-tcp-se -f makefile.1
make i-tcp-cl -f makefile.1
make i-udp-se -f makefile.1
make i-udp-cl -f makefile.1
```

La orden para llevar a cabo la compilación es:

```
$ make todo -f makefile.1
```

11.4 Miscelánea de llamadas y funciones

Las llamadas al sistema estudiadas en los apartados anteriores son fundamentales para construir cualquier tipo de programa que se comunique a través de conectores. En este apartado veremos más llamadas y funciones de la biblioteca estándar que amplían el repertorio estudiado y que facilitan la labor del programador.

11.4.1 Nombres de un conector (`getsockname` y `getpeername`)

Para determinar las direcciones de los procesos conectados a un conector, se utilizan las llamadas `getsockname` y `getpeername`. Estas llamadas tienen aplicación en conectores orientados a conexión y se declaran de la siguiente forma:

```
#include <sys/socket.h>
int getsockname (int sfd, void *addr, int *addrlen);
int getpeername (int sfd, void *addr, int *addrlen);
```

La llamada `getsockname` devuelve en la estructura apuntada por `addr` la dirección del conector cuyo descriptor coincide con `sfd` y `getpeername` devuelve en la estructura apuntada por `addr` la dirección del conector que se encuentra unido al conector descrito por `sfd`. En ambas llamadas, `addrlen` le indica a la función cuál es el tamaño en bytes de la estructura apuntada por `addr`, y al salir de la función contiene el tamaño real de la dirección.

Actualmente, en algunos sistemas, la llamada `getsockname` no está disponible para conectores de la familia `AF_UNIX`, pero en el futuro se corregirá esta situación.

11.4.2 Nombre del nodo actual (`gethostname`)

Para determinar el nombre oficial que tiene un nodo dentro de la red, podemos usar la llamada `gethostname`, que se declara como sigue:

```
#include <unistd.h>
int gethostname (char *hostname, size_t size);
```

Esta llamada devuelve en el array apuntado por `hostname` el nombre oficial del nodo que hace la llamada; `size` indica la longitud del array `hostname`. Esta llamada puede implementarse a partir de la llamada `uname`, que es más general.

11.4.3 Construcción de tuberías a partir de conectores (`socketpair`)

En el sistema BSD, la comunicación clásica mediante tuberías se puede llevar a cabo en base a las facilidades de comunicación que aportan los conectores. La llamada `socketpair` se utiliza para crear un par de conectores que posibilitan, cada uno por separado, una comunicación bidireccional. Esta función se declara como sigue:

```
#include <sys/socket.h>
int socketpair (int family, int type, int protocol, int sockvec[2]);
```

Los parámetros `family`, `type` y `protocol` tienen el mismo significado que vimos para la llamada `socket`:

- `family`, familia a la que pertenecerán los conectores creados. Actualmente sólo pueden ser de la familia `AF_UNIX`.
- `type`, tipo del conector: `SOCK_STREAM` o `SOCK_DGRAM`.
- `protocol`, protocolo a emplear, generalmente vale 0.

El parámetro `sockvec` es un array de dos números `int` donde se devuelven los descriptores de los dos conectores creados —recordemos la llamada `pipe` en el § 9.2 pág. 334—.

Como `family` sólo puede tomar el valor `AF_UNIX`, las dos formas posibles de llamar a `socketpair` son:

```
int par_de_sockets [2];
...
socketpair (AF_UNIX, SOCK_STREAM, 0, par_de_sockets);
/* o bien
socketpair (AF_UNIX, SOCK_DGRAM, 0, par_de_sockets);
```

Si la llamada se ejecuta satisfactoriamente, devolverá el valor 0, en caso contrario, -1.

11.4.4 Cierre de un conector (`shutdown`)

Sabemos que la llamada `close` puede utilizarse para cerrar un conector; sin embargo, existe otra llamada que da un control más fino a la hora de cerrarlo. Los conectores son mecanismos de comunicación bidireccionales que posibilitan en todo momento una comunicación dúplex, y en ellos, los datos que fluyen en un sentido son totalmente independientes de los que fluyen en sentido opuesto. La llamada `shutdown` se utiliza para cerrar total o parcialmente un conector. Se declara como sigue:

```
#include <sys/socket.h>
int shutdown (int sfd, int how);
```

donde `sfd` es el descriptor del conector sobre el que actúa la llamada y `how` es el tipo de acción que llevará a cabo. Los valores de `how` que tienen sentido son:

- `SHUT_RD`, para deshabilitar la recepción de datos del conector.

- `SHUT_WR`, para deshabilitar el envío de datos a través del conector.
- `SHUT_RDWR`, para deshabilitar el envío y la recepción de datos, es equivalente a cerrar el conector con una llamada a `close`.

11.4.5 Lectura del fichero `/etc/hosts`

En el fichero `/etc/hosts` se guarda información sobre las direcciones *Internet* y los nombres de los nodos que hay conectados a una red. Para acceder a esta información podemos usar las funciones estándar descritas en la página `gethostent` del manual de UNIX.

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
struct hostent *gethostent ();
struct hostent *gethostbyname (char *name);
struct hostent *gethostbyaddr (const char *addr, int len, int type);
int sethostname (int stayopen);
int endhostent ();
```

Las funciones `gethostent`, `gethostbyname` y `gethostbyaddr` devuelven un puntero a una estructura local del tipo `hostent`, que se define en `<netdb.h>` como sigue:

```
struct hostent {
    char *h_name;
    char **h_aliases;
    int h_addrtype;
    int h_length;
    char **h_addr_list;
};
#define h_addr h_addr_list[0] // Para compatibilizar con versiones anteriores
```

El significado de los distintos campos es el siguiente:

- `h_name`, nombre oficial del nodo.
- `h_aliases`, array con los nombres alternativos del nodo; este array termina con un elemento `NULL`.
- `h_addrtype`, tipo de la dirección que se maneja, siempre es `AF_INET`.
- `h_addr_list`, array con las direcciones de red a que responde el nodo; este array termina con un elemento `NULL`.
- `h_addr`, primer elemento del array `h_addr_list`.

La función `gethostent` lee la siguiente línea significativa del fichero `/etc/hosts`. Si es necesario, abre el fichero en modo sólo lectura. La apertura se produce en la primera llamada a la función. Cuando la función llega al final del fichero, devuelve un puntero `NULL`.

La función `sethostent` abre el fichero `/etc/hosts` y sitúa el puntero de lectura al principio del mismo. Si el indicador `stayopen` es distinto de 0, la base de datos de nodos no se cierra después de cada llamada a `gethostent`.

La función `endhostent` cierra el fichero abierto con `gethostent` o `sethostent`.

La función `gethostbyname` busca secuencialmente desde el principio de `/etc/hosts` hasta que encuentra un nombre de nodo —oficial o alias— que coincida con el especificado en la zona de memoria apuntada por `name`. Si la función llega al final de fichero sin encontrar el nodo buscado, devolverá un puntero NULL.

Por último, la función `gethostbyaddr` busca secuencialmente desde el principio de `/etc/hosts` hasta que encuentra una dirección de nodo que coincida con la especificada en la zona de memoria apuntada por `addr`. El parámetro `len` es el total de bytes de la dirección calculados mediante la operación `sizeof(in_addr)`, y `type` debe ser la constante `AF_INET`. Si la función llega al final de fichero sin encontrar el nodo buscado, devolverá un puntero NULL.

11.5 Ejemplo de aplicación. Transferencia de ficheros

El siguiente ejemplo está constituido por un par cliente-servidor destinado a la transferencia de ficheros entre los distintos ordenadores que hay conectados a una red. Está diseñado para que ofrezca un servicio parecido al programa estándar `rcp` —copia remota—.

El programa servidor atiende a dos tipos fundamentales de peticiones: recibir un fichero y enviar un fichero. Ambas peticiones proceden de los programas clientes que, como vemos en figura 11.7, actuarán como puente entre los dos servidores que se ven involucrados en cada transferencia.

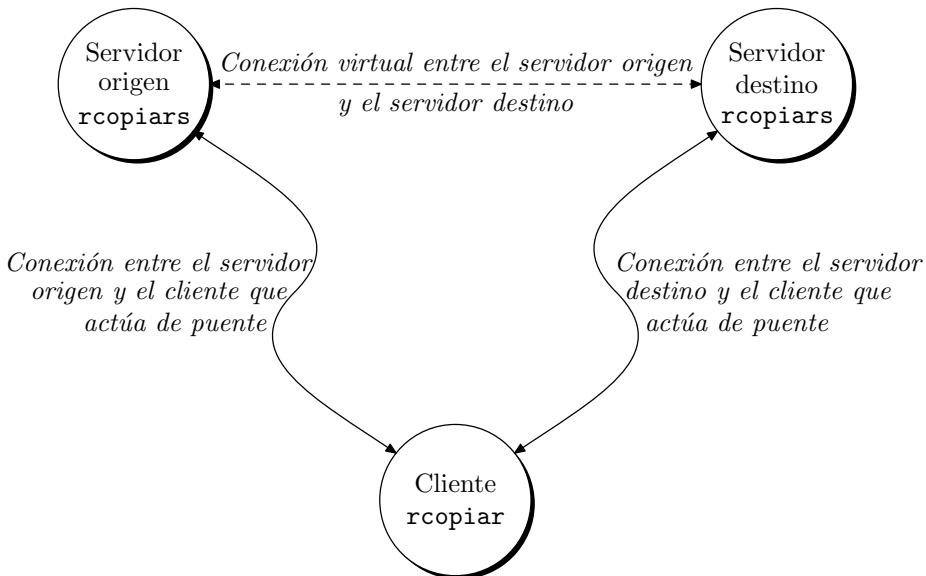


Figura 11.7: Comunicación entre los servidores y el cliente de la aplicación `rcopiar`.

Cada nodo que esté ejecutando el servidor `rcopiars` podrá recibir ficheros procedentes de otros nodos y enviar ficheros a otros nodos. Tanto la lectura del fichero origen como la escritura del fichero destino la realizan los servidores. Cuando un cliente da la orden de copiar un fichero, le enviará un mensaje con el nombre del fichero origen al servidor origen, y el correspondiente mensaje al servidor destino. Acto seguido, el cliente pasa a leer datos procedentes del origen para enviarlos al destino. Estos datos son el contenido del fichero que se está transfiriendo.

Como el servidor que se implementará es del tipo concurrente, la orden `rcopiar` se podrá utilizar para copiar ficheros dentro de una misma máquina; `rcopiar` será, por lo tanto, una generalización de `cp`. Un ejemplo de invocación a `rcopiar` puede ser:

```
$ rcopiar apolo:/etc/hosts marte:/usr/tmp/nodos
```

Esta orden hace que el fichero `/etc/hosts` del nodo `apolo` se copie, con el nombre `nodos`, en el directorio `/usr/tmp` del nodo `mart`. Para que esto funcione correctamente, previamente se debe ejecutar en cada uno de los nodos la orden que arranca al servidor de transferencia remota de ficheros:

```
$ rcopiars 2> rcopiar.log &
```

donde vemos que `rcopiars` es arrancado para que se ejecute en segundo plano, por ello el fichero estándar de salida de errores —descriptor 2— se ha redirigido hacia el fichero `rcopiars.log` para anotar ahí las incidencias que se producen durante la ejecución del servidor.

El primer fichero de la aplicación que vamos a ver es el fichero de cabecera `rcopiar.h`.

Fichero 11.16: Fichero de cabecera (`rcopiar.h`)

```
#ifndef _RCOPIAR_
#define _RCOPIAR_
#include <stdio.h>
#include <fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <signal.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/time.h>
#define MAX_BUF 512
/* Definición de la cabecera de mensaje. */
struct cabecera_mensaje {
    int tipo, /* Tipo del mensaje. */
        longitud; /* Longitud del mensaje sin incluir la cabecera. */
};
/* Definición de mensaje. */
struct mensaje {
    struct cabecera_mensaje cabecera;
    char buf [MAX_BUF];
};
```

```

/* Definición de direcciones y puertos. */
#define PUERTO_SERVIDOR_RCOPIAR 8000

/* Variables externas. */
extern char *nombre_prog;
extern int errno;

/* Códigos empleados por los diferentes tipos de mensajes. */
#define ERROR 1
#define FIN_FICHERO 2
#define ENVIAR_FICHERO 3
#define RECIBIR_FICHERO 4
#define BLOQUE_DATOS 5
#define PERMISOS 6

/* Códigos empleados por la función de contabilidad. */
#define INICIO_CONEXION 1
#define FIN_CONEXION 2

/* Constantes de ayuda a la depuración. */
#define DEP __FILE__, __LINE__

#endif

```

En el módulo `rcop-com.c` hay funciones que serán utilizadas tanto por el programa servidor, `rcopiars.c`, como por el programa cliente, `rcopiar.c`.

Fichero 11.17: Funciones comunes para los clientes y servidores de la utilidad de copia remota (`rcop-com.c`)

```

#include "rcopiar.h"

/* Variables globales. */
char *nombre_prog;

/* Prototipos de funciones locales. */
static enviar_buffer (int sfd, char *buf, int lon_buf);
static recibir_buffer (int sfd, char *buf, int lon_buf);

/**
    error: imprime mensajes de error y termina la ejecución del programa.
***/
error (char *fichero, int linea, char *mensaje)
{
    fprintf (stderr, "PID(%d). ERROR\n", getpid ());
    fprintf (stderr, "\tPrograma: %s\n", nombre_prog);
    fprintf (stderr, "\tFichero fuente: %s\n", fichero);
    fprintf (stderr, "\tLínea de código: %d\n", linea);
}

```

```

    fprintf (stderr, "\tMensaje: %s\n", mensaje);
    fprintf (stderr, "\tNro. error: %d\n", errno);
    fprintf (stderr, "\tError: %s\n", sys_errlist [errno]);
    exit (-1);
}

/****
    aviso: imprime mensajes de error. No termina la ejecución del programa.
****/
aviso (char *fichero, int linea, char *mensaje)
{
    fprintf (stderr, "PID(%d). Aviso\n", getpid ());
    fprintf (stderr, "\tPrograma: %s\n", nombre_prog);
    fprintf (stderr, "\tFichero fuente: %s\n", fichero);
    fprintf (stderr, "\tLínea de código: %d\n", linea);
    fprintf (stderr, "\tMensaje: %s\n", mensaje);
    fprintf (stderr, "\tNro. error: %d\n", errno);
    fprintf (stderr, "\tError: %s\n", sys_errlist [errno]);
}

/****
    recibir: lee un mensaje de un conector del tipo SOCK_STREAM.
****/
recibir (int sfd, struct mensaje *men)
{
    int lon_cab, lon_buf;

    /* Primero leemos la cabecera del mensaje. */
    lon_cab = recibir_buffer (sfd, (char *) &men->cabecera,
                             sizeof (men->cabecera));
    if (lon_cab == -1 || lon_cab != sizeof (men->cabecera)) {
        aviso (DEP, "recibir - cabecera");
        return (-1);
    }

    /* La cabecera informa sobre la longitud del resto del mensaje. */
    if (men->cabecera.longitud == 0)
        return (lon_cab);

    /* Lectura del resto de los datos del mensaje. */
    lon_buf = recibir_buffer (sfd, men->buf, men->cabecera.longitud);
    if (lon_buf == -1 || lon_buf != men->cabecera.longitud) {
        aviso (DEP, "recibir - buffer");
        return (-1);
    }

    return (lon_cab + lon_buf);
}

```

```
}
/****
    enviar: escribe mensajes en un conector del tipo SOCK_STREAM.
****/
int enviar (int sfd, struct mensaje *men)
{
    int lon_cab, lon_buf;

    /* Envío de la cabecera del mensaje. */
    lon_cab = enviar_buffer (sfd, (char *)&men->cabecera,
                             sizeof (men->cabecera));
    if (lon_cab == -1 || lon_cab != sizeof (men->cabecera)) {
        aviso (DEP, "enviar - cabecera");
        return (-1);
    }
    /* Envío de los datos del mensaje. */
    lon_buf = enviar_buffer (sfd, men->buf, men->cabecera.longitud);
    if (lon_buf == -1 || lon_buf != men->cabecera.longitud) {
        aviso (DEP, "enviar - buffer");
        return (-1);
    }

    return (lon_cab + lon_buf);
}

/****
    enviar_buffer: función local para efectuar un envío seguro de mensajes a un
    conector. Esta función fuerza a que el bloque de memoria referenciado por buf
    sea enviado en su totalidad, aunque esté fraccionado.
****/
static enviar_buffer (int sfd, char *buf, int lon_buf)
{
    int bytes_restantes = lon_buf, bytes_enviados;

    while (bytes_restantes > 0) {
        bytes_enviados = send (sfd, buf, bytes_restantes, 0);
        if (bytes_enviados == -1) {
            error (DEP, "enviar_buffer - send");
            return (-1);
        }
        bytes_restantes -= bytes_enviados;
        buf += bytes_enviados;
    }
    return (lon_buf - bytes_restantes);
}
```

```

/****
    recibir_buffer: función local para efectuar una recepción segura de mensajes.
    Esta función fuerza a que el bloque de memoria referenciado por buf sea recibido
    en su totalidad, aunque esté fraccionado.
****/
static recibir_buffer (int sfd, char *buf, int lon_buf)
{
    int bytes_restantes = lon_buf, bytes_recibidos;

    while (bytes_restantes > 0) {
        bytes_recibidos = recv (sfd, buf, bytes_restantes, 0);
        if (bytes_recibidos == -1) {
            error (DEP, "recibir_buffer - recv");
            return (-1);
        }
        bytes_restantes -= bytes_recibidos;
        buf += bytes_recibidos;
    }
    return (lon_buf - bytes_restantes);
}

```

El programa servidor es `rcopiars.c` y su código se muestra a continuación:

Programa 11.18: Servidor de la utilidad de copia remota de ficheros (`rcopiars.c`)

```

/****
    Programa servidor utilizado en el ejemplo de transferencia de ficheros entre
    ordenadores. Este servidor es del tipo concurrente y acepta peticiones de servicio
    a través de conectores de la familia AF_INET del tipo SOCK_STREAM (orientados a
    conexión con el protocolo TCP). La forma de invocar al programa es:

    $ rcopiar 2 > rcopiar_log &

    rcopiar_log es el fichero donde se escriben los mensajes de error y de
    contabilidad.
****/
#include "rcopiar.h"

/****
    main: se encarga de abrir un conector a través del cual se recibirán peticiones de
    servicio de los clientes.
****/
main (int argc, char *argv [])
{
    int sfd, nsfd, pid;
    struct sockaddr_in ser_addr, cli_addr;
    int ser_addr_len, cli_addr_len;

```

```
char hostname [256];
struct hostent *host;
void manejador ();

nombre_prog = argv [0];

/* Ignorar la señal SIGCHLD. */
signal (SIGCHLD, SIG_IGN);
/* Instalación del manejador de señales. */
signal (SIGINT, manejador);
signal (SIGTERM, manejador);

/* Determinación del nombre y dirección del nodo actual. */
if (gethostname (hostname, sizeof (hostname)) == -1)
    error (DEP, "gethostname");
if ((host = gethostbyname (hostname)) == NULL)
    error (DEP, "gethostbyname");

/* Apertura de un conector del tipo STREAM de la familia AF_INET */
if ((sfd = socket (AF_INET, SOCK_STREAM, 0)) == -1)
    error (DEP, "abrir socket");

/* Publicidad de la dirección del servidor. */
printf ("SERVIDOR: %s\n", host->h_name);
printf ("\tDirección: %s\n", inet_ntoa (*(long *)host->h_addr));
ser_addr.sin_family = AF_INET;
ser_addr.sin_addr.s_addr = *(long *)host->h_addr;
ser_addr.sin_port = htons (PUERTO_SERVIDOR_RCOPIAR);
if (bind (sfd, (struct sockaddr *)&ser_addr, sizeof (ser_addr)) == -1)
    error (DEP, "bind");

/* Declaración de una cola con 5 elementos para peticiones de conexión. */
listen (sfd, 5);

/* Bucle de lectura de peticiones de conexión. */
for (;;) {
    cli_addr_len = sizeof (cli_addr);
    if ((nsfd = accept (sfd, (struct sockaddr *) &cli_addr,
                        &cli_addr_len)) == -1)
        error (DEP, "accept");
    contabilidad (nsfd, INICIO_CONEXION);

    /* Creación de un proceso hijo para atender al cliente. */
    if ((pid = fork ()) == -1)
        aviso (DEP, "fork");
    else if (pid == 0) {
```



```

        /* Código del proceso hijo. */
        close (sfd);
        atender_cliente (nsfd);
        close (nsfd);
        contabilidad (nsfd, FIN_CONEXION);
        exit (0);
    }
    /* Código del proceso padre. */
    close (nsfd);
}

}

/****
atender_cliente: función para dar servicio a los clientes. Se podrá atender dos
tipos de peticiones de servicio básicas:
    1 – Petición para recibir un fichero del cliente.
    2 – Petición para enviar un fichero al cliente.
****/
atender_cliente (int sfd)
{
    struct mensaje men, men_err;
    struct stat buf;
    int nbytes, men_len, fd;
    char path [256];

    /* Lectura del primer mensaje procedente del cliente. */
    nbytes = recibir (sfd, &men);
    if (nbytes == -1)
        error (DEP, "atender_cliente (recibir)");
    /* Análisis del tipo de mensaje. */
    switch (men.cabecera.tipo) {
    case ENVIAR_FICHERO:
        /* Apertura del fichero que se le enviará al cliente. */
        if ((fd = open (men.buf, O_RDONLY)) == -1) {
            men_err.cabecera.tipo = ERROR;
            sprintf (men_err.buf, "%s: %d - %s",
                    men.buf, errno, sys_errlist [errno]);
            men_err.cabecera.longitud = strlen (men_err.buf) + 1;
            enviar (sfd, &men_err);
            error (DEP, men_err.buf);
        }
        /* Consultamos los permisos del fichero para enviárselos al cliente. */
        if (fstat (fd, &buf) == -1) {
            men_err.cabecera.tipo = ERROR;
            sprintf (men_err.buf, "%s: %d - %s",
                    "fstat", errno, sys_errlist [errno]);

```

```
        men_err.cabecera.longitud = strlen (men_err.buf) + 1;
        enviar (sfd, &men_err);
        error (DEP, men_err.buf);
    }
    men.cabecera.tipo = PERMISOS;
    memcpy (men.buf, &buf.st_mode, sizeof (buf.st_mode));
    men.cabecera.longitud = sizeof(men.cabecera)+sizeof(buf.st_mode);
    enviar (sfd, &men);

    /* Llamada a la función que le envía los datos del fichero al cliente. */
    enviar_fichero (sfd, fd);
    close (fd);
    break;

case RECIBIR_FICHERO:
    strcpy (path, men.buf);

    /* Recepción de la máscara de permisos del fichero. */
    recibir (sfd, &men);
    if (men.cabecera.tipo != PERMISOS)
        error(DEP, "Esperando mensaje con los permisos del fichero");
    memcpy (&buf.st_mode, men.buf, sizeof (buf.st_mode));
    buf.st_mode &= 0777;
    /* Borramos el fichero destino. */
    if (unlink (path) == -1) {
        men_err.cabecera.tipo = ERROR;
        sprintf (men_err.buf, "%s: %d - %s",
                path, errno, sys_errlist [errno]);
        men_err.cabecera.longitud = strlen (men_err.buf) + 1;
        enviar (sfd, &men_err);
        error (DEP, men_err.buf);
    }
    /* Apertura del fichero donde se almacenarán los datos procedentes del
    cliente. */
    if ((fd=open(path,O_WRONLY|O_TRUNC|O_CREAT,buf.st_mode)) == -1) {
        men_err.cabecera.tipo = ERROR;
        sprintf (men_err.buf, "%s: %d - %s",
                path, errno, sys_errlist [errno]);
        men_err.cabecera.longitud = strlen (path) + 1;
        enviar (sfd, &men_err);
        error (DEP, men_err.buf);
    }
    /* Llamada a la función de recepción de los datos del fichero. */
    recibir_fichero (sfd, fd);
    close (fd);
    break;
```

```

default:
    men_err.cabecera.tipo = ERROR;
    strcpy (men_err.buf, "Mensaje desconocido");
    men.cabecera.longitud = strlen (men_err.buf) + 1;
    enviar (sfd, &men_err);
    error (DEP, men_err.buf);
}

}

/****
    enviar_fichero: función para enviar un fichero a un proceso cliente.
****/
enviar_fichero (int sfd, int fd)
{
    int nbytes, men_len;
    struct mensaje men;

    /* Lectura del contenido del fichero y envío del mismo al proceso cliente. */
    while ((nbytes = read (fd, men.buf, sizeof (men.buf))) != 0) {
        if (nbytes == -1) {
            men.cabecera.tipo = ERROR;
            sprintf (men.buf, "%d - %s", errno, sys_errlist [errno]);
            men.cabecera.longitud = strlen (men.buf) + 1;
            enviar (sfd, &men);
            exit (-1);
        }
        men.cabecera.tipo = BLOQUE_DATOS;
        men.cabecera.longitud = nbytes;
        men_len = sizeof (men.cabecera) + men.cabecera.longitud;
        if (enviar (sfd, &men) != men_len)
            error (DEP, "enviar_fichero (enviar - BLOQUE_DATOS)");
    }
    /* Envío del mensaje de final de fichero. */
    men.cabecera.tipo = FIN_FICHERO;
    men.cabecera.longitud = 0;
    men_len = sizeof (men.cabecera);
    if (enviar (sfd, &men) != men_len)
        error (DEP, "enviar_fichero (enviar - FIN_FICHERO)");
}

/****
    recibir_fichero: función para recibir un fichero procedente del proceso cliente.
****/
recibir_fichero (int sfd, int fd)
{
    int nbytes;

```

```

    struct mensaje men;
    int men_len;

    for (;;) {
        men_len = recibir (sfd, &men);
        if (men_len == -1)
            error (DEP, "recibir_fichero (recibir)");
        switch (men.cabecera.tipo) {
            case FIN_FICHERO:
                return (0);
            case BLOQUE_DATOS:
                nbytes = write (fd, men.buf, men.cabecera.longitud);
                if (nbytes == -1)
                    error (DEP, "recibir_fichero (write)");
                break;
            case ERROR:
                error (DEP, men.buf);
            default:
                error (DEP, "recibir_fichero - tipo de mensaje incorrecto");
        }
    }
}

/****
    contabilidad: determina el tiempo de servicio prestado a cada uno de los clientes.
****/
contabilidad (int sfd, int modo)
{
    struct sockaddr_in dir;
    int lon_dir = sizeof (dir);
    time_t fecha;
    struct hostent *host;

    time (&fecha);
    getpeername (sfd, (struct sockaddr *) &dir, &lon_dir);
    host=gethostbyaddr((char *) &dir.sin_addr,
                      sizeof (dir.sin_addr), dir.sin_family);
    printf ("CLIENTE. %s\n", host->h_name);
    printf ("\tDirección(%s), Puerto (%d)\n",
            inet_ntoa (dir.sin_addr), dir.sin_port);
    if (modo == INICIO_CONEXION)
        printf ("\tInicio de la conexión:");
    else if (modo == FIN_CONEXION)
        printf ("\tFin de la conexión:");
    else
        printf ("\t");
}

```

```

    printf ("%s", asctime (localtime (&fecha)));
}

/****
manejador: manejador de algunas de las señales que puede recibir el programa.
Esta función sacará un mensaje por pantalla indicando el tipo de señal y llamará
a exit para terminar la ejecución del programa.
****/
void manejador (int sig)
{

    printf ("Recibida la señal nro. %d\n", sig);
    exit (0);
}

```

Por último, el programa cliente se encuentra en el fichero `rcopiar.c`, como podemos ver a continuación:

Programa 11.19: Cliente de la utilidad de copia remota de ficheros. (`rcopiar.c`)

```

/****
Programa cliente para transferir ficheros entre los ordenadores conectados a una
red. Su forma de uso es:

$ rcopiar origen destino

origen y destino han de tener la forma:

nombre_nodo:ruta_absoluta
****/
#include "rcopiar.h"

/* Estructura para contener el nombre de un fichero. */
struct nombre_fichero {
    char hostname [256], pathname [256];
};

/****
partir_nombre_fichero: divide el nombre completo de un fichero en dos partes:
    - nombre del nodo
    - ruta absoluta del fichero.
****/
struct nombre_fichero partir_nombre_fichero (char *nombre)
{
    struct nombre_fichero nom;
    int i, lon = strlen (nombre);

```

```

    for (i = 0; i < lon; i++)
        if (nombre [i] == ':') {
            strncpy (nom.hostname, nombre, i);
            nom.hostname [i] = 0;
            strcpy (nom.pathname, nombre + i + 1);
            return (nom);
        }
    nom.hostname[0] = 0;
    strcpy (nom.pathname, nombre);
    return (nom);
}

/****
main: función principal. Se encarga de:
    - Analizar los argumentos de la línea de órdenes.
    - Abrir dos conectores: uno para comunicarse con el servidor donde se
      encuentra el fichero origen y otro para comunicarse con el servidor
      donde se encontrará el fichero destino.
    - Establecer conexión con ambos servidores.
    - Enviar el nombre de cada uno de los ficheros a sus respectivos servidores.
    - Llamar a la función que comunica a los dos servidores.
****/
main (int argc, char *argv [])
{
    int sfdo, sfdd;
    struct sockaddr_in dorigen, ddestino;
    struct nombre_fichero forigen, fdestino;
    struct hostent *host;
    struct mensaje men;
    int lon_men;

    nombre_prog = argv[0];

    /* Análisis de los argumentos de la línea de órdenes.*/
    if (argc != 3) {
        fprintf (stderr, "Forma de uso: %s origen destino\n", nombre_prog);
        exit (-1);
    }
    forigen = partir_nombre_fichero (argv [1]);
    if (forigen.hostname[0] == 0)
        if (gethostname(forigen.hostname, sizeof(forigen.hostname)) == -1)
            error (DEP, "gethostname origen");
    fdestino = partir_nombre_fichero (argv [2]);
    if (fdestino.hostname[0] == 0)
        if (gethostname(fdestino.hostname, sizeof(fdestino.hostname)) == -1)
            error (DEP, "gethostname destino");

```

```

/* Lectura en el fichero /etc/hosts de las direcciones Internet asociadas al nodo
donde reside el fichero origen.*/
if ((host = gethostbyname (forigen.hostname)) == NULL)
    error (DEP, forigen.hostname);
dorigen.sin_family = AF_INET;
dorigen.sin_addr.s_addr = *(long *)host->h_addr;
dorigen.sin_port = htons (PUERTO_SERVIDOR_RCOPIAR);
/* Lectura en el fichero /etc/hosts de las direcciones Internet asociadas al nodo
donde reside el fichero destino.*/
if ((host = gethostbyname (fdestino.hostname)) == NULL)
    error (DEP, fdestino.hostname);
ddestino.sin_family = AF_INET;
ddestino.sin_addr.s_addr = *(long *)host->h_addr;
ddestino.sin_port = htons (PUERTO_SERVIDOR_RCOPIAR);

/* Apertura de un conector de la familia AF_INET orientado a conexión por donde
se recibirán los datos del fichero origen. */
if ((sfdo = socket (AF_INET, SOCK_STREAM, 0)) == -1)
    error (DEP, "apertura del socket origen");
/* Petición de conexión con el servidor donde está el fichero origen. */
if (connect (sfdo, (struct sockaddr *)&dorigen, sizeof(dorigen)) == -1)
    error (DEP, "conexión con origen");
/* Apertura de un conector de la familia AF_INET orientado a conexión por donde
se enviarán los datos hacia el fichero destino. */
if ((sfdd = socket (AF_INET, SOCK_STREAM, 0)) == -1)
    error (DEP, "apertura del socket destino");
/* Petición de conexión con el servidor donde estará el fichero destino. */
if (connect(sfdd, (struct sockaddr *)&ddestino, sizeof(ddestino)) == -1)
    error (DEP, "conexión con destino");

/* Envío del nombre de fichero al servidor origen. */
men.cabecera.tipo = ENVIAR_FICHERO;
strcpy (men.buf, forigen.pathname);
men.cabecera.longitud = strlen (men.buf) + 1;
if (enviar (sfdo, &men) == -1)
    error (DEP, "enviar - nombre origen");

/* Envío del nombre de fichero al servidor destino. */
men.cabecera.tipo = RECIBIR_FICHERO;
strcpy (men.buf, fdestino.pathname);
men.cabecera.longitud = strlen (men.buf) + 1;
if (enviar (sfdd, &men) == -1)
    error (DEP, "enviar - nombre destino");

/* Comunicación entre los dos servidores. */
conectar_sockets (sfdo, sfdd);

```

```

    close (sfdo);
    close (sfdd);
    exit (0);
}

/****
    conectar_sockets: la función hace de puente entre dos conectores. Los datos
    recibidos por el conector origen se envían hacia el conector destino, a menos que
    se reciba un mensaje de error.
****/
conectar_sockets (int sfdo, int sfdd)
{
    struct mensaje men;
    int lon_men, nbytes;

    do {
        lon_men = recibir (sfdo, &men);
        if (lon_men == -1)
            error (DEP, "conectar_sockets (recibir)");
        if (men.cabecera.tipo == ERROR) {
            fprintf (stderr, "(%s) %s\n", nombre_prog, men.buf);
            exit (-1);
        }
        /* Antes de enviar el mensaje recibido, consultamos el conector que nos
        comunica con el servidor destino, por si hay algún mensaje de error
        procedente de él. */
        if (hay_mensaje (sfdd)) {
            lon_men = recibir (sfdd, &men);
            if (lon_men == -1)
                error (DEP, "conectar_sockets (recibir)");
            fprintf (stderr, "(%s) %s\n", nombre_prog, men.buf);
            exit (-1);
        }
        /* Envío del mensaje origen hacia el servidor destino. */
        nbytes = enviar (sfdd, &men);
        if (nbytes == -1 || nbytes != lon_men)
            error (DEP, "conectar_sockets (enviar)");
    } while (men.cabecera.tipo != FIN_FICHERO);

    return (0);
}

/****
    hay_mensaje: función para determinar si hay algún mensaje pendiente de lectura
    en un conector.
****/

```



```

hay_mensaje (int sfd)
{
    fd_set fdread; /* Variable para los descriptores de los ficheros a revisar. */
    struct timeval timeout;

    FD_ZERO (&fdread);
    FD_SET (sfd, &fdread);
    timeout.tv_sec = 0;
    timeout.tv_usec = 0;

    if (select (sfd + 1, &fdread, 0, 0, &timeout) == -1)
        error (DEP, "hay_mensaje (select)");
    if (FD_ISSET (sfd, &fdread))
        return (!0);
    else
        return (0);
}

```

Para compilar la aplicación podemos emplear el siguiente fichero `makefile`.

Fichero 11.20: Fichero para la compilación de las utilidades de transferencia remota de ficheros. (`makefile.2`)

```

#####
# Fichero con la descripción de los módulos que intervienen en la compilación del
# cliente y el servidor de copia remota de ficheros.
#####

rcopiars: makefile.2 rcop-com.o rcopiars.o
    cc -o rcopiars rcopiars.o rcop-com.o

rcopiar: makefile.2 rcop-com.o rcopiar.o
    cc -o rcopiar rcopiar.o rcop-com.o

rcopiars.o: rcopiar.h

rcopiar.o: rcopiar.h

rcop-com.o: rcopiar.h

todo:
    make rcopiars -f makefile.2
    make rcopiar -f makefile.2

```

11.6 Ejercicios

- 11.1** Implemente las funciones `readv` y `writv` a partir de llamadas a `read` y `write`. Observe que ambas llamadas realizan una escritura de tipo atómico, por lo que internamente sólo deben hacer una llamada a `read` o `write`. Esto nos fuerza a manipular las memorias intermedias de las dos llamadas antes de hacer la llamada a `read` o `write`.
- 11.2** Implemente las funciones `inet_addr` e `inet_ntoa` de acuerdo con la funcionalidad descrita en la página `inet(3N)` del manual de UNIX.
- 11.3** A partir de las llamadas `socketpair` y `shutdown`, implemente la función `tubería` con la misma operatividad que la llamada `pipe`. Recuerde que una tubería no es un canal unidireccional.
- 11.4** Modifique los programas cliente y servidor estudiados en el § 10.4.4 pág. 391, que permitían la gestión de una base de datos centralizada utilizando colas de mensajes. Escriba una nueva versión de estos programas, pero a partir de la interfaz de conectores de BSD. Elabore dos nuevas versiones de estos programas: la primera debe utilizar conectores de la familia `AF_UNIX`, y la segunda, conectores de la familia `AF_INET`. Para facilitar su implementación, utilice conectores del tipo flujo de datos —`SOCK_STREAM`—.
- 11.5** Implemente la llamada al sistema `gethostname` a partir de la llamada más general `uname`.
- 11.6** Recodifique el programa cliente `rcopiar.c` para que reconozca los caracteres comodín `*` y `?` a la hora de recibir nombres de ficheros. Recuerde que el análisis de los caracteres comodín lo lleva a cabo el intérprete de órdenes y que, de cara al programa, el uso de estos caracteres se traduce en que recibirá más de un fichero como origen. También queremos que el programa cliente reconozca los nombres `.` —directorio actual— y `..` —directorio padre del actual—. Ejemplos de uso de acuerdo a esta nueva funcionalidad pueden ser:
- ```
$ rcopiar apolo:/usr/include/*.h .

$ rcopiar *.c marte:/usr/local/src
```
- 11.7** Implemente un servidor y un cliente de intercambio de ficheros entre máquinas mediante protocolo `UDP`. Recuerde que este protocolo no garantiza la distribución de los mensajes ni la conservación de su secuencia original. Esto obligará a diseñar un protocolo a nivel de transferencia de ficheros que garantice la fiabilidad de la comunicación. Una posible solución es incluir en cada mensaje un número secuencial que informe al servidor sobre el orden en el que debe ensamblar los mensajes que recibe. Esto también puede ayudar a detectar cuándo se pierde algún mensaje. Si se pierden mensajes, el servidor debe pedir al cliente que los retransmita.



# Parte IV

## Apéndices

*«Porque ¿cómo queréis vos que no me tenga confuso el qué dirá el antiguo legislador que llaman vulgo cuando vea que, al cabo de tantos años como ha que duermo en el silencio del olvido, salgo ahora, con todos mis años auestas, con una leyenda seca como un esparto, ajena de invención, menguada de estilo, pobre de concetos y falta de toda erudición y doctrina, sin acotaciones en las márgenes y sin anotaciones en el fin del libro, como veo que están otros libros, aunque sean fabulosos y profanos, tan llenos de sentencias de Aristóteles, de Platón y de toda la caterva de filósofos, que admiran a los leyentes y tienen a sus autores por hombres leídos, eruditos y elocuentes?»*

*(Cervantes, Quijote I-Prólogo)*

|                                                 |     |
|-------------------------------------------------|-----|
| A El lenguaje de programación C                 | 487 |
| B Desarrollo de aplicaciones en el entorno UNIX | 509 |
| C Resumen de llamadas al sistema                | 541 |
| Referencias                                     | 575 |
| Índice alfabético                               | 581 |



# Apéndice A

## El lenguaje de programación C

*«...y la discreción es la gramática del buen lenguaje, que se acompaña con el uso.»*

*(Cervantes, Quijote II-19)*

---

|     |                                            |     |
|-----|--------------------------------------------|-----|
| A.1 | Introducción . . . . .                     | 487 |
| A.2 | Ciclo de creación de un programa . . . . . | 488 |
| A.3 | Tipos de datos . . . . .                   | 490 |
| A.4 | Expresiones y operadores . . . . .         | 497 |
| A.5 | Sentencias de control de flujo . . . . .   | 499 |
| A.6 | Funciones . . . . .                        | 503 |

---

### A.1 Introducción

C es un lenguaje de programación de alto nivel desarrollado por Dennis Ritchie para codificar el sistema operativo UNIX [Ritchie, 1993]. Las primeras versiones de UNIX se escribieron en ensamblador, pero a partir de 1973 pasaron a escribirse en C. Sólo un pequeño porcentaje del núcleo de UNIX se sigue codificando en ensamblador: en concreto, aquellas partes íntimamente relacionadas con el hardware. Todas las órdenes y aplicaciones estándar que acompañan al sistema UNIX están escritas también en C. Ésta es la razón que hace de este lenguaje la forma natural de comunicarse con el sistema.

En UNIX también se puede programar con otros lenguajes, pero a la hora de desarrollar aplicaciones enfocadas a aprovechar al máximo los recursos del sistema es conveniente tomar la decisión de escribirlas en C.

El lenguaje se puede clasificar dentro del grupo de los lenguajes de alto nivel, aun a pesar de no estar fuertemente tipado; sin embargo, también le ofrece al programador posibilidades que sólo están presentes en los lenguajes de bajo nivel. Así, por ejemplo, en C se pueden manipular bits y aritmética de direcciones. C también permite el desarrollo de la programación estructurada y modular.

## A.2 Ciclo de creación de un programa

A la hora de crear un programa, hemos de empezar por la edición de un fichero que contendrá el código fuente. Este fichero se nombra, por convenio, añadiéndole la extensión `.c`. Si nos valemos del editor `vi`, la forma de editar el programa será:

```
$ vi programa.c
```

El compilador `cc` es el encargado de generar el fichero ejecutable a partir del fichero fuente. Para invocarlo debemos escribir:

```
$ cc programa.c
```

Esta línea de órdenes provoca que se genere el fichero `a.out`, que ya es ejecutable. Si queremos que nuestro programa ejecutable tenga algún nombre en concreto, compilaremos con la orden:

```
$ cc -o programa programa.c
```

lo que provoca que el fichero ejecutable se llame `programa`, en lugar de `a.out`.

El programa `cc` no es realmente el compilador, sino un programa estándar que se encarga de invocar al compilador. Los pasos involucrados en la compilación de `programa.c` se pueden consultar en el § B.2 pág. 511.

Para compilar un programa también podemos utilizar la orden `make`. Si queremos crear el programa `ejemplo` a partir del fichero fuente `ejemplo.c`, bastará con escribir:

```
$ make ejemplo
```

Además, `make` también se emplea para compilar programas compuestos por varios módulos fuente, como se puede ver en el § B.8 pág. 526.

### A.2.1 Componentes léxicos del lenguaje

Existen seis clases de componentes léxicos: identificadores, palabras reservadas, constantes, cadenas de caracteres, operadores y otros separadores.

- *Identificador*. Es una secuencia de letras y dígitos donde el primer elemento debe ser una letra, o los caracteres `_` y `$`. Las letras mayúsculas y minúsculas se consideran distintas. En toda implementación deben ser significativos al menos los 32 primeros caracteres de un identificador.
- *Palabras reservadas*. Las siguientes palabras no se pueden usar como identificadores ya que tienen un significado especial para el compilador:

|                       |                      |                    |                       |                       |                       |
|-----------------------|----------------------|--------------------|-----------------------|-----------------------|-----------------------|
| <code>auto</code>     | <code>default</code> | <code>float</code> | <code>register</code> | <code>struct</code>   | <code>volatile</code> |
| <code>break</code>    | <code>do</code>      | <code>for</code>   | <code>return</code>   | <code>switch</code>   | <code>while</code>    |
| <code>case</code>     | <code>double</code>  | <code>goto</code>  | <code>short</code>    | <code>typedef</code>  |                       |
| <code>char</code>     | <code>else</code>    | <code>if</code>    | <code>signed</code>   | <code>union</code>    |                       |
| <code>const</code>    | <code>enum</code>    | <code>int</code>   | <code>sizeof</code>   | <code>unsigned</code> |                       |
| <code>continue</code> | <code>extern</code>  | <code>long</code>  | <code>static</code>   | <code>void</code>     |                       |

- *Constantes*. Consideramos 4 tipos: enteras, de carácter, flotantes y de enumeración. Veamos ejemplos de cada tipo:

- Enteras: 120, -10 —números enteros—;
  - De carácter: 'a', '\n', '\t', '\xfa', '\0', '\033';
  - Flotantes: 123.42, 1.7E-6, 3.45e10 —números racionales—;
  - Enumeración: enum boolean {NO, SI}.
- *Cadenas de caracteres*. Secuencia de caracteres alfanuméricos entre comillas dobles. Por ejemplo "Esto es una cadena de caracteres".
  - *Operadores*. Caracteres para identificar los operadores unarios, binarios y ternarios. Por ejemplo: +, \*, &&
  - *Otros separadores*: {, }, [, ], (, ), ;, -, >, .
  - *Comentarios*. Secuencia de caracteres que se inicia con /\* y termina con \*/. Los comentarios no se pueden anidar y son ignorados por el compilador. Los comentarios de línea se inician con // y se extienden hasta el final de la línea.

## A.2.2 Estructura de un programa C

Aunque las formas de un programa C pueden ser muy variadas y no hay normas fijas, suele ser común estructurar un programa como sigue:

1. #directrices para el preprocesador.
2. Declaración de variables y funciones externas.
3. Declaración de variables globales y prototipo de las funciones.
4. Funciones, la función `main` debe aparecer en un módulo, y sólo en uno.

Las directrices del preprocesador son órdenes que ejecuta el preprocesador —`cpp`— para generar el fichero con extensión `.i` con el que posteriormente trabajará el compilador. Hay dos directrices que se emplean masivamente en los programas: `include` y `define`.

- `include` se emplea para indicar los ficheros de cabecera donde están definidos tipos derivados y los prototipos de las funciones.
- `define` se emplea para declarar identificadores sinónimos de otros identificadores o constantes. También se emplea para declarar macros.

El siguiente ejemplo nos ayudará a comprender cómo es el aspecto que tiene un fichero fuente C.

```
#include <stdio.h> // Fichero de cabecera para la biblioteca estándar de E/S
// Símbolos que actúan como constantes.
#define VALOR_INICIAL 0
#define VALOR_FINAL 10
#define INCREMENTO 1
```



```
// Función principal
main ()
{
 int i;

 i = VALOR_INICIAL;
 while (i < VALOR_FINAL) {
 printf ("i = %d\n", i);
 i = i+1;
 }
}
```

## A.3 Tipos de datos

En C se consideran dos grandes bloques de datos: los que suministra el lenguaje —tipos fundamentales— y los que define el programador —tipos derivados—.

### A.3.1 Tipos fundamentales

Los tipos fundamentales se clasifican en enteros y reales. Los primeros se utilizan para representar subconjuntos de los números naturales  $\mathbb{N}$  y enteros  $\mathbb{Z}$ , los segundos se emplean para representar un subconjunto de los números racionales  $\mathbb{Q}$ .

#### Tipos enteros

Para declarar variables de alguno de los tipos enteros emplearemos las palabras reservadas `char`, `short`, `int`, `long` y `enum`.

- `char` define un número entero de 8 bits. Su rango es  $[-128, 127]$ . También se emplea para representar el conjunto de caracteres *ASCII*.
- `int` define un número entero de 16 ó 32 bits dependiendo del procesador. Su tamaño suele coincidir con el del *bus* de datos del procesador.
- `long` define un número entero de 32 ó 64 bits.
- `short` define un número entero de tamaño menor o igual que `int`.

Se debe cumplir que  $\text{tamaño}(\text{short}) \leq \text{tamaño}(\text{int}) \leq \text{tamaño}(\text{long})$ .

Estos cuatro tipos pueden ir precedidos del modificador `unsigned` para indicar que el tipo sólo representa números positivos o el cero.

Ejemplos de variables de estos tipos son:

```
int hora;
char carácter;
unsigned short mes;
```

- **enum** se utiliza para definir un subconjunto dentro del conjunto de los números enteros. A este subconjunto se le conoce como enumeración y a cada uno de sus elementos se le asocia un identificador. La definición de un tipo enumerado es:

```
enum tipo {identificador1, identificador2, ..., identificadorn};
```

Un ejemplo de tipo enumerado es:

```
enum meses {enero=1, febrero, marzo, abril, mayo, junio, julio,
 agosto, septiembre, octubre, noviembre, diciembre};
```

## Tipos reales

Para declarar variables de alguno de los tipos reales emplearemos las palabras reservadas **float** y **double**.

- **float** define un número en coma flotante de precisión simple, el tamaño de este tipo suele ser 4 bytes.
- **double** define un número en coma flotante de precisión doble, el tamaño de este tipo suele ser de 8 bytes. El tipo **double** puede ir precedido del modificador **long**, lo que indica que su tamaño pasa a ser de 10 bytes.

Los siguientes son ejemplos de variables en coma flotante:

```
float temperatura = 36.5;
double distancia = 128e64;
```

### A.3.2 Operador sizeof

Para determinar el tamaño, en bytes, tanto de un tipo fundamental como de uno derivado, podemos usar el operador **sizeof**. Esto ayudará a que nuestros programas sean más transportables entre máquinas donde el tamaño de los tipos no coincida.

El siguiente ejemplo es un programa que muestra el tamaño de cada uno de los tipos fundamentales.

```
#include <stdio.h>

main ()
{

 printf ("TIPOS FUNDAMENTALES DE C.\n");
 printf ("char\t\t==>\t %d bytes\n", sizeof (char));
 printf ("unsigned char\t==>\t %d bytes\n", sizeof (unsigned char));
 printf ("short\t\t==>\t %d bytes\n", sizeof (short));
 printf ("unsigned short\t==>\t %d bytes\n", sizeof (unsigned short));
 printf ("int\t\t==>\t %d bytes\n", sizeof (int));
 printf ("unsigned int\t==>\t %d bytes\n", sizeof (unsigned int));
 printf ("long\t\t==>\t %d bytes\n", sizeof (long));
}
```

```

printf ("unsigned long\t==>\t %d bytes\n", sizeof (unsigned long));
printf ("float\t\t==>\t %d bytes\n", sizeof (float));
printf ("double\t\t==>\t %d bytes\n", sizeof (double));
printf ("long double\t==>\t %d bytes\n", sizeof (long double));
}

```

### A.3.3 Tipos derivados

Los tipos derivados se construyen a partir de los tipos fundamentales o de otros tipos derivados. Vamos a estudiar los siguientes: arrays, punteros, estructuras, uniones y campos de bits.

#### Arrays

Los arrays son bloques de elementos del mismo tipo. El tipo base de un array puede ser un tipo fundamental o un tipo derivado. Los elementos individuales del array son accesibles mediante una secuencia de índices. Los índices deben ser variables o constantes de tipo entero. Se define la dimensión de un array como el total de índices que necesitamos para acceder a un elemento particular del array. La definición formal de un array N-dimensional es la siguiente:

```
tipo_array nombre_array [rango1] [rango2] ... [rangoN]
```

A continuación podemos ver ejemplos de arrays:

```

// Matriz de números enteros de 10 filas por 10 columnas
int matriz_entera [10][10];
// Array unidimensional de 5 elementos
float vector_real [5];

```

A los arrays unidimensionales se les llama *vectores*, y a los bidimensionales, *matrices*. Para indexar los elementos de un array, tendremos en cuenta que los índices deben variar entre 0 y  $M - 1$ , donde  $M$  es el tamaño de la dimensión a la que se refiere el índice. Para el array `vector_real` declarado antes, el índice variará entre 0 y 4: `vector_real[0]`, `vector_real[1]`, ..., `vector_real[4]`.

Los arrays unidimensionales de tipo `char` se conocen como *cadenas de caracteres*, y los arrays de cadenas de caracteres —matrices de caracteres— se conocen como *tablas*.

### A.3.4 Punteros

Los punteros son variables que almacenan direcciones de memoria. Se definen también en base a un tipo fundamental o a un tipo derivado. La declaración de un puntero es como sigue:

```
tipo_base *puntero;
```

Por ejemplo, `float *x`; declara una variable de nombre `x` que contiene una dirección de memoria donde se encuentra un número `float`. Hay que poner especial atención para no confundir la dirección a la que apunta el puntero con lo que hay en esa dirección.

Para trabajar con punteros hay definidos dos operadores unarios: `&` y `*`. El operador `&` da la dirección de memoria asociada a una variable y se utiliza para inicializar un puntero. El operador `*` se utiliza para referirse al contenido de una dirección de memoria. El siguiente ejemplo aclara estos conceptos:

```
float x, *px;
...
px = &x; // En px almacenamos la dirección donde se encuentra x
*px = 10.05; // Es equivalente a x=10.05, a través de px se puede acceder
 // de una forma indirecta a la variable x
```

El acceso a una variable a través de un puntero se conoce también como indirección. Tendremos siempre presente que antes de manipular el contenido de un puntero hay que inicializar el puntero para que referencie una zona de memoria válida.

Los punteros admiten las operaciones de incremento, decremento, suma de una constante entera y diferencia de punteros. Al realizar estas operaciones, realmente estamos modificando la dirección a la que referencia el puntero.

```
int array [10];
int *p = array;
...
*p = 5; // Equivale a array[0] = 5
p = p+2;
*p = 3; // Equivale a array [2] = 3
```

El ejemplo anterior muestra cómo podemos acceder a los elementos de un array a través de un puntero. La equivalencia entre arrays y punteros es tan grande que a la hora de indexar los elementos de un array también podemos hacerlo con punteros.

```
int array [10], *p = array, i = 4;
...
*(p + i) = 7; // Esto equivale a array [i] = 7
p [i] = 9; // Esto es equivalente a array [i] = 9
```

Sin embargo, un array no es lo mismo que un puntero. Mientras que un array es una constante —la dirección de inicio del array—, un puntero es una variable que puede tomar como valor cualquier dirección.

Uno de los inconvenientes que plantea el uso de arrays es que sus dimensiones deben ser conocidas para el compilador. Los punteros nos ayudan a solucionar este problema, ya que posibilitan el uso de arrays dinámicos —arrays que se dimensionan en tiempo de ejecución—. Para hacer esto posible, necesitamos valernos de funciones como `malloc`, que reservan memoria según las necesidades del programa. En el § 6.5.1 pág. 202 se pueden consultar más detalles sobre esta función. El ejemplo siguiente ilustra la forma de usarla:

```
int *p;
...
// Reserva memoria para 20 números que serán accesibles través del puntero p
p = (int *) malloc (20*sizeof(int));
if (p == NULL) {
 // Error en la reserva de memoria.
```

```

 // Tratamiento del error.
}

```

## Estructuras

Una estructura es un agregado de tipos fundamentales o derivados y se compone de varios campos. A diferencia de los arrays, cada elemento de la estructura puede ser de un tipo diferente. La forma de definir una estructura es:

```

struct nombre_estructura {
 tipo1 campo1;
 tipo2 campo2;
 ...
 tipon campon;
};

```

En el ejemplo siguiente se define una estructura de nombre `fecha` y una variable de nombre `hoy` cuyo tipo es la estructura `fecha`:

```

struct fecha {
 unsigned short día;
 unsigned short mes;
 unsigned int año;
};
...
struct fecha hoy;

```

Para acceder a los campos de una estructura utilizaremos el operador `..`. Así, para acceder al campo `mes` de la variable `hoy` escribiremos `hoy.mes = 3;`.

Se pueden declarar arrays de estructuras, como muestra el ejemplo siguiente:

```

struct {
 float real, imaginaria;
} vector [10];

```

La variable `vector` es un array unidimensional de números complejos. Para acceder a la parte real del elemento 5, escribiremos `vector[4].real = 10.0;`.

También se pueden declarar punteros a estructuras. En estos casos, el acceso a los campos de la variable se hace por medio del operador `->` como se muestra en el siguiente ejemplo:

```

struct Alumno {
 char nombre [61];
 float nota;
};
struct Alumno alumno, *pa;
...
pa = &alumno;
pa->nota = 10.0;

```

Por último, diremos que puesto que el tipo de cada campo de una estructura puede ser un tipo fundamental o derivado, también puede ser otra estructura. Tendremos así declaradas estructuras dentro de estructuras.

```
struct Alumno {
 char nombre [61];
 struct fecha fecha_nacimiento; // fecha ha sido declarada previamente
 float nota;
};
struct Alumno alumno;
```

Con estas declaraciones podremos hacer asignaciones como `alumno.fecha_nacimiento.mes = 3;`

## Uniones

Las uniones se definen de forma parecida a las estructuras y se emplean para almacenar en un mismo espacio de memoria variables de distintos tipos. La declaración de una unión es la siguiente:

```
union nombre_unión {
 tipo1 campo1;
 tipo2 campo2;
 ...
 tipon campon;
};
```

El tamaño de la unión no es igual a la suma de cada uno de sus campos, como ocurre con las estructuras, sino que es igual al tamaño del mayor de sus campos.

```
union número_mixto {
 float real;
 int entero;
};
...
union número_mixto número;
```

El tamaño de esta unión es de 4 bytes —tamaño del campo **real**—; con esta declaración podremos hacer asignaciones tales como:

```
número.entero = 10;
número.real = 125.0;
```

Hay que insistir en que esta unión no ocupa el espacio de sus dos campos, sino el del mayor de ellos. Si después de las asignaciones anteriores imprimimos el valor de `número.entero`, veremos que no es 10.

## Campos de bits

C brinda la posibilidad de definir variables cuyo tamaño en bits puede no coincidir con un múltiplo de 8. La forma de declararlas es:

```

struct nombre_campo {
 unsigned campo1:tamaño1;
 unsigned campo2:tamaño2;
 ...
 unsigned campon:tamañon;
};

```

Veamos un ejemplo:

```

struct byte {
 unsigned char b0:1;
 unsigned char b1:1;
 unsigned char b2:1;
 unsigned char b3:1;
 unsigned char b4:1;
 unsigned char b5:1;
 unsigned char b6:1;
 unsigned char b7:1;
};
union BYTE {
 unsigned char byte;
 struct byte campo;
};
struct BYTE byte_1, byte_2;
...
byte_1.byte = 0x76;
byte_2.campo.b0 = byte_1.campo.b7;
byte_2.campo.b1 = byte_1.campo.b6;
byte_2.campo.b2 = byte_1.campo.b5;
byte_2.campo.b3 = byte_1.campo.b4;
byte_2.campo.b4 = byte_1.campo.b3;
byte_2.campo.b5 = byte_1.campo.b2;
byte_2.campo.b6 = byte_1.campo.b1;
byte_2.campo.b7 = byte_1.campo.b0;
byte_1.byte = byte_2.byte; // byte_1.byte = 0x6E

```

Con la secuencia de código anterior conseguimos reflejar los bits de un byte. Ese mismo resultado se podría haber conseguido más eficientemente con los operadores de manejo de bits.

### A.3.5 Alias para los nombres de tipo

Para hacer que un identificador sea considerado el nombre de un nuevo tipo, tenemos que emplear la palabra clave `typedef`. Por ejemplo:

```

typedef float REAL;
...
REAL x;

```

Con la definición de `REAL` como sinónimo de `float`, podemos declarar variables de tipo `REAL`. Podemos usar `typedef` para definir alias de cualquier tipo fundamental o derivado.

## A.4 Expresiones y operadores

Una expresión está formada por operadores y operandos. Los operadores establecen la relación entre los operandos y los operandos pueden ser variables, constantes u otras expresiones. Los paréntesis también pueden formar parte de una expresión y se emplean para modificar la precedencia de los operadores.

### A.4.1 Operadores aritméticos

Hay 5 operadores aritméticos:

- `+` Suma
- `-` Resta
- `*` Multiplicación
- `/` División. La división de números enteros produce un truncamiento del cociente, por ejemplo,  $\frac{3}{2} = 1$ .
- `%` Resto de la división entera.

Las expresiones aritméticas se evalúan de izquierda a derecha. Si en una expresión aritmética intervienen variables o constantes de diferentes tipos, el tipo del resultado coincidirá con el tipo mayor que aparezca en la expresión; por ejemplo, si multiplicamos una variable `float` por una variable `int`, el resultado será `float`.

La suma y la diferencia tienen una representación simplificada mediante los operadores `++` y `--`, en el caso de sumar o restar la unidad a una variable. Los siguientes ejemplos aclaran su significado:

```
int x;
...
++x; // Equivalente a x = x+1; preincremento
x++; // Equivalente a x = x+1; posincremento
--x; // Equivalente a x = x-1; predecremento
x--; // Equivalente a x = x-1; posdecremento
```

La diferencia entre la posición prefija y la posición sufija de los operadores anteriores queda patente en el ejemplo siguiente:

```
int x;
...
x = 5;
printf ("%d\n", ++x); // Incrementa x en 1, por lo que imprime 6
x = 5;
printf ("%d\n", x++); // Imprime 5 e incrementa x en 1
```



### A.4.2 Operadores de relación y lógicos

Los operadores de relación y los operadores lógicos se emplean para formar expresiones *booleanas*. Una expresión *booleana* sólo puede tomar dos valores: VERDADERO o FALSO. En C, se considera que una expresión equivale a una expresión booleana FALSA cuando al evaluarla su resultado es 0. Si el resultado de una expresión es distinto de 0 se considera que tiene un valor lógico de VERDAD.

Los operadores de relación son:

- > Mayor
- >= Mayor o igual
- < Menor
- <= Menor o igual
- == Igual, hay que tener cuidado de no confundir el operador de asignación = con el de comparación de igualdad ==.
- != Distinto

Los operadores lógicos son:

- && *And* lógica.
- || *Or* lógica.
- ! Negación lógica.

El la tabla A.1 se muestran las funciones lógicas asociadas a estos operadores.

**Tabla A.1:** Funciones asociadas a los operadores lógicos.

| Exp <sub>1</sub> | Exp <sub>2</sub> | (Exp <sub>1</sub> && Exp <sub>2</sub> ) | (Exp <sub>1</sub>    Exp <sub>2</sub> ) | !Exp <sub>1</sub> |
|------------------|------------------|-----------------------------------------|-----------------------------------------|-------------------|
| VERDAD           | VERDAD           | VERDAD                                  | VERDAD                                  | FALSO             |
| VERDAD           | FALSO            | FALSO                                   | VERDAD                                  | FALSO             |
| FALSO            | VERDAD           | FALSO                                   | VERDAD                                  | VERDAD            |
| FALSO            | FALSO            | FALSO                                   | FALSO                                   | VERDAD            |

### A.4.3 Operadores para el manejo de bits

C también implementa operadores para manipular los bits de las variables o constantes enteras. Estos operadores son:

- & *And* a nivel de bits. Por ejemplo, 1100 & 0101 → 0100
- | *Or* a nivel de bits. Por ejemplo, 1100 | 0101 → 1101

- $\wedge$  Or exclusiva —*XOR*— a nivel de bits. Por ejemplo,  $1100 \wedge 0101 \rightarrow 1001$
- $\sim$  Negación a nivel de bits o complemento a 1. Por ejemplo,  $\sim 1100 \rightarrow 0011$
- $\ll$  Desplazamiento hacia la izquierda. Por ejemplo,  $0110 \ll 1 \rightarrow 1100$
- $\gg$  Desplazamiento hacia la derecha haciendo una extensión del signo. Por ejemplo,  $0110 \gg 1 \rightarrow 0011$ ;  $1011 \gg 1 \rightarrow 1101$

Hay que tener cuidado de no confundir las operaciones a nivel de bit con las operaciones lógicas.

#### A.4.4 Expresiones abreviadas

En la tabla A.2 mostramos una relación de expresiones abreviadas y sus equivalentes:

**Tabla A.2:** Expresiones abreviadas.

| Expresión abreviada | Expresión equivalente |
|---------------------|-----------------------|
| $x += y$            | $x = x + y$           |
| $x -= y$            | $x = x - y$           |
| $x *= y$            | $x = x * y$           |
| $x /= y$            | $x = x / y$           |
| $x \&= y$           | $x = x \& y$          |
| $x  = y$            | $x = x   y$           |
| $x \hat{=} y$       | $x = x \hat{ } y$     |
| $x \ll y$           | $x = x \ll y$         |
| $x \gg y$           | $x = x \gg y$         |

## A.5 Sentencias de control de flujo

### A.5.1 Proposiciones y bloques

Una proposición es una expresión seguida de punto y coma ;. Una proposición compuesta o bloque es un conjunto de declaraciones y proposiciones agrupadas entre llaves { }. Veamos un ejemplo de bloque:

```
{ // Inicio del bloque
 int x; // Declaraciones

 x = 20; // Proposiciones
 x += 10;
 printf ("%d\n", x);
} // Final del bloque
```

### A.5.2 Selección (if-else)

Su sintaxis es:

```
if (expresión)
 proposición1;
else
 proposición2;
```

Si **expresión** es **VERDADERA** —distinta de 0— se ejecuta **proposición<sub>1</sub>**; en caso contrario, se ejecuta **proposición<sub>2</sub>**. Por ejemplo:

```
int x, y, z;
...
// Cálculo del mayor de 2 números
if (x > y)
 z = x;
else
 z = y;
```

Tanto **proposición<sub>1</sub>** como **proposición<sub>2</sub>** pueden ser bloques de código.

El operador ternario **?:** tiene el mismo significado que la proposición **if-else**.

```
(expresión) ? proposición1 : proposición2;
```

Veamos un ejemplo donde se define una macro para calcular el mayor de dos números:

```
#define max(x, y) ((x) > (y)) ? (x) : (y)
...
int z;
z = max (10+3, 11); // z = 13
```

### A.5.3 Selección múltiple (else-if)

Su sintaxis es:

```
if (expr1)
 proposición1;
else if (expr2)
 proposición2;
...
else if (exprm)
 proposiciónm;
else
 proposiciónn;
```

La tabla A.3 muestra cuál es la proposición que se ejecuta en función de los valores de las expresiones.

**Tabla A.3:** Ejecución de la proposición else-if.

| $expr_1$ | $expr_2$ | ...   | $expr_m$ | Proposición que se ejecuta |
|----------|----------|-------|----------|----------------------------|
| VERDAD   | X        | X     | X        | proposición <sub>1</sub>   |
| FALSO    | VERDAD   | X     | X        | proposición <sub>2</sub>   |
| ...      | ...      | ...   | ...      | ...                        |
| FALSO    | FALSO    | FALSO | VERDAD   | proposición <sub>m</sub>   |
| FALSO    | FALSO    | FALSO | FALSO    | proposición <sub>n</sub>   |

#### A.5.4 Selección por casos (switch)

La proposición `switch` es una decisión múltiple que prueba si una expresión coincide con alguno de entre un número de valores constantes enteros y traslada el control adecuadamente. Su sintaxis es:

```
switch (expresión) {
case exp_const1: proposiciones1;
case exp_const2: proposiciones2;
...
case exp_constn: proposicionesn;
default: proposiciones;
}
```

En el ejemplo siguiente se lee el valor de un mes y luego se calcula su número de días:

```
int mes, días_mes;
...
printf ("Mes: ");
scanf ("%d", &mes);
switch (mes) {
case 1:
case 3:
case 5:
case 7:
case 8:
case 10:
case 12:
 días_mes = 31;
 break;
case 4:
case 6:
case 9:
case 11:
 días_mes = 30;
 break;
case 2:
```

```
 días_mes = 28; // No se tienen en cuenta los años bisiestos
 break;
default:
 printf ("[%d] mes erróneo.\n", mes);
}
```

### A.5.5 Bucle for

Su sintaxis es:

```
for (inicialización; expresión; progresión)
 proposición;
```

Mientras **expresión** sea VERDADERA, se estará ejecutando **proposición**; **inicialización** es una expresión, o un conjunto de expresiones, para inicializar las variables de control que intervienen en **expresión**; y **progresión** es una expresión o conjunto de expresiones que indica cómo evolucionan las variables de control. En el ejemplo siguiente utilizamos un bucle **for** para imprimir en el flujo estándar de salida los 10 primeros números naturales:

```
int i;
...
for (i = 0; i < 10; i++)
 printf ("i = %d\n", i);
```

### A.5.6 Bucle (while)

Su sintaxis es:

```
while (expresión)
 proposición;
```

donde **proposición** se estará ejecutando mientras **expresión** sea VERDADERA. El ejemplo siguiente tiene un comportamiento similar al bucle **for** anterior:

```
int i = 0;
...
while (i < 10)
 printf ("i = %d\n", i++);
```

### A.5.7 Bucle (do-while)

Su sintaxis es:

```
do {
 proposición;
} while (expresión);
```

donde **proposición** se estará ejecutando mientras **expresión** sea VERDADERA, la primera vez siempre se ejecuta. En este ejemplo conseguimos el mismo resultado que en los dos anteriores:

```
int i = 0;
...
do {
 printf ("i = %d\n", i++)
}while (i < 10);
```

### A.5.8 break y continue

La proposición **break** produce una salida anticipada de un bucle **for**, **while** o **do-while**, y también produce la salida de la sentencia **switch**. En el ejemplo siguiente **break** provoca que el bucle sólo se ejecute 20 veces:

```
int i = 0;
...
for (i = 0; i < 100; i++) {
 printf ("i = %d\n", i);
 if (i == 20)
 break;
}
```

La proposición **continue** provoca que se inicie la siguiente iteración del bucle **for**, **while** o **do-while** que la contiene. En el ejemplo siguiente el uso de **continue** permite que se impriman sólo los números múltiplos de 5:

```
int i = 0;
...
while (i < 100) {
 if (i++%5)
 continue;
 printf ("%d\n", i-1);
}
```

## A.6 Funciones

Las funciones se utilizan para agrupar bajo un identificador una serie de proposiciones concebidas para realizar alguna tarea específica. La organización de un programa grande en funciones sencillas hará que el programa sea estructurado además de fácil de depurar y mantener. La definición de una función según el estándar *ISO/ANSI* [ISO, 1999] tiene la forma:

```
tipo nombre_función (declaración_parámetros)
{
 variables_locales;
```

```
 proposiciones;
 return (expresión);
}
```

En la primera edición de *El lenguaje de programación C* [Kernighan & Ritchie, 1978], se emplea la siguiente sintaxis para la definición de funciones:

```
tipo nombre_función (parámetros)
declaración_parámetros;
{
 variables_locales;

 proposiciones;
 return (expresión);
}
```

A continuación se muestra un ejemplo de función que calcula el factorial de un número:

```
factorial (int n)
{
 int i, fact = 1;

 for (i = 1; i <= n; i++)
 fact *= i;
 return fact;
}
```

Una función puede ser de cualquier tipo, excepto tipo función o array. Es decir, una función no puede devolver otra función, ni tampoco un array. Si no se especifica nada, el tipo de una función es `int`.

Las variables locales y los parámetros de la función se reservan en la pila de usuario del programa, por lo que al entrar en la función se reserva espacio para ellos, pero al salir de la función desaparecen. Este tipo de almacenamiento se conoce como automático, en contraposición al almacenamiento estático. Si una variable local va precedida del calificador `static`, su almacenamiento será estático y ocupará la zona de memoria reservada a las variables globales; además, esa variable existirá durante todo el tiempo de ejecución del programa.

### A.6.1 Paso de parámetros por valor y por referencia

Es importante tener en cuenta que los parámetros de una función sólo se pueden modificar a nivel local. El programa siguiente imprime el valor 10 a pesar de que aparentemente la función `f` modifica el valor de `x`, pero no es así porque `f` sólo modifica el valor local del parámetro:

```
int f(int x)
{
 x = 20;
}
```

```
main ()
{
 int x = 10;

 f (x);
 printf ("%d\n", x);
}
```

Para modificar un parámetro de forma permanente hay que trabajar con un puntero al mismo. Así, el programa del ejemplo siguiente imprime el valor esperado, 20, porque conseguimos modificar el contenido de la variable `x` definida en `main`:

```
int f(int *x) // La función recibe un puntero
{
 *x = 20; // Accedemos al contenido del puntero
}

main ()
{
 int x = 10;
 f (&x); // Le pasamos a f la dirección de x
 printf ("%d\n", x);
}
```

El paso de arrays como parámetros de funciones siempre se realiza por referencia. Las estructuras se pueden pasar por valor o por referencia. Si pasamos una estructura por valor, las modificaciones de sus campos serán locales mientras que pasándolas por referencia, las modificaciones serán permanentes.

El hecho de que los parámetros pasados por valor sólo sufran modificaciones a nivel local se debe a que el parámetro es una copia en la pila de usuario de la variable a que se refiere. Así, las modificaciones se hacen sobre la copia de la variable que hay en la pila y no sobre la propia variable.

Como ejemplo, veremos a continuación un programa donde se resumen los conceptos de matrices, punteros, estructuras y funciones. El listado siguiente corresponde a un programa para multiplicar matrices.

---

**Programa A.1:** Multiplicación de matrices (`matr.c`)

---

```
#include <stdio.h>
#include <stdlib.h>

/* Definición de la estructura de una matriz. */
struct matriz {
 int filas, columnas;
 float **coef;
};

/* Prototipos de las funciones. */
struct matriz leer_matriz ();
```



```
struct matriz multiplicar_matrices (struct matriz a, struct matriz b);

/****
 Función principal.
****/
main ()
{
 struct matriz a, b, c;

 a = leer_matriz ();
 b = leer_matriz ();

 c = multiplicar_matrices (a, b);

 printf ("Matriz resultado.\n");
 imprimir_matriz (c);
}

/****
 reservar_coeficientes: reserva dinámica de memoria para los coeficientes de
 una matriz.
**/
float **reservar_coeficientes (int filas, int columnas)
{
 float **coef;
 int i;

 if ((coef = (float **)malloc(filas * sizeof(float *))) == NULL)
 error (__FILE__, __LINE__, "Error asignando memoria");
 for (i = 0; i < filas; i++)
 if ((coef[i]=(float *)malloc(columnas*sizeof(float)))==NULL)
 error (__FILE__, __LINE__, "Error asignando memoria");
 return (coef);
}

/****
 leer_matriz: lee el número de filas y columnas junto con los coeficientes de
 una matriz. Devuelve la matriz leída.
****/
struct matriz leer_matriz ()
{
 struct matriz m;
 int i, j;

 printf ("Dimensiones de la matriz.\n");
 scanf ("%d%d", &m.filas, &m.columnas);
```

```
m.coef = reservar_coeficientes (m.filas, m.columnas);
printf ("Coeficientes de la matriz.\n");
for (i = 0; i < m.filas; i++)
 for (j = 0; j < m.columnas; j++)
 scanf ("%f", &m.coef[i][j]);
return (m);
}

/****
 imprimir_matriz: presenta una matriz por pantalla.
****/
imprimir_matriz (struct matriz m)
{
 int i, j;

 for (i = 0; i < m.filas; i++) {
 for (j = 0; j < m.columnas; j++)
 printf ("%g ", m.coef [i][j]);
 printf ("\n");
 }
}

/****
 multiplicar_matrices: recibe como parámetros dos matrices y devuelve el
 resultado de su producto.
****/
struct matriz multiplicar_matrices (struct matriz a, struct matriz b)
{
 struct matriz c;
 int i, j, k;

 if (a.columnas != b.filas)
 error (__FILE__, __LINE__,
 "Las matrices no se pueden multiplicar");
 c.filas = a.filas;
 c.columnas = b.columnas;
 c.coef = reservar_coeficientes (c.filas, c.columnas);

 for (i = 0; i < c.filas; i++)
 for (j = 0; j < c.columnas; j++) {
 c.coef[i][j] = 0;
 for (k = 0; k < a.columnas; k++)
 c.coef[i][j] += a.coef[i][k]*b.coef[k][j];
 }
 return (c);
}
```

---

```
/**
 error: presenta el último error producido e interrumpe la ejecución del programa.
***/
error (char *str, int nro, char *err)
{
 fprintf (stderr, "\n%s (%d). ERROR: %s.\n", str, nro, err);
 exit (-1);
}
```

---

## Apéndice B

# Desarrollo de aplicaciones en el entorno UNIX

---

|      |                                                          |     |
|------|----------------------------------------------------------|-----|
| B.1  | Programación modular . . . . .                           | 509 |
| B.2  | Fases de la compilación . . . . .                        | 511 |
| B.3  | Compilación de programas multimodulares . . . . .        | 512 |
| B.4  | El preprocesador de C . . . . .                          | 513 |
| B.5  | Ejemplo de programa con varios ficheros . . . . .        | 516 |
| B.6  | Gestión de bibliotecas de funciones . . . . .            | 519 |
| B.7  | Distribución de los ficheros de una aplicación . . . . . | 524 |
| B.8  | Automatización de trabajos . . . . .                     | 526 |
| B.9  | Documentación . . . . .                                  | 531 |
| B.10 | Ejercicios . . . . .                                     | 536 |

---

### B.1 Programación modular

El lenguaje de programación C permite el desarrollo de aplicaciones que se ajustan al paradigma de programación modular. Algunas de las características de este modelo son:

1. Los programas grandes se dividen en unidades con una extensión menor, los módulos.
2. La descomposición favorece el análisis descendente que va de lo general a lo particular.
3. Cada uno de los módulos se puede compilar por separado y todos juntos se enlazarán para formar el programa ejecutable final.
4. La depuración se ve facilitada al poder aislar, por lo menos a nivel de módulo, los fallos en la codificación y en la concepción de los programas.

5. La división del trabajo favorece su distribución entre los distintos programadores que intervienen en el desarrollo de una aplicación. Esto se traduce en una mejor gestión de los recursos humanos.

Una gestión inadecuada puede obstaculizar el desarrollo de un proyecto de programación grande. Aunque no existen unos criterios formalizados que permitan unificar la forma de enfocar y desarrollar una aplicación, sí podemos plantear una serie de consejos, fruto de la experiencia, que nos pueden ayudar y facilitar la codificación de un programa de envergadura.

Los puntos que se enumeran a continuación van a ser relevantes cuando estemos desarrollando una aplicación en lenguaje C y en sistema operativo UNIX:

1. Dedicar un directorio o un subárbol de directorios para almacenar los ficheros que componen la aplicación.
2. A la hora de nombrar los distintos ficheros y directorios conviene utilizar nombres significativos. Aunque no hay normas al respecto, algunos de los nombres más utilizados son los siguientes:
  - **src**, directorio donde residen los ficheros en código fuente (.c)
  - **include**, directorio donde residen los ficheros de cabecera (.h)
  - **lib**, directorio donde residen las bibliotecas (.a)
  - **bin**, directorio donde residen los programas ejecutables que se han obtenido como resultado de la compilación y enlace.
  - **doc**, directorio donde reside la documentación externa que acompaña a la aplicación, por ejemplo, manual de usuario, manual del administrador, etc.

Como puede comprobarse, estos nombres se ajustan al modelo anglosajón. Se citan aquí para que al lector le resulte familiar el formato que tienen muchas de las aplicaciones de libre distribución que se pueden encontrar en redes (por ejemplo, *Internet*) de intercambio de aplicaciones. No hay ningún inconveniente, e incluso es recomendable, en utilizar otros nombres que describan mejor la naturaleza del proyecto que se está desarrollando.

3. Dividir el programa fuente en módulos que agrupen funciones y datos que tengan alguna relación semántica. Cuanto más minucioso sea el análisis previo, mayor será el nivel de detalle que se puede alcanzar en esta división modular.

Con respecto a este punto conviene tener en cuenta los apartados siguientes:

- Las partes del programa que no sean transportables de un ordenador a otro deben estar perfectamente localizadas y aisladas del resto del programa. Las incompatibilidades pueden deberse a diferentes causas:
  - El programa accede a recursos hardware mediante procedimientos no estándar. Por ejemplo, acceso directo a la tarjeta de vídeo, reprogramación directa de dispositivos de entrada/salida, etc.

- El programa accede a recursos software mediante procedimientos no estándar. Por ejemplo, acceso a estructuras de control del sistema operativo, uso de llamadas al sistema que no son compatibles, etc.

Hay que decir que utilizando la compilación condicional se pueden subsanar, de una forma elegante, gran parte de los problemas que impiden la transportabilidad del código fuente. Para ello hay que conocer cuáles son los sistemas donde se va a compilar la aplicación y utilizar las directrices del preprocesador C para incluir unas secciones de código u otras<sup>1</sup>.

- Las partes del programa destinadas a la definición de los datos deben formar parte de un fichero de cabecera que se incluirá en los módulos que necesiten conocer esas estructuras de datos. Este planteamiento evitará la existencia de fuentes de errores tales como:
  - Definiciones redundantes de una misma estructura de datos.
  - Inconsistencias producidas al modificar uno de los ficheros donde aparece la definición de los datos sin haber modificado los restantes.

Los elementos que deben aparecer en un fichero de cabecera son:

- Definiciones de aquellos tipos de datos que son compartidos por dos módulos o más.
- Definición de los prototipos de funciones que son compartidas por dos módulos o más.

Bajo ningún concepto debe figurar en los ficheros de cabecera la definición de los datos, es decir, la reserva de memoria para variables de un tipo determinado.

## B.2 Fases de la compilación

El programa `cc` es la orden de compilación estándar para programas escritos en lenguaje C. En realidad `cc` no es el compilador, sino una interfaz entre el usuario y los programas que intervienen en el proceso de generación de un programa ejecutable. Los pasos involucrados en la compilación de un fichero como `prog.c` están reflejados en la figura B.1 y son los siguientes:

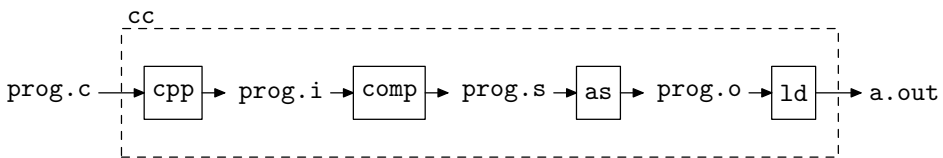


Figura B.1: Fases del proceso de compilación.

- *Preprocesamiento*. Lo realiza el programa `cpp` y como resultado se genera el fichero `prog.i`.

<sup>1</sup>Sobre el uso de las directrices del preprocesador puede consultarse [Ceballos, 2001, cap. 10 *El preprocesador de C*] y [Kernighan & Ritchie, 1988, apartado 4.11 *The C Preprocessor*].

- *Compilación y optimización.* Lo realiza el programa `comp` y como resultado se genera el fichero `prog.s`. Este fichero contiene código fuente ensamblador.
- *Generación del código objeto.* Lo realiza el programa `as` y como resultado se genera el fichero `prog.o`.
- *Enlace.* Lo realiza el programa `ld` a partir de ficheros con código objeto (`.o`) y bibliotecas (`.a`). Como resultado se genera el programa ejecutable.

Los ficheros intermedios que se generan en el proceso de compilación son almacenados temporalmente en el directorio `/tmp`. Como estos ficheros no interesan, son borrados al terminar el proceso global de compilación. Sin embargo, el compilador tiene opciones para generar específicamente alguno de los ficheros anteriores. Estas opciones son:

- `-E`, finaliza la compilación después del preproceso y el resultado es enviado al flujo estándar de salida. En realidad, el compilador no llega a ser invocado. Para generar el fichero preprocesado (`.i`) hay que utilizar esta opción junto con la opción `-o`.

```
$ cc prog.c -E -o prog.i
```

- `-S`, generar el fichero en código ensamblador (`.s`). La compilación termina después de esta fase y el ensamblador no es invocado.

```
$ cc prog.c -s → genera prog.s
```

- `-c`, generar el fichero objeto (`.o`). El enlazador no es invocado, por lo que no se genera el programa ejecutable.

```
$ cc prog.c -c → genera prog.o
```

La compilación puede continuar a partir de los puntos en que se vio interrumpida con las opciones anteriores. Es decir, a partir del fichero preprocesado (`prog.i`), ensamblado (`prog.s`) u objeto (`prog.o`) y con la orden `cc` podemos generar el programa ejecutable.

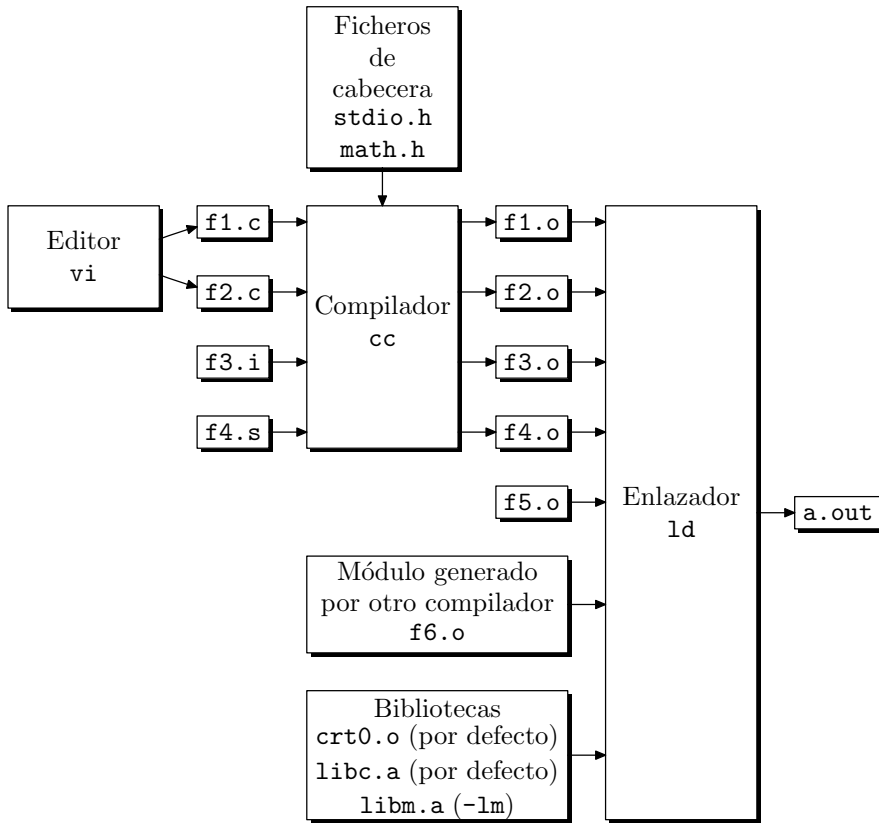
```
$ cc prog.i -o prog
```

```
$ cc prog.s -o prog
```

```
$ cc prog.o -o prog
```

## B.3 Compilación de programas multimodulares

El proceso descrito anteriormente es válido para aplicaciones que se compongan de un solo módulo. La orden `cc` también se utiliza para compilar y enlazar programas constituidos por varios módulos fuente. La procedencia de estos módulos puede ser muy diversa: módulos fuente C, módulos parcialmente compilados —preprocesados o ensamblados—, módulos objeto generados previamente por el compilador, módulos objeto generados por otros compiladores —FORTRAN, Pascal, Ada, etc.—, bibliotecas, etc. En la figura B.2 podemos ver un esquema que resume la creación de un programa ejecutable a partir de varios módulos.



**Figura B.2:** Compilación de un programa formado por varios módulos.

La orden que se debe emitir para generar la aplicación es:

```
$ cc f1.c f2.c f3.i f4.s f5.o f6.o -lm
```

La procedencia de los ficheros que aporta el sistema operativo es la siguiente:

- *Ficheros de cabecera.* Directorio `/usr/include`.
- *Bibliotecas.* Directorios `/lib` y `/usr/lib`.

## B.4 El preprocesador de C

El preprocesador de C es un programa invocado por el compilador antes de proceder a la traducción del código fuente. Las misiones más importantes del preprocesador son:

- Incluir los ficheros de cabecera. Como ya sabemos, en estos ficheros hay definiciones de tipos y prototipos de funciones.



- Sustituir las definiciones de constantes.
- Expandir las macroinstrucciones.

Como ejemplo veamos el resultado del preproceso de un programa sencillo que tenga los elementos anteriores.

```
// prep.c: ejemplo para ilustrar las principales funciones del preprocesador
#include <stdio.h> // Inclusión de un fichero de cabecera
#define CONST 10 // Definición de una constante
#define max(a,b) (((a)>(b)) ? (a) : (b)) // Macroinstrucción

main ()
{
 int i;

 i = max (20, CONST);
 printf ("El mayor de %d y %d es %d\n", 20, CONST, i);
}
```

Como ya sabemos, para generar el fichero preprocesado hay que emitir la orden siguiente:

```
$ cc prep.c -E -o prep.i
```

El resultado es el fichero `prep.i` cuyo contenido, resumido, se muestra a continuación:

```
// prep.i
// Resultado de la expansión del fichero stdio.h
1 "prep.c"
1 "/usr/include/stdio.h" 1 3

// Definición de algunos tipos
typedef long double __long_double_t;
typedef long _G_clock_t;
typedef unsigned short _G_dev_t;
...
struct _IO_FILE {
 int _flags;
 char* _IO_read_ptr;
 char* _IO_read_end;
 char* _IO_read_base;
 char* _IO_write_base;
 char* _IO_write_ptr;
 char* _IO_write_end;
 char* _IO_buf_base;
 char* _IO_buf_end;
 char *_IO_save_base;
 char *_IO_backup_base;
 char *_IO_save_end;
```

```

 struct _IO_marker *_markers;
 struct _IO_FILE *_chain;
 struct _IO_jump_t *_jumps;
 int _fileno;
 int _blksize;
 _G_off_t _offset;
 unsigned short _cur_column;
 char _unused;
 char _shortbuf[1];
};

// Definición de variables externas
extern struct _IO_FILE_plus _IO_stdin_, _IO_stdout_, _IO_stderr_;
extern FILE *stdin, *stdout, *stderr;

// Definición de algunos prototipos de funciones
extern int __underflow (_IO_FILE*);
extern int __overflow (_IO_FILE*, int);
...
extern void clearerr (FILE*);
extern int fclose (FILE*);
extern int feof (FILE*);
extern int ferror (FILE*);
extern int fflush (FILE*);
extern int fgetc (FILE *);
...
extern int getchar (void);
...
extern int printf (__const char* format, ...);
...
extern int putchar (int);
...
// Parte del programa escrita por el usuario
6 "prep.c" 2

main ()
{
 int i;

 // Expansión de la macro max y sustitución de la constante CONST
 i = (((20)>(10)) ? (20) : (10)) ;
 printf ("El mayor de %d y %d es %d\n", 20, 10 , i);
}

```

Es este programa, y no el original escrito por el usuario, el que realmente ve el compilador. Nos podríamos plantear entonces la pregunta siguiente:

«si el compilador no trabaja con el texto del programa original sino con un texto modificado, cuando genera un listado de errores, ¿cómo establece la correspondencia entre la línea del fichero preprocesado donde ha aparecido el error y la línea del fichero original que ha dado lugar al error?»

Tal y como se ha visto antes, el fichero preprocesado puede compilarse con la orden `cc`:

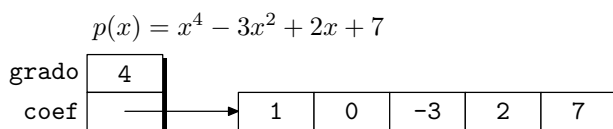
```
$ cc prep.i
```

## B.5 Ejemplo de programa con varios ficheros

Como aplicación de los elementos teóricos expuestos en los apartados anteriores vamos a escribir una aplicación para la manipulación simbólica de polinomios. Los polinomios se representarán mediante la estructura `POLINOMIO` que consta de dos campos:

- **grado**, es un número entero que indica el grado del polinomio (total de coeficientes menos uno).
- **coef**, es un puntero a los coeficientes. Cada coeficiente será un número en coma flotante de simple precisión. Los coeficientes son manipulados a través de un puntero con objeto de no poner límite al grado de los polinomios. Naturalmente, esto exige que la memoria donde se almacenan los coeficientes sea asignada dinámicamente en tiempo de ejecución.

En la figura B.3 podemos ver una representación gráfica de esta estructura.



**Figura B.3:** Estructura de un polinomio.

Las funciones de lectura/escritura de polinomios son:

- `POLINOMIO leer_polinomio (int grado);`

Esta función recibe el grado del polinomio que debe leer y, tras reservar memoria para los coeficientes, los lee del flujo estándar de entrada y devuelve el polinomio leído.

- `void escribir_polinomio (POLINOMIO p);`

Esta función escribe en el flujo estándar de salida el polinomio que recibe a través del parámetro `p`.

Las funciones para realizar operaciones algebraicas con polinomios son:

- `POLINOMIO sumar_polinomios (POLINOMIO p, POLINOMIO q);`

Los polinomios `p` y `q` son los sumandos. La función devuelve el resultado `p+q`. Se tendrá en cuenta que `p` y `q` pueden ser de grados distintos.

- `POLINOMIO multiplicar_polinomios (POLINOMIO p, POLINOMIO q);`

Los polinomios `p` y `q` son los multiplicandos. La función devuelve el resultado `p×q`.

- `POLINOMIO simplificar_polinomios (POLINOMIO p);`

El objetivo de esta función es eliminar los coeficientes altos que sean nulos para así obtener una representación más compacta del polinomio. Esta función se debe aplicar después de cada suma, ya que en esta operación puede haber términos que se anulen. Por ejemplo,  $(x^5 + 5x^2 - 4) + (-x^5 - 5x^2 + x - 2)$  produce el resultado  $(0x^5 + 0x^4 + 0x^3 + 0x^2 + x - 6)$  y al simplificar el polinomio anterior se obtiene  $x - 6$ .

Como hay funciones que devuelven un resultado del tipo `POLINOMIO`, es necesario escribir una función para hacer la reserva dinámica de sus coeficientes. Esta función será `crear_polinomio` y su declaración es:

```
POLINOMIO crear_polinomio (int grado);
```

Los módulos que componen esta aplicación son:

- `pol.h`, fichero de cabecera con la definición de los tipos y los prototipos de las funciones.
- `pol_es.c`, fichero donde está la implementación de las funciones de entrada/salida de datos.
- `pol_arit.c`, fichero donde está la implementación de las funciones de cálculo.
- `pol.c`, fichero donde reside la función principal `main`.

A continuación podemos ver parte de estos módulos:

```
// pol.h
// Fichero de cabecera con la definición de los datos y los prototipos de funciones
// de la aplicación de manipulación simbólica de polinomios.
#ifdef !_POL_H_
#define _POL_H_

// Definición de los datos
typedef struct {
 int grado;
 float *coef;
} POLINOMIO;

// Prototipo de las funciones
POLINOMIO crear_polinomio (int grado);
POLINOMIO leer_polinomio (int grado);
```

```
void escribir_polinomio (POLINOMIO p);
POLINOMIO multiplicar_polinomios (POLINOMIO p, POLINOMIO q);
POLINOMIO sumar_polinomios (POLINOMIO p, POLINOMIO q);
POLINOMIO simplificar_polinomio (POLINOMIO p);

#endif

// pol_es.c (entrada/salida)
// Módulo del programa de manipulación simbólica de polinomios donde se
// implementan las funciones de entrada/salida.
#include <stdio.h>
#include <stdlib.h>
#include "pol.h"

// Función de lectura
POLINOMIO leer_polinomio (int grado)
{...}

// Función de escritura
void escribir_polinomio (POLINOMIO p)
{...}

// Función que reserva memoria para los coeficientes
POLINOMIO crear_polinomio (int grado)
{...}

// pol_arit.c (aritmética)
// Módulo que contiene las funciones de manipulación algebraica de polinomios.
#include "pol.h"

// Función para sumar polinomios
POLINOMIO sumar_polinomios (POLINOMIO p, POLINOMIO q)
{...}

// Función para multiplicar polinomios
POLINOMIO multiplicar_polinomios (POLINOMIO p, POLINOMIO q)
{...}

// Función para simplificar polinomios
POLINOMIO simplificar_polinomio (POLINOMIO p)
{...}

// pol.c
// Módulo donde reside la función principal del programa.
#include <stdio.h>
#include "pol.h"
```

```
// Función principal del programa
main ()
{
 POLINOMIO p, q;
 int grado;

 // Lectura de los polinomios
 printf ("Grado del primer polinomio: ");
 scanf ("%d", &grado);
 printf ("Coeficientes del primer polinomio: ");
 p = leer_polinomio (grado);
 printf ("Grado del segundo polinomio: ");
 scanf ("%d", &grado);
 printf ("Coeficientes del segundo polinomio: ");
 q = leer_polinomio (grado);

 // Operaciones con los polinomios
 // Suma
 printf ("Resultado de la suma.\n");
 escribir_polinomio (
 simplificar_polinomio (sumar_polinomios (p, q)));
 printf ("\n");
 // Multiplicación
 printf ("Resultado de la multiplicación.\n");
 escribir_polinomio (
 simplificar_polinomio (multiplicar_polinomios (p,q)));
 printf ("\n");
}
```

La orden para compilar los módulos anteriores y generar el programa ejecutable es:

```
$ cc pol.c pol_es.c pol_arit.c -o pol
```

## B.6 Gestión de bibliotecas de funciones

Una biblioteca es un fichero compuesto por una colección de otros ficheros llamados miembros de la biblioteca. La estructura de una biblioteca posibilita la extracción de sus miembros.

Cuando se añade un fichero a una biblioteca, los datos de éste y su información administrativa —permisos, fechas, propietario, grupo, etc.— son introducidos en la biblioteca.

### B.6.1 Programas `ar` y `ranlib`

El programa para gestionar bibliotecas es `ar` y sus funciones básicas son crear, modificar y extraer miembros. `ar` puede mantener bibliotecas cuyos miembros tengan nombres de cualquier longitud; sin embargo, dependiendo de la configuración de `ar`, el sistema puede imponer una longitud —15 o 16 caracteres— a los nombres de los miembros.

Los ficheros que forman parte de una biblioteca pueden ser de cualquier naturaleza. Desde el punto de vista de la gestión de proyectos de programación, son los ficheros objeto los que nos va a interesar incluir en las bibliotecas.

Como ejemplo tenemos bibliotecas estáticas que se utilizan en el desarrollo de aplicaciones C —`libc.a`, `libm.a`, etc.— Estos ficheros han sido creados con `ar` a partir de los módulos objeto donde están implementadas las funciones de estas bibliotecas.

La sintaxis de `ar` es la siguiente:

```
$ ar [-]opciones [miembro] biblioteca [ficheros]
```

Entre las **opciones** podemos distinguir aquellas que son obligatorias y los modificadores. Cuando se emite una orden `ar` es necesario que haya una opción obligatoria y sólo una.

Opciones obligatorias:

- **d**, *borrar miembros* de la biblioteca. Los miembros que se van a borrar se indican mediante nombres de fichero.
- **m**, *mover un miembro* en la biblioteca. El orden de los miembros influye en la forma de enlazar los programas. Si no se especifica ningún modificador, el miembro se coloca al final de la biblioteca. Con los modificadores **a**, **b** o **i** se puede indicar la nueva posición.
- **p**, *imprimir un miembro* de la biblioteca en el flujo estándar de salida. Si no se especifica un nombre de fichero, se imprimirán todos los miembros.
- **q**, *añadido rápido*. Añadir ficheros al final de una biblioteca. Como el objetivo de esta operación es la rapidez, el índice de la tabla de símbolos no es actualizado.
- **r**, *reemplazar ficheros* en la biblioteca. Por defecto, el nuevo fichero se añade al final. El punto de inserción se puede controlar con los modificadores **a**, **b** o **i**.
- **t**, *mostrar* una tabla con el *contenido* de la biblioteca.
- **x**, *extraer miembros* de la biblioteca. Si no se especifica ningún fichero, se extraen todos los que haya.

Modificadores:

- **a**, *añadir* nuevos ficheros *detrás* de un miembro existente en la biblioteca. El nombre del miembro se indica en el argumento `[miembro]` que figura antes del nombre de la biblioteca.
- **b**, *añadir* nuevos ficheros *delante* de un miembro existente en la biblioteca. El nombre del miembro se indica en el argumento `[miembro]` que figura antes del nombre de la biblioteca.
- **c**, *crear la biblioteca*. La biblioteca indicada en el argumento `[biblioteca]` es siempre creada en el caso de que no exista. Si no se usa el modificador **c** aparecerá un aviso siempre que se vaya a crear la biblioteca.
- **i**, tiene el mismo significado que **b**.

- *o*, *preservar las fechas originales* de los miembros cuando sean extraídos de la biblioteca. Si no se usa este modificador, las fechas se actualizan con las del instante de extracción.
- *s*, *crear un índice* en la biblioteca o actualizar el existente. Esta opción se puede utilizar sola o como modificador.

Una vez creado el índice, éste es actualizado cada vez que se realiza una operación en la biblioteca, excepto cuando utilizamos la opción *q*.

Una biblioteca con un índice acelera las operaciones de enlazado y permite que las rutinas de la misma se invoquen unas a otras sin que influya la posición que ocupan en la biblioteca.

También se puede utilizar el programa **ranlib** para crear el índice de una biblioteca. Este programa es equivalente a la orden **ar** con la opción **-s**.

- *u*, normalmente, la orden **ar -r...** reemplaza en la biblioteca todos los ficheros indicados. Usando la opción *u* se reemplazan sólo aquellos ficheros que han sido modificados con fecha posterior a los que hay en la biblioteca.
- *v*, con esta opción se muestra información adicional al realizar una operación con la biblioteca.

Como ejemplo, aplicaremos los conceptos explicados a la gestión de la aplicación **polinomios**. Como recordamos, esta aplicación se compone de tres módulos con código: **pol\_es.c**, **pol\_arit.c** y **pol.c**. El tercer módulo contiene la función **main** desde la que son invocadas las funciones de manipulación que hay en los dos primeros módulos. Tanto **pol\_es.c** como **pol\_arit.c** contienen funciones que pueden ser utilizadas para crear otros programas que manejen polinomios como los aquí descritos. Por lo tanto, puede ser útil que estos dos módulos formen parte de una biblioteca que esté disponible para desarrollos futuros. Las órdenes para crear esta biblioteca son:

- Compilación de los módulos **pol\_es.c** y **pol\_arit.c** para crear los módulos objeto,
 

```
$ cc -c pol_es.c
$ cc -c pol_arit.c
```
- Añadir el módulo **pol\_es.o** a la biblioteca **pol.a**,
 

```
$ ar r pol.a pol_es.o
```

Creating archive file '/home/Francisco/make/pol/pol.a'

Como el fichero **pol.a** no existía, aparece un mensaje indicando que se está creando.

- Añadir el módulo **pol\_arit.o** a la biblioteca **pol.a**,
 

```
$ ar r pol.a pol_arit.o
```

Los dos módulos se podrían haber añadido simultáneamente,

```
$ ar r pol.a pol_es.o pol_arit.o
```



- Visualizar el contenido de la biblioteca,

```
$ ar t pol.a

pol_es.o
pol_arit.o
```

- Visualizar el contenido de la biblioteca con más detalle,

```
$ ar tv pol.a

rw-r--r-- 406/ 6 875 Mar 15 10:07 1995 pol_es.o
rw-r--r-- 406/ 6 1037 Mar 15 10:08 1995 pol_arit.o
```

- Generar un índice de símbolos,

```
$ ar s pol.a

Otra posibilidad es,

$ ranlib pol.a
```

- Visualizar de nuevo el contenido de la biblioteca,

```
$ ar tv pol.a

rw-rw-rw- 0/ 0 174 Mar 15 10:31 1995 __.SYMDEF
rw-r--r-- 406/ 6 875 Mar 15 10:07 1995 pol_es.o
rw-r--r-- 406/ 6 1037 Mar 15 10:08 1995 pol_arit.o
```

Como vemos, ha aparecido el miembro `__.SYMDEF`. Este miembro contiene el índice.

- Borrar los ficheros objeto. Ya no los necesitamos puesto que tenemos una copia de ellos en la biblioteca,

```
$ rm pol_es.o

$ rm pol_arit.o
```

- En cualquier momento podemos extraer un módulo de la biblioteca. Por ejemplo, `pol_es.o`,

```
$ ar xv pol.a pol_es.o

x - pol_es.o
```

- Podemos utilizar la biblioteca para compilar programas que emplean funciones implementadas en alguno de los miembros de la biblioteca,

```
$ cc pol.c pol.a -o pol
```

## B.6.2 Programa nm

El contenido de una biblioteca o de un módulo objeto se puede inspeccionar con el programa `nm`. Su sintaxis es la siguiente:

```
$ nm [opciones] [ficheros]
```

Si no se especifica ningún fichero, se considera `a.out` por defecto. Las principales opciones de `nm` son:

- `-g`, mostrar sólo los símbolos externos.
- `-p`, no ordenar la salida alfabéticamente.
- `-n`, ordenar los símbolos según la dirección que ocupen y no según su nombre.
- `-s`, mostrar también el índice, en el caso de que el módulo sea una biblioteca.
- `-o`, preceder cada símbolo con el nombre del fichero al que pertenece.
- `-r`, invertir el sentido de la ordenación.
- `-u`, mostrar sólo los símbolos externos referenciados en cada módulo.

Como ejemplo, a continuación se muestra una sesión de trabajo con `nm` y los módulos de la aplicación `polinomios`.

- Visualizar el contenido de la biblioteca `pol.a`,

```
$ nm pol.a
```

```
pol.a:
```

```
pol_es.o:
```

```
00000000 t __gnu_calloc
00000000 t __gnu_compiled_c
00000050 t __gnu_malloc
 U _calloc
00000170 T _crear_polinomio
00000100 T _escribir_polinomio
 U _exit
00000090 T _leer_polinomio
 U _malloc
 U _perror
 U _printf
 U _scanf
00000000 t gcc2_compiled.
```

```
pol_arit.o:
```

```
00000000 t __gnu_compiled_c
 U _crear_polinomio
```

```

 U _free
00000110 T _multiplicar_polinomios
00000210 T _simplificar_polinomio
00000000 T _sumar_polinomios
00000000 t gcc2_compiled.

```

- Visualizar los símbolos externos de cada uno de los módulos, incluido el índice, de la biblioteca `pol.a`,

```

$ nm -s pol.a

pol.a:

__SYMDEF:
_leer_polinomio in pol_es.o
_crear_polinomio in pol_es.o
_escribir_polinomio in pol_es.o
_sumar_polinomios in pol_arit.o
_multiplicar_polinomios in pol_arit.o
_simplificar_polinomio in pol_arit.o

pol_es.o:
_calloc
_exit
_malloc
_perror
_printf
_scanf

pol_arit.o:
_crear_polinomio
_free

```

## B.7 Distribución de los ficheros de una aplicación

Hasta ahora hemos considerado que todos los ficheros generados para crear una aplicación se encuentran en un mismo directorio. En el caso de aplicaciones grandes, en las que intervienen varios programadores, conviene distribuir los módulos de una aplicación en varios directorios, con el fin de realizar una gestión más eficiente. En el caso de aplicaciones pequeñas como las que estamos estudiando en los ejemplos, esta división puede resultar algo artificiosa e incluso puede complicar innecesariamente el desarrollo de la misma. Hay que recordar que estos ejemplos tienen una finalidad didáctica y que no pretenden constituirse en ejemplos de aplicaciones reales.

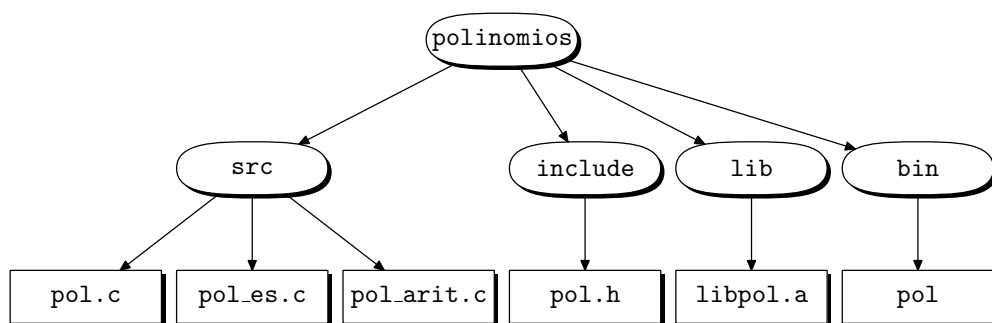
En el primer apartado del capítulo —*Programación modular*— se han mencionado los directorios más importantes que se pueden tener en cuenta a la hora de organizar

una aplicación. Esta distribución es bastante subjetiva y depende tanto del estilo del programador como de las herramientas de desarrollo que está empleando. Algunas herramientas de programación asistida —*CASE*— imponen una estructura de directorios que está condicionada por la metodología de desarrollo a la que se adaptan.

Como ejemplo, seguiremos trabajando con la aplicación **polinomios** y vamos a considerar que sus módulos pertenecen a la jerarquía de directorios siguiente:

- **src**, directorio con los programas en código fuente.
- **include**, directorio para los ficheros de cabecera.
- **lib**, directorio para las bibliotecas.
- **bin**, directorio para los programas ejecutables.

En la figura B.4 podemos ver una representación gráfica de esta estructura. Teniendo en cuenta esta estructura, las órdenes necesarias para compilar la aplicación son las siguientes:



**Figura B.4:** Distribución de los módulos de la aplicación **polinomios**.

1. Compilar los módulos **pol\_es.c** y **pol\_arit.c**,

```
$ cd $HOME/polinomios/src
```

```
$ cc -c pol_es.c -I../include
```

```
$ cc -c pol_arit.c -I../include
```

La opción **-I** se utiliza para indicar un nuevo camino de búsqueda de los ficheros de cabecera. Recordemos que los directorios de búsqueda por defecto son:

- **/usr/include** para los ficheros incluidos con la directriz **#include <fichero.h>**
- *directorio de trabajo actual*  
y  
**/usr/include** para los ficheros incluidos con la directriz **#include "fichero.h"**

El fichero `pol.h` no reside en ninguno de estos directorios, por lo que hay que indicarle al compilador en qué directorio se encuentra (`-I../include`). La ruta que acompaña al parámetro `-I` puede ser absoluta o relativa.

2. Generar la biblioteca `libpol.a`,

```
$ ar r ../lib/libpol.a pol_es.o pol_arit.o
```

3. Generar el programa ejecutable `pol`,

```
$ cc pol.c -I../include -L../lib -lpol -o ../bin/pol
```

En esta orden nos encontramos con los elementos siguientes:

- `-I../include`, indica una ruta en donde buscar los ficheros de cabecera.
- `-L../lib`, indica una ruta en donde buscar las bibliotecas. Las rutas por defecto son `/lib` y `/usr/lib`. Así, cuando en la compilación de un programa ponemos el parámetro `-lm`, la biblioteca matemática se carga del fichero `/usr/lib/libm.a`. Como la biblioteca `libpol.a` se encuentra en el directorio `polinomios/lib` es necesario indicárselo al compilador —no olvidemos que nuestro directorio de trabajo actual es `$HOME/polinomios`—.
- `-lpol`, indica una nueva biblioteca en donde buscar las referencias no resueltas. La biblioteca por defecto es `libc.a`. Con la opción `-lpol` se buscará también en la biblioteca `libpol.a`.  
Como podemos observar, al programa `cc` no se le indica el nombre completo de la biblioteca. Para poder usar la opción `-l` es necesario que el nombre de la biblioteca se ajuste a una plantilla determinada. La forma genérica de esta plantilla es `libX.a`, donde `X` representa cualquier secuencia de letras y dígitos. Así, la forma de indicarle al enlazador que vamos a usar esta biblioteca es `-lX`.
- `-o ../bin/pol`, el programa ejecutable `pol` se deposita en un directorio distinto del directorio de trabajo actual.

## B.8 Automatización de trabajos

Como hemos podido comprobar en el desarrollo de la aplicación `polinomios`, a medida que estructuramos los módulos de una aplicación y el número de éstos se incrementa, se va introduciendo complejidad con respecto a las órdenes que se deben emitir para generar el programa ejecutable. Parece que estuviésemos consiguiendo un efecto contrario al perseguido. Es decir, en lugar de incrementar la elegancia y sencillez de la gestión, parece que estemos introduciendo sofisticaciones que no aportan ventaja alguna.

El objetivo de este apartado es mostrar cómo podemos automatizar la parte relacionada con la gestión de órdenes para compilar las aplicaciones de una forma más cómoda y segura.

La primera solución que podemos aportar consiste en escribir un fichero de procesamiento por lotes en el que se encuentren todas las órdenes necesarias para compilar una aplicación. El programa siguiente es un ejemplo válido para compilar la aplicación `polinomios`.

```
#####
#
Fichero: $HOME/polinomios/src/compilar
Baterías de órdenes para compilar la aplicación polinomios
#
#####
cc -c pol_es.c -I../include -o pol_es.c
cc -c pol_arit.c -I../include -o pol_arit.c
ar r ../lib/libpol.a pol_es.o pol_arit.c
cc pol.c -I../include -L../lib -lpol -o ../bin/pol
```

Así, para compilar la aplicación sólo hay que situarse en el directorio `polinomios/src` y ejecutar el programa `compilar`.

Aunque esta solución es válida y cómoda, sin embargo presenta algunos inconvenientes:

1. Cada vez que se ejecuta la batería se recompilan todos los módulos con código fuente.

Supongamos que la aplicación se compone de 50 módulos y que por necesidades de depuración se han modificado 5 de ellos. Al ejecutar la orden `compilar`, se van a recompilar los 50 módulos cuando sólo es necesario recompilar los 5 que han sido modificados. Esta forma de trabajo acarrea pérdidas de tiempo innecesarias.

2. En el programa `compilar` no quedan reflejadas todas las relaciones que existen entre los distintos ficheros de la aplicación; en concreto, no están reflejadas las relaciones entre ficheros de cabecera y ficheros fuente.

Veámoslos con un ejemplo. En la aplicación `polinomios` utilizamos el fichero de cabecera `pol.h`. En el caso de que modifiquemos este fichero, es necesario recompilar aquellos módulos fuente donde se incluye `pol.h` con la directriz `#include`. Pero ¿cuáles son esos módulos? En el ejemplo `polinomios` la respuesta es muy sencilla: *todos los módulos fuente utilizan el único fichero de cabecera que hay*; pero puede haber aplicaciones con varios módulos cabecera y donde no todos los ficheros fuente incluyan a todos los ficheros de cabecera. En este caso, sólo es necesario recompilar aquellos módulos fuente que incluyan a los ficheros de cabecera modificados.

El programa `make` fue diseñado para solucionar estas y otras deficiencias y poder automatizar la compilación de aplicaciones. Aunque aquí veremos la forma de recompilar aplicaciones escritas en lenguaje C, el programa `make` se puede utilizar para gestionar proyectos escritos en otros lenguajes y para gestionar trabajos que no estén relacionados con el desarrollo de programas. La sintaxis de esta orden es:

```
$ make [-f makefile] [opciones] [objetivos]
```

El programa `make` lee del fichero `makefile` la especificación de cómo están relacionados entre sí los componentes de una aplicación y cómo procesarlos para crear una versión actualizada del programa.

El fichero `makefile` se especifica con la opción `-f makefile`. Si esta opción no está presente, los ficheros que se consideran por defecto son: `makefile` y `Makefile`. El nombre `Makefile` es el utilizado con más frecuencia por los programadores.

El argumento **objetivos** suele ser el nombre del programa o programas que se deben generar como resultado de la actualización realizada por **make**. Conocida la relación entre módulos, **make** revisa la fecha de la última modificación de los distintos ficheros y determina la cantidad mínima de recompilación necesaria para generar una nueva versión del programa.

Una vez determinadas las órdenes que se deben ejecutar para regenerar la aplicación, el programa **make** se encarga de invocarlas.

Como ejemplo, veamos la forma que tiene una primera versión del fichero **Makefile** necesario para compilar la aplicación **polinomios**.

```
#####
#
Fichero: Makefile
Primera versión del fichero para compilar la aplicación polinomios
Compilar escribiendo la orden:
$ make
#
#####

Programas estándar que se van a utilizar y opciones de los mismos
CFLAGS = -I../include -L../lib

Programa ejecutable y ficheros que forman parte del mismo
../bin/pol: pol.o ../lib/libpol.a
 cc pol.o -I../include -L../lib -lpol -o ../bin/pol

Biblioteca de usuario y ficheros que forman parte de la biblioteca
../lib/libpol.a: pol_es.o pol_arit.o
 ar rs ../lib/libpol.a pol_es.o pol_arit.o

Relación entre los ficheros de cabecera y los módulos con código fuente
pol.a pol_es.o pol_arit.o: ../include/pol.h
```

A continuación podemos ver la salida que se produce cuando se emite la orden de compilar la aplicación,

```
$ make

cc -I../include -L../lib -c pol.c -o pol.o
cc -I../include -L../lib -c pol_es.c -o pol_es.o
cc -I../include -L../lib -c pol_arit.c -o pol_arit.o
ar rs ../lib/libpol.a pol_es.o pol_arit.o
Creating archive file '/home/Francisco/make/pol/lib/libpol.a'
cc pol.o -I../include -L../lib -lpol -o ../bin/pol
```

Si una vez que la aplicación ha sido compilada, intentamos recompilarla, podemos ver que no se genera ninguna orden, ya que los ficheros fuente no han sufrido ninguna modificación,

```
$ make
```

```
make: './bin/pol' is up to date.
```

Si borramos el fichero `pol_arit.o`, al recompilar la aplicación sólo será necesario generar el fichero `pol_arit.o`, actualizar la biblioteca `pol.a` y regenerar el programa `pol`,

```
$ rm pol_arit.o
```

```
$ make
```

```
cc -I../include -L../lib -c pol_arit.c -o pol_arit.o
ar rs ../lib/libpol.a pol_es.o pol_arit.o
cc pol.o -I../include -L../lib -lpol -o ../bin/pol
```

Si borramos el fichero `pol.o`, al recompilar la aplicación sólo será necesario generar el fichero `pol.o` y regenerar el programa `pol`,

```
$ rm pol.o
```

```
$ make
```

```
cc -I../include -L../lib -c pol.c -o pol.o
cc pol.o -I../include -L../lib -lpol -o ../bin/pol
```

Para simular que un fichero fuente ha sufrido una modificación, vamos a utilizar el programa `touch`. Este programa cambia la fecha de la última modificación de un fichero para que tome el valor de la fecha y hora actuales.

```
$ touch ../include/pol.h
```

```
$ make
```

```
cc -I../include -L../lib -c pol_es.c -o pol_es.o
cc -I../include -L../lib -c pol_arit.c -o pol_arit.o
ar rs ../lib/libpol.a pol_es.o pol_arit.o
cc pol.o -I../include -L../lib -lpol -o ../bin/pol
```

Si realizamos la misma operación con la biblioteca `libpol.a`, podemos comprobar que la única orden que se genera es la de enlazar el programa `pol`,

```
$ touch ../lib/libpol.a
```

```
$ make
```

```
cc pol.o -I../include -L../lib -lpol -o ../bin/pol
```

Por último, hay que decir que el fichero `Makefile` descrito tiene dos **objetivos**. Los **objetivos** se indican mediante conjuntos de líneas donde figuran las dependencias entre ficheros y las órdenes —encabezadas con un tabulador— que se deben ejecutar para generar el fichero del **objetivo**. En el ejemplo, uno de los **objetivos** es la generación de la biblioteca y se expresa con las líneas siguientes:



```
../lib/libpol.a: pol_es.o pol_arit.o
ar rs ../lib/libpol.a pol_es.o pol_arit.o
```

Si queremos que el programa `make` se ejecute teniendo en cuenta sólo uno de los objetivos, hay que indicárselo a través de la línea de órdenes. En el ejemplo siguiente borramos la biblioteca `pol.a` y ejecutamos `make` con el objetivo de generar esta biblioteca. Aunque el programa `pol` también depende de esta biblioteca, no va a ser enlazado de nuevo,

```
$ rm ../lib/libpol.a

$ make ../lib/libpol.a

ar rs ../lib/libpol.a pol_es.o pol_arit.o
Creating archive file '/home/Francisco/make/pol/lib/libpol.a'
```

El programa `Makefile` tiene una sintaxis mucho más rica y compleja que la explicada en el ejemplo anterior<sup>2</sup>. Sin embargo, con los elementos vistos en el ejemplo se pueden expresar las necesidades de compilación de la mayor parte de las aplicaciones de tamaño medio.

En el ejemplo siguiente se ilustra otra forma muy común de expresar las dependencias entre los módulos de una aplicación. Hacemos uso para ello de la capacidad de manejo de símbolos del programa `make`.

```
#####
#
Fichero: Makefile
Segunda versión del programa para compilar la aplicación polinomios
Compilar escribiendo la orden:
$ make
#
#####

Directorios de la aplicación
INCLUDEDIR = ../include
LIBDIR = ../lib
BINDIR = ../bin

Programas estándar que se utilizan
Archivador
AR = ar
AFLAGS = rs
Compilador
CC = cc
CFLAGS = -I$(INCLUDEDIR) -L$(LIBDIR)
```

---

<sup>2</sup>Para ver una explicación más detallada del uso de `make` se puede consultar [Kernighan & Pike, 1987, Cap. 8 *Desarrollo de programas*].

```
Biblioteca de usuario y ficheros que forman parte de la biblioteca
BIBLIOTECA = $(LIBDIR)/libpol.a
OBJETOS_BIBLIOTECA = pol_es.o pol_arit.o

Programa ejecutable y ficheros que forman parte del mismo
PROGRAMA = $(BINDIR)/pol
OBJETOS_PROGRAMA = pol.o

Generación del programa
$(PROGRAMA): $(OBJETOS_PROGRAMA) $(BIBLIOTECA)
 $(CC) $(OBJETOS_PROGRAMA) $(CFLAGS) -lpol -o $(PROGRAMA)

Creación de la biblioteca
$(BIBLIOTECA): $(OBJETOS_BIBLIOTECA)
 $(AR) $(AFLAGS) $(BIBLIOTECA) $(OBJETOS_BIBLIOTECA)

Relación entre los ficheros de cabecera y los módulos con código fuente
$(OBJETOS_PROGRAMA) $(OBJETOS_BIBLIOTECA): $(INCLUDEDIR)/pol.h
```

## B.9 Documentación

La creación de documentación es una de las fases de gestión de proyectos que no debe dejarse para el final aunque suela hacerse así en muchas ocasiones. El tiempo que se invierte en escribir una buena documentación se amortiza con creces a medida que el proyecto envejece. El motivo es muy sencillo, *un programa no debe ser escrito pensando únicamente en que será ejecutado por un ordenador, también se facilitará que otras personas entiendan el programa y así ayuden a mantenerlo y mejorarlo*. A la hora de hablar de documentación podemos distinguir entre dos niveles:

1. Documentación interna que acompaña al código fuente. Está concebida para que los programadores interpreten mejor el código fuente y la arquitectura de la aplicación. Esto capacita a las distintas personas que intervienen en el proyecto para introducir mejoras en el mismo, crear nuevas versiones, detectar errores, etc.
2. Documentación externa. Está pensada para que el usuario sepa cómo manejar la aplicación. Los usuarios no son programadores y no conocen la estructura de una aplicación. Lo único que deben conocer es la interfaz que tiene esa aplicación para poder manejarla.

Las metodologías de gestión de proyectos se encuentran en fase de formalización. Por el momento no existe una teoría que diga cuál es la forma óptima de realizar proyectos de programación. Bajo el nombre de herramientas *CASE* se engloban un conjunto de aplicaciones que pretenden semiautomatizar el proceso de gestión de proyectos ajustándose a alguna de las metodologías que se han descrito hoy en día. Cada una de estas metodologías y sus herramientas *CASE* asociadas dedican una parte de su normativa a describir la forma de gestionar la documentación asociada a un proyecto.

En este apartado no vamos a describir la forma de gestionar la documentación de un proyecto. Esto correspondería más a un curso de ingeniería del software. Sólo queremos dejar constancia de su importancia.

Como ejemplo práctico y casi anecdótico, vamos a presentar la forma de editar páginas del manual de UNIX. Evidentemente, este ejemplo está sacado fuera de todo contexto donde se describa con rigurosidad la forma de crear la documentación de un proyecto.

Las páginas del manual de UNIX son ficheros de texto que se componen de secuencias de control y cadenas con el texto de la página. Las secuencias de control son interpretadas por el procesador de texto **troff** [Ossanna & Kernighan, 2000] —**groff** es el procesador equivalente a **troff** escrito para el proyecto *GNU*—.

El manual de UNIX está dividido en secciones estándar que se almacenan en directorios estándar. Por ejemplo, en la sección 3 se describen, entre otras, las funciones de las bibliotecas **libc.a**, **libm.a** y **libX11.a**. Las páginas correspondientes a la sección 3 están en el directorio **/usr/man/man3**.

En realidad, el programa **man** es una interfaz con el programa **groff** y cuando ejecutamos una orden como **man fopen** la orden que se ejecuta es:

```
$ groff -man /usr/man/man1/fopen.1 | more
```

El parámetro **-man** le indica al programa **groff** que utilice el conjunto de macroinstrucciones que se han definido para procesar las páginas del manual. Estas macroinstrucciones se encuentran en el fichero **/usr/lib/groff/tman/tmac.an**. Esto exige que las páginas hayan sido escritas ajustándose a este formato.

A continuación podemos ver una plantilla donde se describe la forma que tiene un fichero del manual:

```
.TH Nombre de la orden o función que se describe(Nº sección)
.SH NAME
Orden/función \- breve descripción de la orden/función
.SH SYNOPSIS
.B Nombre de la orden/función
Opciones (caso de orden)/interfaz (caso de función)
.SH DESCRIPTION
Descripción detallada de las órdenes y sus opciones.
En el caso de una función se deben describir los parámetros de su interfaz.
Los párrafos se introducen con .PP
.PP
Éste es un párrafo nuevo.
Las nuevas líneas se introducen con .br
.br
Ésta es una línea nueva.
.SH FILES
Los ficheros que necesita la orden/función. Por ejemplo, la orden passwd(1)
utiliza el fichero /etc/passwd
.SH SEE ALSO
Referencia a documentos afines, incluyendo otras páginas del manual.
.SH BUGS
Detalles sorprendentes (no siempre tienen que ser problemas).
```

**.SH AUTHOR**

*Autor del programa. Se suele incluir su nombre y dirección de contacto (e-mail normalmente).*

Como vemos, la página del manual se compone de un conjunto de epígrafes y cada uno de ellos va encabezado por un título (NAME, SYNOPSIS, DESCRIPTION, etc.). Si algún epígrafe está vacío, su título se omite. La línea .TH y los epígrafes NAME, SYNOPSIS y DESCRIPTION son obligatorios.

Como ejemplo vamos a crear una página del manual para la función `leer_polinomio` de la aplicación `polinomios` que se ha venido describiendo en apartados anteriores. El nombre del fichero que contiene la página es `leer_polinomio.3p`, y una vez presentada tendrá la forma que se puede apreciar en la figura B.5. Para producir esa salida es necesario que el fichero `leer_polinomios.3p` tenga el contenido siguiente:

```
.TH leer_polinomio 3P
.SH NAME
leer_polinomio \- función para leer polinomios
.SH SYNOPSIS
\fB#include "pol.h"\fP
.PP
.B POLINOMIO
leer_polinomio (\fBint grado\fP);
.SH DESCRIPTION
La función
.B leer_polinomios
se utiliza para leer, del fichero estándar de entrada, un polinomio
de grado
.B grado
que se define con la estructura
.B POLINOMIOS.
.PP
El tipo
.B POLINOMIO
se define de la forma siguiente:
.PP
.B typedef struct {
.br
.B int grado;
.br
.B float *coef;
.br
.B }POLINOMIO;
.PP
La forma de uso se muestra en las líneas de código siguientes:
.PP
 POLINOMIO p;
.br
```

```

 int grado;
.br
 printf ("Grado del polinomio\n");
.br
 scanf ("%d", &grado);
.br
 printf ("Coeficientes del polinomio: \n");
.br
 p = leer_polinomio (grado);
.PP
Para introducir el polinomio $X^5+5X^3-2X^2+4$, la comunicación con el
fragmento de programa anterior será la siguiente:
.PP
 Grado del polinomio: \fB5\fP
.br
 Coeficientes del polinomio: \fB1 0 5 -2 0 4\fP
.PP
Para utilizar la función
.B leer_polinomio
es necesario enlazar el programa ejecutable con la biblioteca
.B libpol.a.
Las opciones de compilación serán por lo tanto:
.PP
cc programa.c
.B -Ipolinomios/include -Lpolinomios/lib -lpol
.SH RETURN VALUE
La función devuelve los campos de una estructura \fBPOLINOMIO\fP.
Como la estructura tiene un campo de tipo puntero, \fBfloat *\fP, la
función reserva la memoria necesaria para almacenar los coeficientes.
.SH RELATED FILES
.PP
.B polinomios/include/pol.h
.br
.B polinomios/lib/libpol.a
.SH SEE ALSO
.PP
escribir_polinomio(3P), sumar_polinomios(3P),
multiplicar_polinomios(3P), simplificar_polinomios(3P)
.SH AUTHOR
Francisco Manuel Márquez García (francisco.marquez@uah.es)

```

El significado de las secuencias de control que se han utilizado es el siguiente:

- .TH, cabecera de título —*Title header*—.
- .SH, cabecera de epígrafe o sección —*Section header*—.

**leer\_polinomio(3P)****leer\_polinomio(3P)****NAME**

leer\_polinomio - función para leer polinomios

**SINOPSIS**

```
#include "pol.h"
```

```
POLINOMIO leer_polinomio (int grado);
```

**DESCRIPTIONS**

La función **leer\_polinomios** se utiliza para leer, del fichero estándar de entrada, un polinomio de grado **grado** que se define con la estructura **POLINOMIO**.

El tipo **POLINOMIO** se define de la forma siguiente:

```
typedef struct {
 int grado;
 float *coef;
}POLINOMIO;
```

La forma de uso se muestra en las líneas de código siguientes:

```
POLINOMIO p;
int grado;
printf ("Grado del polinomio\n");
scanf ("%d", &grado);
printf ("Coeficientes del polinomio: \n");
p = leer_polinomio (grado);
```

Para introducir el polinomio  $X^5+5X^3-2X^2+4$ , la comunicación con el fragmento de programa anterior será la siguiente:

```
Grado del polinomio: 5
Coeficientes del polinomio: 1 0 5 -2 0 4
```

Para utilizar la función **leer\_polinomio** es necesario enlazar el programa ejecutable con la biblioteca **libpol.a**. Las opciones de compilación serán por lo tanto:

```
cc programa.c -Ipolinomios/include -Lpolinomios/lib -lpol
```

**RETURN VALUE**

La función devuelve los campos de una estructura **POLINOMIO**. Como la estructura tiene un campo de tipo puntero, **float \***, la función reserva la memoria necesaria para almacenar los coeficientes.

**RELATED FILES**

```
polinomios/include/pol.h
polinomios/lib/libpol.a
```

**SEE ALSO**

```
escribir_polinomio(3P), sumar_polinomios(3P),
multiplicar_polinomios(3P), simplificar_polinomios(3P)
```

**AUTHOR**

Francisco Manuel Márquez García (francisco.maquez@uah.es)

**Figura B.5:** Página del manual para la función leer\_polinomio.

- `.PP`, párrafo nuevo.
- `.B`, texto en **negrita**. Otras opciones válidas son: `.R` letra redonda, `.I` letra *itálica*.
- `.br`, empezar una nueva línea pero no un párrafo nuevo.
- `\-`, representa el carácter `-`
- `\e`, representa el carácter `\`
- `\fB`, cambiar a tipo de letra **negrita**. Otras secuencias válidas son: `\fR` tipo de letra redonda, `\fI` tipo de letra *itálica*.
- `\fP`, volver al tipo de letra anterior.

Una vez escrito el fichero `leer_polinomio.3p` podemos visualizarlo con el programa `groff` emitiendo la orden,

```
$ groff polinomios/doc/man3/leer_polinomio.3p -man | more
```

Como esta forma de trabajo es incómoda, lo mejor será indicarle a `man` que el directorio `polinomios/doc` también va a contener páginas del manual. Para ello se puede utilizar la variable de entorno `MANPATH`.

```
$ MANPATH=/usr/man:$HOME/polinomios/doc
```

```
$ export MANPATH
```

Las líneas anteriores pueden incluirse en el fichero `.profile` o `.bash_profile`. Ahora, la forma de pedir ayuda sobre `leer_polinomio` es mucho más cómoda,

```
$ man leer_polinomio
```

Hay que tener presente que `man` espera que en `polinomios/doc` se reproduzca una estructura de directorio similar a la que hay en `/usr/man`. Esto implica que si la ayuda sobre `leer_polinomio` se cataloga en la sección 3 del manual, el fichero `leer_polinomio.3p` debe estar en el subdirectorio `man3` del directorio `polinomios/doc`.

## B.10 Ejercicios

**B.1** Escriba un programa para manipular polinomios simbólicamente. Los módulos que compondrán este programa son los descritos en el § B.5 pág. 516 y las funciones de cada módulo serán también las descritas en ese apartado. Con objeto de no mezclar ficheros de distintas aplicaciones se recomienda crear un directorio de nombre `polinomios` donde residirán los módulos de la aplicación que se va a crear.

**B.2** Cree una biblioteca con nombre `pol.a` a partir de los módulos de la aplicación polinomios. Con objeto de mejorar la gestión de la biblioteca vamos a dividir el módulo `pol_es.c` en tres y `pol_arit.c` en otros tres. Así, la nueva composición de la aplicación será la siguiente:

- `pol.h`, fichero de cabecera.

- `pol_lee.c`, módulo donde se implementa la función `leer_polinomio`.
- `pol_esc.c`, módulo donde se implementa la función `escribir_polinomio`.
- `pol_cre.c`, módulo donde se implementa la función `crear_polinomio`.
- `pol_mul.c`, módulo donde se implementa la función `multiplicar_polinomios`.
- `pol_sum.c`, módulo donde se implementa la función `sumar_polinomios`.
- `pol_sim.c`, módulo donde se implementa la función `simplificar_polinomio`.
- `pol.c`, módulo donde se implementa la función `main`.

La biblioteca `pol.a` estará compuesta por los módulos `pol_leer.o`, `pol_esc.o`, `pol_cre.o`, `pol_mul.o`, `pol_sum.o` y `pol_sim.o`. Una vez creada la biblioteca se le debe añadir un índice de símbolos.

- B.3** Recompile la aplicación `pol` utilizando la biblioteca creada en el ejercicio anterior.
- B.4** Utilizando la orden `nm` visualice los símbolos que hay en el índice de la biblioteca `libpol.a`.
- B.5** Utilizando la orden `nm` visualice los símbolos y sus direcciones asociadas que hay en el programa `pol`.
- B.6** Distribuya los programas de la aplicación polinomios en una estructura de directorios como la descrita en la figura B.4. En esta figura no está contemplado que el estado actual de la aplicación es distinto, ya que en ejercicios anteriores se han generado seis módulos a partir de los módulos `pol_es.c` y `pol_arit.c`. Para realizar este ejercicio se debe tener en cuenta el estado actual de desarrollo de la aplicación.

Una vez que se hayan distribuido los ficheros con el código fuente, se deben borrar los ficheros objeto, la biblioteca y el fichero ejecutable.

A continuación se deben emitir las órdenes necesarias para recompilar los módulos fuente, generar la biblioteca `libpol.a` y el programa ejecutable `pol` y situarlos en sus directorios correspondientes.

- B.7** Escriba un fichero `Makefile` en el directorio `usr` del árbol dedicado a la aplicación polinomios. El objeto de este fichero es poder compilar la aplicación con la orden `make`. Hay que tener presente que, de acuerdo con los ejercicios anteriores, la forma de la aplicación es ligeramente distinta a la descrita en los ejemplos de esta documentación. Una vez escrito `Makefile`, compile la aplicación.
- B.8** Borre el fichero `libpol.a` de la aplicación polinomios y recompile la aplicación con la orden `make`. ¿Qué órdenes ha generado `make` para recompilar la aplicación?
- B.9** Añada al programa `Makefile` un objetivo para limpiar los directorios de la aplicación. El nombre de este objetivo será `limpiar`, y al ejecutar `make` debe encargarse de borrar los módulos objeto.



**B.10** Cree en el directorio `$HOME/polinomios` el directorio `doc` que estará dedicado a contener documentación sobre la aplicación `polinomios`.

Escriba una página del manual para documentar la función `escribir_polinomio` que forma parte de la biblioteca `libpol.a`.

Modifique la variable de entorno `MANPATH` para que el programa `man` pueda encontrar las páginas del manual que hay en el directorio `$HOME/polinomios/doc`.

**B.11** Este ejercicio y los siguientes tienen como objetivo la repetición de todos los pasos necesarios para crear una aplicación compuesta por varios ficheros. El resultado final de la aplicación será un programa que podrá realizar operaciones básicas con matrices de números complejos.

La aplicación deber residir en el directorio `matrices`. Aquí se creará una jerarquía de subdirectorios como la descrita en los ejemplos:

- `src` para los ficheros fuente,
- `include` para los ficheros de cabecera,
- `lib` para la biblioteca `libmat.a`,
- `bin` para el programa ejecutable `cmat` y
- `doc` para la documentación.

Los módulos fuente que componen la aplicación son: `complejo.h`, `matriz.h`, `complejo.c`, `matriz.c` y `cmat.c`.

A continuación se muestra el contenido de tres de estos módulos:

```
// Fichero: complejo.h
// Fichero de cabecera con la definición de los datos y los prototipos de funciones
// para manejar números complejos.
#if !defined (_COMPLEJO_H_)
#define _COMPLEJO_H_

// Definición de los datos
typedef struct {
 float r, i;
}complejo;

// Interfaz de las funciones
complejo leer_complejo (void);
void escribir_complejo (complejo c);
complejo csum (complejo a, complejo b); // Devuelve a+b
complejo cmul (complejo a, complejo b); // Devuelve a*b

#endif
```

```

// Fichero: matriz.h
// Fichero de cabecera con la definición de los datos y los prototipos de funciones
// para manejar matrices de números complejos
 #if !defined (_MATRIZ_H_)
 #define _MATRIZ_H
 #include "complejo.h"

 // Definición de los datos
 typedef struct {
 int filas, columnas;
 complejo **coef;
 }matriz;

 // Interfaz de las funciones
 matriz leer_matriz (void);
 void escribir_matriz (matriz m);
 // Cálculo de a+b
 matriz sumar_matrices (matriz a, matriz b);
 // Cálculo de a*b
 matriz multiplicar_matrices (matriz a, matriz b);
 // Reservar memoria para los coeficientes de una matriz
 matriz crear_matriz (int filas, int columnas);
 // Liberar la memoria de los coeficientes de una matriz
 void destruir_matriz (matriz m);

 #endif

// Fichero: cmat.c
// Contiene la función principal para probar la biblioteca libmat.a
 #include "matriz.h"

 main ()
 {
 matriz a, b;

 a = leer_matriz ();
 b = leer_matriz ();
 imprimir_matriz (multiplicar_matrices (a, b));
 }

```

En los módulos `complejos.c` y `matriz.c` estarán las funciones que se describen en los ficheros de cabecera `complejos.h` y `matriz.h`. En este ejercicio se deben escribir los ficheros `complejos.c` y `matriz.c`. Para codificar algunas de las funciones que se piden conviene recordar algunos conceptos sobre la aritmética de números complejos y de matrices:

- (a) Si  $a$  y  $b$  son números complejos,  $\begin{cases} \Re(a \times b) = \Re(a) \times \Re(b) - \Im(a) \times \Im(b) \\ \Im(a \times b) = \Re(a) \times \Im(b) + \Im(a) \times \Re(b) \end{cases}$
- (b) Sea  $\mathbf{A} \in M_{m \times n}$  y  $\mathbf{B} \in M_{p \times q}$ . Para que  $\mathbf{A}$  y  $\mathbf{B}$  se puedan multiplicar se debe cumplir que  $n = p$ . El resultado será una matriz  $\mathbf{C} \in M_{m \times q}$  en la que:

$$c_{ij} = \sum_{h=1}^n a_{ih} \cdot b_{hj}; \quad 1 \leq i \leq m, \quad 1 \leq j \leq q$$

**B.12** Para llevar a cabo la compilación de la aplicación, habrá que escribir un fichero **Makefile** que tenga en cuenta las relaciones que existen entre todos los módulos, incluidos los ficheros de cabecera, así como la distribución de los mismos en los directorios que se han indicado en el ejercicio anterior. Hay que tener en cuenta que los módulos **complejo.o** y **matriz.o** se deben utilizar para crear la biblioteca **libmat.a**. El programa **cmat** debe ser enlazado con la biblioteca **libmat.a** y no con los módulos **complejo.o** y **matriz.o**. Una vez escrito el fichero **Makefile** compile la aplicación.

**B.13** Genere una página del manual donde se explique la interfaz y forma de uso de la función **multiplicar\_matrices**. Añada el directorio **matrices/doc** a la ruta de búsqueda del programa **man** y compruebe que se puede hacer la consulta:

```
$ man multiplicar_matrices
```

## Apéndice C

# Resumen de llamadas al sistema

A continuación mostramos un resumen de las llamadas al sistema que hemos explicado a lo largo del libro. Para una referencia más exhaustiva, es aconsejable consultar los manuales de referencia del sistema con el que se está trabajando. La documentación de las llamadas al sistema se encuentra en la sección 2 del manual de UNIX; sin embargo, algunos fabricantes no se ajustan a la forma estándar de numeración de los manuales, optando por una numeración propia.

Los manuales de UNIX también se publican en forma de libro, por ejemplo, la referencia [OSF, 1994] es una guía muy completa y cómoda de manejar. Adicionalmente también la podemos consultar en la página [www.UNIX-systems.org](http://www.UNIX-systems.org).

En [Joy *et al.*, 1993] podemos encontrar una descripción de las llamadas al sistema BSD agrupadas por familias.

Todas las llamadas al sistema devuelven algún resultado. Normalmente, el valor 0 o un valor positivo indican que la llamada se ha ejecutado satisfactoriamente, y el valor -1 indica que se ha producido algún error. En caso de error, la variable global `errno` contendrá un número que codifica el tipo de error producido. Este número tendrá un significado u otro dependiendo de cada llamada concreta.

### `accept`

```
#include <sys/socket.h>
int accept (int sfd, void *addr, int *addrlen);
```

Esta llamada la usan los procesos servidores para leer peticiones de servicio de conectores orientados a conexión. El argumento `sfd` es un descriptor del conector creado por una llamada previa a `socket` y unido a una dirección mediante `bind`. La llamada `accept` extrae la primera petición de conexión que hay en la cola de peticiones pendientes, creada con una llamada previa a `listen`. Una vez extraída la petición, `accept` crea un nuevo conector con las mismas propiedades que `sfd` y reserva un nuevo descriptor de fichero —`nsfd`— para él.

Si no hay peticiones de conexión pendientes y el conector no tiene activo el modo de acceso no bloqueante —`O_NDELAY`—, `accept` permanece bloqueada hasta que se reciba una nueva petición de conexión. Si el modo no bloqueante está activo, `accept` devolverá el control inmediatamente e indicará que se ha producido un error.

El conector original —`sfd`— permanece abierto y puede aceptar nuevas conexiones; sin embargo, el conector recién creado —`nsfd`— no puede usarse para aceptar más conexiones. La llamada `select` puede usarse para determinar si un conector tiene pendiente alguna petición de conexión.

El argumento `addr` debe apuntar a una estructura local con la dirección del conector. La llamada `accept` rellenará esa estructura con la dirección del conector remoto que pide la conexión. El formato de la estructura de dirección dependerá del tipo de conector que se esté manejando. El argumento `addrlen` debe ser un puntero a `int`. Inicialmente, debe contener el tamaño en bytes de la estructura de dirección. La función sobrescribirá en `addrlen` el tamaño real de la dirección leída de la cola de conexiones.

## access

```
#include <unistd.h>
int access (char *path, int amode);
```

Consulta los permisos de acceso al fichero cuya ruta está apuntado por `path`; `amode` puede tomar los siguientes valores:

`R_OK` Permiso de lectura.

`W_OK` Permiso de escritura.

`X_OK` Permiso de ejecución.

`F_OK` Existencia del fichero.

La función devuelve 0 si el proceso tiene permiso de acceso según el modo especificado, y -1 en caso contrario.

## acct

```
int acct (char *path);
```

Habilita o deshabilita la contabilidad del sistema, `path` es un puntero a la ruta del fichero sobre el cual se escribirá la información de contabilidad; si `path` vale `NULL`, la contabilidad se deshabilita. Para ejecutar esta llamada es necesario tener privilegios de superusuario.

## alarm

```
#include <unistd.h>
unsigned long alarm (unsigned long sec);
```

Activa un temporizador descendente en tiempo real de `sec` segundos. Cuando el temporizador llega a 0, el proceso que hizo la llamada recibirá la señal `SIGALRM`. En los sistemas con posibilidad de manejar varios tipos de temporizadores, `alarm` actúa sobre el temporizador `ITIMER_REAL`.

## bind

```
#include <sys/socket.h>
#include <sys/un.h> // Sólo para la familia AF_UNIX
#include <sys/netinet.h> // Sólo para la familia AF_INET
int bind (int sfd, const void *addr, int addrlen);
```

Esta llamada hace que el conector con descriptor `sfd` se una a la dirección de conector especificada en la estructura apuntada por `addr`, `addrlen` indica el tamaño de la dirección.

El formato de la dirección depende de la familia del conector, por lo que habrá que utilizar la estructura adecuada. Para la familia `AF_UNIX` se debe usar la estructura `struct sockaddr_un` y para la familia `AF_INET` la estructura `struct sockaddr_in`.

## brk, sbrk

```
#include <unistd.h>
int brk (char *endds);
char *sbrk (int incr);
```

Llamadas para cambiar el tamaño del segmento de datos de un proceso; `brk` cambia la posición del fin del segmento de datos, haciendo que tome el valor de `endds`; `sbrk` incrementa el tamaño del mismo segmento en la cantidad de bytes especificados por `incr`.

## chdir

```
#include <unistd.h>
int chdir (char *path);
```

Cambia el directorio de trabajo actual asociado a un proceso. El nuevo directorio será el asociado a la ruta apuntada por `path`.

## chmod, fchmod

```
#include <sys/types.h>
#include <sys/stat.h>
int chmod (char *path, mode_t mode);
int fchmod (int fildes, mode_t mode);
```

Llamadas para cambiar la máscara de permisos de un fichero de acuerdo con el valor de `mode`; `chmod` trabaja con una ruta de fichero —`path`—, y `fchmod` lo hace con el descriptor de un fichero ya abierto —`fildes`—; en ambos casos, el argumento `mode` se puede formar a partir de las siguientes constantes:

|                      |       |                                                              |
|----------------------|-------|--------------------------------------------------------------|
| <code>S_ISUID</code> | 04000 | Cambiar el identificador del usuario al ejecutar             |
| <code>S_ISGID</code> | 02000 | Cambiar el identificador del grupo al ejecutar               |
| <code>S_ISVTX</code> | 01000 | Mantener el segmento de texto en memoria después de ejecutar |
| <code>S_IRUSR</code> | 00400 | Permiso de lectura para el propietario                       |
| <code>S_IWUSR</code> | 00200 | Permiso de escritura para el propietario                     |
| <code>S_IXUSR</code> | 00100 | Permiso de ejecución —búsqueda— para el propietario          |
| <code>S_IRGRP</code> | 00040 | Permiso de lectura para el grupo                             |
| <code>S_IWGRP</code> | 00020 | Permiso de escritura para el grupo                           |
| <code>S_IXGRP</code> | 00010 | Permiso de ejecución —búsqueda— para el grupo                |
| <code>S_IROTH</code> | 00004 | Permiso de lectura para otros                                |
| <code>S_IWOTH</code> | 00002 | Permiso de escritura para otros                              |
| <code>S_IXOTH</code> | 00001 | Permiso de ejecución —búsqueda— para otros                   |

## chown, fchown

```
#include <sys/types.h>
#include <unistd.h>
int chown (char *path, uid_t owner, gid_t group);
int fchown (int fildes, uid_t owner, gid_t group);
```

Cambian el propietario y el grupo al que pertenece un fichero de acuerdo con los valores de `owner` —`UID` del nuevo propietario— y `group` —`GID` del nuevo grupo—; `chown` trabaja con el nombre de un fichero y `fchown` lo hace con el descriptor de un fichero ya abierto.

## chroot

```
#include <unistd.h>
int chroot (char *path);
```

Llamada para cambiar el directorio raíz asociado a un proceso. El nuevo directorio raíz pasa a ser el referenciado por el puntero `path`.

## close

```
#include <unistd.h>
int close (int fildes);
```

Libera el descriptor de fichero `fildes` y cierra su fichero asociado en el caso de que no haya más procesos que lo tengan abierto.

## connect

```
#include <sys/socket.h>
#include <sys/un.h> // Sólo para la familia AF_UNIX
#include <sys/netinet.h> // Sólo para la familia AF_INET
int connect (int sfd, const void *addr, int addrlen);
```

Para que un proceso cliente establezca una conexión con un servidor es necesario que haga una llamada a `connect`. El parámetro `sfd` es el descriptor del conector que da acceso al canal, `addr` es un puntero a una estructura que contiene la dirección del conector remoto al que queremos conectarnos y `addrlen` es el tamaño en bytes de la dirección. La estructura de la dirección dependerá de la familia de conectores con la que estemos trabajando.

## creat

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int creat (char *path, mode_t mode);
```

Llamada para crear un nuevo fichero cuya ruta está referenciada por `path`. El fichero se creará con los permisos que se especifican en `mode`. Si el fichero ya existe, `creat` trunca su longitud a 0 bytes. Si la llamada falla, devolverá el valor -1, en caso contrario devolverá un descriptor de fichero que se podrá utilizar en otras llamadas al sistema para acceder al fichero.

## dup

```
#include <unistd.h>
int dup (int fildes);
```



Duplica una entrada en la tabla de descriptores de ficheros, devolviendo un nuevo descriptor que tendrá en común con **fildes** los siguientes campos:

- Apuntará al mismo fichero abierto.
- Igual modo de acceso —lectura, escritura, lectura/escritura—.
- Igual indicador de estado.

El descriptor devuelto es el menor de entre los que haya disponibles.

## execl, execv, execl, execve, execlp, execvp

```
#include <unistd.h>
int execl (char *path, char *arg0, ... char *argn, (char *)0);
int execv (char *path, char *argv[]);
int execl (char *path, char *arg0, ... char *argn, (char *)0, char *envp[]);
int execve (char *path, char *argv[], char *envp[]);
int execlp (char *file, char *arg0, ... char *argn, (char *)0);
int execvp (char *file, char *argv[]);
```

La llamada **exec**, en todas sus formas, carga un programa en memoria reemplazando los segmentos de código y datos del proceso que hace la llamada.

El parámetro **path** es la ruta absoluta o relativa del programa.

El parámetro **file** es el nombre del programa y no su ruta; en este caso, el fichero se busca en los directorios especificados en la variable de entorno **PATH**.

Los parámetros **arg0**, ... **argn** son punteros a cadenas de caracteres que constituyen la lista de argumentos del nuevo programa. Por convenio, al menos **arg0** debe estar presente y apuntará a una cadena idéntica a **path** —o **file**— o a su último componente.

El parámetro **argv** es un array de punteros a cadenas de caracteres que constituyen la lista de argumentos disponibles para el nuevo programa. Por convenio, **argv** debe tener al menos un miembro, que debe apuntar a una cadena idéntica a **path** —o **file**— o a su último componente. El último miembro de **argv** debe ser un puntero **NULL**.

El parámetro **envp** es un array de punteros a cadenas de caracteres. Estas cadenas constituyen el entorno —conjunto de variables de entorno— en el cual se ejecutará el nuevo programa. El último elemento de **envp** debe ser un puntero a **NULL**.

## exit

```
#include <stdlib.h>
void exit (int status);
```

Termina la ejecución del proceso que la llama y devuelve el valor de **status** al sistema operativo para que lo pueda examinar; **exit** se encarga de cerrar los ficheros abiertos por el proceso y de sincronizar sus memorias intermedias con el disco.

## fcntl

```
#include <sys/types.h>
#include <unistd.h>
#include <fcntl.h>
int fcntl (int fildes, int cmd, union {int val; struct flock *lockdes;} arg);
```

Permite realizar un conjunto de operaciones de control sobre ficheros abiertos. El fichero al cual se refiere la acción se referencia por su descriptor **fildes**, **cmd** codifica el tipo de operación a llevar a cabo y **arg** contiene los parámetros necesarios para realizarla. Las constantes siguientes se pueden utilizar como valores de **cmd**:

- F\_DUPFD** Devolver un descriptor de fichero referido al mismo fichero que el descriptor **fildes**. El descriptor devuelto es el menor disponible que cumpla ser menor o igual que **arg**.
- F\_GETFD** Devolver el valor del indicador *close-on-exec*.
- F\_SETFD** Activar —poner a 1— el indicador *close-on-exec*.
- F\_GETFL** Devolver el valor de los indicadores de estado del fichero.
- F\_SETFL** Modificar el valor de los indicadores de estado de acuerdo con el argumento **arg.val**.
- F\_GETLK** Devolver el primer cerrojo que se ajusta a la definición dada por el argumento **arg.lockdes**. La información se devuelve sobrescribiendo en la estructura apuntada por **arg.lockdes**. Si no hay cerrojos que se ajusten a la descripción, la estructura devuelta tendrá activo el bit **F\_UNLCK** en el campo **l\_type**. El tipo **struct flock** se define como sigue:

```
struct flock {
 short l_type; // Tipo de cerrojo:
 // F_RDLCK — lectura
 // F_WRLCK — escritura
 // F_UNLCK — eliminar el cerrojo
 short l_whence; // Punto inicial de la región a bloquear:
 // SEEK_SET — origen del fichero
 // SEEK_CUR — posición actual
 // SEEK_END — final del fichero
 off_t l_start; // Posición relativa de inicio de la región a bloquear.
 // Está referida al punto indicado en l_whence
 off_t l_len; // Longitud de la región a bloquear. Si vale 0, se bloquea
 // desde el punto indicado en l_start hasta el final del fichero
 pid_t l_pid; // Identificador del proceso que ha fijado el cerrojo.
 // Devuelto con la orden F_GETLK
 long l_sysid; // Identificador del sistema que ha fijado el cerrojo.
 // Devuelto con la orden F_GETLK
};
```

- F\_SETLK** Declarar o borrar un cerrojo de acuerdo con la variable de tipo `struct flock` apuntada por `arg.lockdes`. Si el cerrojo no se puede activar, la función devuelve `-1` inmediatamente.
- F\_SETLKW** Igual que **F\_SETLK**, con la diferencia de que la función duerme hasta que el cerrojo se pueda activar.

## fork

```
#include <sys/types.h>
#include <unistd.h>
pid_t fork ();
```

Crea un proceso nuevo, el proceso que llama a `fork` es conocido como padre, y el creado, como hijo. El proceso hijo es una copia exacta del padre, con la diferencia de que `fork` le devuelve al proceso hijo el valor 0, y al padre el valor de identificador —*PID*— del hijo.

## fsync

```
#include <unistd.h>
int fsync (int fildes);
```

Sincroniza con el disco las memorias intermedias que el sistema mantiene referentes al fichero descrito por `fildes`.

## getitimer, setitimer

```
#include <sys/time.h>
getitimer (int wich, struct itimerval *value);
setitimer (int wich, struct itimerval *value, struct itimerval *ovalue);
```

Estas llamadas se utilizan para controlar los tres temporizadores que hay definidos por cada proceso. La llamada `getitimer` se utiliza para leer el estado del temporizador especificado por `wich`. Los valores que puede tomar este argumento son:

- ITIMER\_REAL** Temporizador en tiempo real.
- ITIMER\_VIRTUAL** Temporizador en tiempo virtual. Sólo contabiliza el tiempo mientras el proceso se está ejecutando y no cuando duerme.
- ITIMER\_PROF** Temporizador de perfilado, se usa al crear perfiles de un proceso.

El valor del temporizador es devuelto a través de los campos de la estructura apuntada por `value`. Esta estructura se define como sigue:

```
struct itimerval {
 struct timeval it_interval; // Intervalo del temporizador
 struct timeval it_value; // Valor actual del temporizador
};
```

La estructura `timeval` se define así:

```
struct timeval {
 // Segundos transcurridos desde la Época —origen de tiempos—
 unsigned long tv_sec;
 // Microsegundos. Su rango está comprendido entre 0 y 999.999
 long tv_usec;
};
```

La llamada `setitimer` se utiliza para definir el valor del temporizador especificado por `which`, `value` es un puntero a una estructura con los nuevos valores del temporizador y `ovalue` es un puntero a una estructura donde se devuelven los antiguos valores del temporizador.

## getpid, getpgrp, getppid

```
#include <sys/types.h>
#include <unistd.h>
pid_t getpid ();
pid_t getpgrp ();
pid_t getppid ();
```

Estas llamadas permiten obtener identificadores relacionados con el proceso que las invoca: `getpid` devuelve el identificador del proceso —*PID*— que la llama, `getpgrp` devuelve el identificador del grupo de procesos —*GID*— al que pertenece el proceso que la llama y `getppid` devuelve el identificador del proceso padre —*PPID*— del que la llama.

## gettimeofday, settimeofday

```
#include <sys/time.h>
int gettimeofday (struct timeval *tp, struct timezone *tzp);
int settimeofday (struct timeval *tp, struct timezone *tzp);
```

Estas dos llamadas se utilizan para manipular el reloj interno del sistema. Con `gettimeofday` podemos leer la fecha del sistema expresada en segundos y microsegundos con respecto a la *Época*, `settimeofday` se utiliza para fijar una nueva fecha del sistema.

Los parámetros `tp` y `tzp` son punteros a estructuras del tipo `timeval` y `timezone`, respectivamente. Estas estructuras se definen como sigue:

```
struct timeval {
 // Segundos transcurridos desde la Época
```

```
 unsigned long tv_sec;
 // Microsegundos. Su rango está comprendido entre 0 y 999.999
 long tv_usec;
};

struct timezone {
 int tz_minuteswest; // Corrección local con respecto a la hora UTC
 int tz_dsttime; // Corrección anual —horario de invierno o de verano—
};
```

## getuid, geteuid, getgid, getegid

```
#include <sys/types.h>
#include <unistd.h>
uid_t getuid ();
uid_t geteuid ();
gid_t getgid ();
gid_t getegid ();
```

La llamada `getuid` devuelve el identificador real de usuario *UID* del proceso que la llama, `geteuid` devuelve el identificador efectivo de usuario *EUID* del proceso que la llama, `getgid` devuelve el identificador real de grupo *GID* del proceso que la llama y `getegid` devuelve el identificador efectivo de grupo *EGID* del proceso que la llama.

## ioctl

```
#include <sys/ioctl.h>
ioctl (int fildes, int request, arg);
```

Lleva a cabo un conjunto muy amplio de funciones sobre ficheros de dispositivo, `fildes` es el descriptor del fichero sobre el que queremos trabajar, `request` codifica la acción que se quiere llevar a cabo sobre el fichero y `arg` contiene los parámetros que se van a utilizar para llevar a cabo la acción. Dependiendo del tipo de fichero, `request` podrá tomar unos valores u otros, y `argv` tendrá una forma u otra. En la sección 7 del manual de UNIX podemos encontrar una descripción detallada de todos los dispositivos que maneja el sistema y de los argumentos que podemos pasarle a `ioctl` para programar esos dispositivos.

## kill

```
#include <sys/types.h>
#include <signal.h>
int kill (pid_t pid, int sig);
```

Esta llamada se utiliza para enviarle señales a un proceso o a un conjunto de procesos, donde `pid` es el identificador del proceso al que le enviamos la señal y `sig` es el número de señal a enviar.

Cuando `sig` vale 0 no se envía ninguna señal. Este caso se utiliza para averiguar si el proceso `pid` existe o no. Si el proceso existe, `kill` devolverá el valor 0; en caso contrario, devolverá -1.

El parámetro `pid` puede tomar los siguientes valores:

- `pid > 0` Enviar la señal al proceso cuyo *PID* coincide con `pid`.
- `pid = 0` Enviar la señal a todos aquellos procesos cuyo identificador de grupo de procesos coincide con el *PID* del que hace la llamada a `kill`.
- `Pid = -1` Si el *UID* efectivo del proceso actual es el del superusuario, enviar la señal a todos los procesos. En caso contrario, enviar la señal a todos aquellos procesos cuyo *UID* real es igual al *UID* efectivo del proceso actual.

## link

```
#include <unistd.h>
int link (char *path1, char *path2);
```

Crea una entrada de directorio cuya ruta será igual a la cadena apuntada por `path2`, esta nueva entrada será un enlace con el fichero cuyo nodo-i es el correspondiente a la ruta apuntada por `path1`.

## listen

```
int listen (int sfd, int backlog);
```

La llamada `listen` habilita una cola asociada al conector descrito por `sfd`, esta cola se utiliza para alojar peticiones de conexión procedentes de los procesos cliente, la longitud de esta cola es la especificada en el argumento `backlog`. Para que la llamada a `listen` tenga sentido, el conector debe ser del tipo `SOCK_STREAM` —conector orientado a conexión—.

La cola de conexiones es importante para los servidores de tipo interactivo, porque mientras están atendiendo a un cliente pueden llegar peticiones procedentes de otros. En los de tipo concurrente, el único instante en el que el servidor no atiende la recepción de peticiones es el que dedica a la llamada `fork`. En estos casos también es importante la cola de conexiones.

## lockf

```
#include <unistd.h>
int lockf (int fildes, int function, long size);
```

Proporciona dos modalidades de bloqueo de ficheros: bloqueo consultivo y bloqueo obligatorio; **fildes** es el descriptor del fichero que se desea bloquear; **function** es la acción de bloqueo que se llevará a cabo, puede tomar los siguientes valores:

**F\_ULOCK** Desbloquear una región del fichero.

**F\_LOCK** Bloquear una región del fichero.

**F\_TLOCK** Comprobar si una región está bloqueada. Si no lo está, pasar a bloquearla; en caso contrario, esperar a que quede libre.

**F\_TEST** Preguntar si una región está bloqueada. Si no lo está, la función devuelve el valor 0; en caso contrario, devuelve -1.

La región bloqueada o desbloqueada tiene un tamaño de **size** bytes y empieza en la posición actual del puntero de lectura/escritura.

## lseek

```
#include <sys/types.h>
#include <unistd.h>
off_t lseek (int fildes, off_t offset, int whence);
```

Desplaza el puntero de lectura/escritura del fichero descrito por **fildes** un total de **offset** bytes con respecto a la posición indicada en **whence**, este parámetro puede tomar los valores:

**SEEK\_SET** Principio del fichero.

**SEEK\_CUR** Posición actual del puntero de lectura/escritura.

**SEEK\_END** Final del fichero.

La función devuelve la posición actual del puntero de lectura/escritura, medida en bytes, con respecto al inicio del fichero.

## mkdir

```
#include <sys/types.h>
#include <sys/stat.h>
int mkdir (char *path, mode_t mode);
```

Crea un fichero de directorio con una ruta igual a la cadena apuntada por **path**; **mode** codifica los permisos de lectura, escritura y ejecución para el propietario, grupo de usuarios al que pertenece el propietario y el resto de los usuarios.

## mknod

```
#include <sys/types.h>
#include <sys/stat.h>
int mknod (char *path, mode_t mode, dev_t dev);
```

Llamada para crear un fichero de dispositivo, una tubería con nombre o un directorio, **path** apunta a una cadena de caracteres que contiene la ruta del fichero a crear, **mode** es la máscara de modo del fichero en la que sólo uno de los bits siguientes puede estar activo:

**S\_IFIFO** Crear una tubería con nombre.

**S\_IFCHR** Crear un fichero de dispositivo modo carácter.

**S\_IFBLK** Crear un fichero de dispositivo modo bloque.

**S\_IFDIR** Crear un directorio.

Los 9 bits menos significativos de **mode** codifican los permisos del fichero. Si el fichero a crear es de dispositivo, **dev** debe codificar los *major* y *minor number* del mismo.

## mount

```
int mount (char *spec, char *dir, int rwflag);
```

Monta sobre el directorio **dir** el sistema de ficheros que hay construido en el dispositivo **spec**. Si el bit menos significativo de **rwflag** vale 1, el sistema es montado en modo sólo lectura.

## msgctl

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgctl (int msqid, int cmd, struct msqid_ds *buf);
```

Esta llamada realiza operaciones de control sobre la cola de mensajes identificada por **msqid**, **cmd** indica la operación que se realizará y sus posibles valores son:

**IPC\_STAT** Leer la cabecera de la cola de mensajes y la copia en la estructura apuntada por **buf**.

**IPC\_SET** Modificar los valores de algunos de los campos de la cabecera de la cola, de acuerdo con esos mismos campos de la estructura apuntada por **buf**. Los campos que se modifican son: **msg\_qbytes**, **msg\_perm.uid**, **msg\_perm.gid** y **msg\_perm.mode** —sólo los 9 bits menos significativos—.

**IPC\_RMID** Borrar la cola de mensajes.



La estructura `msgqid_ds` se define como sigue:

```
struct ipc_perm {
 ushort uid; // Identificador del usuario actual
 ushort gid; // Identificador del grupo actual
 ushort cuid; // Identificador del usuario creador de la cola
 ushort cgid; // Identificador del grupo creador de la cola
 ushort mode; // Permisos de la cola
 short pad1; // Usado por el sistema
 long pad2; // Usado por el sistema
};

struct msgqid_ds {
 struct ipc_perm msg_perm; // Estructura de permisos
 short pad1[7]; // Usado por el sistema
 ushort msg_qnum; // Número de mensajes en la cola
 ushort msg_qbytes; // Número máximo de bytes que caben en la cola
 ushort msg_lspid; // PID de la última operación msgsnd
 ushort msg_lrpid; // PID de la última operación msgrcv
 time_t msg_stime; // Fecha de la última operación msgsnd
 time_t msg_rtime; // Fecha de la última operación msgrcv
 time_t msg_ctime; // Fecha del último cambio
};
```

## msgget

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgget (key_t key, int msgflg);
```

Devuelve un identificador de la cola de mensajes cuyo nombre es `key`, este parámetro debe ser una llave válida formada por una llamada previa a `ftok` o el valor `IPC_PRIVATE`. Si `key` vale `IPC_PRIVATE`, `msgget` devuelve un identificador libre que no ha sido reservado por ningún otro proceso y garantiza que sólo el proceso actual tiene acceso a esa cola. La cola pasa a ser de uso exclusivo para el proceso y sus descendientes.

El parámetro `msgflg` puede ser una combinación de los siguientes bits:

`IPC_CREAT` La cola se crea si no ha sido creada ya por otro proceso.

`IPC_EXCL` La llamada falla si la cola ya existe.

## msgop

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
int msgsnd (int msqid, void *msgp, int msgsz, int msgflg);
int msgrcv (int msqid, void *msgp, int msgsz, long msgtyp, int msgflg);
```

La llamada `msgsnd` envía a la cola identificada por `msqid` el mensaje que se encuentra en la zona de memoria apuntada por `msgp` y cuyo tamaño viene indicado por `msgsz`. La estructura del mensaje es la siguiente:

```
struct msgbuf {
 long mtype; // Tipo del mensaje
 char mtext []; // Datos del mensaje
};
```

Si el bit `IPC_NOWAIT` no está activo en el parámetro `msgflg` y la cola está llena, la llamada espera a que quede espacio libre. Si el bit `IPC_NOWAIT` está activo, la llamada devuelve el control inmediatamente en el caso de que no haya espacio libre en la cola.

La llamada `msgrcv` lee alguno de los mensajes que se encuentran en la cola descrita por `msqid`; el mensaje leído se escribe en la zona apuntada por `msgp`, cuyo tamaño es `msgsz`; `msgtyp` indica el tipo de mensaje a leer, sus valores significativos son:

`msgtyp=0` Para leer el primer mensaje que haya en la cola.

`msgtyp>0` Para leer el primer mensaje cuyo tipo coincida con `msgtyp`.

`msgtyp<0` Para leer el primer mensaje con el tipo más bajo que sea menor o igual a `msgtyp`.

Si el bit `IPC_NOWAIT` del parámetro `msgflg` no está activo, la función espera a que se reciba un mensaje del tipo solicitado. Si el bit está activo, la función no espera en el caso de que en la cola no haya ningún mensaje del tipo requerido.

Si se ejecutan satisfactoriamente, `msgsnd` devuelve 0 y `msgrcv` devuelve el número de bytes que hay en el campo `mtext` del mensaje recibido.

## nice

```
#include <unistd.h>
int nice (int incr);
```

Cambia la prioridad de un proceso al añadir el valor `incr` en la siguiente fórmula de cálculo de su prioridad:

$$\text{prioridad} = \frac{\text{uso\_reciente\_de\_CPU}}{\text{cte}} + \text{prioridad\_base} + \text{incr}$$

Los valores altos de `incr` degradan la prioridad dinámica del proceso.

## open

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int open (char *path, int oflag [,mode_t mode]);
```

Abre el fichero cuya ruta coincide con la cadena apuntada por **path**. El modo de apertura se especifica en **oflag** mediante una combinación de los siguientes bits:

- O\_RDONLY** Abrir en modo sólo lectura.
- O\_WRONLY** Abrir en modo sólo escritura.
- O\_RDWR** Abrir para leer y escribir.
- O\_NDELAY** Para ficheros de dispositivo asociados a líneas de comunicaciones, abrir sin esperar la portadora. Para una tubería con nombre, abrir sin esperar a que otro proceso abra la tubería.
- O\_APPEND** Situar el puntero de lectura/escritura al final del fichero para añadir datos.
- O\_CREAT** Si el fichero no existe, crearlo con la máscara de permisos especificada en **mode**.
- O\_TRUNC** Truncar la longitud del fichero a 0 bytes. Se perderán los datos del fichero.
- O\_EXCL** Apertura exclusiva. Si este bit está activo junto con **O\_CREAT** y el fichero existe, la llamada a **open** falla.

Si se ejecuta satisfactoriamente, **open** devuelve un descriptor de fichero que podrá ser usado por otras llamadas.

## pause

```
#include <unistd.h>
int pause (void);
```

Suspende la ejecución del proceso actual hasta que se reciba alguna señal.

## pipe

```
#include <unistd.h>
int pipe (int fildes [2]);
```

Crea una tubería sin nombre y devuelve dos descriptors a través de los cuales se puede leer y escribir en la tubería. El descriptor **fildes[1]** se emplea para escribir en la tubería, y **fildes[0]** para leer de ella.

## plock

```
#include <sys/lock.h>
int plock (int op);
```

Bloquea y desbloquea en memoria los distintos segmentos de un proceso de acuerdo con los siguientes valores de `op`:

**PROLOCK** Bloquear los segmentos de código y datos.

**TXLOCK** Bloquear el segmento de código.

**DATLOCK** Bloquear el segmento de datos.

**UNLOCK** Desbloquear todos los segmentos anteriormente bloqueados.

El bloqueo consiste en dejar residente en memoria de forma inmune al intercambio.

## profil

```
#include <sys/param.h>
void profil (char *buf, int bufsiz, int offset, int scale);
```

Activa o desactiva la creación del perfil de ejecución de un proceso, `buf` es un array donde se llevará la contabilidad de la frecuencia de ejecución de las distintas direcciones del proceso, `bufsiz` es el tamaño del array apuntado por `buf`, `offset` es la primera dirección del proceso que se va a tener en cuenta en la creación del perfil y `scale` es un factor de escala que indica la relación que existe entre las direcciones del proceso y las entradas de `buf`.

## ptrace

```
#include <sys/ptrace.h>
int ptrace (int request, int pid, int addr, int data, int addr2);
```

Esta llamada permite que un proceso padre —proceso depurador— controle la ejecución de su proceso hijo —proceso depurado— y es la base de codificación de los programas depuradores. El *PID* del proceso a depurar se indica mediante el parámetro `pid`, y la acción a llevar a cabo, mediante el parámetro `request`. Los posibles valores de `request` son:

- 0 Habilita la depuración en el proceso hijo (este parámetro lo utiliza sólo el proceso hijo).
- 1, 2 Devuelve el contenido de la dirección de memoria virtual referenciada por `addr` en el proceso hijo.
- 3 Devuelve el contenido de la posición `addr` del área de usuario del proceso hijo.

- 4, 5 Escribe, con el valor **data**, el contenido de la dirección de memoria virtual referenciada por **addr** en el proceso hijo.
- 6 Escribe, con el valor **data**, en la posición **addr** del área de usuario del proceso hijo.
- 7 Le indica al proceso hijo que continúe con su ejecución. El proceso hijo estará parado debido a la recepción de una señal.
- 8 Fuerza a que el proceso hijo termine su ejecución con una llamada a **exit**.
- 9 Le indica al proceso hijo que continúe su ejecución, pero activando el bit de traza del procesador. Esto hace que el proceso interrumpa su ejecución después de cada instrucción máquina. Esta orden se emplea para ejecutar un proceso paso a paso.

## read, readv

```
#include <unistd.h>
int read (int fildes, char *buf, unsigned nbytes);

#include <sys/uio.h>
ssize_t readv (int fildes, const struct iovec *iov, size_t iovcnt);
```

La llamada **read** lee datos procedentes del fichero descrito por **fildes** y los deposita en la zona de memoria apuntada por **buf**. El tamaño de la zona de memoria donde se escribe se indica con el parámetro **nbyte**. Si la llamada funciona correctamente, devuelve el número de bytes leídos.

La llamada **readv** es una generalización de **read** y puede utilizarse para leer datos de cualquier descriptor, ya sea fichero o conector. Esta llamada lee datos procedentes del fichero descrito por **fildes** y los distribuye entre las distintas zonas de memoria intermedia especificadas por el array **iov**. El total de elementos de **iov** viene indicado por el argumento **iovcnt**; así, **iov** describe un total de **iovcnt** memorias intermedias que son: **iov[0]**, **iov[1]**, ..., **iov[iovcnt-1]**.

Cada elemento del array **iov** es del tipo **struct iovec**, que se define con los siguientes campos:

```
struct iovec {
 caddr_t iov_base; // Dirección inicial de la memoria intermedia
 int iov_len; // Tamaño, en bytes, de la memoria intermedia
};
```

A medida que lee datos del fichero, **readv** irá rellenando las distintas memorias intermedias, empezando por la primera. Si llega al final del fichero o completa los bloques de memoria, termina su ejecución, devolviendo el número total de bytes leídos.

## recv, recvfrom, recvmsg

```
#include <sys/socket.h>
int recv (int sfd, void *buf, int len, int flags);
int recvfrom (int sfd, void *buf, int len, int flags, void *from, int fromlen);
int recvmsg (int sfd, struct msghdr msg [], int flags);
```

Estas llamadas se utilizan para leer datos de un conector. En todas ellas `sfd` es el descriptor del conector del cual se leen los datos, `buf` es un puntero a la memoria intermedia donde se escribirán los datos leídos y `len` es el número máximo de bytes que se pueden escribir en la memoria referenciada por `buf`.

La llamada `recvfrom` trabaja de la misma forma que `recv`, pero con la ventaja de que puede devolver la dirección del conector desde el que se enviaron los datos. En el caso de conectores *datagrama* en los que se ha establecido una conexión, la dirección devuelta por `recvfrom` siempre será la misma. Para conectores del tipo *flujo de datos* —*stream*—, `recvfrom` devuelve el mismo tipo de información que `recv` y no devuelve la dirección del conector origen.

Si `from` es un puntero distinto de `NULL`, la dirección del conector origen de los datos se escribirá en la estructura apuntada por `from`; `fromlen` es un parámetro de entrada/salida que al llamar a la función debe contener el tamaño de la estructura apuntada por `from` y, al salir de la función, contendrá el tamaño real de la dirección leída.

Para conectores basados en paso de mensajes, como los conectores *datagrama*, cada mensaje se debe leer en su totalidad en una sola operación. Si el mensaje es demasiado largo para caber en la memoria intermedia indicada, será truncado. En los conectores no basados en el paso de mensajes —flujos de datos—, los datos se devuelven a medida que están disponibles y no se maneja para nada el concepto de tamaño de un mensaje.

La llamada `recvmsg` es una generalización de las dos anteriores. La diferencia con ellas es que los datos leídos pueden distribuirse entre varios bloques de memoria. El argumento `msg` es un array de cabeceras de mensaje, cada una de las cuales tiene la siguiente estructura:

```
struct msghdr {
 caddr_t msg_name; // Dirección, opcional
 int msg_namelen; // Tamaño del campo msg_name
 struct iovec *msg_iov; // Array de bloques de memoria sobre los que
 // distribuir los datos leídos
 int msg_iovlen; // Número de elementos del array msg_iov
 caddr_t msg_accrights; // Derechos de acceso
 int msg_accrightslen; // Tamaño de msg_accrights
};
```

Esta estructura se emplea tanto para leer mensajes de un conector como para escribirlos en él. Los campos `msg_name` y `msg_namelen` se usan para conectores no conectados cuando nos interesa conocer la dirección del conector origen o destino del mensaje. Si no necesitamos conocer esa dirección, el campo `msg_name` puede ser un puntero `NULL`. El campo `msg_iov` es el array de bloques de memoria sobre los que distribuir los datos leídos, o de los que recoger los datos que se van a enviar. En `msg_accrights` se recogen

los derechos de acceso enviados junto con los mensajes. Este campo sólo se utiliza con los conectores de la familia `AF_UNIX` cuando los procesos se intercambian descriptores de fichero. Este campo puede ser un puntero `NULL`, con lo cual no se leen del conector los derechos de acceso.

Para las tres llamadas, el argumento `flags` tiene el mismo significado y sus posibles valores son el valor 0 o una combinación de los bits siguientes:

**MSG\_PEEK** Cualquier dato leído del conector es tratado como si la lectura no se hubiese llevado a cabo, por lo que la siguiente operación de lectura leerá los mismos datos. Esta opción se puede usar para oír si hay datos disponibles en el conector.

**MSG\_OOB** Se utiliza para leer datos que van fuera de banda. Éstos son datos a los que se les da carácter de urgentes, por lo que tienen preferencia en el envío y en la recepción.

En los conectores de la familia `AF_UNIX` no pueden estar activos ninguno de los bits anteriores.

Las tres funciones devuelven el total de bytes leídos del conector.

## rename

```
#include <stdio.h>
int rename (const char *source, const char *target);
```

Hace que el fichero cuya ruta indica `source` pase a llamarse como indica `target`.

## rmdir

```
#include <unistd.h>
int rmdir (char *path);
```

Borra el directorio cuyo nombre viene indicado por `path`. Para que un directorio pueda ser borrado, sus dos únicas entradas deben ser `.` y `...`

## select

```
#include <sys/time.h>
int select (int nfds, int readfds, int writefds, int exceptfds,
 struct timeval *timeout);
```

Examina los descriptores de fichero especificados en las máscaras de bits `readfds`, `writefds` y `exceptfds`. Se examinan los bits comprendidos entre las posiciones 0 y `nfds-1`, ambas inclusive. El fichero de descriptor `fd` se codifica mediante el bit de la posición `1<fd` de la máscara correspondiente.

El significado de cada máscara es el siguiente:

- readfds** Descriptores de los ficheros de lectura por los que preguntamos. Cada bit activo significa que **select** debe interrogar sobre el estado del fichero asociado para ver si tiene datos que leer.
- writefds** Descriptores de los ficheros de escritura por los que preguntamos. Cada bit activo significa que **select** debe interrogar sobre el estado del fichero asociado para ver si se puede escribir en él.
- exceptfds** Descriptores de los ficheros con alguna condición especial por los que preguntamos. Cada bit activo significa que **select** debe interrogar sobre el estado del fichero asociado para ver si se ha producido alguna condición especial.

Si alguna de las condiciones anteriores no nos interesa, lo indicaremos pasándole a **select** el argumento 0.

El parámetro **timeout** es un puntero no nulo que indica el tiempo máximo que se va a esperar desde que **select** entra en ejecución hasta que devuelve el control. La llamada **select** puede devolver el control, bien porque alguno de los ficheros interrogados cumpla los requisitos pedidos, bien porque el tiempo de espera especificado en **timeout** expire. En el caso de que el tiempo de espera se agote, **select** devolverá el valor 0 y en las máscaras aparecerán todos los bits a 0. Si **timeout** es un puntero NULL, la llamada esperará a que se produzca algún cambio de estado en alguno de los ficheros interrogados.

## semctl

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
int semctl (int semid, int semnum, int cmd, arg)
union semun {
 int val;
 struct semid_ds *buf;
 ushort *array;
}arg;
```

Actúa sobre el conjunto de semáforos que responden al identificador **semid** devuelto por una llamada previa a **semget**, **semnum** indica cuál es el semáforo de los que hay bajo **semid** al que queremos acceder, la operación de control que se realiza sobre el semáforo viene indicada en **cmd**, los siguientes son valores válidos para **cmd**:

- GETVAL** Se utiliza para leer el valor de un semáforo. El valor se devuelve a través del nombre de la función.
- SETVAL** Permite inicializar un semáforo a un valor determinado que se especifica en **arg**.



- GETPID** Se usa para leer el *PID* del último proceso que actuó sobre el semáforo. Este valor se devuelve a través del nombre de la función.
- GETNCNT** Permite leer el número de procesos que hay esperando a que se incremente el valor del semáforo. Este número se devuelve a través del nombre de la función.
- GETZCNT** Permite leer el número de procesos que hay esperando a que el semáforo tome el valor 0. Este número se devuelve a través del nombre de la función.
- GETALL** Permite leer el valor de todos los semáforos asociados al identificador **semid**. Estos valores se almacenan en **arg**.
- SETALL** Sirve para inicializar el valor de todos los semáforos asociados al identificador **semid**. Los valores de inicialización deben estar en **arg**.
- IPC\_STAT**
- e
- IPC\_SET** Permiten leer y modificar la información administrativa asociada al identificador **semid**.
- IPC\_RMID** Le indica al núcleo que debe borrar el conjunto de semáforos agrupados bajo el identificador **semid**. La operación de borrado no se efectuará mientras haya algún proceso que esté usando los semáforos.

## semget

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
int semget (key_t key, int nsems, int semflg);
```

Con **semget** podemos crear o acceder a un conjunto de semáforos unidos bajo un identificador común. Si la llamada funciona correctamente, devuelve un identificador con el que podremos acceder a los semáforos en sucesivas llamadas.

El parámetro **key** es la llave que indica a qué grupo de semáforos queremos acceder. Esta llave puede crearse mediante una llamada a **ftok**. Si **key** toma el valor **IPC\_PRIVATE**, la llamada crea un nuevo identificador, sujeto a la disponibilidad de entradas libres en la tabla que gestiona el núcleo para los semáforos.

El parámetro **nsems** es el total de semáforos que están agrupados bajo el identificador devuelto por **semget**.

El parámetro **semflg** es una máscara de bits que indica el modo de adquisición del identificador. Si el bit **IPC\_CREAT** está activo, la facilidad **IPC** se creará, en el supuesto de que otro proceso no la haya creado ya. Si los bits **IPC\_CREAT** e **IPC\_EXCL** están activos simultáneamente, la llamada falla en el caso de que el conjunto de semáforos ya esté creado. Los 9 bits menos significativos de **semflg** indican los permisos del semáforo.

## semop

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
int semop (int semid, struct sembuf *sops, int nsops);
```

Realiza operaciones atómicas sobre los semáforos que hay asociados bajo el identificador `semid`.

El parámetro `sops` es un puntero a un array de estructuras que indican las operaciones que se realizarán sobre los semáforos, `nsops` es el total de elementos que tiene el array de operaciones. Cada elemento del array es una estructura del tipo `sembuf` que se define como sigue:

```
struct sembuf {
 ushort sem_num; // Número del semáforo
 short sem_op; // Operación: incrementar o decrementar
 short sem_flg; // Máscara de bits
};
```

El campo `sem_num` es el número del semáforo y se utiliza como índice para acceder a él. Su valor está en el rango  $0..N - 1$ , donde  $N$  es el total de semáforos que hay agrupados bajo el identificador.

El campo `sem_op` es la operación que se realizará sobre el semáforo especificado en `sem_num`. Si `sem_op` es negativo, el valor del semáforo se decrementará, lo que equivale a una operación  $\mathcal{P}$ , si `sem_op` es positivo, el valor del semáforo se incrementará, lo que equivale a una operación  $\mathcal{V}$ . Si `sem_op` vale 0, el valor del semáforo no sufre ninguna alteración.

El campo `sem_flg` indica la forma de actuar en el caso de que la operación no se pueda realizar de forma inmediata. Así, si el bit `IPC_NOWAIT` está activo, la llamada devuelve el control inmediatamente; en caso contrario, el proceso duerme hasta que la operación se pueda realizar.

## send, sendto, sendmsg

```
#include <sys/socket.h>
int send (int sfd, void *buf, int len, int flags);
int sendto(int sfd, void *buf, int len, int flags, void *to, int tolen);
int sendmsg (int sfd, struct msghdr msg [], int flags);
```

Estas llamadas se utilizan para escribir en un conector. En todas ellas `sfd` es el descriptor del conector en el cual se escriben los datos, `buf` es un puntero a la zona de memoria donde están los datos que se desean enviar y `len` es el número de bytes que hay en esa zona de memoria.

En los conectores orientados a conexión —`SOCK_STREAM`— las llamadas pueden usarse sólo después de haber establecido una conexión previa llamada a `connect`. En los conec-

tores no orientados a conexión —`SOCK_DGRAM`—, debe usarse la función `sendto`, a menos que se especifique una dirección de destino con una llamada a `connect`. Si ya se ha especificado una dirección, al llamar a `sendto` se producirá un error si el parámetro `to` contiene una dirección. La dirección del conector destino está en la posición de memoria apuntada por `to` y su tamaño es `tolen`.

En los conectores del tipo *datagrama* para los que no se ha definido una dirección mediante la llamada correspondiente a `bind`, si se envía un mensaje con `sendto`, el sistema elegirá de forma automática una dirección local, pero no hay garantía de que esa dirección siga siendo la misma en sucesivas llamadas a `sendto`.

La llamada `sendmsg` es una generalización de las dos anteriores. La diferencia con ellas es que los datos a enviar pueden proceder de varios bloques de memoria. El argumento `msg` es un array de cabeceras de mensaje, cada una de las cuales tiene una estructura como la definida para la función `recvmsg` —`struct msghdr`—.

Para las tres llamadas, el argumento `flags` tiene el mismo significado y sus posibles valores son 0 o `MSG_OOB` —mensaje urgente—. En los conectores de la familia `AF_UNIX` no se pueden enviar mensajes urgentes.

Las tres funciones devuelven el total de bytes escritos en el conector.

## setuid, setgid

```
#include <sys/types.h>
#include <unistd.h>
int setuid (uid_t uid);
int setgid (gid_t gid);
```

La llamada `setuid` modifica los *UID* real y efectivo del proceso actual, de acuerdo con el valor de `uid`.

La llamada `setgid` modifica los *GID* real y efectivo del proceso actual, de acuerdo con el valor de `gid`.

## shmctl

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmctl (int shmid, int cmd, struct shmin_ds *buf);
```

Con `shmctl` podemos realizar operaciones de control sobre una zona de memoria compartida, `shmid` es un identificador válido, devuelto por una llamada previa a `shmget`, `cmd` indica el tipo de operación de control a realizar, sus posibles valores son:

**IPC\_STAT** Lee el estado de la estructura de control de la memoria y lo devuelve a través de la zona de memoria apuntada por `buf`.

- IPC\_SET Inicializa algunos de los campos de la estructura de control de la memoria compartida. Los valores de estos campos los toma de la estructura apuntada por `buf`.
- IPC\_RMID Borra del sistema la zona de memoria compartida identificada por `shmid`. Si el segmento de memoria está unido a varios procesos, el borrado no se hace efectivo hasta que todos los procesos liberen la memoria.
- SHM\_LOCK Bloquea en memoria el segmento identificado por `shmid`. Esto quiere decir que no se realiza intercambio sobre él. Sólo los procesos cuyo identificador de usuario efectivo *EUID* sea igual al del superusuario pueden realizar esta operación.
- SHM\_UNLOCK Desbloquea el segmento de memoria compartida, con lo que los mecanismos de intercambio podrán trasladarlo de la memoria principal a la secundaria, y viceversa, cada vez que sea necesario. Sólo los procesos cuyo identificador de usuario efectivo *EUID* sea igual al del superusuario pueden realizar esta operación.

La estructura `shmid_ds` se define como sigue:

```
struct shmid_ds {
 struct ipc_per shm_per; // Estructura de permisos
 int sh_segsz; // Tamaño del área de memoria compartida
 int pad1; // Usado por el sistema
 ushort shm_lpid; // PID del proceso que hizo la última operación con
 // este segmento de memoria
 ushort shm_cpid; // PID del proceso creador del segmento
 ushort shm_nattach; // Número de procesos unidos al segmento
 // de memoria
 short pad2; // Usado por el sistema
 time_t shm_atime; // Fecha de la última unión al segmento de memoria
 time_t shm_dtime; // Fecha de la última separación del segmento
 // de memoria
 time_t shm_ctime; // Fecha del último cambio en el segmento de memoria
};
```

## shmget

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmget (key_t key, int size, int shmflg);
```

Con `shmget` podemos crear una zona de memoria compartida o acceder a una ya existente.

El parámetro `key` es una llave que identifica a la zona de memoria, esta llave puede crearse con una llamada a `ftok`; si `key` toma el valor `IPC_PRIVATE`, la llamada crea un

nuevo identificador, sujeto a la disponibilidad de entradas libres en la tabla que gestiona el núcleo para las zonas de memoria.

El parámetro `size` es el tamaño en bytes de la zona de memoria que queremos crear.

El parámetro `shmflg` es una máscara de bits que indican el modo de adquisición del identificador. Si el bit `IPC_CREAT` está activo, la facilidad *IPC* se creará, en el supuesto de que otro proceso no la haya creado ya. Si los bits `IPC_CREAT` e `IPC_EXCL` están activos simultáneamente, la llamada falla en el caso de que la zona de memoria ya esté creada. Los 9 bits menos significativos de `shmflg` indican los permisos de la zona de memoria.

Si la llamada funciona correctamente, devolverá el identificador —número entero no negativo— asociado a la zona de memoria; este identificador es heredado por los procesos descendientes del actual.

## shmop

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
char *shmat (int shmid, char *shmaddr, int shmflg);
int shmdt (char *shmaddr);
```

La llamada `shmat` une la zona de memoria compartida identificada por `shmid` al espacio de direcciones virtuales que se inicia en `shmaddr`. Si `shmaddr` vale `NULL`, el núcleo es el encargado de elegir la dirección virtual en la que se inicia el segmento de memoria unido.

El parámetro `shmflg` es una máscara de bits que indica la forma de acceso a la memoria. Si el bit `SHM_RDONLY` está activo, la memoria será accesible para leer, pero no para escribir.

La llamada `shmdt` realiza la operación inversa a `shmat`, con lo que el segmento de memoria compartida dejará de ser accesible.

Si las llamadas funcionan correctamente, `shmat` devuelve la dirección a la que está unido el segmento de memoria compartida y `shmdt` devuelve 0. Si algo falla, ambas llamadas devuelven -1.

## shutdown

```
#include <sys/socket.h>
int shutdown (int sfd, int how);
```

Esta llamada se utiliza para cerrar total o parcialmente el conector con descriptor `sfd`. El argumento `how` indica el tipo de acción que llevará a cabo y puede tomar los valores:

`SHUT_RD` Deshabilita la recepción de datos del conector.

`SHUT_WR` Deshabilita el envío de datos a través del conector.

`SHUT_RDWR` Deshabilitar el envío y la recepción de datos. Es equivalente a cerrar el conector con una llamada a `close`.

## sigblock

```
#include <signal.h>
long sigblock (long mask);
```

La llamada **sigblock** permite añadir a la máscara de señales actual aquellas señales que se especifiquen con el parámetro **mask**. Así, con **sigblock** podemos modificar la máscara fijada por **sigsetmask**.

## signal

```
#include <signal.h>
void (*signal (int sig, void (*action) ())) ();
```

La llamada **signal** se emplea para especificar qué tratamiento debe realizar un proceso al recibir una señal, **sig** es el número de la señal sobre la que queremos especificar la forma de tratamiento y **action** es la acción que queremos que se inicie cuando se reciba la señal; **action** puede tomar tres tipos de valores:

- SIG\_DFL** Indica que la acción a realizar, cuando se recibe la señal, es la acción por defecto asociada a la señal. Por lo general, esta acción es terminar el proceso y, en algunos casos, también incluye generar un fichero **core**.
- SIG\_IGN** Indica que la señal se debe ignorar.
- dirección** Es la dirección de la rutina de tratamiento de la señal. La declaración de esta función debe ajustarse al siguiente modelo:

```
#include <signal.h>
void handler (int sig [, int code, struct sigcontext *scp]);
```

Cuando se recibe la señal **sig**, el núcleo es quien se encarga de invocar la rutina **handler** pasándole los parámetros **sig**, **code** y **scp**. El parámetro **sig** es el número de la señal, **code** es una palabra que contiene información sobre el estado del hardware en el momento de invocar a **handler** y **scp** contiene información de contexto definida en **<signal.h>**. Tanto **code** como **scp** son parámetros opcionales y dependientes de la arquitectura de nuestra máquina.

## sigpause

```
#include <signal.h>
long sigpause (long mask);
```

Bloquea la recepción de señales de acuerdo con el valor de **mask**, de la misma forma que hace **setsigmask**. A continuación se pone a esperar la llegada de alguna de las señales

no bloqueadas. Si no queremos que **sigpause** bloquee ninguna señal, le pasaremos la máscara 0L.

Cuando **sigpause** termina su ejecución, restaura la máscara de señales que había antes de llamarla. La ejecución de **sigpause** termina cuando es interrumpida por una señal. Después de tratar la señal, **sigpause** hace que **errno** tome el valor **EINTR** y devuelve el valor -1.

## sigsetmask

```
#include <signal.h>
long sigsetmask (long mask);
```

Fija la máscara de señales actual. Esta máscara indica qué señales tienen bloqueada su recepción. La señal número *i* está bloqueada si el *i-ésimo* bit de **mask** vale 1. Este bit lo podemos fijar con la macro **sigmask(i)**. Naturalmente, las señales que no pueden ser ignoradas ni capturadas tampoco podrán ser bloqueadas.

## sigvec

```
#include <signal.h>
sigvec (int sig, struct sigvec *vec, struct sigvec *ovec);
```

Especifica la forma de tratar la señal número **sig**. Tanto **vec** como **ovec** son punteros a estructuras del tipo **sigvec**. Esta estructura está definida con los siguientes campos:

```
struct sigvec {
 void (*sv_handler) ();
 long sv_mask;
 long sv_flags;
};
```

El campo **sv\_handler** es un puntero a una función que devuelve **void** y tiene el mismo significado que el parámetro **action** de la llamada **signal**. Este campo se utiliza para indicar cuál será la rutina de tratamiento de la señal. Al igual que en el caso de **signal**, puede tomar tres tipos de valores con diferente significado:

**SIG\_DFL** La rutina de tratamiento es la rutina por defecto.

**SIG\_IGN** La señal será ignorada.

**dirección** La rutina de tratamiento empieza en la dirección indicada en **dirección** y ha sido codificada por el usuario.

El campo **sv\_mask** codifica, en cada uno de sus bits, las señales que no deseamos que sean tratadas si son recibidas mientras se está ejecutando la rutina de tratamiento actual. Normalmente, este campo vale 0, lo que indica que mientras se está tratando una señal,

cualquier otra señal puede interrumpir. Si alguno de los bits de `sv_mask` vale 1, vamos a impedir el anidamiento cuando se recibe esa señal.

El campo `sv_flags` codifica cuál será la semántica que se emplee en la recepción de la señal. Los siguientes bits están definidos para este campo:

- SV\_BSDSIG** Usar la semántica de Berkeley. Esto significa, entre otras cosas, que cuando se instala una rutina de tratamiento, permanecerá instalada hasta que se haga otra llamada a `sigvec` que instale una rutina nueva.
- SV\_RESETHAND** Impone la misma semántica que la llamada `signal` del UNIX System V. Es decir, siempre se restaura la rutina de tratamiento por defecto antes de entrar en la rutina de tratamiento suministrada por el usuario.

El parámetro `vec` apunta al nuevo vector de señal y `ovec` devuelve un puntero al vector que había, por si deseamos restaurarlo posteriormente.

## socket

```
#include <sys/types.h>
#include <sys/socket.h>
int socket (int af, int type, int protocol);
```

La llamada crea un punto terminal para conectarse a un canal y devuelve un descriptor. El descriptor del conector devuelto se usará en llamadas posteriores a funciones de la interfaz. El parámetro `af` —*address family*— especifica la familia de conectores o familia de direcciones que se desea emplear. Las familias `AF_INET` y `AF_INET6` suelen estar presentes en todos los sistemas.

El parámetro `type` indica la semántica de la comunicación para el conector y puede tomar los valores:

- SOCK\_STREAM** Conector con un protocolo orientado a conexión.
- SOCK\_DGRAM** Conector con un protocolo no orientado a conexión o *datagrama*.
- SOCK\_RAW** Conector que proporciona acceso directo a los protocolos internos de la red.
- SOCK\_SEQPACKET** Conector con un protocolo no orientado a conexión pero que proporciona un envío fiable y secuencial para *datagramas* con una longitud máxima fija.
- SOCK\_RDM** Conector con un protocolo no orientado a conexión y envío fiable y secuencial de paquetes.

El parámetro `protocol` especifica el protocolo particular que se va a usar con el conector. Normalmente, cada tipo de conector tiene asociado sólo un protocolo pero, si hubiera más de uno, se especificaría mediante este argumento. Si `protocol` toma el valor 0 la elección del protocolo se deja en manos del sistema.



## stat, fstat

```
#include <sys/types.h>
#include <sys/stat.h>
int stat (char *path, struct stat *buf);
int fstat (int fildes, struct stat *buf);
```

La llamada `stat` devuelve, a través de `buf`, información sobre el estado del fichero cuya ruta es `path`; `fstat` hace lo mismo con el fichero descrito por `fildes`. La estructura de `buf` es la siguiente:

```
struct stat {
 dev_t st_dev; // Número de dispositivo del disco donde se encuentra el
 // fichero
 ino_t st_ino; // Nodo-i del fichero
 ushort st_mode; // 16 bits que codifican el modo del fichero,
 // consúltese chmod
 ushort st_nlink; // Número de enlaces al fichero
 uid_t st_uid; // UID del propietario del fichero
 gid_t st_gid; // GID del propietario del fichero
 dev_t st_rdev; // Major y minor number. Tiene significado únicamente
 // para los ficheros especiales en modo carácter y en
 // modo bloque
 off_t st_size; // Tamaño, en bytes, del fichero
 time_t st_atime; // Fecha del último acceso al fichero (lectura)
 time_t st_mtime; // Fecha de la última modificación del fichero
 time_t st_ctime; // Fecha del último cambio de la información
 // administrativa del fichero
};
```

## stime

```
#include <sys/time.h>
int stime (long *tp);
```

Fija la hora y fecha del sistema de acuerdo con el valor apuntado por `tp`. La fecha y hora se especifican en segundos con respecto a la *Época*.

## sync

```
#include <unistd.h>
void sync (void);
```

Sincroniza la memoria intermedia y el disco, forzando así la consistencia de los datos del sistema.

## time

```
#include <time.h>
time_t time (time_t *tloc);
```

Devuelve el número de segundos transcurridos desde la *Época*. Si `tloc` es distinto de `NULL`, contendrá el mismo valor que devuelve la llamada.

## times

```
#include <sys/times.h>
clock_t times (struct tms *buffer);
```

Rellena la estructura apuntada por `buffer` con la información estadística relativa a los tiempos de ejecución empleados por el proceso desde su inicio hasta el momento de invocar a `times`. La estructura `tms` se define de la siguiente forma:

```
struct tms {
 // Tiempo de CPU en modo usuario
 clock_t tms_utime;
 // Tiempo de CPU en modo supervisor
 clock_t tms_stime;
 // Tiempo de CPU en modo usuario de los procesos hijo
 clock_t tms_cutime;
 // Tiempo de CPU en modo supervisor de los procesos hijo
 clock_t tms_cstime;
};
```

El tipo `clock_t` se define para contabilizar los pulsos de reloj. Cada segundo se compone de un total de `CLK_TCK` pulsos. El valor de `CLK_TCK` depende de la implementación del sistema y se puede leer con la llamada `sysconf`.

## ulimit

```
#include <ulimit.h>
long ulimit (int cmd [, ...]);
```

Los procesos tienen limitado el tamaño máximo de los ficheros que pueden crear y el tamaño máximo de memoria que pueden tener asignada. Para consultar o fijar estos parámetros se emplea la llamada `ulimit`.

El parámetro `cmd` le indica a `ulimit` el tipo de operación a realizar. Sus posibles valores son:

**UL\_GETFSIZE** `ulimit` devuelve el tamaño máximo —expresado en bloques de 512 bytes— de los ficheros que puede escribir el proceso. De cara a la lectura de ficheros no hay impuesto ningún límite.

**UL\_SETFSIZE** **ulimit** fija el tamaño máximo de los ficheros que el proceso puede escribir. El tamaño se indica a través del segundo parámetro de la función, este parámetro es de tipo **long**. El tamaño se expresa en unidades de bloques de 512 bytes.

**UL\_GETMAXBRK** **ulimit** devuelve el tamaño máximo de memoria que puede ocupar el proceso —consúltese **brk(2)**—.

## umask

```
#include <sys/types.h>
#include <sys/stat.h>
mode_t umask (mode_t cmask);
```

Fija la máscara de creación de ficheros al valor **cmask** y devuelve su antiguo valor. Cuando se cree un fichero, los bits que **cmask** tenga activos aparecerán inactivos en la máscara de permisos.

## umount

```
int umount (char *name);
```

Desmonta el sistema de ficheros que se encuentra montado sobre el fichero de dispositivo indicado en **name**.

## uname

```
#include <sys/utsname.h>
int uname (struct uname *name);
```

Devuelve información relativa al sistema de acuerdo con la siguiente estructura:

```
struct utsname {
 char sysname [9]; // Nombre del sistema
 char nodename [9]; // Nombre del nodo
 char release [9]; // Edición del sistema
 char version [9]; // Versión del sistema
 char machine [9]; // Tipo de máquina en la que se ejecuta el sistema
};
```

## unlink

```
#include <unistd.h>
int unlink (char *path);
```

Borra la entrada de directorio especificada en la ruta apuntada por `path`.

## ustat

```
#include <sys/types.h>
#include <ustat.h>
int ustat (dev_t dev, struct ustat *buf);
```

Devuelve, en la estructura apuntada por `buf`, información estadística sobre el sistema de ficheros que hay construido en el fichero de dispositivo `dev`. Este parámetro codifica el *major* y *minor number* del fichero. La estructura `ustat` se define como sigue:

```
struct ustat {
 daddr_t f_tfree; // Número de bloques libres
 ino_t f_tinode; // Número de nodos-i libres
 char f_fname [6]; // Nombre del sistema de ficheros
 char f_fpack [6]; // Nombre del paquete del sistema de ficheros
};
```

## utime

```
#include <sys/types.h>
#include <utime.h>
int utime (char *path, struct utimbuf *times);
```

Fija las fechas de último acceso y última modificación del fichero referenciado en `path` de acuerdo con los campos de la estructura apuntada por `times`. Si `times` vale `NULL`, se utilizan la fecha y hora actuales. La estructura `utimbuf` se define con los siguientes campos:

```
struct utimbuf {
 time_t actime; // Fecha de acceso
 time_t modtime; // Fecha de modificación
};
```

## wait

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait (int *stat_loc);
```

Suspende la ejecución del proceso actual hasta que alguno de sus procesos hijo termine por una llamada a `exit` o suspenda su ejecución por recibir una señal cuando está siendo depurado. Si `stat_loc` es distinto de `NULL`, cuando `wait` devuelva el control la dirección a la que apunta contendrá el estado de salida del proceso hijo. Sólo los 16 bits menos significativos aportan información.

La llamada `wait` devuelve el *PID* del proceso hijo que ha terminado o que se ha parado porque está siendo depurado. Si el proceso hijo termina porque ha recibido una señal, `wait` devolverá el valor `-1` y `errno` valdrá `EINTR`.

## write, writev

```
#include <unistd.h>
int write (int fildes, char *buf, unsigned nbyte);

#include <sys/uio.h>
ssize_t writev (int fildes, const struct iovec *iov, size_t iovcnt);
```

La llamada `write` escribe, en el fichero descrito por `fildes`, `nbyte` bytes que se encuentran en la zona de memoria que empieza en la dirección `buf`. Si la llamada se ejecuta satisfactoriamente, `write` devuelve el número de bytes realmente escritos; en caso contrario, devuelve el valor `-1`.

La llamada `writev` es una generalización de `write` y puede utilizarse para escribir datos en cualquier descriptor, ya sea fichero o conector. Esta llamada escribe datos en el fichero descrito por `fildes` y toma esos datos de los distintos bloques de memoria especificados por el array `iov`. El significado del array `iov` y del argumento `iovcnt` es el mismo que se ha explicado en la llamada `readv`.

# Referencias

- [Arnold *et al.*, 2000] ARNOLD, KEN, GOSLING, JAMES, & HOLMES, DAVID. 2000. *The Java Programming Language*. 3rd edn. Reading, MA, USA: Addison-Wesley.
- [Bach, 1986] BACH, MAURICE J. 1986. *The Design of the UNIX Operating System*. Englewood Cliffs, NJ 07632, USA: Prentice-Hall Inc.
- [Ben-Ari, 1990] BEN-ARI, M. 1990. *Principles of Concurrent and Distributed Programming*. Prentice Hall International (UK) Ltd.
- [Black, 1990] BLACK, DAVID L. 1990. Scheduling Support for Concurrency and Parallelism in the Mach Operating System. *IEEE Computer*, **23**(5), 35–43.
- [Burks *et al.*, 1946] BURKS, A. W., GOLDSTINE, H. H., & VON NEUMANN, J. 1946. *Preliminary discussion of the logical design of an electronic computing instrument*. Report to the U.S. Army Ordnance Department. Lo podemos encontrar reproducido en [Taub, 1963].
- [Burns & Wellings, 2001] BURNS, ALAN, & WELLINGS, ANDREW J. 2001. *Real-time systems and programming languages: Ada 95, real-time Java, and real-time POSIX*. Tercera edn. International computer science series. Reading, MA, USA: Addison-Wesley. Existe traducción al español.
- [Butenhof, 1997] BUTENHOF, DAVID R. 1997. *Programming with POSIX Threads*. Reading, MA, USA: Addison-Wesley.
- [Ceballos, 2001] CEBALLOS, FRANCISCO JAVIER. 2001. *C/C++ Curso de programación*. Segunda edn. Madrid: Ra-Ma.
- [Cohen, 1981] COHEN, DANNY. 1981. On Holy Wars and a plea for peace. *Computer*, **14**(10), 48–54.
- [Comer, 2000] COMER, DOUGLAS E. 2000. *Internetworking with TCP/IP Vol.1: Principles, Protocols, and Architecture*. 4th edn. Prentice Hall.
- [Comer, 2001] COMER, DOUGLAS E. 2001. *Internetworking with TCP/IP, Vol. III: Client-Server Programming and Applications, Linux/Posix Sockets Version*. Prentice Hall.

- [Comer & Stevens, 1999] COMER, DOUGLAS E., & STEVENS, DAVID L. 1999. *Internet-working with TCP/IP Vol. II: ANSI C Version: Design, Implementation, and Internals*. 3rd edn. Prentice Hall.
- [Cooper & Draves, 1990] COOPER, ERIC C., & DRAVES, RICHARD P. 1990. *C threads*. Tech. rept. CMU-CS-88-154. School of Computer Science, Carnegie Mellon University, Pittsburgh, PA.
- [Corominas & Pascual, 1996] COROMINAS, JOAN, & PASCUAL, JOSÉ A. 1996. *Diccionario Crítico Etimológico Castellano e Hispánico*. Tercera reimpresión edn. Madrid: Ed. Gredos, S.A.
- [Covarrubias, 1611] COVARRUBIAS, SEBASTIÁN DE. 1611. *Tesoro de la Lengua Española*. Facsímil edn. Se ha empleado el facsímil publicado en 1987 por Ed. Alta Falla de la edición de Martín de Riquer impresa por el barcelonés Joaquín Hurta en 1943. El texto de 1943 no es una edición facsímil de la princeps de 1611 sino un texto preparado por Martín de Riquer que contiene además las adiciones de Benito Remigio Noydens publicadas en 1674 y un índice de voces, expresiones y nombres propios.
- [Dijkstra, 1968a] DIJKSTRA, EDSGER W. 1968a. Cooperating sequential processes. *Pages 43–112 of*: GENUYS, F. (ed), *Programming Languages: NATO Advanced Study Institute*. Academic Press. El documento consultado ha sido realmente el manuscrito mecanografiado disponible en [Dijkstra, 1968b].
- [Dijkstra, 1968b] DIJKSTRA, EDSGER W. 1968b. *Cooperating sequential processes*. Publicado en [Dijkstra, 1968a], se encuentra disponible en formato electrónico en [www.cs.utexas.edu/users/EWD](http://www.cs.utexas.edu/users/EWD).
- [Hoare, 1961] HOARE, C. A. R. 1961. ACM Algorithm 64: Quicksort. *Communications of the ACM*, 4(7), 321.
- [Hoare, 1974] HOARE, C. A. R. 1974. Monitors: An Operating System Structuring Concept. *Communications of the ACM*, 17(10), 549–557.
- [IEEE, 1995] IEEE. 1995. *IEEE 1003.1c-1995: Information Technology – Portable Operating System Interface (POSIX) – System Application Program Interface (API) Amendment 2: Threads Extension (C Language)*. 1109 Spring Street, Suite 300, Silver Spring, MD 20910, USA: IEEE Computer Society Press.
- [IEEE, 2001a] IEEE. 2001a. *1003.1<sup>TM</sup> Standard for Information Technology – Portable Operating System Interface (POSIX<sup>®</sup>) – Base Definitions, Issue 6*. 3 Park Avenue, New York, NY 10016-5997, USA: Institute of Electrical and Electronics Engineers, Inc.
- [IEEE, 2001b] IEEE. 2001b. *1003.1<sup>TM</sup> Standard for Information Technology – Portable Operating System Interface (POSIX<sup>®</sup>) – System Interfaces, Issue 6*. 3 Park Avenue, New York, NY 10016-5997, USA: Institute of Electrical and Electronics Engineers, Inc.
- [ISO, 1984] ISO. 1984. *Information Processing Systems – Open Systems Interconnection – Basic Reference Model*. Tech. rept. International Organization for Standardization and International Electrotechnical Committee. Estándar internacional 7498-1, reemplazado por [ISO/IEC, 1994].

- [ISO, 1999] ISO. 1999. *ISO/IEC 9899:1999: Programming Languages – C*. Ginebra, Suiza: International Organization for Standardization. Se puede encontrar en formato electrónico en [anubis.dkuug.dk/JTC1/SC22/open/n2620/n2620.pdf](http://anubis.dkuug.dk/JTC1/SC22/open/n2620/n2620.pdf) y en [anubis.dkuug.dk/JTC1/SC22/WG14/www/docs/n897.pdf](http://anubis.dkuug.dk/JTC1/SC22/WG14/www/docs/n897.pdf).
- [ISO/IEC, 1994] ISO/IEC. 1994. *Information technology – Open Systems Interconnection – Basic Reference Model: The Basic Model*. Tech. rept. International Organization for Standardization and International Electrotechnical Committee. Estándar Internacional 7498-1. Reemplaza a la primera edición [ISO, 1984].
- [Joy et al., 1993] JOY, WILLIAN N., FABRY, ROBERT S., LEFFLER, SAMUEL J., McKUSICK, MARSHALL K., & KARELS, MICHAEL. 1993. *Berkeley Software Architecture Manual 4.4BSD Edition*. Tech. rept. Computer System Research Group, Computer Science Division, Department of Electrical Engineering and Computer Science, University of California, Berkeley, CA 94720. Disponible en formato electrónico en [docs.freebsd.org/44doc](http://docs.freebsd.org/44doc).
- [Kernighan & Pike, 1984] KERNIGHAN, BRIAN W., & PIKE, ROB. 1984. *The UNIX Programming Environment*. Upper Saddle River, NJ 07458, USA: Prentice-Hall. Existe traducción al español.
- [Kernighan & Pike, 1987] KERNIGHAN, BRIAN W., & PIKE, ROB. 1987. *El entorno de programación UNIX*. México: Prentice-Hall Hispanoamericana, S.A. Traducción al español de [Kernighan & Pike, 1984].
- [Kernighan & Pike, 1999] KERNIGHAN, BRIAN W., & PIKE, ROB. 1999. *The Practice of Programming*. Reading, MA, USA: Addison-Wesley. Existe traducción al español.
- [Kernighan & Pike, 2000] KERNIGHAN, BRIAN W., & PIKE, ROB. 2000. *La Práctica de la Programación*. Primera edn. Pearson Educación, México. Versión en español de [Kernighan & Pike, 1999].
- [Kernighan & Ritchie, 1978] KERNIGHAN, BRIAN W., & RITCHIE, DENNIS M. 1978. *The C Programming Language*. Prentice-Hall, Inc.
- [Kernighan & Ritchie, 1988] KERNIGHAN, BRIAN W., & RITCHIE, DENNIS M. 1988. *The C Programming Language*. 2nd edn. Prentice-Hall. La primera edición de este libro constituye la norma de facto que se ha venido aplicando para programar en C. El estilo de programación que define está hoy muy difundido entre otros autores que desarrollan software en C y en UNIX. La segunda edición aparece como consecuencia de la normalización del lenguaje por parte del *American National Standards Institute* y se ajusta a la definición del ANSI C. Aun a pesar de que la última palabra sobre el lenguaje la tiene la norma, el manual de Kernighan sigue siendo una obra de primera línea sobre la programación en lenguaje C. Existe traducción al español.
- [Khanna et al., 1991] KHANNA, SANDEEP, SEBR'EE, MICHAEL, & ZOLNOWSKY, JOHN. 1991. Realtime Scheduling in SunOS 5.0. *Pages 375–390 of: Proceedings of the Usenix Winter 1992 Technical Conference*. Berkeley, CA, USA: Usenix Association.



- [Knuth, 1973] KNUTH, DONALD E. 1973. *The Art of Computer Programming, Vol.3 – Sorting and Searching*. Reading, MA, USA: Addison-Wesley.
- [Leffler et al., 1989] LEFFLER, S. J., MCKUSICK, M. K., KARELS, M. J., & QUARTERMAN, J. S. 1989. *The Design and Implementation of the 4.3BSD Unix Operating System*. Reading, MA, USA: Addison-Wesley.
- [Leffler et al., 1993] LEFFLER, SAMUEL J., FABRY, ROBERT S., JOY, WILLIAM N., LAPSLEY, PHIL, MILLER, STEVE, & TOREK, CHRIS. 1993. *An Advanced 4.4BSD Interprocess Communication Tutorial*. Tech. rept. Computer System Research Group, Department of Electrical Engineering and Computer Science, University of California, Berkeley, California 94720. Disponible en formato electrónico en [docs.freebsd.org/44doc](http://docs.freebsd.org/44doc).
- [Lehoczký et al., 1989] LEHOCZKY, JOHN P., SHA, LUI, & DING, YE. 1989. The Rate Monotonic Scheduling Algorithm: Exact Characterization and Average Case Behavior. *Pages 166–171 of: PRESS, IEEE COMPUTER SOCIETY (ed), Proceedings of the 10th Real-Time Systems Symposium*. Santa Monica, California, USA: IEEE Computer Society Press.
- [Leung & Whitehead, 1982] LEUNG, J. Y. T., & WHITEHEAD, J. 1982. On the complexity of fixed-priority scheduling of periodic, real-time tasks. *Performance Evaluation North Holland*, **2**(4), 237–250.
- [Lewine, 1994] LEWINE, DONALD A. 1994. *POSIX programmer's guide: writing portable UNIX programs*. 103 Morris Street, Suite A, Sebastopol, CA 95472, USA: O'Reilly & Associates, Inc.
- [Liu & Layland, 1973] LIU, C. L., & LAYLAND, JAMES W. 1973. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *Journal of the ACM*, **20**(1), 46–61.
- [Lázaro-Carreter, 1997] LÁZARO-CARRETER, FERNANDO. 1997. *El dardo en la palabra*. Barcelona: Galaxia Gutenberg – Círculo de Lectores.
- [McKusick et al., 1996] MCKUSICK, MARSHALL KIRK, BOSTIC, KEITH, KARELS, MICHAEL J., & QUARTERMAN, JOHN S. 1996. *The Design and Implementation of the 4.4BSD Operating System*. Reading, MA, USA: Addison-Wesley.
- [Mueller, 1993] MUELLER, FRANK. 1993. A library implementation of POSIX threads under Unix. *Pages 29–41 of: Proceedings of the Winter 1993 USENIX Technical Conference and Exhibition*.
- [Murata, 1989] MURATA, TADAO. 1989. Petri nets: properties, analysis, and applications. *Proceedings of the IEEE*, **77**(4), 541–580. Disponible en formato electrónico en [www.cs.uic.edu/~murata](http://www.cs.uic.edu/~murata).
- [Netzer & Miller, 1992] NETZER, ROBERT H. B., & MILLER, BARTON P. 1992. What Are Race Conditions?: Some Issues and Formalizations. *ACM Letters on Programming Languages and Systems*, **1**(1), 74–88.

- [OSF, 1994] OSF, OPEN SOFTWARE FOUNDATION. 1994. *OSF/1 Programmer's Reference*. Englewood Cliffs, NJ: Prentice Hall.
- [Ossanna & Kernighan, 2000] OSSANNA, JOSEPH F., & KERNIGHAN, BRIAN W. 2000. *Troff User's Manual*. Tech. rept. Computing Sciences Research Center, Bell Laboratories, Murray Hill, NJ, USA. Éste es el manual de referencia original de `troff` revisado en varias ocasiones por B. W. Kernighan. Se puede obtener en formato electrónico en [netlib.bell-labs.com/cm/cs/cstr](http://netlib.bell-labs.com/cm/cs/cstr) y en [plan9.bell-labs.com/sys/doc](http://plan9.bell-labs.com/sys/doc).
- [Powell *et al.*, 1991] POWELL, M. L., KLEIMAN, STEVE R., BARTON, STEVE, SHAH, DEVANG, STEIN, DAN, & WEEKS, MARY. 1991. SunOS Multi-thread Architecture. *Pages 65–80 of: Proceedings of the Winter 1991 USENIX Technical Conference and Exhibition*.
- [RACEFyN, 1996] RACEFyN. 1996. *Vocabulario Científico y Técnico*. Madrid: Real Academia de Ciencias Exactas, Físicas y Naturales.
- [RAE, 2001] RAE. 2001. *Diccionario de la Lengua Española*. Vigésima segunda edn. Madrid: Real Academia Española. Puede ser consultado en la dirección [www.rae.es](http://www.rae.es).
- [Raymond, 2004] RAYMOND, ERIC STEVEN. 2004. *The Art of UNIX Programming*. Reading, MA, USA: Addison-Wesley. En [www.catb.org/~esr/writings/taoup](http://www.catb.org/~esr/writings/taoup) podemos encontrar este libro en formato electrónico.
- [Ritchie, 1993] RITCHIE, DENNIS M. 1993. The development of the C language. *ACM SIGPLAN Notices*, **28**(3), 201–208.
- [Ritchie & Thompson, 1974] RITCHIE, DENNIS M., & THOMPSON, KEN. 1974. The UNIX Time-Sharing System. *Communications of the ACM*, **17**(7).
- [Rochkind, 1985] ROCHKIND, MARCK J. 1985. *Advanced UNIX Programming*. Englewood Cliffs, NJ, USA: Prentice-Hall Inc.
- [Sha & Lehoczky, 1986] SHA, LUI, & LEHOCZKY, JOHN P. 1986. Performance of real-time bus scheduling algorithms. *ACM Performance Evaluation Review*, **14**(1).
- [Sha *et al.*, 1990] SHA, LUI, RAJKUMAR, RAGUNATHAN, & LEHOCZKY, JOHN P. 1990. Priority Inheritance Protocols: An Approach to Real-Time Synchronization. *IEEE Transactions on Computers*, **39**(9), 1175–1185.
- [Silberschatz *et al.*, 2001] SILBERSCHATZ, ABRAHAM, GALVIN, PETER BAER, & GAGNE, GREG. 2001. *Operating System Concepts*. 6th edn. Reading, MA, USA: Addison-Wesley.
- [Silva, 1985] SILVA, MANUEL. 1985. *Las Redes de Petri: en la Automática y la Informática*. Madrid: Editoriar AC.
- [Sánchez, 2001] SÁNCHEZ, SEBASTIÁN. 2001. *UNIX y Linux: Guía práctica*. Segunda edn. Madrid: Ra-Ma.

- [Stein & Shah, 1992] STEIN, DAN, & SHAH, DEVANG. 1992. Implementing Lightweight Threads. *Pages 1–10 of: Proceedings of the Summer 1992 USENIX Technical Conference and Exhibition*. San Antonio, TX: USENIX.
- [Stevens, 1992] STEVENS, RICHARD. 1992. *Advanced Programming in the UNIX Environment*. Reading, MA, USA: Addison-Wesley.
- [Stevens, 1994] STEVENS, W. RICHARD. 1994. *TCP/IP Illustrated, Volume 1: The Protocols*. Reading, MA, USA: Addison Wesley Professional.
- [Stevens, 1996] STEVENS, W. RICHARD. 1996. *TCP/IP Illustrated, Volume 3: TCP for Transactions, HTTP, NNTP, and the UNIX® Domain Protocols*. Reading, MA, USA: Addison Wesley Professional.
- [Tanenbaum, 2001] TANENBAUM, ANDREW S. 2001. *Modern Operating Systems*. 2nd edn. Englewood Cliff, NJ: Prentice-Hall International, Inc.
- [Tanenbaum, 2002] TANENBAUM, ANDREW S. 2002. *Computer Networks*. 4th edn. Englewood Cliff, NJ: Prentice-Hall International, Inc.
- [Taub, 1963] TAUB, A. H. (ed). 1963. *John von Neumann Collected Works*. Vol. V, Design of Computers, Theory of Automata and Numerical Analysis. Oxford: Pergamon.
- [Vahalia, 1996] VAHALIA, URESH. 1996. *UNIX Internals. The New Frontier*. Upper Saddle River, NJ 07458, USA: Prentice-Hall.
- [Webster, 1986] WEBSTER. 1986. *Webster's Third New International Dictionary of the English Language*. Springfield, MA, USA: Merriam-Webster.
- [Wright & Stevens, 1995] WRIGHT, GARY R., & STEVENS, W. RICHARD. 1995. *TCP/IP Illustrated, Volume 2: The Implementation*. Reading, MA, USA: Addison Wesley Professional.

# Índice alfabético

## — A —

a.out, 2, 160  
abort, 242  
abrazo mortal, 89  
accept, 435  
acceso atómico, 25  
access, 81  
ACCOUNTING, 130  
acct, 316  
acctcom, 316  
accton, 316  
adb, 240, 322  
*address family*, 431  
AF\_APPLETALK, 433  
AF\_CCITT, 433  
AF\_CHAOS, 433  
AF\_CNT, 433  
AF\_COIP, 433  
AF\_DATAKIT, 433  
AF\_DECnet, 433  
AF\_DLI, 433  
AF\_ECMA, 433  
AF\_HYLINK, 433  
AF\_IMPLINK, 433  
AF\_INET, 432  
AF\_IPX, 433  
AF\_ISO, 433  
AF\_LAT, 433  
AF\_LINK, 433  
AF\_LOCAL, 433  
AF\_NS, 433  
AF\_PUP, 433  
AF\_ROUTE, 433  
AF\_SNA, 433  
AF\_UNIX, 432  
AF\_XTP, 433  
alarm, 288  
alias, 496

almacenamiento  
    automático, 504  
    estático, 504  
análisis descendente, 509  
antememoria, 27  
ar, 519  
área de usuario, 164–166, 324  
argc, 184  
argv, 184  
array, 492  
as, 512  
*ASCII*, 425  
asíncrono, 239  
at, 291  
atexit, 191  
atof, 305  
atributos  
    de un cerrojo, 402  
    de un hilo, 213  
autómata no finito, 418  
automatización de trabajos, 526

## — B —

*backup*, 14  
bad144, 36  
badblk, 36  
badsect, 36  
badtrk, 36  
biblioteca, 512, 519  
    estándar de entrada/salida, 59  
    índice de símbolos, 521  
    miembro de una, 519  
*big endian*, 456  
bind, 433  
bit  
    de traza, 323  
    pertinaz, 80  
*block*

- device*, 3
  - switch table*, 28
  - started by symbol*, 160
- bloque, 499
  - de arranque, 15
  - de datos, 15
  - de dirección , 17
  - de directorio, 25
- bloqueo
  - consultivo, 87
  - de un semáforo, 397
  - obligatorio, 87
- boot*, 14, 15
- BOOT\_TIME, 130
- brk, 201, 202
- BSIZE, 38
- bss*, 160
- bucle
  - do-while, 502
  - for, 502
  - while, 502
- buffer*, 4
- buffer caché*, 27
- BUFSIZ, 53
- BUS\_ADRALN, 266
- BUS\_ADRERR, 266
- BUS\_OBJERR, 266
- byteorder, 456
- C —
- C-threads*, 170
- cabecera, 160
  - de programa, 160
  - de sección, 185
  - principal, 185
- cadena de caracteres, 489, 492
- cambiar en ejecución el identificador
  - de grupo, 40
  - de usuario, 40
- cambio
  - de contexto, 5, 166
  - de contraseña, 79
- campo de bits, 495
- canal bidireccional, 337
- cancelación
  - asíncrona, 221
  - diferida, 221
- capa
  - de aplicación, 425
  - de contexto, 167
  - de enlace, 424
  - de presentación, 425
  - de red, 425
  - de sesión, 425
  - de transporte, 425
  - física, 424
- capturador de señal, 264
- CASE, 525, 531
- catch, 270
- cc, 2, 488
- cd, 145
- ceiling priority protocol*, 180
- cerrojo, 225, 400
  - atributos, 402
  - de escritura, 73
  - de lectura, 73
  - no bloqueante, 88
  - tipo de atributos, 403
- char, 490
- character device switch table*, 28
- chdir, 121
- chgrp, 114
- chmod, 80, 111
- chown, 82, 114
- chroot, 121
- chunk*, 25
- cinta, 14
- CLD\_CONTINUED, 266
- CLD\_DUMPED, 266
- CLD\_EXITED, 266
- CLD\_KILLED, 266
- CLD\_STOPPED, 266
- CLD\_TRAPPED, 266
- cliente, 428
- CLK\_TCK, 285
- clock\_getres, 294
- clock\_gettime, 294
- CLOCK\_MONOTONIC, 294
- clock\_nanosleep, 295
- CLOCK\_PROF, 294
- CLOCK\_REALTIME, 294
- clock\_settime, 294
- clock\_t, 285
- CLOCK\_VIRTUAL, 294
- clockid\_t, 294
- close, 57
- close-on-exec*, 73
- closedir, 123
- código
  - fuelle ensamblador, 512

- objeto, 512
- cola de mensajes, 369, 387
- comentario, 489
- comp, 512
- compilación, 512
  - condicional, 511
- compilador
  - de lenguaje C, 2
  - opción
    - c, 512
    - de perfilado -p, 315
    - E, 512
    - o, 512
    - S, 512
- comunicación
  - asíncrona, 5
  - dúplex, 350
  - semidúplex, 350
  - síncrona, 5
- comunicación entre procesos, 333
- conurrencia, 169
- condición de carrera, 225
- conector, 334
- conjunto de señales, 268
- connect, 435
- consistencia de un fichero, 59
- consola, 129
- constante, 488
- contabilidad, 315
  - activación, 316
  - llamada acct, 316
  - orden
    - acctcom, 316
    - accton, 316
- contador de programa, 161, 166
- contenido
  - de un módulo objeto, 523
  - de una biblioteca, 523
- contexto
  - cambio de, 5, 166
  - capa de, 167
  - de registros, 166
  - de un proceso, 165
  - del nivel
    - de sistema, 166
    - de usuario, 166, 184
- contraseña, 79
  - lectura de, 155
- copia de seguridad, 14
- coproceso, 362

- core, 246
- corriente, 60
- cp, 51
- CPP, 180
- cpp, 489, 511
- crash, 14
- creat, 57
- criterio
  - DMS, 174
  - EDF, 179
  - RMS, 173
- crypt, 235
- csh, 159
- current work directory, 14
- CWD, 14

## — D —

- DARPA, 432
- Data Base Manager, 391
- data lock, 211
- datagrama, 427
- DATLOCK, 211
- dato urgente, señal de llegada de un, 253
- DBM, 391
- DEAD\_PROCESS, 130
- deadline monotonic scheduling, 174
- define, 489
- DELAY\_TIMER\_MAX, 298
- demonio, 16
- depurador, 185, 322
  - adb, sdb, gdb, 240
- desbloqueo de un semáforo, 397
- descriptor de fichero, 33, 53, 55
- despachador, 5
- despertador, 242
- detachstate, 217
- device driver, 3
- df, 43
- diagrama de transición de estados, 162
- DIR, 122
- dirección
  - del espacio virtual, 166
  - física, 166
- directorio, 14, 24, 117
  - de trabajo, 200
    - actual, 121
    - raíz, 121, 201
- directriz
  - define, 489
  - include, 489

- para el preprocesador, 489
- disco
  - ocupación, 44
  - partición, 35
    - lógica, 14
  - plato, 30
  - sincronización del, 59, 138
- dispatching*, 5
- dispositivo
  - manejador de, 3
  - modo
    - bloque, 3
    - carácter, 3
  - software, 28
- distribución de procesos, 171
- distribuidor, 5
- DMS*, 174
- documentación, 531
  - externa, 531
  - interna, 531
- dominio de una conexión, 427
- double*, 491
- du*, 44
- dup*, 58
- dúplex, comunicación, 350

## — E —

- e2fsk*, 145
- EAGAIN*, 88
- earliest deadline first*, 179
- EBCDIC*, 425
- EBUSY*, 401
- eco local de un terminal, 135
- ed*, 362
- \_edata*, 201
- edata*, 201
- EDEADLK*, 89, 213, 399
- EDF*, 179
- edición
  - páginas del manual, 532
- editor
  - ed*, 362
- EEXIST*, 344
- EGID*, 197
- EINTR*, 249, 399
- EIO*, 324
- ejecución paso a paso, 324
- EMPTY*, 130
- emulator trap instruction*, 242
- encapsulamiento, 102

- \_end*, 201
- end*, 201
- endgrent*, 113
- endhostent*, 467
- endmntent*, 149
- endpwent*, 113
- enlace, 24, 512
  - blando, 110
  - duro, 110
- entero, tipo, 490
- entorno, 198
  - de pila, 250
  - de un proceso, 184
- entrada
  - directa, 21
  - en la tabla de procesos, 166
  - indirecta
    - doble, 21
    - simple, 21
    - triple, 21
- entrada/salida asíncrona, 170
- enum*, 491
- env*, 199
- environ*, 198
- envp*, 184, 198
- EOF*, 62
- \_\_errno*, 215
- errno*, 7, 55, 215
- error
  - de violación de segmento, 242
  - de *bus*, 242
  - en coma flotante, 242
  - tolerancia a fallos, 276
- error\_fatal*, 216
- errores.h*, 216
- estado
  - de un proceso, 161
  - registro de, 166
  - zombi, 164
- estructura, 494
  - de notificación, 295
  - jerárquica de árbol invertido, 117
- \_etext*, 201
- etext*, 201
- ETIMEDOUT*, 399, 401
- EUID*, 197
- evento, 356
  - asíncrono, 239
  - notificación, 295
- excepción, 269

exception\_t, 271  
 exclusión mutua, 399, 400, 416  
 exec, 183  
 execl, 184  
 execle, 184  
 execlp, 184  
 execv, 184  
 execve, 184  
 execvp, 184  
 exit, 189  
 export, 200  
 expresión, 497  
     abreviada, 499  
     booleana, 498  
 ext2, 151  
 ext2\_super\_block, 151  
 extremista  
     mayor, 456  
     menor, 456

## — F —

F\_DUPFD, 72  
 F\_GETFD, 73  
 F\_GETFL, 73  
 F\_GETLK, 73  
 F\_LOCK, 88  
 F\_RDLCK, 73  
 F\_SETFD, 73  
 F\_SETFL, 73  
 F\_SETLK, 73  
 F\_SETLKW, 73  
 F\_TEST, 88  
 F\_TLOCK, 88  
 F\_ULOCK, 88  
 F\_UNLCK, 73  
 F\_WRLCK, 73  
 fallo de alimentación, 243  
 familia  
     de conectores, 427  
     de direcciones, 431  
     de protocolos, 431  
 fchmod, 80  
 fchown, 82  
 fclose, 62  
 fcntl, 72  
 fd\_set, 359  
 fdisk, 35  
 fflush, 148  
 fgetc, 63  
 fichero  
     .i, 511  
     /bin/sh, 184  
     /dev/console, 129  
     /dev/mem, 28  
     /dev/tty, 129  
     /dev, 129  
     /etc/boot, 39  
     /etc/checklist, 41  
     /etc/fstab, 41  
     /etc/group, 87, 113  
     /etc/hosts, 454, 466  
     /etc/inittab, 161  
     /etc/mnttab, 138, 140, 149  
     /etc/mount, 42  
     /etc/passwd, 27, 86, 113, 198, 235  
     /etc/utmp, 130  
     /usr/include/errno.h, 9  
     /usr/lib/libc.a, 3  
     /usr/spool/mail/nombre\_usuario, 234  
     /var/run/utmp, 132  
     <arpa/inet.h>, 455  
     <dirent.h>, 122  
     <errno.h>, 7, 215  
     <grp.h>, 87  
     <linux/ext2\_fs.h>, 151  
     <mntent.h>, 149  
     <mnttab.h>, 140  
     <netdb.h>, 466  
     <netinet/in.h>, 427, 455  
     <pthread.h>, 213  
     <pwd.h>, 86  
     <sched.h>, 228  
     <semaphore.h>, 398  
     <setjmp.h>, 250  
     <signal.h>, 241, 246  
     <stdio.h>, 59  
     <sys/acct.h>, 316  
     <sys/filsys.h>, 15, 140  
     <sys/filsys>, 140  
     <sys/ino.h>, 17  
     <sys/ioctl.h>, 135  
     <sys/lock.h>, 211  
     <sys/mount.h>, 152  
     <sys/netinet.h>, 433  
     <sys/param.h>, 38, 152, 288, 291  
     <sys/ptrace.h>, 324  
     <sys/socket.h>, 427, 432  
     <sys/stat.h>, 78  
     <sys/sysmacros.h>, 118  
     <sys/time.h>, 283



- <sys/times.h>, 285
- <sys/types.h>, 78
- <sys/uio.h>, 436
- <sys/un.h>, 428, 433
- <sys/user.h>, 324
- <sys/utmp.h>, 130
- <sys/utsname.h>, 440
- <sys/wait.h>, 191
- <ucontext.h>, 266
- <unistd.h>, 81, 403, 415
- <ustat.h>, 139
- <utime.h>, 82
- Makefile, 527
- bin, 510
- core, 240, 246
- doc, 510
- include, 510
- lib, 510
- lost+found, 38, 41
- makefile, 71, 527
- mon.out, 315
- src, 510
- conector, 334
- consistencia de un, 59
- de cabecera, 489, 510
  - composición, 511
- de dispositivo, 27
- descriptor de, 53
- especial, 27
- estándar
  - de entrada, 33
  - de salida, 33
  - de salida de mensajes de error, 34
- final de, 62
- ordinario, 23
- preprocesado, 511
- shell script*, 159
- FIFO*, 17, 334
- FILE, 59, 60
- \_\_fillbuf, 64
- final de fichero, 62
- finally, 270
- find, 154
- float, 491
- flock, 114
- \_\_flsbuf, 64
- flujo, 60
  - de directorio, 122
- fopen, 59
- fork, 161, 188

- format, 35
- FPE\_FLTDIV, 266
- FPE\_FLTINV, 266
- FPE\_FLOV, 266
- FPE\_FLTRES, 266
- FPE\_FLTSUB, 266
- FPE\_FLTUND, 266
- FPE\_INTDIV, 266
- FPE\_INTOV, 266
- fread, 61
- free, 202
- fruncate, 83
- fsck, 41
- fsdb, 42
- fstat, 77
- fstatfs, 151
- fsync, 59
- ftok, 371
- función, 503
  - interrumpible, 267
  - reentrante, 226
  - segura, 225
- fundamental
  - tipo, 490
- fwrite, 61

## — G —

- gdb, 240, 322, 361
- gestión de memoria, 5
- GETALL, 375
- getc, 64
- getchar, 64
- getcontext, 266
- getcwd, 121
- getegid, 197
- getenv, 199
- geteuid, 197
- getgid, 197
- getgrent, 113
- getgrent(3C), 87
- getgrgid, 87
- getgrnam, 113
- gethostbyaddr, 467
- gethostbyname, 467
- gethostent, 466
- gethostname, 464
- getitimer, 290
- getmntent, 149
- GETNCNT, 375
- getopt, 307

getpass, 155  
 getpeername, 464  
 getpgrp, 196  
 GETPID, 375  
 getpid, 196  
 getppid, 196  
 getpwent, 87, 113  
 getpwnam, 113  
 getpwuid, 86  
 getsockname, 464  
 gettimeofday, 283  
 getuid, 197  
 getutent, 130  
 GETVAL, 375  
 GETZCNT, 375  
 GID, 197  
 GID\_NO\_CHANGE, 82  
 gid\_t, 197  
 GMT, 281  
 GNU, 532  
 grafo

    bipartido, 418

    dirigido, 162

*Greenwich Mean Time*, 281

groff, 532

    secuencias de control, 534

grupo

    de usuarios, 17

    de cilindros, 30

    de procesos, 196

    efectivo, 197

    real, 197

## — H —

handshake, 3

hebra, 168

herencia

    de prioridades, 180

    del techo de prioridad, 180

    inmediata del techo de prioridad, 181

    simple, 180

hijo, proceso, 161

hilo, 168

    ámbito de contienda, 217

    atributos de un, 213

    cancelación

        asíncrona, 221

        diferida, 221

    del núcleo, 169

    del usuario, 170

    dirección de la pila, 217

    estado de cancelación, 221

    herencia de la planificación, 217

    identificador de, 213

    manejador de limpieza, 222

    parámetros de planificación, 217

    política de distribución, 217

    principal, 213

    punto de cancelación, 221

    tamaño de la pila, 217

    tipo de cancelación, 221

    vinculación, 217

HOME, 199, 235

hostent, 466

htonl, 457

htons, 457

HZ, 291

## — I —

I/O trap instruction, 242

ICCP, 181

identificador, 488

    de hilo, 213

    de proceso, 161, 189, 196

    del grupo

        de procesos, 196

        efectivo, 197

        real, 197

    del proceso padre, 196

    del usuario

        efectivo, 197

        real, 197

ILL\_BADSTK, 266

ILL\_COPROC, 266

ILL\_ILLADR, 265

ILL\_ILLOPC, 265

ILL\_ILLOPN, 265

ILL\_ILLTRP, 266

ILL\_PRVOPC, 266

ILL\_PRVREG, 266

*immediate ceiling priority protocol*, 181

in\_addr, 427

include, 489

indicador *close-on-exec*, 73

índice de símbolos de una biblioteca, 521

inet, 455

inet\_addr, 455

inet\_ntoa, 455

ingeniería del software, 532

*inheritsched*, 217

init, 161  
 INIT\_PROCESS, 130  
 instrucción  
     illegal, 241  
     *test-and-set*, 88  
 int, 490  
 intercambiador, 5  
     proceso, 161  
 intercambio, 5  
*Internet*, 425  
*Internet*  
     *Protocol*, 426  
 intérprete de órdenes, 159  
 interrogación periódica, 351  
 interrupción  
     del reloj, 5  
     manejo de, 5  
     software, 3  
 invariante, 400  
 inversión de prioridades, 179  
 \_iob, 60  
 \_iobuf, 60  
 ioctl, 135  
 \_IOEOF, 60  
 \_IOERR, 60  
 \_IOFBF, 60  
 \_IOLBF, 60  
 \_IOMYBUF, 60  
 \_IONBF, 60  
 \_IOREAD, 60  
 \_IORW, 60  
 \_IOWRT, 60  
*IP*, 426, 452  
*IPC*, 5  
     cola de mensajes, 369  
     memoria compartida, 369  
     semáforo, 369  
 IPC\_CREAT, 370, 374  
 IPC\_EXCL, 370, 374  
 IPC\_NOWAIT, 376  
 IPC\_PRIVATE, 370, 374  
 IPC\_RMID, 375, 380, 390  
 IPC\_SET, 375, 380, 390  
 IPC\_STAT, 375, 380, 390  
 ipcrm, 371  
 ipcs, 371  
 ITIMER\_REAL, 290  
 ITIMER\_VIRTUAL, 290  
 itimerval, 290

## — J —

jmp\_buf, 250

## — K —

*kernel*, 1  
*kernel thread*, 169  
 key\_t, 370  
 kill, 243, 298  
 ksh, 159

## — L —

*LAN*, 458  
 LANG, 199  
 ld, 185, 512  
 lectores-redactores, problema, 105  
 lectura de la contraseña, 155  
 LFNMAX, 140  
*lightweight process*, 169  
 limits, 33  
 link, 120  
 lista  
     de nodos índice, 15  
     simplemente enlazada, 207  
 listen, 434  
*little endian*, 456  
 llamada asíncrona, 246  
 llave, 370  
 ln, 110  
*Local Area Network*, 458  
 localtime, 293  
 LOCK\_EX, 115  
 LOCK\_NB, 115  
 LOCK\_SH, 115  
 LOCK\_UN, 115  
 lockf, 87  
 login, 198, 236  
 LOGIN\_PROCESS, 130  
 LOGNAME, 235  
 long, 490  
 longjmp, 250  
 LPNMAX, 140  
 lseek, 58  
 lstat, 77  
 lugar de una RdP, 418  
*LWP*, 169

## — M —

macro, 514  
 macroinstrucción, 514  
*major number*, 14, 28, 36  
 make, 71, 527

- make file system*, 24
  - makecontext*, 266
  - makedev*, 118, 145
  - malloc*, 202
  - man*, 532
  - manejador
    - de dispositivo, 3, 28
    - de limpieza, 222
    - por defecto, 245
    - suministrado por el usuario, 246
  - manejo de interrupciones, 5
  - MANPATH, 536
  - manual
    - de UNIX, 532
    - edición de páginas, 532
  - máquina virtual, 262
  - marcado de una RdP, 418
  - marco de pila, 160
  - máscara
    - de modo de creación de un fichero, 201
    - de señales, 259
  - matriz, 492
  - MAX\_ALARM, 288, 291
  - MAX\_PROF, 291
  - MAX\_VTALARM, 291
  - MAXBSIZE, 39
  - memoria
    - compartida, 369
    - intermedia, 4
    - virtual, proceso intercambiador, 161
  - miembro de una biblioteca, 519
  - minor number*, 14, 28, 36
  - mkdev*, 38
  - mkdir*, 118
  - mke2fs*, 145
  - mkfifo*, 344
  - mkfs*, 24, 38
  - mknod*, 29, 36, 118
  - mktemp*, 451
  - MNT\_MNTTAB, 149
  - MNTOP\_RO, 149
  - MNTOPT\_RW, 149
  - modelo cliente-servidor, 428
  - modo
    - bloque, 27
    - carácter, 27
    - no canónico del terminal, 135
    - supervisor, 161
    - usuario, 161
  - módulo
    - de control del hardware, 5
    - de gestión de memoria, 5
  - monitor*, 315
  - mount*, 42, 137
  - MSG\_OOB, 438
  - MSG\_PEEK, 438
  - msgctl*, 389, 390
  - msgget*, 371, 389
  - msgrcv*, 390
  - msgsnd*, 390
  - multibyte, 456
  - multiplexación, 351, 357
  - mutex*, 400
  - mv*, 111
- N —
- número
    - de dispositivo, 14
    - mágico, 184
  - namei*, 27
  - nanosleep*, 294
  - NEW\_TIME, 130
  - nm*, 523
  - nodo, 428
    - de un grafo, 162
  - nodo-i, 15, 16
  - norma
    - IEEE 1003.3, 212
    - IEEE 802, 439
  - notación
    - infija, 236
    - polaca inversa, 236
    - postfija, 236
  - ntohl*, 457
  - ntohs*, 457
  - núcleo, 1
- O —
- O\_APPEND, 54
  - O\_CREAT, 54
  - O\_EXCL, 54
  - O\_NDELAY, 54, 351, 435
  - O\_RDONLY, 54
  - O\_RDWR, 54
  - O\_SYNC, 59
  - O\_SYNCW, 59
  - O\_TRUNC, 54
  - O\_WRONLY, 54
  - objetivo para *make*, 528
  - ocupación del disco, 44

OLD\_TIME, 130

open, 53

OPEN\_MAX, 33, 61

opendir, 122

operación

$\mathcal{P}$ , 373

$\mathcal{V}$ , 373

operador, 489

$*$ , 493

$++$ , 497

$-$ , 497

$->$ , 494

$.$ , 494

$\&$ , 493

aritmético, 497

de manejo de bits, 498

de postdecremento, 497

de postincremento, 497

de predecremento, 497

de preincremento, 497

de relación, 498

lógico, 498

ternario  $?:$ , 500

operadores, 497

operandos, 497

optimización, 512

OSI, 423

## — P —

padre, proceso, 161

palabra

multibyte, 456

reservada, 488

parada

de un proceso, 253

señal de, 253

paralelismo, 169

partición

activa, 35

del disco, 35

lógica del disco, 14

tabla, 35

PATH, 184, 199

*path name*, 14

PATH\_MAX, 53

pause, 249

perfil

POSIX, 227

PSE51, 227

PSE52, 227

PSE53, 227

PSE54, 227

perfilado

fichero *mon.out*, 315

función *monitor*, 315

orden *prof*, 315

perfilador, 309

perfiles para sistemas de tiempo real, 227

permiso

de ejecución, 25

de escritura, 25

de lectura, 25

*perror*, 7

*PID*, 161, 164, 196

*pid\_t*, 188

pila

de supervisor, 167

entorno de, 250

marco de, 160

puntero de, 161

*pipe*, 334

planificación, 5

dinámica, 179

*DMS*, 174

*EDF*, 179

estática, 179

*RMS*, 173

planificador, 161, 171

de tareas, 161

plato de un disco, 30

*plock*, 211

POLL\_ERR, 266

POLL\_HUP, 266

POLL\_IN, 266

POLL\_MSG, 266

POLL\_OUT, 266

POLL\_PRI, 266

*polling*, 351

POSIX, 212

\_POSIX\_SEMAPHORES, 397

\_POSIX\_THREAD\_ATTR\_STACKADDR, 218

\_POSIX\_THREAD\_ATTR\_STACKSIZE, 218

\_POSIX\_THREAD\_PRIORITY\_SCHEDULING, 228

\_POSIX\_THREAD\_PROCESS\_SHARED, 403, 415

*PPID*, 196

predicado, 405

*preigion*, 166

preprocesador, directriz, 489

preprocesamiento, 511

prioridad circular, 171

- PRLOCK, 211
- problema
  - de los lectores-redactores, 105
  - de los productores-consumidores, 416
- proceso, 160
  - 0, 188
  - core image*, 192
  - ejecución en segundo plano, 189
  - cliente, 428
  - comunicación entre procesos, 333
  - contexto, 165
  - cooperativo, 87
  - demonio, 16
  - distribución, 171
  - entorno, 184
  - estados, 161
  - grupo de procesos, 196
  - hijo, 161, 188
  - identificador, 161, 189
    - de grupo, 196
  - intercambiador, 161
  - ligero, 169
  - padre, 161, 188
  - parada de un, 253
  - servidor, 428
  - tabla de procesos, 164
  - zombi, 164, 191
- process identification*, 161
- process lock*, 211
- productores-consumidores, 416
- prof, 315
- profil, 310
- program break*, 202
- programa, 159
  - login, 198
  - cabecera, 160
    - de sección, 185
    - principal, 185
  - contador de, 161
  - depurador, 322
  - secciones, 185
  - segmento, 160
    - de datos, 160
    - de pila, 160
    - de texto, 160
  - tablas de símbolos, 185
  - tolerante a fallos, 276
- programación
  - asistida, 525
  - modular, 509
- propietario, 17
- proposición, 499
- protocolo, 423
  - IP*, 452
  - TCP*, 452
  - de herencia de prioridad, 180
  - interno UNIX, 432
  - UDP*, 452
- proyecto
  - GNU*, 532
  - llave de un, 370
- prueba del factor de utilización, 174
- PSE51, 227
- PSE52, 227
- PSE53, 227
- PSE54, 227
- PT\_CONTIN, 324
- PT\_EXIT, 324
- PT\_RDUSER, 324
- PT\_RIUSER, 324
- PT\_RUAREA, 324
- PT\_SETTRC, 324
- PT\_SINGLE, 324
- PT\_WDUSER, 324
- PT\_WIUSER, 324
- PT\_WUAREA, 324
- pthread\_attr\_destroy, 217
- pthread\_attr\_getdetachstate, 217
- pthread\_attr\_getinheritsched, 230
- pthread\_attr\_getschedparam, 229
- pthread\_attr\_getschedpolicy, 228
- pthread\_attr\_getscope, 229
- pthread\_attr\_getstackaddr, 218
- pthread\_attr\_getstacksize, 218
- pthread\_attr\_init, 217
- pthread\_attr\_setdetachstate, 217
- pthread\_attr\_setschedparam, 229
- pthread\_attr\_setschedpolicy, 228
- pthread\_attr\_setscope, 229
- pthread\_attr\_setstackaddr, 218
- pthread\_attr\_setstacksize, 218
- pthread\_attr\_t, 213, 217
- pthread\_cancel, 221
- PTHREAD\_CANCEL\_ASYNCHRONOUS, 221
- PTHREAD\_CANCEL\_DEFERRED, 221
- PTHREAD\_CANCEL\_DISABLE, 221
- PTHREAD\_CANCEL\_ENABLE, 221
- pthread\_clean\_pop, 222
- pthread\_cleanup\_push, 222
- pthread\_cond\_broadcast, 414

pthread\_cond\_destroy, 406  
 pthread\_cond\_init, 406  
 PTHREAD\_COND\_INITIALIZER, 406  
 pthread\_cond\_signal, 407  
 pthread\_cond\_t, 406  
 pthread\_cond\_timedwait, 414  
 pthread\_cond\_wait, 407  
 pthread\_condattr\_destroy, 415  
 pthread\_condattr\_getclock, 415  
 pthread\_condattr\_init, 415  
 pthread\_condattr\_setclock, 415  
 pthread\_condattr\_setpshared, 415  
 pthread\_condattr\_t, 415  
 pthread\_create, 213  
 PTHREAD\_CREATED\_DETACHED, 218  
 PTHREAD\_CREATED\_JOINABLE, 218  
 pthread\_detach, 213  
 pthread\_equal, 213  
 pthread\_exit, 213  
 PTHREAD\_EXPLICIT\_SCHED, 230  
 pthread\_getschedparam, 229  
 PTHREAD\_INHERIT\_SCHED, 230  
 pthread\_join, 213  
 pthread\_kill, 268  
 PTHREAD\_MUTEX\_DEFAULT, 403  
 pthread\_mutex\_destroy, 401  
 PTHREAD\_MUTEX\_ERRORCHECK, 403  
 pthread\_mutex\_getprioceiling, 404  
 pthread\_mutex\_init, 400  
 PTHREAD\_MUTEX\_INITIALIZER, 400  
 pthread\_mutex\_lock, 401  
 PTHREAD\_MUTEX\_NORMAL, 403  
 PTHREAD\_MUTEX\_RECURSIVE, 403  
 pthread\_mutex\_setprioceiling, 404  
 pthread\_mutex\_t, 400  
 pthread\_mutex\_timedlock, 401  
 pthread\_mutex\_trylock, 401  
 pthread\_mutex\_unlock, 401  
 pthread\_mutexattr\_getprioceiling, 404  
 pthread\_mutexattr\_getprotocol, 404  
 pthread\_mutexattr\_gettype, 402  
 pthread\_mutexattr\_init, 402  
 pthread\_mutexattr\_setprioceiling, 404  
 pthread\_mutexattr\_setprotocol, 404  
 pthread\_mutexattr\_setpshared, 403  
 pthread\_mutexattr\_settype, 402  
 pthread\_mutexattr\_t, 402  
 PTHREAD\_PRIO\_INHERIT, 404  
 PTHREAD\_PRIO\_NONE, 404  
 PTHREAD\_PRIO\_PROTECT, 404

PTHREAD\_PROCESS\_PRIVATE, 403, 415  
 PTHREAD\_PROCESS\_SHARED, 403, 415  
 PTHREAD\_SCOPE\_PROCESS, 229  
 PTHREAD\_SCOPE\_SYSTEM, 229  
 pthread\_self, 213  
 pthread\_setcancelstate, 221  
 pthread\_setcanceltype, 221  
 pthread\_setschedparam, 229  
 pthread\_sigmask, 269  
 PTHREAD\_STACK\_MIN, 218  
 pthread\_t, 213  
 pthread\_testcancel, 221  
 pthreads, 170, 212  
 ptrace, 322  
 puerto, 455
 

- libre, 456
- privilegiado, 456
- reservado, 456

 puntero, 492
 

- de pila, 161, 166

 punto
 

- de cancelación, 221
- de parada, 322
- de repliegue, 251

 putc, 64  
 putchar, 64  
 putenv, 199  
 pwd, 145

## — Q —

quot, 44

## — R —

R\_OK, 81  
*race condition*, 225  
 raise, 245  
 rama de un grafo, 162  
 ranlib, 521  
 rc, 43  
 rcp, 425, 467  
 RdP, 418  
 read, 55  
 readdir, 123  
 readdir\_r, 226  
 readv, 436  
 real, tipo, 491  
*recovery system*, 14  
 recubrimiento, 185  
 recv, 436  
 recvfrom, 436

- recvmsg, 436
- red
  - capa
    - de aplicación, 425
    - de enlace, 424
    - de presentación, 425
    - de red, 425
    - de sesión, 425
    - física, 424
  - de ordenadores, 423
  - de Petri, 417
    - comportamiento dinámico, 418
    - disparo de una transición, 418
    - evolución del marcado, 418
    - lugar, 418
    - marcado, 418
    - marcas de un lugar, 418
    - transición, 418
    - transición sensibilizada, 418
  - protocolo de una, 423
  - servicio de una, 423
- redirección del fichero estándar
  - de entrada, 340
  - de salida, 340
- reentrante, función, 226
- registro de estado, 166
- reloj, 294
  - de tiempo real, 294
  - monótono creciente, 294
- rename, 82
- repliegue, punto de, 251
- retardo
  - absoluto, 295
  - relativo, 295
- rewinddir, 124
- rlogin, 425
- rmdir, 119
- RMS, 173
- rodaja, 171
- round robin, 171
- row device, 3
- rresvport, 456
- RTSIG\_MAX, 263
- RUN\_LVL, 130
- ruta, 14, 53
- S —
  - S\_IRGRP, 119
  - S\_IROTH, 119
  - S\_IRUSR, 119
  - S\_IRWXG, 119
  - S\_IRWXO, 119
  - S\_IRWXU, 119
  - S\_ISGID, 80
  - S\_ISUID, 79
  - S\_ISVTX, 80
  - S\_IWGRP, 119
  - S\_IWOTH, 119
  - S\_IWUSR, 119
  - S\_IXGRP, 119
  - S\_IXOTH, 119
  - S\_IXUSR, 119
  - SA\_NOCLDSTOP, 267
  - SA\_NOCLDWAIT, 267
  - SA\_NODEFER, 267
  - SA\_ONSTACK, 267
  - SA\_RESETHAND, 267
  - SA\_RESTART, 267
  - SA\_SIGINFO, 267
  - salto no local, 251
  - sbrk, 202
  - SCHED\_FIFO, 228
  - sched\_get\_priority\_max, 230
  - sched\_get\_priority\_min, 230
  - SCHED\_OTHER, 229
  - SCHED\_RR, 229
  - sched\_yield, 229
  - schedparam, 217
  - schedpolicy, 217
  - scheduler, 5
  - scheduling, 5
  - scope, 217
  - sdb, 240, 322
  - sección, 185
    - crítica, 399, 400
  - SEEK\_CUR, 58
  - SEEK\_END, 58
  - SEEK\_SET, 58
  - seekdir, 124
  - segmento
    - de datos, 160
    - de pila, 160
    - de texto de un programa, 160
    - de un programa, 160
    - violación de, 242
  - segundo plano, 189
  - SEGV\_ACCERR, 266
  - SEGV\_MAPERR, 266
  - selección
    - múltiple else-if, 500



- por casos `switch`, 501
  - simple `if-else`, 500
- `select`, 357
- `sem_close`, 399
- `sem_destroy`, 399
- `SEM_FAILED`, 398
- `sem_getvalue`, 399
- `sem_init`, 398
- `sem_open`, 398
- `sem_post`, 398
- `sem_t`, 398
- `sem_timedwait`, 398
- `sem_trywait`, 398
- `SEM_UNDO`, 377
- `sem_unlink`, 399
- `SEM_VALUE_MAX`, 398
- `sem_wait`, 398
- semáforo, 369, 372, 397
  - anónimo, 397
  - nominal, 397
  - operación
    - $\mathcal{P}$ , 373
    - $\mathcal{V}$ , 373
  - POSIX, 397
- `sembuf`, 376
- `semctl`, 374
- `semget`, 371, 373
- semidúplex, comunicación, 350
- `semop`, 376
- `send`, 438
- `sendmsg`, 438
- `sendto`, 438
- señal, 239
  - cambio de tamaño de una ventana, 253
  - dato urgente, 253
  - de un temporizador, 252
    - en tiempo virtual, 252
  - desconexión, 241
  - emulator trap instruction*, 242
  - entrada/salida asíncrona, 252
  - error
    - de bus, 242
    - en coma flotante, 242
  - fallo de alimentación, 243
  - finalización
    - controlada, 242
    - del tiempo de CPU, 253
  - I/O trap instruction*, 242
  - instrucción ilegal, 241
  - intento de escritura en segundo plano
    - en un terminal, 253
  - intento de lectura en segundo plano
    - de un terminal, 253
  - interrupción, 241
  - lectura de una tubería sin escucha, 242
  - máscara, 259
  - número 1 de usuario, 243
  - número 2 de usuario, 243
  - para continuar, 253
  - para despertar un proceso, 242
  - para salir, 241
  - parada, 253
    - procedente de un terminal, 253
  - superación del tamaño máximo de fichero, 253
  - terminación
    - abrupta, 242
    - del proceso hijo, 243
  - trace trap*, 242
  - vector de tratamiento, 255
  - violación de segmento, 242
- servicio, 423
- servidor, 428
  - concurrente, 429
  - interactivo, 429
- `SETALL`, 375
- `setcontext`, 266
- `setgid`, 198
- `setgrent`, 113
- `sethostent`, 467
- `setitimer`, 290
- `setjmp`, 250
- `setmntent`, 149
- `setpgrp`, 196
- `setpwent`, 113
- `settimeofday`, 283
- `setuid`, 198
- `SETVAL`, 375
- seudodispositivos, 28
- `sh`, 159
- shell script*, 159
- `SHM_LOCK`, 380
- `SHM_RDONLY`, 382
- `SHM_UNLOCK`, 380
- `shmat`, 381
- `shmctl`, 380
- `shmdt`, 381
- `shmget`, 371, 379
- `shmid_ds`, 381
- `short`, 490

- SHUT\_RD, 465
- SHUT\_RDWR, 466
- SHUT\_WR, 466
- shutdown, 16, 465
- SI\_ASYNCIO, 265
- SI\_MSGQ, 265
- SI\_QUEUE, 265
- SI\_TIMER, 265
- SI\_USER, 265
- SIG\_BLOCK, 269
- SIG\_DFL, 245, 254
- SIG\_ERR, 246
- SIG\_IGN, 246, 254
- SIG\_SETMASK, 269
- SIG\_UNBLOCK, 269
- SIGABRT, 263
- sigaction, 262
- sigaddset, 268
- SIGALRM, 164, 242, 263, 288
- sigblock, 259
- SIGBUS, 242, 263
- SIGCHLD, 253, 263
- SIGCLD, 243
- SIGCONT, 253, 263
- sigdelset, 268
- SIGEMT, 242
- SIGEV\_NONE, 296
- SIGEV\_SIGNAL, 296
- SIGEV\_THREAD, 296
- sigfillset, 268
- SIGFPE, 242, 263
- SIGHUP, 241, 263
- SIGILL, 241, 263
- siginfo\_t, 265
- SIGINT, 241, 263
- SIGIO, 252
- SIGIOT, 242
- sigismember, 268
- SIGKILL, 242, 263
- sigmask, 254
- signal, 192, 245
- sigpause, 260
- sigpending, 275
- SIGPIPE, 242, 263
- SIGPOLL, 263
- sigprocmask, 269
- SIGPROF, 252, 264, 290
- SIGPWR, 243
- sigqueue, 267
- SIGQUIT, 241, 263
- SIGRTMAX, 263
- SIGRTMIN, 263
- SIGSEGV, 242, 263
- sigset\_t, 268
- sigsetmask, 259
- SIGSTOP, 253, 263
- sigsuspend, 275
- SIGSYS, 242, 264
- SIGTERM, 242, 263
- sigtimedwait, 276
- SIGTRAP, 242, 264, 323
- SIGTSTP, 253, 263
- SIGTTIN, 253, 263
- SIGTTOU, 253, 263
- SIGURG, 253, 264
- SIGUSR1, 243, 263
- SIGUSR2, 243, 263
- sigvec, 253
- SIGVTALRM, 252, 264, 290
- sigwait, 275
- sigwaitinfo, 275
- SIGWINCH, 253
- SIGXCPU, 253, 264
- SIGXFSZ, 253, 264
- símbolo
  - externo, 523
  - índice de una biblioteca, 521
- sincronización del disco, 59, 138
- sistema
  - de ficheros, 13
  - multihilo, 168
  - operativo, 1
    - de tiempo compartido, 1
    - núcleo, 1
- sizeof, 491
- sleep, 288
- SOCK\_DGRAM, 432
- SOCK\_RAW, 432
- SOCK\_RDM, 433
- SOCK\_SEQPACKET, 433
- SOCK\_STREAM, 432
- sockaddr, 427
- sockaddr\_in, 428
- socket, 334
- socket, 431
- software, ingeniería, 532
- soketpair, 465
- Soporte en Lengua Nativa, 199
- ssize\_t, 436
- stackaddr, 217

*stacksize*, 217  
 stat, 77  
 statfs, 151  
 stderr, 61  
 stdin, 61  
 stdout, 61  
*sticky bit*, 80  
 stime, 281  
 strerror, 215  
 struct acct, 316  
 struct dinode, 17  
 struct dirent, 123  
 struct filsys, 140  
 struct flock, 73  
 struct group, 87  
 struct iovec, 436  
 struct itimerspec, 296  
 struct mntent, 149  
 struct mnttab, 140  
 struct msghdr, 437  
 struct passwd, 87  
 struct sched\_param, 229  
 struct sigaction, 264  
 struct sigevent, 295  
 struct statfs, 152  
 struct timespec, 276  
 struct ustat, 139  
 struct utimbuf, 82  
 struct utsname, 440  
 su, 235  
 subdirectorio, 14  
 subsistema  
     de control de procesos, 3, 5  
     de ficheros, 3, 16  
 superbloque, 15  
     lectura del, 139  
 superusuario, 17  
 SV\_BSDSIG, 254  
 SV\_RESETHAND, 254  
 swap, 5  
 swapcontext, 266  
 swapper, 5  
 swapping, 5  
 \_\_.SYMDEF, 522  
 sync, 138  
 syncer, 17, 42, 138  
 sys\_errlist, 7  
 sysconf, 285  
 system, 194

## — T —

tabla, 492  
     *ASCII*, 425  
     de descriptors de fichero, 31, 33  
     de ficheros, 31  
     de nodos-i, 16, 31  
     de páginas, 166  
     de particiones, 35  
     de procesos, 164  
     de regiones, 166  
         por proceso, 166  
     de símbolos, 185  
     *EBCDIC*, 425  
 tareas, planificador de, 161  
*TCP*, 426, 432, 452  
*TCP/IP*, 452  
 techo de prioridad, 180  
 telldir, 124  
 telnet, 425  
 temporización  
     asíncrona, 295  
     síncrona, 295  
 temporizador, 295  
     armado, 296  
     desarmado, 296  
     en tiempo  
         real, 290  
         virtual, 252, 290  
 TERM, 199  
 terminación abrupta, 242  
 terminal, 129  
     en modo no canónico, 135  
     sin eco local, 135  
 termio(7), 135  
*test-and-set*, 88  
*text lock*, 211  
 texto de un programa, 160  
*thread*, 168  
*thread-safe*, 225  
 throw, 270  
 tiempo  
     de cómputo, 173  
     real, 290  
     virtual, 290  
 time, 101, 282, 286  
*time quantum*, 171  
*time slice*, 171  
 time\_t, 282  
 TIMER\_ABSTIME, 295  
 timer\_create, 295

timer\_delete, 297  
 timer\_getoverrun, 298  
 timer\_gettime, 297  
 timer\_settime, 296  
 times, 285  
 timeval, 283  
 timezone, 283  
 tipo  
     de conexión, 427  
     de notificación, 296  
     derivado, 492  
     entero, 490  
     fundamental, 490  
     real, 491  
 tms, 285  
 tolerancia a fallos, 276  
 touch, 116, 529  
 trace trap, 242  
 transición de una RdP, 418  
*Transmission Control Protocol*, 426, 432  
 TRAP\_BRKPT, 266  
 TRAP\_TRACE, 266  
 troff, 532  
     cabecera  
         de epígrafe o sección .SH, 534  
         de título .TH, 534  
     nueva línea .br, 536  
     párrafo nuevo .PP, 536  
     secuencias de control, 534  
     tipo de letra  
         anterior \fP, 536  
         itálica \fI, 536  
         itálica .I, 536  
         negrita \fB, 536  
         negrita .B, 536  
         redonda \fR, 536  
         redonda .R, 536  
 truncate, 83  
 try, 270  
 tubería, 17, 334  
     con nombre, 28, 334, 342  
     sin nombre, 29, 334  
 TXTLOCK, 211  
 typedef, 496  
 TZ, 283

## — U —

u area, 164  
 ucontext\_t, 266  
 UDP, 426, 432, 452

UID, 164, 197  
 UID\_NO\_CHANGE, 82  
 uid\_t, 197  
 UL\_GETFSIZE, 201  
 UL\_GETMAXBRK, 201  
 UL\_SETFSIZE, 201  
 ulimit, 201  
 umask, 81  
 umount, 43, 138  
 uname, 440  
 unión, 495  
 union signal, 267  
 unlink, 120  
 UNLOCK, 211  
 unsigned, 490  
 update, 42, 138  
 USER, 235  
*User Datagram Protocol*, 426, 432  
 USER\_PROCESS, 130  
 uso de disco, 44  
 ustat, 139  
 usuario

    área de, 164, 165  
     efectivo, 197  
     real, 197  
     u area, 164

utime, 82

utmp, 130

## — V —

va\_arg, 184  
 va\_end, 184  
 va\_start, 184  
 variable  
     de condición, 406  
     de entorno ?, 53, 190  
     estática, 160  
     global, 160  
 vector, 492  
     de señal, 255  
     de tratamiento de una señal, 255  
 vi, 488  
 vinculación de un hilo, 217  
 violación de segmento, 242  
 volcado de la memoria, 192

## — W —

W\_OK, 81  
 wait, 191  
 waitpid, 234

wall, 147  
WCET, 173  
WCOREDUMP, 192  
WEXITSTATUS, 191  
WIFEXITED, 191  
WIFSIGNALED, 192  
WIFSTOPPED, 192  
*worst case execution time*, 173  
write, 56, 130  
writev, 438  
WSTOPSIG, 192  
WTERMSIG, 192

— **X** —

X\_OK, 81  
xxgdb, 361

— **Z** —

zombi, 164, 191





# UNIX

## Programación avanzada

3<sup>a</sup> EDICIÓN

**Usuarios de:**

Sistemas Operativos  
UNIX

**Nivel del libro:**

✓ Iniciación  
✓ Medio  
✓ Avanzado

UNIX se ha convertido en uno de los sistemas operativos más populares en entornos industriales, académicos y, recientemente, incluso domésticos y brinda al usuario un conjunto de herramientas muy variado y completo. La mayor parte de los programas estándar en UNIX están escritos en lenguaje C, y hace uso de unas piezas básicas conocidas como llamadas al sistema (*system calls*).

El conjunto de llamadas al sistema es la interfaz entre el sistema operativo y el programador que utiliza sus recursos y constituye el núcleo de estudio de este libro, junto con técnicas de programación avanzada que nos permitirán aprovechar al máximo la potencia y flexibilidad de UNIX.

El libro está estructurado en tres partes, de acuerdo con la siguiente distribución de capítulos:

- Introducción.

Parte 1. El sistema de ficheros.

- Arquitectura del sistema de ficheros.
- Manejo de ficheros ordinarios.
- Manejo de directorios y ficheros especiales.

Parte 2. Procesos e hilos.

- Estructura de un proceso.
- Gestión de procesos e hilos.
- Señales y funciones de tiempo.
- Perfilado, contabilidad y depuración.

Parte 3. Comunicación entre procesos.

- Comunicación mediante tuberías.
- Comunicación local entre procesos e hilos.
- Comunicaciones en red.

Para ayudar a la comprensión del texto se utilizan alrededor de 90 programas de ejemplo elegidos no sólo por su interés didáctico, sino también porque muchos de ellos responden a la funcionalidad de programas estándar UNIX. Estos programas pueden servirle al lector como elementos de referencia para la construcción de aplicaciones o de nuevas herramientas del sistema.

ISBN 84-7897-603-5



9 788478 976034

**ra-ma.es**



**Ra-Ma®**